

langgraph

38개 문서

목차

Memory

- Add short-term memory
- Add long-term memory
- Manage short-term memory
- Database management

Build a custom RAG agent with LangGraph

- Setup
- Preprocess documents
- Create a retriever tool
- Generate a query or respond
- Rewrite the question
- Configuration file
- Dependencies
- Graphs
- Environment variables

Backward compatibility

- Technical compatibility
- Non-determinism

Case studies

Checkpointers

- Why use checkpointers
- Core concepts
- Get and update state
- Durability modes
- Optimize checkpoint storage
- Checkpointer libraries
- Build a custom checkpointer

Choosing between Graph and Functional APIs

- Quick decision guide
- Detailed comparison
- Combining both APIs
- Migration between APIs
- Summary

Deployment

- LangSmith Cloud

Event streaming

- Quickstart
- How the pieces fit together
- What event streaming provides
- Stream messages

- Stream subgraphs
- Stream state
- Stream multiple projections
- Resume after an interrupt
- Stream all protocol events
- Channels and event lifecycle
- Build your own projection
- Related

Fault tolerance

- Retries
- Timeouts
- Error handling
- Graph defaults
- Graph defaults
- Functional API
- Graceful shutdown
- Limitations

Functional API overview

- Functional API vs. Graph API
- Example
- Entrypoint
- Task
- When to use a task
- Serialization
- Determinism
- Idempotency
- Common pitfalls
- Learn more

Graph API overview

- Graphs
- State
- Nodes
- Edges
- Send
- Command
- Graph migrations
- Runtime context
- Visualization
- Observability and Tracing
- Learn more

Install LangGraph

Requires Python 3.10+

Interrupts

Pause using interrupt

Resuming interrupts

Define an interrupt response schema

Common patterns

Approval workflows: Pause before executing critical actions (API calls, database changes, financial transactions)

Handling multiple interrupts: Pair interrupt IDs with resume values when resuming multiple interrupts in a single invocation

Review and edit: Let humans review and modify LLM outputs or tool calls before continuing

Interrupting tool calls: Pause before executing tool calls to review and edit the tool call before execution

Rules of interrupts

On first run, this might skip the interrupt

On resume, it might not skip it - causing index mismatch

Using with subgraphs called as functions

Debugging with interrupts

Run a local server

Prerequisites

1. Install the LangGraph CLI

Python $\geq$ 3.11 is required.

2. Create a LangGraph app

3. Install dependencies

4. Create a .env file

5. Launch Agent server

6. Test your application in Studio

7. Test the API

Next steps

LangSmith Observability

Prerequisites

Enable tracing

Trace selectively

Log to a project

Add metadata to traces

Use anonymizers to prevent logging of sensitive data in traces

Define the graph

LangGraph overview

Core benefits

LangGraph ecosystem

Acknowledgements

Persistence

Quickstart

Checkpoint vs. store

Troubleshooting common issues

Next steps

LangGraph runtime

Overview

Actors

Channels

Examples

High-level API

Quickstart

Step 1: Read the guide

Step 2: Install dependencies

Step 3: Configure model credentials

Step 4: Implement the calculator agent

Rules

1. Define tools and model

2. Define state

3. Define model node

4. Define tool node

5. Define end logic

6. Build and compile the agent

1. Define tools and model

2. Define model node

3. Define tool node

4. Define agent

Build a custom SQL agent

Setup

1. Select an LLM

2. Configure the database

3. Add tools for database interactions

4. Define application steps

5. Implement the agent

6. Implement human-in-the-loop review

Next steps

Stores

Basic usage

Listing items in a namespace

Semantic search

Using in LangGraph

Build a custom store

Streaming

Get started

Stream modes

Advanced

LangSmith Studio

Prerequisites

Set up local Agent server

Video guide

Test

Prerequisites

Getting started

Testing individual nodes and edges

Partial execution

Thinking in LangGraph

Start with the process you want to automate

Step 1: Map out your workflow as discrete steps

Step 2: Identify what each step needs to do

Step 3: Design your state

Step 4: Build your nodes

Step 5: Wire it together

Summary and next steps

Agent Chat UI

Use the functional API

Creating a simple workflow

Parallel execution

Calling graphs

Call other entrypoints

Streaming

Retry policy

Set task and entrypoint timeouts

Caching tasks

Resuming after an error

Human-in-the-loop

Short-term memory

Long-term memory

Workflows

Integrate with other libraries

Use the graph API

Setup

Define and update state

Add runtime configuration

Add retry policies

Set node timeouts

Configure node timeouts

Handle node errors

Set graph-wide node defaults

Add node caching

Create a sequence of steps

- Create branches
- Map-Reduce and the send API
- Create and control loops
- Async
- Combine control flow and state updates with Command
- Visualize your graph

Subgraphs

- Setup
- Define subgraph communication
- Subgraph persistence
- View subgraph state
- Stream subgraph outputs

Use time-travel

- Overview
- Replay
- Fork
- Interrupts
- Subgraphs

Workflows and agents

- Setup
- LLMs and augmentations
- Prompt chaining

Graph state

Nodes

Simple check - does the joke contain "?" or "!"

Build workflow

Add nodes

Add edges to connect nodes

Compile

Show workflow

Invoke

- Parallelization

Graph state

Nodes

Build workflow

Add nodes

Add edges to connect nodes

Show workflow

Invoke

- Routing

Schema for structured output to use as routing logic

Augment the LLM with schema for structured output

State

Nodes

Run the augmented LLM with structured output to serve as routing logic

Conditional edge function to route to the appropriate node

Build workflow

Add nodes

Add edges to connect nodes

Compile workflow

Show the workflow

Invoke

Orchestrator-worker

Schema for structured output to use in planning

Augment the LLM with schema for structured output

Evaluator-optimizer

Graph state

Schema for structured output to use in evaluation

Augment the LLM with schema for structured output

Nodes

Conditional edge function to route back to joke generator or end based upon feedback from the evaluator

Build workflow

Add the nodes

Add edges to connect nodes

Compile the workflow

Show the workflow

Invoke

Agents

Define tools

Augment the LLM with tools

Execute tools

Invoke

GRAPH_RECURSION_LIMIT

Troubleshooting

INVALID_CHAT_HISTORY

Troubleshooting

INVALID_CONCURRENT_GRAPH_UPDATE

Troubleshooting

INVALID_GRAPH_NODE_RETURN_VALUE

Troubleshooting

MISSING_CHECKPOINTER

Troubleshooting

Related

MULTIPLE_SUBGRAPHS

Troubleshooting

Related

langgraph

Memory

langgraph/add-memory.mdx

AI applications need [memory](#) to share context across multiple interactions. In LangGraph, you can add two types of memory:

- [Add short-term memory](#) as a part of your agent's [state](#) to enable multi-turn conversations.
- [Add long-term memory](#) to store user-specific or application-level data across sessions.

Add short-term memory

Short-term memory (thread-level [persistence](#)) enables agents to track multi-turn conversations. To add short-term memory:

python

```
from langgraph.checkpoint.memory import InMemorySaver # [!code highlight]
from langgraph.graph import StateGraph

checkpointer = InMemorySaver() # [!code highlight]

builder = StateGraph(...)
graph = builder.compile(checkpointer=checkpointer) # [!code highlight]

graph.invoke(
    {"messages": [{"role": "user", "content": "hi! i am Bob"}]},
    {"configurable": {"thread_id": "1"}}, # [!code highlight]
)
```

js

```
import { MemorySaver, StateGraph } from "@langchain/langgraph";

const checkpointer = new MemorySaver();

const builder = new StateGraph(...);
const graph = builder.compile({ checkpointer });

await graph.invoke(
  { messages: [{ role: "user", content: "hi! i am Bob" }] },
  { configurable: { thread_id: "1" } }
);
```

Use in production

In production, use a checkpointer backed by a database:

python

```
from langgraph.checkpoint.postgres import PostgresSaver

DB_URI = "postgresql://postgres:postgres@localhost:5432/postgres?sslmode=disable"
with PostgresSaver.from_conn_string(DB_URI) as checkpointer: # [!code highlight]
    builder = StateGraph(...)
    graph = builder.compile(checkpointer=checkpointer) # [!code highlight]
```

js

Postgres

```
import { PostgresSaver } from "@langchain/langgraph-checkpoint-postgres";

const DB_URI = "postgresql://postgres:postgres@localhost:5432/postgres?sslmode=disable";
const checkpointer = PostgresSaver.fromConnString(DB_URI);

const builder = new StateGraph(...);
const graph = builder.compile({ checkpointer });
```

MongoDB

```
import { MongoClient } from "mongodb";
import { MongoDBSaver } from "@langchain/langgraph-checkpoint-mongodb";

const client = new MongoClient("mongodb://user:password@localhost:27017");
const checkpointer = new MongoDBSaver({ client });

const builder = new StateGraph(...);
const graph = builder.compile({ checkpointer });
```

Example: using Postgres checkpointer

python

```
pip install -U "psycopg[binary,pool]" langgraph langgraph-checkpoint-postgres
```

팁

You need to call `checkpointer.setup()` the first time you're using Postgres checkpointer

Sync

```
from langchain.chat_models import init_chat_model
from langgraph.graph import StateGraph, MessagesState, START
from langgraph.checkpoint.postgres import PostgresSaver # [!code highlight]

model = init_chat_model(model="claude-haiku-4-5-20251001")

DB_URI = "postgresql://postgres:postgres@localhost:5432/postgres?sslmode=disable"
with PostgresSaver.from_conn_string(DB_URI) as checkpointer: # [!code highlight]
    # checkpointer.setup()

def call_model(state: MessagesState):
    response = model.invoke(state["messages"])
    return {"messages": response}

builder = StateGraph(MessagesState)
builder.add_node(call_model)
builder.add_edge(START, "call_model")

graph = builder.compile(checkpointer=checkpointer) # [!code highlight]

config = {
    "configurable": {
        "thread_id": "1" # [!code highlight]
    }
}

stream = graph.stream_events(
    {"messages": [{"role": "user", "content": "hi! I'm bob"}]},
    config, # [!code highlight]
    version="v3",
)
for snapshot in stream.values:
    print(snapshot)

    stream = graph.stream_events(
        {"messages": [{"role": "user", "content": "what's my name?"}]},
        config, # [!code highlight]
        version="v3",
    )
    for snapshot in stream.values:
        print(snapshot)
```

Async

```
from langchain.chat_models import init_chat_model
from langgraph.graph import StateGraph, MessagesState, START
from langgraph.checkpoint.postgres.aio import AsyncPostgresSaver # [!code highlight]
```

```

model = init_chat_model(model="claude-haiku-4-5-20251001")

DB_URI = "postgresql://postgres:postgres@localhost:5432/postgres?sslmode=disable"
async with AsyncPostgresSaver.from_conn_string(DB_URI) as checkpointer: # [!code highlight]
    # await checkpointer.setup()

async def call_model(state: MessagesState):
    response = await model.ainvoke(state["messages"])
    return {"messages": response}

builder = StateGraph(MessagesState)
builder.add_node(call_model)
builder.add_edge(START, "call_model")

graph = builder.compile(checkpointer=checkpointer) # [!code highlight]

config = {
    "configurable": {
        "thread_id": "1" # [!code highlight]
    }
}

stream = await graph.astream_events(
    {"messages": [{"role": "user", "content": "hi! I'm bob"}]},
    config, # [!code highlight]
    version="v3",
)

async for message in stream.messages:
    async for token in message.text:
        print(token, end="", flush=True)

        stream = await graph.astream_events(
            {"messages": [{"role": "user", "content": "what's my name?"}]},
            config, # [!code highlight]
            version="v3",
        )
        async for message in stream.messages:
            async for token in message.text:
                print(token, end="", flush=True)

```

js

```
npm install @langchain/langgraph-checkpoint-postgres
```

팁

You need to call `checkpointer.setup()` the first time you're using Postgres checkpointer

```

import { ChatAnthropic } from "@langchain/anthropic";
import { StateGraph, StateSchema, MessagesValue, GraphNode, START } from "@langchain/langgraph";
import { PostgresSaver } from "@langchain/langgraph-checkpoint-postgres";

const State = new StateSchema({
  messages: MessagesValue,
});

const model = new ChatAnthropic({ model: "claude-haiku-4-5-20251001" });

const DB_URI = "postgresql://postgres:postgres@localhost:5432/postgres?sslmode=disable";
const checkpointer = PostgresSaver.fromConnString(DB_URI);
// await checkpointer.setup();

const callModel: GraphNode<typeof State> = async (state) => {
  const response = await model.invoke(state.messages);
  return { messages: [response] };
};

const builder = new StateGraph(State)
  .addNode("call_model", callModel)
  .addEdge(START, "call_model");

const graph = builder.compile({ checkpointer });

const config = {
  configurable: {
    thread_id: "1"
  }
};

const stream1 = await graph.streamEvents(
  { messages: [{ role: "user", content: "hi! I'm bob" }] },
  { ...config, version: "v3" }
);

for await (const snapshot of stream1.values) {
  console.log(snapshot);
}

const stream2 = await graph.streamEvents(
  { messages: [{ role: "user", content: "what's my name?" }] },
  { ...config, version: "v3" }
);

for await (const snapshot of stream2.values) {
  console.log(snapshot);
}

```

Example: using MongoDB checkpointer

python

```
pip install -U pymongo langgraph langgraph-checkpoint-mongodb
```

팁

Setup To use the [MongoDB checkpointer](#), you will need a MongoDB cluster. Follow [this guide](#) to create a cluster if you don't already have one. For an agent-focused walkthrough, see [short-term memory with MongoDB Atlas](#).

Sync

```
from langchain.chat_models import init_chat_model
from langgraph.graph import StateGraph, MessagesState, START
from langgraph.checkpoint.mongodb import MongoDBSaver # [!code highlight]

model = init_chat_model(model="claude-haiku-4-5-20251001")

MONGODB_URI = "localhost:27017"
with MongoDBSaver.from_conn_string(MONGODB_URI) as checkpointer: # [!code highlight]

    def call_model(state: MessagesState):
        response = model.invoke(state["messages"])
        return {"messages": response}

    builder = StateGraph(MessagesState)
    builder.add_node(call_model)
    builder.add_edge(START, "call_model")

    graph = builder.compile(checkpointer=checkpointer) # [!code highlight]

    config = {
        "configurable": {
            "thread_id": "1" # [!code highlight]
        }
    }

    stream = graph.stream_events(
        {"messages": [{"role": "user", "content": "hi! I'm bob"}]},
        config, # [!code highlight]
        version="v3",
    )
    for snapshot in stream.values:
        print(snapshot)

        stream = graph.stream_events(
            {"messages": [{"role": "user", "content": "what's my name?"}]},
            config, # [!code highlight]
```



```

        version="v3",
    )
    for snapshot in stream.values:
        print(snapshot)

```

Async

```

from langchain.chat_models import init_chat_model
from langgraph.graph import StateGraph, MessagesState, START
from langgraph.checkpoint.mongodb.aio import AsyncMongoDBSaver # [!code highlight]

model = init_chat_model(model="claude-haiku-4-5-20251001")

MONGODB_URI = "localhost:27017"
async with AsyncMongoDBSaver.from_conn_string(MONGODB_URI) as checkpoint: # [!code highli
    ght]

    async def call_model(state: MessagesState):
        response = await model.ainvoke(state["messages"])
        return {"messages": response}

    builder = StateGraph(MessagesState)
    builder.add_node(call_model)
    builder.add_edge(START, "call_model")

    graph = builder.compile(checkpointer=checkpoint) # [!code highlight]

    config = {
        "configurable": {
            "thread_id": "1" # [!code highlight]
        }
    }

    stream = await graph.astream_events(
        {"messages": [{"role": "user", "content": "hi! I'm bob"}]},
        config, # [!code highlight]
        version="v3",
    )
    async for message in stream.messages:
        async for token in message.text:
            print(token, end="", flush=True)

            stream = await graph.astream_events(
                {"messages": [{"role": "user", "content": "what's my name?"}]},
                config, # [!code highlight]
                version="v3",
            )
            async for message in stream.messages:

```

```
async for token in message.text:
    print(token, end="", flush=True)
```

js

```
npm install @langchain/langgraph-checkpoint-mongodb
```

팁

Setup To use `MongoDBSaver`, you will need a MongoDB cluster. Follow [this guide](#) to create a cluster if you don't already have one. For an agent-focused walkthrough, see [short-term memory with MongoDB Atlas](#).

```
import { ChatAnthropic } from "@langchain/anthropic";
import { StateGraph, StateSchema, MessagesValue, GraphNode, START } from "@langchain/langgraph";
import { MongoDBSaver } from "@langchain/langgraph-checkpoint-mongodb";
import { MongoClient } from "mongodb";

const State = new StateSchema({
  messages: MessagesValue,
});

const model = new ChatAnthropic({ model: "claude-haiku-4-5-20251001" });

const client = new MongoClient("mongodb://user:password@localhost:27017");
const checkpointer = new MongoDBSaver({ client, dbName: "langgraph" });

const callModel: GraphNode<typeof State> = async (state) => {
  const response = await model.invoke(state.messages);
  return { messages: [response] };
};

const builder = new StateGraph(State)
  .addNode("call_model", callModel)
  .addEdge(START, "call_model");

const graph = builder.compile({ checkpointer });

const config = { configurable: { thread_id: "1" } };

const stream1 = await graph.streamEvents(
  { messages: [{ role: "user", content: "hi! I'm bob" } ] },
  { ...config, version: "v3" }
);

for await (const snapshot of stream1.values) {
  console.log(snapshot);
}
```

```

}

const stream2 = await graph.streamEvents(
  { messages: [{ role: "user", content: "what's my name?" }] },
  { ...config, version: "v3" }
);
for await (const snapshot of stream2.values) {
  console.log(snapshot);
}

```

python

Example: using Redis checkpointer

```
pip install -U langgraph langgraph-checkpoint-redis
```

팁

You need to call `checkpointer.setup()` the first time you're using Redis checkpointer.

Sync

```

from langchain.chat_models import init_chat_model
from langgraph.graph import StateGraph, MessagesState, START
from langgraph.checkpoint.redis import RedisSaver # [!code highlight]

model = init_chat_model(model="claude-haiku-4-5-20251001")

DB_URI = "redis://localhost:6379"
with RedisSaver.from_conn_string(DB_URI) as checkpointer: # [!code highlight]
    # checkpointer.setup()

def call_model(state: MessagesState):
    response = model.invoke(state["messages"])
    return {"messages": response}

builder = StateGraph(MessagesState)
builder.add_node(call_model)
builder.add_edge(START, "call_model")

graph = builder.compile(checkpointer=checkpointer) # [!code highlight]

config = {
    "configurable": {
        "thread_id": "1" # [!code highlight]
    }
}

```

```

stream = graph.stream_events(
    {"messages": [{"role": "user", "content": "hi! I'm bob"}]},
    config, # [!code highlight]
    version="v3",
)
for snapshot in stream.values:
    print(snapshot)

stream = graph.stream_events(
    {"messages": [{"role": "user", "content": "what's my name?"}]},
    config, # [!code highlight]
    version="v3",
)
for snapshot in stream.values:
    print(snapshot)

```

Async

```

from langchain.chat_models import init_chat_model
from langgraph.graph import StateGraph, MessagesState, START
from langgraph.checkpoint.redis.aio import AsyncRedisSaver # [!code highlight]

model = init_chat_model(model="claude-haiku-4-5-20251001")

DB_URI = "redis://localhost:6379"
async with AsyncRedisSaver.from_conn_string(DB_URI) as checkpointer: # [!code highlight]
    # await checkpointer.asetup()

    async def call_model(state: MessagesState):
        response = await model.ainvoke(state["messages"])
        return {"messages": response}

    builder = StateGraph(MessagesState)
    builder.add_node(call_model)
    builder.add_edge(START, "call_model")

    graph = builder.compile(checkpointer=checkpointer) # [!code highlight]

    config = {
        "configurable": {
            "thread_id": "1" # [!code highlight]
        }
    }

    stream = await graph.astream_events(
        {"messages": [{"role": "user", "content": "hi! I'm bob"}]},
        config, # [!code highlight]
        version="v3",
    )

```

```

async for message in stream.messages:
    async for token in message.text:
        print(token, end="", flush=True)

    stream = await graph.astream_events(
        {"messages": [{"role": "user", "content": "what's my name?"}]},
        config, # [!code highlight]
        version="v3",
    )
    async for message in stream.messages:
        async for token in message.text:
            print(token, end="", flush=True)

```

Example: using Oracle checkpointer

```

pip install -U langgraph langgraph-oracledb

```

참고

Setup To use the [Oracle checkpointer](#), you will need an Oracle AI Database instance. A local container (for example `gvenzl/oracle-free:23-slim`) or an Oracle Autonomous Database in OCI both work.

팁

You need to call `checkpointer.setup()` the first time you're using the Oracle checkpointer.

Sync

```

from langchain.chat_models import init_chat_model
from langgraph.graph import StateGraph, MessagesState, START
from langgraph_oracledb.checkpoint.oracle import OracleSaver # [!code highlight]

model = init_chat_model(model="claude-haiku-4-5-20251001")

DB_URI = "user/password@localhost:1521/FREEDB1"
with OracleSaver.from_conn_string(DB_URI) as checkpointer: # [!code highlight]
    # checkpointer.setup()

def call_model(state: MessagesState):
    response = model.invoke(state["messages"])
    return {"messages": response}

builder = StateGraph(MessagesState)
builder.add_node(call_model)
builder.add_edge(START, "call_model")

```

```

graph = builder.compile(checkpointer=checkpointer) # [!code highlight]

config = {
    "configurable": {
        "thread_id": "1" # [!code highlight]
    }
}

stream = graph.stream_events(
    {"messages": [{"role": "user", "content": "hi! I'm bob"}]},
    config, # [!code highlight]
    version="v3",
)
for snapshot in stream.values:
    print(snapshot)

    stream = graph.stream_events(
        {"messages": [{"role": "user", "content": "what's my name?"}]},
        config, # [!code highlight]
        version="v3",
    )
    for snapshot in stream.values:
        print(snapshot)

```

Async

```

from langchain.chat_models import init_chat_model
from langgraph.graph import StateGraph, MessagesState, START
from langgraph_oracledb.checkpoint.oracle import AsyncOracleSaver # [!code highlight]

model = init_chat_model(model="claude-haiku-4-5-20251001")

DB_URI = "user/password@localhost:1521/FREEDB1"
async with AsyncOracleSaver.from_conn_string(DB_URI) as checkpointer: # [!code highlight]
    # await checkpointer.setup()

    async def call_model(state: MessagesState):
        response = await model.ainvoke(state["messages"])
        return {"messages": response}

    builder = StateGraph(MessagesState)
    builder.add_node(call_model)
    builder.add_edge(START, "call_model")

    graph = builder.compile(checkpointer=checkpointer) # [!code highlight]

    config = {
        "configurable": {
            "thread_id": "1" # [!code highlight]
        }
    }

```

```

}

stream = await graph.astream_events(
    {"messages": [{"role": "user", "content": "hi! I'm bob"}]},
    config, # [!code highlight]
    version="v3",
)
async for message in stream.messages:
    async for token in message.text:
        print(token, end="", flush=True)

        stream = await graph.astream_events(
            {"messages": [{"role": "user", "content": "what's my name?"}]},
            config, # [!code highlight]
            version="v3",
        )
    async for message in stream.messages:
        async for token in message.text:
            print(token, end="", flush=True)

```

Use in subgraphs

If your graph contains [subgraphs](#), you only need to provide the checkpointer when compiling the parent graph. LangGraph will automatically propagate the checkpointer to the child subgraphs.

python

```

from langgraph.graph import START, StateGraph
from langgraph.checkpoint.memory import InMemorySaver
from typing import TypedDict

class State(TypedDict):
    foo: str

# Subgraph

def subgraph_node_1(state: State):
    return {"foo": state["foo"] + "bar"}

subgraph_builder = StateGraph(State)
subgraph_builder.add_node(subgraph_node_1)
subgraph_builder.add_edge(START, "subgraph_node_1")
subgraph = subgraph_builder.compile() # [!code highlight]

# Parent graph

builder = StateGraph(State)
builder.add_node("node_1", subgraph) # [!code highlight]

```

```
builder.add_edge(START, "node_1")

checkpointer = InMemorySaver()
graph = builder.compile(checkpointer=checkpointer) # [!code highlight]
```

js

```
import { StateGraph, StateSchema, START, MemorySaver } from "@langchain/langgraph";
import { z } from "zod/v4";

const State = new StateSchema({ foo: z.string() });

const subgraphBuilder = new StateGraph(State)
  .addNode("subgraph_node_1", (state) => {
    return { foo: state.foo + "bar" };
  })
  .addEdge(START, "subgraph_node_1");
const subgraph = subgraphBuilder.compile();

const builder = new StateGraph(State)
  .addNode("node_1", subgraph)
  .addEdge(START, "node_1");

const checkpointer = new MemorySaver();
const graph = builder.compile({ checkpointer });
```

You can configure subgraph-specific checkpointing behavior. See [subgraph persistence](#) for details on persistence levels including interrupt support and stateful continuations.

python

```
subgraph_builder = StateGraph(...)
subgraph = subgraph_builder.compile(checkpointer=True) # [!code highlight]
```

js

```
const subgraphBuilder = new StateGraph(...);
const subgraph = subgraphBuilder.compile({ checkpointer: true }); // [!code highlight]
```

Add long-term memory

Use long-term memory to store user-specific or application-specific data across conversations.

python

```
from langgraph.store.memory import InMemoryStore # [!code highlight]
from langgraph.graph import StateGraph

store = InMemoryStore() # [!code highlight]

builder = StateGraph(...)
graph = builder.compile(store=store) # [!code highlight]
```

js

```
import { InMemoryStore, StateGraph } from "@langchain/langgraph";

const store = new InMemoryStore();

const builder = new StateGraph(...);
const graph = builder.compile({ store });
```

Access the store inside nodes

Once you compile a graph with a store, LangGraph automatically injects the store into your node functions. The recommended way to access the store is through the `Runtime` object.

python

```
from dataclasses import dataclass
from langgraph.runtime import Runtime
from langgraph.graph import StateGraph, MessagesState, START
import uuid

@dataclass
class Context:
    user_id: str

async def call_model(state: MessagesState, runtime: Runtime[Context]): # [!code highlight]
    user_id = runtime.context.user_id # [!code highlight]
    namespace = (user_id, "memories")

    # Search for relevant memories
    memories = await runtime.store.asearch( # [!code highlight]
        namespace, query=state["messages"][-1].content, limit=3
    )
    info = "\n".join([d.value["data"] for d in memories])

    # ... Use memories in model call
```

```

# Store a new memory
await runtime.store.aput( # [!code highlight]
    namespace, str(uuid.uuid4()), {"data": "User prefers dark mode"}
)

builder = StateGraph(MessagesState, context_schema=Context) # [!code highlight]
builder.add_node(call_model)
builder.add_edge(START, "call_model")
graph = builder.compile(store=store)

# Pass context at invocation time
graph.invoke(
    {"messages": [{"role": "user", "content": "hi"}]},
    {"configurable": {"thread_id": "1"}},
    context=Context(user_id="1"), # [!code highlight]
)

```

js

```

import { StateGraph, StateSchema, MessagesValue, GraphNode, START } from "@langchain/langgraph";

const State = new StateSchema({
  messages: MessagesValue,
});

const callModel: GraphNode<typeof State> = async (state, runtime) => {
  const userId = runtime.context?.userId;
  const namespace = [userId, "memories"];

  // Search for relevant memories
  const memories = await runtime.store?.search(namespace, {
    query: state.messages.at(-1)?.content,
    limit: 3,
  });
  const info = memories?.map((d) => d.value.data).join("\n") || "";

  // ... Use memories in model call

  // Store a new memory
  await runtime.store?.put(namespace, crypto.randomUUID(), { data: "User prefers dark mode"
});
};

const builder = new StateGraph(State)
  .addNode("call_model", callModel)
  .addEdge(START, "call_model");
const graph = builder.compile({ store });

```

```
// Pass context at invocation time
await graph.invoke(
  { messages: [{ role: "user", content: "hi" }] },
  { configurable: { thread_id: "1" }, context: { userId: "1" } }
);
```

Use in production

In production, use a store backed by a database:

python

```
from langgraph.store.postgres import PostgresStore

DB_URI = "postgresql://postgres:postgres@localhost:5432/postgres?sslmode=disable"
with PostgresStore.from_conn_string(DB_URI) as store: # [!code highlight]
    builder = StateGraph(...)
    graph = builder.compile(store=store) # [!code highlight]
```

js

Postgres

```
import { PostgresStore } from "@langchain/langgraph-checkpoint-postgres/store";

const DB_URI = "postgresql://postgres:postgres@localhost:5432/postgres?sslmode=disable";
const store = PostgresStore.fromConnString(DB_URI);

const builder = new StateGraph(...);
const graph = builder.compile({ store });
```

MongoDB

```
import { MongoDBStore } from "@langchain/langgraph-checkpoint-mongodb";

const MONGODB_URI = "mongodb://user:password@localhost:27017";
const store = await MongoDBStore.fromConnString(MONGODB_URI, {
  dbName: "langgraph",
  collectionName: "store",
});

const builder = new StateGraph(...);
const graph = builder.compile({ store });
```

Example: using Postgres store

python

```
pip install -U "psycopg[binary,pool]" langgraph langgraph-checkpoint-postgres
```

팁

You need to call `store.setup()` the first time you're using Postgres store

Async

```
from dataclasses import dataclass
from langchain.chat_models import init_chat_model
from langgraph.graph import StateGraph, MessagesState, START
from langgraph.checkpoint.postgres.aio import AsyncPostgresSaver
from langgraph.store.postgres.aio import AsyncPostgresStore # [!code highlight]
from langgraph.runtime import Runtime # [!code highlight]
import uuid

model = init_chat_model(model="claude-haiku-4-5-20251001")

@dataclass
class Context:
    user_id: str

async def call_model( # [!code highlight]
    state: MessagesState,
    runtime: Runtime[Context], # [!code highlight]
):
    user_id = runtime.context.user_id # [!code highlight]
    namespace = ("memories", user_id)
    memories = await runtime.store.asearch(namespace, query=str(state["messages"][-1].content)) # [!code highlight]
    info = "\n".join([d.value["data"] for d in memories])
    system_msg = f"You are a helpful assistant talking to the user. User info: {info}"

    # Store new memories if the user asks the model to remember
    last_message = state["messages"][-1]
    if "remember" in last_message.content.lower():
        memory = "User name is Bob"
        await runtime.store.aput(namespace, str(uuid.uuid4()), {"data": memory}) # [!code highlight]

    response = await model.ainvoke(
        [{"role": "system", "content": system_msg}] + state["messages"]
    )
    return {"messages": response}
```

```

DB_URI = "postgresql://postgres:postgres@localhost:5432/postgres?sslmode=disable"

async with (
    AsyncPostgresStore.from_conn_string(DB_URI) as store, # [!code highlight]
    AsyncPostgresSaver.from_conn_string(DB_URI) as checkpointer,
):
    # await store.setup()
    # await checkpointer.setup()

builder = StateGraph(MessagesState, context_schema=Context) # [!code highlight]
builder.add_node(call_model)
builder.add_edge(START, "call_model")

graph = builder.compile(
    checkpointer=checkpointer,
    store=store, # [!code highlight]
)

config = {"configurable": {"thread_id": "1"}}
stream = await graph.astream_events(
    {"messages": [{"role": "user", "content": "Hi! Remember: my name is Bob"}]},
    config,
    version="v3",
    context=Context(user_id="1"), # [!code highlight]
)
async for message in stream.messages:
    async for token in message.text:
        print(token, end="", flush=True)

    config = {"configurable": {"thread_id": "2"}}
    stream = await graph.astream_events(
        {"messages": [{"role": "user", "content": "what is my name?"}]},
        config,
        version="v3",
        context=Context(user_id="1"), # [!code highlight]
    )
    async for message in stream.messages:
        async for token in message.text:
            print(token, end="", flush=True)

```

Sync

```

from dataclasses import dataclass
from langchain.chat_models import init_chat_model
from langgraph.graph import StateGraph, MessagesState, START
from langgraph.checkpoint.postgres import PostgresSaver
from langgraph.store.postgres import PostgresStore # [!code highlight]
from langgraph.runtime import Runtime # [!code highlight]
import uuid

```

```

model = init_chat_model(model="claude-haiku-4-5-20251001")

@dataclass
class Context:
    user_id: str

def call_model( # [!code highlight]
    state: MessagesState,
    runtime: Runtime[Context], # [!code highlight]
):
    user_id = runtime.context.user_id # [!code highlight]
    namespace = ("memories", user_id)
    memories = runtime.store.search(namespace, query=str(state["messages"][-1].content)) #
[!code highlight]
    info = "\n".join([d.value["data"] for d in memories])
    system_msg = f"You are a helpful assistant talking to the user. User info: {info}"

# Store new memories if the user asks the model to remember
last_message = state["messages"][-1]
if "remember" in last_message.content.lower():
    memory = "User name is Bob"
    runtime.store.put(namespace, str(uuid.uuid4()), {"data": memory}) # [!code highlight]

response = model.invoke(
    [{"role": "system", "content": system_msg}] + state["messages"]
)
return {"messages": response}

DB_URI = "postgresql://postgres:postgres@localhost:5432/postgres?sslmode=disable"

with (
    PostgresStore.from_conn_string(DB_URI) as store, # [!code highlight]
    PostgresSaver.from_conn_string(DB_URI) as checkpointer,
):
    # store.setup()
    # checkpointer.setup()

builder = StateGraph(MessagesState, context_schema=Context) # [!code highlight]
builder.add_node(call_model)
builder.add_edge(START, "call_model")

graph = builder.compile(
    checkpointer=checkpointer,
    store=store, # [!code highlight]
)

config = {"configurable": {"thread_id": "1"}}
stream = graph.stream_events(
    {"messages": [{"role": "user", "content": "Hi! Remember: my name is Bob"}]},
    config,

```

```

    version="v3",
    context=Context(user_id="1"), # [!code highlight]
)
for snapshot in stream.values:
    print(snapshot)

    config = {"configurable": {"thread_id": "2"}}
    stream = graph.stream_events(
        {"messages": [{"role": "user", "content": "what is my name?"}]},
        config,
        version="v3",
        context=Context(user_id="1"), # [!code highlight]
    )
    for snapshot in stream.values:
        print(snapshot)

```

js

```
npm install @langchain/langgraph-checkpoint-postgres
```

팁

You need to call `store.setup()` the first time you're using Postgres store

```

import { ChatAnthropic } from "@langchain/anthropic";
import { StateGraph, StateSchema, MessagesValue, GraphNode, START } from "@langchain/langgraph";
import { PostgresSaver } from "@langchain/langgraph-checkpoint-postgres";
import { PostgresStore } from "@langchain/langgraph-checkpoint-postgres/store";

const State = new StateSchema({
  messages: MessagesValue,
});

const model = new ChatAnthropic({ model: "claude-haiku-4-5-20251001" });

const callModel: GraphNode<typeof State> = async (state, runtime) => {
  const userId = runtime.context?.userId;
  const namespace = ["memories", userId];
  const memories = await runtime.store?.search(namespace, { query: state.messages.at(-1)?.content });
  const info = memories?.map(d => d.value.data).join("\n") || "";
  const systemMsg = `You are a helpful assistant talking to the user. User info: ${info}`;

  // Store new memories if the user asks the model to remember
  const lastMessage = state.messages.at(-1);
  if (lastMessage?.content?.toLowerCase().includes("remember")) {

```

```

const memory = "User name is Bob";
await runtime.store?.put(namespace, crypto.randomUUID(), { data: memory });
}

const response = await model.invoke([
  { role: "system", content: systemMsg },
  ...state.messages
]);
return { messages: [response] };
};

const DB_URI = "postgresql://postgres:postgres@localhost:5432/postgres?sslmode=disable";

const store = PostgresStore.fromConnString(DB_URI);
const checkpointer = PostgresSaver.fromConnString(DB_URI);
// await store.setup();
// await checkpointer.setup();

const builder = new StateGraph(State)
  .addNode("call_model", callModel)
  .addEdge(START, "call_model");

const graph = builder.compile({
  checkpointer,
  store,
});

const stream1 = await graph.streamEvents(
  { messages: [{ role: "user", content: "Hi! Remember: my name is Bob" }] },
  { configurable: { thread_id: "1" }, context: { userId: "1" }, version: "v3" }
);
for await (const snapshot of stream1.values) {
  console.log(snapshot);
}

const stream2 = await graph.streamEvents(
  { messages: [{ role: "user", content: "what is my name?" }] },
  { configurable: { thread_id: "2" }, context: { userId: "1" }, version: "v3" }
);
for await (const snapshot of stream2.values) {
  console.log(snapshot);
}

```

Example: using MongoDB store

js

```
npm install @langchain/langgraph-checkpoint-mongodb
```


팁

For an agent-focused walkthrough, see [long-term memory with MongoDB Atlas](#).

```
import { ChatAnthropic } from "@langchain/anthropic";
import { MemorySaver, StateGraph, StateSchema, MessagesValue, GraphNode, START } from "@langchain/langgraph";
import { MongoDBStore } from "@langchain/langgraph-checkpoint-mongodb";

const State = new StateSchema({
  messages: MessagesValue,
});

const model = new ChatAnthropic({ model: "claude-sonnet-4-6" });

const callModel: GraphNode<typeof State> = async (state, runtime) => {
  const userId = runtime.context?.userId;
  const namespace = ["memories", userId];
  const memories = await runtime.store?.search(namespace);
  const info = memories?.map(d => d.value.data).join("\n") || "n/a";
  const systemMsg = `You are a helpful assistant talking to the user. User info: ${info}`;

  // Store new memories if the user asks the model to remember
  const lastMessage = state.messages.at(-1);
  if (lastMessage?.content?.toLowerCase().includes("remember")) {
    const memory = "User name is Bob";
    await runtime.store?.put(namespace, crypto.randomUUID(), { data: memory });
  }

  const response = await model.invoke([
    { role: "system", content: systemMsg },
    ...state.messages
  ]);
  return { messages: [response] };
};

const MONGODB_URI = "mongodb://user:password@localhost:27017";

const store = await MongoDBStore.fromConnString(MONGODB_URI, {
  dbName: "langgraph",
  collectionName: "store",
});

const checkpointer = new MemorySaver();

const builder = new StateGraph(State)
  .addNode("call_model", callModel)
  .addEdge(START, "call_model");
```

```

const graph = builder.compile({ checkpointer, store });

const stream1 = await graph.streamEvents(
  { messages: [{ role: "user", content: "Hi! Remember: my name is Bob" }] },
  { configurable: { thread_id: "1" }, context: { userId: "1" }, version: "v3" }
);
for await (const snapshot of stream1.values) {
  console.log(snapshot);
}

const stream2 = await graph.streamEvents(
  { messages: [{ role: "user", content: "what is my name?" }] },
  { configurable: { thread_id: "2" }, context: { userId: "1" }, version: "v3" }
);
for await (const snapshot of stream2.values) {
  console.log(snapshot);
}

```

python

Example: using Redis store

```
pip install -U langgraph langgraph-checkpoint-redis
```

팁

You need to call `store.setup()` the first time you're using [Redis store](#).

Async

```

from dataclasses import dataclass
from langchain.chat_models import init_chat_model
from langgraph.graph import StateGraph, MessagesState, START
from langgraph.checkpoint.redis.aio import AsyncRedisSaver
from langgraph.store.redis.aio import AsyncRedisStore # [!code highlight]
from langgraph.runtime import Runtime # [!code highlight]
import uuid

model = init_chat_model(model="claude-haiku-4-5-20251001")

@dataclass
class Context:
    user_id: str

async def call_model( # [!code highlight]
    state: MessagesState,
    runtime: Runtime[Context], # [!code highlight]

```

```

):
    user_id = runtime.context.user_id # [!code highlight]
    namespace = ("memories", user_id)
    memories = await runtime.store.asearch(namespace, query=str(state["messages"][-1].content)) # [!code highlight]
    info = "\n".join([d.value["data"] for d in memories])
    system_msg = f"You are a helpful assistant talking to the user. User info: {info}"

# Store new memories if the user asks the model to remember
last_message = state["messages"][-1]
if "remember" in last_message.content.lower():
    memory = "User name is Bob"
    await runtime.store.aput(namespace, str(uuid.uuid4()), {"data": memory}) # [!code highlight]

response = await model.ainvoke(
    [{"role": "system", "content": system_msg}] + state["messages"]
)
return {"messages": response}

DB_URI = "redis://localhost:6379"

async with (
    AsyncRedisStore.from_conn_string(DB_URI) as store, # [!code highlight]
    AsyncRedisSaver.from_conn_string(DB_URI) as checkpointer,
):
    # await store.setup()
    # await checkpointer.asetup()

builder = StateGraph(MessagesState, context_schema=Context) # [!code highlight]
builder.add_node(call_model)
builder.add_edge(START, "call_model")

graph = builder.compile(
    checkpointer=checkpointer,
    store=store, # [!code highlight]
)

config = {"configurable": {"thread_id": "1"}}
stream = await graph.astream_events(
    {"messages": [{"role": "user", "content": "Hi! Remember: my name is Bob"}]},
    config,
    version="v3",
    context=Context(user_id="1"), # [!code highlight]
)
async for snapshot in stream.values:
    snapshot["messages"][-1].pretty_print()

    config = {"configurable": {"thread_id": "2"}}
    stream = await graph.astream_events(
        {"messages": [{"role": "user", "content": "what is my name?"}]},

```

```

        config,
        version="v3",
        context=Context(user_id="1"), # [!code highlight]
    )
    async for snapshot in stream.values:
        snapshot["messages"][-1].pretty_print()

```

Sync

```

from dataclasses import dataclass
from langchain.chat_models import init_chat_model
from langgraph.graph import StateGraph, MessagesState, START
from langgraph.checkpoint.redis import RedisSaver
from langgraph.store.redis import RedisStore # [!code highlight]
from langgraph.runtime import Runtime # [!code highlight]
import uuid

model = init_chat_model(model="claude-haiku-4-5-20251001")

@dataclass
class Context:
    user_id: str

def call_model( # [!code highlight]
    state: MessagesState,
    runtime: Runtime[Context], # [!code highlight]
):
    user_id = runtime.context.user_id # [!code highlight]
    namespace = ("memories", user_id)
    memories = runtime.store.search(namespace, query=str(state["messages"][-1].content)) # [!code highlight]
    info = "\n".join([d.value["data"] for d in memories])
    system_msg = f"You are a helpful assistant talking to the user. User info: {info}"

# Store new memories if the user asks the model to remember
last_message = state["messages"][-1]
if "remember" in last_message.content.lower():
    memory = "User name is Bob"
    runtime.store.put(namespace, str(uuid.uuid4()), {"data": memory}) # [!code highlight]

response = model.invoke(
    [{"role": "system", "content": system_msg}] + state["messages"]
)
return {"messages": response}

DB_URI = "redis://localhost:6379"

with (
    RedisStore.from_conn_string(DB_URI) as store, # [!code highlight]
    RedisSaver.from_conn_string(DB_URI) as checkpointer,

```

```

):
    store.setup()
    checkpointer.setup()

builder = StateGraph(MessagesState, context_schema=Context) # [!code highlight]
builder.add_node(call_model)
builder.add_edge(START, "call_model")

graph = builder.compile(
    checkpointer=checkpointer,
    store=store, # [!code highlight]
)

config = {"configurable": {"thread_id": "1"}}
stream = graph.stream_events(
    {"messages": [{"role": "user", "content": "Hi! Remember: my name is Bob"}]},
    config,
    version="v3",
    context=Context(user_id="1"), # [!code highlight]
)
for snapshot in stream.values:
    snapshot["messages"][-1].pretty_print()

    config = {"configurable": {"thread_id": "2"}}
    stream = graph.stream_events(
        {"messages": [{"role": "user", "content": "what is my name?"}]},
        config,
        version="v3",
        context=Context(user_id="1"), # [!code highlight]
    )
    for snapshot in stream.values:
        snapshot["messages"][-1].pretty_print()

```

Example: using Oracle store

```
pip install -U langgraph langgraph-oracledb langchain-openai
```

참고

Setup To use the [Oracle store](#), you will need an Oracle AI Database instance — the vector index used for semantic search requires [Oracle AI Vector Search](#).

팁

You need to call `store.setup()` and `checkpointer.setup()` the first time you're using the Oracle store and checkpointer.

Sync

```
import uuid

from langchain.chat_models import init_chat_model
from langchain.embeddings import init_embeddings
from langchain_core.runnables import RunnableConfig
from langgraph.graph import StateGraph, MessagesState, START
from langgraph.store.base import BaseStore
from langgraph_oracledb.checkpoint.oracle import OracleSaver
from langgraph_oracledb.store.oracle import OracleStore # [!code highlight]

model = init_chat_model(model="claude-haiku-4-5-20251001")
embeddings = init_embeddings("openai:text-embedding-3-small")

DB_URI = "user/password@localhost:1521/FREEPDB1"

with (
    OracleStore.from_conn_string( # [!code highlight]
        DB_URI,
        index={"embed": embeddings, "dims": 1536}, # [!code highlight]
    ) as store,
    OracleSaver.from_conn_string(DB_URI) as checkpointer,
):
    store.setup()
    checkpointer.setup()

def call_model(
    state: MessagesState,
    config: RunnableConfig,
    *,
    store: BaseStore, # [!code highlight]
):
    user_id = config["configurable"]["user_id"]
    namespace = ("memories", user_id)
    memories = store.search(namespace, query=str(state["messages"][-1].content)) # [!code h
ighlight]
    info = "\n".join([d.value["data"] for d in memories])
    system_msg = f"You are a helpful assistant talking to the user. User info: {info}"

    # Store new memories if the user asks the model to remember
    last_message = state["messages"][-1]
    if "remember" in last_message.content.lower():
        memory = "User name is Bob"
        store.put(namespace, str(uuid.uuid4()), {"data": memory}) # [!code highlight]

    response = model.invoke(
        [{"role": "system", "content": system_msg}] + state["messages"]
    )
    return {"messages": response}
```

```

builder = StateGraph(MessagesState)
builder.add_node(call_model)
builder.add_edge(START, "call_model")

graph = builder.compile(
    checkpointer=checkpointer,
    store=store, # [!code highlight]
)

config = {
    "configurable": {
        "thread_id": "1", # [!code highlight]
        "user_id": "1", # [!code highlight]
    }
}

stream = graph.stream_events(
    {"messages": [{"role": "user", "content": "Hi! Remember: my name is Bob"}]},
    config, # [!code highlight]
    version="v3",
)

for snapshot in stream.values:
    snapshot["messages"][-1].pretty_print()

config = {
    "configurable": {
        "thread_id": "2", # [!code highlight]
        "user_id": "1",
    }
}

stream = graph.stream_events(
    {"messages": [{"role": "user", "content": "what is my name?"}]},
    config, # [!code highlight]
    version="v3",
)

for snapshot in stream.values:
    snapshot["messages"][-1].pretty_print()

```

Async

```

import uuid

from langchain.chat_models import init_chat_model
from langchain.embeddings import init_embeddings
from langchain_core.runnables import RunnableConfig
from langgraph.graph import StateGraph, MessagesState, START
from langgraph.store.base import BaseStore
from langgraph_oracledb.checkpoint.oracle import AsyncOracleSaver
from langgraph_oracledb.store.oracle import AsyncOracleStore # [!code highlight]

```

```

model = init_chat_model(model="claude-haiku-4-5-20251001")
embeddings = init_embeddings("openai:text-embedding-3-small")

DB_URI = "user/password@localhost:1521/FREEDB1"

async with (
    AsyncOracleStore.from_conn_string( # [!code highlight]
        DB_URI,
        index={"embed": embeddings, "dims": 1536}, # [!code highlight]
    ) as store,
    AsyncOracleSaver.from_conn_string(DB_URI) as checkpointer,
):
    await store.setup()
    await checkpointer.setup()

async def call_model(
    state: MessagesState,
    config: RunnableConfig,
    *,
    store: BaseStore, # [!code highlight]
):
    user_id = config["configurable"]["user_id"]
    namespace = ("memories", user_id)
    memories = await store.asearch(namespace, query=str(state["messages"][-1].content)) #
[!code highlight]
    info = "\n".join([d.value["data"] for d in memories])
    system_msg = f"You are a helpful assistant talking to the user. User info: {info}"

# Store new memories if the user asks the model to remember
last_message = state["messages"][-1]
if "remember" in last_message.content.lower():
    memory = "User name is Bob"
    await store.aput(namespace, str(uuid.uuid4()), {"data": memory}) # [!code highlight]

response = await model.ainvoke(
    [{"role": "system", "content": system_msg}] + state["messages"]
)
return {"messages": response}

builder = StateGraph(MessagesState)
builder.add_node(call_model)
builder.add_edge(START, "call_model")

graph = builder.compile(
    checkpointer=checkpointer,
    store=store, # [!code highlight]
)

config = {
    "configurable": {

```



```

        "thread_id": "1", # [!code highlight]
        "user_id": "1", # [!code highlight]
    }
}
stream = await graph.astream_events(
    {"messages": [{"role": "user", "content": "Hi! Remember: my name is Bob"}]},
    config, # [!code highlight]
    version="v3",
)
async for snapshot in stream.values:
    snapshot["messages"][-1].pretty_print()

config = {
    "configurable": {
        "thread_id": "2", # [!code highlight]
        "user_id": "1",
    }
}

    stream = await graph.astream_events(
        {"messages": [{"role": "user", "content": "what is my name?"}]},
        config, # [!code highlight]
        version="v3",
    )
    async for snapshot in stream.values:
        snapshot["messages"][-1].pretty_print()

```

Use semantic search

Enable semantic search in your graph's memory store to let graph agents search for items in the store by semantic similarity.

python

```

from langchain.embeddings import init_embeddings
from langgraph.store.memory import InMemoryStore

# Create store with semantic search enabled
embeddings = init_embeddings("openai:text-embedding-3-small")
store = InMemoryStore(
    index={
        "embed": embeddings,
        "dims": 1536,
    }
)

store.put(("user_123", "memories"), "1", {"text": "I love pizza"})
store.put(("user_123", "memories"), "2", {"text": "I am a plumber"})

```

```
items = store.search(
    ("user_123", "memories"), query="I'm hungry", limit=1
)
```

js

```
import { OpenAIEmbeddings } from "@langchain/openai";
import { InMemoryStore } from "@langchain/langgraph";

// Create store with semantic search enabled
const embeddings = new OpenAIEmbeddings({ model: "text-embedding-3-small" });
const store = new InMemoryStore({
  index: {
    embeddings,
    dims: 1536,
  },
});

await store.put(["user_123", "memories"], "1", { text: "I love pizza" });
await store.put(["user_123", "memories"], "2", { text: "I am a plumber" });

const items = await store.search(["user_123", "memories"], {
  query: "I'm hungry",
  limit: 1,
});
```

js

팁

`InMemoryStore` is suitable for development. For production, use a persistent store like `PostgresStore`, `MongoDBStore`, OR `RedisStore`.

Long-term memory with semantic search

python

```
from langchain.embeddings import init_embeddings
from langchain.chat_models import init_chat_model
from langgraph.store.memory import InMemoryStore
from langgraph.graph import START, MessagesState, StateGraph
from langgraph.runtime import Runtime # [!code highlight]
```

```

model = init_chat_model("gpt-5.4-mini")

# Create store with semantic search enabled
embeddings = init_embeddings("openai:text-embedding-3-small")
store = InMemoryStore(
    index={
        "embed": embeddings,
        "dims": 1536,
    }
)

store.put(("user_123", "memories"), "1", {"text": "I love pizza"})
store.put(("user_123", "memories"), "2", {"text": "I am a plumber"})

async def chat(state: MessagesState, runtime: Runtime): # [!code highlight]
    # Search based on user's last message
    items = await runtime.store.asearch( # [!code highlight]
        ("user_123", "memories"), query=state["messages"][-1].content, limit=2
    )
    memories = "\n".join(item.value["text"] for item in items)
    memories = f"## Memories of user\n{memories}" if memories else ""
    response = await model.ainvoke(
        [
            {"role": "system", "content": f"You are a helpful assistant.\n{memories}"},
            *state["messages"],
        ]
    )
    return {"messages": [response]}

builder = StateGraph(MessagesState)
builder.add_node(chat)
builder.add_edge(START, "chat")
graph = builder.compile(store=store)

stream = await graph.astream_events(
    {"messages": [{"role": "user", "content": "I'm hungry"}]},
    version="v3",
)

async for message in stream.messages:
    async for token in message.text:
        print(token, end="", flush=True)

```

js

InMemoryStore

```

import { OpenAIEmbeddings, ChatOpenAI } from "@langchain/openai";
import { StateGraph, StateSchema, MessagesValue, GraphNode, START, InMemoryStore } from "@langchain/langgraph";

```

```

const State = new StateSchema({
  messages: MessagesValue,
});

const model = new ChatOpenAI({ model: "gpt-5.4-mini" });

// Create store with semantic search enabled
const embeddings = new OpenAIEmbeddings({ model: "text-embedding-3-small" });
const store = new InMemoryStore({
  index: {
    embeddings,
    dims: 1536,
  }
});

await store.put(["user_123", "memories"], "1", { text: "I love pizza" });
await store.put(["user_123", "memories"], "2", { text: "I am a plumber" });

const chat: GraphNode<typeof State> = async (state, runtime) => {
  // Search based on user's last message
  const items = await runtime.store.search(
    ["user_123", "memories"],
    { query: state.messages.at(-1)?.content, limit: 2 }
  );
  const memories = items.map(item => item.value.text).join("\n");
  const memoriesText = memories ? `## Memories of user\n${memories}` : "";

  const response = await model.invoke([
    { role: "system", content: `You are a helpful assistant.\n${memoriesText}` },
    ...state.messages,
  ]);

  return { messages: [response] };
};

const builder = new StateGraph(State)
  .addNode("chat", chat)
  .addEdge(START, "chat");
const graph = builder.compile({ store });

const stream = await graph.streamEvents(
  { messages: [{ role: "user", content: "I'm hungry" }] },
  { version: "v3" }
);
for await (const message of stream.messages) {
  for await (const token of message.text) {
    process.stdout.write(token);
  }
}

```

MongoDB (manual embedding)

```
import { ChatOpenAI, OpenAIEmbeddings } from "@langchain/openai";
import { MongoDBStore } from "@langchain/langgraph-checkpoint-mongodb";
import { StateGraph, StateSchema, MessagesValue, GraphNode, START } from "@langchain/langgraph";

const State = new StateSchema({
  messages: MessagesValue,
});

const model = new ChatOpenAI({ model: "gpt-5.4-mini" });

// Create store with semantic search enabled
const MONGODB_URI = "mongodb://user:password@localhost:27017";
const store = await MongoDBStore.fromConnString(MONGODB_URI, {
  dbName: "langgraph",
  collectionName: "store",
  embeddings: new OpenAIEmbeddings({ model: "text-embedding-3-small" }),
  indexConfig: {
    name: "store_vector_index",
    dims: 1536,
    embeddingKey: "text",
  },
});

await store.put(["user_123", "memories"], "1", { text: "I love pizza" });
await store.put(["user_123", "memories"], "2", { text: "I am a plumber" });

const chat: GraphNode<typeof State> = async (state, runtime) => {
  // Search based on user's last message
  const items = await runtime.store.search(
    ["user_123", "memories"],
    { query: state.messages.at(-1)?.content, limit: 2 }
  );
  const memories = items.map(item => item.value.text).join("\n");
  const memoriesText = memories ? `## Memories of user\n${memories}` : "";

  const response = await model.invoke([
    { role: "system", content: `You are a helpful assistant.\n${memoriesText}` },
    ...state.messages,
  ]);

  return { messages: [response] };
};

const builder = new StateGraph(State)
  .addNode("chat", chat)
  .addEdge(START, "chat");
const graph = builder.compile({ store });
```

```

const stream = await graph.streamEvents(
  { messages: [{ role: "user", content: "I'm hungry" }] },
  { version: "v3" }
);
for await (const message of stream.messages) {
  for await (const token of message.text) {
    process.stdout.write(token);
  }
}

```

MongoDB (auto embedding)

참고

Auto embedding requires MongoDB Atlas. MongoDB generates embeddings server-side via Voyage AI. See the [Automated Embedding documentation](#) for more information.

```

import { StateGraph, StateSchema, MessagesValue, GraphNode, START } from "@langchain/langgraph";
import { MongoDBStore } from "@langchain/langgraph-checkpoint-mongodb";
import { ChatOpenAI } from "@langchain/openai";

const State = new StateSchema({
  messages: MessagesValue,
});

const model = new ChatOpenAI({ model: "gpt-5.4-mini" });

// Auto embedding: no embeddings instance needed.
// Configure the Voyage AI model and the field path MongoDB will read server-side.
const MONGODB_URI = "mongodb://user:password@localhost:27017";
const store = await MongoDBStore.fromConnString(MONGODB_URI, {
  dbName: "langgraph",
  collectionName: "store",
  indexConfig: {
    name: "store_vector_index",
    path: "value.content", // MongoDB reads this field and embeds it server-side
    model: "voyage-4",     // Voyage AI model used by MongoDB Atlas
  },
});

// Values must have the content field matching the configured path (value.content)
await store.put(["user_123", "memories"], "1", { content: "I love pizza" });
await store.put(["user_123", "memories"], "2", { content: "I am a plumber" });

const chat: GraphNode<typeof State> = async (state, runtime) => {
  // MongoDB generates the query embedding server-side
  const items = await runtime.store.search(

```

```

    ["user_123", "memories"],
    { query: state.messages.at(-1)?.content, limit: 2 }
  );
  const memories = items.map(item => item.value.content).join("\n");
  const memoriesText = memories ? `## Memories of user\n${memories}` : "";

  const response = await model.invoke([
    { role: "system", content: `You are a helpful assistant.\n${memoriesText}` },
    ...state.messages,
  ]);

  return { messages: [response] };
};

const builder = new StateGraph(State)
  .addNode("chat", chat)
  .addEdge(START, "chat");
const graph = builder.compile({ store });

const stream = await graph.streamEvents(
  { messages: [{ role: "user", content: "I'm hungry" } ] },
  { version: "v3" }
);
for await (const message of stream.messages) {
  for await (const token of message.text) {
    process.stdout.write(token);
  }
}

```

Manage short-term memory

With [short-term memory](#) enabled, long conversations can exceed the LLM's context window. Common solutions are:

- [Trim messages](#): Remove first or last N messages (before calling LLM)
- [Delete messages](#) from LangGraph state permanently
- [Summarize messages](#): Summarize earlier messages in the history and replace them with a summary
- [Manage checkpoints](#) to store and retrieve message history
- Custom strategies (e.g., message filtering, etc.)

This allows the agent to keep track of the conversation without exceeding the LLM's context window.

Trim messages

python

Most LLMs have a maximum supported context window (denominated in tokens). One way to decide when to truncate messages is to count the tokens in the message history and truncate whenever it approaches that limit. If you're using LangChain, you can use the trim messages utility and specify the number of tokens to keep from the list, as well as the `strategy` (e.g., keep the last `max_tokens`) to use for handling the boundary.

js

Most LLMs have a maximum supported context window (denominated in tokens). One way to decide when to truncate messages is to count the tokens in the message history and truncate whenever it approaches that limit. If you're using LangChain, you can use the trim messages utility and specify the number of tokens to keep from the list, as well as the `strategy` (e.g., keep the last `maxTokens`) to use for handling the boundary.

python

To trim message history, use the [trim_messages](#) function:

```
from langchain_core.messages.utils import ( # [!code highlight]
    trim_messages, # [!code highlight]
    count_tokens_approximately # [!code highlight]
) # [!code highlight]

def call_model(state: MessagesState):
    messages = trim_messages( # [!code highlight]
        state["messages"],
        strategy="last",
        token_counter=count_tokens_approximately,
        max_tokens=128,
        start_on="human",
        end_on=("human", "tool"),
    )
    response = model.invoke(messages)
    return {"messages": [response]}

builder = StateGraph(MessagesState)
builder.add_node(call_model)
...
```

js

To trim message history, use the [trimMessages](#) function:

```
import { trimMessages } from "@langchain/core/messages";
import { StateSchema, MessagesValue, GraphNode } from "@langchain/langgraph";
```



```

const State = new StateSchema({
  messages: MessagesValue,
});

const callModel: GraphNode<typeof State> = async (state) => {
  const messages = trimMessages(state.messages, {
    strategy: "last",
    maxTokens: 128,
    startOn: "human",
    endOn: ["human", "tool"],
  });
  const response = await model.invoke(messages);
  return { messages: [response] };
};

const builder = new StateGraph(State)
  .addNode("call_model", callModel);
// ...

```

Full example: trim messages

python

```

from langchain_core.messages.utils import (
    trim_messages, # [!code highlight]
    count_tokens_approximately # [!code highlight]
)
from langchain.chat_models import init_chat_model
from langgraph.graph import StateGraph, START, MessagesState

model = init_chat_model("claude-sonnet-4-6")
summarization_model = model.bind(max_tokens=128)

def call_model(state: MessagesState):
    messages = trim_messages( # [!code highlight]
        state["messages"],
        strategy="last",
        token_counter=count_tokens_approximately,
        max_tokens=128,
        start_on="human",
        end_on=("human", "tool"),
    )
    response = model.invoke(messages)
    return {"messages": [response]}

checkpointer = InMemorySaver()
builder = StateGraph(MessagesState)
builder.add_node(call_model)

```

```

builder.add_edge(START, "call_model")
graph = builder.compile(checkpointer=checkpointer)

config = {"configurable": {"thread_id": "1"}}
graph.invoke({"messages": "hi, my name is bob"}, config)
graph.invoke({"messages": "write a short poem about cats"}, config)
graph.invoke({"messages": "now do the same but for dogs"}, config)
final_response = graph.invoke({"messages": "what's my name?"}, config)

final_response["messages"][-1].pretty_print()

```

===== Ai Message =====

Your name is Bob, as you mentioned when you first introduced yourself.

js

```

import { trimMessages } from "@langchain/core/messages";
import { ChatAnthropic } from "@langchain/anthropic";
import { StateGraph, StateSchema, MessagesValue, GraphNode, START, MemorySaver } from "@langchain/langgraph";

const State = new StateSchema({
  messages: MessagesValue,
});

const model = new ChatAnthropic({ model: "claude-3-5-sonnet-20241022" });

const callModel: GraphNode<typeof State> = async (state) => {
  const messages = trimMessages(state.messages, {
    strategy: "last",
    maxTokens: 128,
    startOn: "human",
    endOn: ["human", "tool"],
    tokenCounter: model,
  });
  const response = await model.invoke(messages);
  return { messages: [response] };
};

const checkpointer = new MemorySaver();
const builder = new StateGraph(State)
  .addNode("call_model", callModel)
  .addEdge(START, "call_model");
const graph = builder.compile({ checkpointer });

const config = { configurable: { thread_id: "1" } };
await graph.invoke({ messages: [{ role: "user", content: "hi, my name is bob" }] }, config);
await graph.invoke({ messages: [{ role: "user", content: "write a short poem about cats" }]

```

```

    }, config);
    await graph.invoke({ messages: [{ role: "user", content: "now do the same but for dogs" }]
    }, config);
    const finalResponse = await graph.invoke({ messages: [{ role: "user", content: "what's my name?" }] }, config);

    console.log(finalResponse.messages.at(-1)?.content);

```

Your name is Bob, as you mentioned when you first introduced yourself.

Delete messages

You can delete messages from the graph state to manage the message history. This is useful when you want to remove specific messages or clear the entire message history.

python

To delete messages from the graph state, you can use the `RemoveMessage`. For `RemoveMessage` to work, you need to use a state key with `@[add_messages]` [reducer](#), like [MessagesState](#).

To remove specific messages:

```

from langchain.messages import RemoveMessage # [!code highlight]

def delete_messages(state):
    messages = state["messages"]
    if len(messages) > 2:
        # remove the earliest two messages
        return {"messages": [RemoveMessage(id=m.id) for m in messages[:2]]} # [!code highlight]

```

To remove **all** messages:

```

from langgraph.graph.message import REMOVE_ALL_MESSAGES # [!code highlight]

def delete_messages(state):
    return {"messages": [RemoveMessage(id=REMOVE_ALL_MESSAGES)]} # [!code highlight]

```

js

To delete messages from the graph state, you can use the `RemoveMessage`. For `RemoveMessage` to work, you need to use a state key with `@[messagesStateReducer]` [reducer](#), like `MessagesValue`.

To remove specific messages:

```
import { RemoveMessage } from "@langchain/core/messages";

const deleteMessages = (state) => {
  const messages = state.messages;
  if (messages.length > 2) {
    // remove the earliest two messages
    return {
      messages: messages
        .slice(0, 2)
        .map((m) => new RemoveMessage({ id: m.id })),
    };
  }
};
```

주의

When deleting messages, **make sure** that the resulting message history is valid. Check the limitations of the LLM provider you're using. For example:

- Some providers expect message history to start with a `user` message
- Most providers require `assistant` messages with tool calls to be followed by corresponding `tool` result messages.

Full example: delete messages

python

```
from langchain.messages import RemoveMessage # [!code highlight]

def delete_messages(state):
    messages = state["messages"]
    if len(messages) > 2:
        # remove the earliest two messages
        return {"messages": [RemoveMessage(id=m.id) for m in messages[:2]]} # [!code highlight]

def call_model(state: MessagesState):
    response = model.invoke(state["messages"])
    return {"messages": response}

builder = StateGraph(MessagesState)
builder.add_sequence([call_model, delete_messages])
builder.add_edge(START, "call_model")

checkpointer = InMemorySaver()
app = builder.compile(checkpointer=checkpointer)
config = {"configurable": {"thread_id": "1"}}
```

```

stream = app.stream_events(
    {"messages": [{"role": "user", "content": "hi! I'm bob"}]},
    config,
    version="v3"
)
for snapshot in stream.values:
    print([(message.type, message.content) for message in snapshot["messages"]])

stream = app.stream_events(
    {"messages": [{"role": "user", "content": "what's my name?"}]},
    config,
    version="v3"
)
for snapshot in stream.values:
    print([(message.type, message.content) for message in snapshot["messages"]])

```

```

[('human', "hi! I'm bob")]
[('human', "hi! I'm bob"), ('ai', 'Hi Bob! How are you doing today? Is there anything I can help you with?')]
[('human', "hi! I'm bob"), ('ai', 'Hi Bob! How are you doing today? Is there anything I can help you with?'), ('human', "what's my name?")]
[('human', "hi! I'm bob"), ('ai', 'Hi Bob! How are you doing today? Is there anything I can help you with?'), ('human', "what's my name?"), ('ai', 'Your name is Bob.')]
[('human', "what's my name?"), ('ai', 'Your name is Bob.')]

```

js

```

import { RemoveMessage } from "@langchain/core/messages";
import { ChatAnthropic } from "@langchain/anthropic";
import { StateGraph, StateSchema, MessagesValue, GraphNode, START, MemorySaver } from "@langchain/langgraph";

const State = new StateSchema({
    messages: MessagesValue,
});

const model = new ChatAnthropic({ model: "claude-3-5-sonnet-20241022" });

const deleteMessages: GraphNode<typeof State> = (state) => {
    const messages = state.messages;
    if (messages.length > 2) {
        // remove the earliest two messages
        return { messages: messages.slice(0, 2).map(m => new RemoveMessage({ id: m.id })) };
    }
    return {};
};

const callModel: GraphNode<typeof State> = async (state) => {

```

```

const response = await model.invoke(state.messages);
return { messages: [response] };
};

const builder = new StateGraph(State)
  .addNode("call_model", callModel)
  .addNode("delete_messages", deleteMessages)
  .addEdge(START, "call_model")
  .addEdge("call_model", "delete_messages");

const checkpointer = new MemorySaver();
const app = builder.compile({ checkpointer });

const config = { configurable: { thread_id: "1" } };

const stream1 = await app.streamEvents(
  { messages: [{ role: "user", content: "hi! I'm bob" } ] },
  { ...config, version: "v3" }
);
for await (const snapshot of stream1.values) {
  console.log(snapshot.messages.map(message => [message.getType(), message.content]));
}

const stream2 = await app.streamEvents(
  { messages: [{ role: "user", content: "what's my name?" } ] },
  { ...config, version: "v3" }
);
for await (const snapshot of stream2.values) {
  console.log(snapshot.messages.map(message => [message.getType(), message.content]));
}

```

```

[['human', "hi! I'm bob"]]
[['human', "hi! I'm bob"], ['ai', 'Hi Bob! How are you doing today? Is there anything I can help you with?']]
[['human', "hi! I'm bob"], ['ai', 'Hi Bob! How are you doing today? Is there anything I can help you with?'], ['human', "what's my name?"]]
[['human', "hi! I'm bob"], ['ai', 'Hi Bob! How are you doing today? Is there anything I can help you with?'], ['human', "what's my name?"], ['ai', 'Your name is Bob.']]
[['human', "what's my name?"], ['ai', 'Your name is Bob.']]

```

Summarize messages

The problem with trimming or removing messages, as shown above, is that you may lose information from culling of the message queue. Because of this, some applications benefit from a more sophisticated approach of summarizing the message history using a chat model.



Summary

python

Prompting and orchestration logic can be used to summarize the message history. For example, in LangGraph you can extend the [MessagesState](#) to include a `summary` key:

```
from langgraph.graph import MessagesState
class State(MessagesState):
    summary: str
```

Then, you can generate a summary of the chat history, using any existing summary as context for the next summary. This `summarize_conversation` node can be called after some number of messages have accumulated in the `messages` state key.

```
def summarize_conversation(state: State):

    # First, we get any existing summary
    summary = state.get("summary", "")

    # Create our summarization prompt
    if summary:

        # A summary already exists
        summary_message = (
            f"This is a summary of the conversation to date: {summary}\n\n"
            "Extend the summary by taking into account the new messages above:"
        )

    else:
        summary_message = "Create a summary of the conversation above:"

    # Add prompt to our history
    messages = state["messages"] + [HumanMessage(content=summary_message)]
    response = model.invoke(messages)

    # Delete all but the 2 most recent messages
    delete_messages = [RemoveMessage(id=m.id) for m in state["messages"][:-2]]
    return {"summary": response.content, "messages": delete_messages}
```

js

Prompting and orchestration logic can be used to summarize the message history. For example, in LangGraph you can include a `summary` key in the state alongside the `messages` key:

```
import { StateSchema, MessagesValue, GraphNode } from "@langchain/langgraph";
import { z } from "zod/v4";

const State = new StateSchema({
```

```

    messages: MessagesValue,
    summary: z.string().optional(),
  });

```

Then, you can generate a summary of the chat history, using any existing summary as context for the next summary. This `summarizeConversation` node can be called after some number of messages have accumulated in the `messages` state key.

```

import { RemoveMessage, HumanMessage } from "@langchain/core/messages";

const summarizeConversation: GraphNode<typeof State> = async (state) => {
  // First, we get any existing summary
  const summary = state.summary || "";

  // Create our summarization prompt
  let summaryMessage: string;
  if (summary) {
    // A summary already exists
    summaryMessage =
      `This is a summary of the conversation to date: ${summary}\n\n` +
      "Extend the summary by taking into account the new messages above:";
  } else {
    summaryMessage = "Create a summary of the conversation above:";
  }

  // Add prompt to our history
  const messages = [
    ...state.messages,
    new HumanMessage({ content: summaryMessage })
  ];
  const response = await model.invoke(messages);

  // Delete all but the 2 most recent messages
  const deleteMessages = state.messages
    .slice(0, -2)
    .map(m => new RemoveMessage({ id: m.id }));

  return {
    summary: response.content,
    messages: deleteMessages
  };
};

```

Full example: summarize messages

python


```

from typing import Any, TypedDict

from langchain.chat_models import init_chat_model
from langchain.messages import AnyMessage
from langchain_core.messages.utils import count_tokens_approximately
from langgraph.graph import StateGraph, START, MessagesState
from langgraph.checkpoint.memory import InMemorySaver
from langmem.short_term import SummarizationNode, RunningSummary # [!code highlight]

model = init_chat_model("claude-sonnet-4-6")
summarization_model = model.bind(max_tokens=128)

class State(MessagesState):
    context: dict[str, RunningSummary] # [!code highlight]

class LLMInputState(TypedDict): # [!code highlight]
    summarized_messages: list[AnyMessage]
    context: dict[str, RunningSummary]

summarization_node = SummarizationNode( # [!code highlight]
    token_counter=count_tokens_approximately,
    model=summarization_model,
    max_tokens=256,
    max_tokens_before_summary=256,
    max_summary_tokens=128,
)

def call_model(state: LLMInputState): # [!code highlight]
    response = model.invoke(state["summarized_messages"])
    return {"messages": [response]}

checkpointer = InMemorySaver()
builder = StateGraph(State)
builder.add_node(call_model)
builder.add_node("summarize", summarization_node) # [!code highlight]
builder.add_edge(START, "summarize")
builder.add_edge("summarize", "call_model")
graph = builder.compile(checkpointer=checkpointer)

# Invoke the graph
config = {"configurable": {"thread_id": "1"}}
graph.invoke({"messages": "hi, my name is bob"}, config)
graph.invoke({"messages": "write a short poem about cats"}, config)
graph.invoke({"messages": "now do the same but for dogs"}, config)
final_response = graph.invoke({"messages": "what's my name?"}, config)

final_response["messages"][-1].pretty_print()
print("\nSummary:", final_response["context"]["running_summary"].summary)

```

1. We will keep track of our running summary in the `context` field

(expected by the `SummarizationNode`).

1. Define private state that will be used only for filtering

the inputs to `call_model` node.

1. We're passing a private input state here to isolate the messages returned by the summarization node

```
``` ===== Ai Message
=====
```

From our conversation, I can see that you introduced yourself as Bob. That's the name you shared with me when we began talking.

Summary: In this conversation, I was introduced to Bob, who then asked me to write a poem about cats. I composed a poem titled "The Mystery of Cats" that captured cats' graceful movements, independent nature, and their special relationship with humans. Bob then requested a similar poem about dogs, so I wrote "The Joy of Dogs," which highlighted dogs' loyalty, enthusiasm, and loving companionship. Both poems were written in a similar style but emphasized the distinct characteristics that make each pet special.

```
</div>

<div class="callout js" markdown="1">

<p class="callout-label">js</p>

```typescript
import { ChatAnthropic } from "@langchain/anthropic";
import {
  SystemMessage,
  HumanMessage,
  RemoveMessage,
} from "@langchain/core/messages";
import {
  StateGraph,
  StateSchema,
  MessagesValue,
  GraphNode,
  ConditionalEdgeRouter,
  START,
  END,
  MemorySaver,
} from "@langchain/langgraph";
import * as z from "zod";

const memory = new MemorySaver();
```

```

// We will add a `summary` attribute (in addition to `messages` key)
const GraphState = new StateSchema({
  messages: MessagesValue,
  summary: z.string().default(""),
});

// We will use this model for both the conversation and the summarization
const model = new ChatAnthropic({ model: "claude-haiku-4-5-20251001" });

// Define the logic to call the model
const callModel: GraphNode<typeof GraphState> = async (state) => {
  // If a summary exists, we add this in as a system message
  const { summary } = state;
  let { messages } = state;
  if (summary) {
    const systemMessage = new SystemMessage({
      id: crypto.randomUUID(),
      content: `Summary of conversation earlier: ${summary}`,
    });
    messages = [systemMessage, ...messages];
  }
  const response = await model.invoke(messages);
  // We return an object, because this will get added to the existing state
  return { messages: [response] };
};

// We now define the logic for determining whether to end or summarize the conversation
const shouldContinue: ConditionalEdgeRouter<{ InputSchema: typeof GraphState; Nodes: "summarize_conversation" }> = (state) => {
  const messages = state.messages;
  // If there are more than six messages, then we summarize the conversation
  if (messages.length > 6) {
    return "summarize_conversation";
  }
  // Otherwise we can just end
  return END;
};

const summarizeConversation: GraphNode<typeof GraphState> = async (state) => {
  // First, we summarize the conversation
  const { summary, messages } = state;
  let summaryMessage: string;
  if (summary) {
    // If a summary already exists, we use a different system prompt
    // to summarize it than if one didn't
    summaryMessage =
      `This is summary of the conversation to date: ${summary}\n\n` +
      "Extend the summary by taking into account the new messages above:";
  } else {
    summaryMessage = "Create a summary of the conversation above:";
  }

```

```

}

const allMessages = [
  ...messages,
  new HumanMessage({ id: crypto.randomUUID(), content: summaryMessage }),
];

const response = await model.invoke(allMessages);

// We now need to delete messages that we no longer want to show up
// I will delete all but the last two messages, but you can change this
const deleteMessages = messages
  .slice(0, -2)
  .map((m) => new RemoveMessage({ id: m.id! }));

if (typeof response.content !== "string") {
  throw new Error("Expected a string response from the model");
}

return { summary: response.content, messages: deleteMessages };
};

// Define a new graph
const workflow = new StateGraph(GraphState)
  // Define the conversation node and the summarize node
  .addNode("conversation", callModel)
  .addNode("summarize_conversation", summarizeConversation)
  // Set the entrypoint as conversation
  .addEdge(START, "conversation")
  // We now add a conditional edge
  .addConditionalEdges(
    // First, we define the start node. We use `conversation`.
    // This means these are the edges taken after the `conversation` node is called.
    "conversation",
    // Next, we pass in the function that will determine which node is called next.
    shouldContinue,
  )
  // We now add a normal edge from `summarize_conversation` to END.
  // This means that after `summarize_conversation` is called, we end.
  .addEdge("summarize_conversation", END);

// Finally, we compile it!
const app = workflow.compile({ checkpointers: memory });

```

Manage checkpoints

You can view and delete the information stored by the checkpointers.

View thread state

python

Graph/Functional API

```
config = {
    "configurable": {
        "thread_id": "1", # [!code highlight]
        # optionally provide an ID for a specific checkpoint,
        # otherwise the latest checkpoint is shown
        # "checkpoint_id": "1f029ca3-1f5b-6704-8004-820c16b69a5a" # [!code highlight]
    }
}
graph.get_state(config) # [!code highlight]
```

```
StateSnapshot(
    values={'messages': [HumanMessage(content="hi! I'm bob"), AIMessage(content='Hi Bob! How
are you doing today?'), HumanMessage(content="what's my name?"), AIMessage(content='Your name
is Bob.')]}, next=(),
    config={'configurable': {'thread_id': '1', 'checkpoint_ns': '', 'checkpoint_id': '1f029c
a3-1f5b-6704-8004-820c16b69a5a'}},
    metadata={
        'source': 'loop',
        'writes': {'call_model': {'messages': AIMessage(content='Your name is Bob.')}},
        'step': 4,
        'parents': {},
        'thread_id': '1'
    },
    created_at='2025-05-05T16:01:24.680462+00:00',
    parent_config={'configurable': {'thread_id': '1', 'checkpoint_ns': '', 'checkpoint_id':
'1f029ca3-1790-6b0a-8003-baf965b6a38f'}},
    tasks=(),
    interrupts=()
)
```

Checkpoint API

```
config = {
    "configurable": {
        "thread_id": "1", # [!code highlight]
        # optionally provide an ID for a specific checkpoint,
        # otherwise the latest checkpoint is shown
        # "checkpoint_id": "1f029ca3-1f5b-6704-8004-820c16b69a5a" # [!code highlight]
    }
}
```

```
}
checkpointer.get_tuple(config) # [!code highlight]
```

```
CheckpointTuple(
    config={'configurable': {'thread_id': '1', 'checkpoint_ns': '', 'checkpoint_id': '1f029c
a3-1f5b-6704-8004-820c16b69a5a'}},
    checkpoint={
        'v': 3,
        'ts': '2025-05-05T16:01:24.680462+00:00',
        'id': '1f029ca3-1f5b-6704-8004-820c16b69a5a',
        'channel_versions': {'__start__': '00000000000000000000000000000005.0.52906785676018
59', 'messages': '00000000000000000000000000000006.0.3205149138784782', 'branch:to:call_mode
l': '00000000000000000000000000000006.0.14611156755133758'}, 'versions_seen': {'__input__':
{}}, '__start__': {'__start__': '00000000000000000000000000000004.0.5736472536395331'}}, 'call
_model': {'branch:to:call_model': '00000000000000000000000000000005.0.1410174088651449'}},
        'channel_values': {'messages': [HumanMessage(content="hi! I'm bob"), AIMessage(conte
nt='Hi Bob! How are you doing today?'), HumanMessage(content="what's my name?"), AIMessage(co
ntent='Your name is Bob.')]},
    },
    metadata={
        'source': 'loop',
        'writes': {'call_model': {'messages': AIMessage(content='Your name is Bob.')}},
        'step': 4,
        'parents': {},
        'thread_id': '1'
    },
    parent_config={'configurable': {'thread_id': '1', 'checkpoint_ns': '', 'checkpoint_id':
'1f029ca3-1790-6b0a-8003-baf965b6a38f'}},
    pending_writes=[]
)
```

js

```
const config = {
  configurable: {
    thread_id: "1",
    // optionally provide an ID for a specific checkpoint,
    // otherwise the latest checkpoint is shown
    // checkpoint_id: "1f029ca3-1f5b-6704-8004-820c16b69a5a"
  },
};
await graph.getState(config);
```

```
{
  values: { messages: [HumanMessage(...), AIMessage(...), HumanMessage(...), AIMessage(...)]
},
  next: [],
  config: { configurable: { thread_id: '1', checkpoint_ns: '', checkpoint_id: '1f029ca3-1f5b
```

```
-6704-8004-820c16b69a5a' } },
  metadata: {
    source: 'loop',
    writes: { call_model: { messages: AIMessage(...) } },
    step: 4,
    parents: {},
    thread_id: '1'
  },
  createdAt: '2025-05-05T16:01:24.680462+00:00',
  parentConfig: { configurable: { thread_id: '1', checkpoint_ns: '', checkpoint_id: '1f029ca3-1790-6b0a-8003-baf965b6a38f' } },
  tasks: [],
  interrupts: []
}
```

View the history of the thread

python

Graph/Functional API

```
```python
config = {
 "configurable": {
 "thread_id": "1" # [!code highlight]
 }
}
list(graph.get_state_history(config)) # [!code highlight]
```

```
[StateSnapshot(values={'messages': [HumanMessage(content="hi! I'm bob"),
AIMessage(content='Hi Bob! How are you doing today? Is there anything I can help you with?'),
HumanMessage(content="what's my name?"), AIMessage(content='Your name is Bob.')]}, next=
(), config={'configurable': {'thread_id': '1', 'checkpoint_ns': '', 'checkpoint_id': '1f029ca3-1f5b-
6704-8004-820c16b69a5a'}}, metadata={'source': 'loop', 'writes': {'call_model': {'messages':
AIMessage(content='Your name is Bob.')}}, 'step': 4, 'parents': {}, 'thread_id': '1'},
created_at='2025-05-05T16:01:24.680462+00:00', parent_config={'configurable': {'thread_id': '1',
'checkpoint_ns': '', 'checkpoint_id': '1f029ca3-1790-6b0a-8003-baf965b6a38f'}}, tasks=(),
interrupts=()), StateSnapshot(values={'messages': [HumanMessage(content="hi! I'm bob"),
AIMessage(content='Hi Bob! How are you doing today? Is there anything I can help you with?'),
HumanMessage(content="what's my name?")]}, next=('call_model',), config={'configurable':
{'thread_id': '1', 'checkpoint_ns': '', 'checkpoint_id': '1f029ca3-1790-6b0a-8003-baf965b6a38f'}},
metadata={'source': 'loop', 'writes': None, 'step': 3, 'parents': {}, 'thread_id': '1'},
created_at='2025-05-05T16:01:23.863421+00:00', parent_config={...} tasks=
```

```
(PregelTask(id='8ab4155e-6b15-b885-9ce5-bed69a2c305c', name='call_model', path=
('pregel_pull', 'call_model'), error=None, interrupts=(), state=None, result={'messages':
AIMessage(content='Your name is Bob.')}), interrupts=()), StateSnapshot(values=
{'messages': [HumanMessage(content="hi! I'm bob"), AIMessage(content='Hi Bob! How
are you doing today? Is there anything I can help you with?')]}), next=('__start',), config={...},
metadata={'source': 'input', 'writes': {'start': {'messages': [{'role': 'user', 'content': "what's my
name?"}]}}, 'step': 2, 'parents': {}, 'thread_id': '1'}, created_at='2025-05-
05T16:01:23.863173+00:00', parent_config={...} tasks=(PregelTask(id='24ba39d6-6db1-4c9b-f4c5-
682aeaf38dcd', name='start', path=('pregel_pull', '__start'), error=None, interrupts=(),
state=None, result={'messages': [{'role': 'user', 'content': "what's my name?"}]}), interrupts=()),
StateSnapshot(values={'messages': [HumanMessage(content="hi! I'm bob"),
AIMessage(content='Hi Bob! How are you doing today? Is there anything I can help you
with?')]}), next=(), config={...}, metadata={'source': 'loop', 'writes': {'call_model': {'messages':
AIMessage(content='Hi Bob! How are you doing today? Is there anything I can help you
with?')}}, 'step': 1, 'parents': {}, 'thread_id': '1'}, created_at='2025-05-05T16:01:23.862295+00:00',
parent_config={...} tasks=(), interrupts=()), StateSnapshot(values={'messages':
[HumanMessage(content="hi! I'm bob")]}), next=('call_model',), config={...}, metadata={'source':
'loop', 'writes': None, 'step': 0, 'parents': {}, 'thread_id': '1'}, created_at='2025-05-
05T16:01:22.278960+00:00', parent_config={...} tasks=(PregelTask(id='8cbd75e0-3720-b056-04f7-
71ac805140a0', name='call_model', path=('pregel_pull', 'call_model'), error=None,
interrupts=(), state=None, result={'messages': AIMessage(content='Hi Bob! How are you
doing today? Is there anything I can help you with?')}), interrupts=()), StateSnapshot(
values={'messages': []}, next=('__start',), config={'configurable': {'thread_id': '1',
'checkpoint_ns': '', 'checkpoint_id': '1f029ca3-0870-6ce2-bfff-1f3f14c3e565'}}, metadata=
{'source': 'input', 'writes': {'start': {'messages': [{'role': 'user', 'content': "hi! I'm bob"}]}}, 'step':
-1, 'parents': {}, 'thread_id': '1'}, created_at='2025-05-05T16:01:22.277497+00:00',
parent_config=None, tasks=(PregelTask(id='d458367b-8265-812c-18e2-33001d199ce6',
name='start', path=('pregel_pull', '__start'), error=None, interrupts=(), state=None, result=
{'messages': [{'role': 'user', 'content': "hi! I'm bob"}]}), interrupts=())]
```

**\*\*Checkpoint API\*\***

```
```python
config = {
    "configurable": {
        "thread_id": "1" # [!code highlight]
    }
}
list(checkpointer.list(config)) # [!code highlight]
```

```
[
    CheckpointTuple(
        config={'configurable': {'thread_id': '1', 'checkpoint_ns': '', 'checkpoint_id': '1f
029ca3-1f5b-6704-8004-820c16b69a5a'}},
```



```

        checkpoint={
            'v': 3,
            'ts': '2025-05-05T16:01:24.680462+00:00',
            'id': '1f029ca3-1f5b-6704-8004-820c16b69a5a',
            'channel_versions': {'__start__': '000000000000000000000000000005.0.5290678567
601859', 'messages': '00000000000000000000000000000006.0.3205149138784782', 'branch:to:call_
model': '00000000000000000000000000000006.0.14611156755133758'},
            'versions_seen': {'__input__': {}, '__start__': {'__start__': '0000000000000000
000000000000004.0.5736472536395331'}, 'call_model': {'branch:to:call_model': '00000000000000
00000000000000005.0.1410174088651449'}},
            'channel_values': {'messages': [HumanMessage(content="hi! I'm bob"), AIMessage(c
ontent='Hi Bob! How are you doing today? Is there anything I can help you with?'), HumanMess
age(content="what's my name?"), AIMessage(content='Your name is Bob.')]},
        },
        metadata={'source': 'loop', 'writes': {'call_model': {'messages': AIMessage(content
='Your name is Bob.')}}, 'step': 4, 'parents': {}, 'thread_id': '1'},
        parent_config={'configurable': {'thread_id': '1', 'checkpoint_ns': '', 'checkpoint_i
d': '1f029ca3-1790-6b0a-8003-baf965b6a38f'}},
        pending_writes=[],
    ),
    CheckpointTuple(
        config={'configurable': {'thread_id': '1', 'checkpoint_ns': '', 'checkpoint_id': '1f
029ca3-1790-6b0a-8003-baf965b6a38f'}},
        checkpoint={
            'v': 3,
            'ts': '2025-05-05T16:01:23.863421+00:00',
            'id': '1f029ca3-1790-6b0a-8003-baf965b6a38f',
            'channel_versions': {'__start__': '000000000000000000000000000005.0.5290678567
601859', 'messages': '00000000000000000000000000000006.0.3205149138784782', 'branch:to:call_
model': '00000000000000000000000000000006.0.14611156755133758'},
            'versions_seen': {'__input__': {}, '__start__': {'__start__': '0000000000000000
000000000000004.0.5736472536395331'}, 'call_model': {'branch:to:call_model': '00000000000000
00000000000000005.0.1410174088651449'}},
            'channel_values': {'messages': [HumanMessage(content="hi! I'm bob"), AIMessage(c
ontent='Hi Bob! How are you doing today? Is there anything I can help you with?'), HumanMess
age(content="what's my name?")], 'branch:to:call_model': None}
        },
        metadata={'source': 'loop', 'writes': None, 'step': 3, 'parents': {}, 'thread_id':
'1'},
        parent_config={...},
        pending_writes=[('8ab4155e-6b15-b885-9ce5-bed69a2c305c', 'messages', AIMessage(conte
nt='Your name is Bob.'))]
    ),
    CheckpointTuple(
        config={...},
        checkpoint={
            'v': 3,
            'ts': '2025-05-05T16:01:23.863173+00:00',
            'id': '1f029ca3-1790-616e-8002-9e021694a0cd',
            'channel_versions': {'__start__': '000000000000000000000000000004.0.5736472536
395331', 'messages': '000000000000000000000000000003.0.7056767754077798', 'branch:to:call_

```


js

python

js

```
const threadId = "1";  
await checkpointer.deleteThread(threadId);
```

Database management

If you are using any database-backed persistence implementation (such as Postgres, Redis, or Oracle) to store short and/or long-term memory, you will need to run migrations to set up the required schema before you can use it with your database.

By convention, most database-specific libraries define a `setup()` method on the checkpointer or store instance that runs the required migrations. However, you should check with your specific implementation of `@[BaseCheckpointSaver]` or `@[BaseStore]` to confirm the exact method name and usage.

We recommend running migrations as a dedicated deployment step, or you can ensure they're run as part of server startup.

Build a custom RAG agent with LangGraph

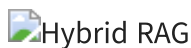
langgraph/agent-rag.mdx

Build a [retrieval](#) agent with LangGraph that decides when to search a vector store versus answering the user directly.

LangChain offers built-in [agent](#) implementations built on [LangGraph](#) primitives. When you need deeper customization, implement the agent directly in LangGraph. This tutorial walks through one retrieval-agent pattern.

In this tutorial you will:

1. Fetch and preprocess documents for retrieval.
2. Index those documents for semantic search and create a retriever tool for the agent.
3. Build an agentic RAG system that can decide when to use the retriever tool.



Concepts

This tutorial covers the following concepts:

- [Retrieval](#) using
- [document loaders](#),
- [text splitters](#), [embeddings](#), and
- [vector stores](#)
- The LangGraph [Graph API](#), including state, nodes, edges, and conditional edges.

Setup

Install the required packages and set your API keys:

python

```
pip install -U langgraph langchain langchain-openai langchain-text-splitters beautifulsoup4 requests
```

js

```
`` `bash npm npm install @langchain/langgraph @langchain/openai @langchain/textsplitters cheerio
```

```
`` `bash pnpm
```

```
pnpm install @langchain/langgraph @langchain/openai @langchain/textsplitters cheerio
```

```
```bash yarn add @langchain/langgraph @langchain/openai @langchain/textsplitters cheerio
```

```
```bash bun  
bun add @langchain/langgraph @langchain/openai @langchain/textsplitters cheerio
```

Set up LangSmith

RAG applications run retrieval and generation in sequence. When you run the examples in this tutorial, [LangSmith](#) logs a trace for each query so you can inspect retrieval, tool calls, and model responses. After you [sign up for LangSmith](#), set your environment variables to start logging traces:

```
export LANGSMITH_TRACING="true"  
export LANGSMITH_API_KEY="..."
```

python

Or, set them in Python:

```
import getpass  
import os  
  
os.environ["LANGSMITH_TRACING"] = "true"  
os.environ["LANGSMITH_API_KEY"] = getpass.getpass()
```

팁

If you are building a production agent, we also recommend you set up [LangSmith Engine](#) which monitors your traces, detects issues, and proposes fixes.

Preprocess documents

python

Fetch documents

Use three posts from [Lilian Weng's blog](#). Fetch page content with a minimal helper built on `requests` and `BeautifulSoup`.

Split documents

Split the fetched documents into smaller chunks for indexing into the vector store:

js

Fetch documents

Use three recent posts from [Lilian Weng's blog](#). Fetch page content with a minimal helper built on `fetch` and `cheerio`:

Split documents

Split the fetched documents into smaller chunks for indexing into the vector store:

Create a retriever tool

Index the split documents into a vector store for semantic search.

python

Index documents

Use an in-memory vector store and OpenAI embeddings:

Create the retriever tool

Create a retriever tool using the `@tool` decorator:

Test the tool

js

Index documents and create the tool

Use an in-memory vector store and OpenAI embeddings, then create a retriever tool with LangChain's prebuilt `createRetrieverTool`:

Test the tool

Generate a query or respond

With the retriever tool ready, start building the agent as a LangGraph graph. In the [Graph API](#), a graph is made of:

python

- **State**: Shared data that nodes read and update. This tutorial uses [MessagesState](#) , which stores a `messages` list of [chat messages](#).

js

- **State**: Shared data that nodes read and update. This tutorial uses [MessagesAnnotation](#) , which stores a `messages` list of [chat messages](#).

- **Nodes**: Functions that take the current state, run a step (for example, call a model or a tool), and return state updates.
- **Edges**: Connections that define which node runs next, including [conditional edges](#) that branch based on the state.

The first node is the agent decision point. Given the conversation so far, the model either answers the user directly or calls the retriever tool when the question needs blog context. That choice is what makes the system agentic rather than a fixed retrieve-then-generate pipeline: retrieval runs only when the model requests it.

python

Build the node

Build a `generate_query_or_respond` node that calls the model on the current messages and binds the `retriever_tool` with `.bind_tools`:

Try a simple greeting

Output:

```
```text wrap ===== Ai Message
=====
```

Hello! How can I help you today?

```
Ask a retrieval question
```

```
Ask a question that requires semantic search:
```

```
Output:
```

```
```text wrap
===== Ai Message =====
Tool Calls:
retrieve_blog_posts (call_tYQxgfIlNQUdMdtAhdbXNwIM)
Call ID: call_tYQxgfIlNQUdMdtAhdbXNwIM
```



```
Args:
  query: types of reward hacking
```

js

Build the node

Build a `generateQueryOrRespond` node that calls the model on the current messages and binds the tools with `.bindTools`:

Try a simple greeting

```
import { HumanMessage } from "@langchain/core/messages";

const input = { messages: [new HumanMessage("hello!")] };
const result = await generateQueryOrRespond(input);
console.log(result.messages[0]);
```

Output:

```
```text wrap AIMessage { content: "Hello! How can I help you today?", tool_calls: [] }
```

```
Ask a retrieval question
```

Ask a question that requires semantic search:

```
```typescript
const input = {
  messages: [
    new HumanMessage("What does Lilian Weng say about types of reward hacking?")
  ]
};
const result = await generateQueryOrRespond(input);
console.log(result.messages[0]);
```

Output:

```
```text wrap AIMessage { content: "", tool_calls: [ { name: "retrieve_blog_posts", args: { query: "types of reward hacking" }, id: "call_...", type: "tool_call" } ] }
```

```
</div>
```

```
Grade documents
```

A normal edge always sends the graph to the same next node. A [conditional edge](/oss/langgraph/graph-api#conditional-edges) chooses the next node at runtime by running a function over

the current state. After retrieval, use that pattern to grade whether the documents are relevant: continue to answer generation if they are, or rewrite the question and try again if they are not.

```
<div class="callout python" markdown="1">
```

```
<p class="callout-label">python</p>
```

**\*\*Add document grading\*\***

Add a `grade_documents` routing function that uses a model with a structured output schema `GradeDocuments`. It returns the name of the next node based on the grading decision (`generate_answer` or `rewrite_question`):

**\*\*Test with irrelevant documents\*\***

Run this with irrelevant documents in the tool response:

**\*\*Test with relevant documents\*\***

Confirm that relevant documents are classified as such:

```
</div>
```

```
<div class="callout js" markdown="1">
```

```
<p class="callout-label">js</p>
```

**\*\*Add document grading\*\***

Add a `gradeDocuments` node that uses a model with structured output (Zod), and falls back to a plain yes or no response if structured parsing fails. Route with a conditional edge according to the result (`generate` or `rewrite`):

**\*\*Test with irrelevant documents\*\***

Run this with irrelevant documents in the tool response:

```
```typescript
```

```
import { ToolMessage } from "@langchain/core/messages";
```

```
const input = {
  messages: [
    new HumanMessage("What does Lilian Weng say about types of reward hacking?"),
    new AIMessage({
      tool_calls: [
        {
          type: "tool_call",
          name: "retrieve_blog_posts",
          args: { query: "types of reward hacking" },
          id: "1",
```

```

    }
  ]
}),
new ToolMessage({
  content: "meow",
  tool_call_id: "1",
})
]
}
const result = await gradeDocuments(input);

```

Test with relevant documents

Confirm that relevant documents are classified as such:

```

const input = {
  messages: [
    new HumanMessage("What does Lilian Weng say about types of reward hacking?"),
    new AIMessage({
      tool_calls: [
        {
          type: "tool_call",
          name: "retrieve_blog_posts",
          args: { query: "types of reward hacking" },
          id: "1",
        }
      ]
    }),
    new ToolMessage({
      content: "reward hacking can be categorized into two types: environment or goal misspecification, and reward tampering",
      tool_call_id: "1",
    })
  ]
}
const result = await gradeDocuments(input);

```

Rewrite the question

If the grader marks the retrieved documents as irrelevant, the graph should not answer from that context. Instead, rewrite the original user question into a clearer search query, then send control back to the generate-query-or-respond node so the agent can retrieve again. This retry loop is how the agent recovers from a weak first retrieval instead of stopping or hallucinating an answer.

python

Build the rewrite node

Build the `rewrite_question` node to improve the original user question when retrieval misses:

Try it out

Output:

```text wrap What are the different types of reward hacking described by Lilian Weng, and how does she explain them?

```
</div>

<div class="callout js" markdown="1">

<p class="callout-label">js</p>

Build the rewrite node

Build the `rewrite` node to improve the original user question when retrieval misses:

Try it out

```typescript
import { HumanMessage, AIMessage, ToolMessage } from "@langchain/core/messages";

const input = {
  messages: [
    new HumanMessage("What does Lilian Weng say about types of reward hacking?"),
    new AIMessage({
      content: "",
      tool_calls: [
        {
          id: "1",
          name: "retrieve_blog_posts",
          args: { query: "types of reward hacking" },
          type: "tool_call"
        }
      ]
    }),
    new ToolMessage({ content: "meow", tool_call_id: "1" })
  ]
};

const response = await rewrite(input);
console.log(response.messages[0].content);
```

Output:

```text wrap What are the different types of reward hacking described by Lilian Weng, and how does she explain them?

</div>

## Generate an answer

When the grader accepts the retrieved documents, the graph moves to answer generation. This node is the classic RAG step: combine the original user question with the tool message that holds the retrieved context, then ask the model to produce a grounded reply. Keep the prompt tight so the model answers from the provided context instead of inventing details.

<div class="callout python" markdown="1">

<p class="callout-label">python</p>

**\*\*Build the answer node\*\***

Build the `generate\_answer` node to produce the final reply from the question and retrieved context:

**\*\*Try it\*\***

**\*\*Output:\*\***

```text wrap

===== Ai Message =====

Lilian Weng categorizes reward hacking into two types: environment or goal misspecification, and reward tampering. She considers reward hacking as a broad concept that includes both of these categories. Reward hacking occurs when an agent exploits flaws or ambiguities in the reward function to achieve high rewards without performing the intended behaviors.

js

Build the answer node

Build the `generate` node to produce the final reply from the question and retrieved context:

Try it

```
import { HumanMessage, AIMessage, ToolMessage } from "@langchain/core/messages";

const input = {
  messages: [
    new HumanMessage("What does Lilian Weng say about types of reward hacking?"),
    new AIMessage({
      content: "",
      tool_calls: [
        {
          id: "1",
```

```

        name: "retrieve_blog_posts",
        args: { query: "types of reward hacking" },
        type: "tool_call"
    }
]
}),
new ToolMessage({
    content: "reward hacking can be categorized into two types: environment or goal misspecification, and reward tampering",
    tool_call_id: "1"
})
]
};

const response = await generate(input);
console.log(response.messages[0].content);

```

Output:

``text wrap Lilian Weng categorizes reward hacking into two types: environment or goal misspecification, and reward tampering. She considers reward hacking as a broad concept that includes both of these categories. Reward hacking occurs when an agent exploits flaws or ambiguities in the reward function to achieve high rewards without performing the intended behaviors.

```

</div>

## Assemble the graph

Assemble the nodes and edges into a complete graph:

<div class="callout python" markdown="1">

<p class="callout-label">python</p>

- Start with `generate_query_or_respond` and determine whether to call `retriever_tool`.
- Route to the next step based on whether the model made tool calls:
  - If `generate_query_or_respond` returned `tool_calls`, call `retriever_tool` to retrieve context.
  - Otherwise, respond directly to the user.
- Grade retrieved document content for relevance to the question (`grade_documents`) and route to the next step:
  - If not relevant, rewrite the question using `rewrite_question` and then call `generate_query_or_respond` again.
  - If relevant, proceed to `generate_answer` and generate the final response using the @ToolMessage with the retrieved document context.

Visualize the graph:

```

```



</div>

<div class="callout js" markdown="1">

<p class="callout-label">js</p>

- Start with `generateQueryOrRespond` and determine whether to call the retriever tool.
- Route to the next step using a conditional edge:
  - If `generateQueryOrRespond` returned `tool_calls`, call the retriever tool to retrieve c
ontext.
  - Otherwise, respond directly to the user.
- Grade retrieved document content for relevance to the question (`gradeDocuments`) and rout
e to the next step:
  - If not relevant, rewrite the question using `rewrite` and then call `generateQueryOrResp
ond` again.
  - If relevant, proceed to `generate` and generate the final response using the @[ToolMessa
ge] with the retrieved document context.

</div>

## Run the agentik RAG

Test the complete graph by running it with a question:

<div class="callout python" markdown="1">

<p class="callout-label">python</p>

</div>

<div class="callout js" markdown="1">

<p class="callout-label">js</p>

</div>

## See also

- [Retrieval](/oss/langchain/retrieval)
- [Graph API](/oss/langgraph/graph-api)
- [Agents](/oss/langchain/agents)
- [Build a RAG agent](/oss/deepagents/rag)
- [Build a semantic search engine](/oss/langchain/knowledge-base)

```

```
<div class="page-break"></div>
```

Application structure

```
<p class="source">langgraph/application-structure.mdx</p>
```

A LangGraph application consists of one or more graphs, a configuration file (``langgraph.json``), a file that specifies dependencies, and an optional ``.env`` file that specifies environment variables.

This guide shows a typical structure of an application and shows you how to provide the required configuration to deploy an application with [LangSmith Deployment](/langsmith/deployment).

```
<div class="callout info" markdown="1">
```

```
<p class="callout-label">정보</p>
```

LangSmith Deployment is a managed hosting platform for deploying and scaling LangGraph agents. It handles the infrastructure, scaling, and operational concerns so you can deploy your stateful, long-running agents directly from your repository. Learn more in the [Deployment documentation](/langsmith/deployment).

```
</div>
```

Key concepts

To deploy using LangSmith, the following information should be provided:

1. A [LangGraph configuration file](#configuration-file-concepts) (``langgraph.json``) that specifies the dependencies, graphs, and environment variables to use for the application.
2. The [graphs](#graphs) that implement the logic of the application.
3. A file that specifies [dependencies](#dependencies) required to run the application.
4. [Environment variables](#environment-variables) that are required for the application to run.

File structure

Below are examples of directory structures for applications:

```
<div class="callout python" markdown="1">
```

```
<p class="callout-label">python</p>
```

```
**Python (requirements.txt)**
```

```
```plaintext
```

```
my-app/
```

```
├─ my_agent # all project code lies within here
```

```
| └─ utils # utilities for your graph
```



```

| | | └─ __init__.py
| | | └─ tools.py # tools for your graph
| | | └─ nodes.py # node functions for your graph
| | | └─ state.py # state definition of your graph
| | └─ __init__.py
| └─ agent.py # code for constructing your graph
└─ .env # environment variables
└─ requirements.txt # package dependencies
└─ langgraph.json # configuration file for LangGraph

```

## Python (pyproject.toml)

```

my-app/
└─ my_agent # all project code lies within here
| └─ utils # utilities for your graph
| | └─ __init__.py
| | └─ tools.py # tools for your graph
| | └─ nodes.py # node functions for your graph
| | └─ state.py # state definition of your graph
| └─ __init__.py
| └─ agent.py # code for constructing your graph
└─ .env # environment variables
└─ langgraph.json # configuration file for LangGraph
└─ pyproject.toml # dependencies for your project

```

## js

```

my-app/
└─ src # all project code lies within here
| └─ utils # optional utilities for your graph
| | └─ tools.ts # tools for your graph
| | └─ nodes.ts # node functions for your graph
| | └─ state.ts # state definition of your graph
| └─ agent.ts # code for constructing your graph
└─ package.json # package dependencies
└─ .env # environment variables
└─ langgraph.json # configuration file for LangGraph

```

## 참고

The directory structure of a LangGraph application can vary depending on the programming language and the package manager used.

# Configuration file

The `langgraph.json` file is a JSON file that specifies the dependencies, graphs, environment variables, and other settings required to deploy a LangGraph application.

See the [LangGraph configuration file reference](#) for details on all supported keys in the JSON file.

## 팁

The [LangGraph CLI](#) defaults to using the configuration file `langgraph.json` in the current directory.

## Examples

### python

- The dependencies involve a custom local package and the `langchain_openai` package.
- A single graph will be loaded from the file `./your_package/your_file.py` with the variable `variable`.
- The environment variables are loaded from the `.env` file.

```
{
 "dependencies": ["langchain_openai", "./your_package"],
 "graphs": {
 "my_agent": "./your_package/your_file.py:agent"
 },
 "env": "./.env"
}
```

### js

- The dependencies will be loaded from a dependency file in the local directory (e.g., `package.json`).
- A single graph will be loaded from the file `./your_package/your_file.js` with the function `agent`.
- The environment variable `OPENAI_API_KEY` is set inline.

```
{
 "dependencies": ["."],
 "graphs": {
 "my_agent": "./your_package/your_file.js:agent"
 },
 "env": {
 "OPENAI_API_KEY": "secret-key"
 }
}
```

```
}
}
```

## Dependencies

---

### python

A LangGraph application may depend on other Python packages.

### js

A LangGraph application may depend on other TypeScript/JavaScript libraries.

You will generally need to specify the following information for dependencies to be set up correctly:

### python

1. A file in the directory that specifies the dependencies (e.g. `requirements.txt`, `pyproject.toml`, or `package.json`).

### js

1. A file in the directory that specifies the dependencies (e.g. `package.json`).

1. A `dependencies` key in the [LangGraph configuration file](#) that specifies the dependencies required to run the LangGraph application.
2. Any additional binaries or system libraries can be specified using `dockerfile_lines` key in the [LangGraph configuration file](#).

## Graphs

---

Use the `graphs` key in the [LangGraph configuration file](#) to specify which graphs will be available in the deployed LangGraph application.

You can specify one or more graphs in the configuration file. Each graph is identified by a name (which should be unique) and a path for either: (1) the compiled graph or (2) a function that makes a graph is defined.

## Environment variables

---

If you're working with a deployed LangGraph application locally, you can configure environment variables in the `env` key of the [LangGraph configuration file](#).

For a production deployment, you will typically want to configure the environment variables in the deployment environment.

# Backward compatibility

---

langgraph/backward-compatibility.mdx

Software needs to change in production. New requirements, bug fixes, and refactors all eventually land in your graph code. Because LangGraph runs the latest deployed graph against state that has been [persisted](#) for existing threads, every change you ship is effectively a backward-compatible API change with respect to your existing checkpoints.

Unlike workflow engines that pin a run to the version of code it started with, LangGraph applies the latest graph immediately to *every* thread, both new threads and threads that resume from a checkpoint. This is convenient: bug fixes propagate to in-flight conversations and agents without ceremony. It also means you must reason about how each change interacts with runs that started under the previous version of the code.

There are three categories of compatibility issues to watch for, in roughly the order you will encounter them:

1. [Technical compatibility](#): The most common; the new code must still load and execute against existing State.
2. [Business compatibility](#): Less common; existing runs should keep following the old business logic even though the code has changed.
3. [Non-determinism](#): Only applies to the [Functional API](#).

## 팁

For a short summary of which graph topology and state changes the runtime supports by default, see [Graph migrations](#). The rest of this page covers the patterns you can apply when a change falls outside that supported set.

## Technical compatibility

---

Technical compatibility is the equivalent of an API breaking change in a microservice. The "API" here is the contract between your graph code and the data already persisted by the [checkpointer](#) for existing threads. When a thread resumes, LangGraph deserializes the saved state, dispatches it to a node by name, and expects the node to return values that fit the state schema.

Common technical breakages:

- **Renaming or removing a node** while threads are paused at or about to enter that node, for example at an `@[ interrupt ]` or via a checkpointed conditional edge that still routes to the old name. On resume, LangGraph cannot find the node by its saved name and the run fails. The starting point for resuming a run is the beginning of the node where execution stopped, so a missing node has nowhere to resume from.

- **Renaming or removing a State key** that older checkpoints still contain or that downstream nodes still read.
- **Tightening a State field**, such as making an `Optional` field required, narrowing a type, or adding a new required field with no default. Existing checkpoints will not satisfy the new schema.

Edge topology itself is *not* persisted in the checkpoint. Adding, removing, or rerouting edges between nodes that still exist is safe for in-flight threads. Per the [Graph migrations](#) summary, the only topology change that can break an interrupted thread is renaming or removing a node.

## Recommended patterns

### python

- Add new state fields as `NotRequired` (or `Optional[...] = None`) so old checkpoints still validate:

```
```python from typing import NotRequired from typing_extensions import TypedDict
```

```
class State(TypedDict): messages: list summary: NotRequired[str] # [!code ++]
```

- Treat removals as deprecations. Keep the field defined on the state for at least one drain cycle, even if no node reads it, so existing checkpoints continue to load.
- Rename through **add-then-remove**. Add the new field or node alongside the old one, dual-write or route to both for a deprecation window, then remove the old one once you have confirmed no in-flight thread depends on it.
- Keep node functions tolerant of unknown keys. ``TypedDict`` ignores extra keys at runtime, so leftover state from an older code version will not raise unless a node explicitly reads a missing key.
- Use `[time travel](/oss/langgraph/use-time-travel)` and `@[graph.get_state][get_state]` to spot-check existing threads against the new code in a staging deployment before rolling out.

</div>

<div class="callout js" markdown="1">

<p class="callout-label">js</p>

- Mark new state fields as optional (``z.string().optional()`` or ``.nullish()``) so old checkpoints still validate.
- Treat removals as deprecations: keep the field on the schema for at least one drain cycle so existing checkpoints continue to load.
- Rename via add-then-remove: add the new field or node alongside the old one, dual-write or route to both for a deprecation window, then remove the old one once no in-flight thread depends on it.
- Use `[time travel](/oss/langgraph/use-time-travel)` and `@[graph.getState][get_state]` to spot-check existing threads against the new code in a staging deployment before rolling out.

</div>

Detecting in-flight threads

Before you remove a node, rename a State key, or otherwise make a change that older threads cannot tolerate, you want to know whether any threads are currently parked on the version of the code you are about to drop. LangGraph itself does not maintain a search index over thread state, so the answer depends on where your graph runs.

****If you deploy to [LangSmith](/langsmith/deployment).**** Use the Agent Server's thread search to filter by status. The ``status`` field accepts ``idle``, ``busy``, ``interrupted``, and ``error``, so you can bulk-query for ``interrupted`` or ``busy`` threads, optionally narrowed with metadata filters. See [Filter by thread status](/langsmith/use-threads#filter-by-thread-status) and [List threads](/langsmith/use-threads#list-threads).

****Anywhere LangGraph runs.**** Use [LangSmith tracing](/oss/langgraph/observability) to monitor which nodes are being entered and exited in production. This is the most reliable signal that a node or state field is no longer reachable in any active code path.

****When you already have a ``thread_id``.** Inspect that single thread directly:

<div class="callout python" markdown="1">

<p class="callout-label">python</p>

- @[`graph.get\_state(config)`][get_state] returns the latest checkpoint, including which node the thread is paused at and any pending interrupts.
- @[`graph.get\_state\_history(config)`][get_state_history] returns the full chronological list of checkpoints for the thread.

</div>

<div class="callout js" markdown="1">

<p class="callout-label">js</p>

- @[`graph.getState(config)`][get_state] returns the latest checkpoint, including which node the thread is paused at and any pending interrupts.
- @[`graph.getStateHistory(config)`][get_state_history] returns the full chronological list of checkpoints for the thread.

</div>

When in doubt, keep the deprecated node or field in place until both the Agent Server thread list and tracing show no further activity on it.

Business compatibility

Sometimes a change is technically valid (every existing checkpoint still loads and every node still resolves), but the *meaning* of the new graph differs from the old one. The new behavior is correct for new threads, and you do not want to retroactively apply it to threads th

at started under the old logic.

For example, suppose your graph runs ``intake` → `trriage` → `respond``, and you decide to insert a new ``policy_check`` step between ``trriage`` and ``respond``:

- Threads that have already passed ``trriage`` should continue straight to ``respond`` (the old flow).
- New threads should run the full new flow.

The recommended pattern is to record the relevant **behavioral version** on the state at thread start, then branch on it with a [conditional edge](/oss/langgraph/graph-api#conditional-edges):

```
<div class="callout python" markdown="1">
```

```
<p class="callout-label">python</p>
```

```
```python
from typing import NotRequired
from typing_extensions import TypedDict

from langgraph.graph import END, START, StateGraph

class State(TypedDict):
 request: str
 flow_version: NotRequired[int]
 response: NotRequired[str]

def intake(state: State) -> dict:
 # Stamp new threads with the current flow version. Existing threads
 # that resume past `intake` keep whatever value was already saved.
 return {"flow_version": state.get("flow_version", 2)}

def triage(state: State) -> dict: ...
def policy_check(state: State) -> dict: ...
def respond(state: State) -> dict: ...

def after_triage(state: State) -> str:
 if state.get("flow_version", 1) >= 2:
 return "policy_check"
 return "respond"

builder = StateGraph(State)
builder.add_node("intake", intake)
builder.add_node("trriage", triage)
builder.add_node("policy_check", policy_check)
builder.add_node("respond", respond)
builder.add_edge(START, "intake")
builder.add_edge("intake", "trriage")
builder.add_conditional_edges("trriage", after_triage, ["policy_check", "respond"])
builder.add_edge("policy_check", "respond")
```



```
builder.add_edge("respond", END)

graph = builder.compile()
```

Old threads that resume after `triage` read `flow_version` from their saved state (or fall through to the v1 default) and skip `policy_check`. New threads start at `intake`, are stamped with `flow_version=2`, and run the new path. Once all v1 threads have completed, you can remove the version flag and the conditional edge.

This pattern only works if you set the version *at thread start*, before any branch that needs to be versioned. Setting it later means existing threads will not have it set when they need it.

## Non-determinism

This category only applies to the [Functional API](#) and to `tasks` or `@interrupt` calls inside a [Graph API node](#). Plain Graph API **nodes** [re-run from the start of the node function](#) on resume; design side effects to be idempotent, but you do not need to preserve task call order unless you use `tasks` or `@interrupt` in that **node**.

A Functional API **entrypoint** compiles to a single **node** that replays the entrypoint body from the beginning when a run resumes, using cached `@task` results to skip work that has already been done. Two kinds of changes break this model:

- **Adding, removing, or reordering `@task` calls or `@interrupt` calls** that come *before* the resume point. LangGraph matches cached results and resume values to calls by their position in the replay, so shifting that position can cause the wrong cached value to be replayed against a different call.
- **Introducing non-deterministic operations outside of a `@task`**, such as `time.time()`, `random.random()`, or a network call inlined in the entrypoint body. On replay these produce different values than they did on the first run, which can change the control flow.

For a deeper treatment with examples, see [Determinism](#) and [Common pitfalls](#) in the Functional API guide.

If you need to make non-trivial code changes to an `@entrypoint` that has in-flight runs, the safest options are:

- Let in-flight runs drain before deploying the change.
- Wrap any new logic in a new `@task` so its results are checkpointed independently.
- Register a new entrypoint under a new graph name in `langgraph.json` for the new behavior, and route new threads to it.

# Case studies

langgraph/case-studies.mdx

This list of companies using LangGraph and their success stories is compiled from public sources. If your company uses LangGraph, we'd love for you to share your story and add it to the list. You're also welcome to contribute updates based on publicly available information from other companies, such as blog posts or press releases.

Company	Industry	Use case	Reference
<a href="#">AirTop</a>	Software & Technology (GenAI Native)	Browser automation for AI agents	<a href="#">Case study, 2024</a>
<a href="#">AppFolio</a>	Real Estate	Copilot for domain-specific task	<a href="#">Case study, 2024</a>
<a href="#">Athena Intelligence</a>	Software & Technology (GenAI Native)	Research & summarization	<a href="#">Case study, 2024</a>
<a href="#">BlackRock</a>	Financial Services	Copilot for domain-specific task	<a href="#">Interrupt talk, 2025</a>
<a href="#">Captide</a>	Software & Technology (GenAI Native)	Data extraction	<a href="#">Case study, 2025</a>
<a href="#">Cisco CX</a>	Software & Technology	Customer support	<a href="#">Interrupt Talk, 2025</a>
<a href="#">Cisco Outshift</a>	Software & Technology	DevOps	<a href="#">Video story, 2025</a> ; <a href="#">Case study, 2025</a> ; <a href="#">Blog post, 2025</a>
<a href="#">Cisco TAC</a>	Software & Technology	Customer support	<a href="#">Video story, 2025</a>
<a href="#">City of Hope</a>	Non-profit	Copilot for domain-specific task	<a href="#">Video story, 2025</a>
<a href="#">C.H. Robinson</a>	Logistics	Automation	<a href="#">Case study, 2025</a>
<a href="#">Definely</a>	Legal	Copilot for domain-specific task	<a href="#">Case study, 2025</a>
<a href="#">Docent Pro</a>	Travel	GenAI embedded product experiences	<a href="#">Case study, 2025</a>
<a href="#">Elastic</a>	Software & Technology	Copilot for domain-specific task	<a href="#">Blog post, 2025</a>
<a href="#">Exa</a>	Software & Technology (GenAI Native)	Search	<a href="#">Case study, 2025</a>

Company	Industry	Use case	Reference
<a href="#">GitLab</a>	Software & Technology	Code generation	<a href="#">Duo workflow docs</a>
<a href="#">Harmonic</a>	Software & Technology	Search	<a href="#">Case study, 2025</a>
<a href="#">Inconvo</a>	Software & Technology	Code generation	<a href="#">Case study, 2025</a>
<a href="#">Infor</a>	Software & Technology	GenAI embedded product experiences; customer support; copilot	<a href="#">Case study, 2025</a>
<a href="#">J.P. Morgan</a>	Financial Services	Copilot for domain-specific task	<a href="#">Interrupt talk, 2025</a>
<a href="#">Klarna</a>	Fintech	Copilot for domain-specific task	<a href="#">Case study, 2025</a>
<a href="#">Komodo Health</a>	Healthcare	Copilot for domain-specific task	<a href="#">Blog post</a>
<a href="#">LinkedIn</a>	Social Media	Code generation; Search & discovery	<a href="#">Interrupt talk, 2025</a> ; <a href="#">Blog post, 2025</a> ; <a href="#">Blog post, 2024</a>
<a href="#">Minimal</a>	E-commerce	Customer support	<a href="#">Case study, 2025</a>
<a href="#">Modern Treasury</a>	Fintech	GenAI embedded product experiences	<a href="#">Video story, 2025</a>
<a href="#">Monday</a>	Software & Technology	GenAI embedded product experiences	<a href="#">Interrupt talk, 2025</a>
<a href="#">Morningstar</a>	Financial Services	Research & summarization	<a href="#">Video story, 2025</a>
<a href="#">OpenRecovery</a>	Healthcare	Copilot for domain-specific task	<a href="#">Case study, 2024</a>
<a href="#">Pigment</a>	Fintech	GenAI embedded product experiences	<a href="#">Video story, 2025</a>
<a href="#">Prosper</a>	Fintech	Customer support	<a href="#">Video story, 2025</a>
<a href="#">Qodo</a>	Software & Technology (GenAI Native)	Code generation	<a href="#">Blog post, 2025</a>
<a href="#">Rakuten</a>	E-commerce / Fintech	Copilot for domain-specific task	<a href="#">Video story, 2025</a> ; <a href="#">Blog post, 2025</a>
<a href="#">Replit</a>	Software & Technology	Code generation	<a href="#">Blog post, 2024</a> ; <a href="#">Breakout agent story, 2024</a> ; <a href="#">Fireside chat video, 2024</a>
<a href="#">Rexera</a>	Real Estate (GenAI Native)	Copilot for domain-specific task	<a href="#">Case study, 2024</a>
<a href="#">Abu Dhabi Government</a>	Government	Search	<a href="#">Case study, 2025</a>

Company	Industry	Use case	Reference
<a href="#">Tradestack</a>	Software & Technology (GenAI Native)	Copilot for domain-specific task	<a href="#">Case study, 2024</a>
<a href="#">Uber</a>	Transportation	Developer productivity; Code generation	<a href="#">Interrupt talk, 2025</a> ; <a href="#">Presentation, 2024</a> ; <a href="#">Video, 2024</a>
<a href="#">Unify</a>	Software & Technology (GenAI Native)	Copilot for domain-specific task	<a href="#">Interrupt talk, 2025</a> ; <a href="#">Blog post, 2024</a>
<a href="#">Vizient</a>	Healthcare	Copilot for domain-specific task	<a href="#">Video story, 2025</a> ; <a href="#">Case study, 2025</a>
<a href="#">Vodafone</a>	Telecommunications	Code generation; internal search	<a href="#">Case study, 2025</a>
<a href="#">WebToon</a>	Media & Entertainment	Data extraction	<a href="#">Case study, 2025</a>
<a href="#">11x</a>	Software & Technology (GenAI Native)	Research & outreach	<a href="#">Interrupt talk, 2025</a>

# Checkpointers

---

langgraph/checkpointers.mdx

A checkpointer saves a snapshot of graph state at each super-step, organized into **threads**. Compile a graph with a checkpointer to enable human-in-the-loop workflows, time travel debugging, fault-tolerant execution, and conversational memory.



## 정보

**Agent Server handles checkpointing automatically** When using the [Agent Server](#), you do not need to implement or configure checkpointers manually. The server handles all persistence infrastructure for you behind the scenes.

## 팁

Trace checkpointed state and debug how your agent resumes across sessions with [LangSmith](#). Follow the [tracing quickstart](#) to get set up.

## Why use checkpointers

---

Checkpointers are required for the following features:

- **Human-in-the-loop:** Checkpointers facilitate [human-in-the-loop workflows](#) by allowing humans to inspect, interrupt, and approve graph steps. Checkpointers are needed for these workflows as the person has to be able to view the state of a graph at any point in time, and the graph has to be able to resume execution after the person has made any updates to the state. See [Interrupts](#) for examples.
- **Memory:** Checkpointers allow for ["memory"](#) between interactions. In the case of repeated human interactions (like conversations) any follow up messages can be sent to that thread, which will retain its memory of previous ones. See [Add memory](#) for information on how to add and manage conversation memory using checkpointers.
- **Time travel:** Checkpointers allow for ["time travel"](#), allowing users to replay prior graph executions to review and / or debug specific graph steps. In addition, checkpointers make it possible to fork the graph state at arbitrary checkpoints to explore alternative trajectories.
- **Fault-tolerance:** Checkpointing provides fault-tolerance and error recovery: if one or more nodes fail at a given superstep, you can restart your graph from the last successful step.
- **Pending writes:** When a graph node fails mid-execution at a given [super-step](#), LangGraph stores pending checkpoint writes from any other nodes that completed successfully at that

super-step. When you resume graph execution from that super-step you don't re-run the successful nodes.

## Core concepts

---

### Threads

A thread is a unique ID or thread identifier assigned to each checkpoint saved by a checkpointer. It contains the accumulated state of a sequence of [runs](#). When a run is executed, the [state](#) of the underlying graph of the assistant will be persisted to the thread.

When invoking a graph with a checkpointer, you **must** specify a `thread_id` as part of the `configurable` portion of the config:

#### python

```
{"configurable": {"thread_id": "1"}}
```

#### js

```
{
 configurable: {
 thread_id: "1";
 }
}
```

A thread's current and historical state can be retrieved. To persist state, a thread must be created prior to executing a run. The LangSmith API provides several endpoints for creating and managing threads and thread state. See the [API reference](#) for more details.

The checkpointer uses `thread_id` as the primary key for storing and retrieving checkpoints. Without it, the checkpointer cannot save state or resume execution after an [interrupt](#), since the checkpointer uses `thread_id` to load the saved state.

### Checkpoints

The state of a thread at a particular point in time is called a checkpoint. A checkpoint is a snapshot of the graph state saved at each [super-step](#) and is represented by a `StateSnapshot` object (see [StateSnapshot fields](#) for the full field reference).

### Super-steps

LangGraph creates a checkpoint at each **super-step** boundary. A super-step is a single "tick" of the graph where all nodes scheduled for that step execute (potentially in parallel). For a sequential

graph like `START -> A -> B -> END`, there are separate super-steps for the input, node A, and node B — producing a checkpoint after each one. Understanding super-step boundaries is important for [time travel](#), because you can only resume execution from a checkpoint (i.e., a super-step boundary).

In addition to super-step checkpoints, LangGraph also persists writes at the **node (task) level**. As each node within a super-step finishes, its outputs are written to the checkpoint's `checkpoint_writes` table as task entries linked to the in-progress checkpoint. These per-task writes are what enable [pending writes](#) recovery: if another node in the same super-step fails, the successful nodes' writes are already durable and don't need to be re-run on resume. The full state snapshot is then committed once the super-step completes.

LangGraph also persists writes from individual node executions within a super-step. These writes are stored as tasks and used for fault tolerance: if another node in the same super-step fails, successful node writes do not need to be recomputed when you resume. These task writes are not full `StateSnapshot` checkpoints, so time travel resumes from full checkpoints at super-step boundaries.

Checkpoints are persisted and can be used to restore the state of a thread at a later time.

Let's see what checkpoints are saved when a simple graph is invoked as follows:

#### python

```
from langgraph.graph import StateGraph, START, END
from langgraph.checkpoint.memory import InMemorySaver
from langchain_core.runnables import RunnableConfig
from typing import Annotated
from typing_extensions import TypedDict
from operator import add

class State(TypedDict):
 foo: str
 bar: Annotated[list[str], add]

def node_a(state: State):
 return {"foo": "a", "bar": ["a"]}

def node_b(state: State):
 return {"foo": "b", "bar": ["b"]}

workflow = StateGraph(State)
workflow.add_node(node_a)
workflow.add_node(node_b)
workflow.add_edge(START, "node_a")
workflow.add_edge("node_a", "node_b")
workflow.add_edge("node_b", END)
```

```

checkpointer = InMemorySaver()
graph = workflow.compile(checkpointer=checkpointer)

config: RunnableConfig = {"configurable": {"thread_id": "1"}}
graph.invoke({"foo": "", "bar": []}, config)

```

## js

```

import { StateGraph, StateSchema, ReducedValue, START, END, MemorySaver } from "@langchain/langgraph";
import { z } from "zod/v4";

const State = new StateSchema({
 foo: z.string(),
 bar: new ReducedValue(
 z.array(z.string()).default(() => []),
 {
 inputSchema: z.array(z.string()),
 reducer: (x, y) => x.concat(y),
 }
),
});

const workflow = new StateGraph(State)
 .addNode("nodeA", (state) => {
 return { foo: "a", bar: ["a"] };
 })
 .addNode("nodeB", (state) => {
 return { foo: "b", bar: ["b"] };
 })
 .addEdge(START, "nodeA")
 .addEdge("nodeA", "nodeB")
 .addEdge("nodeB", END);

const checkpointer = new MemorySaver();
const graph = workflow.compile({ checkpointer });

const config = { configurable: { thread_id: "1" } };
await graph.invoke({ foo: "", bar: [] }, config);

```

## python

After you run the graph, there will be exactly 4 checkpoints:

- Empty checkpoint with `@[START]` as the next node to be executed
- Checkpoint with the user input `{'foo': '', 'bar': []}` and `node_a` as the next node to be executed



- Checkpoint with the outputs of `node_a` `{'foo': 'a', 'bar': ['a']}` and `node_b` as the next node to be executed
- Checkpoint with the outputs of `node_b` `{'foo': 'b', 'bar': ['a', 'b']}` and no next nodes to be executed

Note that the `bar` channel values contain outputs from both nodes because this example has a reducer for the `bar` channel.

js

After you run the graph, there will be exactly 4 checkpoints:

- Empty checkpoint with `@[START]` as the next node to be executed
- Checkpoint with the user input `{'foo': '', 'bar': []}` and `nodeA` as the next node to be executed
- Checkpoint with the outputs of `nodeA` `{'foo': 'a', 'bar': ['a']}` and `nodeB` as the next node to be executed
- Checkpoint with the outputs of `nodeB` `{'foo': 'b', 'bar': ['a', 'b']}` and no next nodes to be executed

Note that the `bar` channel values contain outputs from both nodes because this example has a reducer for the `bar` channel.

## Checkpoint namespace

Each checkpoint has a `checkpoint_ns` (checkpoint namespace) field that identifies which graph or subgraph it belongs to:

- `""` (empty string): The checkpoint belongs to the parent (root) graph.
- `"node_name:uuid"`: The checkpoint belongs to a subgraph invoked as the given node. For nested subgraphs, namespaces are joined with `|` separators (e.g., `"outer_node:uuid|inner_node:uuid"`).

You can access the checkpoint namespace from within a node via the config:

python

```
from langchain_core.runnables import RunnableConfig

def my_node(state: State, config: RunnableConfig):
 checkpoint_ns = config["configurable"]["checkpoint_ns"]
 # "" for the parent graph, "node_name:uuid" for a subgraph
```

js

```
import { RunnableConfig } from "@langchain/core/runnables";

function myNode(state: typeof State.Type, config: RunnableConfig) {
 const checkpointNs = config.configurable?.checkpoint_ns;
 // "" for the parent graph, "node_name:uuid" for a subgraph
}
```

See [Subgraphs](#) for more details on working with subgraph state and checkpoints.

## Get and update state

### Get state

#### python

When interacting with the saved graph state, you **must** specify a [thread identifier](#). You can view the *latest* state of the graph by calling `graph.get_state(config)`. This will return a `StateSnapshot` object that corresponds to the latest checkpoint associated with the thread ID provided in the config or a checkpoint associated with a checkpoint ID for the thread, if provided.

```
get the latest state snapshot
config = {"configurable": {"thread_id": "1"}}
graph.get_state(config)

get a state snapshot for a specific checkpoint_id
config = {"configurable": {"thread_id": "1", "checkpoint_id": "1ef663ba-28fe-6528-8002-5a559208592c"}}
graph.get_state(config)
```

#### js

When interacting with the saved graph state, you **must** specify a [thread identifier](#). You can view the *latest* state of the graph by calling `graph.getState(config)`. This will return a `StateSnapshot` object that corresponds to the latest checkpoint associated with the thread ID provided in the config or a checkpoint associated with a checkpoint ID for the thread, if provided.

```
// get the latest state snapshot
const config = { configurable: { thread_id: "1" } };
await graph.getState(config);

// get a state snapshot for a specific checkpoint_id
const config = {
 configurable: {
 thread_id: "1",
 checkpoint_id: "1ef663ba-28fe-6528-8002-5a559208592c",
 },
};
```

```

 },
 });
 await graph.getState(config);

```

## python

In this example, the output of `get_state` will look like this:

```

StateSnapshot(
 values={'foo': 'b', 'bar': ['a', 'b']},
 next=(),
 config={'configurable': {'thread_id': '1', 'checkpoint_ns': '', 'checkpoint_id': '1ef663ba-28fe-6528-8002-5a559208592c'}},
 metadata={'source': 'loop', 'writes': {'node_b': {'foo': 'b', 'bar': ['b']}}, 'step': 2},
 created_at='2024-08-29T19:19:38.821749+00:00',
 parent_config={'configurable': {'thread_id': '1', 'checkpoint_ns': '', 'checkpoint_id': '1ef663ba-28f9-6ec4-8001-31981c2c39f8'}},
 tasks=()
)

```

## js

In this example, the output of `getState` will look like this:

```

StateSnapshot {
 values: { foo: 'b', bar: ['a', 'b'] },
 next: [],
 config: {
 configurable: {
 thread_id: '1',
 checkpoint_ns: '',
 checkpoint_id: '1ef663ba-28fe-6528-8002-5a559208592c'
 }
 },
 metadata: {
 source: 'loop',
 writes: { nodeB: { foo: 'b', bar: ['b'] } },
 step: 2
 },
 createdAt: '2024-08-29T19:19:38.821749+00:00',
 parentConfig: {
 configurable: {
 thread_id: '1',
 checkpoint_ns: '',
 checkpoint_id: '1ef663ba-28f9-6ec4-8001-31981c2c39f8'
 }
 }
}

```

```

 },
 tasks: []
}

```

## StateSnapshot fields

### python

Field	Type	Description
values	dict	State channel values at this checkpoint.
next	tuple[str, ...]	Node names to execute next. Empty () means the graph is complete.
config	dict	Contains thread_id, checkpoint_ns, and checkpoint_id.
metadata	dict	Execution metadata. Contains source ("input", "loop", or "update"), writes (node outputs), and step (super-step counter).
created_at	str	ISO 8601 timestamp of when this checkpoint was created.
parent_config	dict \   None	Config of the previous checkpoint. None for the first checkpoint.
tasks	tuple[Prege1Task, ...]	Tasks to execute at this step. Each task has id, name, error, interrupts, and optionally state (subgraph snapshot, when using subgraphs=True).

### js

Field	Type	Description
values	object	State channel values at this checkpoint.
next	string[]	Node names to execute next. Empty [] means the graph is complete.
config	object	Contains thread_id, checkpoint_ns, and checkpoint_id.
metadata	object	Execution metadata. Contains source ("input", "loop", or "update"), writes (node outputs), and step (super-step counter).
createdAt	string	ISO 8601 timestamp of when this checkpoint was created.
parentConfig	object \   null	Config of the previous checkpoint. null for the first checkpoint.
tasks	PregelTask[]	Tasks to execute at this step. Each task has id, name, error, interrupts, and optionally state (subgraph snapshot, when using subgraphs: true).

## Get state history

### python

You can get the full history of the graph execution for a given thread by calling `@[graph.get_state_history(config)][get_state_history]`. This will return a list of `StateSnapshot` objects associated with the thread ID provided in the config. Importantly, the checkpoints will be ordered chronologically with the most recent checkpoint / `StateSnapshot` being the first in the list.

```
config = {"configurable": {"thread_id": "1"}}
list(graph.get_state_history(config))
```

### js

You can get the full history of the graph execution for a given thread by calling `graph.getStateHistory(config)`. This will return a list of `StateSnapshot` objects associated with the thread ID provided in the config. Importantly, the checkpoints will be ordered chronologically with the most recent checkpoint / `StateSnapshot` being the first in the list.

```
const config = { configurable: { thread_id: "1" } };
for await (const state of graph.getStateHistory(config)) {
 console.log(state);
}
```

### python

In this example, the output of `@[get_state_history]` will look like this:

```
[
 StateSnapshot(
 values={'foo': 'b', 'bar': ['a', 'b']},
 next=(),
 config={'configurable': {'thread_id': '1', 'checkpoint_ns': '', 'checkpoint_id': '1ef663ba-28fe-6528-8002-5a559208592c'}},
 metadata={'source': 'loop', 'writes': {'node_b': {'foo': 'b', 'bar': ['b']}}, 'step': 2},
 created_at='2024-08-29T19:19:38.821749+00:00',
 parent_config={'configurable': {'thread_id': '1', 'checkpoint_ns': '', 'checkpoint_id': '1ef663ba-28f9-6ec4-8001-31981c2c39f8'}},
 tasks=(),
),
 StateSnapshot(
 values={'foo': 'a', 'bar': ['a']},
 next=('node_b',),
```

```

 config={'configurable': {'thread_id': '1', 'checkpoint_ns': '', 'checkpoint_id': '1ef663ba-28f9-6ec4-8001-31981c2c39f8'}},
 metadata={'source': 'loop', 'writes': {'node_a': {'foo': 'a', 'bar': ['a']}}, 'step': 1},
 created_at='2024-08-29T19:19:38.819946+00:00',
 parent_config={'configurable': {'thread_id': '1', 'checkpoint_ns': '', 'checkpoint_id': '1ef663ba-28f4-6b4a-8000-ca575a13d36a'}},
 tasks=(PregelTask(id='6fb7314f-f114-5413-a1f3-d37dfe98ff44', name='node_b', error=None, interrupts=()),),
),
 StateSnapshot(
 values={'foo': '', 'bar': []},
 next=('node_a',),
 config={'configurable': {'thread_id': '1', 'checkpoint_ns': '', 'checkpoint_id': '1ef663ba-28f4-6b4a-8000-ca575a13d36a'}},
 metadata={'source': 'loop', 'writes': None, 'step': 0},
 created_at='2024-08-29T19:19:38.817813+00:00',
 parent_config={'configurable': {'thread_id': '1', 'checkpoint_ns': '', 'checkpoint_id': '1ef663ba-28f0-6c66-bfff-6723431e8481'}},
 tasks=(PregelTask(id='f1b14528-5ee5-579c-949b-23ef9bfbed58', name='node_a', error=None, interrupts=()),),
),
 StateSnapshot(
 values={'bar': []},
 next=('__start__',),
 config={'configurable': {'thread_id': '1', 'checkpoint_ns': '', 'checkpoint_id': '1ef663ba-28f0-6c66-bfff-6723431e8481'}},
 metadata={'source': 'input', 'writes': {'foo': ''}, 'step': -1},
 created_at='2024-08-29T19:19:38.816205+00:00',
 parent_config=None,
 tasks=(PregelTask(id='6d27aa2e-d72b-5504-a36f-8620e54a76dd', name='__start__', error=None, interrupts=()),),
)
]

```

js

In this example, the output of `getStateHistory` will look like this:

```

[
 StateSnapshot {
 values: { foo: 'b', bar: ['a', 'b'] },
 next: [],
 config: {
 configurable: {
 thread_id: '1',
 checkpoint_ns: '',
 checkpoint_id: '1ef663ba-28fe-6528-8002-5a559208592c'
 }
 }
 }
]

```

```

 },
 metadata: {
 source: 'loop',
 writes: { nodeB: { foo: 'b', bar: ['b'] } },
 step: 2
 },
 createdAt: '2024-08-29T19:19:38.821749+00:00',
 parentConfig: {
 configurable: {
 thread_id: '1',
 checkpoint_ns: '',
 checkpoint_id: '1ef663ba-28f9-6ec4-8001-31981c2c39f8'
 }
 },
 tasks: []
 },
 StateSnapshot {
 values: { foo: 'a', bar: ['a'] },
 next: ['nodeB'],
 config: {
 configurable: {
 thread_id: '1',
 checkpoint_ns: '',
 checkpoint_id: '1ef663ba-28f9-6ec4-8001-31981c2c39f8'
 }
 },
 metadata: {
 source: 'loop',
 writes: { nodeA: { foo: 'a', bar: ['a'] } },
 step: 1
 },
 createdAt: '2024-08-29T19:19:38.819946+00:00',
 parentConfig: {
 configurable: {
 thread_id: '1',
 checkpoint_ns: '',
 checkpoint_id: '1ef663ba-28f4-6b4a-8000-ca575a13d36a'
 }
 },
 tasks: [
 PregelTask {
 id: '6fb7314f-f114-5413-a1f3-d37dfe98ff44',
 name: 'nodeB',
 error: null,
 interrupts: []
 }
]
 },
 StateSnapshot {
 values: { foo: '', bar: [] },
 next: ['node_a'],

```

```

config: {
 configurable: {
 thread_id: '1',
 checkpoint_ns: '',
 checkpoint_id: '1ef663ba-28f4-6b4a-8000-ca575a13d36a'
 }
},
metadata: {
 source: 'loop',
 writes: null,
 step: 0
},
createdAt: '2024-08-29T19:19:38.817813+00:00',
parentConfig: {
 configurable: {
 thread_id: '1',
 checkpoint_ns: '',
 checkpoint_id: '1ef663ba-28f0-6c66-bfff-6723431e8481'
 }
},
tasks: [
 PregelTask {
 id: 'f1b14528-5ee5-579c-949b-23ef9bfbbed58',
 name: 'node_a',
 error: null,
 interrupts: []
 }
],
StateSnapshot {
 values: { bar: [] },
 next: ['__start__'],
 config: {
 configurable: {
 thread_id: '1',
 checkpoint_ns: '',
 checkpoint_id: '1ef663ba-28f0-6c66-bfff-6723431e8481'
 }
 },
 metadata: {
 source: 'input',
 writes: { foo: '' },
 step: -1
 },
 createdAt: '2024-08-29T19:19:38.816205+00:00',
 parentConfig: null,
 tasks: [
 PregelTask {
 id: '6d27aa2e-d72b-5504-a36f-8620e54a76dd',
 name: '__start__',
 error: null,

```



```

 interrupts: []
 }
]
}
]

```



State

## Find a specific checkpoint

You can filter the state history to find checkpoints matching specific criteria:

### python

```

history = list(graph.get_state_history(config))

Find the checkpoint before a specific node executed
before_node_b = next(s for s in history if s.next == ("node_b",))

Find a checkpoint by step number
step_2 = next(s for s in history if s.metadata["step"] == 2)

Find checkpoints created by update_state
forks = [s for s in history if s.metadata["source"] == "update"]

Find the checkpoint where an interrupt occurred
interrupted = next(
 s for s in history
 if s.tasks and any(t.interrupts for t in s.tasks)
)

```

### js

```

const history: StateSnapshot[] = [];
for await (const state of graph.getStateHistory(config)) {
 history.push(state);
}

// Find the checkpoint before a specific node executed
const beforeNodeB = history.find((s) => s.next.includes("nodeB"));

// Find a checkpoint by step number
const step2 = history.find((s) => s.metadata.step === 2);

// Find checkpoints created by updateState
const forks = history.filter((s) => s.metadata.source === "update");

```

```
// Find the checkpoint where an interrupt occurred
const interrupted = history.find(
 (s) => s.tasks.length > 0 && s.tasks.some((t) => t.interrupts.length > 0)
);
```

## Replay

Replay re-executes steps from a prior checkpoint. Invoke the graph with a prior `checkpoint_id` to re-run nodes after that checkpoint. Nodes before the checkpoint are skipped (their results are already saved). Nodes after the checkpoint re-execute, including any LLM calls, API requests, or [interrupts](#) — which are always re-triggered during replay.

See [Time travel](#) for full details and code examples on replaying past executions.



Replay

## Update state

### python

You can edit the graph state using `@[update_state]`. This creates a new checkpoint with the updated values — it does not modify the original checkpoint. The update is treated the same as a node update: values are passed through [reducer](#) functions when defined, so channels with reducers *accumulate* values rather than overwrite them.

You can optionally specify `as_node` to control which node the update is treated as coming from, which affects which node executes next. See [Time travel: as\\_node](#) for details.

### js

You can edit the graph state using `graph.updateState()`. This creates a new checkpoint with the updated values — it does not modify the original checkpoint. The update is treated the same as a node update: values are passed through [reducer](#) functions when defined, so channels with reducers *accumulate* values rather than overwrite them.

You can optionally specify `asNode` to control which node the update is treated as coming from, which affects which node executes next. See [Time travel: asNode](#) for details.



Update

## Durability modes

LangGraph supports three durability modes that let you balance performance and data consistency. You can specify the durability mode when calling any graph execution method:

## python

```
graph.stream(
 {"input": "test"},
 durability="sync"
)
```

## js

```
await graph.stream(
 { input: "test" },
 { durability: "sync" }
)
```

The durability modes, from least to most durable, are as follows:

- `"exit"`: LangGraph persists changes only when graph execution exits — successfully, with an error, or due to a human-in-the-loop interrupt. This provides the best performance for long-running graphs but means intermediate state is not saved, so you cannot recover from system failures (like process crashes) mid-execution.
- `"async"`: LangGraph persists changes asynchronously while the next step executes. This provides good performance and durability, but there is a small risk that LangGraph does not write checkpoints if the process crashes during execution.
- `"sync"`: LangGraph persists changes synchronously before the next step starts. This ensures that LangGraph writes every checkpoint before continuing execution, providing high durability at the cost of some performance overhead.

## Optimize checkpoint storage

### python

By default, LangGraph checkpoints write the full value of every state channel at each super-step. For long-running threads with large accumulations—such as multi-turn conversations—this can produce significant storage growth over time.

`@[DeltaChannel]` stores only incremental deltas instead of the full accumulated value, substantially reducing checkpoint size for append-heavy channels. See [DeltaChannel](#) for usage and the storage-vs-latency tradeoff.

### 주의

DeltaChannel requires langgraph>=1.2 and is currently in beta. The API may change in future releases.

## Checkpointing libraries

Under the hood, checkpointing is powered by checkpointer objects that conform to `@[ BaseCheckpointSaver ]` interface. LangGraph provides several checkpointer implementations, all implemented via standalone, installable libraries.

### python

#### 참고

See [checkpointer integrations](#) for available providers.

- `langgraph-checkpoint` : The base interface for checkpointer savers (`@[ BaseCheckpointSaver ]`) and serialization/deserialization interface (`@[ SerializerProtocol ]`). Includes in-memory checkpointer implementation (`@[ InMemorySaver ]`) for experimentation. LangGraph comes with `langgraph-checkpoint` included.
- `langgraph-checkpoint-sqlite` : An implementation of LangGraph checkpointer that uses SQLite database (`@[ SqliteSaver ]` / `@[ AsyncSqliteSaver ]`). Ideal for experimentation and local workflows. Needs to be installed separately.
- `langgraph-checkpoint-postgres` : An advanced checkpointer that uses Postgres database (`@[ PostgresSaver ]` / `@[ AsyncPostgresSaver ]`), used in LangSmith. Ideal for using in production. Needs to be installed separately.
- `langgraph-checkpoint-mongodb` : An advanced checkpointer that uses MongoDB (`MongoDBSaver` / `AsyncMongoDBSaver`). Ideal for using in production. Needs to be installed separately.
- `langchain-azure-cosmosdb` : An implementation of LangGraph checkpointer that uses Azure Cosmos DB for NoSQL (`@[ CosmosDBSaverSync ]` / `@[ CosmosDBSaver ]`). Ideal for using in production with Azure. Supports both sync and async operations, with Microsoft Entra ID authentication. Needs to be installed separately.

### js

- `@langchain/langgraph-checkpoint` : The base interface for checkpointer savers (`@[ BaseCheckpointSaver ]`) and serialization/deserialization interface (`@[ SerializerProtocol ]`). Includes in-memory checkpointer implementation (`@[ MemorySaver ]`) for experimentation. LangGraph comes with `@langchain/langgraph-checkpoint` included.

- `@langchain/langgraph-checkpoint-sqlite` : An implementation of LangGraph checkpointer that uses SQLite database (`@[SqliteSaver]`). Ideal for experimentation and local workflows. Needs to be installed separately.
- `@langchain/langgraph-checkpoint-postgres` : An advanced checkpointer that uses Postgres database (`@[PostgresSaver]`), used in LangSmith. Ideal for using in production. Needs to be installed separately.
- `@langchain/langgraph-checkpoint-mongodb` : An advanced checkpointer (`MongoDBSaver`) and long-term memory store (`MongoDBStore`) backed by MongoDB. The store supports cross-thread persistence with optional integrated vector search. Ideal for production use. Needs to be installed separately.
- `@langchain/langgraph-checkpoint-redis` : An advanced checkpointer that uses Redis database (`RedisSaver`). Ideal for using in production. Needs to be installed separately.

## Checkpoint interface

### python

Each checkpointer conforms to `@[BaseCheckpointSaver]` interface and implements the following methods:

- `.put` - Store a checkpoint with its configuration and metadata.
- `.put_writes` - Store intermediate writes linked to a checkpoint (i.e. [pending writes](#)).
- `.get_tuple` - Fetch a checkpoint tuple using for a given configuration (`thread_id` and `checkpoint_id`). This is used to populate `StateSnapshot` in `graph.get_state()`.
- `.list` - List checkpoints that match a given configuration and filter criteria. This is used to populate state history in `graph.get_state_history()`

If the checkpointer is used with asynchronous graph execution (i.e. executing the graph via `.ainvoke`, `.astream`, `.abatch`), asynchronous versions of the above methods will be used (`.aput`, `.aput_writes`, `.aget_tuple`, `.alist`).

### 참고

For running your graph asynchronously, you can use `@[InMemorySaver]`, or async versions of Sqlite/Postgres checkpointers -- `@[AsyncSqliteSaver]` / `@[AsyncPostgresSaver]` checkpointers.

### js

Each checkpointer conforms to the `@[BaseCheckpointSaver]` interface and implements the following methods:

- `.put` - Store a checkpoint with its configuration and metadata.

- `.putWrites` - Store intermediate writes linked to a checkpoint (i.e. [pending writes](#)).
- `.getTuple` - Fetch a checkpoint tuple using for a given configuration ( `thread_id` and `checkpoint_id` ). This is used to populate `StateSnapshot` in `graph.getState()` .
- `.list` - List checkpoints that match a given configuration and filter criteria. This is used to populate state history in `graph.getStateHistory()`

## python

### Serializer

When checkpointers save the graph state, they need to serialize the channel values in the state. This is done using serializer objects.

`langgraph_checkpoint` defines `@[protocol][SerializerProtocol]` for implementing serializers provides a default implementation (`@[JsonPlusSerializer]`) that handles a wide variety of types, including `LangChain` and `LangGraph` primitives, datetimes, enums and more.

#### Serialization with `pickle`

The default serializer, `@[JsonPlusSerializer]`, uses `ormsgpack` and `JSON` under the hood, which is not suitable for all types of objects.

If you want to fallback to `pickle` for objects not currently supported by the `msgpack` encoder (such as `Pandas` dataframes), you can use the `pickle_fallback` argument of the `@[JsonPlusSerializer]`:

```
from langgraph.checkpoint.memory import InMemorySaver
from langgraph.checkpoint.serde.jsonplus import JsonPlusSerializer

... Define the graph ...
graph.compile(
 checkpointer=InMemorySaver(serde=JsonPlusSerializer(pickle_fallback=True))
)
```

### Encryption

Checkpointers can optionally encrypt all persisted state. To enable this, pass an instance of `@[EncryptedSerializer]` to the `serde` argument of any `@[BaseCheckpointSaver]` implementation. The easiest way to create an encrypted serializer is via `@[from_pycryptodome_aes]`, which reads the AES key from the `LANGGRAPH_AES_KEY` environment variable (or accepts a `key` argument):

```
import sqlite3

from langgraph.checkpoint.serde.encrypted import EncryptedSerializer
from langgraph.checkpoint.sqlite import SqliteSaver
```

```
serde = EncryptedSerializer.from_pycryptodome_aes() # reads LANGGRAPH_AES_KEY
checkpointer = SqliteSaver(sqlite3.connect("checkpoint.db"), serde=serde)
```

```
from langgraph.checkpoint.serde.encrypted import EncryptedSerializer
from langgraph.checkpoint.postgres import PostgresSaver

serde = EncryptedSerializer.from_pycryptodome_aes()
checkpointer = PostgresSaver.from_conn_string("postgresql://...", serde=serde)
checkpointer.setup()
```

When running on LangSmith, encryption is automatically enabled whenever `LANGGRAPH_AES_KEY` is present, so you only need to provide the environment variable. Other encryption schemes can be used by implementing `@[CipherProtocol]` and supplying it to `@[EncryptedSerializer]`.

## Build a custom checkpointer

### python

#### 팁

Validate your implementation as you build using the [conformance test suite](#). It covers all five base methods and extended capabilities including delta channels. Run it in CI before shipping.

This section covers implementing `BaseCheckpointSaver` from scratch for a custom storage backend. If you already have a working checkpointer and only need to add delta channel support, jump to [Delta channel support](#).

### Overview

LangGraph's persistence layer is built on two storage abstractions:

- **Checkpoints table** — one row per superstep; stores the serialized graph state (`channel_values`, `channel_versions`, `versions_seen`) and links to its parent checkpoint.
- **Writes table** — one row per node output within a superstep; stores (`task_id`, `channel`, `value`) tuples linked to a checkpoint.

Your checkpointer manages both tables. `put` writes a checkpoint row; `put_writes` writes node-output rows; `get_tuple` reads both back into a `CheckpointTuple`.

## Base contract

Subclass `BaseCheckpointSaver` and implement these five methods. All are required — a missing base method raises `NotImplementedError` at runtime.

```
from collections.abc import AsyncIterator, Iterator, Sequence
from typing import Any
from langchain_core.runnables import RunnableConfig
from langgraph.checkpoint.base import (
 BaseCheckpointSaver,
 ChannelVersions,
 Checkpoint,
 CheckpointMetadata,
 CheckpointTuple,
)

class MyCheckpointier(BaseCheckpointSaver):
 async def aput(
 self,
 config: RunnableConfig,
 checkpoint: Checkpoint,
 metadata: CheckpointMetadata,
 new_versions: ChannelVersions,
) -> RunnableConfig:
 ...

 async def aput_writes(
 self,
 config: RunnableConfig,
 writes: Sequence[tuple[str, Any]],
 task_id: str,
 task_path: str = "",
) -> None:
 ...

 async def aget_tuple(self, config: RunnableConfig) -> CheckpointTuple | None:
 ...

 async def alist(
 self,
 config: RunnableConfig | None,
 *,
 filter: dict[str, Any] | None = None,
 before: RunnableConfig | None = None,
 limit: int | None = None,
) -> AsyncIterator[CheckpointTuple]:
 ...
 yield # make this an async generator
```



```

async def adelete_thread(self, thread_id: str) -> None:
 ...

```

## put / aput

Store one checkpoint row. Return an updated config with the stored `checkpoint_id`.

Key requirements:

- Serialize the checkpoint using `self.serde.dumps_typed(checkpoint)` — this handles all LangGraph-native types including `_DeltaSnapshot` blobs used by delta channels.
- Store `metadata` in full — do not strip unknown keys. LangGraph adds new metadata fields (such as `counters_since_delta_snapshot` for delta channels) in minor releases; discarding them silently breaks features.
- Store `config["configurable"].get("checkpoint_id")` as the parent checkpoint ID so `get_tuple` can populate `parent_config`.

```

async def aput(self, config, checkpoint, metadata, new_versions):
 thread_id = config["configurable"]["thread_id"]
 checkpoint_ns = config["configurable"]["checkpoint_ns"]
 checkpoint_id = checkpoint["id"]
 parent_id = config["configurable"].get("checkpoint_id")

 type_, blob = self.serde.dumps_typed(checkpoint)
 serialized_metadata = self.serde.dumps_typed(metadata)

 await self.db.execute(
 "INSERT INTO checkpoints (...) VALUES (...)",
 thread_id, checkpoint_ns, checkpoint_id, parent_id,
 type_, blob, *serialized_metadata,
)
 return {
 "configurable": {
 "thread_id": thread_id,
 "checkpoint_ns": checkpoint_ns,
 "checkpoint_id": checkpoint_id,
 }
 }

```

## put\_writes / aput\_writes

Store node-output rows for a single task within the current superstep. These rows are linked to the checkpoint by `(thread_id, checkpoint_ns, checkpoint_id)`.

```

async def aput_writes(self, config, writes, task_id, task_path=""):
 thread_id = config["configurable"]["thread_id"]
 checkpoint_ns = config["configurable"]["checkpoint_ns"]

```

```

 checkpoint_id = config["configurable"]["checkpoint_id"]

 rows = []
 for idx, (channel, value) in enumerate(writes):
 type_, blob = self.serde.dumps_typed(value)
 final_idx = WRITES_IDX_MAP.get(channel, idx)
 rows.append((thread_id, checkpoint_ns, checkpoint_id,
 task_id, task_path, final_idx, channel, type_, blob))

 await self.db.executemany("INSERT INTO writes (...) VALUES (...)", rows)

```

Import `WRITES_IDX_MAP` from `langgraph.checkpoint.base`. It maps special channels (`__error__`, `__interrupt__`, etc.) to reserved negative indices so they do not collide with regular write indices.

## get\_tuple / aget\_tuple

Retrieve a checkpoint. The config may contain:

- **No `checkpoint_id`** — return the latest checkpoint for the thread + namespace.
- **A specific `checkpoint_id`** — return that exact checkpoint.

**Both paths must work correctly.** The specific-id path is used for time travel and — critically — for delta channel state reconstruction on every graph invocation (see [Delta channel support](#)). A broken specific-id lookup silently corrupts delta channel state.

```

async def aget_tuple(self, config):
 thread_id = config["configurable"]["thread_id"]
 checkpoint_ns = config["configurable"].get("checkpoint_ns", "")
 checkpoint_id = config["configurable"].get("checkpoint_id")

 if checkpoint_id:
 row = await self.db.fetchone(
 "SELECT * FROM checkpoints "
 "WHERE thread_id=? AND checkpoint_ns=? AND checkpoint_id=?",
 thread_id, checkpoint_ns, checkpoint_id,
)
 else:
 row = await self.db.fetchone(
 "SELECT * FROM checkpoints "
 "WHERE thread_id=? AND checkpoint_ns=? "
 "ORDER BY checkpoint_id DESC LIMIT 1",
 thread_id, checkpoint_ns,
)

 if row is None:
 return None

 writes = await self.db.fetchall(

```

```

"SELECT task_id, channel, type, value FROM writes "
"WHERE thread_id=? AND checkpoint_ns=? AND checkpoint_id=? "
"ORDER BY task_id, idx",
thread_id, checkpoint_ns, row["checkpoint_id"],
)
pending_writes = [
 (w["task_id"], w["channel"], self.serde.loads_typed((w["type"], w["value"])))
 for w in writes
]

checkpoint = self.serde.loads_typed((row["type"], row["blob"]))
metadata = self.serde.loads_typed((row["metadata_type"], row["metadata"]))

parent_config = None
if row["parent_checkpoint_id"]:
 parent_config = {
 "configurable": {
 "thread_id": thread_id,
 "checkpoint_ns": checkpoint_ns,
 "checkpoint_id": row["parent_checkpoint_id"],
 }
 }

return CheckpointTuple(
 config={
 "configurable": {
 "thread_id": thread_id,
 "checkpoint_ns": checkpoint_ns,
 "checkpoint_id": row["checkpoint_id"],
 }
 },
 checkpoint=checkpoint,
 metadata=metadata,
 parent_config=parent_config,
 pending_writes=pending_writes,
)

```

## 주의

**Row key / index design matters for the specific-id lookup.** If your storage uses a time-ordered key (e.g., a reversed timestamp) that does not embed `checkpoint_id`, you cannot do a direct row read by id. You must either encode `checkpoint_id` in the row key, or build a secondary index. A scan with a value filter on every lookup works but does not scale.

## list / alist

Return checkpoints for a thread, newest first. Respect `before` (return only checkpoints older than that config's `checkpoint_id`) and `limit`.

## delete\_thread / adelete\_thread

Delete all checkpoints and writes for a thread. Both checkpoint rows and write rows must be deleted.

## Row key / index design

How you store and index checkpoints directly affects correctness and performance.

### Recommended schema (SQL):

```
CREATE TABLE checkpoints (
 thread_id TEXT NOT NULL,
 checkpoint_ns TEXT NOT NULL DEFAULT '',
 checkpoint_id TEXT NOT NULL, -- ULID, lexicographically sortable newest-last
 parent_checkpoint_id TEXT,
 type TEXT,
 checkpoint BYTEA,
 metadata JSONB,
 PRIMARY KEY (thread_id, checkpoint_ns, checkpoint_id)
);

CREATE TABLE writes (
 thread_id TEXT NOT NULL,
 checkpoint_ns TEXT NOT NULL DEFAULT '',
 checkpoint_id TEXT NOT NULL,
 task_id TEXT NOT NULL,
 task_path TEXT NOT NULL DEFAULT '',
 idx INTEGER NOT NULL,
 channel TEXT NOT NULL,
 type TEXT,
 value BYTEA,
 PRIMARY KEY (thread_id, checkpoint_ns, checkpoint_id, task_id, task_path, idx)
);
```

Because `checkpoint_id` is a ULID, it sorts lexicographically — larger values are newer. "Get latest" is `ORDER BY checkpoint_id DESC LIMIT 1`; "get by id" is an equality lookup on the primary key.

**For non-SQL stores:** the same principle applies. Whatever key scheme you use, direct lookup by `(thread_id, checkpoint_ns, checkpoint_id)` must be  $O(1)$  or close to it. Avoid designs where the only way to find a checkpoint by id is to scan all rows for a thread.

## Serialization

Always use `self.serde` (inherited from `BaseCheckpointSaver`, defaults to `JsonPlusSerializer`) for checkpoints, writes, and metadata. Do not use `pickle` directly for metadata — it works, but `JsonPlusSerializer` produces human-readable output and handles versioning better.

`JsonPlusSerializer` handles all LangGraph-native types automatically: - `_DeltaSnapshot` — the sentinel blob used by delta channels (msgpack ext code 7) - Pydantic v2 models, dataclasses, numpy arrays, datetimes, enums, and more

If you write a custom serializer, make sure it can round-trip `_DeltaSnapshot` from `langgraph.checkpoint.serde.types`.

## Extended capabilities

These methods are optional but unlock additional Agent Server features. Implement them if your storage backend can support them efficiently.

Method	What it enables
<code>delete_for_runs</code>	Rollback multitask strategy
<code>copy_thread</code>	Efficient thread forking
<code>prune</code>	Thread history pruning
<code>get_delta_channel_history</code>	Efficient delta channel state reconstruction (see below)

Agent Server auto-detects which capabilities your checkpointer implements at startup and activates the corresponding features.

## Delta channel support

### 정보

**DeltaChannel is in beta.** The API and on-disk representation may change while the design stabilizes.

`DeltaChannel` is a reducer channel that stores only a sentinel (`MISSING`) in checkpoint blobs instead of the full channel value. State is reconstructed by replaying ancestor writes through the reducer. This makes checkpoint blobs  $O(1)$  per step instead of  $O(N)$  for channels like `messages` that accumulate over time.

### What the runtime needs

When loading a checkpoint whose delta channels are absent from `channel_values`, LangGraph calls `saver.get_delta_channel_history(config=config, channels=[...])`. This returns, for each channel:

- `writes` — all writes to that channel in the ancestor chain, oldest first, up to the nearest snapshot.
- `seed` (optional) — the stored `_DeltaSnapshot` blob at the nearest ancestor that has one; absent if the walk reaches the root without finding a snapshot.

The runtime then calls `channel.from_checkpoint(seed)` and `channel.replay_writes(writes)` to reconstruct the live value.

## Default implementation

`BaseCheckpointSaver` provides a default `get_delta_channel_history` that works with any correct `get_tuple` implementation:

```
Simplified from BaseCheckpointSaver
def get_delta_channel_history(self, *, config, channels):
 target = self.get_tuple(config) # load the head checkpoint
 cursor = target.parent_config # walk from its parent
 collected = {ch: [] for ch in channels}
 seed = {}
 remaining = set(channels)

 while cursor and remaining:
 tup = self.get_tuple(cursor) # ← requires correct by-id lookup
 if tup is None:
 break
 for write in reversed(tup.pending_writes or []):
 if write[1] in remaining:
 collected[write[1]].append(write)
 for ch in list(remaining):
 if ch in tup.checkpoint["channel_values"]:
 seed[ch] = tup.checkpoint["channel_values"][ch]
 remaining.discard(ch)
 cursor = tup.parent_config

 return {
 ch: {"writes": list(reversed(collected[ch])), **({"seed": seed[ch]} if ch in seed else {})}
 for ch in channels
 }
```

**The critical dependency:** `get_tuple(cursor)` is always called with a specific `checkpoint_id` (the parent's id). If that lookup returns `None`, the walk stops immediately and every delta channel reconstructs as empty — silently, with no error. This is why the specific-id path in `get_tuple` must be correct.

## Performance override

The default walk issues one `get_tuple` call per ancestor checkpoint. For backends with good query support, override `get_delta_channel_history` (and its async twin) to retrieve the ancestor chain and writes in two queries:

```
async def aget_delta_channel_history(self, *, config, channels):
 if not channels:
```

```

 return {}

thread_id = config["configurable"]["thread_id"]
checkpoint_ns = config["configurable"].get("checkpoint_ns", "")
checkpoint_id = config["configurable"]["checkpoint_id"]

Stage 1: stream ancestors newest-first until every channel has a seed
ancestors = await self.db.fetchall(
 "SELECT checkpoint_id, parent_checkpoint_id, type, checkpoint "
 "FROM checkpoints "
 "WHERE thread_id=? AND checkpoint_ns=? AND checkpoint_id < ? "
 "ORDER BY checkpoint_id DESC",
 thread_id, checkpoint_ns, checkpoint_id,
)

chain_by_ch: dict[str, list[str]] = {ch: [] for ch in channels}
seed_by_ch: dict[str, Any] = {}
remaining = set(channels)
cur_id = config["configurable"]["checkpoint_id"]

for row in ancestors:
 if not remaining:
 break
 parent_id = row["parent_checkpoint_id"]
 ckpt = self.serde.loads_typed((row["type"], row["checkpoint"]))
 cv = ckpt.get("channel_values") or {}
 for ch in list(remaining):
 chain_by_ch[ch].append(row["checkpoint_id"])
 if ch in cv:
 seed_by_ch[ch] = cv[ch]
 remaining.discard(ch)
 cur_id = parent_id

Stage 2: fetch writes for each channel's ancestor chain in one query
result: dict[str, DeltaChannelHistory] = {}
for ch in channels:
 chain = chain_by_ch[ch]
 if not chain:
 entry: DeltaChannelHistory = {"writes": []}
 if ch in seed_by_ch:
 entry["seed"] = seed_by_ch[ch]
 result[ch] = entry
 continue

write_rows = await self.db.fetchall(
 f"SELECT checkpoint_id, task_id, idx, type, value FROM writes "
 f"WHERE thread_id=? AND checkpoint_ns=? AND channel=? "
 f"AND checkpoint_id IN ({','.join('?' * len(chain))})"
 f"ORDER BY checkpoint_id, task_id, idx",
 thread_id, checkpoint_ns, ch, *chain,
)

```

```

writes_by_cid: dict[str, list[PendingWrite]] = {}
for row in write_rows:
 cid = row["checkpoint_id"]
 value = self.serde.loads_typed((row["type"], row["value"]))
 writes_by_cid.setdefault(cid, []).append((row["task_id"], ch, value))

chain is newest-first; iterate oldest-first to get correct replay order
collected: list[PendingWrite] = []
for cid in reversed(chain):
 collected.extend(writes_by_cid.get(cid, []))

entry = {"writes": collected}
if ch in seed_by_ch:
 entry["seed"] = seed_by_ch[ch]
result[ch] = entry

return result

```

## Pruning with delta channels

`DeltaChannel` state is not self-contained in a single checkpoint — it depends on the ancestor write chain back to the nearest `_DeltaSnapshot`. If you implement `prune` or `delete_for_runs`, you must not delete write rows that a surviving checkpoint's delta channels depend on.

Safe options:

1. **Walk before pruning** — for each checkpoint you intend to keep, walk its ancestor chain and mark all write rows up to the nearest `_DeltaSnapshot` as non-deletable.
2. **Force a snapshot before pruning** — rewrite `channel_values[ch] = _DeltaSnapshot(reconstructed_value)` on the checkpoint you are keeping, then delete ancestors freely.
3. **Skip pruning for delta-channel threads** — the safest short-term option if you do not yet need pruning.

## Copy thread with delta channels

When implementing `copy_thread`, copy the complete ancestor chain — not just the head checkpoint. The target thread must have write rows going back to at least one `_DeltaSnapshot` for every delta channel, or those channels will reconstruct as empty after the copy.

## Testing with the conformance suite

`langgraph-checkpoint-conformance` validates your implementation against the full contract, including delta channel history:

```
pip install langgraph-checkpoint-conformance
```



```

import asyncio
from langgraph.checkpoint.conformance import checkpoint_test, validate

@checkpoint_test(name="MyCheckpoint")
async def my_checkpoint():
 async with MyCheckpoint.create() as saver:
 yield saver

async def main():
 report = await validate(my_checkpoint)
 report.print_report()
 # Fails the process if any base capability is missing or broken
 if not report.passed_all_base():
 raise RuntimeError("Checkpoint failed conformance suite")

asyncio.run(main())

```

The suite auto-detects which extended capabilities your checkpoint implements (including `aget_delta_channel_history`) and runs the relevant tests for each. Run it as part of your CI before shipping.

# Choosing between Graph and Functional APIs

---

langgraph/choosing-apis.mdx

LangGraph provides two different APIs to build agent workflows: the **Graph API** and the **Functional API**. Both APIs share the same underlying runtime and can be used together in the same application, but they are designed for different use cases and development preferences.

This guide will help you understand when to use each API based on your specific requirements.

## Quick decision guide

---

Use the **Graph API** when you need: - **Complex workflow visualization** for debugging and documentation - **Explicit state management** with shared data across multiple nodes - **Conditional branching** with multiple decision points - **Parallel execution paths** that need to merge later - **Team collaboration** where visual representation aids understanding

Use the **Functional API** when you want: - **Minimal code changes** to existing procedural code - **Standard control flow** (if/else, loops, function calls) - **Function-scoped state** without explicit state management - **Rapid prototyping** with less boilerplate - **Linear workflows** with simple branching logic

## Detailed comparison

---

### When to use the Graph API

The [Graph API](#) uses a declarative approach where you define nodes, edges, and shared state to create a visual graph structure.

#### 1. Complex decision trees and branching logic

When your workflow has multiple decision points that depend on various conditions, the Graph API makes these branches explicit and easy to visualize.

python

```
Graph API: Clear visualization of decision paths
from langgraph.graph import StateGraph
from typing import TypedDict

class AgentState(TypedDict):
 messages: list
 current_tool: str
 retry_count: int
```

```

def should_continue(state):
 if state["retry_count"] > 3:
 return "end"
 elif state["current_tool"] == "search":
 return "process_search"
 else:
 return "call_llm"

workflow = StateGraph(AgentState)
workflow.add_node("call_llm", call_llm_node)
workflow.add_node("process_search", search_node)
workflow.add_conditional_edges("call_llm", should_continue)

```

js

```

import * as z from "zod";
import {
 StateGraph,
 StateSchema,
 MessagesValue,
 START,
 END,
 type GraphNode,
 type ConditionalEdgeRouter,
} from "@langchain/langgraph";

// Graph API: Clear visualization of decision paths
const AgentState = new StateSchema({
 messages: MessagesValue,
 currentTool: z.string(),
 retryCount: z.number().default(0),
});

const shouldContinue: ConditionalEdgeRouter<typeof AgentState> = (state) => {
 if (state.retryCount > 3) {
 return END;
 } else if (state.currentTool === "search") {
 return "processSearch";
 } else {
 return "callLlm";
 }
};

const workflow = new StateGraph(AgentState)
 .addNode("callLlm", callLlmNode)
 .addNode("processSearch", searchNode)
 .addConditionalEdges("callLlm", shouldContinue);

```

## 2. State management across multiple components

When you need to share and coordinate state between different parts of your workflow, the Graph API's explicit state management is beneficial.

### python

```
Multiple nodes can access and modify shared state
class WorkflowState(TypedDict):
 user_input: str
 search_results: list
 generated_response: str
 validation_status: str

def search_node(state):
 # Access shared state
 results = search(state["user_input"])
 return {"search_results": results}

def validation_node(state):
 # Access results from previous node
 is_valid = validate(state["generated_response"])
 return {"validation_status": "valid" if is_valid else "invalid"}
```

### js

```
import * as z from "zod";
import { StateSchema, type GraphNode } from "@langchain/langgraph";

// Multiple nodes can access and modify shared state
const WorkflowState = new StateSchema({
 userInput: z.string(),
 searchResults: z.array(z.string()).default([]),
 generatedResponse: z.string().optional(),
 validationStatus: z.string().optional(),
});

const searchNode: GraphNode<typeof WorkflowState> = async (state) => {
 // Access shared state
 const results = await search(state.userInput);
 return { searchResults: results };
};

const validationNode: GraphNode<typeof WorkflowState> = async (state) => {
 // Access results from previous node
 const isValid = await validate(state.generatedResponse);
```

```
return { validationStatus: isValid ? "valid" : "invalid" };
};
```

### 3. Parallel processing with synchronization

When you need to run multiple operations in parallel and then combine their results, the Graph API handles this naturally.

#### python

```
Parallel processing of multiple data sources
workflow.add_node("fetch_news", fetch_news)
workflow.add_node("fetch_weather", fetch_weather)
workflow.add_node("fetch_stocks", fetch_stocks)
workflow.add_node("combine_data", combine_all_data)

All fetch operations run in parallel
workflow.add_edge(START, "fetch_news")
workflow.add_edge(START, "fetch_weather")
workflow.add_edge(START, "fetch_stocks")

Combine waits for all parallel operations to complete
workflow.add_edge("fetch_news", "combine_data")
workflow.add_edge("fetch_weather", "combine_data")
workflow.add_edge("fetch_stocks", "combine_data")
```

#### js

```
import { START } from "@langchain/langgraph";

// Parallel processing of multiple data sources
workflow
 .addNode("fetchNews", fetchNews)
 .addNode("fetchWeather", fetchWeather)
 .addNode("fetchStocks", fetchStocks)
 .addNode("combineData", combineAllData)
 // All fetch operations run in parallel
 .addEdge(START, "fetchNews")
 .addEdge(START, "fetchWeather")
 .addEdge(START, "fetchStocks")
 // Combine waits for all parallel operations to complete
 .addEdge("fetchNews", "combineData")
 .addEdge("fetchWeather", "combineData")
 .addEdge("fetchStocks", "combineData");
```

### 4. Team development and documentation

The visual nature of the Graph API makes it easier for teams to understand, document, and maintain complex workflows.

### python

```
Clear separation of concerns - each team member can work on different nodes
workflow.add_node("data_ingestion", data_team_function)
workflow.add_node("ml_processing", ml_team_function)
workflow.add_node("business_logic", product_team_function)
workflow.add_node("output_formatting", frontend_team_function)
```

### js

```
// Clear separation of concerns - each team member can work on different nodes
workflow
 .addNode("dataIngestion", dataTeamFunction)
 .addNode("mlProcessing", mlTeamFunction)
 .addNode("businessLogic", productTeamFunction)
 .addNode("outputFormatting", frontendTeamFunction);
```

## When to use the Functional API

The [Functional API](#) uses an imperative approach that integrates LangGraph features into standard procedural code.

### 1. Existing procedural code

When you have existing code that uses standard control flow and want to add LangGraph features with minimal refactoring.

### python

```
Functional API: Minimal changes to existing code
from langgraph.func import entrypoint, task

@task
def process_user_input(user_input: str) -> dict:
 # Existing function with minimal changes
 return {"processed": user_input.lower().strip()}

@entrypoint(checkpointer=checkpointer)
def workflow(user_input: str) -> str:
 # Standard Python control flow
 processed = process_user_input(user_input).result()

 if "urgent" in processed["processed"]:
```

```

 response = handle_urgent_request(processed).result()
 else:
 response = handle_normal_request(processed).result()

 return response

```

**js**

```

import { task, entrypoint } from "@langchain/langgraph";

// Functional API: Minimal changes to existing code
const processUserInput = task(
 "processUserInput",
 async (userInput: string) => {
 // Existing function with minimal changes
 return { processed: userInput.toLowerCase().trim() };
 }
);

const workflow = entrypoint(
 { checkpointer },
 async (userInput: string) => {
 // Standard control flow
 const processed = await processUserInput(userInput);

 let response: string;
 if (processed.processed.includes("urgent")) {
 response = await handleUrgentRequest(processed);
 } else {
 response = await handleNormalRequest(processed);
 }

 return response;
 }
);

```

## 2. Linear workflows with simple logic

When your workflow is primarily sequential with straightforward conditional logic.

**python**

```

@entrypoint(checkpointer=checkpointer)
def essay_workflow(topic: str) -> dict:
 # Linear flow with simple branching
 outline = create_outline(topic).result()

 if len(outline["points"]) < 3:

```

```

 outline = expand_outline(outline).result()

draft = write_draft(outline).result()

Human review checkpoint
feedback = interrupt({"draft": draft, "action": "Please review"})

if feedback == "approve":
 final_essay = draft
else:
 final_essay = revise_essay(draft, feedback).result()

 return {"essay": final_essay}

```

js

```

import { entrypoint, interrupt } from "@langchain/langgraph";

const essayWorkflow = entrypoint(
 { checkpointer },
 async (topic: string) => {
 // Linear flow with simple branching
 let outline = await createOutline(topic);

 if (outline.points.length < 3) {
 outline = await expandOutline(outline);
 }

 const draft = await writeDraft(outline);

 // Human review checkpoint
 const feedback = interrupt({ draft, action: "Please review" });

 let finalEssay: string;
 if (feedback === "approve") {
 finalEssay = draft;
 } else {
 finalEssay = await reviseEssay(draft, feedback);
 }

 return { essay: finalEssay };
 }
);

```

### 3. Rapid prototyping

When you want to quickly test ideas without the overhead of defining state schemas and graph structures.



## python

```
@entrypoint(checkpointer=checkpointer)
def quick_prototype(data: dict) -> dict:
 # Fast iteration - no state schema needed
 step1_result = process_step1(data).result()
 step2_result = process_step2(step1_result).result()

 return {"final_result": step2_result}
```

## js

```
import { entrypoint } from "@langchain/langgraph";

const quickPrototype = entrypoint(
 { checkpointer },
 async (data: Record<string, unknown>) => {
 // Fast iteration - no state schema needed
 const step1Result = await processStep1(data);
 const step2Result = await processStep2(step1Result);

 return { finalResult: step2Result };
 }
);
```

## 4. Function-scoped state management

When your state is naturally scoped to individual functions and doesn't need to be shared broadly.

## python

```
@task
def analyze_document(document: str) -> dict:
 # Local state management within function
 sections = extract_sections(document)
 summaries = [summarize(section) for section in sections]
 key_points = extract_key_points(summaries)

 return {
 "sections": len(sections),
 "summaries": summaries,
 "key_points": key_points
 }

@entrypoint(checkpointer=checkpointer)
def document_processor(document: str) -> dict:
 analysis = analyze_document(document).result()
```

```
State is passed between functions as needed
return generate_report(analysis).result()
```

js

```
import { task, entrypoint } from "@langchain/langgraph";

const analyzeDocument = task("analyzeDocument", async (document: string) => {
 // Local state management within function
 const sections = extractSections(document);
 const summaries = await Promise.all(sections.map(summarize));
 const keyPoints = extractKeyPoints(summaries);

 return {
 sections: sections.length,
 summaries,
 keyPoints,
 };
});

const documentProcessor = entrypoint(
 { checkpointer },
 async (document: string) => {
 const analysis = await analyzeDocument(document);
 // State is passed between functions as needed
 return await generateReport(analysis);
 }
);
```

## Combining both APIs

You can use both APIs together in the same application. This is useful when different parts of your system have different requirements.

python

```
from langgraph.graph import StateGraph
from langgraph.func import entrypoint

Complex multi-agent coordination using Graph API
coordination_graph = StateGraph(CoordinationState)
coordination_graph.add_node("orchestrator", orchestrator_node)
coordination_graph.add_node("agent_a", agent_a_node)
coordination_graph.add_node("agent_b", agent_b_node)

Simple data processing using Functional API
```

```

@entrypoint()
def data_processor(raw_data: dict) -> dict:
 cleaned = clean_data(raw_data).result()
 transformed = transform_data(cleaned).result()
 return transformed

Use the functional API result in the graph
def orchestrator_node(state):
 processed_data = data_processor.invoke(state["raw_data"])
 return {"processed_data": processed_data}

```

js

```

import * as z from "zod";
import {
 StateGraph,
 StateSchema,
 entrypoint,
 type GraphNode,
} from "@langchain/langgraph";

// Define state for complex multi-agent coordination
const CoordinationState = new StateSchema({
 rawData: z.record(z.string(), z.unknown()),
 processedData: z.record(z.string(), z.unknown()).optional(),
});

// Simple data processing using Functional API
const dataProcessor = entrypoint({}, async (rawData: Record<string, unknown>) => {
 const cleaned = await cleanData(rawData);
 const transformed = await transformData(cleaned);
 return transformed;
});

// Use the functional API result in the graph
const orchestratorNode: GraphNode<typeof CoordinationState> = async (state) => {
 const processedData = await dataProcessor.invoke(state.rawData);
 return { processedData };
};

// Complex multi-agent coordination using Graph API
const coordinationGraph = new StateGraph(CoordinationState)
 .addNode("orchestrator", orchestratorNode)
 .addNode("agentA", agentANode)
 .addNode("agentB", agentBNode);

```

# Migration between APIs

## From Functional to Graph API

When your functional workflow grows complex, you can migrate to the Graph API:

### python

```
Before: Functional API
@entrypoint(checkpointer=checkpointer)
def complex_workflow(input_data: dict) -> dict:
 step1 = process_step1(input_data).result()

 if step1["needs_analysis"]:
 analysis = analyze_data(step1).result()
 if analysis["confidence"] > 0.8:
 result = high_confidence_path(analysis).result()
 else:
 result = low_confidence_path(analysis).result()
 else:
 result = simple_path(step1).result()

 return result

After: Graph API
class WorkflowState(TypedDict):
 input_data: dict
 step1_result: dict
 analysis: dict
 final_result: dict

def should_analyze(state):
 return "analyze" if state["step1_result"]["needs_analysis"] else "simple_path"

def confidence_check(state):
 return "high_confidence" if state["analysis"]["confidence"] > 0.8 else "low_confidence"

workflow = StateGraph(WorkflowState)
workflow.add_node("step1", process_step1_node)
workflow.add_conditional_edges("step1", should_analyze)
workflow.add_node("analyze", analyze_data_node)
workflow.add_conditional_edges("analyze", confidence_check)
... add remaining nodes and edges
```

### js

```
import * as z from "zod";
import { entrypoint } from "@langchain/langgraph";
```

```
// Before: Functional API
const complexWorkflow = entrypoint(
 { checkpointer },
 async (inputData: Record<string, unknown>) => {
 const step1 = await processStep1(inputData);

 let result: unknown;
 if (step1.needsAnalysis) {
 const analysis = await analyzeData(step1);
 if (analysis.confidence > 0.8) {
 result = await highConfidencePath(analysis);
 } else {
 result = await lowConfidencePath(analysis);
 }
 } else {
 result = await simplePath(step1);
 }

 return result;
 }
);

// After: Graph API
import {
 StateGraph,
 StateSchema,
 type GraphNode,
 type ConditionalEdgeRouter,
} from "@langchain/langgraph";

const WorkflowState = new StateSchema({
 inputData: z.record(z.string(), z.unknown()),
 step1Result: z.record(z.string(), z.unknown()).optional(),
 analysis: z.record(z.string(), z.unknown()).optional(),
 finalResult: z.unknown().optional(),
});

const shouldAnalyze: ConditionalEdgeRouter<typeof WorkflowState> = (state) => {
 return state.step1Result?.needsAnalysis ? "analyze" : "simplePath";
};

const confidenceCheck: ConditionalEdgeRouter<typeof WorkflowState> = (state) => {
 return (state.analysis?.confidence as number) > 0.8
 ? "highConfidence"
 : "lowConfidence";
};

const workflow = new StateGraph(WorkflowState)
 .addNode("step1", processStep1Node)
 .addConditionalEdges("step1", shouldAnalyze)
```

```
.addNode("analyze", analyzeDataNode)
.addConditionalEdges("analyze", confidenceCheck);
// ... add remaining nodes and edges
```

## From Graph to Functional API

When your graph becomes overly complex for simple linear processes:

### python

```
Before: Over-engineered Graph API
class SimpleState(TypedDict):
 input: str
 step1: str
 step2: str
 result: str

After: Simplified Functional API
@entrypoint(checkpointer=checkpointer)
def simple_workflow(input_data: str) -> str:
 step1 = process_step1(input_data).result()
 step2 = process_step2(step1).result()
 return finalize_result(step2).result()
```

### js

```
import { z } from "zod/v4";
import { StateGraph, StateSchema, entrypoint } from "@langchain/langgraph";

// Before: Over-engineered Graph API
const SimpleState = new StateSchema({
 input: z.string(),
 step1: z.string().optional(),
 step2: z.string().optional(),
 result: z.string().optional(),
});

// After: Simplified Functional API
const simpleWorkflow = entrypoint(
 { checkpointer },
 async (inputData: string) => {
 const step1 = await processStep1(inputData);
 const step2 = await processStep2(step1);
 return await finalizeResult(step2);
 }
);
```

## Summary

---

Choose the **Graph API** when you need explicit control over workflow structure, complex branching, parallel processing, or team collaboration benefits.

Choose the **Functional API** when you want to add LangGraph features to existing code with minimal changes, have simple linear workflows, or need rapid prototyping capabilities.

Both APIs provide the same core LangGraph features (persistence, streaming, human-in-the-loop, memory) but package them in different paradigms to suit different development styles and use cases.

# Deployment

---

langgraph/deploy.mdx

When you are ready to deploy your LangGraph agent to production, choose a hosting model that fits your stack. [LangSmith Cloud](#) provides fully managed infrastructure for stateful, long-running agents with persistent state and background execution.

js

You can also deploy on **JavaScript frameworks and platforms** such as Next.js, SvelteKit, Nuxt, Cloudflare Workers, and Deno Deploy using the same [Agent Streaming Protocol](#).

팁

LangSmith offers multiple deployment options beyond Cloud, including [hybrid](#), [standalone servers](#), and [self-hosted with control plane](#). For more information, see the [LangSmith Deployment overview](#).

## LangSmith Cloud

---

This section walks through deploying your agent to LangSmith Cloud from a GitHub repository. LangSmith handles infrastructure, scaling, and operational concerns.

### Prerequisites

Before you begin, ensure you have the following:

- A [GitHub account](#)
- A [LangSmith account](#) (free to sign up)

### Deploy your agent

#### 1. Create a repository on GitHub

Your application's code must reside in a GitHub repository to be deployed on LangSmith. Both public and private repositories are supported. For this quickstart, first make sure your app is LangGraph-compatible by following the [local server setup guide](#). Then, push your code to the repository.

#### 2. Deploy to LangSmith

#### Navigate to LangSmith Deployment



Log in to [LangSmith](#). In the left sidebar, select **Deployments**.

## Create new deployment

Click the + **New Deployment** button. A pane will open where you can fill in the required fields.

## Link repository

If you are a first time user or adding a private repository that has not been previously connected, click the **Add new account** button and follow the instructions to connect your GitHub account.

## Deploy repository

Select your application's repository. Click **Submit** to deploy. This may take about 15 minutes to complete. You can check the status in the **Deployment details** view.

## 3. Test your application in Studio

Once your application is deployed:

1. Select the deployment you just created to view more details.
2. Click the **Studio** button in the top right corner. Studio will open to display your graph.

## 4. Get the API URL for your deployment

1. In the **Deployment details** view in LangGraph, click the **API URL** to copy it to your clipboard.
2. Click the `URL` to copy it to the clipboard.

## 5. Test the API

You can now test the API:

### python

#### Python

1. Install LangGraph SDK:

```
pip install langgraph-sdk
```

2. Send a message to the agent:

```
from langgraph_sdk import get_sync_client # or get_client for async

client = get_sync_client(url="your-deployment-url", api_key="your-langsmith-api-key")

for chunk in client.runs.stream(
 None, # Threadless run
 "agent", # Name of agent. Defined in langgraph.json.
```

```

input={
 "messages": [{
 "role": "human",
 "content": "What is LangGraph?",
 }],
},
stream_mode="updates",
):
 print(f"Receiving new event of type: {chunk.event}...")
 print(chunk.data)
 print("\n\n")

```

js

## TypeScript

### 1. Install LangGraph SDK:

```
npm install @langchain/langgraph-sdk
```

### 2. Send a message to the agent:

```

import { Client } from "@langchain/langgraph-sdk";

const client = new Client({ apiUrl: "your-deployment-url", apiKey: "your-langsmith-api-key"
});

const streamResponse = client.runs.stream(
 null, // Threadless run
 "agent", // Name of agent. Defined in langgraph.json.
 {
 input: {
 "messages": [
 { "role": "user", "content": "What is LangGraph?" }
]
 },
 streamMode: "messages",
 }
);

for await (const chunk of streamResponse) {
 console.log(`Receiving new event of type: ${chunk.event}...`);
 console.log(JSON.stringify(chunk.data));
 console.log("\n\n");
}

```

## Rest API

```
curl -s --request POST \
 --url <DEPLOYMENT_URL>/runs/stream \
 --header 'Content-Type: application/json' \
 --header "X-API-Key: <LANGSMITH API KEY> \
 --data "{
 \"assistant_id\": \"agent\", `# Name of agent. Defined in langgraph.json.`
 \"input\": {
 \"messages\": [
 {
 \"role\": \"human\",
 \"content\": \"What is LangGraph?\"
 }
]
 },
 \"stream_mode\": \"updates\"
 }"
```

# Event streaming

---

langgraph/event-streaming.mdx

Event streaming is the recommended in-process streaming model for most LangGraph application code. It returns a run stream object that can be consumed in multiple ways at the same time.

## Quickstart

---

### python

```
stream = graph.stream_events({
 "messages": [{"role": "user", "content": "What is 42 * 17?"}],
}, version="v3")

for message in stream.messages:
 for token in message.text:
 print(token, end="", flush=True)

final_state = stream.output
```

### js

```
const stream = await graph.streamEvents(
 { messages: [{ role: "user", content: "What is 42 * 17?" }] },
 { version: "v3" }
);

for await (const message of stream.messages) {
 for await (const token of message.text) {
 process.stdout.write(token);
 }
}

const finalState = await stream.output;
```

To stream against a graph deployed behind an Agent Server, see the [LangSmith Streaming API](#).

## How the pieces fit together

---

The streaming stack has two main layers:

1. **Streaming** emits raw graph execution events from the Pregel engine.

2. **Event streaming** normalizes those events, runs them through stream transformers, and exposes typed projections.

Pregel engine

Runs graph steps

emits

Raw Pregel events

`updates`, `values`, `messages`, `custom`, `checkpoints`, `tasks`, `debug`

sent to

Event router

Routes each event through the transformer pipeline

cascades through

Stream transformers

ValuesTransformer

MessagesTransformer

...

Custom transformers

produces

Event Stream

Projected events for application code

The event router is the bridge between the two layers. It receives normalized Pregel events and passes each event through the registered stream transformers. Built-in transformers create standard projections such as `stream.messages`, `stream.values`, `stream.subgraphs`, and `stream.output`. Custom transformers can add application-specific projections under `stream.extensions`.

## What event streaming provides

---

The run stream exposes typed projections over one underlying event flow:

Projection	Use
<code>stream</code>	Iterate every protocol event.
<code>stream.messages</code>	Stream chat model messages and token deltas.
<code>stream.values</code>	Iterate state snapshots and await the final value.
<code>stream.output</code>	Await the final output.
<code>stream.subgraphs</code>	Discover and observe nested graph executions.
<code>stream.interrupts</code>	Inspect human-in-the-loop interrupt payloads.
<code>stream.interrupted</code>	Check whether the run paused for human input.
<code>stream.extensions</code>	Consume custom stream transformer projections.

Multiple consumers can read these projections concurrently. Reading `stream.messages` does not consume events needed by `stream.values`, `stream.subgraphs`, or `stream.output`.

Event streaming sits one level above [streaming](#), which exposes raw graph execution events through `stream_mode` modes such as `updates`, `values`, `messages`, `custom`, `checkpoints`, `tasks`, and `debug`. Use streaming when you need low-level access to those modes; use event streaming when application code benefits from typed projections.

## Stream messages

---

Use `stream.messages` for chat model output:

### python

```
stream = graph.stream_events(input, version="v3")

for message in stream.messages:
 text = str(message.text)
 usage = message.output.usage_metadata

 print(text)
 print(usage)
```

### js

```
const stream = await graph.streamEvents(input, { version: "v3" });

for await (const message of stream.messages) {
 const text = await message.text;
 const usage = await message.usage;

 console.log(text);
 console.log(usage);
}
```

### python

`message.text` is iterable in synchronous code. Iterate it for token-by-token output, or call `str(message.text)` for the complete text.

`message.reasoning` exposes reasoning deltas, and `message.tool_calls` exposes tool-call argument chunks. If you need text, reasoning, and tool-call chunks in exact arrival order, iterate the message stream's raw events instead of each projection separately.

js

`message.text` is both an async iterable and a promise-like value. Iterate it for token-by-token output, or await it for the complete text.

## Stream subgraphs

Use `stream.subgraphs` to observe nested graph work without parsing namespace strings:

python

```
stream = graph.stream_events(input, version="v3")

for subgraph in stream.subgraphs:
 print(subgraph.graph_name, subgraph.path)

 for message in subgraph.messages:
 print(message.text)
```

js

```
const stream = await graph.streamEvents(input, { version: "v3" });

for await (const subgraph of stream.subgraphs) {
 console.log(subgraph.name, subgraph.path);

 for await (const message of subgraph.messages) {
 console.log(await message.text);
 }
}
```

`subgraph.graph_name` is the `name` of the compiled graph or agent. A named agent dispatched from a tool (for example, a `create_agent(name=...)` invoked through the Deep Agents `task` tool) surfaces here under that name, and the `lifecycle` event that opens the scope carries a `cause` linking back to the dispatching tool call. See [Lifecycle](#) for more information.

For product-specific streams, see [Deep Agents streaming](#) for subagent streams and [LangChain agent streaming](#) for tool calls and middleware events.

## Stream state

Use `stream.values` to stream full state snapshots after each step:

## python

```
stream = graph.stream_events(input, version="v3")

for snapshot in stream.values:
 print(snapshot)

final_state = stream.output
```

## js

```
const stream = await graph.streamEvents(input, { version: "v3" });

for await (const snapshot of stream.values) {
 console.log(snapshot);
}

const finalState = await stream.output;
```

# Stream multiple projections

## python

For concurrent consumption in async code, use `astream_events` with `asyncio.gather`:

```
import asyncio

stream = await graph.astream_events(input, version="v3")

async def consume_messages():
 async for message in stream.messages:
 print(f"[llm] node={message.node}")

async def consume_subgraphs():
 async for subgraph in stream.subgraphs:
 print(f"[subgraph] path={subgraph.path}")

await asyncio.gather(consume_messages(), consume_subgraphs())
```

For synchronous code, use `stream.interleave(...)` to consume multiple projections in strict arrival order:

```
stream = graph.stream_events(input, version="v3")

for name, item in stream.interleave("values", "messages", "subgraphs"):
```



```

if name == "values":
 print(f"[state] keys={list(item)}")
elif name == "messages":
 print(f"[llm] node={item.node}")
elif name == "subgraphs":
 print(f"[subgraph] path={item.path}")

```

## js

Use concurrent consumers when you need multiple projections in JavaScript:

```

await Promise.all([
 (async () => {
 for await (const message of stream.messages) {
 console.log(await message.text);
 }
 })(),
 (async () => {
 for await (const subgraph of stream.subgraphs) {
 console.log(subgraph.path);
 }
 })(),
]);

```

## Resume after an interrupt

When a graph pauses for human input, inspect `stream.interrupted` and `stream.interrupts`, then resume by calling `stream_events(..., version="v3")` again with `Command`.

Resume requires a graph compiled with a checkpointer and a config carrying a thread ID — see [persistence](#).

## python

```

from langgraph.types import Command

stream = graph.stream_events(input, version="v3")

for message in stream.messages:
 print(message.text)

if stream.interrupted:
 print(stream.interrupts)

stream = graph.stream_events(
 Command(resume={"decisions": [{"type": "approve"}]}),

```

```

 version="v3",
)
 final_state = stream.output

```

**js**

```

import { Command } from "@langchain/langgraph";

let stream = await graph.streamEvents(input, { version: "v3" });

for await (const message of stream.messages) {
 console.log(await message.text);
}

if (stream.interrupted) {
 console.log(stream.interrupts);
}

stream = await graph.streamEvents(
 new Command({ resume: { decisions: [{ type: "approve" }] } }),
 { version: "v3" }
);
const finalState = await stream.output;

```

## Stream all protocol events

Use the run object itself when you want the raw protocol event stream:

**python**

```

stream = graph.stream_events({
 "messages": [{"role": "user", "content": "What is 42 * 17?"}],
}, version="v3")

for event in stream:
 namespace = event["params"]["namespace"]
 print(namespace, event["method"], event["params"]["data"])

```

**js**

```

const stream = await graph.streamEvents(
 { messages: [{ role: "user", content: "What is 42 * 17?" }] },
 { version: "v3" }
);

```

```
for await (const event of stream) {
 const namespace = event.params.namespace;
 console.log(namespace, event.method, event.params.data);
}
```

Each event is a `ProtocolEvent` envelope wrapping a channel-specific payload. The same shape is what a transformer's `process(event)` receives.

## python

```
class ProtocolEvent(TypedDict):
 seq: int # strictly increasing within a run; use for ordering
 method: str # channel name: "messages", "values", "updates", "custom",
 "tools", "lifecycle", ...
 params: ProtocolEventParams

class ProtocolEventParams(TypedDict):
 namespace: list[str] # path of "<name>:<runtime_id>" segments from the root graph; [] is the root
 timestamp: int # wall-clock milliseconds; can drift, don't rely on for ordering
 data: Any # channel-specific payload; shape depends on `method`
```

## js

```
interface ProtocolEvent {
 readonly seq: number; // strictly increasing within a run; use for ordering
 readonly method: string; // channel name: "messages", "values", "updates", "custom",
 "tools", "lifecycle", ...
 readonly params: {
 readonly namespace: string[]; // path of "<name>:<runtime_id>" segments from the root graph; [] is the root
 readonly timestamp: number; // wall-clock milliseconds; can drift, don't rely on for ordering
 readonly node?: string; // graph node that emitted this event, when applicable
 readonly data: unknown; // channel-specific payload; shape depends on `method`
 };
}
```

The `namespace` is a path from the root graph to the scope that emitted the event. The root is the empty array `[]`. Each child execution adds one `"name:runtime_id"` segment, so a nested tool call inside a subgraph looks like `["researcher:6f4d", "tools:91ac"]`. The name before `:` is the stable graph or node name; the suffix is a per-invocation runtime ID. Filter raw events by namespace yourself when you only care about a specific subtree — `stream.subgraphs` already does this for nested graph executions.

# Channels and event lifecycle

Raw events flow on channels. The channel name appears as the event's `method` ; each channel emits a specific event shape.

Channel	Purpose
<code>values</code>	Full graph state snapshots.
<code>updates</code>	Per-node state deltas.
<code>messages</code>	Content-block-centric chat model output.
<code>tools</code>	Tool call start, streamed output, finish, and error events.
<code>lifecycle</code>	Run, subgraph, and subagent status changes.
<code>checkpoints</code>	Lightweight checkpoint envelopes for branching and time travel.
<code>input</code>	Human-in-the-loop input requests and responses.
<code>tasks</code>	Pregel task creation and result events.
<code>custom</code>	User-defined payloads from graph code.
<code>custom:&lt;name&gt;</code>	Application-defined stream transformer output.

The typed projections ( `stream.messages` , `stream.values` , etc.) are built from these channels. The channel name appears as the `method` field on raw events when you iterate the run object directly.

## Messages

The `messages` channel models output as content blocks. The data's `event` field is one of:

- `message-start`
- `content-block-start`
- `content-block-delta`
- `content-block-finish`
- `message-finish`

Content blocks have explicit boundaries: a block starts, emits zero or more deltas, and finishes before the next block in the same message starts. This makes token streaming, reasoning blocks, tool-call blocks, and multimodal content explicit without requiring provider-specific formats. `message-finish` may include token usage; unrecoverable model-call failures arrive as message error events.

To consume raw content-block events directly instead of using the `stream.messages` projection:

python

```

for event in stream:
 if event["method"] != "messages":
 continue

 data = event["params"]["data"][0]
 if not isinstance(data, dict):
 continue
 if data.get("event") != "content-block-delta":
 continue

 block = data.get("delta") or {}
 if block.get("type") == "text-delta":
 print(block.get("text", ""), end="", flush=True)
 elif block.get("type") == "reasoning-delta":
 print(f"[thinking]{block.get('reasoning', '')}", end="", flush=True)

```

js

```

for await (const event of stream) {
 if (event.method !== "messages") continue;

 const data = event.params.data;
 if (data.event !== "content-block-delta") continue;

 const block = data.delta ?? {};
 if (block.type === "text-delta") {
 process.stdout.write(block.text ?? "");
 } else if (block.type === "reasoning-delta") {
 process.stdout.write(`[thinking]${block.reasoning ?? ""}`);
 }
}

```

## Tools

The `tools` channel exposes tool execution. The data's `event` field is one of:

- `tool-started`
- `tool-output-delta`
- `tool-finished`
- `tool-error`

Tool events are correlated by tool call ID, so a tool execution can be joined back to its originating tool-call content block on the `messages` channel.

## Lifecycle

The `lifecycle` channel tracks root run, subgraph, and subagent status. The data's `event` field is one of:

- `started`
- `running`
- `completed`
- `failed`
- `interrupted`

Beyond `event`, lifecycle data may include an optional `graph_name`, `error`, and `cause` describing why a child scope started (parent tool call, fan-out send, edge transition).

## Build your own projection

---

Stream transformers are the projection layer in event streaming. They observe protocol events, keep their own state, and expose derived views of a run — things like tool activity, token totals, progress events, artifacts, or messages for another protocol. `StreamChannel` is the projection primitive transformers use to publish those views.

Built-in projections (`stream.messages`, `stream.values`, `stream.subgraphs`, `stream.output`) and product-specific projections (LangChain's `stream.tool_calls`, Deep Agents' `stream.subagents`) are themselves transformers using this same contract. User transformers stack on top via compile-time or call-time registration, and their projections appear under `stream.extensions`.

Write one when the existing projections don't match the shape an application needs.

## How transformers work

Event streaming starts with streaming output from the LangGraph Pregel engine. The runtime normalizes those chunks into protocol events, then a stream handler routes each event through a stack of stream transformers.

```
flowchart TD
 A[Pregel modes] --> B[Events]
 B --> C[Built-in projections]
 C --> D[User transformers]
 D --> E[Run projections]
```

The stream handler is the central dispatcher for one stream. For every protocol event, it:

1. Calls each registered transformer's `process(event)` hook in order.
2. Wires named `StreamChannel` pushes back onto the protocol event stream.
3. Stores the event in the run stream unless a transformer suppresses it.
4. Calls `finalize()` or `fail()` on every transformer when the run ends.

Transformers are observational. They do not call back into the graph runtime. Instead, they consume events and push derived values into `StreamChannel`, promises, or other projection objects.

## Transformer shape

A transformer implements the `StreamTransformer` interface:

### python

```
from langgraph.stream import ProtocolEvent, StreamTransformer

class MyTransformer(StreamTransformer):
 def init(self) -> dict:
 ...

 def process(self, event: ProtocolEvent) -> bool:
 ...

 def finalize(self) -> None:
 ...

 def fail(self, err: BaseException) -> None:
 ...
```

### js

```
interface StreamTransformer<TProjection = unknown> {
 init(): TProjection;
 process(event: ProtocolEvent): boolean;
 finalize?(): void | PromiseLike<void>;
 fail?(err: unknown): void;
}
```

- `init()` creates the projection object. User transformer projections appear under `stream.extensions`.
- `process()` observes each protocol event. See [Stream all protocol events](#) for the `ProtocolEvent` shape. Return `false` only when you intentionally want to suppress the original event.
- `finalize()` closes or resolves non-channel projections after a successful stream.
- `fail()` propagates errors to non-channel projections.

## Declaring required stream modes

`required_stream_modes` controls which Pregel stream modes the underlying graph emits during the stream. The runtime takes the union of every registered transformer's `required_stream_modes` and passes that union as the `stream_mode` argument to the graph's `.stream()` call. **Modes that no**

**transformer requests are never emitted** — declaring `("custom",)` is what causes `custom` events to flow through the run at all.

python

```
class CustomTransformer(StreamTransformer):
 required_stream_modes = ("custom",) # [!code highlight]

 def process(self, event: ProtocolEvent) -> bool:
 if event["method"] == "custom":
 ...
 return True
```

`process()` receives every event the graph emits and is responsible for filtering by `event["method"]`. The declaration turns on upstream emission; it does not narrow what `process()` sees. Valid values are the Pregel stream modes: `"messages"`, `"tools"`, `"custom"`, `"values"`, `"updates"`, `"checkpoints"`, `"tasks"`, `"debug"`. Each transformer must declare every mode it acts on — an omitted mode is not emitted by the graph and never reaches `process()`.

StreamChannel

`StreamChannel` is the projection primitive a transformer uses for streaming values. It always exposes an iterable stream on `stream.extensions.<name>`. The constructor argument decides whether each `push()` also flows into the run's main event stream as a `custom:<name>` event—that is, whether the projection's values show up when iterating raw protocol events.

js

Need	Use
Side-channel projection only	<code>`new StreamChannel</code>

`()` | | Also flow each push into the main event stream | `new StreamChannel`  
`(name)`` |

python

Need	Use
Side-channel projection only	<code>StreamChannel()</code>
Also flow each push into the main event stream	<code>StreamChannel(name)</code>

Named channel payloads must be serializable, because each pushed value also becomes a `custom:<name>` protocol event in the main stream. Keep promises, async iterables, class instances, and other in-process handles in unnamed channels.



The stream handler owns channel lifecycle. Once `init()` returns a channel, the handler closes or fails it for you when the run ends. Transformers only push values.

## Example: named channel

Pass a string name to `StreamChannel` to expose a streaming projection through `stream.extensions` and forward each pushed value into the run's main event stream as a `custom:<name>` protocol event:

### python

```
from typing import TypedDict

from langgraph.stream import ProtocolEvent, StreamChannel, StreamTransformer

class ToolActivity(TypedDict):
 name: str
 status: str

class ToolActivityTransformer(StreamTransformer):
 required_stream_modes = ("tools",)

 def __init__(self, scope: tuple[str, ...] = ()) -> None:
 super().__init__(scope)
 self.activity = StreamChannel[ToolActivity]("tool_activity")

 def init(self) -> dict:
 return {"tool_activity": self.activity}

 def process(self, event: ProtocolEvent) -> bool:
 if event["method"] != "tools":
 return True

 data = event["params"]["data"]
 if isinstance(data, dict) and data.get("tool_name") and data.get("event"):
 status = "error" if data["event"] == "tool-error" else "started"
 self.activity.push({"name": data["tool_name"], "status": status})
 return True
```

### js

```
import { StreamChannel } from "@langchain/langgraph";

const toolActivityTransformer = () => {
 const activity = new StreamChannel<{
 name: string;
 status: "started" | "finished" | "error";
 }>("toolActivity");
```

```

return {
 init: () => ({ toolActivity: activity }),
 process(event) {
 if (event.method === "tools") {
 const data = event.params.data as { tool_name?: string; event?: string };
 if (data.tool_name && data.event) {
 activity.push({
 name: data.tool_name,
 status: data.event === "tool-error" ? "error" : "started",
 });
 }
 }
 return true;
 },
};
};

```

## Example: unnamed channel

Without a name, the channel is a side-channel projection only — accessible on `stream.extensions` but not visible to consumers iterating raw events. This is the right choice for projections that hold in-process handles (promises, async iterables, class instances) that can't be serialized onto the main event stream.

The example below pairs an unnamed channel with `get_stream_writer`, which lets graph nodes emit `custom`-channel events that the transformer then drains into the projection:

### python

```

from langgraph.config import get_stream_writer
from langgraph.stream import ProtocolEvent, StreamChannel, StreamTransformer

def node(state):
 writer = get_stream_writer()
 writer({"kind": "progress", "message": "retrieving context"})
 return state

class CustomTransformer(StreamTransformer):
 required_stream_modes = ("custom",)

 def __init__(self, scope: tuple[str, ...] = ()) -> None:
 super().__init__(scope)
 self.log = StreamChannel()

 def init(self) -> dict:
 return {"custom": self.log}

 def process(self, event: ProtocolEvent) -> bool:

```

```

 if event["method"] == "custom":
 self.log.push(event["params"]["data"])
 return True

stream = graph.stream_events(input, version="v3", transformers=[CustomTransformer])

for item in stream.extensions["custom"]:
 print(item)

```

js

```

import { StreamChannel } from "@langchain/langgraph";

const customTransformer = () => {
 const custom = new StreamChannel<unknown>();

 return {
 init: () => ({ custom }),
 process(event) {
 if (event.method === "custom") {
 custom.push(event.params.data);
 }
 return true;
 },
 };
};

```

## Example: final-value projection

Use unnamed streams, promises, or other in-process objects when the projection should not flow into the main event stream:

python

```

from langgraph.stream import ProtocolEvent, StreamChannel, StreamTransformer

class StatsTransformer(StreamTransformer):
 required_stream_modes = ("messages",)

 def __init__(self, scope: tuple[str, ...] = ()) -> None:
 super().__init__(scope)
 self.total_tokens = 0
 self.total_tokens_log = StreamChannel[int]()

 def init(self) -> dict:
 return {"total_tokens": self.total_tokens_log}

```

```

def process(self, event: ProtocolEvent) -> bool:
 data = event["params"]["data"]
 if isinstance(data, dict):
 usage = data.get("usage") or {}
 self.total_tokens += usage.get("output_tokens") or 0
 return True

def finalize(self) -> None:
 self.total_tokens_log.push(self.total_tokens)
 self.total_tokens_log.close()

```

js

```

const statsTransformer = () => {
 let totalTokens = 0;
 let resolveTotal!: (value: number) => void;
 const totalTokensPromise = new Promise<number>((resolve) => {
 resolveTotal = resolve;
 });

 return {
 init: () => ({ totalTokens: totalTokensPromise }),
 process(event) {
 if (event.method === "messages") {
 const data = event.params.data as { usage?: { output_tokens?: number } };
 totalTokens += data.usage?.output_tokens ?? 0;
 }
 return true;
 },
 finalize: () => resolveTotal(totalTokens),
 };
};

```

## Register at call time or compile time

Pass transformers at call time for local experimentation:

python

```

stream = graph.stream_events(
 input,
 version="v3",
 transformers=[StatsTransformer, ToolActivityTransformer],
)

```

js

```
const stream = await graph.streamEvents(input, {
 version: "v3",
 transformers: [statsTransformer, toolActivityTransformer],
});
```

Compile transformers into the graph when every run of that graph should produce the projection:

### python

```
graph = builder.compile(
 transformers=[StatsTransformer, ToolActivityTransformer],
)
```

### js

```
const graph = builder.compile({
 transformers: [statsTransformer, toolActivityTransformer],
});
```

## Built-in: `ToolCallTransformer`

### python

LangGraph ships `ToolCallTransformer` as a built-in. Register it to expose `stream.tool_calls` on a plain `StateGraph`:

```
from langgraph.prebuilt import ToolCallTransformer

stream = graph.stream_events(input, version="v3", transformers=[ToolCallTransformer])

for tool_call in stream.tool_calls:
 print(tool_call.tool_name, tool_call.input)
```

## Related

LangGraph defines the streaming primitives. For using streaming with LangChain or Deep Agents, review the relevant product docs:

- [LangChain agent streaming](#) covers ReAct-style agent messages, tool calls, and middleware updates.
- [Deep Agents streaming](#) covers subagents, nested messages, and subagent tool calls.

- [LangChain frontend patterns](#) and [LangGraph frontend patterns](#) show UI use cases built on top of streamed state.
- [LangSmith Streaming API](#) covers streaming against a graph deployed behind an Agent Server.

The wire-level event and command formats are defined in the [Agent Protocol](#) repository and consumable as [langchain-protocol](#) on PyPI and [@langchain/protocol](#) on npm.

# Fault tolerance

langgraph/fault-tolerance.mdx

When a node fails—from a slow external API, a transient network error, or an unhandled exception—LangGraph gives you three composable mechanisms to respond:

- [Retries](#): automatically re-run failed attempts based on exception type and backoff settings
- [Timeouts](#): cap how long a single attempt may run
- [Error handling](#): run a recovery function after all retries are exhausted

## python

Use [set\\_node\\_defaults](#) to configure these mechanisms once for all nodes instead of repeating them on every `add_node` call.

## js

Use [setNodeDefaults](#) to configure these mechanisms once for all nodes instead of repeating them on every `addNode` call.

These compose in a fixed order: when a node attempt raises any exception (including `@[NodeTimeoutError]` from a timeout), the retry policy decides whether to retry. Only after retries are exhausted does the error handler run.

For stopping a run cleanly at a superstep boundary and resuming later, see [Graceful shutdown](#).

## python

### 참고

Per-node timeouts and node-level error handlers require `langgraph>=1.2`.

## js

### 참고

Per-node timeouts and node-level error handlers require `@langchain/langgraph>=1.4.0`.

```
%%{init:{'theme':'base','themeVariables':{'lineColor':'#40668D','primaryColor':'#E5F4FF','primaryTextColor':'#030710','primaryBorderColor':'#006DDD'}}}%
flowchart LR
```

```

start([Attempt starts]) --> exec[Run node]
exec -->|"success"| done([Continue graph])
exec -->|"any exception
including NodeTimeoutError"| retry{retry_policy
matches?}
retry -->|"yes, attempts left"| exec
retry -->|"exhausted or absent"| handler{error_handler?}
handler -->|"yes"| run_handler["Invoke handler
with NodeError"]
run_handler --> route([Update state +
Command goto])
handler -->|"no"| bubble([Exception
bubbles up])

```

```

classDef process fill:#E5F4FF,stroke:#006DDD,stroke-width:2px,color:#030710
classDef decision fill:#FDF3FF,stroke:#7E65AE,stroke-width:2px,color:#504B5F
classDef alert fill:#F8E8E6,stroke:#B27D75,stroke-width:2px,color:#634643
classDef output fill:#EBD0F0,stroke:#885270,stroke-width:2px,color:#441E33

```

```

class exec,run_handler process
class retry,handler decision
class bubble alert
class done,route,start output

```

## Retries

A retry policy automatically re-runs a failed node attempt based on exception type and backoff settings.

### python

Pass `retry_policy=` to `@[add_node]`:

```

from langgraph.types import RetryPolicy

builder.add_node(
 "call_api",
 call_api,
 retry_policy=RetryPolicy(max_attempts=3),
)

```

### js

Pass `retryPolicy` to `@[addNode]`:

```

import { StateGraph } from "@langchain/langgraph";

const graph = new StateGraph(State)
 .addNode("callApi", callApi, { retryPolicy: { maxAttempts: 3 } })
 .compile();

```



## Default behavior

### python

By default, `retry_on` uses `default_retry_on`, which retries on **any** exception except the following (and their subclasses):

- `ValueError`
- `TypeError`
- `ArithmeticError`
- `ImportError`
- `LookupError`
- `NameError`
- `SyntaxError`
- `RuntimeError`
- `ReferenceError`
- `StopIteration`
- `StopAsyncIteration`
- `OSError`

For exceptions from popular HTTP libraries such as `requests` and `httpx`, it only retries on 5xx status codes. `@[NodeTimeoutError]` is retryable by default.

### js

Retries are opt-in. A node retries only when it has a `retryPolicy` configured, either directly or through graph defaults with [setNodeDefaults](#). An empty policy (`{}`) is enough. Without a policy, the first failure ends the attempt and `LangGraph` does not call `retryOn`.

If the policy omits `retryOn`, `LangGraph` uses a built-in handler that retries thrown errors except:

- Abort and cancellation errors: `error.name === "AbortError"`, or `error.message` starts with `"Cancel"` or `"AbortError"`
- `GraphValueError`, matched by `error.name`
- Aborted connections: `error.code === "ECONNABORTED"`
- HTTP client errors with status 400, 401, 402, 403, 404, 405, 406, 407, or 409, read from `error.response?.status` or `error.status` for clients such as `fetch`, `Axios`, and similar clients
- OpenAI-style quota errors: `error.error?.code === "insufficient_quota"`

Other HTTP statuses, including 408 and 5xx responses, are retryable unless you override `retryOn`. `@[NodeTimeoutError]` is not on this blacklist, so it is retryable when a retry policy is configured.

Some failures bypass `retryOn`. Graph control-flow errors, such as `GraphInterrupt` and `Command routing`, bubble up without retrying. An aborted run signal also stops the retry loop, even if `retryOn` would return `true`.

## Parameters

### python

Parameter	Type	Default	Description
<code>max_attempts</code>	<code>int</code>	<code>3</code>	Maximum number of attempts, including the first.
<code>initial_interval</code>	<code>float</code>	<code>0.5</code>	Seconds before the first retry.
<code>backoff_factor</code>	<code>float</code>	<code>2.0</code>	Multiplier applied to the interval after each retry.
<code>max_interval</code>	<code>float</code>	<code>128.0</code>	Maximum seconds between retries.
<code>jitter</code>	<code>bool</code>	<code>True</code>	Add random jitter to the interval.
<code>retry_on</code>	<code>type[Exception] \</code> <code>Sequence[type[Exception]] \</code> <code>Callable[[Exception], bool]</code>	<code>default_retry_on</code>	Exceptions to retry on, or a callable returning <code>True</code> for retryable exceptions.

### js

Parameter	Type	Default	Description
<code>maxAttempts</code>	<code>number</code>	<code>3</code>	Maximum number of attempts, including the first.
<code>initialInterval</code>	<code>number</code>	<code>500</code>	Milliseconds before the first retry.
<code>backoffFactor</code>	<code>number</code>	<code>2.0</code>	Multiplier applied to the interval after each retry.
<code>maxInterval</code>	<code>number</code>	<code>128000</code>	Maximum milliseconds between retries.
<code>jitter</code>	<code>boolean</code>	<code>true</code>	Add random jitter to the interval.
<code>retryOn</code>	<code>(error: unknown) =&gt; boolean</code>	built-in handler (when policy is set)	Callable returning <code>true</code> for retryable exceptions. Only used when <code>retryPolicy</code> is configured.

Parameter	Type	Default	Description
logWarning	boolean	true	Whether to log a warning when a retry is attempted.

## Custom retry logic

### python

Pass a callable or exception type to `retry_on`. Import `default_retry_on` to extend the default behavior:

```
from langgraph.types import RetryPolicy, default_retry_on

def custom_retry_on(exc: BaseException) -> bool:
 if isinstance(exc, MyCustomError):
 return False
 return default_retry_on(exc)

builder.add_node(
 "call_api",
 call_api,
 retry_policy=RetryPolicy(max_attempts=3, retry_on=custom_retry_on),
)
```

### js

Pass a callable to `retryOn`. Unlike Python, there is no exported `defaultRetryOn` helper—implement your own predicate:

```
import { StateGraph } from "@langchain/langgraph";

class MyCustomError extends Error {}

const graph = new StateGraph(State)
 .addNode("callApi", callApi, {
 retryPolicy: {
 maxAttempts: 3,
 retryOn: (error: unknown) => {
 if (error instanceof MyCustomError) return false;
 // Retry on other errors
 return true;
 },
 },
 },
);
```

```
})
.compile();
```

## Inspect retry state

Use execution info inside a node to inspect the current attempt number. This is useful for switching to a fallback when the primary call keeps failing:

### python

```
from langgraph.graph import StateGraph, START, END
from langgraph.runtime import Runtime
from langgraph.types import RetryPolicy
from typing_extensions import TypedDict

class State(TypedDict):
 result: str

def my_node(state: State, runtime: Runtime) -> State:
 if runtime.execution_info.node_attempt > 1: # [!code highlight]
 return {"result": call_fallback_api()}
 return {"result": call_primary_api()}

builder = StateGraph(State)
builder.add_node("my_node", my_node, retry_policy=RetryPolicy(max_attempts=3))
builder.add_edge(START, "my_node")
builder.add_edge("my_node", END)
```

`execution_info` exposes the following fields:

Attribute	Type	Description
<code>node_attempt</code>	<code>int</code>	Current attempt number (1-indexed). 1 on the first try, 2 on the first retry, etc.
<code>node_first_attempt_time</code>	<code>float</code> \   <code>None</code>	Unix timestamp of when the first attempt started. Constant across retries.
<code>thread_id</code>	<code>str</code> \   <code>None</code>	Thread ID for the current execution. <code>None</code> without a checkpoint.
<code>run_id</code>	<code>str</code> \   <code>None</code>	Run ID for the current execution. <code>None</code> when not provided in config.
<code>checkpoint_id</code>	<code>str</code>	Checkpoint ID for the current execution.
<code>task_id</code>	<code>str</code>	Task ID for the current execution.

`execution_info` is available even without a retry policy— `node_attempt` defaults to 1.

js

```
import { StateGraph, StateSchema, START, END, type Runtime } from "@langchain/langgraph";
import * as z from "zod";

const State = new StateSchema({
 result: z.string(),
});

const myNode = async (state: typeof State.State, runtime: Runtime<typeof State>) => {
 if ((runtime.executionInfo?.nodeAttempt ?? 1) > 1) { // [!code highlight]
 return { result: await callFallbackApi() };
 }
 return { result: await callPrimaryApi() };
};

const graph = new StateGraph(State)
 .addNode("myNode", myNode, { retryPolicy: { maxAttempts: 3 } })
 .addEdge(START, "myNode")
 .addEdge("myNode", END)
 .compile();
```

`executionInfo` exposes the following fields:

Attribute	Type	Description
<code>nodeAttempt</code>	number	Current attempt number (1-indexed). 1 on the first try, 2 on the first retry, etc.
<code>nodeFirstAttemptTime</code>	number \  undefined	Unix timestamp (ms) of when the first attempt started. Constant across retries.
<code>threadId</code>	string \  undefined	Thread ID for the current execution. undefined without a checkpoint.
<code>runId</code>	string \  undefined	Run ID for the current execution. undefined when not provided in config.
<code>checkpointId</code>	string	Checkpoint ID for the current execution.
<code>checkpointNs</code>	string	Checkpoint namespace for the current execution.
<code>taskId</code>	string	Task ID for the current execution.

`executionInfo` is available even without a retry policy—`nodeAttempt` defaults to 1.

## Timeouts

python

## 참고

Requires `langgraph>=1.2`.

The `timeout=` parameter on `@[ add_node ]` caps how long a single node attempt may run. Pass a number (seconds), a `timedelta`, or a `@[ TimeoutPolicy ]` for separate run and idle limits:

```
from datetime import timedelta
from langgraph.types import TimeoutPolicy

Simple wall-clock cap
builder.add_node("call_model", call_model, timeout=60)
builder.add_node("call_model", call_model, timeout=timedelta(minutes=2))

Separate run and idle limits
builder.add_node(
 "call_model",
 call_model,
 timeout=TimeoutPolicy(run_timeout=120, idle_timeout=30),
)
```

## 주의

Node timeouts only apply to **async** nodes. Sync nodes with a `timeout` are rejected at compile time. To wrap blocking I/O, use `asyncio.to_thread` inside an async node.

## js

## 참고

Requires `@langchain/langgraph>=1.4.0`.

The `timeout` parameter on `@[ addNode ]` caps how long a single node attempt may run. Pass a number (milliseconds) or a `@[ TimeoutPolicy ]` for separate run and idle limits:

```
import { StateGraph, type TimeoutPolicy } from "@langchain/langgraph";

// Simple wall-clock cap (60 seconds)
new StateGraph(State).addNode("callModel", callModel, { timeout: 60_000 });

// Separate run and idle limits
new StateGraph(State).addNode("callModel", callModel, {
```

```
 timeout: { runTimeout: 120_000, idleTimeout: 30_000 },
 });
```

## Run timeout

### python

`run_timeout` is a hard wall-clock cap on a single attempt. It is never refreshed, regardless of node activity:

```
from langgraph.types import TimeoutPolicy

builder.add_node(
 "call_model",
 call_model,
 timeout=TimeoutPolicy(run_timeout=120),
)
```

### js

`runTimeout` is a hard wall-clock cap on a single attempt. It is never refreshed, regardless of node activity:

```
const graph = new StateGraph(State)
 .addNode("callModel", callModel, {
 timeout: { runTimeout: 120_000 },
 })
 .compile();
```

When the limit is exceeded, LangGraph raises `@[NodeTimeoutError]`, clears any writes from the failed attempt, and lets the retry policy decide whether to retry.

## Idle timeout

### python

`idle_timeout` is a progress-resetting cap. It fires only when the node stops making observable progress for the specified duration—unlike `run_timeout`, the clock resets whenever the node produces a progress signal:

```
builder.add_node(
 "call_model",
 call_model,
```

```
 timeout=TimeoutPolicy(idle_timeout=30),
)
```

## js

`idleTimeout` is a progress-resetting cap. It fires only when the node stops making observable progress for the specified duration—unlike `runTimeout`, the clock resets whenever the node produces a progress signal:

```
const graph = new StateGraph(State)
 .addNode("callModel", callModel, {
 timeout: { idleTimeout: 30_000 },
 })
 .compile();
```

## python

You can set `run_timeout` and `idle_timeout` together. Whichever fires first cancels the attempt.

## js

You can set `runTimeout` and `idleTimeout` together. Whichever fires first cancels the attempt.

## Progress signals

### python

Under the default `refresh_on="auto"`, the idle clock resets on any of the following:

- State writes via `CONFIG_KEY_SEND`
- Stream output (yielded async stream chunks)
- Child-task scheduling
- Runtime stream-writer calls
- Any LangChain callback event from the node or its descendants (LLM tokens, tool calls, chain start/end, etc.)

## js

Under the default `refreshOn: "auto"`, the idle clock resets on any of the following:

- State writes through the graph write path
- Custom stream output via `runtime.writer`
- Child-task scheduling



- Any LangChain callback event from the node or its descendants (LLM tokens, tool calls, chain start/end, etc.)

## Heartbeat mode

### python

Set `refresh_on="heartbeat"` to narrow the refresh source to explicit `runtime.heartbeat()` calls only. This is useful when you want a strict idle definition that isn't reset by chatty subordinates:

```
builder.add_node(
 "call_model",
 call_model,
 timeout=TimeoutPolicy(idle_timeout=30, refresh_on="heartbeat"),
)
```

### js

Set `refreshOn: "heartbeat"` to narrow the refresh source to explicit `runtime.heartbeat()` calls only. This is useful when you want a strict idle definition that isn't reset by chatty subordinates:

```
const graph = new StateGraph(State)
 .addNode("callModel", callModel, {
 timeout: { idleTimeout: 30_000, refreshOn: "heartbeat" },
 })
.compile();
```

## Manual heartbeats

For long-running work that doesn't naturally emit progress signals, call `runtime.heartbeat()` to manually reset the idle clock:

### python

```
from langgraph.graph import StateGraph, START, END
from langgraph.runtime import Runtime
from langgraph.types import TimeoutPolicy
from typing_extensions import TypedDict

class State(TypedDict):
 result: str

async def long_running_node(state: State, runtime: Runtime) -> State:
 for batch in fetch_batches():
```

```

 process(batch)
 runtime.heartbeat() # [!code highlight]
 return {"result": "done"}

builder = StateGraph(State)
builder.add_node(
 "long_running_node",
 long_running_node,
 timeout=TimeoutPolicy(idle_timeout=30, refresh_on="heartbeat"),
)
builder.add_edge(START, "long_running_node")
builder.add_edge("long_running_node", END)

```

js

```

import {
 StateGraph,
 StateSchema,
 START,
 END,
 type Runtime,
} from "@langchain/langgraph";
import * as z from "zod";

const State = new StateSchema({
 result: z.string(),
});

const longRunningNode = async (
 state: typeof State.State,
 runtime: Runtime<typeof State>
) => {
 for (const batch of fetchBatches()) {
 process(batch);
 runtime.heartbeat?.(); // [!code highlight]
 }
 return { result: "done" };
};

const graph = new StateGraph(State)
 .addNode("longRunningNode", longRunningNode, {
 timeout: { idleTimeout: 30_000, refreshOn: "heartbeat" },
 })
 .addEdge(START, "longRunningNode")
 .addEdge("longRunningNode", END)
 .compile();

```

`runtime.heartbeat()` is a no-op outside an idle-timed attempt, so you can call it unconditionally.

## NodeTimeoutError

When a timeout fires, LangGraph raises `@[NodeTimeoutError]` with structured context about which limit was hit:

### python

Attribute	Type	Description
node	str	Name of the node whose execution timed out.
elapsed	float	Seconds elapsed before the timeout fired.
kind	Literal["idle", "run"]	Which timeout fired.
idle_timeout	float \   None	The configured idle timeout (seconds), if any.
run_timeout	float \   None	The configured run timeout (seconds), if any.

### js

Attribute	Type	Description
node	string	Name of the node whose execution timed out.
elapsed	number	Milliseconds elapsed before the timeout fired.
kind	"idle" \   "run"	Which timeout fired.
timeout	number	The value (ms) of the timeout that fired.
idleTimeout	number \   undefined	The configured idle timeout (milliseconds), if any.
runTimeout	number \   undefined	The configured run timeout (milliseconds), if any.

Use `isNodeTimeoutError(error)` to narrow caught errors in TypeScript.

`NodeTimeoutError` is retryable by default. Combining `timeout` with a retry policy works out of the box—the timeout clock resets on each new attempt, and writes from a timed-out attempt are cleared before the next retry:

### python

```
from langgraph.types import RetryPolicy, TimeoutPolicy

builder.add_node(
 "call_model",
 call_model,
 timeout=TimeoutPolicy(idle_timeout=30),
 retry_policy=RetryPolicy(max_attempts=3),
)
```

js

```
const graph = new StateGraph(State)
 .addNode("callModel", callModel, {
 timeout: { idleTimeout: 30_000 },
 retryPolicy: { maxAttempts: 3 },
 })
 .compile();
```

## Dynamic timeouts with Send

When using `@[ send ]` to dispatch nodes dynamically (for example, in map-reduce patterns), you can pass a timeout directly on the `send` to override the target node's static timeout for that specific push:

python

```
from langgraph.types import Send, TimeoutPolicy

def fan_out(state: OverallState):
 return [
 Send("process_item", {"item": item}, timeout=TimeoutPolicy(idle_timeout=15))
 for item in state["items"]
]
```

js

```
import { Send } from "@langchain/langgraph";

const fanOut = (state: typeof State.State) =>
 state.items.map(
 (item) =>
 new Send("processItem", { item }, { timeout: { idleTimeout: 15_000 } })
);
```

python

If the timeout is omitted on the `send`, the target node's timeout (set at `@[ add_node ]` time) applies. This lets you set a default timeout on the node and tighten it for individual calls.

js

If the timeout is omitted on the `send`, the target node's timeout (set at `@[ addNode ]` time) applies. This lets you set a default timeout on the node and tighten it for individual calls.

## Error handling

### python

#### 참고

Requires `langgraph>=1.2`.

### js

#### 참고

Requires `@langchain/langgraph>=1.4.0`.

An error handler runs after a node fails and all retries are exhausted. It receives the current state and can update it or route to a different node using `@[ Command ]`. This is useful for compensation flows (Saga patterns) where you want to recover gracefully rather than abort the entire graph.

### python

Pass `error_handler=` to `@[ add_node ]`:

```
from langgraph.errors import NodeError
from langgraph.types import Command, RetryPolicy
from langgraph.graph import StateGraph, START
from typing_extensions import TypedDict

class State(TypedDict):
 status: str

def charge_payment(state: State) -> State:
 raise RuntimeError("payment gateway timeout")

def payment_error_handler(state: State, error: NodeError) -> Command:
 return Command(
 update={"status": f"compensated: {error.error}"},
 goto="finalize",
)

def finalize(state: State) -> State:
 return state
```

```

graph = (
 StateGraph(State)
 .add_node(
 "charge_payment",
 charge_payment,
 retry_policy=RetryPolicy(max_attempts=3, retry_on=ConnectionError),
 error_handler=payment_error_handler,
)
 .add_node("finalize", finalize)
 .add_edge(START, "charge_payment")
 .compile()
)

```

js

Pass `errorHandler` to `@[addNode]` on `@[StateGraph]` only (not the base `Graph` class):

```

import {
 StateGraph,
 StateSchema,
 START,
 Command,
 NodeError,
} from "@langchain/langgraph";
import * as z from "zod";

class ConnectionError extends Error {}

const State = new StateSchema({
 status: z.string(),
});

const chargePayment = () => {
 throw new Error("payment gateway timeout");
};

const paymentErrorHandler = (
 state: typeof State.State,
 error: NodeError
) =>
 new Command({
 update: { status: `compensated: ${error.error.message}` },
 goto: "finalize",
 });

const finalize = (state: typeof State.State) => state;

const graph = new StateGraph(State)

```

```

.addNode("chargePayment", chargePayment, {
 retryPolicy: {
 maxAttempts: 3,
 retryOn: (err) => err instanceof ConnectionError,
 },
 errorHandler: paymentErrorHandler,
})
.addNode("finalize", finalize)
.addEdge(START, "chargePayment")
.compile();

```

The handler fires only after the retry policy is exhausted, or immediately if no retry policy is configured. The retry policy and the error handler stay decoupled: configure when to retry and when to compensate independently.

## NodeError

### python

Error handlers receive failure context through a typed `error: NodeError` parameter, injected by type annotation (the same pattern as `runtime: Runtime`):

```

from langgraph.errors import NodeError

def my_handler(state: State, error: NodeError) -> Command:
 print(f"Node {error.node} failed with: {error.error}")
 return Command(update={"status": "recovered"}, goto="next_step")

```

`@[NodeError]` is a frozen dataclass with two fields:

Attribute	Type	Description
<code>node</code>	<code>str</code>	Name of the node whose execution failed.
<code>error</code>	<code>BaseException</code>	The exception raised by the failed node.

The `error: NodeError` parameter is opt-in. Handlers that don't need failure context can use simpler signatures like `(state)` or `(state, runtime)`.

### js

Error handlers receive failure context through a typed `error: NodeError` parameter:

```

import { Command, NodeError } from "@langchain/langgraph";

const myHandler = (state: typeof State.State, error: NodeError) => {
 console.log(`Node ${error.node} failed with: ${error.error.message}`);

```

```

return new Command({
 update: { status: "recovered" },
 goto: "nextStep",
});
};

```

@[ `NodeError` ] is a class with two fields:

Attribute	Type	Description
node	string	Name of the node whose execution failed.
error	Error	The exception thrown by the failed node.

The `error: NodeError` parameter is opt-in. Handlers that don't need failure context can omit the second argument and accept only `state`.

## Route with Command

Error handlers can return a @[ `Command` ] to update state and route to a specific node, enabling Saga / compensation patterns:

### python

```

from langgraph.errors import NodeError
from langgraph.types import Command, RetryPolicy
from langgraph.graph import StateGraph, START
from typing_extensions import TypedDict

class State(TypedDict):
 status: str

def reserve_inventory(state: State) -> State:
 return {"status": "reserved"}

def charge_payment(state: State) -> State:
 raise RuntimeError("payment timeout")

def payment_error_handler(state: State, error: NodeError) -> Command:
 return Command(
 update={"status": f"compensated_after_{error.node}": {error.error}},
 goto="finalize",
)

def finalize(state: State) -> State:
 return state

graph = (
 StateGraph(State)

```



```

.add_node("reserve_inventory", reserve_inventory)
.add_node(
 "charge_payment",
 charge_payment,
 retry_policy=RetryPolicy(max_attempts=3, retry_on=ConnectionError),
 error_handler=payment_error_handler,
)
.add_node("finalize", finalize)
.add_edge(START, "reserve_inventory")
.add_edge("reserve_inventory", "charge_payment")
.compile()
)

```

`charge_payment` retries on `ConnectionError` up to 3 times. If retries are exhausted (or the error isn't a `ConnectionError`), the handler compensates by updating state and routing to `finalize` instead of aborting the graph.

js

```

import {
 StateGraph,
 StateSchema,
 START,
 Command,
 NodeError,
} from "@langchain/langgraph";
import * as z from "zod";

class ConnectionError extends Error {}

const State = new StateSchema({
 status: z.string(),
});

const reserveInventory = () => ({ status: "reserved" });

const chargePayment = () => {
 throw new Error("payment timeout");
};

const paymentErrorHandler = (
 state: typeof State.State,
 error: NodeError
) =>
 new Command({
 update: {
 status: `compensated_after_${error.node}: ${error.error.message}`,
 },
 goto: "finalize",
 });

```

```
});

const finalize = (state: typeof State.State) => state;

const graph = new StateGraph(State)
 .addNode("reserveInventory", reserveInventory)
 .addNode("chargePayment", chargePayment, {
 retryPolicy: {
 maxAttempts: 3,
 retryOn: (err) => err instanceof ConnectionError,
 },
 errorHandler: paymentErrorHandler,
 })
 .addNode("finalize", finalize)
 .addEdge(START, "reserveInventory")
 .addEdge("reserveInventory", "chargePayment")
 .compile();
```

`chargePayment` retries on `ConnectionError` up to 3 times. If retries are exhausted (or the error isn't a `ConnectionError`), the handler compensates by updating state and routing to `finalize` instead of aborting the graph.

## Resume-safe failures

### 참고

Failure provenance is checkpointed. If the graph is interrupted or the process crashes after a node fails but before the handler completes, the handler sees the same `NodeError` context when the graph resumes from its checkpoint.

## Behavior with `interrupt()`

### 주의

`interrupt()` raised inside a node is **not** routed to the error handler. Interrupts use the `GraphBubbleUp` mechanism to pause graph execution for human-in-the-loop workflows, bypassing both retry policies and error handlers. The graph pauses as usual.

## Subgraph failures

If a node wraps a subgraph and the subgraph raises an unhandled exception, that exception surfaces to the parent node. If the parent node has an error handler, the handler fires with the subgraph's exception in `error.error`.

## Graph defaults

### 참고

Requires `langgraph>=1.2`.

Instead of repeating the same `retry_policy=`, `error_handler=`, `timeout=`, or `cache_policy=` on every `add_node` call, use `@[ set_node_defaults ]` to configure graph-wide defaults in one place:

```
from langgraph.errors import NodeError
from langgraph.types import RetryPolicy, TimeoutPolicy
from langgraph.graph import StateGraph, START
from typing_extensions import TypedDict

class State(TypedDict):
 status: str

def default_error_handler(state: State, error: NodeError) -> State:
 return {"status": f"handled: {error.error}"}

graph = (
 StateGraph(State)
 .set_node_defaults(
 retry_policy=RetryPolicy(max_attempts=3),
 error_handler=default_error_handler,
 timeout=TimeoutPolicy(run_timeout=30),
)
 .add_node("step_a", step_a)
 .add_node("step_b", step_b)
 .add_edge(START, "step_a")
 .compile()
)
```

Both `step_a` and `step_b` now share the same retry policy, error handler, and timeout without any duplication.

### Precedence

Per-node values passed directly to `add_node()` always override the defaults set by `set_node_defaults()`. Defaults are resolved at `compile()` time, so you can call `set_node_defaults()` before or after `add_node()` in any order:

```
graph = (
 StateGraph(State)
```

```

 .set_node_defaults(error_handler=default_error_handler)
 .add_node("step_a", step_a) # uses default_error_handler
 .add_node("step_b", step_b, error_handler=custom_error_handler) # uses custom_error_handler
 .add_edge(START, "step_a")
 .compile()
)

```

## Default error handler

The `error_handler` default is particularly valuable when every graph run maps to an external process (for example a background job row) and any unhandled node failure should mark that process as failed, without repeating `error_handler=` on every `add_node`. Per-node handlers still take precedence when a step needs its own logic:

```

from langgraph.errors import NodeError
from langgraph.graph import StateGraph, START
from langgraph.types import Command, RetryPolicy
from typing_extensions import TypedDict

class State(TypedDict):
 process_id: str
 status: str

def fetch_data(state: State) -> State:
 return {"status": "fetched"}

def charge_payment(state: State) -> State:
 raise RuntimeError("payment timeout")

def finalize(state: State) -> State:
 return state

def mark_process_failed(state: State, error: NodeError) -> State:
 # Persist failure on the external process row keyed by process_id.
 return {"status": f"failed at {error.node}: {error.error}"}

def refund_payment(state: State, error: NodeError) -> Command:
 return Command(
 update={"status": f"compensated after {error.node}"},
 goto="finalize",
)

graph = (
 StateGraph(State)
 .set_node_defaults(
 retry_policy=RetryPolicy(max_attempts=3),
 error_handler=mark_process_failed,
)
)

```

```

)
.add_node("fetch_data", fetch_data) # uses mark_process_failed
.add_node(
 "charge_payment",
 charge_payment,
 error_handler=refund_payment, # overrides the graph-wide default
)
.add_node("finalize", finalize)
.add_edge(START, "fetch_data")
.add_edge("fetch_data", "charge_payment")
.compile()
)

```

If `fetch_data` fails after retries, `mark_process_failed` runs. If `charge_payment` fails after retries, `refund_payment` runs instead because the per-node handler overrides the default.

The handler accepts the same `(state, error: NodeError)` signature described in [Error handling](#). It also accepts `RunnableConfig` as an optional third argument if you need access to config values such as `thread_id`:

```

from langchain_core.runnables import RunnableConfig

def mark_process_failed(
 state: State, error: NodeError, config: RunnableConfig
) -> State:
 thread_id = config["configurable"].get("thread_id")
 return {"status": f"failed on thread {thread_id}: {error.error}"}

```

## Applicability matrix

Not all defaults apply to all node types. Error-handler nodes (those registered via `add_node(error_handler=...)`) are excluded from certain defaults to prevent unsafe behavior:

set_node_defaults parameter	Applies to regular nodes	Applies to error-handler nodes	Reason
retry_policy	✓	✓	Handlers should be retried on transient failures
timeout	✓	✓	Stuck handlers should be cancelled like stuck regular nodes
error_handler	✓	✗	Handlers must never catch themselves
cache_policy	✓	✗	Caching handler results is unsafe

## Scope

Defaults set on a parent graph are **not** inherited by subgraphs. Each graph maintains its own defaults.

js

## Graph defaults

### 참고

Requires `@langchain/langgraph>=1.4.0`.

Instead of repeating the same `retryPolicy`, `errorHandler`, `timeout`, or `cachePolicy` on every `addNode` call, use [setNodeDefaults](#) to configure graph-wide defaults in one place:

```
import { StateGraph, START, NodeError } from "@langchain/langgraph";

const defaultErrorHandler = (
 state: typeof State.State,
 error: NodeError
) => ({ status: `handled: ${error.error.message}` });

const graph = new StateGraph(State)
 .setNodeDefaults({
 retryPolicy: { maxAttempts: 3 },
 errorHandler: defaultErrorHandler,
 timeout: { runTimeout: 30_000 },
 cachePolicy: { ttl: 60 },
 })
 .addNode("stepA", stepA)
 .addNode("stepB", stepB)
 .addEdge(START, "stepA")
 .compile();
```

Both `stepA` and `stepB` now share the same retry policy, error handler, timeout, and cache policy without any duplication.

## Precedence

Per-node values passed directly to `addNode()` always override defaults set by `setNodeDefaults()`. Defaults are resolved at `compile()` time, so you can call `setNodeDefaults()` before or after `addNode()` in any order:

```
import { StateGraph, START } from "@langchain/langgraph";

const graph = new StateGraph(State)
 .setNodeDefaults({ errorHandler: defaultErrorHandler })
 .addNode("stepA", stepA) // uses defaultErrorHandler
 .addNode("stepB", stepB, { errorHandler: customErrorHandler }) // overrides default
 .addEdge(START, "stepA")
 .compile();
```

## Default error handler

The `errorHandler` default is particularly valuable when every graph run maps to an external process (for example a background job row) and any unhandled node failure should mark that process as failed, without repeating `errorHandler` on every `addNode`. Per-node handlers still take precedence when a step needs its own compensation logic:

```
import { Command, NodeError, StateGraph, START } from "@langchain/langgraph";

const markProcessFailed = (
 state: typeof State.State,
 error: NodeError
) => {
 // Persist failure on the external process row keyed by processId.
 return { status: `failed at ${error.node}: ${error.error.message}` };
};

const refundPayment = (state: typeof State.State, error: NodeError) =>
 new Command({
 update: { status: `compensated after ${error.node}` },
 goto: "finalize",
 });

const graph = new StateGraph(State)
 .setNodeDefaults({
 retryPolicy: { maxAttempts: 3 },
 errorHandler: markProcessFailed,
 })
 .addNode("fetchData", fetchData) // uses markProcessFailed
 .addNode("chargePayment", chargePayment, {
 errorHandler: refundPayment, // overrides the graph-wide default
 })
 .addNode("finalize", finalize)
 .addEdge(START, "fetchData")
 .addEdge("fetchData", "chargePayment")
 .compile();
```

If `fetchData` fails after retries, `markProcessFailed` runs. If `chargePayment` fails after retries, `refundPayment` runs instead because the per-node handler overrides the default.

## Applicability matrix

Not all defaults apply to all node types. Error-handler nodes (those registered via `addNode(..., { errorHandler })`) are excluded from certain defaults to prevent unsafe behavior:

setNodeDefaults parameter	Applies to regular nodes	Applies to error-handler nodes	Reason
retryPolicy	✓	✓	Handlers should be retried on transient failures
timeout	✓	✓	Stuck handlers should be cancelled like stuck regular nodes
errorHandler	✓	✗	Handlers must never catch themselves
cachePolicy	✓	✗	Caching handler results is unsafe

## Scope

Defaults set on a parent graph are **not** inherited by subgraphs. Each graph maintains its own defaults.

## Functional API

### python

The same `timeout=` and `retry_policy=` parameters are available on `@task` and `@entrypoint` in the functional API:

```
from langgraph.func import entrypoint, task
from langgraph.types import RetryPolicy, TimeoutPolicy

@task(
 timeout=TimeoutPolicy(idle_timeout=30),
 retry_policy=RetryPolicy(max_attempts=3),
)
async def call_api(url: str) -> str:
 response = await fetch(url)
 return response.text

@entrypoint(timeout=60)
async def my_workflow(inputs: dict) -> str:
 result = await call_api("https://api.example.com/data")
 return result
```



The behavior is identical to `add_node`: `NodeTimeoutError` is raised on timeout, buffered writes are cleared, and the retry policy decides whether to retry.

## js

The `timeout` option is available on `task` and `entrypoint`; `task` also accepts a `retry` option (not `retryPolicy`):

```
import { entrypoint, task } from "@langchain/langgraph";

const callApi = task(
 {
 name: "callApi",
 timeout: { idleTimeout: 30_000 },
 retry: { maxAttempts: 3 },
 },
 async (url: string) => {
 const response = await fetch(url);
 return response.text();
 }
);

const myWorkflow = entrypoint(
 { name: "myWorkflow", timeout: 60_000 },
 async (inputs: { url: string }) => {
 return await callApi(inputs.url);
 }
);
```

The behavior matches `addNode`: `NodeTimeoutError` is raised on timeout, buffered writes are cleared, and the retry policy decides whether to retry. Error handlers are not available on `task` / `entrypoint` in the JavaScript/TypeScript SDK—use `StateGraph.addNode(..., { errorHandler })` instead.

## Graceful shutdown

Cooperative shutdown lets you stop an in-flight graph run after the current superstep completes and save a resumable checkpoint. This is useful for handling SIGTERM signals or any external supervisor that needs to reclaim resources without losing work.

## python

### 참고

Requires `langgraph>=1.2`.

Create a `@[RunControl]` and pass it as `control=` to `invoke` or `stream`. Call `request_drain()` from any thread to signal that the run should stop:

```
from langgraph.runtime import RunControl
from langgraph.errors import GraphDrained

control = RunControl()

In a signal handler or supervisor:
control.request_drain("sigterm")

try:
 result = graph.invoke(inputs, config, control=control)
except GraphDrained as e:
 # The graph stopped early and saved a checkpoint.
 # Resume later with the same config.
 print(f"Drained: {e.reason}")
```

js

## 참고

Requires `@langchain/langgraph>=1.4.0`.

Create a `@[RunControl]` and pass it as `control` to `invoke` or `stream`. Call `requestDrain()` from any context to signal that the run should stop:

```
import { RunControl, GraphDrained } from "@langchain/langgraph";

const control = new RunControl();

// In a signal handler or supervisor:
// control.requestDrain("sigterm");

try {
 const result = await graph.invoke(inputs, { ...config, control });
} catch (e) {
 if (e instanceof GraphDrained) {
 // The graph stopped early and saved a checkpoint.
 // Resume later with the same config.
 console.log(`Drained: ${e.reason}`);
 } else {
 throw e;
 }
}
```

## Semantics

Drain is cooperative and operates between supersteps, never preempting work that is already running:

### python

Scenario	Behavior
Node mid-execution	Runs to completion. Drain takes effect on the next superstep.
Node with a retry policy currently retrying	Retry loop runs to exhaustion or success. Drain takes effect after.
Graph finishes naturally on the same tick as drain	Returns normally. Inspect <code>control.drain_requested</code> to distinguish from a normal run.
More supersteps remain	Raises <code>GraphDrained(reason)</code> . Checkpoint is saved and resumable.
Subgraph requests drain	<code>GraphDrained</code> bubbles up through the parent and stops it at its own next superstep boundary.

### js

Scenario	Behavior
Node mid-execution	Runs to completion. Drain takes effect on the next superstep.
Node with a retry policy currently retrying	Retry loop runs to exhaustion or success. Drain takes effect after.
Graph finishes naturally on the same tick as drain	Returns normally. Inspect <code>control.drainRequested</code> to distinguish from a normal run.
More supersteps remain	Raises <code>GraphDrained(reason)</code> . Checkpoint is saved and resumable.
Subgraph requests drain	<code>GraphDrained</code> bubbles up through the parent and stops it at its own next superstep boundary.

## Resume after drain

### python

Resume a drained run with `invoke(None, config)` using the same `thread_id`:

```
result = graph.invoke(None, config)
```

### js

Resume a drained run with `invoke(null, config)` using the same `thread_id`:

```
const result = await graph.invoke(null, config);
```

## Read drain state inside a node

Access drain state through the `runtime` parameter to adjust node behavior before the superstep boundary is reached:

### python

```
from langgraph.runtime import Runtime

async def my_node(state: State, runtime: Runtime) -> State:
 if runtime.drain_requested:
 # Skip expensive work and return a minimal result
 return {"status": "skipped", "reason": runtime.drain_reason}
 return {"status": await do_work() }
```

### js

```
import { type Runtime } from "@langchain/langgraph";

const myNode = async (state: typeof State.State, runtime: Runtime<typeof State>) => {
 if (runtime.control?.drainRequested) {
 // Skip expensive work and return a minimal result
 return { status: "skipped", reason: runtime.control.drainReason };
 }
 return { status: await doWork() };
};
```

## SIGTERM hook pattern

The recommended pattern for handling process shutdown:

### python

```
import signal
from langgraph.runtime import RunControl
from langgraph.errors import GraphDrained

control = RunControl()
signal.signal(signal.SIGTERM, lambda *_: control.request_drain("sigterm"))

try:
 result = graph.invoke(inputs, config, control=control)
```

```
except GraphDrained as e:
 log.info("graph drained: %s", e.reason)
 # Resume on next startup with the same config
```

## 참고

`request_drain()` does not cancel running asyncio tasks or kill threads. For a hard upper bound, pair drain with a graceful timeout and task cancellation.

## js

```
import process from "node:process";
import { RunControl, GraphDrained } from "@langchain/langgraph";

const control = new RunControl();
process.on("SIGTERM", () => control.requestDrain("sigterm"));

try {
 const result = await graph.invoke(inputs, { ...config, control });
} catch (e) {
 if (e instanceof GraphDrained) {
 console.log(`graph drained: ${e.reason}`);
 // Resume on next startup with the same config
 } else {
 throw e;
 }
}
```

## js

## 참고

`requestDrain()` does not cancel in-flight async work. For a hard upper bound, pair drain with a graceful timeout and an `AbortSignal`.

# Limitations

## python

- **Timeouts are async-only:** sync nodes with a `timeout` are rejected at compile time.
- **One handler per node:** each node can have at most one `error_handler`.
- **Handler failures bubble up:** if the error handler itself raises, that exception propagates as if the node had no handler.

- `set_node_defaults` **is not inherited by subgraphs**: each graph manages its own defaults independently.

js

- `setNodeDefaults` **is not inherited by subgraphs**: each graph manages its own defaults independently.
- **Error handlers are StateGraph -only**: pass `errorHandler` to `StateGraph.addNode`, not the base `Graph` class. Error handlers are not available on `task` / `entrypoint`.
- **One handler per node**: each node can have at most one `errorHandler`.
- **Handler failures bubble up**: if the error handler itself throws, that exception propagates as if the node had no handler.

# Functional API overview

---

langgraph/functional-api.mdx

The **Functional API** allows you to add LangGraph's key features ([persistence](#), [memory](#), [human-in-the-loop](#), and [streaming](#)) to your applications with minimal changes to your existing code.

It is designed to integrate these features into existing code that may use standard language primitives for branching and control flow, such as `if` statements, `for` loops, and function calls. Unlike many data orchestration frameworks that require restructuring code into an explicit pipeline or DAG, the Functional API allows you to incorporate these capabilities without enforcing a rigid execution model.

The Functional API uses two key building blocks:

## python

- **@entrypoint** : Marks a function as the starting point of a workflow, encapsulating logic and managing execution flow, including handling long-running tasks and interrupts.
- **@task** : Represents a discrete unit of work, such as an API call or data processing step, that can be executed asynchronously within an entrypoint. Tasks return a future-like object that can be awaited or resolved synchronously.

## js

- **entrypoint** : An entrypoint encapsulates workflow logic and manages execution flow, including handling long-running tasks and interrupts.
- **task** : Represents a discrete unit of work, such as an API call or data processing step, that can be executed asynchronously within an entrypoint. Tasks return a future-like object that can be awaited or resolved synchronously.

This provides a minimal abstraction for building workflows with state management and streaming.

## 팁

For information on how to use the functional API, see [Use Functional API](#).

## Functional API vs. Graph API

---

For users who prefer a more declarative approach, LangGraph's [Graph API](#) allows you to define workflows using a Graph paradigm. Both APIs share the same underlying runtime, so you can use them together in the same application.

Here are some key differences:

- **Control flow:** The Functional API does not require thinking about graph structure. You can use standard Python constructs to define workflows. This will usually trim the amount of code you need to write.
- **Short-term memory:** The **GraphAPI** requires declaring a [State](#) and may require defining [reducers](#) to manage updates to the graph state. `@entrypoint` and `@tasks` do not require explicit state management as their state is scoped to the function and is not shared across functions.
- **Checkpointing:** Both APIs generate and use checkpoints. In the **Graph API** a new checkpoint is generated after every [superstep](#). In the **Functional API**, when tasks are executed, their results are saved to an existing checkpoint associated with the given entrypoint instead of creating a new checkpoint.
- **Visualization:** The Graph API makes it easy to visualize the workflow as a graph which can be useful for debugging, understanding the workflow, and sharing with others. The Functional API does not support visualization as the graph is dynamically generated during runtime.

## Example

---

Below we demonstrate a simple application that writes an essay and [interrupts](#) to request human review.

### python

```
from langgraph.checkpoint.memory import InMemorySaver
from langgraph.func import entrypoint, task
from langgraph.types import interrupt

@task
def write_essay(topic: str) -> str:
 """Write an essay about the given topic."""
 time.sleep(1) # A placeholder for a long-running task.
 return f"An essay about topic:{topic}"

@entrypoint(checkpointer=InMemorySaver())
def workflow(topic: str) -> dict:
 """A simple workflow that writes an essay and asks for a review."""
 essay = write_essay("cat").result()
 is_approved = interrupt({
 # Any json-serializable payload provided to interrupt as argument.
 # It will be surfaced on the client side as an Interrupt when streaming data
 # from the workflow.
 "essay": essay, # The essay we want reviewed.
 # We can add any additional information that we need.
 # For example, introduce a key called "action" with some instructions.
 "action": "Please approve/reject the essay",
 })
```



```

return {
 "essay": essay, # The essay that was generated
 "is_approved": is_approved, # Response from HIL
}

```

js

```

import { MemorySaver, entrypoint, task, interrupt } from "@langchain/langgraph";

const writeEssay = task("writeEssay", async (topic: string) => {
 // A placeholder for a long-running task.
 await new Promise((resolve) => setTimeout(resolve, 1000));
 return `An essay about topic: ${topic}`;
});

const workflow = entrypoint(
 { checkpointer: new MemorySaver(), name: "workflow" },
 async (topic: string) => {
 const essay = await writeEssay(topic);
 const isApproved = interrupt({
 // Any json-serializable payload provided to interrupt as argument.
 // It will be surfaced on the client side as an Interrupt when streaming data
 // from the workflow.
 essay, // The essay we want reviewed.
 // We can add any additional information that we need.
 // For example, introduce a key called "action" with some instructions.
 action: "Please approve/reject the essay",
 });

 return {
 essay, // The essay that was generated
 isApproved, // Response from HIL
 };
 }
);

```

## Detailed Explanation

This workflow will write an essay about the topic "cat" and then pause to get a review from a human. The workflow can be interrupted for an indefinite amount of time until a review is provided.

When the workflow is resumed, it executes from the very start, but because the result of the `writeEssay` task was already saved, the task result will be loaded from the checkpoint instead of being recomputed.

## python

An essay has been written and is ready for review. Once the review is provided, we can resume the workflow:

The workflow has been completed and the review has been added to the essay.

## js

An essay has been written and is ready for review. Once the review is provided, we can resume the workflow:

The workflow has been completed and the review has been added to the essay.

# Entrypoint

## python

The `@entrypoint` decorator can be used to create a workflow from a function. It encapsulates workflow logic and manages execution flow, including handling *long-running tasks* and [interrupts](#).

## js

The `entrypoint` function can be used to create a workflow from a function. It encapsulates workflow logic and manages execution flow, including handling *long-running tasks* and [interrupts](#).

# Definition

## python

An **entrypoint** is defined by decorating a function with the `@entrypoint` decorator.

The function **must accept a single positional argument**, which serves as the workflow input. If you need to pass multiple pieces of data, use a dictionary as the input type for the first argument.

Decorating a function with an `entrypoint` produces a `@Pregel[Pregel.stream]` instance which helps to manage the execution of the workflow (e.g., handles streaming, resumption, and checkpointing).

You will usually want to pass a **checkpoint** to the `@entrypoint` decorator to enable persistence and use features like **human-in-the-loop**.

## Sync

```
from langgraph.func import entrypoint

@entrypoint(checkpointer=checkpointer)
def my_workflow(some_input: dict) -> int:
 # some logic that may involve long-running tasks like API calls,
 # and may be interrupted for human-in-the-loop.
 ...
 return result
```

## Async

```
from langgraph.func import entrypoint

@entrypoint(checkpointer=checkpointer)
async def my_workflow(some_input: dict) -> int:
 # some logic that may involve long-running tasks like API calls,
 # and may be interrupted for human-in-the-loop
 ...
 return result
```

## js

An **entrypoint** is defined by calling the `entrypoint` function with configuration and a function.

The function **must accept a single positional argument**, which serves as the workflow input. If you need to pass multiple pieces of data, use an object as the input type for the first argument.

Creating an entrypoint with a function produces a workflow instance which helps to manage the execution of the workflow (e.g., handles streaming, resumption, and checkpointing).

You will often want to pass a **checkpoint** to the `entrypoint` function to enable persistence and use features like **human-in-the-loop**.

```
import { entrypoint } from "@langchain/langgraph";

const myWorkflow = entrypoint(
 { checkpointer, name: "workflow" },
 async (someInput: Record<string, any>): Promise<number> => {
 // some logic that may involve long-running tasks like API calls,
 // and may be interrupted for human-in-the-loop
 return result;
 }
);
```

```
}
);
```

## 주의

**Serialization** The **inputs** and **outputs** of entrypoints must be JSON-serializable to support checkpointing. Please see the [serialization](#) section for more details.

## python

### Injectable parameters

When declaring an `entrypoint`, you can request access to additional parameters that will be injected automatically at runtime. These parameters include:

Parameter	Description
<b>previous</b>	Access the state associated with the previous <code>checkpoint</code> for the given thread. See <a href="#">short-term-memory</a> .
<b>store</b>	An instance of <code>[BaseStore][langgraph.store.base.BaseStore]</code> . Useful for <a href="#">long-term memory</a> .
<b>writer</b>	Use to access the <code>StreamWriter</code> when working with Async Python < 3.11. See <a href="#">streaming with functional API for details</a> .
<b>config</b>	For accessing run time configuration. See <a href="#">RunnableConfig</a> for information.

## 주의

Declare the parameters with the appropriate name and type annotation.

### Requesting Injectable Parameters

```
from langchain_core.runnables import RunnableConfig
from langgraph.func import entrypoint
from langgraph.store.base import BaseStore
from langgraph.store.memory import InMemoryStore
from langgraph.checkpoint.memory import InMemorySaver
from langgraph.types import StreamWriter

in_memory_checkpointer = InMemorySaver(...)
in_memory_store = InMemoryStore(...) # An instance of InMemoryStore for long-term memory

@entrypoint(
 checkpointer=in_memory_checkpointer, # Specify the checkpointer
 store=in_memory_store # Specify the store
)
```

```
def my_workflow(
 some_input: dict, # The input (e.g., passed via `invoke`)
 *,
 previous: Any = None, # For short-term memory
 store: BaseStore, # For long-term memory
 writer: StreamWriter, # For streaming custom data
 config: RunnableConfig # For accessing the configuration passed to the entrypoint
) -> ...:
```

## Executing

### python

Using the [@entrypoint](#) yields a `@[Pregel][Pregel.stream]` object that can be executed using the `invoke`, `ainvoke`, `stream`, and `astream` methods.

### Invoke

```
config = {
 "configurable": {
 "thread_id": "some_thread_id"
 }
}
my_workflow.invoke(some_input, config) # Wait for the result synchronously
```

### Async Invoke

```
config = {
 "configurable": {
 "thread_id": "some_thread_id"
 }
}
await my_workflow.ainvoke(some_input, config) # Await result asynchronously
```

### Stream

```
config = {
 "configurable": {
 "thread_id": "some_thread_id"
 }
}

stream = my_workflow.stream_events(some_input, config, version="v3")
for message in stream.messages:
 for token in message.text:
 print(token, end="", flush=True)
```

## Async Stream

```
config = {
 "configurable": {
 "thread_id": "some_thread_id"
 }
}

stream = await my_workflow.astream_events(some_input, config, version="v3")
async for message in stream.messages:
 async for token in message.text:
 print(token, end="", flush=True)
```

## js

Using the [entrypoint](#) function will return an object that can be executed using the `invoke` and `stream` methods.

### Invoke

```
const config = {
 configurable: {
 thread_id: "some_thread_id"
 }
};
await myWorkflow.invoke(someInput, config); // Wait for the result
```

### Stream

```
const config = {
 configurable: {
 thread_id: "some_thread_id"
 }
};

const stream = await myWorkflow.streamEvents(someInput, config, { version: "v3" });
for await (const message of stream.messages) {
 for await (const token of message.text) {
 process.stdout.write(token);
 }
}
```

## Resuming

## python

Resuming an execution after an `@[interrupt][interrupt]` can be done by passing a **resume** value to the `@[Command]` primitive.

## Invoke

```
from langgraph.types import Command

config = {
 "configurable": {
 "thread_id": "some_thread_id"
 }
}

my_workflow.invoke(Command(resume=some_resume_value), config)
```

## Async Invoke

```
from langgraph.types import Command

config = {
 "configurable": {
 "thread_id": "some_thread_id"
 }
}

await my_workflow.ainvoke(Command(resume=some_resume_value), config)
```

## Stream

```
from langgraph.types import Command

config = {
 "configurable": {
 "thread_id": "some_thread_id"
 }
}

stream = my_workflow.stream_events(Command(resume=some_resume_value), config, version="v3")
for message in stream.messages:
 for token in message.text:
 print(token, end="", flush=True)
```

## Async Stream

```
from langgraph.types import Command

config = {
 "configurable": {
```

```

 "thread_id": "some_thread_id"
 }
}

stream = await my_workflow.astream_events(Command(resume=some_resume_value), config, version="v3")
async for message in stream.messages:
 async for token in message.text:
 print(token, end="", flush=True)

```

## js

Resuming an execution after an `@[interrupt][interrupt]` can be done by passing a **resume** value to the `@[Command]` primitive.

### Invoke

```

import { Command } from "@langchain/langgraph";

const config = {
 configurable: {
 thread_id: "some_thread_id"
 }
};

await myWorkflow.invoke(new Command({ resume: someResumeValue }), config);

```

### Stream

```

import { Command } from "@langchain/langgraph";

const config = {
 configurable: {
 thread_id: "some_thread_id"
 }
};

const stream = await myWorkflow.streamEvents(
 new Command({ resume: someResumableValue }),
 config,
 { version: "v3" },
);

for await (const message of stream.messages) {
 for await (const token of message.text) {
 process.stdout.write(token);
 }
}

```



```
}
}
```

## python

### Resuming after an error

To resume after an error, run the `entrypoint` with a `None` and the same **thread id** (config).

This assumes that the underlying **error** has been resolved and execution can proceed successfully.

### Invoke

```
config = {
 "configurable": {
 "thread_id": "some_thread_id"
 }
}

my_workflow.invoke(None, config)
```

### Async Invoke

```
config = {
 "configurable": {
 "thread_id": "some_thread_id"
 }
}

await my_workflow.ainvoke(None, config)
```

### Stream

```
config = {
 "configurable": {
 "thread_id": "some_thread_id"
 }
}

stream = my_workflow.stream_events(None, config, version="v3")
for message in stream.messages:
 for token in message.text:
 print(token, end="", flush=True)
```

## Async Stream

```
config = {
 "configurable": {
 "thread_id": "some_thread_id"
 }
}

stream = await my_workflow.astream_events(None, config, version="v3")
async for message in stream.messages:
 async for token in message.text:
 print(token, end="", flush=True)
```

## js

### Resuming after an error

To resume after an error, run the `entrypoint` with `null` and the same **thread id** (config).

This assumes that the underlying **error** has been resolved and execution can proceed successfully.

### Invoke

```
const config = {
 configurable: {
 thread_id: "some_thread_id"
 }
};

await myWorkflow.invoke(null, config);
```

### Stream

```
const config = {
 configurable: {
 thread_id: "some_thread_id"
 }
};

const stream = await myWorkflow.streamEvents(null, config, { version: "v3" });
for await (const message of stream.messages) {
 for await (const token of message.text) {
 process.stdout.write(token);
 }
}
```

## Short-term memory

When an `entrypoint` is defined with a `checkpointer`, it stores information between successive invocations on the same **thread id** in [checkpoints](#).

### python

This allows accessing the state from the previous invocation using the `previous` parameter.

By default, the `previous` parameter is the return value of the previous invocation.

```
@entrypoint(checkpointer=checkpointer)
def my_workflow(number: int, *, previous: Any = None) -> int:
 previous = previous or 0
 return number + previous

config = {
 "configurable": {
 "thread_id": "some_thread_id"
 }
}

my_workflow.invoke(1, config) # 1 (previous was None)
my_workflow.invoke(2, config) # 3 (previous was 1 from the previous invocation)
```

### js

This allows accessing the state from the previous invocation using the `getPreviousState` function.

By default, the `getPreviousState` function returns the return value of the previous invocation.

```
import { entrypoint, getPreviousState } from "@langchain/langgraph";

const myWorkflow = entrypoint(
 { checkpointer, name: "workflow" },
 async (number: number) => {
 const previous = getPreviousState<number>() ?? 0;
 return number + previous;
 }
);

const config = {
 configurable: {
 thread_id: "some_thread_id",
 },
};
```

```
await myWorkflow.invoke(1, config); // 1 (previous was undefined)
await myWorkflow.invoke(2, config); // 3 (previous was 1 from the previous invocation)
```

## entrypoint.final

### python

@[entrypoint.final][entrypoint.final] is a special primitive that can be returned from an entrypoint and allows **decoupling** the value that is **saved in the checkpoint** from the **return value of the entrypoint**.

The first value is the return value of the entrypoint, and the second value is the value that will be saved in the checkpoint. The type annotation is `entrypoint.final[return_type, save_type]`.

```
@entrypoint(checkpointer=checkpointer)
def my_workflow(number: int, *, previous: Any = None) -> entrypoint.final[int, int]:
 previous = previous or 0
 # This will return the previous value to the caller, saving
 # 2 * number to the checkpoint, which will be used in the next invocation
 # for the `previous` parameter.
 return entrypoint.final(value=previous, save=2 * number)

config = {
 "configurable": {
 "thread_id": "1"
 }
}

my_workflow.invoke(3, config) # 0 (previous was None)
my_workflow.invoke(1, config) # 6 (previous was 3 * 2 from the previous invocation)
```

### js

@[entrypoint.final][entrypoint.final] is a special primitive that can be returned from an entrypoint and allows **decoupling** the value that is **saved in the checkpoint** from the **return value of the entrypoint**.

The first value is the return value of the entrypoint, and the second value is the value that will be saved in the checkpoint.

```
import { entrypoint, getPreviousState } from "@langchain/langgraph";

const myWorkflow = entrypoint(
 { checkpointer, name: "workflow" },
 async (number: number) => {
 const previous = getPreviousState<number>() ?? 0;
```

```
// This will return the previous value to the caller, saving
// 2 * number to the checkpoint, which will be used in the next invocation
// for the `previous` parameter.
return entrypoint.final({
 value: previous,
 save: 2 * number,
});
}
);

const config = {
 configurable: {
 thread_id: "1",
 },
};

await myWorkflow.invoke(3, config); // 0 (previous was undefined)
await myWorkflow.invoke(1, config); // 6 (previous was 3 * 2 from the previous invocation)
```

## Task

A **task** represents a discrete unit of work, such as an API call or data processing step. It has two key characteristics:

- **Asynchronous Execution:** Tasks are designed to be executed asynchronously, allowing multiple operations to run concurrently without blocking.
- **Checkpointing:** Task results are saved to a checkpoint, enabling resumption of the workflow from the last saved state. (See [persistence](#) for more details).

## Definition

### python

Tasks are defined using the `@task` decorator, which wraps a regular Python function.

```
from langgraph.func import task

@task()
def slow_computation(input_value):
 # Simulate a long-running operation
 ...
 return result
```

### js

Tasks are defined using the `task` function, which wraps a regular function.

```
import { task } from "@langchain/langgraph";

const slowComputation = task("slowComputation", async (inputValue: any) => {
 // Simulate a long-running operation
 return result;
});
```

## 주의

**Serialization** The **outputs** of tasks must be JSON-serializable to support checkpointing.

## Execution

**Tasks** can only be called from within an **entrypoint**, another **task**, or a [state graph node](#).

Tasks *cannot* be called directly from the main application code.

### python

When you call a **task**, it returns *immediately* with a future object. A future is a placeholder for a result that will be available later.

To obtain the result of a **task**, you can either wait for it synchronously (using `result()`) or await it asynchronously (using `await`).

### Synchronous Invocation

```
@entrypoint(checkpointer=checkpointer)
def my_workflow(some_input: int) -> int:
 future = slow_computation(some_input)
 return future.result() # Wait for the result synchronously
```

### Asynchronous Invocation

```
@entrypoint(checkpointer=checkpointer)
async def my_workflow(some_input: int) -> int:
 return await slow_computation(some_input) # Await result asynchronously
```

### js

When you call a **task**, it returns a Promise that can be awaited.

```
const myWorkflow = entrypoint(
 { checkpointer, name: "workflow" },
 async (someInput: number): Promise<number> => {
```

```
 return await slowComputation(someInput);
 }
);
```

## When to use a task

---

**Tasks** are useful in the following scenarios:

- **Checkpointing**: When you need to save the result of a long-running operation to a checkpoint, so you don't need to recompute it when resuming the workflow.
- **Human-in-the-loop**: If you're building a workflow that requires human intervention, you **MUST** use **tasks** to encapsulate any randomness (e.g., API calls) to ensure that the workflow can be resumed correctly. See the [determinism](#) section for more details.
- **Parallel Execution**: For I/O-bound tasks, **tasks** enable parallel execution, allowing multiple operations to run concurrently without blocking (e.g., calling multiple APIs).
- **Observability**: Wrapping operations in **tasks** provides a way to track the progress of the workflow and monitor the execution of individual operations using [LangSmith](#).
- **Retryable Work**: When work needs to be retried to handle failures or inconsistencies, **tasks** provide a way to encapsulate and manage the retry logic.

## Serialization

---

There are two key aspects to serialization in LangGraph:

1. `entrypoint` inputs and outputs must be JSON-serializable.
2. `task` outputs must be JSON-serializable.

### python

These requirements are necessary for enabling checkpointing and workflow resumption. Use python primitives like dictionaries, lists, strings, numbers, and booleans to ensure that your inputs and outputs are serializable.

### js

These requirements are necessary for enabling checkpointing and workflow resumption. Use primitives like objects, arrays, strings, numbers, and booleans to ensure that your inputs and outputs are serializable.

Serialization ensures that workflow state, such as task results and intermediate values, can be reliably saved and restored. This is critical for enabling human-in-the-loop interactions, fault tolerance, and parallel execution.

Providing non-serializable inputs or outputs will result in a runtime error when a workflow is configured with a checkpoint.

## Determinism

---

When you resume a workflow run, the code does **NOT** resume from the **same line of code** where execution stopped. Execution returns to a checkpoint boundary, and the workflow **replays** forward until it reaches the pause again.

For the Functional API, replay starts at the beginning of the **entrypoint** while LangGraph restores completed [task](#) and [subgraph](#) results from the checkpoint instead of recomputing them. That preserves the recorded order of steps across pauses, including for long-running or non-deterministic **task** outputs.

To use features like **human-in-the-loop**, you must place non-deterministic work (for example, random values) and side effects (for example, file writes or API calls) in [tasks](#).

Different runs of a workflow can produce different results, but resuming a **specific** thread should replay the same persisted **task** and **subgraph** results.

To ensure that your workflow is deterministic and can be consistently replayed, follow these guidelines:

- **Avoid repeating work:** In an **entrypoint**, if you chain several side effects (for example, logging, file writes, or network calls), give each its own **task** so resume restores their outputs from the checkpoint instead of running them again.
- **Encapsulate non-deterministic operations:** Keep values that can change between attempts (for example, random numbers or wall-clock reads) inside **tasks**, so replay lines up with what was checkpointed.
- **Use idempotent operations:** For partial task failures and retries, see [Idempotency](#).

## Idempotency

---

Idempotency ensures that running the same operation multiple times produces the same result. This helps prevent duplicate API calls and redundant processing if a step is rerun due to a failure. Always place API calls inside **tasks** functions for checkpointing, and design them to be idempotent in case of re-execution. This is particularly important for operations that result in data writes. When a workflow resumes, LangGraph replays completed **task** results from the checkpoint. A **task** that started but did not finish may run again on that resume, so design side effects to be idempotent. Use idempotency keys or verify existing results to avoid unintended duplication.



# Common pitfalls

## Handling side effects

Encapsulate side effects (e.g., writing to a file, sending an email) in tasks to ensure they are not executed multiple times when resuming a workflow.

### Incorrect

In this example, a side effect (writing to a file) is directly included in the workflow, so it will be executed a second time when resuming the workflow.

#### python

```
```python
@entrypoint(checkpointer=checkpointer)
def my_workflow(inputs: dict) -> int:
    # This code will be executed a second time when resuming the workflow.
    # Which is likely not what you want.
    with open("output.txt", "w") as f: # [!code highlight]
        f.write("Side effect executed") # [!code highlight]
    value = interrupt("question")
    return value

```

</div>

<div class="callout js" markdown="1">

<p class="callout-label">js</p>

```
```typescript
```

```
import { entrypoint, interrupt } from "@langchain/langgraph";
import fs from "fs";

const myWorkflow = entrypoint(
 { checkpointer, name: "workflow" },
 async (inputs: Record<string, any>) => {
 // This code will be executed a second time when resuming the workflow.
 // Which is likely not what you want.
 fs.writeFileSync("output.txt", "Side effect executed");
 const value = interrupt("question");
 return value;
 }
);
```

### Correct

In this example, the side effect is encapsulated in a task, ensuring consistent execution upon resumption.

## python

```
from langgraph.func import task

@task # [!code highlight]
def write_to_file(): # [!code highlight]
 with open("output.txt", "w") as f:
 f.write("Side effect executed")

@entrypoint(checkpointer=checkpointer)
def my_workflow(inputs: dict) -> int:
 # The side effect is now encapsulated in a task.
 write_to_file().result()
 value = interrupt("question")
 return value
```

## js

```
import { entrypoint, task, interrupt } from "@langchain/langgraph";
import * as fs from "fs";

const writeToFile = task("writeToFile", async () => {
 fs.writeFileSync("output.txt", "Side effect executed");
});

const myWorkflow = entrypoint(
 { checkpointer, name: "workflow" },
 async (inputs: Record<string, any>) => {
 // The side effect is now encapsulated in a task.
 await writeToFile();
 const value = interrupt("question");
 return value;
 }
);
```

## Non-deterministic control flow

Operations that might give different results each time (like getting current time or random numbers) should be encapsulated in tasks to ensure that on resume, the same result is returned.

- In a task: Get random number (5) → interrupt → resume → (returns 5 again) → ...
- Not in a task: Get random number (5) → interrupt → resume → get new random number (7) → ...

This is especially important when using **human-in-the-loop** workflows with multiple interrupt calls. LangGraph keeps a list of resume values for each task/entrypoint. When an interrupt is encountered, it's matched with the corresponding resume value. This matching is strictly **index-based**, so the order of the resume values should match the order of the interrupts.

If order of execution is not maintained when resuming, one `@interrupt` call may be matched with the wrong `resume` value, leading to incorrect results.

Please read the section on [determinism](#) for more details.

## Incorrect

In this example, the workflow uses the current time to determine which task to execute. This is non-deterministic because the result of the workflow depends on the time at which it is executed.

### python

```
from langgraph.func import entrypoint

@entrypoint(checkpointer=checkpointer)
def my_workflow(inputs: dict) -> int:
 t0 = inputs["t0"]
 t1 = time.time() # [!code highlight]

 delta_t = t1 - t0

 if delta_t > 1:
 result = slow_task(1).result()
 value = interrupt("question")
 else:
 result = slow_task(2).result()
 value = interrupt("question")

 return {
 "result": result,
 "value": value
 }
```

### js

```
import { entrypoint, interrupt } from "@langchain/langgraph";

const myWorkflow = entrypoint(
 { checkpointer, name: "workflow" },
 async (inputs: { t0: number }) => {
 const t1 = Date.now();

 const deltaT = t1 - inputs.t0;
```

```

 if (deltaT > 1000) {
 const result = await slowTask(1);
 const value = interrupt("question");
 return { result, value };
 } else {
 const result = await slowTask(2);
 const value = interrupt("question");
 return { result, value };
 }
 }
};

```

**Correct**

## python

In this example, the workflow uses the input `t0` to determine which task to execute. This is deterministic because the result of the workflow depends only on the input.

```

import time

from langgraph.func import task

@task # [!code highlight]
def get_time() -> float: # [!code highlight]
 return time.time()

@entrypoint(checkpointer=checkpointer)
def my_workflow(inputs: dict) -> int:
 t0 = inputs["t0"]
 t1 = get_time().result() # [!code highlight]

 delta_t = t1 - t0

 if delta_t > 1:
 result = slow_task(1).result()
 value = interrupt("question")
 else:
 result = slow_task(2).result()
 value = interrupt("question")

 return {
 "result": result,
 "value": value
 }

```

js

In this example, the workflow uses the input `t0` to determine which task to execute. This is deterministic because the result of the workflow depends only on the input.

```
import { entrypoint, task, interrupt } from "@langchain/langgraph";

const getTime = task("getTime", () => Date.now());

const myWorkflow = entrypoint(
 { checkpoint, name: "workflow" },
 async (inputs: { t0: number }): Promise<any> => {
 const t1 = await getTime();

 const deltaT = t1 - inputs.t0;

 if (deltaT > 1000) {
 const result = await slowTask(1);
 const value = interrupt("question");
 return { result, value };
 } else {
 const result = await slowTask(2);
 const value = interrupt("question");
 return { result, value };
 }
 }
);
```

## Learn more

---

- [How to use the Functional API](#)
- [Graph API conceptual overview](#)
- [Choosing between Graph API and Functional API](#)

# Graph API overview

---

langgraph/graph-api.mdx

## Graphs

---

At its core, LangGraph models agent workflows as graphs. You define the behavior of your agents using three key components:

1. [State](#): A shared data structure that represents the current snapshot of your application. It can be any data type, but is typically defined using a shared state schema.
2. [Nodes](#): Functions that encode the logic of your agents. They receive the current state as input, perform some computation or side-effect, and return an updated state.
3. [Edges](#): Functions that determine which [Node](#) to execute next based on the current state. They can be conditional branches or fixed transitions.

By composing [Nodes](#) and [Edges](#), you can create complex, looping workflows that evolve the state over time. The real power, though, comes from how LangGraph manages that state.

To emphasize: [Nodes](#) and [Edges](#) are nothing more than functions—they can contain an LLM or just good ol' code.

In short: *nodes do the work, edges tell what to do next.*

LangGraph's underlying graph algorithm uses [message passing](#) to define a general program. When a Node completes its operation, it sends messages along one or more edges to other node(s). These recipient nodes then execute their functions, pass the resulting messages to the next set of nodes, and the process continues. Inspired by Google's [Pregel](#) system, the program proceeds in discrete "super-steps."

A super-step can be considered a single iteration over the graph nodes. Nodes that run in parallel are part of the same super-step, while nodes that run sequentially belong to separate super-steps. At the start of graph execution, all nodes begin in an [inactive](#) state. A node becomes [active](#) when it receives a new message (state) on any of its incoming edges (or "channels"). The active node then runs its function and responds with updates. At the end of each super-step, nodes with no incoming messages vote to [halt](#) by marking themselves as [inactive](#). The graph execution terminates when all nodes are [inactive](#) and no messages are in transit.

## StateGraph

The `@[StateGraph]` class is the main graph class to use. This is parameterized by a user defined [State](#) object.

## Compiling your graph

To build your graph, you first define the [state](#), you then add [nodes](#) and [edges](#), and then you compile it. What exactly is compiling your graph and why is it needed?

Compiling is a pretty simple step. It provides a few basic checks on the structure of your graph (no orphaned nodes, etc). It is also where you can specify runtime args like [checkpointers](#) and breakpoints. You compile your graph by just calling the `.compile` method:

### python

```
graph = graph_builder.compile(...)
```

### js

```
const graph = new StateGraph(StateAnnotation)
 .addNode("nodeA", nodeA)
 .addEdge(START, "nodeA")
 .addEdge("nodeA", END)
 .compile();
```

### 주의

You **MUST** compile your graph before you can use it.

## State

### python

The first thing you do when you define a graph is define the `state` of the graph. The `state` consists of the [schema of the graph](#) as well as [reducer functions](#) which specify how to apply updates to the state. The schema of the `state` will be the input schema to all `Nodes` and `Edges` in the graph, and can be either a `TypedDict` or a `Pydantic` model. All `Nodes` will emit updates to the `state` which are then applied using the specified `reducer` function.

### js

The first thing you do when you define a graph is define the `state` of the graph. The `state` consists of the [schema of the graph](#) as well as [reducer functions](#) which specify how to apply updates to the state. The schema of the `state` will be the input schema to all `Nodes` and `Edges` in the graph. You define state using the `StateSchema` class, which accepts any [standard schemas](#) (like [Zod](#)) for individual fields along with special value types like `ReducedValue` and

`MessagesValue`. All `Nodes` will emit updates to the `state` which are then applied using the specified `reducer` function.

## Schema

### python

The main documented way to specify the schema of a graph is by using a [TypedDict](#). If you want to provide default values in your state, use a [dataclass](#). We also support using a Pydantic [BaseModel](#) as your graph state if you want recursive data validation (though note that Pydantic is less performant than a `TypedDict` or `dataclass`).

By default, the graph will have the same input and output schemas. If you want to change this, you can also specify explicit input and output schemas directly. This is useful when you have a lot of keys, and some are explicitly for input and others for output. See the [guide](#) for more information.

### 정보

The higher-level [create\\_agent](#) factory in `langchain` does not support Pydantic state schemas.

### js

The main way to specify the schema of a graph is by using the `StateSchema` class. Each field in the schema can be:

- A **Standard schema** for simple fields (becomes a "last value" channel that overwrites on update)
- A `ReducedValue` for fields that need a custom reducer function (when nodes are run in parallel)
- A `MessagesValue` for chat message lists (prebuilt with message-aware reducer)
- An `UntrackedValue` for transient state that should not be checkpointed

```
import {
 StateSchema,
 ReducedValue,
 MessagesValue,
 UntrackedValue
} from "@langchain/langgraph";
import { z } from "zod/v4";

const AgentState = new StateSchema({
 // Prebuilt messages value with built-in reducer
```



```

messages: MessagesValue,

// Simple fields use Zod schemas directly
currentStep: z.string(),

// Fields with defaults
retryCount: z.number().default(0),

// Custom reducer for accumulating values
allSteps: new ReducedValue(
 z.array(z.string()).default(() => []),
 {
 inputSchema: z.string(),
 reducer: (current, newStep) => [...current, newStep],
 }
),

// Transient state not saved to checkpoints
tempCache: new UntrackedValue(z.record(z.string(), z.unknown())),
});

// Type extraction
type State = typeof AgentState.State; // Full state type
type Update = typeof AgentState.Update; // Partial update type

// Use in graph
const graph = new StateGraph(AgentState)
 .addNode("myNode", ...)
 .compile();

```

By default, the graph will have the same input and output schemas. If you want to change this, you can also specify explicit input and output schemas directly. This is useful when you have a lot of keys, and some are explicitly for input and others for output.

## Multiple schemas

Typically, all graph nodes communicate with a single schema. This means that they will read and write to the same state channels. But, there are cases where we want more control over this:

- Internal nodes can pass information that is not required in the graph's input / output.
- We may also want to use different input / output schemas for the graph. The output might, for example, only contain a single relevant output key.

It is possible to have nodes write to private state channels inside the graph for internal node communication. We can simply define a private schema, `PrivateState`.

It is also possible to define explicit input and output schemas for a graph. In these cases, we define an "internal" schema that contains *all* keys relevant to graph operations. But, we also define `input`

and `output` schemas that are sub-sets of the "internal" schema to constrain the input and output of the graph. See [Define input and output schemas](#) for more detail.

Let's look at an example:

**python**

**js**

There are two subtle and important points to note here:

**python**

1. We pass `state: InputState` as the input schema to `node_1`. But, we write out to `foo`, a channel in `OverallState`. How can we write out to a state channel that is not included in the input schema? This is because a node *can write to any state channel in the graph state*. The graph state is the union of the state channels defined at initialization, which includes `OverallState` and the filters `InputState` and `OutputState`.
2. We initialize the graph with:

```
python StateGraph(OverallState, input_schema=InputState, output_schema=OutputState)
```

How can we write to `PrivateState` in `node_2`? How does the graph gain access to this schema if it was not passed in the `StateGraph` initialization?

We can do this because `_nodes` can also declare additional state `channels_` as long as the state schema definition exists. In this case, the `PrivateState` schema is defined, so we can add `bar` as a new state channel in the graph and write to it.

**js**

1. We pass `state` as the input schema to `node1`. But, we write out to `foo`, a channel in `OverallState`. How can we write out to a state channel that is not included in the input schema? This is because a node *can write to any state channel in the graph state*. The graph state is the union of the state channels defined at initialization, which includes `OverallState` and the filters `InputState` and `OutputState`.
2. We initialize the graph with `StateGraph({ state: OverallState, input: InputState, output: OutputState })`. How can we write to `PrivateState` in `node2`? How does the graph gain access to this schema if it was not passed in the `StateGraph` initialization? We can do this because *nodes can also declare additional state channels* as long as the state schema definition exists. In this case, the `PrivateState` schema is defined, so we can add `bar` as a new state channel in the graph and write to it.

## python

### 주의

#### Private channels are not redacted when streaming.

Input, output, and private schemas constrain what each node *reads* (its input schema) and what *invoke* *returns* (the output schema). They do **not** hide channels from *stream*.

When you stream with `stream_mode="values"`, the graph emits **all** of its state channels by default, including private ones, because values streaming defaults to the full set of state channels rather than the output schema. This is why a private channel like `bar` is hidden by *invoke* but visible while streaming:

To restrict the streamed values to a specific set of channels (e.g. only the output schema), pass `output_keys`:

```
stream = graph.stream_events(
 {"user_input": "My"},
 version="v3",
 output_keys=["graph_output"], # [!code highlight]
)
for snapshot in stream.values:
 print(snapshot)
{'graph_output': 'My name is Lance'}
```

If you only need the channels a node actually produced each step (rather than the full accumulated state), use `stream_mode="updates"` instead.

## js

### 주의

#### Private channels are not redacted when streaming.

Input, output, and private schemas constrain what each node *reads* (its input schema) and what *invoke* *returns* (the output schema). They do **not** hide channels from *stream*.

When you stream with `streamMode: "values"`, the graph emits **all** of its state channels by default — including private ones — because values streaming defaults to the full set of state channels rather than the output schema. This is why a private channel like `bar` is hidden by *invoke* but visible while streaming:

To restrict the streamed values to a specific set of channels (e.g. only the output schema), pass `outputKeys`:

```
const stream = await graph.streamEvents(
 { userInput: "My" },
 { version: "v3", outputKeys: ["graphOutput"] } // [!code highlight]
);
for await (const snapshot of stream.values) {
 console.log(snapshot);
}
// { graphOutput: 'My name is Lance' }
```

If you only need the channels a node actually produced each step (rather than the full accumulated state), use `streamMode: "updates"` instead.

## Reducers

Reducers are key to understanding how updates from nodes are applied to the `state`. Each key in the `state` has its own independent reducer function. If no reducer function is explicitly specified then it is assumed that all updates to that key should override it. There are a few different types of reducers, starting with the default type of reducer:

### Reducer arguments

Every reducer is a binary function with two positional arguments:

- **Left argument:** The current value already stored in state for that key.
- **Right argument:** The update for that key returned by a node.

When a node returns a partial update, LangGraph calls the reducer for each updated key and saves the return value as the new state value:

#### python

```
new_value = reducer(left=current_state[key], right=node_update[key])
```

#### js

```
const newValue = reducer(currentState[key], nodeUpdate[key]); // left, right
```

The left argument always comes from accumulated state. The right argument always comes from the latest node update. The following example names both arguments explicitly:

#### python

Suppose the state is `{"tags": ["draft"]}` and a node returns `{"tags": ["review"]}`. `LangGraph` calls:

The new state value for `tags` is `["draft", "review"]`.

**js**

Suppose the state is `{ tags: ["draft"] }` and a node returns `{ tags: ["review"] }`. `LangGraph` calls:

The new state value for `tags` is `["draft", "review"]`.

Custom reducers combine the left and right arguments. The [default reducer](#) discards the left argument and keeps only the right.

### Default reducer

The default reducer ignores the left argument and replaces the state value with the right argument. This example shows how to use the default reducer:

**python**

**js**

In this example, no reducer functions are specified for any key. Let's assume the input to the graph is:

**python**

`{"foo": 1, "bar": ["hi"]}`. Let's then assume the first `Node` returns `{"foo": 2}`. This is treated as an update to the state. Notice that the `Node` does not need to return the whole `State` schema - just an update. After applying this update, the `State` would then be `{"foo": 2, "bar": ["hi"]}`. If the second node returns `{"bar": ["bye"]}` then the `State` would then be `{"foo": 2, "bar": ["bye"]}`

**js**

`{ foo: 1, bar: ["hi"] }`. Let's then assume the first `Node` returns `{ foo: 2 }`. This is treated as an update to the state. Notice that the `Node` does not need to return the whole `State` schema - just an update. After applying this update, the `State` would then be `{ foo: 2, bar: ["hi"] }`. If the second node returns `{ bar: ["bye"] }` then the `State` would then be `{ foo: 2, bar: ["bye"] }`

## Custom reducers

A custom reducer combines the left and right arguments instead of replacing the state value, which is useful for accumulating values, such as appending updates to a list. This example shows how to specify a custom reducer:

### python

In this example, we've used the `Annotated` type to specify a reducer function (`operator.add`) for the second key (`bar`). Note that the first key remains unchanged. Let's assume the input to the graph is `{"foo": 1, "bar": ["hi"]}`. Let's then assume the first `Node` returns `{"foo": 2}`. This is treated as an update to the state. Notice that the `Node` does not need to return the whole `state` schema - just an update. After applying this update, the `state` would then be `{"foo": 2, "bar": ["hi"]}`. If the second node returns `{"bar": ["bye"]}` then the `state` would then be `{"foo": 2, "bar": ["hi", "bye"]}`. Notice here that the `bar` key is updated by adding the two lists together.

### js

In this example, we've used `ReducedValue` to specify a reducer function for the second key (`bar`). Note that the first key remains unchanged. Let's assume the input to the graph is `{ foo: 1, bar: ["hi"] }`. Let's then assume the first `Node` returns `{ foo: 2 }`. This is treated as an update to the state. Notice that the `Node` does not need to return the whole `state` schema - just an update. After applying this update, the `state` would then be `{ foo: 2, bar: ["hi"] }`. If the second node returns `{ bar: ["bye"] }` then the `state` would then be `{ foo: 2, bar: ["hi", "bye"] }`. Notice here that the `bar` key is updated by concatenating the two arrays together.

### python

#### Overwrite

##### 팁

In some cases, you may want to bypass a reducer and directly overwrite a state value. LangGraph provides the `Overwrite` type for this purpose. [Learn how to use Overwrite here.](#)

## Resetting a reducer field

A common source of confusion with reducers: with a merging reducer, returning an empty value does **not** clear the field. Because the reducer merges the right argument into the left one, an empty update is merged in and previously accumulated values are kept.

This pattern matters for error buffers or retry counters that must be cleared between retry attempts:

## python

To clear the field while keeping a merging reducer, wrap the update with [Overwrite](#):

For more information, see [Bypass reducers with Overwrite](#).

## js

To let a node reset (clear) the field, define a custom reducer that replaces the accumulated value instead of merging it:

Alternatively, wrap the update with `overwrite` to bypass the reducer for a single update, while keeping the field's normal merging behavior for every other update:

For more information, see [Bypass reducers with Overwrite](#).

## Untracked values

`UntrackedValue` is used for state fields that should exist during graph execution but should **never be checkpointed**. When a graph resumes from a checkpoint, untracked values will be reset to their initial state (or be unavailable).

This is useful for: - **Database connections** that can't be serialized - **Temporary caches** that should be rebuilt on resume - **Large objects** you don't want to persist - **Runtime-only configuration** that should be passed fresh each time

```
import { StateSchema, UntrackedValue, MessagesValue } from "@langchain/langgraph";
import { z } from "zod/v4";

const State = new StateSchema({
 messages: MessagesValue,

 // Untracked: throws if multiple nodes write in same step (guard: true is default)
 dbConnection: new UntrackedValue<DatabaseConnection>(),

 // Untracked with guard: false allows multiple writes, keeps last value
 tempCache: new UntrackedValue(
 z.record(z.string(), z.unknown()),
 { guard: false }
),

 // Untracked without a schema (for maximum flexibility)
 runtimeConfig: new UntrackedValue(),
});
```

**Behavior:** - During execution: Values are stored and accessible like normal state - On checkpoint: Untracked values are **excluded** from the checkpoint data - On resume: Untracked values start fresh (empty or with their default value) - With `guard: true` (default): Throws error if multiple nodes write in the same step - With `guard: false`: Multiple writes allowed, last value wins

## 주의

Don't use `UntrackedValue` for data you need to persist across interrupts or time travel. Use regular state fields or `ReducedValue` for persistent data.

## Type utilities

LangGraph provides several type utilities for better TypeScript type safety when defining nodes and conditional edges.

### GraphNode

Use `GraphNode` to type node functions defined outside the graph builder:

```
import { GraphNode, StateSchema, Command } from "@langchain/langgraph";
import { z } from "zod/v4";

const State = new StateSchema({
 count: z.number().default(0),
 result: z.string(),
});

// Basic node - receives state, returns partial update
const incrementNode: GraphNode<typeof State> = (state) => {
 return { count: state.count + 1 };
};

// Async node
const fetchNode: GraphNode<typeof State> = async (state, config) => {
 const response = await fetch(`/api/data/${state.count}`);
 return { result: await response.text() };
};

// Node with Command routing - specify valid destinations
const routerNode: GraphNode<{ InputSchema: typeof State; Nodes: "process" | "done" }> = (state)
=> {
 if (state.count >= 10) {
 return new Command({ goto: "done" });
 }
 return new Command({
 update: { count: state.count + 1 },
 goto: "process"
 });
};
```



```
});
};
```

### **State.Node** shorthand

Each `StateSchema` instance has a `Node` property that provides a shorthand for typing nodes:

```
const State = new StateSchema({
 messages: MessagesValue,
 step: z.string(),
});

// These are equivalent:
const myNode1: GraphNode<typeof State> = (state) => ({ step: "done" });
const myNode2: typeof State.Node = (state) => ({ step: "done" });
```

### **ConditionalEdgeRouter**

Use `ConditionalEdgeRouter` for routing functions in conditional edges (no state updates, just routing):

```
import { ConditionalEdgeRouter, END } from "@langchain/langgraph";

const State = new StateSchema({
 shouldContinue: z.boolean(),
 step: z.string(),
});

// Router returns node name(s) or END
const router: ConditionalEdgeRouter<{ InputSchema: typeof State; Nodes: "process" | "summarize" }> = (state) => {
 if (!state.shouldContinue) {
 return END;
 }
 return state.step === "initial" ? "process" : "summarize";
};

// Use in graph
graph.addConditionalEdges("check", router);
```

### **StateSchema.State** and **StateSchema.Update**

Extract the state and update types from a schema for use in custom type definitions:

```
import { StateSchema } from "@langchain/langgraph";

const MyStateSchema = new StateSchema({
```

```

 messages: MessagesValue,
 count: z.number().default(0),
 });

 // Extract the full state type
 type MyState = typeof MyStateSchema.State;
 // { messages: BaseMessage[], count: number }

 // Extract the update type (partial, with reducer input types)
 type MyUpdate = typeof MyStateSchema.Update;
 // { messages?: Messages, count?: number }

```

## Working with messages in graph state

### Why use messages?

Most modern LLM providers have a chat model interface that accepts a list of messages as input. LangChain's [chat model interface](#) in particular accepts a list of message objects as inputs. These messages come in a variety of forms such as `@[ HumanMessage ]` (user input) or `@[ AIMessage ]` (LLM response).

To read more about what message objects are, please refer to the [Messages conceptual guide](#).

### Using messages in your graph

#### python

In many cases, it is helpful to store prior conversation history as a list of messages in your graph state. To do so, we can add a key (channel) to the graph state that stores a list of `Message` objects and annotate it with a reducer function (see `messages` key in the example below). The reducer function is vital to telling the graph how to update the list of `Message` objects in the state with each state update (for example, when a node sends an update). If you don't specify a reducer, every state update will overwrite the list of messages with the most recently provided value. If you wanted to simply append messages to the existing list, you could use `operator.add` as a reducer.

However, you might also want to manually update messages in your graph state (e.g. human-in-the-loop). If you were to use `operator.add`, the manual state updates you send to the graph would be appended to the existing list of messages, instead of updating existing messages. To avoid that, you need a reducer that can keep track of message IDs and overwrite existing messages, if updated. To achieve this, you can use the prebuilt `@[ add_messages ]` function. For brand new messages, it will simply append to existing list, but it will also handle the updates for existing messages correctly.

#### js

In many cases, it is helpful to store prior conversation history as a list of messages in your graph state. To do so, you can use the prebuilt `MessagesValue` which provides a message-aware reducer that handles message IDs, updates, and deletions automatically.

The `MessagesValue` reducer is vital to telling the graph how to update the list of `Message` objects in the state with each state update. If you don't specify a reducer, every state update will overwrite the list of messages with the most recently provided value. `MessagesValue` handles this correctly: for brand new messages, it appends to the existing list, and for existing messages (matched by ID), it updates them in place.

#### 팁

`MessagesValue` is actually a special case of `ReducedValue`, preconfigured with an internal `messagesStateReducer` that handles message lists and updates. This provides convenient, message-aware state management for chat message history in LangGraph graphs.

## Serialization

### python

In addition to keeping track of message IDs, the `@[add_messages]` function will also try to deserialize messages into LangChain `Message` objects whenever a state update is received on the `messages` channel.

For more information, see [LangChain serialization/deserialization](#). This allows sending graph inputs / state updates in the following format:

```
this is supported
{"messages": [HumanMessage(content="message")]}

and this is also supported
{"messages": [{"type": "human", "content": "message"}]}
```

Since the state updates are always deserialized into LangChain `Messages` when using `@[add_messages]`, you should use dot notation to access message attributes, like `state["messages"][-1].content`.

Below is an example of a graph that uses `@[add_messages]` as its reducer function.

```
from langchain.messages import AnyMessage
from langgraph.graph.message import add_messages
from typing import Annotated
from typing_extensions import TypedDict
```

```
class GraphState(TypedDict):
 messages: Annotated[list[AnyMessage], add_messages]
```

## js

In addition to keeping track of message IDs, `MessagesValue` will also try to deserialize messages into LangChain `Message` objects whenever a state update is received on the `messages` channel. This allows sending graph inputs / state updates in the following format:

```
// this is supported
{
 messages: [new HumanMessage("message")];
}

// and this is also supported
{
 messages: [{ role: "human", content: "message" }];
}
```

Since the state updates are always deserialized into LangChain `Messages` when using `MessagesValue`, you should use dot notation to access message attributes, like `state.messages.at(-1).content`. Below is an example of a graph that uses `MessagesValue`:

```
import { StateGraph, StateSchema, MessagesValue } from "@langchain/langgraph";

const State = new StateSchema({
 messages: MessagesValue,
});

const graph = new StateGraph(State)
...
```

The `messages` field is defined as a `MessagesValue` which is a list of `@[BaseMessage]` objects with a built-in reducer. Typically, there is more state to track than just messages, so we see people extend this state and add more fields, like:

```
import { StateSchema, MessagesValue } from "@langchain/langgraph";
import * as z from "zod";

const State = new StateSchema({
 messages: MessagesValue,
 documents: z.array(z.string()),
});
```

## python

## MessagesState

Since having a list of messages in your state is so common, there exists a prebuilt state called `MessagesState` which makes it easy to use messages. `MessagesState` is defined with a single `messages` key which is a list of `AnyMessage` objects and uses the `@[ add_messages ]` reducer. Typically, there is more state to track than just messages, so we see people subclass this state and add more fields, like:

```
from langgraph.graph import MessagesState

class State(MessagesState):
 documents: list[str]
```

## Nodes

### python

In LangGraph, nodes are Python functions (either synchronous or asynchronous) that accept the following arguments:

1. `state` —The [state](#) of the graph
2. `config` —A `@[ RunnableConfig ]` object that contains configuration information like `thread_id` and tracing information like `tags`
3. `runtime` —A `Runtime` object that contains [runtime context](#) and other information like `store`, `stream_writer`, `execution_info`, `server_info`, `heartbeat` (for idle timeout refresh), and `control` (for [graceful shutdown](#))

Similar to `NetworkX`, you add these nodes to a graph using the `@[ add_node ]` method:

```
from dataclasses import dataclass
from typing_extensions import TypedDict

from langgraph.graph import StateGraph
from langgraph.runtime import Runtime

class State(TypedDict):
 input: str
 results: str

@dataclass
class Context:
 user_id: str

builder = StateGraph(State)

def plain_node(state: State):
```

```

 return state

def node_with_runtime(state: State, runtime: Runtime[Context]):
 print("In node: ", runtime.context.user_id)
 return {"results": f"Hello, {state['input']}!"}

def node_with_execution_info(state: State, runtime: Runtime):
 print("In node with thread_id: ", runtime.execution_info.thread_id) # [!code highlight]
 return {"results": f"Hello, {state['input']}!"}

builder.add_node("plain_node", plain_node)
builder.add_node("node_with_runtime", node_with_runtime)
builder.add_node("node_with_execution_info", node_with_execution_info)
...

```

## js

In LangGraph, nodes are typically functions (sync or async) that accept the following arguments:

1. `state` —The [state](#) of the graph
2. `config` —A `@RunnableConfig` object that contains configuration information like `thread_id` and tracing information like `tags`

You can add nodes to a graph using the `addNode` method. For better type safety, use the `GraphNode` type utility or `State.Node` to type your node functions:

```

import { StateGraph, StateSchema, GraphNode } from "@langchain/langgraph";
import * as z from "zod";

const State = new StateSchema({
 input: z.string(),
 results: z.string(),
});

// Option 1: Use GraphNode type utility
const myNode: GraphNode<typeof State> = (state, config) => {
 console.log("In node: ", config?.configurable?.user_id);
 return { results: `Hello, ${state.input}!` };
};

// Option 2: Use State.Node shorthand
const otherNode: typeof State.Node = (state) => {
 return state;
};

const builder = new StateGraph(State)
 .addNode("myNode", myNode)

```

```
.addNode("otherNode", otherNode)
...
```

Behind the scenes, functions are converted to `@[RunnableLambda]`, which add batch and async support to your function, along with [native tracing and debugging](#).

If you add a node to a graph without specifying a name, it will be given a default name equivalent to the function name.

### python

```
builder.add_node(my_node)
You can then create edges to/from this node by referencing it as `my_node`
```

### js

```
builder.addNode(myNode);
// You can then create edges to/from this node by referencing it as `myNode`
```

## Re-execution and idempotency

### python

When you compile with a [checkpointer](#), LangGraph saves checkpoints at [super-step](#) boundaries, not mid-function inside a node. If execution stops and later resumes (for example after an [interrupt](#) or a [retry](#)), the affected **node** runs again from the start of its function. Code and side effects before the pause run again.

**Idempotency.** Design **node** logic so re-execution does not corrupt state. If a node inserts a database row, running it twice should not create duplicate rows unless that is intentional. Use idempotency keys, upserts, or read-before-write checks. For effects around `interrupt()`, see [Side effects called before interrupt must be idempotent](#).

**Graph changes.** [Determinism](#) rules about code changes do not apply to graph structure. You can add or remove **nodes** and edges without breaking resume for existing threads. Resumed runs use saved state and execute whatever graph you compile now.

**Tasks and interrupts inside a node.** If a **node** calls [tasks](#) or `@[interrupt]`, stricter determinism rules apply on resume. LangGraph restores completed **task** results from the checkpointer, but changing **task** or `@[interrupt]` order in code before the resume point can mismatch cached values. A [Functional API](#) **entrypoint** compiles to a single **node** that runs the whole entrypoint method this way. See [Determinism](#), [Idempotency](#), and [Using tasks in nodes](#).

js

When you compile with a [checkpointer](#), LangGraph saves checkpoints at [super-step](#) boundaries, not mid-function inside a node. If execution stops and later resumes (for example after an [interrupt](#) or a retry), the affected **node** runs again from the start of its function. Code and side effects before the pause run again.

**Idempotency.** Design **node** logic so re-execution does not corrupt state. If a node inserts a database row, running it twice should not create duplicate rows unless that is intentional. Use idempotency keys, upserts, or read-before-write checks. For effects around `interrupt()`, see [Side effects called before `interrupt` must be idempotent](#).

**Graph changes.** [Determinism](#) rules about code changes do not apply to graph structure. You can add or remove **nodes** and edges without breaking resume for existing threads. Resumed runs use saved state and execute whatever graph you compile now.

**Tasks and interrupts inside a node.** If a **node** calls [tasks](#) or `@[ interrupt ]`, stricter determinism rules apply on resume. LangGraph restores completed **task** results from the checkpointer, but changing **task** or `@[ interrupt ]` order in code before the resume point can mismatch cached values. A [Functional API](#) **entrypoint** compiles to a single **node** that runs the whole entrypoint method this way. See [Determinism](#), [Idempotency](#), and [Using tasks in nodes](#).

## Using tasks in nodes

If a [node](#) contains multiple operations, you may find it easier to implement each operation as a [task](#) instead of splitting the logic across multiple nodes. Task results are checkpointed when the graph uses a checkpointer, so resuming a thread can skip completed **task** work inside the node.

python

Original

With task

js

Original

With task

### START node

The `@[ START ]` Node is a special node that represents the node that sends user input to the graph. The main purpose for referencing this node is to determine which nodes should be called first.



## python

```
from langgraph.graph import START

graph.add_edge(START, "node_a")
```

## js

```
import { START } from "@langchain/langgraph";

graph.addEdge(START, "nodeA");
```

## END node

The **END** Node is a special node that represents a terminal node. This node is referenced when you want to denote which edges have no actions after they are done.

## python

```
from langgraph.graph import END

graph.add_edge("node_a", END)
```

## js

```
import { END } from "@langchain/langgraph";

graph.addEdge("nodeA", END);
```

## Node caching

### python

LangGraph supports caching of tasks/nodes based on the input to the node. To use caching:

- Specify a cache when compiling a graph (or specifying an entrypoint)
- Specify a cache policy for nodes. Each cache policy supports:
  - `key_func` used to generate a cache key based on the input to a node, which defaults to a `hash` of the input with `pickle`.
  - `ttl`, the time to live for the cache in seconds. If not specified, the cache will never expire.

For example:

```

import time
from typing_extensions import TypedDict
from langgraph.graph import StateGraph
from langgraph.cache.memory import InMemoryCache
from langgraph.types import CachePolicy

class State(TypedDict):
 x: int
 result: int

builder = StateGraph(State)

def expensive_node(state: State) -> dict[str, int]:
 # expensive computation
 time.sleep(2)
 return {"result": state["x"] * 2}

builder.add_node("expensive_node", expensive_node, cache_policy=CachePolicy(ttl=3))
builder.set_entry_point("expensive_node")
builder.set_finish_point("expensive_node")

graph = builder.compile(cache=InMemoryCache())

print(graph.invoke({"x": 5}, stream_mode='updates')) # [!code highlight]
[{'expensive_node': {'result': 10}}]
print(graph.invoke({"x": 5}, stream_mode='updates')) # [!code highlight]
[{'expensive_node': {'result': 10}, '__metadata__': {'cached': True}}]

```

## 참고

`set_entry_point(node)` defines the first node the graph will execute. It is equivalent to `builder.add_edge(START, node)`.

`set_finish_point(node)` defines the last node in the graph. It is equivalent to `builder.add_edge(node, END)`.

Both methods are valid but `add_edge(START, ...)` and `add_edge(..., END)` are the recommended modern syntax.

1. First run takes two seconds to run (due to mocked expensive computation).
2. Second run utilizes cache and returns quickly.

## js

LangGraph supports caching of tasks/nodes based on the input to the node. To use caching:

- Specify a cache when compiling a graph (or specifying an entrypoint)

- Specify a cache policy for nodes. Each cache policy supports:
- `keyFunc`, which is used to generate a cache key based on the input to a node.
- `ttl`, the time to live for the cache in seconds. If not specified, the cache will never expire.

```
import { StateGraph, StateSchema, GraphNode, START } from "@langchain/langgraph";
import { InMemoryCache } from "@langchain/langgraph-checkpoint";
import { z } from "zod/v4";

const State = new StateSchema({
 x: z.number(),
 result: z.number(),
});

const expensiveNode: GraphNode<typeof State> = async (state) => {
 // Simulate an expensive operation
 await new Promise((resolve) => setTimeout(resolve, 3000));
 return { result: state.x * 2 };
};

const graph = new StateGraph(State)
 .addNode("expensive_node", expensiveNode, { cachePolicy: { ttl: 3 } })
 .addEdge(START, "expensive_node")
 .compile({ cache: new InMemoryCache() });

await graph.invoke({ x: 5 }, { streamMode: "updates" }); // [!code highlight]
// [{"expensive_node": {"result": 10}}]
await graph.invoke({ x: 5 }, { streamMode: "updates" }); // [!code highlight]
// [{"expensive_node": {"result": 10}, "__metadata__": {"cached": true}}]
```

## Edges

Edges define how the logic is routed and how the graph decides to stop. This is a big part of how your agents work and how different nodes communicate with each other. There are a few key types of edges:

- Normal Edges: Go directly from one node to the next.
- Conditional Edges: Call a function to determine which node(s) to go to next.
- Entry Point: Which node to call first when user input arrives.
- Conditional Entry Point: Call a function to determine which node(s) to call first when user input arrives.

A node can have multiple outgoing edges. If a node has multiple outgoing edges, **all** of those destination nodes will be executed in parallel as a part of the next superstep.

### 주의

For each node, choose one routing mechanism: use normal edges for static routing, or use conditional edges / @[ Command ] for dynamic routing. Do not mix normal edges and dynamic routing from the same node, because both paths can execute and make graph behavior harder to reason about.

## Normal edges

### python

If you **always** want to go from node A to node B, you can use the @[ add\_edge ] method directly.

```
graph.add_edge("node_a", "node_b")
```

### js

If you **always** want to go from node A to node B, you can use the @[ addEdge ] method directly.

```
graph.addEdge("nodeA", "nodeB");
```

## Conditional edges

### python

If you want to **optionally** route to one or more edges (or optionally terminate), you can use the @[ add\_conditional\_edges ] method. This method accepts the name of a node and a "routing function" to call after that node is executed:

```
graph.add_conditional_edges("node_a", routing_function)
```

Similar to nodes, the routing\_function accepts the current state of the graph and returns a value.

By default, the return value routing\_function is used as the name of the node (or list of nodes) to send the state to next. All those nodes will be run in parallel as a part of the next superstep.

You can optionally provide a dictionary that maps the routing\_function's output to the name of the next node.

```
graph.add_conditional_edges("node_a", routing_function, {True: "node_b", False: "node_c"})
```

### js

If you want to **optionally** route to one or more edges (or optionally terminate), you can use the `@[ addConditionalEdges ]` method. This method accepts the name of a node and a "routing function" to call after that node is executed:

```
graph.addConditionalEdges("nodeA", routingFunction);
```

Similar to nodes, the `routingFunction` accepts the current `state` of the graph and returns a value.

By default, the return value `routingFunction` is used as the name of the node (or list of nodes) to send the state to next. All those nodes will be run in parallel as a part of the next superstep.

You can optionally provide an object that maps the `routingFunction`'s output to the name of the next node.

```
graph.addConditionalEdges("nodeA", routingFunction, {
 true: "nodeB",
 false: "nodeC",
});
```

### 팁

Use [Command](#) instead of conditional edges if you want to combine state updates and routing in a single function.

## Entry point

### python

The entry point is the first node(s) that are run when the graph starts. You can use the `@[ add_edge ]` method from the virtual `@[ START ]` node to the first node to execute to specify where to enter the graph.

```
from langgraph.graph import START

graph.add_edge(START, "node_a")
```

### js

The entry point is the first node(s) that are run when the graph starts. You can use the `@[ addEdge ]` method from the virtual `@[ START ]` node to the first node to execute to specify where to enter the graph.

```
import { START } from "@langchain/langgraph";

graph.addEdge(START, "nodeA");
```

## Conditional entry point

### python

A conditional entry point lets you start at different nodes depending on custom logic. You can use `@[ add_conditional_edges ]` from the virtual `@[ START ]` node to accomplish this.

```
from langgraph.graph import START

graph.add_conditional_edges(START, routing_function)
```

You can optionally provide a dictionary that maps the `routing_function`'s output to the name of the next node.

```
graph.add_conditional_edges(START, routing_function, {True: "node_b", False: "node_c"})
```

### js

A conditional entry point lets you start at different nodes depending on custom logic. You can use `@[ addConditionalEdges ]` from the virtual `@[ START ]` node to accomplish this.

```
import { START } from "@langchain/langgraph";

graph.addConditionalEdges(START, routingFunction);
```

You can optionally provide an object that maps the `routingFunction`'s output to the name of the next node.

```
graph.addConditionalEdges(START, routingFunction, {
 true: "nodeB",
 false: "nodeC",
});
```

Send

### python

By default, `Nodes` and `Edges` are defined ahead of time and operate on the same shared state. However, there can be cases where the exact edges are not known ahead of time and/or you may want different versions of `state` to exist at the same time. A common example of this is with [map-reduce](#) design patterns. In this design pattern, a first node may generate a list of objects, and you may want to apply some other node to all those objects. The number of objects may be unknown ahead of time (meaning the number of edges may not be known) and the input `state` to the downstream `Node` should be different (one for each generated object).

To support this design pattern, LangGraph supports returning `@[Send]` objects from conditional edges. `Send` takes two arguments: first is the name of the node, and second is the state to pass to that node.

```
from langgraph.types import Send

def continue_to_jokes(state: OverallState):
 return [Send("generate_joke", {"subject": s}) for s in state['subjects']]

graph.add_conditional_edges("node_a", continue_to_jokes)
```

## js

By default, `Nodes` and `Edges` are defined ahead of time and operate on the same shared state. However, there can be cases where the exact edges are not known ahead of time and/or you may want different versions of `state` to exist at the same time. A common example of this is with map-reduce design patterns. In this design pattern, a first node may generate a list of objects, and you may want to apply some other node to all those objects. The number of objects may be unknown ahead of time (meaning the number of edges may not be known) and the input `state` to the downstream `Node` should be different (one for each generated object).

To support this design pattern, LangGraph supports returning `@[Send]` objects from conditional edges. `Send` takes two arguments: first is the name of the node, and second is the state to pass to that node.

```
import { Send } from "@langchain/langgraph";

graph.addConditionalEdges("nodeA", (state) => {
 return state.subjects.map(
 (subject) => new Send("generateJoke", { subject })
);
});
```

## Command

`@[Command]` is a versatile primitive for controlling graph execution. It accepts four parameters:

- `update`: Apply state updates (similar to returning updates from a node).
- `goto`: Navigate to specific nodes (similar to [conditional edges](#)).
- `graph`: Target a parent graph when navigating from [subgraphs](#).
- `resume`: Provide a value to resume execution after an [interrupt](#).

`Command` is used in three contexts:

- [Return from nodes](#): Use `update`, `goto`, and `graph` to combine state updates with control flow.
- [Input to invoke or stream](#): Use `resume` to continue execution after an interrupt.
- [Return from tools](#): Similar to return from nodes, combine state updates and control flow from inside a tool.

## Return from nodes

### `update` and `goto`

Return `@[Command]` from node functions to update state and route to the next node in a single step:

#### python

```
def my_node(state: State) -> Command[Literal["my_other_node"]]:
 return Command(
 # state update
 update={"foo": "bar"},
 # control flow
 goto="my_other_node"
)
```

With `@[Command]` you can also achieve dynamic control flow behavior (identical to [conditional edges](#)):

```
def my_node(state: State) -> Command[Literal["my_other_node"]]:
 if state["foo"] == "bar":
 return Command(update={"foo": "baz"}, goto="my_other_node")
```

Use `@[Command]` when you need to **both** update state **and** route to a different node. If you only need to route without updating state, use [conditional edges](#) instead.

### 참고

When returning `@[Command]` in your node functions, you must add return type annotations with the list of node names the node is routing to, e.g. `Command[Literal["my_other_node"]]`. This is necessary for the graph rendering and tells LangGraph that `my_node` can navigate to `my_other_node`.



js

```
import { Command } from "@langchain/langgraph";

graph.addNode("myNode", (state) => {
 return new Command({
 update: { foo: "bar" },
 goto: "myOtherNode",
 });
});
```

With `@[Command]` you can also achieve dynamic control flow behavior (identical to [conditional edges](#)):

```
import { Command } from "@langchain/langgraph";

graph.addNode("myNode", (state) => {
 if (state.foo === "bar") {
 return new Command({
 update: { foo: "baz" },
 goto: "myOtherNode",
 });
 }
});
```

Use `@[Command]` when you need to **both** update state **and** route to a different node. If you only need to route without updating state, use [conditional edges](#) instead.

When using `@[Command]` in your node functions, you must add the `ends` parameter when adding the node to specify which nodes it can route to:

```
builder.addNode("myNode", myNode, {
 ends: ["myOtherNode", END],
});
```

## 주의

`@[Command]` only adds dynamic edges—static edges defined with `add_edge` / `addEdge` still execute. For example, if `node_a` returns `Command(goto="my_other_node")` and you also have `graph.add_edge("node_a", "node_b")`, both `node_b` and `my_other_node` will run. For each node, use either `@[Command]` or static edges to route to the next nodes, not both.

Check out this [how-to guide](#) for an end-to-end example of how to use `@[Command]`.

## python

If you are using [subgraphs](#), you can navigate from a node within a subgraph to a different node in the parent graph by specifying `graph=Command.PARENT` in `@[Command]`:

```
def my_node(state: State) -> Command[Literal["other_subgraph"]]:
 return Command(
 update={"foo": "bar"},
 goto="other_subgraph", # where `other_subgraph` is a node in the parent graph
 graph=Command.PARENT
)
```

## 참고

Setting `graph` to `Command.PARENT` will navigate to the closest parent graph.

When you send updates from a subgraph node to a parent graph node for a key that's shared by both parent and subgraph [state schemas](#), you **must** define a [reducer](#) for the key you're updating in the parent graph state. See this [example](#).

## js

If you are using [subgraphs](#), you can navigate from a node within a subgraph to a different node in the parent graph by specifying `graph: Command.PARENT` in `Command`:

```
import { Command } from "@langchain/langgraph";

graph.addNode("myNode", (state) => {
 return new Command({
 update: { foo: "bar" },
 goto: "otherSubgraph", // where `otherSubgraph` is a node in the parent graph
 graph: Command.PARENT,
 });
});
```

## 참고

Setting `graph` to `Command.PARENT` will navigate to the closest parent graph.

When you send updates from a subgraph node to a parent graph node for a key that's shared by both parent and subgraph [state schemas](#), you **must** define a [reducer](#) for the key you're updating in the parent graph state.

This is particularly useful when implementing [multi-agent handoffs](#). Check out [Navigate to a node in a parent graph](#) for detail.

## Input to `invoke` or `stream`

### python

#### 주의

`Command(resume=...)` is the **only** `Command` pattern intended as input to `invoke()` / `stream()` (optionally combined with `update=...` to also apply a state change while resuming). Do not use `Command(update=...)` alone as input to continue multi-turn conversations—because passing any `Command` as input resumes from the latest checkpoint (i.e. the last step that ran, not `__start__`), the graph will appear stuck if it already finished. To continue a conversation on an existing thread, pass a plain input dict:

```
WRONG - graph resumes from the latest checkpoint
(last step that ran), appears stuck
graph.invoke(Command(update={ # [!code --]
 "messages": [{"role": "user", "content": "follow up"}] # [!code --]
}), config) # [!code --]

CORRECT - plain dict restarts from __start__
graph.invoke({ # [!code ++]
 "messages": [{"role": "user", "content": "follow up"}] # [!code ++]
}, config) # [!code ++]
```

### js

#### 주의

`new Command({ resume: ... })` is the **only** `Command` pattern intended as input to `invoke()` / `stream()` (optionally combined with `update` to also apply a state change while resuming). Do not use `new Command({ update: ... })` alone as input to continue multi-turn conversations—because passing any `Command` as input resumes from the latest checkpoint (i.e. the last step that ran, not `__start__`), the graph will appear stuck if it already finished. To continue a conversation on an existing thread, pass a plain input object:

```
// WRONG - graph resumes from the latest checkpoint
// (last step that ran), appears stuck
await graph.invoke(new Command({ update: { messages: [{ role: "user", content: "follow
up" }] } })), config); // [!code --]

// CORRECT - plain object restarts from __start__
```

```
await graph.invoke({ messages: [{ role: "user", content: "follow up" }] }, config); //
[!code ++]
```

## resume

### python

Use `Command(resume=...)` to provide a value and resume graph execution after an [interrupt](#). The value passed to `resume` becomes the return value of the `interrupt()` call inside the paused node:

Check out the [interrupts conceptual guide](#) for full details on interrupt patterns, including multiple interrupts and validation loops.

### js

Use `new Command({ resume: ... })` to provide a value and resume graph execution after an [interrupt](#). The value passed to `resume` becomes the return value of the `interrupt()` call inside the paused node:

```
import { Command, interrupt } from "@langchain/langgraph";

const humanReview = async (state: typeof StateAnnotation.State) => {
 // Pauses the graph and waits for a value
 const answer = interrupt("Do you approve?");
 return { messages: [{ role: "user", content: answer }] };
};

// First invocation - hits the interrupt and pauses
const result = await graph.invoke({ messages: [...] }, config);

// Resume with a value - the interrupt() call returns "yes"
const resumed = await graph.invoke(new Command({ resume: "yes" }), config);
```

Check out the [interrupts conceptual guide](#) for full details on interrupt patterns, including multiple interrupts and validation loops.

## Return from tools

You can return `@[Command]` from tools to update graph state and control flow. Use `update` to modify state (e.g., saving customer information looked up during a conversation) and `goto` to route to a specific node after the tool completes.

## 주의

When used inside tools, `goto` adds a dynamic edge—any static edges already defined on the node that called the tool will still execute. For each node, use either tool-driven dynamic routing or static edges to route to the next nodes, not both.

Refer to [Use inside tools](#) for detail.

## Graph migrations

LangGraph can easily handle migrations of graph definitions (nodes, edges, and state) even when using a checkpointer to track state.

- For threads at the end of the graph (i.e. not interrupted) you can change the entire topology of the graph (i.e. all nodes and edges, remove, add, rename, etc)
- For threads currently interrupted, we support all topology changes other than renaming / removing nodes (as that thread could now be about to enter a node that no longer exists) -- if this is a blocker please reach out and we can prioritize a solution.
- For modifying state, we have full backwards and forwards compatibility for adding and removing keys
- State keys that are renamed lose their saved state in existing threads
- State keys whose types change in incompatible ways could currently cause issues in threads with state from before the change -- if this is a blocker please reach out and we can prioritize a solution.

## 팁

For changes that are technically compatible but alter business logic, such as rewriting the tool set or restructuring conversation flow, see [Business compatibility](#). That page covers pinning a behavioral version in state so existing threads keep the old path while new threads pick up the latest version.

## Runtime context

### python

When creating a graph, you can specify a `context_schema` for runtime context passed to nodes. This is useful for passing information to nodes that is not part of the graph state. For example, you might want to pass dependencies such as model name or a database connection.

### js

When creating a graph, you can specify a `contextSchema` for runtime context passed to nodes. This is useful for passing information to nodes that is not part of the graph state. For example, you might want to pass dependencies such as model name or a database connection.

### python

```
@dataclass
class ContextSchema:
 llm_provider: str = "openai"

graph = StateGraph(State, context_schema=ContextSchema)
```

You can then pass this context into the graph using the `context` parameter of the `invoke` method.

```
graph.invoke(inputs, context={"llm_provider": "anthropic"})
```

### js

```
import { StateGraph, StateSchema } from "@langchain/langgraph";
import * as z from "zod";

const State = new StateSchema({
 input: z.string(),
 output: z.string(),
});

const ContextSchema = z.object({
 llm: z.union([z.literal("openai"), z.literal("anthropic")]),
});

const graph = new StateGraph(State, ContextSchema);
```

You can then pass this configuration into the graph using the `context` property.

```
const config = { context: { llm: "anthropic" } };

await graph.invoke(inputs, config);
```

You can then access and use this context inside a node or conditional edge:

### python

```
from langgraph.runtime import Runtime

def node_a(state: State, runtime: Runtime[ContextSchema]):
 llm = get_llm(runtime.context.llm_provider)
 # ...
```

js

```
import { Runtime, GraphNode } from "@langchain/langgraph";
import * as z from "zod";

const nodeA: GraphNode<typeof State> = (state, config) => {
 const llm = getLLM(runtime.context?.llm);
 // ...
 return {};
};
```

See [Add runtime configuration](#) for a full breakdown on configuration.

js

```
graph.addNode("myNode", (state, config) => {
 const llmType = config.context?.llm || "openai";
 const llm = getLLM(llmType);
 return { results: `Hello, ${state.input}!` };
});
```

## Recursion limit

python

The recursion limit sets the maximum number of [super-steps](#) the graph can execute during a single execution. Once the limit is reached, LangGraph will raise `GraphRecursionError`. Starting in version 1.0.6, the default recursion limit is set to 1000 steps. The recursion limit can be set on any graph at runtime, and is passed to `invoke` / `stream` via the config dictionary. Importantly, `recursion_limit` is a standalone config key and should not be passed inside the `configurable` key as all other user-defined configuration. See the example below:

```
graph.invoke(inputs, config={"recursion_limit": 5}, context={"llm": "anthropic"})
```

Read [Recursion limit](#) to learn more about how the recursion limit works.

js

The recursion limit sets the maximum number of [super-steps](#) the graph can execute during a single execution. Once the limit is reached, LangGraph will raise `GraphRecursionError`. By default this value is set to 25 steps. The recursion limit can be set on any graph at runtime, and is passed to `invoke/stream` via the config object. Importantly, `recursionLimit` is a standalone config key and should not be passed inside the `configurable` key as all other user-defined configuration. See the example below:

```
await graph.invoke(inputs, {
 recursionLimit: 5,
 context: { llm: "anthropic" },
});
```

## Accessing and handling the recursion counter

### python

The current step counter is accessible in `config["metadata"]["langgraph_step"]` within any node, allowing for proactive recursion handling before hitting the recursion limit. This enables you to implement graceful degradation strategies within your graph logic.

### js

The current step counter is accessible in `config.metadata.langgraph_step` within any node, allowing for proactive recursion handling before hitting the recursion limit. This enables you to implement graceful degradation strategies within your graph logic.

## How it works

### python

The step counter is stored in `config["metadata"]["langgraph_step"]`. LangGraph increments this counter as the graph executes and raises a `GraphRecursionError` once the configured `recursion_limit` is exceeded.

### js

The step counter is stored in `config.metadata.langgraph_step`. LangGraph increments this counter as the graph executes and raises a `GraphRecursionError` once the configured `recursionLimit` is exceeded.



## Accessing the current step counter

You can access the current step counter within any node to monitor execution progress.

### python

```
from langchain_core.runnables import RunnableConfig
from langgraph.graph import StateGraph

def my_node(state: dict, config: RunnableConfig) -> dict:
 current_step = config["metadata"]["langgraph_step"]
 print(f"Currently on step: {current_step}")
 return state
```

### js

```
import { RunnableConfig } from "@langchain/core/runnables";
import { StateGraph } from "@langchain/langgraph";

const myNode: GraphNode<typeof State> = async (state, config) => {
 const currentStep = config.metadata?.langgraph_step;
 console.log(`Currently on step: ${currentStep}`);
 return state;
}
```

### python

#### Proactive recursion handling

LangGraph provides a `RemainingSteps` managed value that tracks how many steps remain before hitting the recursion limit. This allows for graceful degradation within your graph.

```
from typing import Annotated, Literal
from langgraph.graph import StateGraph, START, END
from langgraph.managed import RemainingSteps

class State(TypedDict):
 messages: Annotated[list, lambda x, y: x + y]
 remaining_steps: RemainingSteps # Managed value - tracks steps until limit

def reasoning_node(state: State) -> dict:
 # RemainingSteps is automatically populated by LangGraph
 remaining = state["remaining_steps"]

 # Check if we're running low on steps
 if remaining <= 2:
```

```

 return {"messages": ["Approaching limit, wrapping up..."]}

Normal processing
return {"messages": ["thinking..."]}

def route_decision(state: State) -> Literal["reasoning_node", "fallback_node"]:
 """Route based on remaining steps"""
 if state["remaining_steps"] <= 2:
 return "fallback_node"
 return "reasoning_node"

def fallback_node(state: State) -> dict:
 """Handle cases where recursion limit is approaching"""
 return {"messages": ["Reached complexity limit, providing best effort answer"]}

Build graph
builder = StateGraph(State)
builder.add_node("reasoning_node", reasoning_node)
builder.add_node("fallback_node", fallback_node)
builder.add_edge(START, "reasoning_node")
builder.add_conditional_edges("reasoning_node", route_decision)
builder.add_edge("fallback_node", END)

graph = builder.compile()

RemainingSteps works with any recursion_limit
result = graph.invoke({"messages": []}, {"recursion_limit": 10})

```

js

Design your graph with explicit termination conditions, and catch `GraphRecursionError` as a safety net:

```

import {
 StateGraph,
 StateSchema,
 ReducedValue,
 GraphNode,
 ConditionalEdgeRouter,
 END,
 GraphRecursionError
} from "@langchain/langgraph";
import { z } from "zod/v4";

const State = new StateSchema({
 messages: new ReducedValue(
 z.array(z.string()).default(() => []),
 { reducer: (x, y) => x.concat(y) }
),
},

```

```

});

// Build graph with explicit termination logic
const graph = new StateGraph(State)
 .addNode("reasoning", async (state) => {
 // Normal processing - design your graph with explicit termination conditions
 return {
 messages: ["thinking..."]
 };
 })
 .addConditionalEdges("reasoning", (state) => {
 // Add your termination condition here
 if (state.messages.length >= 5) {
 return END;
 }
 return "reasoning";
 });

const app = graph.compile();

// Catch GraphRecursionError as a safety net
try {
 const result = await app.invoke(
 { messages: [] },
 { recursionLimit: 10 }
);
} catch (error) {
 if (error instanceof GraphRecursionError) {
 console.log("Recursion limit reached, handling gracefully");
 // Handle the error - return partial results, notify user, etc.
 }
}

```

## Proactive vs reactive approaches

There are two main approaches to handling recursion limits: proactive (monitoring within the graph) and reactive (catching errors externally).

### python

```

from typing import Annotated, Literal, TypedDict
from langgraph.graph import StateGraph, START, END
from langgraph.managed import RemainingSteps
from langgraph.errors import GraphRecursionError

class State(TypedDict):
 messages: Annotated[list, lambda x, y: x + y]
 remaining_steps: RemainingSteps

```

```

Proactive Approach (recommended) - using RemainingSteps
def agent_with_monitoring(state: State) -> dict:
 """Proactively monitor and handle recursion within the graph"""
 remaining = state["remaining_steps"]

Early detection - route to internal handling
if remaining <= 2:
 return {
 "messages": ["Approaching limit, returning partial result"]
 }

Normal processing
return {"messages": [f"Processing... ({remaining} steps remaining)"]}

def route_decision(state: State) -> Literal["agent", END]:
 if state["remaining_steps"] <= 2:
 return END
 return "agent"

Build graph
builder = StateGraph(State)
builder.add_node("agent", agent_with_monitoring)
builder.add_edge(START, "agent")
builder.add_conditional_edges("agent", route_decision)
graph = builder.compile()

Proactive: Graph completes gracefully
result = graph.invoke({"messages": []}, {"recursion_limit": 10})

Reactive Approach (fallback) - catching error externally
try:
 result = graph.invoke({"messages": []}, {"recursion_limit": 10})
except GraphRecursionError as e:
 # Handle externally after graph execution fails
 result = {"messages": ["Fallback: recursion limit exceeded"]}

```

js

```

import {
 StateGraph,
 StateSchema,
 ReducedValue,
 GraphNode,
 ConditionalEdgeRouter,
 END,
 GraphRecursionError
} from "@langchain/langgraph";
import { z } from "zod/v4";

```

```

const State = new StateSchema({
 messages: new ReducedValue(
 z.array(z.string()).default(() => []),
 { reducer: (x, y) => x.concat(y) }
),
});

// Build graph with explicit termination logic
const builder = new StateGraph(State)
 .addNode("agent", async (state) => {
 return {
 messages: ["Processing..."]
 };
 })
 .addConditionalEdges("agent", (state) => {
 // Design termination conditions into your graph
 if (state.messages.length >= 5) {
 return END;
 }
 return "agent";
 });

const graph = builder.compile();

// Reactive Approach - catch GraphRecursionError as safety net
try {
 const result = await graph.invoke(
 { messages: [] },
 { recursionLimit: 10 }
);
} catch (error) {
 if (error instanceof GraphRecursionError) {
 // Handle externally after graph execution fails
 console.log("Recursion limit exceeded, handling gracefully");
 }
}

```

## python

The key differences between these approaches are:

Approach	Detection	Handling	Control Flow
Proactive (using RemainingSteps )	Before limit reached	Inside graph via conditional routing	Graph continues to completion node
Reactive (catching GraphRecursionError )	After limit exceeded	Outside graph in try/catch	Graph execution terminated

### Proactive advantages:

- Graceful degradation within the graph
- Can save intermediate state in checkpoints
- Better user experience with partial results
- Graph completes normally (no exception)

### Reactive advantages:

- Simpler implementation
- No need to modify graph logic
- Centralized error handling

js

The reactive approach catches `GraphRecursionError` after the limit is exceeded. Design your graph with explicit termination conditions to avoid hitting the limit in the first place.

Approach	Detection	Handling	Control Flow
Reactive (catching <code>GraphRecursionError</code> )	After limit exceeded	Outside graph in try/catch	Graph execution terminated

### Reactive advantages:

- Simple implementation
- No need to modify graph logic
- Centralized error handling

## Other available metadata

python

Along with `langgraph_step`, the following metadata is also available in `config["metadata"]`:

```
def inspect_metadata(state: dict, config: RunnableConfig) -> dict:
 metadata = config["metadata"]

 print(f"Step: {metadata['langgraph_step']}")
 print(f"Node: {metadata['langgraph_node']}")
 print(f"Triggers: {metadata['langgraph_triggers']}")
 print(f"Path: {metadata['langgraph_path']}")
 print(f"Checkpoint NS: {metadata['langgraph_checkpoint_ns']}")

 return state
```

js

Along with `langgraph_step`, the following metadata is also available in `config.metadata`:

```
const inspectMetadata: GraphNode<typeof State> = async (state, config) => {
 const metadata = config.metadata;

 console.log(`Step: ${metadata?.langgraph_step}`);
 console.log(`Node: ${metadata?.langgraph_node}`);
 console.log(`Triggers: ${metadata?.langgraph_triggers}`);
 console.log(`Path: ${metadata?.langgraph_path}`);
 console.log(`Checkpoint NS: ${metadata?.langgraph_checkpoint_ns}`);

 return state;
}
```

## Visualization

---

It's often nice to be able to visualize graphs, especially as they get more complex. LangGraph comes with several built-in ways to visualize graphs. See [Visualize your graph](#) for more info.

## Observability and Tracing

---

To trace, debug and evaluate your agents, use [LangSmith](#).

## Learn more

---

- [How to use the Graph API](#)
- [Functional API conceptual overview](#)
- [Choosing between Graph API and Functional API](#)

# Install LangGraph

---

langgraph/install.mdx

To install the base LangGraph package:

## python

```
```bash pip pip install -U langgraph
```

```
```bash uv
uv add langgraph
```

## js

```
```bash npm npm install @langchain/langgraph @langchain/core
```

```
```bash pnpm
pnpm add @langchain/langgraph @langchain/core
```

```
```bash yarn yarn add @langchain/langgraph @langchain/core
```

```
```bash bun
bun add @langchain/langgraph @langchain/core
```

To use LangGraph you will usually want to access LLMs and define tools. You can do this however you see fit.

One way to do this (which we will use in the docs) is to use [LangChain](#).

Install LangChain with:

## python

```
```bash pip pip install -U langchain
```

Requires Python 3.10+

```
```bash uv
```



```
uv add langchain
Requires Python 3.10+
```

## js

```
```bash npm npm install langchain
```

```
```bash pnpm
pnpm add langchain
```

```
```bash yarn yarn add langchain
```

```
```bash bun
bun add langchain
```

To work with specific LLM provider packages, you will need install them separately.

Refer to the [integrations](#) page for provider-specific installation instructions.

# Interrupts

langgraph/interrupts.mdx

Interrupts allow you to pause graph execution at specific points and wait for external input before continuing. This enables human-in-the-loop patterns where you need external input to proceed. When an interrupt is triggered, LangGraph saves the graph state using its [persistence](#) layer and waits indefinitely until you resume execution.

Interrupts work by calling the `interrupt()` function at any point in your graph nodes. The function accepts any JSON-serializable value which is surfaced to the caller. When you're ready to continue, you resume execution by re-invoking the graph using `Command`, which then becomes the return value of the `interrupt()` call from inside the node.

Unlike static breakpoints (which pause before or after specific nodes), interrupts are **dynamic**: they can be placed anywhere in your code and can be conditional based on your application logic.

## python

- **Checkpointing keeps your place:** the checkpointer writes the exact graph state so you can resume later, even when in an error state.
- **thread\_id is your pointer:** set `config={"configurable": {"thread_id": ...}}` to tell the checkpointer which state to load.
- **Interrupt payloads surface via `stream.interrupts`:** when using [event streaming](#) (`graph.stream_events(..., version="v3")`), the values you pass to `interrupt()` appear on `stream.interrupts`, and `stream.interrupted` is `True` when the run pauses for input.

## js

- **Checkpointing keeps your place:** the checkpointer writes the exact graph state so you can resume later, even when in an error state.
- **thread\_id is your pointer:** use `{ configurable: { thread_id: ... } }` as options to the `invoke` method to tell the checkpointer which state to load.
- **Interrupt payloads surface as `__interrupt__`:** the values you pass to `interrupt()` return to the caller in the `__interrupt__` field so you know what the graph is waiting on.

The `thread_id` you choose is effectively your persistent cursor. Reusing it resumes the same checkpoint; using a new value starts a brand-new thread with an empty state.

## Pause using `interrupt`

The `@interrupt` function pauses graph execution and returns a value to the caller. When you call `@interrupt` within a node, LangGraph saves the current graph state and waits for you to resume execution with input.

To use `@interrupt`, you need: 1. A **checkpointer** to persist the graph state (use a durable checkpointer in production) 2. A **thread ID** in your config so the runtime knows which state to resume from 3. To call `interrupt()` where you want to pause (payload must be JSON-serializable)

### python

```
from langgraph.types import interrupt

def approval_node(state: State):
 # Pause and ask for approval
 approved = interrupt("Do you approve this action?")

 # When you resume, Command(resume=...) returns that value here
 return {"approved": approved}
```

### js

```
import { interrupt } from "@langchain/langgraph";

async function approvalNode(state: State) {
 // Pause and ask for approval
 const approved = interrupt("Do you approve this action?");

 // Command({ resume: ... }) provides the value returned into this variable
 return { approved };
}
```

When you call `@interrupt`, here's what happens:

1. **Graph execution gets suspended** at the exact point where `@interrupt` is called
2. **State is saved** using the checkpointer so execution can be resumed later, In production, this should be a persistent checkpointer (e.g. backed by a database)

### python

3. **Value is returned** to the caller on `stream.interrupts` when using [event streaming](#) (`graph.stream_events(..., version="v3")`), or under `__interrupt__` with the default `invoke()` API; it can be any JSON-serializable value (string, object, array, etc.)

js

3. **Value is returned** to the caller under `__interrupt__`; it can be any JSON-serializable value (string, object, array, etc.)

4. **Graph waits indefinitely** until you resume execution with a response

5. **Response is passed back** into the node when you resume, becoming the return value of the `interrupt()` call

## Resuming interrupts

After an interrupt pauses execution, you resume the graph by invoking it again with a `Command` that contains the resume value. The resume value is passed back to the `interrupt` call, allowing the node to continue execution with the external input.

python

The recommended way to drive a graph that may interrupt is [event streaming](#) — it surfaces interrupts via `stream.interrupts` and `stream.interrupted`, and exposes the final state through `stream.output`.

### 참고

The default `graph.invoke(...)` API still works and surfaces interrupts under `result["__interrupt__"]`. Use it when you don't need streamed projections; otherwise prefer `graph.stream_events(..., version="v3")`.

### Key points about resuming:

- You must use the **same thread ID** when resuming that was used when the interrupt occurred
- The value passed to `Command(resume=...)` becomes the return value of the `@[interrupt]` call
- The node restarts from the beginning of the node where the `@[interrupt]` was called when resumed, so any code before the `@[interrupt]` runs again
- You can pass any JSON-serializable value as the resume value, subject to validation when you specify a typed `response_schema`

### 주의

`Command(resume=...)` is the **only** `Command` pattern intended as input to `invoke()` / `stream()` / `stream_events()`. The other `Command` parameters (`update`, `goto`, `graph`)

are designed for [returning from node functions](#). Do not pass `Command(update=...)` as input to continue multi-turn conversations—pass a plain input dict instead.

js

```
import { Command } from "@langchain/langgraph";

// Initial run - hits the interrupt and pauses
// thread_id is the durable pointer back to the saved checkpoint
const config = { configurable: { thread_id: "thread-1" } };
const result = await graph.invoke({ input: "data" }, config);

// Check what was interrupted
// __interrupt__ mirrors every payload you passed to interrupt()
console.log(result.__interrupt__);
// [{ value: 'Do you approve this action?', ... }]

// Resume with the human's response
// Command({ resume }) returns that value from interrupt() in the node
await graph.invoke(new Command({ resume: true }), config);
```

### Key points about resuming:

- You must use the **same thread ID** when resuming that was used when the interrupt occurred
- The value passed to `new Command({ resume: ... })` becomes the return value of the `@[ interrupt ]` call
- The node restarts from the beginning of the node where the `@[ interrupt ]` was called when resumed, so any code before the `@[ interrupt ]` runs again
- You can pass any JSON-serializable value as the resume value, subject to validation when you specify a Zod `responseSchema`

### 주의

`new Command({ resume: ... })` is the **only** Command pattern intended as input to `invoke()` / `stream()` / `stream_events()`. The other Command parameters (`update`, `goto`, `graph`) are designed for [returning from node functions](#). Do not pass `new Command({ update: ... })` as input to continue multi-turn conversations—pass a plain input object instead.

## Define an interrupt response schema

python

The `response_schema` argument to `@[ interrupt ]` describes the input expected when you resume the graph. Clients can use the schema to render typed input forms.

## 참고

Interrupt response schemas require `langgraph>=1.2.12`.

Pass a Pydantic model class, a `TypedDict` from `typing_extensions`, or a dataclass to validate resume values. LangGraph converts the type to JSON Schema and includes it in `Interrupt.response_schema`, separately from the interrupt's `value` payload. The schema is available on interrupts returned to the caller and in checkpoint task interrupts.

This example uses the approval schema from LangGraph's interrupt tests:

```
from typing import Any

from pydantic import BaseModel
from typing_extensions import TypedDict

from langgraph.checkpoint.memory import InMemorySaver
from langgraph.graph import START, StateGraph
from langgraph.types import Command, interrupt

class Decision(BaseModel):
 approved: bool
 note: str | None = None

class State(TypedDict):
 answer: Any

def node(state: State) -> State:
 answer = interrupt({"question": "approve?"}, response_schema=Decision)
 return {"answer": answer}

graph = (
 StateGraph(State)
 .add_node("node", node)
 .add_edge(START, "node")
 .compile(checkpointer=InMemorySaver())
)

config = {"configurable": {"thread_id": "1"}}

result = graph.invoke({"answer": None}, config)
pending = result["__interrupt__"][0]
assert pending.response_schema == Decision.model_json_schema()

result = graph.invoke(Command(resume={"approved": True}), config)
assert result["answer"] == Decision(approved=True)
```

With a typed schema, `interrupt()` returns the validated object: a Pydantic model instance, a dictionary for a `TypedDict`, or a dataclass instance. Pydantic's validation and coercion rules apply. Invalid input raises `pydantic.ValidationError`; you can retry with a corrected `Command(resume=...)` using the same thread ID.

You can also pass a JSON Schema dictionary as `response_schema`. LangGraph exposes that dictionary unchanged but does **not** validate resume values against it. When you omit `response_schema`, its value is `None`, and resume values pass through without schema validation.

js

The `responseSchema` option to `@[interrupt]` describes the input expected when you resume the graph. Clients can use the schema to render typed input forms.

### 참고

Interrupt response schemas require `@langchain/langgraph` version 1.4.16 or later.

Pass a Zod schema to validate resume values. LangGraph converts it to JSON Schema and includes it in `Interrupt.response_schema`, separately from the interrupt's `value` payload. The schema is available on interrupts returned to the caller and in checkpoint task interrupts.

This example uses the approval schema from LangGraph's interrupt tests:

```
import {
 Annotation,
 Command,
 MemorySaver,
 START,
 StateGraph,
 interrupt,
} from "@langchain/langgraph";
import { z } from "zod/v4";

const Decision = z.object({
 approved: z.boolean(),
 note: z.string().default(""),
});

const State = Annotation.Root({
 answer: Annotation<unknown>(),
});

const graph = new StateGraph(State)
 .addNode("node", () => ({
 answer: interrupt({ question: "approve?" }, { responseSchema: Decision }),
 }));
```

```

.addEdge(START, "node")
.compile({ checkpointer: new MemorySaver() });

const config = { configurable: { thread_id: "1" } };

const result = await graph.invoke({ answer: null }, config);
console.log(result.__interrupt__?.[0].response_schema);

const resumed = await graph.invoke(
 new Command({ resume: { approved: true } }),
 config
);
console.log(resumed.answer);

```

With a Zod schema, `interrupt()` returns the parsed value. In this example, the result is `{ approved: true, note: "" }`, including the default `note`. Invalid input raises `ZodError`; you can retry with a corrected `new Command({ resume: ... })` using the same thread ID.

You can also pass a JSON Schema object as `responseSchema`. `LangGraph` exposes that object unchanged but does **not** validate resume values against it. When you omit `responseSchema`, interrupts have no `response_schema` field, and resume values pass through without schema validation.

Studio renders interrupt response schemas as typed input fields instead of a JSON editor. This preview of the Studio component uses example data with approval, note, and numeric amount fields.



Studio interrupt form with an approval toggle, a note field, and a numeric amount set to 125.50

## Common patterns

The key thing that interrupts unlock is the ability to pause execution and wait for external input. This is useful for a variety of use cases, including:

-



**Approval workflows**: Pause before executing critical actions (API calls, database changes, financial transactions)

---

**Handling multiple interrupts**: Pair interrupt IDs with resume values when resuming multiple interrupts in a single invocation

---

**Review and edit**: Let humans review and modify LLM outputs or tool calls before continuing

---

**Interrupting tool calls**: Pause before executing tool calls to review and edit the tool call before execution

---

**Validating human input**: Pause before proceeding to the next step to validate human input

## Stream with human-in-the-loop (HITL) interrupts

When building interactive agents with human-in-the-loop workflows, you can use [event streaming](#) to consume message chunks and state snapshots concurrently while handling interrupts.

Use the typed projections returned by `graph.stream_events(..., version="v3")` in a loop until the run finishes:

- Stream AI responses token-by-token via `stream.messages`
- Observe per-step state snapshots via `stream.values`
- Detect interrupts via `stream.interrupted` and read their payloads from `stream.interrupts`
- Resume execution by calling `stream_events` again with `Command(resume=...)` and repeat until `stream.interrupted` is false

### python

- `stream.messages` : Chat-model output as content blocks; iterate each `message.text` for token deltas. For nested subgraphs, read message chunks from `stream.subgraphs[*].messages` .
- `stream.values` : Full state snapshots after each step
- `stream.interrupted` / `stream.interrupts` : After each run, check whether the graph paused; read payloads from `stream.interrupts`
- `Command(resume=...)` : Pass as the next `stream_events` input to resume; loop until the run completes without interrupting

### js

- `stream.messages` : Chat-model output as content blocks; iterate `message.text` for token deltas. For nested subgraphs, read message chunks from `stream.subgraphs[*].messages` .
- `stream.values` : Full state snapshots after each step

- `stream.interrupted / stream.interrupts` : After each run, check whether the graph paused; read payloads from `stream.interrupts`
- `Command(resume=...)` : Pass as the next `streamEvents` input to resume; loop until the run completes without interrupting

## Handling multiple interrupts

When parallel branches interrupt simultaneously (for example, fan-out to multiple nodes that each call `interrupt()`), you may need to resume multiple interrupts in a single invocation. When resuming multiple interrupts with a single invocation, map each interrupt ID to its resume value. This ensures each response is paired with the correct interrupt at runtime.

### python

### js

```
import {
 Annotation,
 Command,
 END,
 INTERRUPT,
 MemorySaver,
 START,
 StateGraph,
 interrupt,
 isInterrupted,
} from "@langchain/langgraph";

const State = Annotation.Root({
 vals: Annotation<string[]>({
 reducer: (left, right) =>
 left.concat(Array.isArray(right) ? right : [right]),
 default: () => [],
 }),
});

function nodeA(_state: typeof State.State) {
 const answer = interrupt("question_a") as string;
 return { vals: [`a:${answer}`] };
}

function nodeB(_state: typeof State.State) {
 const answer = interrupt("question_b") as string;
 return { vals: [`b:${answer}`] };
}

const graph = new StateGraph(State)
```

```

.addNode("a", nodeA)
.addNode("b", nodeB)
.addEdge(START, "a")
.addEdge(START, "b")
.addEdge("a", END)
.addEdge("b", END)
.compile({ checkpointer: new MemorySaver() });

const config = { configurable: { thread_id: "1" } };

async function main() {
 // Step 1: invoke - both parallel nodes hit interrupt() and pause
 const interruptedResult = await graph.invoke({ vals: [] }, config);
 console.log(interruptedResult);
 /*
 {
 vals: [],
 __interrupt__: [
 { id: '...', value: 'question_a' },
 { id: '...', value: 'question_b' }
]
 }
 */

 // Step 2: resume all pending interrupts at once
 const resumeMap: Record<string, string> = {};
 if (isInterrupted(interruptedResult)) {
 for (const i of interruptedResult[INTERRUPT]) {
 if (i.id !== null) {
 resumeMap[i.id] = `answer for ${i.value}`;
 }
 }
 }
 const result = await graph.invoke(new Command({ resume: resumeMap }), config);

 console.log("Final state:", result);
 //> Final state: { vals: ['a:answer for question_a', 'b:answer for question_b'] }
}

main().catch(console.error);

```

## Approve or reject

One of the most common uses of interrupts is to pause before a critical action and ask for approval. For example, you might want to ask a human to approve an API call, a database change, or any other important decision.

**python**

```

from typing import Literal
from langgraph.types import interrupt, Command

def approval_node(state: State) -> Command[Literal["proceed", "cancel"]]:
 # Pause execution; payload shows up on stream.interrupts (with stream_events) or result
 ["__interrupt__"] (with invoke)
 is_approved = interrupt({
 "question": "Do you want to proceed with this action?",
 "details": state["action_details"]
 })

 # Route based on the response
 if is_approved:
 return Command(goto="proceed") # Runs after the resume payload is provided
 else:
 return Command(goto="cancel")

```

js

```

import { interrupt, Command } from "@langchain/langgraph";

const approvalNode: typeof State.Node = (state) => {
 // Pause execution; payload surfaces in result.__interrupt__
 const isApproved = interrupt({
 question: "Do you want to proceed?",
 details: state.actionDetails
 });

 // Route based on the response
 if (isApproved) {
 return new Command({ goto: "proceed" }); // Runs after the resume payload is provided
 } else {
 return new Command({ goto: "cancel" });
 }
}

```

python

When you resume the graph, pass `True` to approve or `False` to reject:

```

To approve
graph.stream_events(Command(resume=True), config=config, version="v3").output

To reject
graph.stream_events(Command(resume=False), config=config, version="v3").output

```

js

When you resume the graph, pass `true` to approve or `false` to reject:

```
// To approve
await graph.invoke(new Command({ resume: true }), config);

// To reject
await graph.invoke(new Command({ resume: false }), config);
```

## Full example

python

js

```
import {
 Command,
 MemorySaver,
 START,
 END,
 StateGraph,
 StateSchema,
 interrupt,
} from "@langchain/langgraph";
import * as z from "zod";

const State = new StateSchema({
 actionDetails: z.string(),
 status: z.enum(["pending", "approved", "rejected"]).nullable(),
});

const graphBuilder = new StateGraph(State)
 .addNode("approval", async (state) => {
 // Expose details so the caller can render them in a UI
 const decision = interrupt({
 question: "Approve this action?",
 details: state.actionDetails,
 });
 return new Command({ goto: decision ? "proceed" : "cancel" });
 }, { ends: ['proceed', 'cancel'] })
 .addNode("proceed", () => ({ status: "approved" }))
 .addNode("cancel", () => ({ status: "rejected" }))
 .addEdge(START, "approval")
 .addEdge("proceed", END)
 .addEdge("cancel", END);

// Use a more durable checkpoint in production
```

```

const checkpointer = new MemorySaver();
const graph = graphBuilder.compile({ checkpointer });

const config = { configurable: { thread_id: "approval-123" } };
const initial = await graph.invoke(
 { actionDetails: "Transfer $500", status: "pending" },
 config,
);
console.log(initial.__interrupt__);
// [{ value: { question: ..., details: ... } }]

// Resume with the decision; true routes to proceed, false to cancel
const resumed = await graph.invoke(new Command({ resume: true }), config);
console.log(resumed.status); // -> "approved"

```

## Review and edit state

Sometimes you want to let a human review and edit part of the graph state before continuing. This is useful for correcting LLMs, adding missing information, or making adjustments.

### python

```

from langgraph.types import interrupt

def review_node(state: State):
 # Pause and show the current content for review (payload surfaces on stream.interrupts)
 edited_content = interrupt({
 "instruction": "Review and edit this content",
 "content": state["generated_text"]
 })

 # Update the state with the edited version
 return {"generated_text": edited_content}

```

### js

```

import { interrupt } from "@langchain/langgraph";

const reviewNode: typeof State.Node = (state) => {
 // Pause and show the current content for review (surfaces in result.__interrupt__)
 const editedContent = interrupt({
 instruction: "Review and edit this content",
 content: state.generatedText
 });

 // Update the state with the edited version

```

```
 return { generatedText: editedContent };
}
```

When resuming, provide the edited content:

### python

```
graph.stream_events(
 Command(resume="The edited and improved text"), # Value becomes the return from interrupt()
 config=config,
 version="v3",
).output
```

### js

```
await graph.invoke(
 new Command({ resume: "The edited and improved text" }), // Value becomes the return from interrupt()
 config
);
```

## Full example

### python

### js

```
import {
 Command,
 MemorySaver,
 START,
 END,
 StateGraph,
 StateSchema,
 interrupt,
} from "@langchain/langgraph";
import * as z from "zod";

const State = new StateSchema({
 generatedText: z.string(),
});

const builder = new StateGraph(State)
 .addNode("review", async (state) => {
```

```

 // Ask a reviewer to edit the generated content
 const updated = interrupt({
 instruction: "Review and edit this content",
 content: state.generatedText,
 });
 return { generatedText: updated };
 })
 .addEdge(START, "review")
 .addEdge("review", END);

const checkpointer = new MemorySaver();
const graph = builder.compile({ checkpointer });

const config = { configurable: { thread_id: "review-42" } };
const initial = await graph.invoke({ generatedText: "Initial draft" }, config);
console.log(initial.__interrupt__);
// [{ value: { instruction: ..., content: ... } }]

// Resume with the edited text from the reviewer
const finalState = await graph.invoke(
 new Command({ resume: "Improved draft after review" }),
 config,
);
console.log(finalState.generatedText); // -> "Improved draft after review"

```

## Interrupts in tools

You can also place interrupts directly inside tool functions. This makes the tool itself pause for approval whenever it's called, and allows for human review and editing of the tool call before it is executed.

First, define a tool that uses `@interrupt`:

### python

```

from langchain.tools import tool
from langgraph.types import interrupt

@tool
def send_email(to: str, subject: str, body: str):
 """Send an email to a recipient."""

 # Pause before sending; payload surfaces on stream.interrupts when using event streaming
 response = interrupt({
 "action": "send_email",
 "to": to,
 "subject": subject,
 "body": body,
 })

```



```

 "message": "Approve sending this email?"
 })

 if response.get("action") == "approve":
 # Resume value can override inputs before executing
 final_to = response.get("to", to)
 final_subject = response.get("subject", subject)
 final_body = response.get("body", body)
 return f"Email sent to {final_to} with subject '{final_subject}'"
 return "Email cancelled by user"

```

js

```

import { tool } from "@langchain/core/tools";
import { interrupt } from "@langchain/langgraph";
import * as z from "zod";

const sendEmailTool = tool(
 async ({ to, subject, body }) => {
 // Pause before sending; payload surfaces in result.__interrupt__
 const response = interrupt({
 action: "send_email",
 to,
 subject,
 body,
 message: "Approve sending this email?",
 });

 if (response?.action === "approve") {
 // Resume value can override inputs before executing
 const finalTo = response.to ?? to;
 const finalSubject = response.subject ?? subject;
 const finalBody = response.body ?? body;
 return `Email sent to ${finalTo} with subject '${finalSubject}'`;
 }
 return "Email cancelled by user";
 },
 {
 name: "send_email",
 description: "Send an email to a recipient",
 schema: z.object({
 to: z.string(),
 subject: z.string(),
 body: z.string(),
 }),
 },
);

```

This approach is useful when you want the approval logic to live with the tool itself, making it reusable across different parts of your graph. The LLM can call the tool naturally, and the interrupt will pause execution whenever the tool is invoked, allowing you to approve, edit, or cancel the action.

## Full example

### python

```
import sqlite3
import operator
from typing import TypedDict, Annotated, Literal
from langchain.tools import tool
from langchain_anthropic import ChatAnthropic
from langgraph.checkpoint.sqlite import SqliteSaver
from langgraph.graph import StateGraph, START, END
from langgraph.types import Command, interrupt
from langchain.messages import AnyMessage, SystemMessage, ToolMessage

class AgentState(TypedDict):
 messages: Annotated[list[AnyMessage], operator.add]

@tool
def send_email(to: str, subject: str, body: str):
 """Send an email to a recipient."""

 # Pause before sending; payload surfaces on stream.interrupts when using event streaming
 response = interrupt({
 "action": "send_email",
 "to": to,
 "subject": subject,
 "body": body,
 "message": "Approve sending this email?",
 })

 if response.get("action") == "approve":
 final_to = response.get("to", to)
 final_subject = response.get("subject", subject)
 final_body = response.get("body", body)

 # Actually send the email (your implementation here)
 print(f"[send_email] to={final_to} subject={final_subject} body={final_body}")
 return f"Email sent to {final_to}"

 return "Email cancelled by user"

model = ChatAnthropic(model="claude-sonnet-4-6").bind_tools([send_email])
tools_by_name = {"send_email": send_email}

def agent_node(state: AgentState):
```

```

LLM may decide to call the tool; interrupt pauses before sending
result = model.invoke(state["messages"])
return {"messages": [result]}

def tool_node(state: AgentState):
 """Performs the tool call"""
 result = []
 for tool_call in state["messages"][-1].tool_calls:
 tool = tools_by_name[tool_call["name"]]
 observation = tool.invoke(tool_call["args"])
 result.append(ToolMessage(content=observation, tool_call_id=tool_call["id"]))
 return {"messages": result}

def should_continue(state: AgentState) -> Literal["tool_node", END]:
 """Decide if we should continue the loop or stop based upon whether the LLM made a tool
 call"""
 messages = state["messages"]
 last_message = messages[-1]

 if last_message.tool_calls:
 return "tool_node"
 return END

builder = StateGraph(AgentState)
builder.add_node("agent", agent_node)
builder.add_node("tool_node", tool_node)

builder.add_edge(START, "agent")
builder.add_conditional_edges("agent", should_continue, ["tool_node", END]) # Routes to "to
ols" or END
builder.add_edge("tool_node", "agent") # Loop back after tools

checkpointer = SqliteSaver(
 sqlite3.connect("tool-approval.db", check_same_thread=False)
)
graph = builder.compile(checkpointer=checkpointer)

config = {"configurable": {"thread_id": "email-workflow"}}
initial = graph.stream_events(
 {
 "messages": [
 {"role": "user", "content": "Send an email to alice@example.com about the meetin
g"}
],
 },
 config=config,
 version="v3",
)
initial.output # drive the stream to completion
print(initial.interrupts) # -> (Interrupt(value={'action': 'send_email', ...}),)

```

```
Resume with approval and optionally edited arguments
resumed = graph.stream_events(
 Command(resume={"action": "approve", "subject": "Updated subject"}),
 config=config,
 version="v3",
)
print(resumed.output["messages"][-1]) # -> Tool result returned by send_email
```

js

```
import { tool } from "@langchain/core/tools";
import { ChatAnthropic } from "@langchain/anthropic";
import {
 Command,
 MemorySaver,
 START,
 END,
 StateGraph,
 StateSchema,
 MessagesValue,
 GraphNode,
 interrupt,
} from "@langchain/langgraph";
import * as z from "zod";

const sendEmailTool = tool(
 async ({ to, subject, body }) => {
 // Pause before sending; payload surfaces in result.__interrupt__
 const response = interrupt({
 action: "send_email",
 to,
 subject,
 body,
 message: "Approve sending this email?",
 });

 if (response?.action === "approve") {
 const finalTo = response.to ?? to;
 const finalSubject = response.subject ?? subject;
 const finalBody = response.body ?? body;
 console.log("[sendEmailTool]", finalTo, finalSubject, finalBody);
 return `Email sent to ${finalTo}`;
 }
 return "Email cancelled by user";
 },
 {
 name: "send_email",
 description: "Send an email to a recipient",
 schema: z.object({
```

```

 to: z.string(),
 subject: z.string(),
 body: z.string(),
 })),
 },
);

const model = new ChatAnthropic({ model: "claude-sonnet-4-6" }).bindTools([sendEmailTool]);

const State = new StateSchema({
 messages: MessagesValue,
});

const agent: typeof State.Node = async (state) => {
 // LLM may decide to call the tool; interrupt pauses before sending
 const response = await model.invoke(state.messages);
 return { messages: [response] };
};

const graphBuilder = new StateGraph(State)
 .addNode("agent", agent)
 .addEdge(START, "agent")
 .addEdge("agent", END);

const checkpointer = new MemorySaver();
const graph = graphBuilder.compile({ checkpointer });

const config = { configurable: { thread_id: "email-workflow" } };
const initial = await graph.invoke(
 {
 messages: [
 { role: "user", content: "Send an email to alice@example.com about the meeting" },
],
 },
 config,
);
console.log(initial.__interrupt__); // -> [{ value: { action: 'send_email', ... } }]

// Resume with approval and optionally edited arguments
const resumed = await graph.invoke(
 new Command({
 resume: { action: "approve", subject: "Updated subject" },
 }),
 config,
);
console.log(resumed.messages.at(-1)); // -> Tool result returned by send_email

```

## Validating human input

Sometimes you need to validate input from humans and re-prompt if the value is invalid. The recommended approach is to call `interrupt()` **once per node invocation**, return from the node with the error message stored in state, and use a **conditional edge** to loop back to the node until a valid value is provided.

### 주의

**Avoid `while True + interrupt()` loops inside a single node.** Because the node re-runs from the beginning on every resume (see [Rules of interrupts](#)), a loop that calls `interrupt()` multiple times causes each resume to replay all previous iterations: the first resume replays 1 iteration, the second replays 2, and so on. The result is exponential re-execution of any code inside the loop body.

### python

The correct pattern:

1. Store the re-prompt question in state (e.g. `pending_question`).
2. In the node, call `interrupt()` **exactly once**, passing the current question from state.
3. If the answer is invalid, return the updated `pending_question` so the next invocation re-prompts.
4. Use `add_conditional_edges` to route back to the node until a valid value is collected.

Each resume invokes `get_age_node` exactly once, runs the `interrupt()` call once, and exits. When the answer is invalid, the conditional edge loops back and the next interrupt re-prompts with the updated question. No code runs more than once per resume.

### js

The correct pattern:

1. Store the re-prompt question in state (e.g. `pendingQuestion`).
2. In the node, call `interrupt()` **exactly once**, passing the current question from state.
3. If the answer is invalid, return the updated `pendingQuestion` so the next invocation re-prompts.
4. Use `addConditionalEdges` to route back to the node until a valid value is collected.

Each resume invokes `getAgeNode` exactly once, runs the `interrupt()` call once, and exits. When the answer is invalid, the conditional edge loops back and the next interrupt re-prompts with the updated question.

## Full example

```

import {
 Command,
 MemorySaver,
 START,
 END,
 StateGraph,
 StateSchema,
 interrupt,
} from "@langchain/langgraph";
import * as z from "zod";

const State = new StateSchema({
 age: z.number().nullable(),
 pendingQuestion: z.string().nullable(),
});

const builder = new StateGraph(State)
 .addNode("collectAge", (state) => {
 const question = state.pendingQuestion ?? "What is your age?";
 const answer = interrupt(question); // called exactly once per invocation

 if (typeof answer === "number" && answer > 0) {
 return { age: answer, pendingQuestion: null };
 }
 return { pendingQuestion: ` '${answer}' is not a valid age. Please enter a positive number.` };
 })
 .addEdge(START, "collectAge")
 .addConditionalEdges("collectAge", (state) =>
 state.age !== null ? END : "collectAge"
);

const checkpointer = new MemorySaver();
const graph = builder.compile({ checkpointer });

const config = { configurable: { thread_id: "form-1" } };
const first = await graph.invoke({ age: null, pendingQuestion: null }, config);
console.log(first.__interrupt__); // -> [{ value: "What is your age?", ... }]

// Provide invalid data; the node re-prompts via the conditional edge
const retry = await graph.invoke(new Command({ resume: "thirty" }), config);
console.log(retry.__interrupt__); // -> [{ value: "'thirty' is not a valid age...", ... }]

// Provide valid data; route returns END and the graph finishes

```

```
const final = await graph.invoke(new Command({ resume: 30 })), config);
console.log(final.age); // -> 30
```

## Rules of interrupts

When you call `@[ interrupt ]` within a node, LangGraph suspends execution by raising an exception that signals the runtime to pause. This exception propagates up through the call stack and is caught by the runtime, which notifies the graph to save the current state and wait for external input.

When execution resumes (after you provide the requested input), the runtime restarts the entire node from the beginning—it does not resume from the exact line where `@[ interrupt ]` was called. This means any code that ran before the `@[ interrupt ]` will execute again. Because of this, there's a few important rules to follow when working with interrupts to ensure they behave as expected.

### python

#### Do not wrap `interrupt` calls in try/except

The way that `@[ interrupt ]` pauses execution at the point of the call is by throwing a special exception. If you wrap the `@[ interrupt ]` call in a try/except block, you will catch this exception and the interrupt will not be passed back to the graph.

-  Separate `@[ interrupt ]` calls from error-prone code
-  Use specific exception types in try/except blocks

```
```python Separating logic def node_a(state: State): #  Good: interrupting first, then
handling # error conditions separately interrupt("What's your name?") try: fetch_data() # This
can fail except Exception as e: print(e) return state
```

```
```python Explicit exception handling
def node_a(state: State):
 #  Good: catching specific exception types
 # will not catch the interrupt exception
 try:
 name = interrupt("What's your name?")
 fetch_data() # This can fail
 except NetworkException as e:
 print(e)
 return state
```

-  Do not wrap `@[ interrupt ]` calls in bare try/except blocks

```
def node_a(state: State):
 #  Bad: wrapping interrupt in bare try/except
 # will catch the interrupt exception
 try:
```



```

 interrupt("What's your name?")
 except Exception as e:
 print(e)
 return state

```

js

## Do not wrap `interrupt` calls in try/catch

The way that `@[ interrupt ]` pauses execution at the point of the call is by throwing a special exception. If you wrap the `@[ interrupt ]` call in a try/catch block, you will catch this exception and the interrupt will not be passed back to the graph.

-  Separate `@[ interrupt ]` calls from error-prone code
-  Conditionally catch errors if needed

```

```typescript
Separating logic
const nodeA: GraphNode = async (state) => { //  Good:
  interrupting first, then handling error conditions separately
  const name = interrupt("What's your name?");
  try { await fetchData(); // This can fail } catch (err) { console.error(error); }
  return state; }

```

```

```typescript
Conditionally handling errors
const nodeA: GraphNode<typeof State> = async (state) => {
 //  Good: re-throwing the exception will
 // allow the interrupt to be passed back to
 // the graph
 try {
 const name = interrupt("What's your name?");
 await fetchData(); // This can fail
 } catch (err) {
 if (error instanceof NetworkError) {
 console.error(error);
 }
 throw error;
 }
 return state;
}

```

-  Do not wrap `@[ interrupt ]` calls in bare try/catch blocks

```

async function nodeA(state: State) {
 //  Bad: wrapping interrupt in bare try/catch will catch the interrupt exception
 try {
 const name = interrupt("What's your name?");
 } catch (err) {
 console.error(error);
 }
}

```

```
 return state;
}
```

## Do not reorder `interrupt` calls within a node

It's common to use multiple interrupts in a single node, however this can lead to unexpected behavior if not handled carefully.

When a node contains multiple interrupt calls, LangGraph keeps a list of resume values specific to the task executing the node. Whenever execution resumes, it starts at the beginning of the node. For each interrupt encountered, LangGraph checks if a matching value exists in the task's resume list. Matching is **strictly index-based**, so the order of interrupt calls within the node is important.

-  Keep `@[interrupt]` calls consistent across node executions

### python

```
def node_a(state: State):
 #  Good: interrupt calls happen in the same order every time
 name = interrupt("What's your name?")
 age = interrupt("What's your age?")
 city = interrupt("What's your city?")

 return {
 "name": name,
 "age": age,
 "city": city
 }
```

### js

```
async function nodeA(state: State) {
 //  Good: interrupt calls happen in the same order every time
 const name = interrupt("What's your name?");
 const age = interrupt("What's your age?");
 const city = interrupt("What's your city?");

 return {
 name,
 age,
 city
 };
}
```

-  Do not conditionally skip `@[interrupt]` calls within a node

-  Do not loop @`[ interrupt ]` calls using logic that isn't deterministic across executions, including `while True` validation loops. Use a conditional edge instead (see [Validating human input](#))

## python

```
```python Skipping interrupts def node_a(state: State): #  Bad: conditionally skipping interrupts changes the order name = interrupt("What's your name?")
```

On first run, this might skip the interrupt

On resume, it might not skip it - causing index mismatch

```
if state.get("needs_age"): age = interrupt("What's your age?")
```

```
city = interrupt("What's your city?")
```

```
return {"name": name, "city": city}
```

```
```python Looping interrupts
def node_a(state: State):
 #  Bad: looping based on non-deterministic data
 # The number of interrupts changes between executions
 results = []
 for item in state.get("dynamic_list", []): # List might change between runs
 result = interrupt(f"Approve {item}?")
 results.append(result)

 return {"results": results}
```

## js

```
```typescript Skipping interrupts const nodeA: GraphNode = async (state) => { //  Bad: conditionally skipping interrupts changes the order const name = interrupt("What's your name?");
```

```
// On first run, this might skip the interrupt // On resume, it might not skip it - causing index mismatch if (state.needsAge) { const age = interrupt("What's your age?"); }
```

```
const city = interrupt("What's your city?");
```

```
return { name, city }; }
```

```

```typescript Looping interrupts
const nodeA: GraphNode<typeof State> = async (state) => {
 // ❌ Bad: looping based on non-deterministic data
 // The number of interrupts changes between executions
 const results = [];
 for (const item of state.dynamicList || []) { // List might change between runs
 const result = interrupt(`Approve ${item}?`);
 results.push(result);
 }

 return { results };
}

```

## Do not return complex values in `interrupt` calls

Depending on which checkpointer is used, complex values may not be serializable (e.g. you can't serialize a function). To make your graphs adaptable to any deployment, it's best practice to only use values that can be reasonably serialized.

- ✅ Pass simple, JSON-serializable types to `@[interrupt]`
- ✅ Pass dictionaries/objects with simple values

### python

```

```python Simple values
def node_a(state: State): # ✅ Good: passing simple types that are
    name = interrupt("What's your name?")
    count = interrupt(42)
    approved = interrupt(True)

```

```

    return {"name": name, "count": count, "approved": approved}

```

```

```python Structured data
def node_a(state: State):
 # ✅ Good: passing dictionaries with simple values
 response = interrupt({
 "question": "Enter user details",
 "fields": ["name", "email", "age"],
 "current_values": state.get("user", {})
 })

 return {"user": response}

```

### js

```

```typescript Simple values
const nodeA: GraphNode = async (state) => { // ✅ Good: passing
    simple types that are serializable
    const name = interrupt("What's your name?");
    const count =

```

```
interrupt(42); const approved = interrupt(true);

return { name, count, approved }; }
```

```
```typescript Structured data
const nodeA: GraphNode<typeof State> = async (state) => {
 // ✅ Good: passing objects with simple values
 const response = interrupt({
 question: "Enter user details",
 fields: ["name", "email", "age"],
 currentValues: state.user || {}
 });

 return { user: response };
}
```

- 🚫 Do not pass functions, class instances, or other complex objects to `@[ interrupt ]`

## python

```
```python Functions def validate_input(value): return len(value) > 0
```

```
def node_a(state: State): # ❌ Bad: passing a function to interrupt # The function cannot be
serialized response = interrupt({ "question": "What's your name?", "validator": validate_input #
This will fail }) return {"name": response}
```

```
```python Class instances
class DataProcessor:
 def __init__(self, config):
 self.config = config

def node_a(state: State):
 processor = DataProcessor({"mode": "strict"})

 # ❌ Bad: passing a class instance to interrupt
 # The instance cannot be serialized
 response = interrupt({
 "question": "Enter data to process",
 "processor": processor # This will fail
 })
 return {"result": response}
```

## js

```
```typescript Functions function validateInput(value: string): boolean { return value.length > 0;
}
```

```
const nodeA: GraphNode = async (state) => { // ❌ Bad: passing a function to interrupt // The
function cannot be serialized const response = interrupt({ question: "What's your name?",
validator: validateInput // This will fail }); return { name: response }; }
```

```
```typescript Class instances
class DataProcessor {
 constructor(private config: any) {}
}

const nodeA: GraphNode<typeof State> = async (state) => {
 const processor = new DataProcessor({ mode: "strict" });

 // ❌ Bad: passing a class instance to interrupt
 // The instance cannot be serialized
 const response = interrupt({
 question: "Enter data to process",
 processor: processor // This will fail
 });
 return { result: response };
}
```

## Side effects called before `interrupt` must be idempotent

Because interrupts work by re-running the nodes they were called from, side effects called before `@[ interrupt ]` should (ideally) be idempotent. For context, idempotency means that the same operation can be applied multiple times without changing the result beyond the initial execution.

As an example, you might have an API call to update a record inside of a node. If `@[ interrupt ]` is called after that call is made, it will be re-run multiple times when the node is resumed, potentially overwriting the initial update or creating duplicate records.

- ✅ Use idempotent operations before `@[ interrupt ]`
- ✅ Place side effects after `@[ interrupt ]` calls
- ✅ Separate side effects into separate nodes when possible

### python

```
```python Idempotent operations def node_a(state: State): # ✅ Good: using upsert operation
which is idempotent # Running this multiple times will have the same result db.upsert_user(
user_id=state["user_id"], status="pending_approval" )
```

```
approved = interrupt("Approve this change?")
```

```
return {"approved": approved}
```

```

python Side effects after interrupt
def node_a(state: State):
    # ✅ Good: placing side effect after the interrupt
    # This ensures it only runs once after approval is received
    approved = interrupt("Approve this change?")

    if approved:
        db.create_audit_log(
            user_id=state["user_id"],
            action="approved"
        )

    return {"approved": approved}

```

```

python Separating into different nodes
def approval_node(state: State): # ✅ Good: only
    handling the interrupt in this node
    approved = interrupt("Approve this change?")

```

```

    return {"approved": approved}

```

```

def notification_node(state: State): # ✅ Good: side effect happens in a separate node # This
    runs after approval, so it only executes once if (state.approved): send_notification(
        user_id=state["user_id"], status="approved" )

```

```

    return state

```

</div>

<div class="callout js" markdown="1">

<p class="callout-label">js</p>

```

typescript Idempotent operations
const nodeA: GraphNode<typeof State> = async (state) => {
    // ✅ Good: using upsert operation which is idempotent
    // Running this multiple times will have the same result
    await db.upsertUser({
        userId: state.userId,
        status: "pending_approval"
    });

    const approved = interrupt("Approve this change?");

    return { approved };
}

```

```

typescript Side effects after interrupt
const nodeA: GraphNode = async (state) => { // ✅
    Good: placing side effect after the interrupt // This ensures it only runs once after approval is

```

```
received const approved = interrupt("Approve this change?");

if (approved) { await db.createAuditLog({ userId: state.userId, action: "approved" }); }

return { approved }; }
```

```
```typescript Separating into different nodes
const approvalNode: GraphNode<typeof State> = async (state) => {
 // ✅ Good: only handling the interrupt in this node
 const approved = interrupt("Approve this change?");

 return { approved };
}

const notificationNode: GraphNode<typeof State> = async (state) => {
 // ✅ Good: side effect happens in a separate node
 // This runs after approval, so it only executes once
 if (state.approved) {
 await sendNotification({
 userId: state.userId,
 status: "approved",
 });
 }

 return state;
}
```

- Do not perform non-idempotent operations before `@[ interrupt ]`
- Do not create new records without checking if they exist

## python

```
```python Creating records def node_a(state: State): # ❌ Bad: creating a new record before
interrupt # This will create duplicate records on each resume audit_id = db.create_audit_log({
"user_id": state["user_id"], "action": "pending_approval", "timestamp": datetime.now() })
```

```
approved = interrupt("Approve this change?")
```

```
return {"approved": approved, "audit_id": audit_id}
```

```
```python Appending to lists
def node_a(state: State):
 # ❌ Bad: appending to a list before interrupt
 # This will add duplicate entries on each resume
 db.append_to_history(state["user_id"], "approval_requested")

 approved = interrupt("Approve this change?")
```



```
return {"approved": approved}
```

js

```
````typescript Creating records const nodeA: GraphNode = async (state) => { // ❌ Bad: creating
a new record before interrupt // This will create duplicate records on each resume const auditId
= await db.createAuditLog({ userId: state.userId, action: "pending_approval", timestamp: new
Date() });
```

```
const approved = interrupt("Approve this change?");
```

```
return { approved, auditId }; }
```

```
````typescript Appending to arrays
const nodeA: GraphNode<typeof State> = async (state) => {
 // ❌ Bad: appending to an array before interrupt
 // This will add duplicate entries on each resume
 await db.appendToHistory(state.userId, "approval_requested");

 const approved = interrupt("Approve this change?");

 return { approved };
}
```

## Using with subgraphs called as functions

When invoking a subgraph within a node, the parent graph will resume execution from the **beginning of the node** where the subgraph was invoked and the `@[interrupt]` was triggered. Similarly, the **subgraph** will also resume from the beginning of the node where `@[interrupt]` was called.

python

```
def node_in_parent_graph(state: State):
 some_code() # <-- This will re-execute when resumed
 # Invoke a subgraph as a function.
 # The subgraph contains an `interrupt` call.
 subgraph_result = subgraph.invoke(some_input)
 # ...

def node_in_subgraph(state: State):
 some_other_code() # <-- This will also re-execute when resumed
```

```
result = interrupt("What's your name?")
...
```

js

```
async function nodeInParentGraph(state: State) {
 someCode(); // <-- This will re-execute when resumed
 // Invoke a subgraph as a function.
 // The subgraph contains an `interrupt` call.
 const subgraphResult = await subgraph.invoke(someInput);
 // ...
}

async function nodeInSubgraph(state: State) {
 someOtherCode(); // <-- This will also re-execute when resumed
 const result = interrupt("What's your name?");
 // ...
}
```

## Debugging with interrupts

python

To debug and test a graph, you can use static interrupts as breakpoints to step through the graph execution one node at a time. Static interrupts are triggered at defined points either before or after a node executes. You can set these by specifying `interrupt_before` and `interrupt_after` when compiling the graph.

js

To debug and test a graph, you can use static interrupts as breakpoints to step through the graph execution one node at a time. Static interrupts are triggered at defined points either before or after a node executes. You can set these by specifying `interruptBefore` and `interruptAfter` when compiling the graph.

참고

Static interrupts are **not** recommended for human-in-the-loop workflows. Use the `@[ interrupt ]` function instead.

**At compile time**

## python

```
graph = builder.compile(
 interrupt_before=["node_a"], # [!code highlight]
 interrupt_after=["node_b", "node_c"], # [!code highlight]
 checkpointer=checkpointer,
)

Pass a thread ID to the graph
config = {
 "configurable": {
 "thread_id": "some_thread"
 }
}

Run the graph until the breakpoint
graph.invoke(inputs, config=config) # [!code highlight]

Resume the graph
graph.invoke(None, config=config) # [!code highlight]
```

1. The breakpoints are set during `compile` time.
2. `interrupt_before` specifies the nodes where execution should pause before the node is executed.
3. `interrupt_after` specifies the nodes where execution should pause after the node is executed.
4. A checkpointer is required to enable breakpoints.
5. The graph is run until the first breakpoint is hit.
6. The graph is resumed by passing in `None` for the input. This will run the graph until the next breakpoint is hit.

## js

```
const graph = builder.compile({
 interruptBefore: ["node_a"], // [!code highlight]
 interruptAfter: ["node_b", "node_c"], // [!code highlight]
 checkpointer,
});

// Pass a thread ID to the graph
const config = {
 configurable: {
 thread_id: "some_thread"
 }
};

// Run the graph until the breakpoint
```

```
await graph.invoke(inputs, config);# [!code highlight]
```

```
await graph.invoke(null, config); # [!code highlight]
```

1. The breakpoints are set during `compile` time.
2. `interruptBefore` specifies the nodes where execution should pause before the node is executed.
3. `interruptAfter` specifies the nodes where execution should pause after the node is executed.
4. A checkpointer is required to enable breakpoints.
5. The graph is run until the first breakpoint is hit.
6. The graph is resumed by passing in `null` for the input. This will run the graph until the next breakpoint is hit.

## At run time

### python

```
config = {
 "configurable": {
 "thread_id": "some_thread"
 }
}

Run the graph until the breakpoint
graph.invoke(
 inputs,
 interrupt_before=["node_a"], # [!code highlight]
 interrupt_after=["node_b", "node_c"], # [!code highlight]
 config=config,
)

Resume the graph
graph.invoke(None, config=config) # [!code highlight]
```

1. `graph.invoke` is called with the `interrupt_before` and `interrupt_after` parameters. This is a run-time configuration and can be changed for every invocation.
2. `interrupt_before` specifies the nodes where execution should pause before the node is executed.
3. `interrupt_after` specifies the nodes where execution should pause after the node is executed.
4. The graph is run until the first breakpoint is hit.
5. The graph is resumed by passing in `None` for the input. This will run the graph until the next breakpoint is hit.

js

```
// Run the graph until the breakpoint
graph.invoke(inputs, {
 interruptBefore: ["node_a"], // [!code highlight]
 interruptAfter: ["node_b", "node_c"], // [!code highlight]
 configurable: {
 thread_id: "some_thread"
 }
});

// Resume the graph
await graph.invoke(null, config); // [!code highlight]
```

1. `graph.invoke` is called with the `interruptBefore` and `interruptAfter` parameters. This is a run-time configuration and can be changed for every invocation.
2. `interruptBefore` specifies the nodes where execution should pause before the node is executed.
3. `interruptAfter` specifies the nodes where execution should pause after the node is executed.
4. The graph is run until the first breakpoint is hit.
5. The graph is resumed by passing in `null` for the input. This will run the graph until the next breakpoint is hit.

#### 팁

To debug your interrupts, use [LangSmith](#).

## Using LangSmith Studio

You can use [LangSmith Studio](#) to set static interrupts in your graph in the UI before running the graph. You can also use the UI to inspect the graph state at any point in the execution.

image

# Run a local server

---

langgraph/local-server.mdx

This guide shows you how to run a LangGraph application locally.

## Prerequisites

---

Before you begin, ensure you have the following:

- An API key for [LangSmith](#) - free to sign up

## 1. Install the LangGraph CLI

---

**python**

```
```bash pip
```

Python >= 3.11 is required.

```
pip install -U "langgraph-cli[inmem]"
```

```
```bash uv
Python >= 3.11 is required.
uv add "langgraph-cli[inmem]"
```

**js**

```
npm install --save-dev @langchain/langgraph-cli
```

## 2. Create a LangGraph app

---

**python**

Create a new app from the [new-langgraph-project-python template](#). This template demonstrates a single-node application you can extend with your own logic.

```
langgraph new path/to/your/app --template new-langgraph-project-python
```

**팁**

**Additional templates** If you use `langgraph new` without specifying a template, you will be presented with an interactive menu that will allow you to choose from a list of available templates.

js

Create a new app from the [new-langgraph-project-js template](#). This template demonstrates a single-node application you can extend with your own logic.

```
npm create langgraph
```

### Adding LangGraph to an existing project

If you have an existing project with LangGraph agents, you can automatically generate a `langgraph.json` configuration file using the `config` command:

```
npm create langgraph config
```

This command scans your project for LangGraph agents (such as `createAgent()`, `StateGraph.compile()`, or `workflow.compile()` patterns) and generates a configuration file with all exported agents.

Example output:

```
{
 "node_version": "24",
 "graphs": {
 "agent": "./src/agent.ts:agent",
 "searchAgent": "./src/search.ts:searchAgent"
 },
 "env": ".env"
}
```

#### 팁

Only **exported** agents are included in the configuration. If an agent is not exported, the command will warn you so you can add the `export` keyword.

## 3. Install dependencies

In the root of your new LangGraph app, install the dependencies in `edit` mode so your local changes are used by the server:

python

```
```bash pip cd path/to/your/app pip install -e .
```

```
``bash uv
cd path/to/your/app
uv sync
```

js

```
cd path/to/your/app
npm install
```

4. Create a `.env` file

You will find a `.env.example` in the root of your new LangGraph app. Create a `.env` file in the root of your new LangGraph app and copy the contents of the `.env.example` file into it, filling in the necessary API keys:

LANGSMITH_API_KEY=1sv2...

5. Launch Agent server

Start the LangGraph API server locally:

python

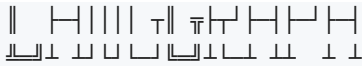
```
langgraph dev
```

js

```
npx @langchain/langgraph-cli dev
```

Sample output:

```
INFO:langgraph_api.cli:
Welcome to
```

- 🚀 API: `http://127.0.0.1:2024`
- 🎨 Studio UI: `https://smith.langchain.com/studio/?baseUrl=http://127.0.0.1:2024`
- 📖 API Docs: `http://127.0.0.1:2024/docs`

This in-memory server is designed for development and testing.
For production use, please use LangSmith Deployment.

The `langgraph dev` command starts Agent Server in an in-memory mode. This mode is suitable for development and testing purposes. For production use, deploy Agent Server with access to a persistent storage backend. For more information, see the [Platform setup overview](#).

6. Test your application in Studio

[Studio](#) is a specialized UI that you can connect to LangGraph API server to visualize, interact with, and debug your application locally. Test your graph in Studio by visiting the URL provided in the output of the `langgraph dev` command:

```
> - LangGraph Studio Web UI: https://smith.langchain.com/studio/?baseUrl=http://127.0.0.1:2024
```

For an Agent Server running on a custom host/port, update the `baseUrl` query parameter in the URL. For example, if your server is running on `http://myhost:3000`:

```
https://smith.langchain.com/studio/?baseUrl=http://myhost:3000
```

Safari compatibility

Use the `--tunnel` flag with your command to create a secure tunnel, as Safari has limitations when connecting to localhost servers:

```
langgraph dev --tunnel
```

7. Test the API

python

Python SDK (async)

1. Install the LangGraph Python SDK: `shell pip install langgraph-sdk`
2. Send a message to the assistant (threadless run): `python from langgraph_sdk import get_client import asyncio`

```
client = get_client(url="http://localhost:2024")
```

```
async def main(): async for chunk in client.runs.stream( None, # Threadless run "agent", #  
Name of assistant. Defined in langgraph.json. input={ "messages": [{ "role": "human",  
"content": "What is LangGraph?", }, ], }, ): print(f"Receiving new event of type: {chunk.event}...")  
print(chunk.data) print("\n\n")
```

```
asyncio.run(main())
```

****Python SDK (sync)****

1. Install the LangGraph Python SDK:

```
```shell  
pip install langgraph-sdk
```
```

2. Send a message to the assistant (threadless run):

```
```python  
from langgraph_sdk import get_sync_client
```

```
client = get_sync_client(url="http://localhost:2024")
```

```
for chunk in client.runs.stream(
 None, # Threadless run
 "agent", # Name of assistant. Defined in langgraph.json.
 input={
 "messages": [{
 "role": "human",
 "content": "What is LangGraph?",
 }],
 },
 stream_mode="messages-tuple",
):
 print(f"Receiving new event of type: {chunk.event}...")
 print(chunk.data)
 print("\n\n")
```

## Rest API

```
curl -s --request POST \
 --url "http://localhost:2024/runs/stream" \
 --header 'Content-Type: application/json' \
 --data "{
 \"assistant_id\": \"agent\",
 \"input\": {
 \"messages\": [
 {
 \"role\": \"human\",
 \"content\": \"What is LangGraph?\"
 }
]
 }
 }"
```

```

 }
]
},
\"stream_mode\": \"messages-tuple\"
}"

```

js

## JavaScript SDK

1. Install the LangGraph JS SDK: `shell npm install @langchain/langgraph-sdk`
2. Send a message to the assistant (threadless run): ````js import { Client } from "@langchain/langgraph-sdk";`

// only set the apiUrl if you changed the default port when calling langgraph dev const client = new Client({ apiUrl: "http://localhost:2024"});

```
const streamResponse = client.runs.stream(null, // Threadless run "agent", // Assistant ID {
input: { "messages": [{ "role": "user", "content": "What is LangGraph?" }] }, streamMode:
"messages-tuple", });
```

```
for await (const chunk of streamResponse) { console.log(Receiving new event of type:
${chunk.event}...); console.log(JSON.stringify(chunk.data)); console.log("\n\n"); }
```

**\*\*Rest API\*\***

```

```bash
curl -s --request POST \
  --url "http://localhost:2024/runs/stream" \
  --header 'Content-Type: application/json' \
  --data "{
    \"assistant_id\": \"agent\",
    \"input\": {
      \"messages\": [
        {
          \"role\": \"human\",
          \"content\": \"What is LangGraph?\"
        }
      ]
    },
    \"stream_mode\": \"messages-tuple\"
  }"

```

Next steps

Now that you have a LangGraph app running locally, take your journey further by exploring deployment and advanced features:

- [Deployment quickstart](#): Deploy your LangGraph app using LangSmith.
- [LangSmith](#): Learn about foundational LangSmith concepts.

python

- [SDK Reference](#): Explore the SDK API Reference.

js

- [SDK Reference](#): Explore the SDK API Reference.

LangSmith Observability

langgraph/observability.mdx

Traces are a series of steps that your application takes to go from input to output. Each of these individual steps is represented by a run. You can use [LangSmith](#) to visualize these execution steps. To use it, [enable tracing for your application](#). This enables you to do the following:

- [Debug a locally running application](#).
- [Evaluate the application performance](#).
- [Monitor the application](#).

Prerequisites

Before you begin, ensure you have the following:

- **A LangSmith account:** Sign up (for free) or log in at smith.langchain.com.
- **A LangSmith API key:** Follow the [Create an API key](#) guide.

Enable tracing

To enable tracing for your application, set the following environment variables:

```
export LANGSMITH_TRACING=true
export LANGSMITH_API_KEY=<your-api-key>
```

By default, the trace will be logged to the project with the name `default`. To configure a custom project name, see [Log to a project](#).

For more information, see [Trace with LangGraph](#).

Trace selectively

You may opt to trace specific invocations or parts of your application using LangSmith's `tracing_context` context manager:

python

```
import langsmith as ls

# This WILL be traced
with ls.tracing_context(enabled=True):
    agent.invoke({"messages": [{"role": "user", "content": "Send a test email to alice@example.com"}]})
```

```
# This will NOT be traced (if LANGSMITH_TRACING is not set)
agent.invoke({"messages": [{"role": "user", "content": "Send another email"}]})
```

js

```
import { LangChainTracer } from "@langchain/core/tracers/tracer_langchain";

// This WILL be traced
const tracer = new LangChainTracer();
await agent.invoke(
  {
    messages: [{role: "user", content: "Send a test email to alice@example.com"}],
  },
  { callbacks: [tracer] }
);

// This will NOT be traced (if LANGSMITH_TRACING is not set)
await agent.invoke(
  {
    messages: [{role: "user", content: "Send another email"}]
  }
);
```

Log to a project

Statically

You can set a custom project name for your entire application by setting the `LANGSMITH_PROJECT` environment variable:

```
export LANGSMITH_PROJECT=my-agent-project
```

Dynamically

You can set the project name programmatically for specific operations:

python

```
import langsmith as ls

with ls.tracing_context(project_name="email-agent-test", enabled=True):
    response = agent.invoke({
        "messages": [{"role": "user", "content": "Send a welcome email"}]
    })
```

js

```
import { LangChainTracer } from "@langchain/core/tracers/tracer_langchain";

const tracer = new LangChainTracer({ projectName: "email-agent-test" });
await agent.invoke(
  {
    messages: [{role: "user", content: "Send a test email to alice@example.com"}]
  },
  { callbacks: [tracer] }
);
```

Add metadata to traces

You can annotate your traces with custom metadata and tags:

python

```
response = agent.invoke(
  {"messages": [{"role": "user", "content": "Send a welcome email"}]},
  config={
    "tags": ["production", "email-assistant", "v1.0"],
    "metadata": {
      "user_id": "user_123",
      "session_id": "session_456",
      "environment": "production"
    }
  }
)
```

`tracing_context` also accepts tags and metadata for fine-grained control:

```
with ls.tracing_context(
  project_name="email-agent-test",
  enabled=True,
  tags=["production", "email-assistant", "v1.0"],
  metadata={"user_id": "user_123", "session_id": "session_456", "environment": "production"}):
  response = agent.invoke(
    {"messages": [{"role": "user", "content": "Send a welcome email"}]}
  )
```

js

```
import { LangChainTracer } from "@langchain/core/tracers/tracer_langchain";

const tracer = new LangChainTracer({ projectName: "email-agent-test" });
await agent.invoke(
  {
    messages: [{role: "user", content: "Send a test email to alice@example.com"}]
  },
  config: {
    tags: ["production", "email-assistant", "v1.0"],
    metadata: {
      userId: "user123",
      sessionId: "session456",
      environment: "production"
    }
  },
);
```

This custom metadata and tags will be attached to the trace in LangSmith.

팁

To learn more about how to use traces to debug, evaluate, and monitor your agents, see the [LangSmith documentation](#).

Use anonymizers to prevent logging of sensitive data in traces

You may want to mask sensitive data to prevent it from being logged to LangSmith. You can create [anonymizers](#) and apply them to your graph using configuration. This example will redact anything matching the Social Security Number format XXX-XX-XXXX from traces sent to LangSmith.

python

```
` `` python Python from langchain_core.tracers.langchain import LangChainTracer from
langgraph.graph import StateGraph, MessagesState from langsmith import Client from
langsmith.anonymizer import create_anonymizer

anonymizer = create_anonymizer([ # Matches SSNs { "pattern": r"\b\d{3}-?\d{2}-?\d{4}\b",
"replace": "" } ])

tracer_client = Client(anonymizer=anonymizer) tracer = LangChainTracer(client=tracer_client)
```


Define the graph

```
graph = ( StateGraph(MessagesState) ... .compile() .with_config({'callbacks': [tracer]}) )
```

```
</div>
```

```
<div class="callout js" markdown="1">
```

```
<p class="callout-label">js</p>
```

```
```typescript TypeScript
```

```
import { StateGraph } from "@langchain/langgraph";
```

```
import { LangChainTracer } from "@langchain/core/tracers/tracer_langchain";
```

```
import { StateAnnotation } from "./state.js";
```

```
import { createAnonymizer } from "langsmith/anonymizer"
```

```
import { Client } from "langsmith"
```

```
const anonymizer = createAnonymizer([
```

```
 // Matches SSNs
```

```
 { pattern: /\b\d{3}-?\d{2}-?\d{4}\b/, replace: "<ssn>" }
```

```
])
```

```
const langsmithClient = new Client({ anonymizer })
```

```
const tracer = new LangChainTracer({
```

```
 client: langsmithClient,
```

```
});
```

```
export const graph = new StateGraph(StateAnnotation)
```

```
 .compile()
```

```
 .withConfig({ callbacks: [tracer] });
```

# LangGraph overview

---

langgraph/overview.mdx

Trusted by companies shaping the future of agents—including Klarna, Uber, J.P. Morgan, and more—LangGraph is a low-level orchestration framework and runtime for building, managing, and deploying long-running, stateful agents. LangGraph gives you fine-grained control to mix deterministic, hand-coded steps with LLM-driven agentic steps in the same graph, so you can build bespoke agents that behave exactly the way your application requires.

LangGraph is very low-level, and focused entirely on agent **orchestration**. Before using LangGraph, we recommend you familiarize yourself with some of the components used to build agents, starting with [models](#) and [tools](#).

We will commonly use [LangChain](#) components throughout the documentation to integrate models and tools, but you don't need to use LangChain to use LangGraph. If you are just getting started with agents or want a higher-level abstraction, we recommend you use LangChain's [agents](#) that provide prebuilt architectures for common LLM and tool-calling loops.

LangGraph is focused on the underlying capabilities important for agent orchestration: durable execution, streaming, human-in-the-loop, and more.

One of LangGraph's core strengths is the ability to mix deterministic steps with LLM-driven agentic steps in a single graph. This lets you build bespoke workflows where parts of the logic are fully predictable and auditable while other parts are flexible and model-driven, giving you fine-grained control over exactly where and how AI is applied.

## how LangChain products fit together

- [Deep Agents](#) is an [agent harness](#): planning, subagents, filesystem tools, and context management on top of LangGraph.
- [LangChain](#) is the agent framework: abstractions and integrations for models, tools, and agent loops.
- [LangGraph](#) is the orchestration runtime: durable execution, streaming, human-in-the-loop, and persistence.
- [LangSmith](#) is the platform for tracing, evaluation, prompts, and deployment across frameworks.
- [LangSmith Engine](#) detects issues in your LangGraph agent traces and proposes fixes. You can open a pull request with the proposed fix directly from the Engine tab.
- [LangSmith Fleet](#) is the no-code agent builder for templates, integrations, and routine automation.

Read [Frameworks, runtimes, and harnesses](#) for a comparison of the open source stack.

## Install

### python

```
```bash pip pip install -U langgraph
```

```
```bash uv  
uv add langgraph
```

### js

```
```bash npm npm install @langchain/langgraph @langchain/core
```

```
```bash pnpm  
pnpm add @langchain/langgraph @langchain/core
```

```
```bash yarn yarn add @langchain/langgraph @langchain/core
```

```
```bash bun  
bun add @langchain/langgraph @langchain/core
```

Then, create a simple hello world example:

### python

```
from langgraph.graph import StateGraph, MessagesState, START, END

def mock_llm(state: MessagesState):
 return {"messages": [{"role": "ai", "content": "hello world"}]}

graph = StateGraph(MessagesState)
graph.add_node(mock_llm)
graph.add_edge(START, "mock_llm")
graph.add_edge("mock_llm", END)
graph = graph.compile()

graph.invoke({"messages": [{"role": "user", "content": "hi!"}]})
```

### js

```
import { StateSchema, MessagesValue, type GraphNode, StateGraph, START, END } from "@langchain/langgraph";

const State = new StateSchema({
 messages: MessagesValue,
});

const mockLlm: GraphNode<typeof State> = (state) => {
 return { messages: [{ role: "ai", content: "hello world" }] };
};

const graph = new StateGraph(State)
 .addNode("mock_llm", mockLlm)
 .addEdge(START, "mock_llm")
 .addEdge("mock_llm", END)
 .compile();

await graph.invoke({ messages: [{ role: "user", content: "hi!" }] });
```

### 팁

Use [LangSmith](#) to trace requests, debug agent behavior, and evaluate outputs. Set `LANGSMITH_TRACING=true` and your API key to get started. Follow the [tracing quickstart](#) to get set up. We recommend you also set up [LangSmith Engine](#) which monitors your traces, detects issues, and proposes fixes.

## Core benefits

LangGraph provides low-level supporting infrastructure for *any* long-running, stateful workflow or agent. LangGraph does not abstract prompts or architecture, and provides the following central benefits:

- **Mix deterministic and agentic steps:** Combine hand-coded, deterministic logic with LLM-driven decision-making in a single graph. Use deterministic steps where you need reliability and predictability, and agentic steps where you need flexibility—giving you precise control over every part of your agent's behavior.
- [Persistence](#): Build agents that persist through failures and can run for extended periods, resuming from where they left off.
- [Human-in-the-loop](#): Incorporate human oversight by inspecting and modifying agent state at any point.
- [Comprehensive memory](#): Create stateful agents with both short-term working memory for ongoing reasoning and long-term memory across sessions.
- [Debugging with LangSmith](#): Gain deep visibility into complex agent behavior with visualization tools that trace execution paths, capture state transitions, and provide detailed

runtime metrics.

- [Production-ready deployment](#): Deploy sophisticated agent systems confidently with scalable infrastructure designed to handle the unique challenges of stateful, long-running workflows.

## LangGraph ecosystem

---

While LangGraph can be used standalone, it also integrates seamlessly with any LangChain product, giving developers a full suite of tools for building agents. To improve your LLM application development, pair LangGraph with:

### [LangSmith Observability](#)

Trace requests, evaluate outputs, and monitor deployments in one place. Prototype locally with LangGraph, then move to production with integrated observability and evaluation to build more reliable agent systems.

### [LangSmith Deployment](#)

Deploy and scale agents effortlessly with a purpose-built deployment platform for long running, stateful workflows. Discover, reuse, configure, and share agents across teams — and iterate quickly with visual prototyping in Studio.

### [LangChain](#)

Provides integrations and composable components to streamline LLM application development. Contains agent abstractions built on top of LangGraph.

## Acknowledgements

---

LangGraph is inspired by [Pregel](#) and [Apache Beam](#). The public interface draws inspiration from [NetworkX](#). LangGraph is built by LangChain Inc, the creators of LangChain, but can be used without LangChain.

# Persistence

---

langgraph/persistence.mdx

Persistence lets LangGraph applications keep useful information beyond a single graph run. It matters when an agent needs to continue a conversation, resume after an interruption, recover from a failure, or remember information across interactions.

LangGraph provides two complementary persistence systems:

- [Checkpointers](#) persist a thread's graph state as checkpoints. Use them for short-term, thread-scoped memory, including conversation continuity, human-in-the-loop workflows, time travel, and fault tolerance.
- [Stores](#) persist application-defined data outside the graph state. Use them for long-term, cross-thread memory, including user preferences, facts, and shared knowledge.

Most applications can use both: a [checkpointer](#) tracks the current thread, and a [store](#) tracks durable information across threads.

## Quickstart

---

Compile your graph with a checkpointer, a store, or both:

### python

```
from langgraph.checkpoint.memory import InMemorySaver
from langgraph.store.memory import InMemoryStore

checkpointer = InMemorySaver()
store = InMemoryStore()

graph = builder.compile(checkpointer=checkpointer, store=store)

result = graph.invoke(
 {"messages": [{"role": "user", "content": "Hi, my name is Bob."}]},
 {"configurable": {"thread_id": "thread-1"}},
)
```

### js

```
import { MemorySaver, MemoryStore } from "@langchain/langgraph";

const checkpointer = new MemorySaver();
const store = new MemoryStore();
```

```
const graph = builder.compile({ checkpointer, store });

const result = await graph.invoke(
 { messages: [{ role: "user", content: "Hi, my name is Bob." }] },
 { configurable: { thread_id: "thread-1" } }
);
```

## 정보

**Agent Server handles persistence automatically** When using the [Agent Server](#), you do not need to implement or configure checkpointers or stores manually. The server handles persistence infrastructure behind the scenes.

## Checkpointer vs. store

	Checkpointer	Store
Persists	Graph state snapshots	Application-defined key-value data
Scope	A single thread	Across threads
Memory type	Short-term, thread-scoped memory	Long-term, cross-thread memory
Use for	Conversation continuity, human-in-the-loop, time travel, and fault tolerance	User preferences, facts, and shared knowledge
Access pattern	Pass a <code>thread_id</code> in graph config	Read and write items from nodes or application code
Full guide	<a href="#">Checkpointers</a>	<a href="#">Stores</a>

## Troubleshooting common issues

### PostgresSaver: `thread_id` too long

When using `PostgresSaver` (or `AsyncPostgresSaver`), the `thread_id` is stored in a column with limited length. If your `thread_id` exceeds the column size, you will see a database error.

**Fix:** Keep `thread_id` values under 255 characters. Use a UUID or hash if you need deterministic IDs:

### python

```
import uuid

config = {"configurable": {"thread_id": str(uuid.uuid4())[:255]}}
```

## `MemorySaver` does not persist between restarts

`MemorySaver` and `InMemorySaver` store checkpoints in RAM. When the process restarts, all checkpoints are lost.

**Fix:** Use a persistent checkpointer for production:

- `PostgresSaver`: PostgreSQL with async support
- `SqliteSaver`: Local file-based storage for development

## Checkpoints growing unboundedly

Over long conversations, checkpoints accumulate. This can increase latency and storage costs.

**Fix:** Prune old checkpoints periodically or set a retention policy:

### python

```
from langgraph.checkpoint.postgres import PostgresSaver

checkpointer = PostgresSaver.from_conn_string("postgresql://...")
checkpointer.setup() # Creates tables with indexes
Consider adding a cron job to delete checkpoints older than N days
```

## State access from parent graph to subgraph

When a subgraph updates state, the parent graph may not see the changes immediately. This is because each subgraph manages its own checkpoint namespace.

**Fix:** Use [shared state via Store](#) for data that needs to cross graph boundaries, or configure your subgraph to write to the parent checkpoint.

## Next steps

---

- [Use checkpointer](#) to persist and inspect thread state.
- [Use stores](#) to persist durable data across threads.



# LangGraph runtime

---

langgraph/pregel.mdx

## python

`@[ Pregel ]` implements LangGraph's runtime, managing the execution of LangGraph applications.

Compiling a `@[StateGraph][StateGraph]` or creating an `@[@entrypoint ]` produces a `@[ Pregel ]` instance that can be invoked with input.

## js

`@[ Pregel ]` implements LangGraph's runtime, managing the execution of LangGraph applications.

Compiling a `@[StateGraph][StateGraph]` or creating an `@[entrypoint][entrypoint]` produces a `@[ Pregel ]` instance that can be invoked with input.

This guide explains the runtime at a high level and provides instructions for directly implementing applications with Pregel.

## python

**Note:** The `@[ Pregel ]` runtime is named after [Google's Pregel algorithm](#), which describes an efficient method for large-scale parallel computation using graphs.

## js

**Note:** The `@[ Pregel ]` runtime is named after [Google's Pregel algorithm](#), which describes an efficient method for large-scale parallel computation using graphs.

## Overview

---

In LangGraph, Pregel combines [actors](#) and **channels** into a single application. **Actors** read data from channels and write data to channels. Pregel organizes the execution of the application into multiple steps, following the **Pregel Algorithm/Bulk Synchronous Parallel** model.

Each step consists of three phases:

- **Plan:** Determine which **actors** to execute in this step. For example, in the first step, select the **actors** that subscribe to the special **input** channels; in subsequent steps, select the **actors** that subscribe to channels updated in the previous step.
- **Execution:** Execute all selected **actors** in parallel, until all complete, or one fails, or a timeout is reached. During this phase, channel updates are invisible to actors until the next step.
- **Update:** Update the channels with the values written by the **actors** in this step.

Repeat until no **actors** are selected for execution, or a maximum number of steps is reached.

## Actors

---

An **actor** is a `PregelNode`. It subscribes to channels, reads data from them, and writes data to them. It can be thought of as an **actor** in the Pregel algorithm. `PregelNodes` implement LangChain's `Runnable` interface.

## Channels

---

Channels are used to communicate between actors (`PregelNodes`). Each channel has a value type, an update type, and an update function—which takes a sequence of updates and modifies the stored value. Channels can be used to send data from one chain to another, or to send data from a chain to itself in a future step.

### LastValue

`@[LastValue]` is the default channel type. It stores the last value written to it, overwriting any previous value. Use it for input and output values, or for passing data from one step to the next.

#### python

```
from langgraph.channels import LastValue

channel: LastValue[int] = LastValue(int)
```

#### js

```
import { LastValue } from "@langchain/langgraph/channels";

const channel = new LastValue<number>();
```

## Topic

`@[Topic]` is a configurable PubSub channel useful for sending multiple values between actors or accumulating output across steps. It can be configured to deduplicate values or to accumulate all

values written during a run.

### python

```
from langgraph.channels import Topic

Accumulate all values written across steps
channel: Topic[str] = Topic(str, accumulate=True)
```

### js

```
import { Topic } from "@langchain/langgraph/channels";

// Accumulate all values written across steps
const channel = new Topic<string>({ accumulate: true });
```

## BinaryOperatorAggregate

@[ BinaryOperatorAggregate ] stores a persistent value that is updated by applying a binary operator to the current value and each new update. Use it to compute running aggregates across steps.

### python

```
import operator
from langgraph.channels import BinaryOperatorAggregate

Running total: each write adds to the current value
total = BinaryOperatorAggregate(int, operator.add)
```

### js

```
import { BinaryOperatorAggregate } from "@langchain/langgraph/channels";

// Running total: each write adds to the current value
const total = new BinaryOperatorAggregate<number>({ operator: (a, b) => a + b });
```

### python

## DeltaChannel

주의

`DeltaChannel` requires `langgraph>=1.2` and is currently in **beta**. The API may change in future releases.

`@[DeltaChannel]` stores only the incremental delta at each step rather than the full accumulated value. This is most useful for channels that are written frequently and accumulate large values over time—for example, a conversation message list in a long-running thread. Without delta storage, the full list is re-serialized into every checkpoint; with `DeltaChannel`, only the new messages written at each step are stored.

### 팁

Consider `DeltaChannel` when a channel is both written to frequently and grows large over time. A good signal: if you notice checkpoint sizes growing linearly with thread length for a particular channel, `DeltaChannel` is likely a good fit.

Use `DeltaChannel` in an `Annotated` type annotation the same way you would use a plain reducer:

```
from typing import Annotated, Sequence
from typing_extensions import TypedDict
from langgraph.channels import DeltaChannel

def my_reducer(state: list[str], writes: Sequence[list[str]]) -> list[str]:
 result = list(state)
 for write in writes:
 result.extend(write)
 return result

class State(TypedDict):
 messages: Annotated[list[str], DeltaChannel(my_reducer)]
```

## Bulk reducer requirement

The `reducer` passed to `DeltaChannel` is a **bulk reducer**: it receives the current state and a *sequence* of all writes from the current step in a single call—not pairwise like a standard reducer. This differs from the per-key reducers used with `Annotated` in a `StateGraph`, where the reducer is called once per update.

### 주의

The bulk reducer **must be associative** (batching-invariant):

```
reducer(reducer(state, [xs]), [ys]) == reducer(state, [xs, ys])
```

If your reducer is not associative, the reconstructed state may differ depending on how LangGraph batches writes across steps, producing inconsistent behavior.

## 주의

**The reducer runs on reconstruction, not on write.** Unlike a `@BinaryOperatorAggregate`, whose reducer is invoked at write time so the combined value is what gets serialized into the checkpoint, a `DeltaChannel` reducer is invoked when the channel value is *rebuilt* from its persisted writes. The raw per-step writes are what get serialized; the reducer is only called when the value is materialized—on the next read, on the next step's actors, or when replaying history.

Practical consequences when designing a reducer:

- **Make it a pure function of (state, writes).** Any side effects, randomness, or wall-clock reads (e.g., `uuid.uuid4()`, `datetime.now()`) execute every time the value is reconstructed and produce different results on each replay. They are *not* baked into the persisted writes.
- **Do not rely on mutations to incoming writes being persisted.** If your reducer mutates a write object (for example, assigning a stable ID to an item that arrived without one), that mutation lives only in the reconstructed value. The stored write still has the original shape, so the next reconstruction will see the un-mutated input again.
- **Attach identity and other stable metadata upstream.** If downstream code needs to reference an item by ID across turns (e.g., to update or remove it later), assign that ID before the value is written to the channel—not inside the reducer.

Here are bulk reducers for the two most common cases:

```
from typing import Any, Sequence

List: append all writes in order
def list_reducer(state: list[Any], writes: Sequence[list[Any]]) -> list[Any]:
 result = list(state)
 for write in writes:
 result.extend(write)
 return result

Dict: merge all writes, last write wins on key conflicts
def dict_reducer(
 state: dict[str, Any], writes: Sequence[dict[str, Any]]
) -> dict[str, Any]:
 result = dict(state)
 for write in writes:
```

```
result.update(write)
return result
```

Both are associative: applying batches one at a time produces the same result as applying them together.

### Use `snapshot_frequency` for bounded read latency

Without snapshots, reading a `DeltaChannel` value requires replaying the full write history— $O(N)$  for a thread with  $N$  steps. Setting `snapshot_frequency=K` writes a full snapshot every  $K$  pregel steps, bounding read depth to at most  $K$  steps:

```
class State(TypedDict):
 messages: Annotated[
 list[str],
 DeltaChannel(my_reducer, snapshot_frequency=5),
]
```

Higher values of `snapshot_frequency` reduce storage overhead but increase read latency. Lower values bound latency more tightly at the cost of larger checkpoints. `None` (the default) skips snapshots entirely—appropriate when reads are rare or threads are short.

### Version compatibility and rollbacks

#### 주의

Changing a persisted channel from `DeltaChannel` to a non-delta channel is not recommended. Checkpoints encode these channel types differently, so changing the type for an existing thread can cause incomplete or incorrect state reconstruction. Keep channel definitions stable for the lifetime of a thread. Before changing a channel type, migrate affected threads to the new representation, or discard them and start new threads.

**Rolling back to a version without `DeltaChannel` support is not supported.** `langgraph>=1.2` writes delta channel checkpoints in a new format that earlier versions cannot read. Once a thread has used `DeltaChannel`, downgrading `LangGraph` leaves those checkpoints unreadable as older runtimes do not understand the delta format and cannot reconstruct channel state. If you need to roll back, use the [delta-channel-dump recovery script](#) to migrate affected threads, or discard them, before downgrading.

## Examples

### python

While most users will interact with Pregel through the `@[StateGraph][StateGraph]` API or the `@[entrypoint]` decorator, it is possible to interact with Pregel directly.

js

While most users will interact with Pregel through the `@[StateGraph]` API or the `@[entrypoint]` decorator, it is possible to interact with Pregel directly.

Below are a few different examples to give you a sense of the Pregel API.

## Single node

python

```
from langgraph.channels import EphemeralValue
from langgraph.pregel import Pregel, NodeBuilder

node1 = (
 NodeBuilder().subscribe_only("a")
 .do(lambda x: x + x)
 .write_to("b")
)

app = Pregel(
 nodes={"node1": node1},
 channels={
 "a": EphemeralValue(str),
 "b": EphemeralValue(str),
 },
 input_channels=["a"],
 output_channels=["b"],
)

app.invoke({"a": "foo"})
```

```
{'b': 'foofoo'}
```

js

```
import { EphemeralValue } from "@langchain/langgraph/channels";
import { Pregel, NodeBuilder } from "@langchain/langgraph/pregel";

const node1 = new NodeBuilder()
 .subscribeOnly("a")
 .do((x: string) => x + x)
 .writeTo("b");
```

```
const app = new Pregel({
 nodes: { node1 },
 channels: {
 a: new EphemeralValue<string>(),
 b: new EphemeralValue<string>(),
 },
 inputChannels: ["a"],
 outputChannels: ["b"],
});

await app.invoke({ a: "foo" });
```

```
{ b: 'foofoo' }
```

## Multiple nodes

### python

```
from langgraph.channels import LastValue, EphemeralValue
from langgraph.pregel import Pregel, NodeBuilder

node1 = (
 NodeBuilder().subscribe_only("a")
 .do(lambda x: x + x)
 .write_to("b")
)

node2 = (
 NodeBuilder().subscribe_only("b")
 .do(lambda x: x + x)
 .write_to("c")
)

app = Pregel(
 nodes={"node1": node1, "node2": node2},
 channels={
 "a": EphemeralValue(str),
 "b": LastValue(str),
 "c": EphemeralValue(str),
 },
 input_channels=["a"],
 output_channels=["b", "c"],
)

app.invoke({"a": "foo"})
```



```
{'b': 'foofoo', 'c': 'foofoofoofoo'}
```

## js

```
import { LastValue, EphemeralValue } from "@langchain/langgraph/channels";
import { Pregel, NodeBuilder } from "@langchain/langgraph/pregel";

const node1 = new NodeBuilder()
 .subscribeOnly("a")
 .do((x: string) => x + x)
 .writeTo("b");

const node2 = new NodeBuilder()
 .subscribeOnly("b")
 .do((x: string) => x + x)
 .writeTo("c");

const app = new Pregel({
 nodes: { node1, node2 },
 channels: {
 a: new EphemeralValue<string>(),
 b: new LastValue<string>(),
 c: new EphemeralValue<string>(),
 },
 inputChannels: ["a"],
 outputChannels: ["b", "c"],
});

await app.invoke({ a: "foo" });
```

```
{ b: 'foofoo', c: 'foofoofoofoo' }
```

## Topic

### python

```
from langgraph.channels import EphemeralValue, Topic
from langgraph.pregel import Pregel, NodeBuilder

node1 = (
 NodeBuilder().subscribe_only("a")
 .do(lambda x: x + x)
 .write_to("b", "c")
)

node2 = (
```

```

 NodeBuilder().subscribe_to("b")
 .do(lambda x: x["b"] + x["b"])
 .write_to("c")
)

app = Pregel(
 nodes={"node1": node1, "node2": node2},
 channels={
 "a": EphemeralValue(str),
 "b": EphemeralValue(str),
 "c": Topic(str, accumulate=True),
 },
 input_channels=["a"],
 output_channels=["c"],
)

app.invoke({"a": "foo"})

```

```
{'c': ['foofoo', 'foofoofoofoo']}
```

## js

```

import { EphemeralValue, Topic } from "@langchain/langgraph/channels";
import { Pregel, NodeBuilder } from "@langchain/langgraph/pregel";

const node1 = new NodeBuilder()
 .subscribeOnly("a")
 .do((x: string) => x + x)
 .writeTo("b", "c");

const node2 = new NodeBuilder()
 .subscribeTo("b")
 .do((x: { b: string }) => x.b + x.b)
 .writeTo("c");

const app = new Pregel({
 nodes: { node1, node2 },
 channels: {
 a: new EphemeralValue<string>(),
 b: new EphemeralValue<string>(),
 c: new Topic<string>({ accumulate: true }),
 },
 inputChannels: ["a"],
 outputChannels: ["c"],
});

await app.invoke({ a: "foo" });

```

```
{ c: ['foofoo', 'foofoofoofoo'] }
```

## BinaryOperatorAggregate

This example demonstrates how to use the `@[BinaryOperatorAggregate]` channel to implement a reducer.

### python

```
from langgraph.channels import EphemeralValue, BinaryOperatorAggregate
from langgraph.pregel import Pregel, NodeBuilder

node1 = (
 NodeBuilder().subscribe_only("a")
 .do(lambda x: x + x)
 .write_to("b", "c")
)

node2 = (
 NodeBuilder().subscribe_only("b")
 .do(lambda x: x + x)
 .write_to("c")
)

def reducer(current, update):
 if current:
 return current + " | " + update
 else:
 return update

app = Pregel(
 nodes={"node1": node1, "node2": node2},
 channels={
 "a": EphemeralValue(str),
 "b": EphemeralValue(str),
 "c": BinaryOperatorAggregate(str, operator=reducer),
 },
 input_channels=["a"],
 output_channels=["c"],
)

app.invoke({"a": "foo"})
```

```
{ 'c': 'foofoo | foofoofoofoo' }
```

### js

```

import { EphemeralValue, BinaryOperatorAggregate } from "@langchain/langgraph/channels";
import { Pregel, NodeBuilder } from "@langchain/langgraph/pregel";

const node1 = new NodeBuilder()
 .subscribeOnly("a")
 .do((x: string) => x + x)
 .writeTo("b", "c");

const node2 = new NodeBuilder()
 .subscribeOnly("b")
 .do((x: string) => x + x)
 .writeTo("c");

const reducer = (current: string, update: string) => {
 if (current) {
 return current + " | " + update;
 } else {
 return update;
 }
};

const app = new Pregel({
 nodes: { node1, node2 },
 channels: {
 a: new EphemeralValue<string>(),
 b: new EphemeralValue<string>(),
 c: new BinaryOperatorAggregate<string>({ operator: reducer }),
 },
 inputChannels: ["a"],
 outputChannels: ["c"],
});

await app.invoke({ a: "foo" });

```

## Cycle

### python

This example demonstrates how to introduce a cycle in the graph, by having a chain write to a channel it subscribes to. Execution will continue until a `None` value is written to the channel.

```

from langgraph.channels import EphemeralValue
from langgraph.pregel import Pregel, NodeBuilder, ChannelWriteEntry

example_node = (
 NodeBuilder().subscribe_only("value")
 .do(lambda x: x + x if len(x) < 10 else None)
 .write_to(ChannelWriteEntry("value", skip_none=True))

```

```

)

app = Pregel(
 nodes={"example_node": example_node},
 channels={
 "value": EphemeralValue(str),
 },
 input_channels=["value"],
 output_channels=["value"],
)

app.invoke({"value": "a"})

```

```
{'value': 'aaaaaaaaaaaaaaaa'}
```

js

This example demonstrates how to introduce a cycle in the graph, by having a chain write to a channel it subscribes to. Execution will continue until a `null` value is written to the channel.

```

import { EphemeralValue } from "@langchain/langgraph/channels";
import { Pregel, NodeBuilder, ChannelWriteEntry } from "@langchain/langgraph/pregel";

const exampleNode = new NodeBuilder()
 .subscribeOnly("value")
 .do((x: string) => x.length < 10 ? x + x : null)
 .writeTo(new ChannelWriteEntry("value", { skipNone: true }));

const app = new Pregel({
 nodes: { exampleNode },
 channels: {
 value: new EphemeralValue<string>(),
 },
 inputChannels: ["value"],
 outputChannels: ["value"],
});

await app.invoke({ value: "a" });

```

```
{ value: 'aaaaaaaaaaaaaaaa' }
```

## High-level API

LangGraph provides two high-level APIs for creating a Pregel application: the [StateGraph \(Graph API\)](#) and the [Functional API](#).

## StateGraph (Graph API)

### python

The `@StateGraph (Graph API)[StateGraph]` is a higher-level abstraction that simplifies the creation of Pregel applications. It allows you to define a graph of nodes and edges. When you compile the graph, the StateGraph API automatically creates the Pregel application for you.

```
from typing import TypedDict

from langgraph.constants import START
from langgraph.graph import StateGraph

class Essay(TypedDict):
 topic: str
 content: str | None
 score: float | None

def write_essay(essay: Essay):
 return {
 "content": f"Essay about {essay['topic']}",
 }

def score_essay(essay: Essay):
 return {
 "score": 10
 }

builder = StateGraph(Essay)
builder.add_node(write_essay)
builder.add_node(score_essay)
builder.add_edge(START, "write_essay")
builder.add_edge("write_essay", "score_essay")

Compile the graph.
This will return a Pregel instance.
graph = builder.compile()
```

### js

The `@StateGraph (Graph API)[StateGraph]` is a higher-level abstraction that simplifies the creation of Pregel applications. It allows you to define a graph of nodes and edges. When you compile the graph, the StateGraph API automatically creates the Pregel application for you.

```
import { START, StateGraph } from "@langchain/langgraph";

interface Essay {
 topic: string;
```

```

 content?: string;
 score?: number;
}

const writeEssay = (essay: Essay) => {
 return {
 content: `Essay about ${essay.topic}`,
 };
};

const scoreEssay = (essay: Essay) => {
 return {
 score: 10
 };
};

const builder = new StateGraph<Essay>({
 channels: {
 topic: null,
 content: null,
 score: null,
 }
})

 .addNode("writeEssay", writeEssay)
 .addNode("scoreEssay", scoreEssay)
 .addEdge(START, "writeEssay")
 .addEdge("writeEssay", "scoreEssay");

// Compile the graph.
// This will return a Pregel instance.
const graph = builder.compile();

```

The compiled Pregel instance will be associated with a list of nodes and channels. You can inspect the nodes and channels by printing them.

## python

```
print(graph.nodes)
```

You will see something like this:

```
{'__start__': <langgraph.pregel.read.PregelNode at 0x7d05e3ba1810>,
 'write_essay': <langgraph.pregel.read.PregelNode at 0x7d05e3ba14d0>,
 'score_essay': <langgraph.pregel.read.PregelNode at 0x7d05e3ba1710>}
```

```
print(graph.channels)
```

You should see something like this

```
{'topic': <langgraph.channels.last_value.LastValue at 0x7d05e3294d80>,
 'content': <langgraph.channels.last_value.LastValue at 0x7d05e3295040>,
 'score': <langgraph.channels.last_value.LastValue at 0x7d05e3295980>,
 '__start__': <langgraph.channels.ephemeral_value.EphemeralValue at 0x7d05e3297e00>,
 'write_essay': <langgraph.channels.ephemeral_value.EphemeralValue at 0x7d05e32960c0>,
 'score_essay': <langgraph.channels.ephemeral_value.EphemeralValue at 0x7d05e2d8ab80>,
 'branch:__start__:__self__:write_essay': <langgraph.channels.ephemeral_value.EphemeralValue
at 0x7d05e32941c0>,
 'branch:__start__:__self__:score_essay': <langgraph.channels.ephemeral_value.EphemeralValue
at 0x7d05e2d88800>,
 'branch:write_essay:__self__:write_essay': <langgraph.channels.ephemeral_value.EphemeralVal
ue at 0x7d05e3295ec0>,
 'branch:write_essay:__self__:score_essay': <langgraph.channels.ephemeral_value.EphemeralVal
ue at 0x7d05e2d8ac00>,
 'branch:score_essay:__self__:write_essay': <langgraph.channels.ephemeral_value.EphemeralVal
ue at 0x7d05e2d89700>,
 'branch:score_essay:__self__:score_essay': <langgraph.channels.ephemeral_value.EphemeralVal
ue at 0x7d05e2d8b400>,
 'start:write_essay': <langgraph.channels.ephemeral_value.EphemeralValue at 0x7d05e2d8b280>}
```

js

```
console.log(graph.nodes);
```

You will see something like this:

```
{
 __start__: PregelNode { ... },
 writeEssay: PregelNode { ... },
 scoreEssay: PregelNode { ... }
}
```

```
console.log(graph.channels);
```

You should see something like this

```
{
 topic: LastValue { ... },
 content: LastValue { ... },
 score: LastValue { ... },
 __start__: EphemeralValue { ... },
 writeEssay: EphemeralValue { ... },
 scoreEssay: EphemeralValue { ... },
 'branch:__start__:__self__:writeEssay': EphemeralValue { ... },
 'branch:__start__:__self__:scoreEssay': EphemeralValue { ... },
}
```



```

'branch:writeEssay:__self__:writeEssay': EphemeralValue { ... },
'branch:writeEssay:__self__:scoreEssay': EphemeralValue { ... },
'branch:scoreEssay:__self__:writeEssay': EphemeralValue { ... },
'branch:scoreEssay:__self__:scoreEssay': EphemeralValue { ... },
'start:writeEssay': EphemeralValue { ... }
}

```

## Functional API

### python

In the [Functional API](#), you can use an `@[@entrypoint]` to create a Pregel application. The `entrypoint` decorator allows you to define a function that takes input and returns output.

```

from typing import TypedDict

from langgraph.checkpoint.memory import InMemorySaver
from langgraph.func import entrypoint

class Essay(TypedDict):
 topic: str
 content: str | None
 score: float | None

checkpointer = InMemorySaver()

@entrypoint(checkpointer=checkpointer)
def write_essay(essay: Essay):
 return {
 "content": f"Essay about {essay['topic']}",
 }

print("Nodes: ")
print(write_essay.nodes)
print("Channels: ")
print(write_essay.channels)

```

```

Nodes:
{'write_essay': <langgraph.pregel.read.PregelNode object at 0x7d05e2f9aad0>}
Channels:
{'__start__': <langgraph.channels.ephemeral_value.EphemeralValue object at 0x7d05e2c906c0>,
 '__end__': <langgraph.channels.last_value.LastValue object at 0x7d05e2c90c40>, '__previous__': <langgraph.channels.last_value.LastValue object at 0x7d05e1007280>}

```

### js

In the [Functional API](#), you can use an `@[entrypoint][entrypoint]` to create a Pregel application. The `entrypoint` decorator allows you to define a function that takes input and returns output.

```
import { MemorySaver } from "@langchain/langgraph";
import { entrypoint } from "@langchain/langgraph/func";

interface Essay {
 topic: string;
 content?: string;
 score?: number;
}

const checkpointer = new MemorySaver();

const writeEssay = entrypoint(
 { checkpointer, name: "writeEssay" },
 async (essay: Essay) => {
 return {
 content: `Essay about ${essay.topic}`,
 };
 }
);

console.log("Nodes: ");
console.log(writeEssay.nodes);
console.log("Channels: ");
console.log(writeEssay.channels);
```

```
Nodes:
{ writeEssay: PregelNode { ... } }
Channels:
{
 __start__: EphemeralValue { ... },
 __end__: LastValue { ... },
 __previous__: LastValue { ... }
}
```

# Quickstart

---

langgraph/quickstart.mdx

This quickstart demonstrates how to build a calculator agent using the LangGraph Graph API or the Functional API.

Build a LangGraph calculator agent in this working directory by following the LangGraph quickstart.

## Step 1: Read the guide

---

Detect whether this project uses Python or TypeScript/JavaScript. Fetch and follow the matching page; treat it as the source of truth for package names, model strings, and code:

- Python: <https://docs.langchain.com/oss/python/langgraph/quickstart.md>
- TypeScript: <https://docs.langchain.com/oss/javascript/langgraph/quickstart.md>

Ask the user whether to use the Graph API or the Functional API. If they have no preference, use the Graph API path.

## Step 2: Install dependencies

---

Install the packages required by the chosen path with the package manager already used in this project.

## Step 3: Configure model credentials

---

This quickstart uses Anthropic by default. Check whether `ANTHROPIC_API_KEY` is set. If it is not, ask the user to create a key and set it in the shell or a `.env` file, then stop and wait for confirmation. Do not invent, hardcode, or commit API keys. If the user prefers another chat model provider from the integrations docs, adapt the example accordingly after confirming.

## Step 4: Implement the calculator agent

---

Implement the selected quickstart path end to end: tools for add, multiply, and divide; the agent graph or functional workflow; and a sample invocation that exercises tool calling. Print the final result so the user can verify the run.

## Rules

---

- Stay scoped to this quickstart. Do not add deployment, evals, or unrelated frameworks unless the user asks.

- Prefer the APIs and structure shown on the fetched guide over inventing a different agent pattern.
- Ask rather than guess when a secret, API choice, or project convention is unclear.

## 팁

### Using an AI coding assistant?

- Install the [LangChain Docs MCP servers](#) to give your agent access to up-to-date LangChain documentation and examples.
  - Install [LangChain Skills](#) to improve your agent's performance on LangChain ecosystem tasks.
- [Use the Graph API](#) if you prefer to define your agent as a graph of nodes and edges.
- [Use the Functional API](#) if you prefer to define your agent as a single function.

For conceptual information, see [Graph API overview](#) and [Functional API overview](#).

## 정보

For this example, you will need to set up a [Claude \(Anthropic\)](#) account and get an API key. Then, set the `ANTHROPIC_API_KEY` environment variable in your terminal. See [chat model integrations](#) for all available providers. If you use [LangSmith Gateway](#), you can [bring your own provider keys](#) or use [Gateway Credits](#) to access models without a provider key.

## Use the Graph API

# 1. Define tools and model

In this example, we'll use the Claude Sonnet 4.5 model and define tools for addition, multiplication, and division.

## python

```
from langchain.tools import tool
from langchain.chat_models import init_chat_model

model = init_chat_model(
 "claude-sonnet-4-6",
 temperature=0
)

Define tools
@tool
def multiply(a: int, b: int) -> int:
```

```

 """Multiply `a` and `b`.

Args:
 a: First int
 b: Second int
"""
 return a * b

@tool
def add(a: int, b: int) -> int:
 """Adds `a` and `b`.

Args:
 a: First int
 b: Second int
"""
 return a + b

@tool
def divide(a: int, b: int) -> float:
 """Divide `a` and `b`.

Args:
 a: First int
 b: Second int
"""
 return a / b

Augment the LLM with tools
tools = [add, multiply, divide]
tools_by_name = {tool.name: tool for tool in tools}
model_with_tools = model.bind_tools(tools)

```

js

```

import { ChatAnthropic } from "@langchain/anthropic";
import { tool } from "@langchain/core/tools";
import * as z from "zod";

const model = new ChatAnthropic({
 model: "claude-sonnet-4-6",
 temperature: 0,
});

// Define tools
const add = tool(({ a, b }) => a + b, {
 name: "add",
 description: "Add two numbers",
 schema: z.object({

```

```

 a: z.number().describe("First number"),
 b: z.number().describe("Second number"),
 }},
});

const multiply = tool(({ a, b }) => a * b, {
 name: "multiply",
 description: "Multiply two numbers",
 schema: z.object({
 a: z.number().describe("First number"),
 b: z.number().describe("Second number"),
 }),
});

const divide = tool(({ a, b }) => a / b, {
 name: "divide",
 description: "Divide two numbers",
 schema: z.object({
 a: z.number().describe("First number"),
 b: z.number().describe("Second number"),
 }),
});

// Augment the LLM with tools
const toolsByName = {
 [add.name]: add,
 [multiply.name]: multiply,
 [divide.name]: divide,
};
const tools = Object.values(toolsByName);
const modelWithTools = model.bindTools(tools);

```

## 2. Define state

The graph's state is used to store the messages and the number of LLM calls.

### python

#### 팁

State in LangGraph persists throughout the agent's execution.

The `Annotated` type with `operator.add` ensures that new messages are appended to the existing list rather than replacing it.

### js

## 팁

State in LangGraph persists throughout the agent's execution.

The `MessagesValue` provides a built-in reducer for appending messages. The `llmCalls` field uses a `ReducedValue` with `(x, y) => x + y` to accumulate the count.

## python

```
from langchain.messages import AnyMessage
from typing_extensions import TypedDict, Annotated
import operator

class MessagesState(TypedDict):
 messages: Annotated[list[AnyMessage], operator.add]
 llm_calls: int
```

## js

```
import {
 StateGraph,
 StateSchema,
 MessagesValue,
 ReducedValue,
 GraphNode,
 ConditionalEdgeRouter,
 START,
 END,
} from "@langchain/langgraph";
import { z } from "zod/v4";

const MessagesState = new StateSchema({
 messages: MessagesValue,
 llmCalls: new ReducedValue(
 z.number().default(0),
 { reducer: (x, y) => x + y }
),
});
```

## 3. Define model node

The model node is used to call the LLM and decide whether to call a tool or not.

## python

```

from langchain.messages import SystemMessage

def llm_call(state: dict):
 """LLM decides whether to call a tool or not"""

 return {
 "messages": [
 model_with_tools.invoke(
 [
 SystemMessage(
 content="You are a helpful assistant tasked with performing arithmetic on a set of inputs."
)
]
 + state["messages"]
)
],
 "llm_calls": state.get('llm_calls', 0) + 1
 }

```

js

```

import { SystemMessage } from "@langchain/core/messages";

const llmCall: GraphNode<typeof MessagesState> = async (state) => {
 const response = await modelWithTools.invoke([
 new SystemMessage(
 "You are a helpful assistant tasked with performing arithmetic on a set of inputs."
),
 ...state.messages,
]);
 return {
 messages: [response],
 llmCalls: 1,
 };
};

```

## 4. Define tool node

The tool node is used to call the tools and return the results.

python

```

from langchain.messages import ToolMessage

def tool_node(state: dict):

```



```

"""Performs the tool call"""

result = []
for tool_call in state["messages"][-1].tool_calls:
 tool = tools_by_name[tool_call["name"]]
 observation = tool.invoke(tool_call["args"])
 result.append(ToolMessage(content=observation, tool_call_id=tool_call["id"]))
return {"messages": result}

```

js

```

import { AIMessage, ToolMessage } from "@langchain/core/messages";

const toolNode: GraphNode<typeof MessagesState> = async (state) => {
 const lastMessage = state.messages.at(-1);

 if (lastMessage == null || !AIMessage.isInstance(lastMessage)) {
 return { messages: [] };
 }

 const result: ToolMessage[] = [];
 for (const toolCall of lastMessage.tool_calls ?? []) {
 const tool = toolsByName[toolCall.name];
 const observation = await tool.invoke(toolCall);
 result.push(observation);
 }

 return { messages: result };
};

```

## 5. Define end logic

The conditional edge function is used to route to the tool node or end based upon whether the LLM made a tool call.

python

```

from typing import Literal
from langgraph.graph import StateGraph, START, END

def should_continue(state: MessagesState) -> Literal["tool_node", END]:
 """Decide if we should continue the loop or stop based upon whether the LLM made a tool
 call"""

 messages = state["messages"]
 last_message = messages[-1]

```

```
If the LLM makes a tool call, then perform an action
if last_message.tool_calls:
 return "tool_node"

Otherwise, we stop (reply to the user)
return END
```

js

```
const shouldContinue: ConditionalEdgeRouter<{ InputSchema: typeof MessagesState; Nodes: "toolNode" }> = (state) => {
 const lastMessage = state.messages.at(-1);

 // Check if it's an AIMessage before accessing tool_calls
 if (!lastMessage || !AIMessage.isInstance(lastMessage)) {
 return END;
 }

 // If the LLM makes a tool call, then perform an action
 if (lastMessage.tool_calls?.length) {
 return "toolNode";
 }

 // Otherwise, we stop (reply to the user)
 return END;
};
```

## 6. Build and compile the agent

The agent is built using the `@[StateGraph]` class and compiled using the `@[compile]` `[StateGraph.compile]` method.

python

```
Build workflow
agent_builder = StateGraph(MessagesState)

Add nodes
agent_builder.add_node("llm_call", llm_call)
agent_builder.add_node("tool_node", tool_node)

Add edges to connect nodes
agent_builder.add_edge(START, "llm_call")
agent_builder.add_conditional_edges(
 "llm_call",
```

```

 should_continue,
 ["tool_node", END]
)
 agent_builder.add_edge("tool_node", "llm_call")

 # Compile the agent
 agent = agent_builder.compile()

 # Show the agent
 from IPython.display import Image, display
 display(Image(agent.get_graph(xray=True).draw_mermaid_png()))

 # Invoke
 from langchain.messages import HumanMessage
 messages = [HumanMessage(content="Add 3 and 4.")]
 messages = agent.invoke({"messages": messages})
 for m in messages["messages"]:
 m.pretty_print()

```

js

```

const agent = new StateGraph(MessagesState)
 .addNode("llmCall", llmCall)
 .addNode("toolNode", toolNode)
 .addEdge(START, "llmCall")
 .addConditionalEdges("llmCall", shouldContinue, ["toolNode", END])
 .addEdge("toolNode", "llmCall")
 .compile();

// Invoke
import { HumanMessage } from "@langchain/core/messages";
const result = await agent.invoke({
 messages: [new HumanMessage("Add 3 and 4.")],
});

for (const message of result.messages) {
 console.log(`[${message.type}]: ${message.text}`);
}

```

## 팁

Trace and debug your agent with [LangSmith](#). Follow the [tracing quickstart](#) to get set up. When ready for production, see [Deploy](#) for hosting options.

We recommend you also set up [LangSmith Engine](#) which monitors your traces, detects issues, and proposes fixes.

Congratulations! You've built your first agent using the LangGraph Graph API.

## Full code example

### python

```
Step 1: Define tools and model

from langchain.tools import tool
from langchain.chat_models import init_chat_model

model = init_chat_model(
 "claude-sonnet-4-6",
 temperature=0
)

Define tools
@tool
def multiply(a: int, b: int) -> int:
 """Multiply `a` and `b`."""

 Args:
 a: First int
 b: Second int
 """
 return a * b

@tool
def add(a: int, b: int) -> int:
 """Adds `a` and `b`."""

 Args:
 a: First int
 b: Second int
 """
 return a + b

@tool
def divide(a: int, b: int) -> float:
 """Divide `a` and `b`."""

 Args:
 a: First int
 b: Second int
 """
 return a / b

Augment the LLM with tools
tools = [add, multiply, divide]
tools_by_name = {tool.name: tool for tool in tools}
```

```

model_with_tools = model.bind_tools(tools)

Step 2: Define state

from langchain.messages import AnyMessage
from typing_extensions import TypedDict, Annotated
import operator

class MessagesState(TypedDict):
 messages: Annotated[list[AnyMessage], operator.add]
 llm_calls: int

Step 3: Define model node
from langchain.messages import SystemMessage

def llm_call(state: MessagesState):
 """LLM decides whether to call a tool or not"""

 return {
 "messages": [
 model_with_tools.invoke(
 [
 SystemMessage(
 content="You are a helpful assistant tasked with performing arithmetic o
n a set of inputs."
)
]
 + state["messages"]
)
],
 "llm_calls": state.get('llm_calls', 0) + 1
 }

Step 4: Define tool node

from langchain.messages import ToolMessage

def tool_node(state: MessagesState):
 """Performs the tool call"""

 result = []
 for tool_call in state["messages"][-1].tool_calls:
 tool = tools_by_name[tool_call["name"]]
 observation = tool.invoke(tool_call["args"])
 result.append(ToolMessage(content=observation, tool_call_id=tool_call["id"]))
 return {"messages": result}

Step 5: Define logic to determine whether to end

from typing import Literal
from langgraph.graph import StateGraph, START, END

```

```

Conditional edge function to route to the tool node or end based upon whether the LLM made
a tool call
def should_continue(state: MessagesState) -> Literal["tool_node", END]:
 """Decide if we should continue the loop or stop based upon whether the LLM made a tool
 call"""

 messages = state["messages"]
 last_message = messages[-1]

 # If the LLM makes a tool call, then perform an action
 if last_message.tool_calls:
 return "tool_node"

 # Otherwise, we stop (reply to the user)
 return END

Step 6: Build agent

Build workflow
agent_builder = StateGraph(MessagesState)

Add nodes
agent_builder.add_node("llm_call", llm_call)
agent_builder.add_node("tool_node", tool_node)

Add edges to connect nodes
agent_builder.add_edge(START, "llm_call")
agent_builder.add_conditional_edges(
 "llm_call",
 should_continue,
 ["tool_node", END]
)
agent_builder.add_edge("tool_node", "llm_call")

Compile the agent
agent = agent_builder.compile()

from IPython.display import Image, display
Show the agent
display(Image(agent.get_graph(xray=True).draw_mermaid_png()))

Invoke
from langchain.messages import HumanMessage
messages = [HumanMessage(content="Add 3 and 4.")]
messages = agent.invoke({"messages": messages})
for m in messages["messages"]:
 m.pretty_print()

```

js

```
// Step 1: Define tools and model

import { ChatAnthropic } from "@langchain/anthropic";
import { tool } from "@langchain/core/tools";
import * as z from "zod";

const model = new ChatAnthropic({
 model: "claude-sonnet-4-6",
 temperature: 0,
});

// Define tools
const add = tool(({ a, b }) => a + b, {
 name: "add",
 description: "Add two numbers",
 schema: z.object({
 a: z.number().describe("First number"),
 b: z.number().describe("Second number"),
 }),
});

const multiply = tool(({ a, b }) => a * b, {
 name: "multiply",
 description: "Multiply two numbers",
 schema: z.object({
 a: z.number().describe("First number"),
 b: z.number().describe("Second number"),
 }),
});

const divide = tool(({ a, b }) => a / b, {
 name: "divide",
 description: "Divide two numbers",
 schema: z.object({
 a: z.number().describe("First number"),
 b: z.number().describe("Second number"),
 }),
});

// Augment the LLM with tools
const toolsByName = {
 [add.name]: add,
 [multiply.name]: multiply,
 [divide.name]: divide,
};

const tools = Object.values(toolsByName);
const modelWithTools = model.bindTools(tools);
```

```
// Step 2: Define state
```

```
import {
 StateGraph,
 StateSchema,
 MessagesValue,
 ReducedValue,
 GraphNode,
 ConditionalEdgeRouter,
 START,
 END,
} from "@langchain/langgraph";
import * as z from "zod";

const MessagesState = new StateSchema({
 messages: MessagesValue,
 llmCalls: new ReducedValue(
 z.number().default(0),
 { reducer: (x, y) => x + y }
),
});
```

```
// Step 3: Define model node
```

```
import { SystemMessage, AIMessage, ToolMessage } from "@langchain/core/messages";

const llmCall: GraphNode<typeof MessagesState> = async (state) => {
 return {
 messages: [await modelWithTools.invoke([
 new SystemMessage(
 "You are a helpful assistant tasked with performing arithmetic on a set of inputs."
),
 ...state.messages,
])],
 llmCalls: 1,
 };
};
```

```
// Step 4: Define tool node
```

```
const toolNode: GraphNode<typeof MessagesState> = async (state) => {
 const lastMessage = state.messages.at(-1);

 if (lastMessage == null || !AIMessage.isInstance(lastMessage)) {
 return { messages: [] };
 }

 const result: ToolMessage[] = [];
 for (const toolCall of lastMessage.tool_calls ?? []) {
 const tool = toolsByName[toolCall.name];
```



```

 const observation = await tool.invoke(toolCall);
 result.push(observation);
 }

 return { messages: result };
};

```

```

// Step 5: Define logic to determine whether to end
import { ConditionalEdgeRouter, END } from "@langchain/langgraph";

const shouldContinue: ConditionalEdgeRouter<{ InputSchema: typeof MessagesState; Nodes: "toolNode" }> = (state) => {
 const lastMessage = state.messages.at(-1);

 // Check if it's an AIMessage before accessing tool_calls
 if (!lastMessage || !AIMessage.isInstance(lastMessage)) {
 return END;
 }

 // If the LLM makes a tool call, then perform an action
 if (lastMessage.tool_calls?.length) {
 return "toolNode";
 }

 // Otherwise, we stop (reply to the user)
 return END;
};

```

```

// Step 6: Build and compile the agent
import { HumanMessage } from "@langchain/core/messages";
import { StateGraph, START, END } from "@langchain/langgraph";

const agent = new StateGraph(MessagesState)
 .addNode("llmCall", llmCall)
 .addNode("toolNode", toolNode)
 .addEdge(START, "llmCall")
 .addConditionalEdges("llmCall", shouldContinue, ["toolNode", END])
 .addEdge("toolNode", "llmCall")
 .compile();

// Invoke
const result = await agent.invoke({
 messages: [new HumanMessage("Add 3 and 4.")],
});

for (const message of result.messages) {
 console.log(`[${message.type}]: ${message.text}`);
}

```

# 1. Define tools and model

---

In this example, we'll use the Claude Sonnet 4.5 model and define tools for addition, multiplication, and division.

**python**

```
from langchain.tools import tool
from langchain.chat_models import init_chat_model

model = init_chat_model(
 "claude-sonnet-4-6",
 temperature=0
)

Define tools
@tool
def multiply(a: int, b: int) -> int:
 """Multiply `a` and `b`.
 Args:
 a: First int
 b: Second int
 """
 return a * b

@tool
def add(a: int, b: int) -> int:
 """Adds `a` and `b`.
 Args:
 a: First int
 b: Second int
 """
 return a + b

@tool
def divide(a: int, b: int) -> float:
 """Divide `a` and `b`.
 Args:
 a: First int
 b: Second int
 """
 return a / b

Augment the LLM with tools
```

```

tools = [add, multiply, divide]
tools_by_name = {tool.name: tool for tool in tools}
model_with_tools = model.bind_tools(tools)

from langgraph.graph import add_messages
from langchain.messages import (
 SystemMessage,
 HumanMessage,
 ToolCall,
)
from langchain_core.messages import BaseMessage
from langgraph.func import entrypoint, task

```

js

```

import { ChatAnthropic } from "@langchain/anthropic";
import { tool } from "@langchain/core/tools";
import * as z from "zod";

const model = new ChatAnthropic({
 model: "claude-sonnet-4-6",
 temperature: 0,
});

// Define tools
const add = tool(({ a, b }) => a + b, {
 name: "add",
 description: "Add two numbers",
 schema: z.object({
 a: z.number().describe("First number"),
 b: z.number().describe("Second number"),
 }),
});

const multiply = tool(({ a, b }) => a * b, {
 name: "multiply",
 description: "Multiply two numbers",
 schema: z.object({
 a: z.number().describe("First number"),
 b: z.number().describe("Second number"),
 }),
});

const divide = tool(({ a, b }) => a / b, {
 name: "divide",
 description: "Divide two numbers",
 schema: z.object({
 a: z.number().describe("First number"),
 b: z.number().describe("Second number"),
 }),
});

```

```

 }),
 });

 // Augment the LLM with tools
 const toolsByName = {
 [add.name]: add,
 [multiply.name]: multiply,
 [divide.name]: divide,
 };
 const tools = Object.values(toolsByName);
 const modelWithTools = model.bindTools(tools);

```

## 2. Define model node

The model node is used to call the LLM and decide whether to call a tool or not.

### python

#### 팁

The `@task` decorator marks a function as a task that can be executed as part of the agent. Tasks can be called synchronously or asynchronously within your entrypoint function.

```

@task
def call_llm(messages: list[BaseMessage]):
 """LLM decides whether to call a tool or not"""
 return model_with_tools.invoke(
 [
 SystemMessage(
 content="You are a helpful assistant tasked with performing arithmetic on a
set of inputs."
)
]
 + messages
)

```

### js

```

import { task, entrypoint } from "@langchain/langgraph";
import { SystemMessage } from "@langchain/core/messages";

const callLlm = task({ name: "callLlm" }, async (messages: BaseMessage[]) => {
 return modelWithTools.invoke([

```

```

 new SystemMessage(
 "You are a helpful assistant tasked with performing arithmetic on a set of inputs."
),
 ...messages,
]);
});

```

### 3. Define tool node

The tool node is used to call the tools and return the results.

#### python

```

@task
def call_tool(tool_call: ToolCall):
 """Performs the tool call"""
 tool = tools_by_name[tool_call["name"]]
 return tool.invoke(tool_call)

```

#### js

```

import type { ToolCall } from "@langchain/core/messages/tool";

const callTool = task({ name: "callTool" }, async (toolCall: ToolCall) => {
 const tool = toolsByName[toolCall.name];
 return tool.invoke(toolCall);
});

```

### 4. Define agent

#### python

The agent is built using the `@entrypoint` function.

#### 참고

In the Functional API, instead of defining nodes and edges explicitly, you write standard control flow logic (loops, conditionals) within a single function.

```

@entrypoint()
def agent(messages: list[BaseMessage]):
 model_response = call_llm(messages).result()

```

```

while True:
 if not model_response.tool_calls:
 break

 # Execute tools
 tool_result_futures = [
 call_tool(tool_call) for tool_call in model_response.tool_calls
]
 tool_results = [fut.result() for fut in tool_result_futures]
 messages = add_messages(messages, [model_response, *tool_results])
 model_response = call_llm(messages).result()

 messages = add_messages(messages, model_response)
 return messages

Invoke
messages = [HumanMessage(content="Add 3 and 4.")]
stream = agent.stream_events(messages, version="v3")
for snapshot in stream.values:
 print(snapshot)
 print("\n")

```

js

```

import { addMessages } from "@langchain/langgraph";
import { type BaseMessage } from "@langchain/core/messages";

const agent = entrypoint({ name: "agent" }, async (messages: BaseMessage[]) => {
 let modelResponse = await callLlm(messages);

 while (true) {
 if (!modelResponse.tool_calls?.length) {
 break;
 }

 // Execute tools
 const toolResults = await Promise.all(
 modelResponse.tool_calls.map((toolCall) => callTool(toolCall))
);
 messages = addMessages(messages, [modelResponse, ...toolResults]);
 modelResponse = await callLlm(messages);
 }

 return messages;
});

// Invoke
import { HumanMessage } from "@langchain/core/messages";

```

```
const result = await agent.invoke([new HumanMessage("Add 3 and 4.")]);

for (const message of result) {
 console.log(`[${message.getType()}]: ${message.text}`);
}
```

## 팁

Trace and debug your agent with [LangSmith](#). Follow the [tracing quickstart](#) to get set up. When ready for production, see [Deploy](#) for hosting options.

We recommend you also set up [LangSmith Engine](#) which monitors your traces, detects issues, and proposes fixes.

Congratulations! You've built your first agent using the LangGraph Functional API.

## Full code example

### python

```
Step 1: Define tools and model

from langchain.tools import tool
from langchain.chat_models import init_chat_model

model = init_chat_model(
 "claude-sonnet-4-6",
 temperature=0
)

Define tools
@tool
def multiply(a: int, b: int) -> int:
 """Multiply `a` and `b`.

Args:

a: First int

b: Second int

"""
 return a * b

@tool
def add(a: int, b: int) -> int:
 """Adds `a` and `b`.

Args:

a: First int


```

```

 b: Second int
 """
 return a + b

@tool
def divide(a: int, b: int) -> float:
 """Divide `a` and `b`."""

 Args:
 a: First int
 b: Second int
 """
 return a / b

Augment the LLM with tools
tools = [add, multiply, divide]
tools_by_name = {tool.name: tool for tool in tools}
model_with_tools = model.bind_tools(tools)

from langgraph.graph import add_messages
from langchain.messages import (
 SystemMessage,
 HumanMessage,
 ToolCall,
)
from langchain_core.messages import BaseMessage
from langgraph.func import entrypoint, task

Step 2: Define model node

@task
def call_llm(messages: list[BaseMessage]):
 """LLM decides whether to call a tool or not"""
 return model_with_tools.invoke(
 [
 SystemMessage(
 content="You are a helpful assistant tasked with performing arithmetic on a set of inputs."
)
]
 + messages
)

Step 3: Define tool node

@task
def call_tool(tool_call: ToolCall):
 """Performs the tool call"""
 tool = tools_by_name[tool_call["name"]]
 return tool.invoke(tool_call)

```



```

Step 4: Define agent

@entrypoint()
def agent(messages: list[BaseMessage]):
 model_response = call_llm(messages).result()

 while True:
 if not model_response.tool_calls:
 break

 # Execute tools
 tool_result_futures = [
 call_tool(tool_call) for tool_call in model_response.tool_calls
]
 tool_results = [fut.result() for fut in tool_result_futures]
 messages = add_messages(messages, [model_response, *tool_results])
 model_response = call_llm(messages).result()

 messages = add_messages(messages, model_response)
 return messages

Invoke
messages = [HumanMessage(content="Add 3 and 4.")]
stream = agent.stream_events(messages, version="v3")
for snapshot in stream.values:
 print(snapshot)
 print("\n")

```

js

```

import { ChatAnthropic } from "@langchain/anthropic";
import { tool } from "@langchain/core/tools";
import {
 task,
 entrypoint,
 addMessages,
} from "@langchain/langgraph";
import {
 SystemMessage,
 HumanMessage,
 type BaseMessage,
} from "@langchain/core/messages";
import type { ToolCall } from "@langchain/core/messages/tool";
import * as z from "zod";

// Step 1: Define tools and model

const model = new ChatAnthropic({
 model: "claude-sonnet-4-6",

```

```

 temperature: 0,
 });

// Define tools
const add = tool(({ a, b }) => a + b, {
 name: "add",
 description: "Add two numbers",
 schema: z.object({
 a: z.number().describe("First number"),
 b: z.number().describe("Second number"),
 }),
});

const multiply = tool(({ a, b }) => a * b, {
 name: "multiply",
 description: "Multiply two numbers",
 schema: z.object({
 a: z.number().describe("First number"),
 b: z.number().describe("Second number"),
 }),
});

const divide = tool(({ a, b }) => a / b, {
 name: "divide",
 description: "Divide two numbers",
 schema: z.object({
 a: z.number().describe("First number"),
 b: z.number().describe("Second number"),
 }),
});

// Augment the LLM with tools
const toolsByName = {
 [add.name]: add,
 [multiply.name]: multiply,
 [divide.name]: divide,
};
const tools = Object.values(toolsByName);
const modelWithTools = model.bindTools(tools);

// Step 2: Define model node

const callLlm = task({ name: "callLlm" }, async (messages: BaseMessage[]) => {
 return modelWithTools.invoke([
 new SystemMessage(
 "You are a helpful assistant tasked with performing arithmetic on a set of inputs."
),
 ...messages,
]);
});

```

```

// Step 3: Define tool node

const callTool = task({ name: "callTool" }, async (toolCall: ToolCall) => {
 const tool = toolsByName[toolCall.name];
 return tool.invoke(toolCall);
});

// Step 4: Define agent

const agent = entrypoint({ name: "agent" }, async (messages: BaseMessage[]) => {
 let modelResponse = await callLlm(messages);

 while (true) {
 if (!modelResponse.tool_calls?.length) {
 break;
 }

 // Execute tools
 const toolResults = await Promise.all(
 modelResponse.tool_calls.map((toolCall) => callTool(toolCall))
);
 messages = addMessages(messages, [modelResponse, ...toolResults]);
 modelResponse = await callLlm(messages);
 }

 return messages;
});

// Invoke

const result = await agent.invoke([new HumanMessage("Add 3 and 4.")]);

for (const message of result) {
 console.log(`[${message.type}]: ${message.text}`);
}

```

# Build a custom SQL agent

---

langgraph/sql-agent.mdx

In this tutorial we will build a custom agent that can answer questions about a SQL database using LangGraph.

LangChain offers built-in [agent](#) implementations, implemented using [LangGraph](#) primitives. If deeper customization is required, agents can be implemented directly in LangGraph. This guide demonstrates an example implementation of a SQL agent. For a practical introduction, see [building a SQL agent using higher-level LangChain abstractions](#).

## 주의

Building Q&A systems of SQL databases requires executing model-generated SQL queries. There are inherent risks in doing this. Make sure that your database connection permissions are always scoped as narrowly as possible for your agent's needs. This will mitigate, though not eliminate, the risks of building a model-driven system.

The [prebuilt agent](#) lets us get started quickly, but we relied on the system prompt to constrain its behavior—for example, we instructed the agent to always start with the "list tables" tool, and to always run a query-checker tool before executing the query.

We can enforce a higher degree of control in LangGraph by customizing the agent. Here, we implement a simple ReAct-agent setup, with dedicated nodes for specific tool-calls. We will use the same [state] as the prebuilt agent.

## Concepts

We will cover the following concepts:

- [Tools](#) for reading from SQL databases
- The LangGraph [Graph API](#), including state, nodes, edges, and conditional edges.
- [Human-in-the-loop](#) processes

## Setup

---

### Installation

python

```
```bash pip install langchain langgraph
```

```
</div>
```

```
<div class="callout js" markdown="1">
```

```
<p class="callout-label">js</p>
```

```
```bash npm
```

```
npm i langchain @langchain/core @langchain/langgraph @langchain/openai sqlite3 zod
```

```
```bash yarn yarn add langchain @langchain/core @langchain/langgraph @langchain/openai  
sqlite3 zod
```

```
```bash pnpm
```

```
pnpm add langchain @langchain/core @langchain/langgraph @langchain/openai sqlite3 zod
```

## LangSmith

Set up [LangSmith](#) to inspect what is happening inside your chain or agent. Then set the following environment variables:

```
export LANGSMITH_TRACING="true"
export LANGSMITH_API_KEY="..."
```

## 1. Select an LLM

Select a model that supports [tool-calling](#):

python

js

The output shown in the examples below used OpenAI.

## 2. Configure the database

You will be creating a [SQLite database](#) for this tutorial. SQLite is a lightweight database that is easy to set up and use. We will be loading the `chinook` database, which is a sample database that represents a digital media store.

For convenience, we have hosted the database ( `chinook.db` ) on a public GCS bucket.

python

We will use Python's built-in `sqlite3` module to interact with the database:

```
Dialect: sqlite
Available tables: ['Album', 'Artist', 'Customer', 'Employee', 'Genre', 'Invoice', 'InvoiceLine', 'MediaType', 'Playlist', 'PlaylistTrack', 'Track']
Sample output: [(1, 'AC/DC'), (2, 'Accept'), (3, 'Aerosmith'), (4, 'Alanis Morissette'), (5, 'Alice In Chains')]
```

## js

We will use the `sqlite3` library to interact with the database:

```
Dialect: sqlite
Available tables: Album, Artist, Customer, Employee, Genre, Invoice, InvoiceLine, MediaType, Playlist, PlaylistTrack, Track
Sample output: [{"ArtistId":1,"Name":"AC/DC"}, {"ArtistId":2,"Name":"Accept"}, {"ArtistId":3,"Name":"Aerosmith"}, {"ArtistId":4,"Name":"Alanis Morissette"}, {"ArtistId":5,"Name":"Alice In Chains"}]
```

## 3. Add tools for database interactions

### 주의

The following database tools are minimal wrappers for demonstration purposes only. They are not intended to be secure or used in production. Use narrowly scoped database permissions and add application-specific validation before executing model-generated SQL.

## python

We can implement database [tools](#) as thin wrappers using the `@tool` decorator from `langchain.tools`:

`sql_db_list_tables`: Input is an empty string, output is a comma-separated list of tables in the database.

`sql_db_schema`: Input to this tool is a comma-separated list of tables, output is the schema and sample rows for those tables.

Be sure that the tables actually exist by calling `sql_db_list_tables` first!

Example Input: table1, table2, table3

`sql_db_query`: Input to this tool is a detailed and correct SQL query, output is a result from the database.

If the query is not correct, an error message will be returned.

If an error is returned, rewrite the query, check the query, and try again.

If you encounter an issue with Unknown column 'xxxx' in 'field list', use `sql_db_schema` to query the correct table fields.

js

We'll create custom tools to interact with the database:

`sql_db_list_tables`: Input is an empty string, output is a comma-separated list of tables in the database.

`sql_db_schema`: Input to this tool is a comma-separated list of tables, output is the schema and sample rows for those tables. Be sure that the tables actually exist by calling `sql_db_list_tables` first! Example Input: `table1, table2, table3`

`sql_db_query`: Input to this tool is a detailed and correct SQL query, output is a result from the database. If the query is not correct, an error message will be returned. If an error is returned, rewrite the query, check the query, and try again.

## 4. Define application steps

We construct dedicated nodes for the following steps:

- Listing DB tables
- Calling the "get schema" tool
- Generating a query
- Checking the query

Putting these steps in dedicated nodes lets us (1) force tool-calls when needed, and (2) customize the prompts associated with each step.

python

js

## 5. Implement the agent

We can now assemble these steps into a workflow using the [Graph API](#). We define a [conditional edge](#) at the query generation step that will route to the query checker if a query is generated, or end if there are no tool calls present, such that the LLM has delivered a response to the query.

python

We visualize the application below:

js

We visualize the application below:

 SQL agent graph

We can now invoke the graph:

python

js

```
===== Human Message =====

Which genre on average has the longest tracks?
===== Ai Message =====

Available tables: Album, Artist, Customer, Employee, Genre, Invoice, InvoiceLine, MediaType, Playlist, PlaylistTrack, Track
===== Ai Message =====

Tool Calls:
 sql_db_schema (call_yzje0tj7JK3TEzDx4QnRR3lL)
Call ID: call_yzje0tj7JK3TEzDx4QnRR3lL
Args:
 table_names: Genre, Track
===== Tool Message =====
Name: sql_db_schema

CREATE TABLE "Genre" (
 "GenreId" INTEGER NOT NULL,
 "Name" NVARCHAR(120),
 PRIMARY KEY ("GenreId")
)

/*
3 rows from Genre table:
GenreId Name
1 Rock
2 Jazz
3 Metal
*/

CREATE TABLE "Track" (
 "TrackId" INTEGER NOT NULL,
 "Name" NVARCHAR(200) NOT NULL,
 "AlbumId" INTEGER,
 "MediaTypeId" INTEGER NOT NULL,
 "GenreId" INTEGER,
```



```

"Composer" NVARCHAR(220),
"Milliseconds" INTEGER NOT NULL,
"Bytes" INTEGER,
"UnitPrice" NUMERIC(10, 2) NOT NULL,
PRIMARY KEY ("TrackId"),
FOREIGN KEY("MediaTypeId") REFERENCES "MediaType" ("MediaTypeId"),
FOREIGN KEY("GenreId") REFERENCES "Genre" ("GenreId"),
FOREIGN KEY("AlbumId") REFERENCES "Album" ("AlbumId")
)

/*
3 rows from Track table:
TrackId Name AlbumId MediaTypeId GenreId Composer Milliseconds Bytes UnitPrice
1 For Those About To Rock (We Salute You) 1 1 1 Angus Young, Malcolm Young, Brian Johnson 343719 11170334 0.99
2 Balls to the Wall 2 2 1 U. Dirkschneider, W. Hoffmann, H. Frank, P. Baltes, S. Kaufmann, G. Hoffmann 342562 5510424 0.99
3 Fast As a Shark 3 2 1 F. Baltes, S. Kaufman, U. Dirkschneider & W. Hoffman 230619 3990994 0.99
*/

===== Ai Message =====
Tool Calls:
sql_db_query (call_cb9ApLfZLSq7CWg6jd0im90b)
Call ID: call_cb9ApLfZLSq7CWg6jd0im90b
Args:
query: SELECT Genre.Name, AVG(Track.Milliseconds) AS AvgMilliseconds FROM Track JOIN Genre ON Track.GenreId = Genre.GenreId GROUP BY Genre.GenreId ORDER BY AvgMilliseconds DESC LIMIT 5;
===== Ai Message =====
Tool Calls:
sql_db_query (call_DMVALfnQ4kJsuF3Yl6jxbeAU)
Call ID: call_DMVALfnQ4kJsuF3Yl6jxbeAU
Args:
query: SELECT Genre.Name, AVG(Track.Milliseconds) AS AvgMilliseconds FROM Track JOIN Genre ON Track.GenreId = Genre.GenreId GROUP BY Genre.GenreId ORDER BY AvgMilliseconds DESC LIMIT 5;
===== Tool Message =====
Name: sql_db_query

[('Sci Fi & Fantasy', 2911783.0384615385), ('Science Fiction', 2625549.076923077), ('Drama', 2575283.78125), ('TV Shows', 2145041.0215053763), ('Comedy', 1585263.705882353)]
===== Ai Message =====

The genre with the longest tracks on average is "Sci Fi & Fantasy," with an average track length of approximately 2,911,783 milliseconds. Other genres with relatively long tracks include "Science Fiction," "Drama," "TV Shows," and "Comedy."

```

## python

팁

See [LangSmith trace](#) for the above run.

js

팁

See [LangSmith trace](#) for the above run.

## 6. Implement human-in-the-loop review

It can be prudent to check the agent's SQL queries before they are executed for any unintended actions or inefficiencies.

Here we leverage LangGraph's [human-in-the-loop](#) features to pause the run before executing a SQL query and wait for human review. Using LangGraph's [persistence layer](#), we can pause the run indefinitely (or at least as long as the persistence layer is alive).

Let's wrap the `sql_db_query` tool in a node that receives human input. We can implement this using the [interrupt](#) function. Below, we allow for input to approve the tool call, edit its arguments, or provide user feedback.

python

js

참고

The above implementation follows the [tool interrupt example](#) in the broader [human-in-the-loop](#) guide. Refer to that guide for details and alternatives.

Let's now re-assemble our graph. We will replace the programmatic check with human review. Note that we now include a [checkpoint](#); this is required to pause and resume the run.

python

js

We can invoke the graph as before. This time, execution is interrupted:

python

js

```
...

INTERRUPTED:
{
 "action": "sql_db_query",
 "args": {
 "query": "SELECT Genre.Name, AVG(Track.Milliseconds) AS AvgLength FROM Track JOIN Genre ON T
rack.GenreId = Genre.GenreId GROUP BY Genre.Name ORDER BY AvgLength DESC LIMIT 5;"
 },
 "description": "Please review the tool call"
}
```

We can accept or edit the tool call using [Command](#):

python

js

```
===== Ai Message =====
Tool Calls:
 sql_db_query (call_t4yXkD6shwdTPuelXEmY3sAY)
Call ID: call_t4yXkD6shwdTPuelXEmY3sAY
Args:
 query: SELECT Genre.Name, AVG(Track.Milliseconds) AS AvgLength FROM Track JOIN Genre ON Trac
k.GenreId = Genre.GenreId GROUP BY Genre.Name ORDER BY AvgLength DESC LIMIT 5;
===== Tool Message =====
Name: sql_db_query

[(('Sci Fi & Fantasy', 2911783.0384615385), ('Science Fiction', 2625549.076923077), ('Drama', 257
5283.78125), ('TV Shows', 2145041.0215053763), ('Comedy', 1585263.705882353))]
===== Ai Message =====

The genre with the longest average track length is "Sci Fi & Fantasy" with an average length of
about 2,911,783 milliseconds. Other genres with long average track lengths include "Science Fict
ion," "Drama," "TV Shows," and "Comedy."
```

Refer to the [human-in-the-loop guide](#) for details.

## Next steps

Check out the [Evaluate a graph](#) guide for evaluating LangGraph applications, including SQL agents like this one, using LangSmith.

# Stores

---

langgraph/stores.mdx

Stores let agents persist information across threads, including user preferences, accumulated knowledge, and facts that should survive beyond a single conversation. Unlike [checkpointer](#)s, which save the full graph state scoped to one thread, stores hold arbitrary key-value data accessible from any thread.



Model of shared state

## 정보

**Agent Server handles stores automatically** When using the [Agent Server](#), you do not need to implement or configure stores manually. The API handles all storage infrastructure for you behind the scenes.

## 참고

`@InMemoryStore` is suitable for development and testing. For production, use a persistent store like `PostgresStore`, `MongoDBStore`, `RedisStore`, OR `UpstashStore`. All implementations extend `@BaseStore`, which is the type annotation to use in node function signatures.

## 참고

See [store integrations](#) for the full list of available providers.

## Basic usage

---

The following code snippet shows the `@InMemoryStore` in isolation without using LangGraph:

### python

```
from langgraph.store.memory import InMemoryStore
store = InMemoryStore()
```

### js

```
import { MemoryStore } from "@langchain/langgraph";

const memoryStore = new MemoryStore();
```

Memories are namespaced by a `tuple`, which is `(<user_id>, "memories")` in the following example. The namespace can be any length and represent anything, does not have to be user specific.

#### python

```
user_id = "1"
namespace_for_memory = (user_id, "memories")
```

#### js

```
const userId = "1";
const namespaceForMemory = [userId, "memories"];
```

Use the `store.put` method to save memories to the namespace in the store. Specify the namespace, as defined above, and a key-value pair for the memory: the key is simply a unique identifier for the memory (`memory_id`) and the value (a dictionary) is the memory itself.

#### python

```
memory_id = str(uuid.uuid4())
memory = {"food_preference" : "I like pizza"}
store.put(namespace_for_memory, memory_id, memory)
```

#### js

```
const memoryId = crypto.randomUUID();
const memory = { food_preference: "I like pizza" };
await memoryStore.put(namespaceForMemory, memoryId, memory);
```

Read out memories from your namespace using the `store.search` method, which returns memories for a given user as a list, up to the `limit` argument (default `10`). With `InMemoryStore`, items are returned in insertion order, so the most recent memory is last in the list; other backends may order memories differently (see [Listing items in a namespace](#)).

#### python

```
memories = store.search(namespace_for_memory)
memories[-1].dict()
{'value': {'food_preference': 'I like pizza'},
 'key': '07e0caf4-1631-47b7-b15f-65515d4c1843',
 'namespace': ['1', 'memories'],
```

```
'created_at': '2024-10-02T17:22:31.590602+00:00',
'updated_at': '2024-10-02T17:22:31.590605+00:00'}
```

Each memory type is a Python class ([Item](#)) with certain attributes. We can access it as a dictionary by converting with `.dict`.

The attributes it has are:

- `value`: The value (itself a dictionary) of this memory
- `key`: A unique key for this memory in this namespace
- `namespace`: A tuple of strings, the namespace of this memory type

## 참고

While the type is `tuple[str, ...]`, it may be serialized as a list when converted to JSON (for example, `['1', 'memories']`).

- `created_at`: Timestamp for when this memory was created
- `updated_at`: Timestamp for when this memory was updated

## js

```
const memories = await memoryStore.search(namespaceForMemory);
memories[memories.length - 1];

// {
// value: { food_preference: 'I like pizza' },
// key: '07e0caf4-1631-47b7-b15f-65515d4c1843',
// namespace: ['1', 'memories'],
// createdAt: '2024-10-02T17:22:31.590602+00:00',
// updatedAt: '2024-10-02T17:22:31.590605+00:00'
// }
```

The attributes it has are:

- `value`: The value of this memory
- `key`: A unique key for this memory in this namespace
- `namespace`: A tuple of strings, the namespace of this memory type

## 참고

While the type is `tuple`, it may be serialized as a list when converted to JSON (for example, `['1', 'memories']`).

- `createdAt`: Timestamp for when this memory was created

- `updatedAt` : Timestamp for when this memory was updated

## Listing items in a namespace

### python

Calling `store.search` (or the async `store.asearch`) with no `query` and no `filter` returns the items stored under `namespace_prefix`, up to `limit`. Use this to enumerate everything in a namespace when you don't need semantic ranking.

### js

Calling `store.search` with no `query` and no `filter` returns the items stored under the namespace prefix, up to `limit`. Use this to enumerate everything in a namespace when you don't need semantic ranking.

### python

### js

Three behaviors to keep in mind:

- `namespace_prefix` **matches by prefix, not exactly.** `("alice",)` also returns items under `("alice", "memories")`, `("alice", "preferences")`, and so on. To restrict to a single level, pass the full namespace or filter the returned items client-side on `item.namespace`.
- **Results past `limit` are silently truncated.** There is no overflow signal—set `limit` above your expected maximum, or paginate with `offset`.
- **Default ordering depends on the store backend.** `PostgresStore` and `AsyncPostgresStore` return results ordered by `updated_at` descending (most recently updated first). `InMemoryStore` returns results in insertion order (most recently inserted last). Do not rely on a specific order across implementations; sort client-side on `item.updated_at` if order matters.

### python

To page through a large namespace:

### js

To page through a large namespace:

### python

To discover which namespaces exist (for example, to iterate over every user before listing their memories), use `store.list_namespaces` or `store.alist_namespaces`:

**js**

To discover which namespaces exist (for example, to iterate over every user before listing their memories), use `store.listNamespaces`:

## Semantic search

Beyond simple retrieval, the store also supports semantic search, allowing you to find memories based on meaning rather than exact matches. To enable this, configure the store with an embedding model:

**python**

```
from langchain.embeddings import init_embeddings

store = InMemoryStore(
 index={
 "embed": init_embeddings("openai:text-embedding-3-small"), # Embedding provider
 "dims": 1536, # Embedding dimensions
 "fields": ["food_preference", "$"] # Fields to embed
 }
)
```

**js**

```
import { OpenAIEmbeddings } from "@langchain/openai";

const store = new InMemoryStore({
 index: {
 embeddings: new OpenAIEmbeddings({ model: "text-embedding-3-small" }),
 dims: 1536,
 fields: ["food_preference", "$"], // Fields to embed
 },
});
```

Now when searching, you can use natural language queries to find relevant memories:

**python**

```
Find memories about food preferences
(This can be done after putting memories into the store)
```



```

memories = store.search(
 namespace_for_memory,
 query="What does the user like to eat?",
 limit=3 # Return top 3 matches
)

```

js

```

// Find memories about food preferences
// (This can be done after putting memories into the store)
const memories = await store.search(namespaceForMemory, {
 query: "What does the user like to eat?",
 limit: 3, // Return top 3 matches
});

```

You can control which parts of your memories get embedded by configuring the `fields` parameter or by specifying the `index` parameter when storing memories:

python

```

Store with specific fields to embed
store.put(
 namespace_for_memory,
 str(uuid.uuid4()),
 {
 "food_preference": "I love Italian cuisine",
 "context": "Discussing dinner plans"
 },
 index=["food_preference"] # Only embed "food_preferences" field
)

Store without embedding (still retrievable, but not searchable)
store.put(
 namespace_for_memory,
 str(uuid.uuid4()),
 {"system_info": "Last updated: 2024-01-01"},
 index=False
)

```

js

```

// Store with specific fields to embed
await store.put(
 namespaceForMemory,
 crypto.randomUUID(),
 {

```

```

 food_preference: "I love Italian cuisine",
 context: "Discussing dinner plans",
 },
 { index: ["food_preference"] } // Only embed "food_preferences" field
);

// Store without embedding (still retrievable, but not searchable)
await store.put(
 namespaceForMemory,
 crypto.randomUUID(),
 { system_info: "Last updated: 2024-01-01" },
 { index: false }
);

```

## Using in LangGraph

### python

The store works hand-in-hand with the checkpointer: the checkpointer saves state to threads, as discussed above, and the store allows you to store arbitrary information for access *across* threads. Compile the graph with both the checkpointer and the store as follows.

```

from dataclasses import dataclass
from langgraph.checkpoint.memory import InMemorySaver

@dataclass
class Context:
 user_id: str

We need this because we want to enable threads (conversations)
checkpointer = InMemorySaver()

... Define the graph ...

Compile the graph with the checkpointer and store
builder = StateGraph(MessagesState, context_schema=Context)
... add nodes and edges ...
graph = builder.compile(checkpointer=checkpointer, store=store)

```

### js

The `memoryStore` works hand-in-hand with the checkpointer: the checkpointer saves state to threads, as discussed above, and the `memoryStore` allows you to store arbitrary information for access *across* threads. Compile the graph with both the checkpointer and the `memoryStore` as follows.

```
import { MemorySaver } from "@langchain/langgraph";

// We need this because we want to enable threads (conversations)
const checkpointer = new MemorySaver();

// ... Define the graph ...

// Compile the graph with the checkpointer and store
const graph = workflow.compile({ checkpointer, store: memoryStore });
```

Then invoke the graph with a `thread_id`, as before, and also with a `user_id`, which serves as the namespace for memories for this particular user as before.

### python

```
Invoke the graph
config = {"configurable": {"thread_id": "1"}}

First let's just say hi to the AI
for update in graph.stream(
 {"messages": [{"role": "user", "content": "hi"}]},
 config,
 stream_mode="updates",
 context=Context(user_id="1"),
):
 print(update)
```

### js

```
// Invoke the graph
const userId = "1";
const config = { configurable: { thread_id: "1" }, context: { userId } };

// First let's just say hi to the AI
for await (const update of await graph.stream(
 { messages: [{ role: "user", content: "hi" }] },
 { ...config, streamMode: "updates" }
)) {
 console.log(update);
}
```

### python

You can access the store and the `user_id` from *any node* by using the `Runtime` object. The `Runtime` is automatically injected by LangGraph when you add it as a parameter to your node

function. You can use it to save memories:

```
from langgraph.runtime import Runtime
from dataclasses import dataclass

@dataclass
class Context:
 user_id: str

async def update_memory(state: MessagesState, runtime: Runtime[Context]):

 # Get the user id from the runtime context
 user_id = runtime.context.user_id

 # Namespace the memory
 namespace = (user_id, "memories")

 # ... Analyze conversation and create a new memory

 # Create a new memory ID
 memory_id = str(uuid.uuid4())

 # We create a new memory
 await runtime.store.aput(namespace, memory_id, {"memory": memory})
```

js

You can access the store and the `userId` from *any node* with the `runtime` argument. You can use it to save memories:

```
import { StateSchema, MessagesValue, Runtime } from "@langchain/langgraph";

const MessagesState = new StateSchema({
 messages: MessagesValue,
});

const updateMemory: GraphNode<typeof MessagesState> = async (state, runtime) => {
 // Get the user id from the config
 const userId = runtime.context?.user_id;
 if (!userId) throw new Error("User ID is required");

 // Namespace the memory
 const namespace = [userId, "memories"];

 // ... Analyze conversation and create a new memory
 const memory = "Some memory content";
```

```
// Create a new memory ID
const memoryId = crypto.randomUUID();

// We create a new memory
await runtime.store?.put(namespace, memoryId, { memory });
};
```

You can also access the store from any node and use the `store.search` method to get memories. Memories are returned as a list of objects that can be converted to a dictionary.

### python

```
memories[-1].dict()
{'value': {'food_preference': 'I like pizza'},
 'key': '07e0caf4-1631-47b7-b15f-65515d4c1843',
 'namespace': ['1', 'memories'],
 'created_at': '2024-10-02T17:22:31.590602+00:00',
 'updated_at': '2024-10-02T17:22:31.590605+00:00'}
```

### js

```
memories[memories.length - 1];
// {
// value: { food_preference: 'I like pizza' },
// key: '07e0caf4-1631-47b7-b15f-65515d4c1843',
// namespace: ['1', 'memories'],
// createdAt: '2024-10-02T17:22:31.590602+00:00',
// updatedAt: '2024-10-02T17:22:31.590605+00:00'
// }
```

You access the memories and use them in model calls.

### python

```
from dataclasses import dataclass
from langgraph.runtime import Runtime

@dataclass
class Context:
 user_id: str

async def call_model(state: MessagesState, runtime: Runtime[Context]):
 # Get the user id from the runtime context
 user_id = runtime.context.user_id

 # Namespace the memory
```

```

namespace = (user_id, "memories")

Search based on the most recent message
memories = await runtime.store.asearch(
 namespace,
 query=state["messages"][-1].content,
 limit=3
)
info = "\n".join([d.value["memory"] for d in memories])

... Use memories in the model call

```

## js

```

const callModel: GraphNode<typeof MessagesState> = async (state, runtime) => {
 // Get the user id from the config
 const userId = runtime.context?.user_id;

 // Namespace the memory
 const namespace = [userId, "memories"];

 // Search based on the most recent message
 const memories = await runtime.store?.search(namespace, {
 query: state.messages[state.messages.length - 1].content,
 limit: 3,
 });
 const info = memories.map((d) => d.value.memory).join("\n");

 // ... Use memories in the model call
};

```

If you create a new thread, you can still access the same memories so long as the `user_id` is the same.

## python

```

Invoke the graph on a new thread
config = {"configurable": {"thread_id": "2"}}

Let's say hi again
for update in graph.stream(
 {"messages": [{"role": "user", "content": "hi, tell me about my memories"}]},
 config,
 stream_mode="updates",
 context=Context(user_id="1"),

```

```
):
 print(update)
```

js

```
// Invoke the graph
const config = { configurable: { thread_id: "2" }, context: { userId: "1" } };

// Let's say hi again
for await (const update of await graph.stream(
 { messages: [{ role: "user", content: "hi, tell me about my memories" }] },
 { ...config, streamMode: "updates" }
)) {
 console.log(update);
}
```

When you use LangSmith locally (e.g., in [Studio](#)) or [hosted](#), the base store is available to use by default and you do not need to specify it during graph compilation. To enable semantic search, however, you **do** need to configure the indexing settings in your `langgraph.json` file. For example:

```
{
 ...
 "store": {
 "index": {
 "embed": "openai:text-embeddings-3-small",
 "dims": 1536,
 "fields": ["$"]
 }
 }
}
```

See the [deployment guide](#) for more details and configuration options.

python

## Build a custom store

To use a storage backend other than the built-in implementations, subclass `@[BaseStore]` and implement its required methods. The built-in `@[InMemoryStore]` is the simplest reference implementation.

### Base contract

All five async methods are required. Sync counterparts (`put`, `get`, `delete`, `search`, `list_namespaces`) are optional but recommended for compatibility with sync graph execution.

Method	Description
<code>aput(namespace, key, value, index=None)</code>	Store or overwrite a single item
<code>aget(namespace, key)</code>	Retrieve a single item by key; return <code>None</code> if missing
<code>adelete(namespace, key)</code>	Delete a single item
<code>asearch(namespace_prefix, *, query=None, filter=None, limit=10, offset=0)</code>	Search items under a namespace prefix; optionally by semantic query
<code>alist_namespaces(*, prefix=None, suffix=None, max_depth=None, limit=100, offset=0)</code>	List namespaces matching a prefix/suffix pattern

Look up exact signatures before implementing:

```
import inspect
from langgraph.store.base import BaseStore
print(inspect.getsource(BaseStore))
```

## Namespace design

Namespaces are tuples of strings, e.g. `("user_id", "memories")`. Store implementations must support:

- **Prefix matching:** `asearch(("alice",))` returns items under `("alice",)`, `("alice", "memories")`, and any other sub-namespace.
- **Exact key lookup:** `aget(("alice", "memories"), "some-key")` must be  $O(1)$  or close to it.

For SQL backends, a common schema:

```
CREATE TABLE store_items (
 namespace TEXT[] NOT NULL,
 key TEXT NOT NULL,
 value JSONB NOT NULL,
 created_at TIMESTAMPTZ DEFAULT now(),
 updated_at TIMESTAMPTZ DEFAULT now(),
 PRIMARY KEY (namespace, key)
);

CREATE INDEX ON store_items USING gin(namespace);
```

## Serialization

Store values are plain Python dicts — no special serializer is required. Serialize with `json.dumps` / `json.loads` or a JSONB column directly. Do not store raw Python objects that are not JSON-serializable.



## Semantic search support

If your backend supports vector search, implement the `query` parameter on `asearch`:

- Accept a `query: str | None` argument.
- When `query` is not `None`, embed it and rank results by cosine similarity.
- Results should include a `score` field on each `Item` when `query` is provided.

If your backend does not support vector search, raise `NotImplementedError` when `query` is passed.

## Testing

There is currently no conformance suite for custom stores. Test against `@[InMemoryStore]` as the reference:

```
import pytest
from langgraph.store.memory import InMemoryStore
from your_module import YourStore

@pytest.fixture
async def store():
 async with YourStore.create() as s:
 yield s

@pytest.fixture
def reference():
 return InMemoryStore()

async def test_put_and_get(store, reference):
 ns = ("test", "ns")
 for s in [store, reference]:
 await s.aput(ns, "k1", {"val": 1})
 item = await s.aget(ns, "k1")
 assert item is not None
 assert item.value == {"val": 1}

async def test_delete(store, reference):
 ns = ("test", "ns")
 for s in [store, reference]:
 await s.aput(ns, "k1", {"val": 1})
 await s.delete(ns, "k1")
 assert await s.aget(ns, "k1") is None

async def test_search_prefix(store, reference):
 for s in [store, reference]:
 await s.aput(("user", "memories"), "m1", {"text": "likes pizza"})
```

```
results = await s.asearch(("user",))
assert any(r.key == "m1" for r in results)
```

## Next steps

- [Add a custom store to Agent Server](#) — deploying your implementation
- [Checkpointers](#) — thread-scoped state persistence

# Streaming

langgraph/streaming.mdx

## 팁

For new applications, we recommend [event streaming](#)—the typed-projection API introduced in LangGraph v1.2. Event streaming gives you separate iterators per projection (messages, values, subgraphs, output) so you can consume them independently instead of branching on `stream_mode` chunks.

This page covers LangGraph's stream-mode API. It exposes graph execution through stream modes such as `updates`, `values`, `messages`, `custom`, `checkpoints`, `tasks`, and `debug`. Use it when you need direct access to graph-runtime events or specific stream-mode output.

## Get started

### Basic usage

#### python

LangGraph graphs expose the `@[stream][Pregel.stream]` (sync) and `@[astream][Pregel.astream]` (async) methods to yield streamed outputs as iterators. Pass one or more [stream modes](#) to control what data you receive.

```
for chunk in graph.stream(
 {"topic": "ice cream"},
 stream_mode=["updates", "custom"], # [!code highlight]
 version="v2", # [!code highlight]
):
 if chunk["type"] == "updates":
 for node_name, state in chunk["data"].items():
 print(f"Node {node_name} updated: {state}")
 elif chunk["type"] == "custom":
 print(f"Status: {chunk['data']['status']}")
```

```
```shell title="Output" Status: thinking of a joke... Node generate_joke updated: {'joke': 'Why did the ice cream go to school? To get a sundae education!'}
```

```
**Full example**
```

```
```python
from typing import TypedDict
```

```

from langgraph.graph import StateGraph, START, END
from langgraph.config import get_stream_writer

class State(TypedDict):
 topic: str
 joke: str

def generate_joke(state: State):
 writer = get_stream_writer()
 writer({"status": "thinking of a joke..."})
 return {"joke": f"Why did the {state['topic']} go to school? To get a sundae education!"}

graph = (
 StateGraph(State)
 .add_node(generate_joke)
 .add_edge(START, "generate_joke")
 .add_edge("generate_joke", END)
 .compile()
)

for chunk in graph.stream(
 {"topic": "ice cream"},
 stream_mode=["updates", "custom"],
 version="v2",
):
 if chunk["type"] == "updates":
 for node_name, state in chunk["data"].items():
 print(f"Node {node_name} updated: {state}")
 elif chunk["type"] == "custom":
 print(f"Status: {chunk['data']['status']}")

```

```

```shell title="Output" Status: thinking of a joke... Node generate_joke updated: {'joke': 'Why did the ice cream go to school? To get a sundae education!'}

```

</div>

<div class="callout js" markdown="1">

<p class="callout-label">js</p>

LangGraph graphs expose the `@[`stream`][Pregel.stream]` method to yield streamed outputs as iterators.

```

```typescript

```

```

for await (const chunk of await graph.stream(inputs, {
 streamMode: "updates",
})) {

```

```
console.log(chunk);
}
```

## 팁

Debug streaming events, inspect token-by-token LLM output, and monitor latency with [LangSmith](#). Follow the [tracing quickstart](#) to get set up.

## python

### Stream output format (v2)

#### 참고

Requires LangGraph 1.1 or later. All examples on this page use `version="v2"`.

Pass `version="v2"` to `stream()` or `astream()` to get a unified output format. Every chunk is a `StreamPart` dict with a consistent shape — regardless of stream mode, number of modes, or subgraph settings:

```
{
 "type": "values" | "updates" | "messages" | "custom" | "checkpoints" | "tasks" | "debug",
 "ns": (), # namespace tuple, populated for subgraph events
 "data": ..., # the actual payload (type varies by stream mode)
}
```

Each stream mode has a corresponding `TypedDict` containing `@[ValuesStreamPart]`, `@[UpdatesStreamPart]`, `@[MessagesStreamPart]`, `@[CustomStreamPart]`, `@[CheckpointStreamPart]`, `@[TasksStreamPart]`, `@[DebugStreamPart]`. You can import these types from `langgraph.types`. The union type `@[StreamPart]` is a disjoining union on `part["type"]`, enabling full type narrowing in editors and type checkers.

With v1 (default), the output format changes based on your streaming options (single mode returns raw data, multiple modes return `(mode, data)` tuples, subgraphs return `(namespace, data)` tuples). With v2, the format is always the same:

```
```python v2 (new) for chunk in graph.stream(inputs, stream_mode="updates", version="v2"):
    print(chunk["type"]) # "updates" print(chunk["ns"]) # () print(chunk["data"]) # {"node_name":
    {"key": "value"}}
```

```
```python v1 (current default)
for chunk in graph.stream(inputs, stream_mode="updates"):
 print(chunk) # {"node_name": {"key": "value"}}
```

The v2 format also enables type narrowing, which means you can filter chunks by `chunk["type"]` and get the correct payload type. Each branch narrows `part["data"]` to the specific type for that mode:

```

for part in graph.stream(
 {"topic": "ice cream"},
 stream_mode=["values", "updates", "messages", "custom"],
 version="v2",
):
 if part["type"] == "values":
 # ValuesStreamPart – full state snapshot after each step
 print(f"State: topic={part['data']['topic']}")
 elif part["type"] == "updates":
 # UpdatesStreamPart – only the changed keys from each node
 for node_name, state in part["data"].items():
 print(f"Node `{node_name}` updated: {state}")
 elif part["type"] == "messages":
 # MessagesStreamPart – (message_chunk, metadata) from LLM calls
 msg, metadata = part["data"]
 print(msg.content, end="", flush=True)
 elif part["type"] == "custom":
 # CustomStreamPart – arbitrary data from get_stream_writer()
 print(f"Progress: {part['data']['progress']}%")

```

## Stream modes

### python

Pass one or more of the following stream modes as a list to the `@[ stream ]` `[CompiledStateGraph.stream]` or `@[ astream ]``[CompiledStateGraph.astream]` methods:

Mode	Type	Description
<a href="#">values</a>	<code>@[ ValuesStreamPart ]</code>	Full state after each step.
<a href="#">updates</a>	<code>@[ UpdatesStreamPart ]</code>	State updates after each step. Multiple updates in the same step are streamed separately.
<a href="#">messages</a>	<code>@[ MessagesStreamPart ]</code>	2-tuples of (LLM token, metadata) from LLM calls.
<a href="#">custom</a>	<code>@[ CustomStreamPart ]</code>	Custom data emitted from nodes via <code>@[ get_stream_writer ]</code> .
<a href="#">checkpoints</a>	<code>@[ CheckpointStreamPart ]</code>	Checkpoint events (same format as <code>get_state()</code> ). Requires a checkpointer.

Mode	Type	Description
<a href="#">tasks</a>	@[ TasksStreamPart ]	Task start/finish events with results and errors. Requires a checkpointer.
<a href="#">debug</a>	@[ DebugStreamPart ]	All available info — combines checkpoints and tasks with extra metadata.

## js

Pass one or more of the following stream modes as a list to the @[ stream ] [CompiledStateGraph.stream] method:

Mode	Description
<a href="#">values</a>	Full state after each step.
<a href="#">updates</a>	State updates after each step. Multiple updates in the same step are streamed separately.
<a href="#">messages</a>	2-tuples of (LLM token, metadata) from LLM calls.
<a href="#">custom</a>	Custom data emitted from nodes via the writer config parameter.
<a href="#">tools</a>	Tool-call lifecycle events ( on_tool_start , on_tool_event , on_tool_end , on_tool_error ).
<a href="#">debug</a>	All available info throughout graph execution.

## Graph state

Use the stream modes updates and values to stream the state of the graph as it executes.

- updates streams the **updates** to the state after each step of the graph.
- values streams the **full value** of the state after each step of the graph.

## python

```
from typing import TypedDict
from langgraph.graph import StateGraph, START, END

class State(TypedDict):
 topic: str
 joke: str

def refine_topic(state: State):
 return {"topic": state["topic"] + " and cats"}

def generate_joke(state: State):
 return {"joke": f"This is a joke about {state['topic']}"}

graph = (
 StateGraph(State)
 .add_node(refine_topic)
```

```

 .add_node(generate_joke)
 .add_edge(START, "refine_topic")
 .add_edge("refine_topic", "generate_joke")
 .add_edge("generate_joke", END)
 .compile()
)

```

js

```

import { StateGraph, StateSchema, START, END } from "@langchain/langgraph";
import { z } from "zod/v4";

const State = new StateSchema({
 topic: z.string(),
 joke: z.string(),
});

const graph = new StateGraph(State)
 .addNode("refineTopic", (state) => {
 return { topic: state.topic + " and cats" };
 })
 .addNode("generateJoke", (state) => {
 return { joke: `This is a joke about ${state.topic}` };
 })
 .addEdge(START, "refineTopic")
 .addEdge("refineTopic", "generateJoke")
 .addEdge("generateJoke", END)
 .compile();

```

## updates

Use this to stream only the **state updates** returned by the nodes after each step. The streamed outputs include the name of the node as well as the update.

python

```

for chunk in graph.stream(
 {"topic": "ice cream"},
 stream_mode="updates", # [!code highlight]
 version="v2", # [!code highlight]
):
 if chunk["type"] == "updates":
 for node_name, state in chunk["data"].items():
 print(f"Node `{node_name}` updated: {state}")

```

```

``shell title="Output" Node refine_topic updated: {'topic': 'ice cream and cats'}
Node generate_joke` updated: {'joke': 'This is a joke about ice cream and cats'}

```



```

</div>

<div class="callout js" markdown="1">

<p class="callout-label">js</p>

```typescript
for await (const chunk of await graph.stream(
  { topic: "ice cream" },
  { streamMode: "updates" }
)) {
  for (const [nodeName, state] of Object.entries(chunk)) {
    console.log(`Node ${nodeName} updated:`, state);
  }
}

```

values

Use this to stream the **full state** of the graph after each step.

python

```

for chunk in graph.stream(
    {"topic": "ice cream"},
    stream_mode="values", # [!code highlight]
    version="v2", # [!code highlight]
):
    if chunk["type"] == "values":
        print(f"topic: {chunk['data']['topic']}, joke: {chunk['data']['joke']}")

```

```

```shell title="Output" topic: ice cream, joke: topic: ice cream and cats, joke: topic: ice cream
and cats, joke: This is a joke about ice cream and cats

```

```

</div>

<div class="callout js" markdown="1">

<p class="callout-label">js</p>

```typescript
for await (const chunk of await graph.stream(
  { topic: "ice cream" },
  { streamMode: "values" }
)) {

```

```
console.log(`topic: ${chunk.topic}, joke: ${chunk.joke}`);
}
```

LLM tokens

Use the `messages` streaming mode to stream Large Language Model (LLM) outputs **token by token** from any part of your graph, including nodes, tools, subgraphs, or tasks.

python

The streamed output from [messages mode](#) is a tuple `(message_chunk, metadata)` where:

- `message_chunk`: the token or message segment from the LLM.
- `metadata`: a dictionary containing details about the graph node and LLM invocation.

If your LLM is not available as a LangChain integration, you can stream its outputs using `custom` mode instead. See [use with any LLM](#) for details.

주의

Manual config required for async in Python < 3.11 When using Python < 3.11 with async code, you must explicitly pass `@[RunnableConfig]` to `ainvoke()` to enable proper streaming. See [Async with Python < 3.11](#) for details or upgrade to Python 3.11+.

```
from dataclasses import dataclass

from langchain.chat_models import init_chat_model
from langgraph.graph import StateGraph, START

@dataclass
class MyState:
    topic: str
    joke: str = ""

model = init_chat_model(model="gpt-5.4-mini")

def call_model(state: MyState):
    """Call the LLM to generate a joke about a topic"""
    # Note that message events are emitted even when the LLM is run using .invoke rather than .stream
    model_response = model.invoke( # [!code highlight]
        [
            {"role": "user", "content": f"Generate a joke about {state.topic}"}
        ]
    )
```

```

        return {"joke": model_response.content}

graph = (
    StateGraph(MyState)
        .add_node(call_model)
        .add_edge(START, "call_model")
        .compile()
)

# The "messages" stream mode streams LLM tokens with metadata
# Use version="v2" for a unified StreamPart format
for chunk in graph.stream(
    {"topic": "ice cream"},
    stream_mode="messages", # [!code highlight]
    version="v2", # [!code highlight]
):
    if chunk["type"] == "messages":
        message_chunk, metadata = chunk["data"]
        if message_chunk.content:
            print(message_chunk.content, end="|", flush=True)

```

js

The streamed output from [messages mode](#) is a tuple [message_chunk, metadata] where:

- `message_chunk`: the token or message segment from the LLM.
- `metadata`: a dictionary containing details about the graph node and LLM invocation.

If your LLM is not available as a LangChain integration, you can stream its outputs using `custom` mode instead. See [use with any LLM](#) for details.

```

import { ChatOpenAI } from "@langchain/openai";
import { StateGraph, StateSchema, GraphNode, START } from "@langchain/langgraph";
import * as z from "zod";

const MyState = new StateSchema({
  topic: z.string(),
  joke: z.string().default(""),
});

const model = new ChatOpenAI({ model: "gpt-5.4-mini" });

const callModel: GraphNode<typeof MyState> = async (state) => {
  // Call the LLM to generate a joke about a topic
  // Note that message events are emitted even when the LLM is run using .invoke rather than
  .stream
  const modelResponse = await model.invoke([

```

```

    { role: "user", content: `Generate a joke about ${state.topic}` },
  ]});
  return { joke: modelResponse.content };
};

const graph = new StateGraph(MyState)
  .addNode("callModel", callModel)
  .addEdge(START, "callModel")
  .compile();

// The "messages" stream mode returns an iterator of tuples [messageChunk, metadata]
// where messageChunk is the token streamed by the LLM and metadata is a dictionary
// with information about the graph node where the LLM was called and other information
for await (const [messageChunk, metadata] of await graph.stream(
  { topic: "ice cream" },
  { streamMode: "messages" }
)) {
  if (messageChunk.content) {
    console.log(messageChunk.content + "|");
  }
}

```

Filter by LLM invocation

You can associate `tags` with LLM invocations to filter the streamed tokens by LLM invocation.

python

```

from langchain.chat_models import init_chat_model

# model_1 is tagged with "joke"
model_1 = init_chat_model(model="gpt-5.4-mini", tags=['joke'])
# model_2 is tagged with "poem"
model_2 = init_chat_model(model="gpt-5.4-mini", tags=['poem'])

graph = ... # define a graph that uses these LLMs

# The stream_mode is set to "messages" to stream LLM tokens
# The metadata contains information about the LLM invocation, including the tags
async for chunk in graph.astream(
    {"topic": "cats"},
    stream_mode="messages", # [!code highlight]
    version="v2", # [!code highlight]
):
    if chunk["type"] == "messages":
        msg, metadata = chunk["data"]
        # Filter the streamed tokens by the tags field in the metadata to only include
        # the tokens from the LLM invocation with the "joke" tag

```

```
if metadata["tags"] == ["joke"]:
    print(msg.content, end="|", flush=True)
```

js

```
import { ChatOpenAI } from "@langchain/openai";

// model1 is tagged with "joke"
const model1 = new ChatOpenAI({
  model: "gpt-5.4-mini",
  tags: ['joke']
});

// model2 is tagged with "poem"
const model2 = new ChatOpenAI({
  model: "gpt-5.4-mini",
  tags: ['poem']
});

const graph = // ... define a graph that uses these LLMs

// The streamMode is set to "messages" to stream LLM tokens
// The metadata contains information about the LLM invocation, including the tags
for await (const [msg, metadata] of await graph.stream(
  { topic: "cats" },
  { streamMode: "messages" }
)) {
  // Filter the streamed tokens by the tags field in the metadata to only include
  // the tokens from the LLM invocation with the "joke" tag
  if (metadata.tags?.includes("joke")) {
    console.log(msg.content + "|");
  }
}
```

Extended example: filtering by tags

python

```
from typing import TypedDict

from langchain.chat_models import init_chat_model
from langgraph.graph import START, StateGraph

# The joke_model is tagged with "joke"
joke_model = init_chat_model(model="gpt-5.4-mini", tags=["joke"])
# The poem_model is tagged with "poem"
poem_model = init_chat_model(model="gpt-5.4-mini", tags=["poem"])
```

```

class State(TypedDict):
    topic: str
    joke: str
    poem: str

async def call_model(state, config):
    topic = state["topic"]
    print("Writing joke...")
    # Note: Passing the config through explicitly is required for python < 3.11
    # Since context var support wasn't added before then: https://docs.python.org/3/library/asyncio-task.html#creating-tasks
    # The config is passed through explicitly to ensure the context vars are propagated correctly
    # This is required for Python < 3.11 when using async code. Please see the async section for more details
    joke_response = await joke_model.ainvoke(
        [{"role": "user", "content": f"Write a joke about {topic}"}],
        config,
    )
    print("\n\nWriting poem...")
    poem_response = await poem_model.ainvoke(
        [{"role": "user", "content": f"Write a short poem about {topic}"}],
        config,
    )
    return {"joke": joke_response.content, "poem": poem_response.content}

graph = (
    StateGraph(State)
    .add_node(call_model)
    .add_edge(START, "call_model")
    .compile()
)

# The stream_mode is set to "messages" to stream LLM tokens
# The metadata contains information about the LLM invocation, including the tags
async for chunk in graph.astream(
    {"topic": "cats"},
    stream_mode="messages",
    version="v2",
):
    if chunk["type"] == "messages":
        msg, metadata = chunk["data"]
        if metadata["tags"] == ["joke"]:
            print(msg.content, end="|", flush=True)

```

js

```

import { ChatOpenAI } from "@langchain/openai";
import { StateGraph, StateSchema, GraphNode, START } from "@langchain/langgraph";

```

```

import * as z from "zod";

// The jokeModel is tagged with "joke"
const jokeModel = new ChatOpenAI({
  model: "gpt-5.4-mini",
  tags: ["joke"]
});

// The poemModel is tagged with "poem"
const poemModel = new ChatOpenAI({
  model: "gpt-5.4-mini",
  tags: ["poem"]
});

const State = new StateSchema({
  topic: z.string(),
  joke: z.string(),
  poem: z.string(),
});

const callModel: GraphNode<typeof State> = async (state) => {
  const topic = state.topic;
  console.log("Writing joke...");

  const jokeResponse = await jokeModel.invoke([
    { role: "user", content: `Write a joke about ${topic}` }
  ]);

  console.log("\n\nWriting poem...");
  const poemResponse = await poemModel.invoke([
    { role: "user", content: `Write a short poem about ${topic}` }
  ]);

  return {
    joke: jokeResponse.content,
    poem: poemResponse.content
  };
};

const graph = new StateGraph(State)
  .addNode("callModel", callModel)
  .addEdge(START, "callModel")
  .compile();

// The streamMode is set to "messages" to stream LLM tokens
// The metadata contains information about the LLM invocation, including the tags
for await (const [msg, metadata] of await graph.stream(
  { topic: "cats" },
  { streamMode: "messages" }
)) {
  // Filter the streamed tokens by the tags field in the metadata to only include
  // the tokens from the LLM invocation with the "joke" tag

```

```

if (metadata.tags?.includes("joke")) {
  console.log(msg.content + "|");
}
}

```

Omit messages from the stream

Use the `nostream` tag to exclude LLM output from the stream entirely. Invocations tagged with `nostream` still run and produce output; their tokens are simply not emitted in `messages` mode.

This is useful when:

- You need LLM output for internal processing (for example structured output) but do not want to stream it to the client
- You stream the same content through a different channel (for example custom UI messages) and want to avoid duplicate output in the `messages` stream

python

js

Filter by node

To stream tokens only from specific nodes, use `stream_mode="messages"` and filter the outputs by the `langgraph_node` field in the streamed metadata:

python

```

# The "messages" stream mode streams LLM tokens with metadata
# Use version="v2" for a unified StreamPart format
for chunk in graph.stream(
    inputs,
    stream_mode="messages", # [!code highlight]
    version="v2", # [!code highlight]
):
    if chunk["type"] == "messages":
        msg, metadata = chunk["data"]
        # Filter the streamed tokens by the langgraph_node field in the metadata
        # to only include the tokens from the specified node
        if msg.content and metadata["langgraph_node"] == "some_node_name":
            ...

```

js


```
// The "messages" stream mode returns a tuple of [messageChunk, metadata]
// where messageChunk is the token streamed by the LLM and metadata is a dictionary
// with information about the graph node where the LLM was called and other information
for await (const [msg, metadata] of await graph.stream(
  inputs,
  { streamMode: "messages" }
)) {
  // Filter the streamed tokens by the langgraph_node field in the metadata
  // to only include the tokens from the specified node
  if (msg.content && metadata.langgraph_node === "some_node_name") {
    // ...
  }
}
```

Extended example: streaming LLM tokens from specific nodes

python

```
from typing import TypedDict
from langgraph.graph import START, StateGraph
from langchain_openai import ChatOpenAI

model = ChatOpenAI(model="gpt-5.4-mini")

class State(TypedDict):
    topic: str
    joke: str
    poem: str

def write_joke(state: State):
    topic = state["topic"]
    joke_response = model.invoke(
        [{"role": "user", "content": f"Write a joke about {topic}"}]
    )
    return {"joke": joke_response.content}

def write_poem(state: State):
    topic = state["topic"]
    poem_response = model.invoke(
        [{"role": "user", "content": f"Write a short poem about {topic}"}]
    )
    return {"poem": poem_response.content}

graph = (
    StateGraph(State)
    .add_node(write_joke)
    .add_node(write_poem)
    # write both the joke and the poem concurrently
```

```

        .add_edge(START, "write_joke")
        .add_edge(START, "write_poem")
        .compile()
    )

    # The "messages" stream mode streams LLM tokens with metadata
    # Use version="v2" for a unified StreamPart format
    for chunk in graph.stream(
        {"topic": "cats"},
        stream_mode="messages", # [!code highlight]
        version="v2", # [!code highlight]
    ):
        if chunk["type"] == "messages":
            msg, metadata = chunk["data"]
            # Filter the streamed tokens by the langgraph_node field in the metadata
            # to only include the tokens from the write_poem node
            if msg.content and metadata["langgraph_node"] == "write_poem":
                print(msg.content, end="|", flush=True)

```

js

```

import { ChatOpenAI } from "@langchain/openai";
import { StateGraph, StateSchema, GraphNode, START } from "@langchain/langgraph";
import * as z from "zod";

const model = new ChatOpenAI({ model: "gpt-5.4-mini" });

const State = new StateSchema({
    topic: z.string(),
    joke: z.string(),
    poem: z.string(),
});

const writeJoke: GraphNode<typeof State> = async (state) => {
    const topic = state.topic;
    const jokeResponse = await model.invoke([
        { role: "user", content: `Write a joke about ${topic}` }
    ]);
    return { joke: jokeResponse.content };
};

const writePoem: GraphNode<typeof State> = async (state) => {
    const topic = state.topic;
    const poemResponse = await model.invoke([
        { role: "user", content: `Write a short poem about ${topic}` }
    ]);
    return { poem: poemResponse.content };
};

```

```

const graph = new StateGraph(State)
  .addNode("writeJoke", writeJoke)
  .addNode("writePoem", writePoem)
  // write both the joke and the poem concurrently
  .addEdge(START, "writeJoke")
  .addEdge(START, "writePoem")
  .compile();

// The "messages" stream mode returns a tuple of [messageChunk, metadata]
// where messageChunk is the token streamed by the LLM and metadata is a dictionary
// with information about the graph node where the LLM was called and other information
for await (const [msg, metadata] of await graph.stream(
  { topic: "cats" },
  { streamMode: "messages" }
)) {
  // Filter the streamed tokens by the langgraph_node field in the metadata
  // to only include the tokens from the writePoem node
  if (msg.content && metadata.langgraph_node === "writePoem") {
    console.log(msg.content + "|");
  }
}

```

Custom data

python

To send **custom user-defined data** from inside a LangGraph node or tool, follow these steps:

1. Use `@[get_stream_writer]` to access the stream writer and emit custom data.
2. Set `stream_mode="custom"` when calling `.stream()` or `.astream()` to get the custom data in the stream. You can combine multiple modes (e.g., `["updates", "custom"]`), but at least one must be `"custom"`.

주의

No `@[get_stream_writer]` in async for Python < 3.11 In async code running on Python < 3.11, `@[get_stream_writer]` will not work. Instead, add a `writer` parameter to your node or tool and pass it manually. See [Async with Python < 3.11](#) for usage examples.

node

```

from typing import TypedDict
from langgraph.config import get_stream_writer
from langgraph.graph import StateGraph, START

class State(TypedDict):

```

```

    query: str
    answer: str

def node(state: State):
    # Get the stream writer to send custom data
    writer = get_stream_writer()
    # Emit a custom key-value pair (e.g., progress update)
    writer({"custom_key": "Generating custom data inside node"})
    return {"answer": "some data"}

graph = (
    StateGraph(State)
    .add_node(node)
    .add_edge(START, "node")
    .compile()
)

inputs = {"query": "example"}

# Set stream_mode="custom" to receive the custom data in the stream
for chunk in graph.stream(inputs, stream_mode="custom", version="v2"):
    if chunk["type"] == "custom":
        print(f"Custom event: {chunk['data']['custom_key']}")

```

tool

```

from langchain.tools import tool
from langgraph.config import get_stream_writer

@tool
def query_database(query: str) -> str:
    """Query the database."""
    # Access the stream writer to send custom data
    writer = get_stream_writer() # [!code highlight]
    # Emit a custom key-value pair (e.g., progress update)
    writer({"data": "Retrieved 0/100 records", "type": "progress"}) # [!code highlight]
    # perform query
    # Emit another custom key-value pair
    writer({"data": "Retrieved 100/100 records", "type": "progress"})
    return "some-answer"

graph = ... # define a graph that uses this tool

# Set stream_mode="custom" to receive the custom data in the stream
for chunk in graph.stream(inputs, stream_mode="custom", version="v2"):
    if chunk["type"] == "custom":
        print(f"{chunk['data']['type']}: {chunk['data']['data']}")

```

To send **custom user-defined data** from inside a LangGraph node or tool, follow these steps:

1. Use the `writer` parameter from the `LangGraphRunnableConfig` to emit custom data.
 2. Set `streamMode: "custom"` when calling `.stream()` to get the custom data in the stream.
- You can combine multiple modes (e.g., `["updates", "custom"]`), but at least one must be `"custom"`.

node

```
import { StateGraph, StateSchema, GraphNode, START, LangGraphRunnableConfig } from "@langchain/langgraph";
import * as z from "zod";

const State = new StateSchema({
  query: z.string(),
  answer: z.string(),
});

const node: GraphNode<typeof State> = async (state, config) => {
  // Use the writer to emit a custom key-value pair (e.g., progress update)
  config.writer({ custom_key: "Generating custom data inside node" });
  return { answer: "some data" };
};

const graph = new StateGraph(State)
  .addNode("node", node)
  .addEdge(START, "node")
  .compile();

const inputs = { query: "example" };

// Set streamMode: "custom" to receive the custom data in the stream
for await (const chunk of await graph.stream(inputs, { streamMode: "custom" })) {
  console.log(chunk);
}
```

tool

```
import { tool } from "@langchain/core/tools";
import { LangGraphRunnableConfig } from "@langchain/langgraph";
import * as z from "zod";

const queryDatabase = tool(
  async (input, config: LangGraphRunnableConfig) => {
    // Use the writer to emit a custom key-value pair (e.g., progress update)
    config.writer({ data: "Retrieved 0/100 records", type: "progress" });
    // perform query
    // Emit another custom key-value pair
    config.writer({ data: "Retrieved 100/100 records", type: "progress" });
  }
);
```

```

    return "some-answer";
  },
  {
    name: "query_database",
    description: "Query the database.",
    schema: z.object({
      query: z.string().describe("The query to execute."),
    }),
  }
];

const graph = // ... define a graph that uses this tool

// Set streamMode: "custom" to receive the custom data in the stream
for await (const chunk of await graph.stream(inputs, { streamMode: "custom" })) {
  console.log(chunk);
}

```

js

Tool progress

Use the `tools` stream mode to receive real-time lifecycle events for tool executions. This is useful for showing progress indicators, partial results, and error states in your UI while tools are running.

The `tools` stream mode emits four event types:

Event	When	Payload
<code>on_tool_start</code>	Tool invocation begins	<code>name</code> , <code>input</code> , <code>toolCallId</code>
<code>on_tool_event</code>	Tool yields intermediate data	<code>name</code> , <code>data</code> , <code>toolCallId</code>
<code>on_tool_end</code>	Tool returns its final result	<code>name</code> , <code>output</code> , <code>toolCallId</code>
<code>on_tool_error</code>	Tool throws an error	<code>name</code> , <code>error</code> , <code>toolCallId</code>

Define tools that stream progress

To emit `on_tool_event` events, define your tool function as an **async generator** (`async function*`). Each `yield` sends intermediate data to the stream, and the `return` value is used as the tool's final result.

```

import { tool } from "@langchain/core/tools";
import { z } from "zod/v4";

const searchFlights = tool(
  async function* (input) {

```

```

const airlines = ["United", "Delta", "American", "JetBlue"];
const completed: string[] = [];

for (let i = 0; i < airlines.length; i++) {
  await new Promise((r) => setTimeout(r, 500));
  completed.push(airlines[i]);

  // Each yield emits an on_tool_event to the stream
  yield {
    message: `Searching ${airlines[i]}...`,
    progress: (i + 1) / airlines.length,
    completed,
  };
}

// The return value becomes the tool result (ToolMessage.content)
return JSON.stringify({
  flights: [
    { airline: "United", price: 450, duration: "5h 30m" },
    { airline: "Delta", price: 520, duration: "5h 15m" },
  ],
});
},
{
  name: "search_flights",
  description: "Search for available flights to a destination.",
  schema: z.object({
    destination: z.string(),
    date: z.string(),
  }),
}
);

```

참고

Existing tools that return a `Promise` are fully compatible. They emit `on_tool_start` and `on_tool_end` events but no `on_tool_event` events.

Consume tool events server-side

Pass `streamMode: ["tools"]` (or combine with other modes) to `graph.stream()`:

```

for await (const [mode, chunk] of await graph.stream(
  { messages: [{ role: "user", content: "Find flights to Tokyo" } ] },
  { streamMode: ["updates", "tools"] }) {
  if (mode === "tools") {
    switch (chunk.event) {

```

```

    case "on_tool_start":
      console.log(`Tool started: ${chunk.name}`, chunk.input);
      break;
    case "on_tool_event":
      console.log(`Tool progress: ${chunk.name}`, chunk.data);
      break;
    case "on_tool_end":
      console.log(`Tool finished: ${chunk.name}`, chunk.output);
      break;
    case "on_tool_error":
      console.error(`Tool failed: ${chunk.name}`, chunk.error);
      break;
  }
}
}

```

Use tool progress in React with `useStream`

The `@langchain/langgraph-sdk/react` exposes a `toolProgress` array when you include `"tools"` in your stream modes. Each entry is a `ToolProgress` object that tracks the current state of a running tool:

Field	Description
<code>name</code>	The tool name
<code>state</code>	Current lifecycle state: <code>"starting"</code> , <code>"running"</code> , <code>"completed"</code> , or <code>"error"</code>
<code>toolCallId</code>	The tool call ID from the LLM
<code>input</code>	The tool's input arguments
<code>data</code>	The most recent yielded data from <code>on_tool_event</code>
<code>result</code>	The final result, set on <code>on_tool_end</code>
<code>error</code>	The error, set on <code>on_tool_error</code>

```

import { useStream } from "@langchain/langgraph-sdk/react";

function Chat() {
  const stream = useStream({
    assistantId: "my-agent",
    streamMode: ["values", "tools"],
  });

  // Filter for actively running tools
  const activeTools = stream.toolProgress.filter(
    (t) => t.state === "starting" || t.state === "running"
  );

  return (

```



```

<div>
  {stream.messages.map((msg) => (
    <MessageBubble key={msg.id} message={msg} />
  ))}

  {/* Show progress cards for running tools */}
  {activeTools.map((tool) => (
    <ToolProgressCard
      key={tool.toolCallId ?? tool.name}
      name={tool.name}
      state={tool.state}
      data={tool.data}
    />
  ))}
</div>
);
}

```

Extended example: travel planning agent with tool progress

This example shows a complete agent with async-generator tools that stream search progress to a React UI.

Agent definition:

```

import { tool } from "@langchain/core/tools";
import { ChatOpenAI } from "@langchain/openai";
import { createAgent } from "@langchain/langgraph";
import { MemorySaver } from "@langchain/langgraph-checkpoint-memory";
import { z } from "zod/v4";

const searchFlights = tool(
  async function* (input) {
    const airlines = ["United", "Delta", "American", "JetBlue"];
    const completed: string[] = [];

    for (let i = 0; i < airlines.length; i++) {
      await new Promise((r) => setTimeout(r, 600));
      completed.push(`${airlines[i]}: checked`);
      yield {
        message: `Searching ${airlines[i]}...`,
        progress: (i + 1) / airlines.length,
        completed,
      };
    }
  }

  return JSON.stringify({
    flights: [
      { airline: "United", price: 450, duration: "5h 30m" },
      { airline: "Delta", price: 520, duration: "5h 15m" },
    ],
  })

```

```

    ],
  });
},
{
  name: "search_flights",
  description: "Search for available flights.",
  schema: z.object({
    destination: z.string(),
    departure_date: z.string(),
  }),
}
);

const checkHotels = tool(
  async function* (input) {
    const hotels = ["Grand Hyatt", "Marriott", "Hilton"];
    const completed: string[] = [];

    for (let i = 0; i < hotels.length; i++) {
      await new Promise((r) => setTimeout(r, 400));
      completed.push(`${hotels[i]}: available`);
      yield {
        message: `Checking ${hotels[i]}...`,
        progress: (i + 1) / hotels.length,
        completed,
      };
    }

    return JSON.stringify({
      hotels: [
        { name: "Grand Hyatt", price: 250, rating: 4.5 },
        { name: "Marriott", price: 180, rating: 4.2 },
      ],
    });
  },
{
  name: "check_hotels",
  description: "Check hotel availability.",
  schema: z.object({
    city: z.string(),
    check_in: z.string(),
    nights: z.number(),
  }),
}
);

export const agent = createAgent({
  model: new ChatOpenAI({ model: "gpt-5.4-mini" }),
  tools: [searchFlights, checkHotels],
});

```

```
    checkpointer: new MemorySaver(),
  });
```

React component with progress cards:

```
import { useStream } from "@langchain/langgraph-sdk/react";

function TravelPlanner() {
  const stream = useStream<typeof agent>({
    assistantId: "travel-agent",
    streamMode: ["values", "tools"],
  });

  const activeTools = stream.toolProgress.filter(
    (t) => t.state === "starting" || t.state === "running"
  );

  return (
    <div>
      {stream.messages.map((msg) => (
        <div key={msg.id}>{msg.content}</div>
      ))}

      {activeTools.map((tool) => {
        const data = tool.data as {
          message?: string;
          progress?: number;
          completed?: string[];
        } | undefined;

        return (
          <div key={tool.toolCallId ?? tool.name}>
            <strong>{tool.name}</strong>
            {data?.message && <p>{data.message}</p>}
            {data?.progress != null && (
              <div style={{ width: "100%", background: "#eee" }}>
                <div
                  style={{
                    width: `${data.progress * 100}%`,
                    background: "#4CAF50",
                    height: 8,
                    transition: "width 0.3s ease",
                  }}
                />
              </div>
            )}
            {data?.completed?.map((step, i) => (
              <div key={i}>#{10003; {step}</div>
            ))}
          </div>
        );
      })}
```

```

    );
  }}}
</div>
);
}

```

tools VS custom stream mode

Both stream modes can surface tool progress, but they serve different purposes:

- **tools** —automatically emits structured lifecycle events (`on_tool_start` , `on_tool_event` , `on_tool_end` , `on_tool_error`) with no code changes needed in your tools beyond using `async function*` . The `useStream` hook provides the reactive `toolProgress` array out of the box.
- **custom** —gives you full control over what data is emitted and when using `config.writer()` . Use this when you need freeform data that doesn't map to the tool lifecycle, or when you want to stream from nodes (not just tools).

Subgraph outputs

python

To include outputs from [subgraphs](#) in the streamed outputs, you can set `subgraphs=True` in the `.stream()` method of the parent graph. This will stream outputs from both the parent graph and any subgraphs.

The outputs will be streamed as tuples (`namespace` , `data`) , where `namespace` is a tuple with the path to the node where a subgraph is invoked, e.g. ("parent_node:<task_id>" , "child_node:<task_id>") .

v2 (LangGraph 1.1 or later)

With `version="v2"` , subgraph events use the same `StreamPart` format. The `ns` field identifies the source:

```

for chunk in graph.stream(
    {"foo": "foo"},
    subgraphs=True, # [!code highlight]
    stream_mode="updates",
    version="v2", # [!code highlight]
):
    print(chunk["type"]) # "updates"
    print(chunk["ns"])   # () for root, ("node_name:<task_id>") for subgraph
    print(chunk["data"]) # {"node_name": {"key": "value"}}

```

v1 (default)

```

for chunk in graph.stream(
    {"foo": "foo"},
    # Set subgraphs=True to stream outputs from subgraphs
    subgraphs=True, # [!code highlight]
    stream_mode="updates",
):
    print(chunk)

```

js

To include outputs from [subgraphs](#) in the streamed outputs, you can set `subgraphs: true` in the `.stream()` method of the parent graph. This will stream outputs from both the parent graph and any subgraphs.

The outputs will be streamed as tuples `[namespace, data]`, where `namespace` is a tuple with the path to the node where a subgraph is invoked, e.g. `["parent_node:<task_id>", "child_node:<task_id>"]`.

```

for await (const chunk of await graph.stream(
  { foo: "foo" },
  {
    // Set subgraphs: true to stream outputs from subgraphs
    subgraphs: true,
    streamMode: "updates",
  }
)) {
  console.log(chunk);
}

```

python

참고

This applies to every `stream_mode`, including `"messages"`. Agent builders like `@[ create_agent ]` return a **compiled graph**, so adding one as a node turns it into a subgraph. Without `subgraphs=True`, `stream_mode="messages"` on the parent graph will not emit token chunks from the inner agent's LLM calls. Invoking `agent.stream(...)` directly will, which is why this often shows up only after wrapping.

```

from langchain.agents import create_agent
from langgraph.graph import END, START, StateGraph

graph = (
    StateGraph(State)
    .add_node("agent", create_agent(model, tools, state_schema=State))

```

```

        .add_edge(START, "agent")
        .add_edge("agent", END)
        .compile()
    )

    for chunk in graph.stream(
        {"messages": [{"role": "user", "content": "..."}]},
        stream_mode="messages",
        subgraphs=True, # [!code highlight]
        version="v2",
    ):
        print(chunk["type"]) # "messages"
        print(chunk["ns"])   # () for root, ("agent:<task_id>",) for subgraph
        print(chunk["data"]) # (token, metadata)

```

js

참고

This applies to every `streamMode`, including `"messages"`. `@[createAgent]` returns a `ReactAgent` wrapper; pass `agent.graph` when adding it as a node so the parent treats it as a subgraph. With `subgraphs: true`, message chunks are `[namespace, [token, metadata]]`, so you can tell which subgraph emitted them.

```

import { createAgent } from "langchain";
import { END, START, StateGraph } from "@langchain/langgraph";

const agent = createAgent({ model, tools, stateSchema: State });

const graph = new StateGraph(State)
    .addNode("agent", agent.graph)
    .addEdge(START, "agent")
    .addEdge("agent", END)
    .compile();

for await (const [ns, data] of await graph.stream(
    { messages: [{ role: "user", content: "..."}] },
    {
        streamMode: "messages",
        subgraphs: true, // [!code highlight]
    }
)) {
    const [token, metadata] = data;
    console.log(ns, token, metadata);
}

```

Extended example: streaming from subgraphs

python

```
from langgraph.graph import START, StateGraph
from typing import TypedDict

# Define subgraph
class SubgraphState(TypedDict):
    foo: str # note that this key is shared with the parent graph state
    bar: str

def subgraph_node_1(state: SubgraphState):
    return {"bar": "bar"}

def subgraph_node_2(state: SubgraphState):
    return {"foo": state["foo"] + state["bar"]}

subgraph_builder = StateGraph(SubgraphState)
subgraph_builder.add_node(subgraph_node_1)
subgraph_builder.add_node(subgraph_node_2)
subgraph_builder.add_edge(START, "subgraph_node_1")
subgraph_builder.add_edge("subgraph_node_1", "subgraph_node_2")
subgraph = subgraph_builder.compile()

# Define parent graph
class ParentState(TypedDict):
    foo: str

def node_1(state: ParentState):
    return {"foo": "hi! " + state["foo"]}

builder = StateGraph(ParentState)
builder.add_node("node_1", node_1)
builder.add_node("node_2", subgraph)
builder.add_edge(START, "node_1")
builder.add_edge("node_1", "node_2")
graph = builder.compile()

for chunk in graph.stream(
    {"foo": "foo"},
    stream_mode="updates",
    # Set subgraphs=True to stream outputs from subgraphs
    subgraphs=True, # [!code highlight]
    version="v2", # [!code highlight]
):
    if chunk["type"] == "updates":
        if chunk["ns"]:
            print(f"Subgraph {chunk['ns']}: {chunk['data']}")
```

```
else:
    print(f"Root: {chunk['data']}")
```

js

```
import { StateGraph, StateSchema, START } from "@langchain/langgraph";
import { z } from "zod/v4";

// Define subgraph
const SubgraphState = new StateSchema({
  foo: z.string(), // note that this key is shared with the parent graph state
  bar: z.string(),
});

const subgraphBuilder = new StateGraph(SubgraphState)
  .addNode("subgraphNode1", (state) => {
    return { bar: "bar" };
  })
  .addNode("subgraphNode2", (state) => {
    return { foo: state.foo + state.bar };
  })
  .addEdge(START, "subgraphNode1")
  .addEdge("subgraphNode1", "subgraphNode2");
const subgraph = subgraphBuilder.compile();

// Define parent graph
const ParentState = new StateSchema({
  foo: z.string(),
});

const builder = new StateGraph(ParentState)
  .addNode("node1", (state) => {
    return { foo: "hi! " + state.foo };
  })
  .addNode("node2", subgraph)
  .addEdge(START, "node1")
  .addEdge("node1", "node2");
const graph = builder.compile();

for await (const chunk of await graph.stream(
  { foo: "foo" },
  {
    streamMode: "updates",
    // Set subgraphs: true to stream outputs from subgraphs
    subgraphs: true,
  }
)) {
```



```
console.log(chunk);
}
```

python

```
Root: {'node_1': {'foo': 'hi! foo'}}
Subgraph ('node_2:dfddc4ba-c3c5-6887-5012-a243b5b377c2',): {'subgraph_node_1': {'bar': 'bar'}}
Subgraph ('node_2:dfddc4ba-c3c5-6887-5012-a243b5b377c2',): {'subgraph_node_2': {'foo': 'hi! foobar'}}
Root: {'node_2': {'foo': 'hi! foobar'}}
```

js

```
[[[], {'node1': {'foo': 'hi! foo'}}]]
[['node2:dfddc4ba-c3c5-6887-5012-a243b5b377c2'], {'subgraphNode1': {'bar': 'bar'}}]
[['node2:dfddc4ba-c3c5-6887-5012-a243b5b377c2'], {'subgraphNode2': {'foo': 'hi! foobar'}}]
[[[], {'node2': {'foo': 'hi! foobar'}}]]
```

Note that we are receiving not just the node updates, but we also the namespaces which tell us what graph (or subgraph) we are streaming from.

python

Checkpoints

Use the `checkpoints` streaming mode to receive checkpoint events as the graph executes. Each checkpoint event has the same format as the output of `get_state()`. Requires a [checkpointer](#).

```
from langgraph.checkpoint.memory import MemorySaver

graph = (
    StateGraph(State)
    .add_node(refine_topic)
    .add_node(generate_joke)
    .add_edge(START, "refine_topic")
    .add_edge("refine_topic", "generate_joke")
    .add_edge("generate_joke", END)
    .compile(checkpointer=MemorySaver())
)

config = {"configurable": {"thread_id": "1"}}

for chunk in graph.stream(
    {"topic": "ice cream"},
```

```

config=config,
stream_mode="checkpoints", # [!code highlight]
version="v2", # [!code highlight]
):
    if chunk["type"] == "checkpoints":
        print(chunk["data"])

```

Tasks

Use the `tasks` streaming mode to receive task start and finish events as the graph executes. Task events include information about which node is running, its results, and any errors. Requires a [checkpointeer](#).

```

from langgraph.checkpoint.memory import MemorySaver

graph = (
    StateGraph(State)
    .add_node(refine_topic)
    .add_node(generate_joke)
    .add_edge(START, "refine_topic")
    .add_edge("refine_topic", "generate_joke")
    .add_edge("generate_joke", END)
    .compile(checkpointer=MemorySaver())
)

config = {"configurable": {"thread_id": "1"}}

for chunk in graph.stream(
    {"topic": "ice cream"},
    config=config,
    stream_mode="tasks", # [!code highlight]
    version="v2", # [!code highlight]
):
    if chunk["type"] == "tasks":
        print(chunk["data"])

```

Debug

Use the `debug` streaming mode to stream as much information as possible throughout the execution of the graph. The streamed outputs include the name of the node as well as the full state.

python

```

for chunk in graph.stream(
    {"topic": "ice cream"},
    stream_mode="debug", # [!code highlight]
    version="v2", # [!code highlight]

```

```

):
    if chunk["type"] == "debug":
        print(chunk["data"])

```

js

```

for await (const chunk of await graph.stream(
  { topic: "ice cream" },
  { streamMode: "debug" }
)) {
  console.log(chunk);
}

```

python

참고

The `debug` mode combines `checkpoints` and `tasks` events with additional metadata. Use `checkpoints` or `tasks` directly if you only need a subset of the debug information.

Multiple modes at once

python

You can pass a list as the `stream_mode` parameter to stream multiple modes at once.

With `version="v2"`, every chunk is a `StreamPart` dict. Use `chunk["type"]` to distinguish between modes:

```

python v2 for chunk in graph.stream(inputs, stream_mode=["updates", "custom"], version="v2"):
    if chunk["type"] == "updates": for node_name, state in chunk["data"].items(): print(f"Node ")
    elif chunk["type"] == "custom": print(f"Custom event: {chunk['data']}")` updated: {state

```

```

python v1
for mode, chunk in graph.stream(inputs, stream_mode=["updates", "custom"]):
    print(chunk)

```

js

You can pass an array as the `streamMode` parameter to stream multiple modes at once.

The streamed outputs will be tuples of `[mode, chunk]` where `mode` is the name of the stream mode and `chunk` is the data streamed by that mode.

```

for await (const [mode, chunk] of await graph.stream(inputs, {
  streamMode: ["updates", "custom"],
})) {
  console.log(chunk);
}

```

Advanced

Use with any LLM

python

You can use `stream_mode="custom"` to stream data from **any LLM API**—even if that API does **not** implement the LangChain chat model interface.

This lets you integrate raw LLM clients or external services that provide their own streaming interfaces, making LangGraph highly flexible for custom setups.

```

from langgraph.config import get_stream_writer

def call_arbitrary_model(state):
    """Example node that calls an arbitrary model and streams the output"""
    # Get the stream writer to send custom data
    writer = get_stream_writer() # [!code highlight]
    # Assume you have a streaming client that yields chunks
    # Generate LLM tokens using your custom streaming client
    for chunk in your_custom_streaming_client(state["topic"]):
        # Use the writer to send custom data to the stream
        writer({"custom_llm_chunk": chunk}) # [!code highlight]
    return {"result": "completed"}

graph = (
    StateGraph(State)
    .add_node(call_arbitrary_model)
    # Add other nodes and edges as needed
    .compile()
)

# Set stream_mode="custom" to receive the custom data in the stream
for chunk in graph.stream(
    {"topic": "cats"},
    stream_mode="custom", # [!code highlight]
    version="v2", # [!code highlight]
):
    if chunk["type"] == "custom":
        # The chunk data will contain the custom data streamed from the llm
        print(chunk["data"])

```

js

You can use `streamMode: "custom"` to stream data from **any LLM API**—even if that API does **not** implement the LangChain chat model interface.

This lets you integrate raw LLM clients or external services that provide their own streaming interfaces, making LangGraph highly flexible for custom setups.

```
import { StateGraph, GraphNode, StateSchema } from "@langchain/langgraph";
import * as z from "zod";

const State = new StateSchema({ result: z.string() });

const callArbitraryModel: GraphNode<typeof State> = async (state, config) => {
  // Example node that calls an arbitrary model and streams the output
  // Assume you have a streaming client that yields chunks
  // Generate LLM tokens using your custom streaming client
  for await (const chunk of yourCustomStreamingClient(state.topic)) {
    // Use the writer to send custom data to the stream
    config.writer({ custom_llm_chunk: chunk });
  }
  return { result: "completed" };
};

const graph = new StateGraph(State)
  .addNode("callArbitraryModel", callArbitraryModel)
  // Add other nodes and edges as needed
  .compile();

// Set streamMode: "custom" to receive the custom data in the stream
for await (const chunk of await graph.stream(
  { topic: "cats" },
  { streamMode: "custom" }
)) {
  // The chunk will contain the custom data streamed from the llm
  console.log(chunk);
}
```

Extended example: streaming arbitrary chat model

python

```
import operator
import json

from typing import TypedDict
from typing_extensions import Annotated
from langgraph.graph import StateGraph, START
```

```

from openai import AsyncOpenAI

openai_client = AsyncOpenAI()
model_name = "gpt-5.4-mini"

async def stream_tokens(model_name: str, messages: list[dict]):
    response = await openai_client.chat.completions.create(
        messages=messages, model=model_name, stream=True
    )
    role = None
    async for chunk in response:
        delta = chunk.choices[0].delta

    if delta.role is not None:
        role = delta.role

    if delta.content:
        yield {"role": role, "content": delta.content}

# this is our tool
async def get_items(place: str) -> str:
    """Use this tool to list items one might find in a place you're asked about."""
    writer = get_stream_writer()
    response = ""
    async for msg_chunk in stream_tokens(
        model_name,
        [
            {
                "role": "user",
                "content": (
                    "Can you tell me what kind of items "
                    f"i might find in the following place: '{place}'."
                    "List at least 3 such items separating them by a comma. "
                    "And include a brief description of each item."
                ),
            },
        ],
    ):
        response += msg_chunk["content"]
        writer(msg_chunk)

    return response

class State(TypedDict):
    messages: Annotated[list[dict], operator.add]

# this is the tool-calling graph node
async def call_tool(state: State):
    ai_message = state["messages"][-1]
    tool_call = ai_message["tool_calls"][-1]

```

```

function_name = tool_call["function"]["name"]
if function_name != "get_items":
    raise ValueError(f"Tool {function_name} not supported")

function_arguments = tool_call["function"]["arguments"]
arguments = json.loads(function_arguments)

function_response = await get_items(**arguments)
tool_message = {
    "tool_call_id": tool_call["id"],
    "role": "tool",
    "name": function_name,
    "content": function_response,
}
return {"messages": [tool_message]}

graph = (
    StateGraph(State)
    .add_node(call_tool)
    .add_edge(START, "call_tool")
    .compile()
)

```

Let's invoke the graph with an `@[AIMessage]` that includes a tool call:

```

inputs = {
    "messages": [
        {
            "content": None,
            "role": "assistant",
            "tool_calls": [
                {
                    "id": "1",
                    "function": {
                        "arguments": '{"place":"bedroom"}',
                        "name": "get_items",
                    },
                    "type": "function",
                }
            ],
        }
    ]
}

async for chunk in graph.astream(
    inputs,
    stream_mode="custom",
    version="v2",
):

```

```
if chunk["type"] == "custom":
    print(chunk["data"]["content"], end="|", flush=True)
```

js

```
import { StateGraph, StateSchema, MessagesValue, GraphNode, START, LangGraphRunnableConfig }
from "@langchain/langgraph";
import { tool } from "@langchain/core/tools";
import * as z from "zod";
import OpenAI from "openai";

const openaiClient = new OpenAI();
const modelName = "gpt-5.4-mini";

async function* streamTokens(modelName: string, messages: any[]) {
    const response = await openaiClient.chat.completions.create({
        messages,
        model: modelName,
        stream: true,
    });

    let role: string | null = null;
    for await (const chunk of response) {
        const delta = chunk.choices[0]?.delta;

        if (delta?.role) {
            role = delta.role;
        }

        if (delta?.content) {
            yield { role, content: delta.content };
        }
    }
}

// this is our tool
const.getItems = tool(
    async (input, config: LangGraphRunnableConfig) => {
        let response = "";
        for await (const msgChunk of streamTokens(
            modelName,
            [
                {
                    role: "user",
                    content: `Can you tell me what kind of items i might find in the following place:
                    '${input.place}'. List at least 3 such items separating them by a comma. And include a brief
                    description of each item.`
                }
            ]
        )) {
            response += msgChunk.content;
        }
    }
);
```



```

    )) {
      response += msgChunk.content;
      config.writer?.(msgChunk);
    }
    return response;
  },
  {
    name: "get_items",
    description: "Use this tool to list items one might find in a place you're asked about.",
    schema: z.object({
      place: z.string().describe("The place to look up items for."),
    }),
  }
];

const State = new StateSchema({
  messages: MessagesValue,
});

const callTool: GraphNode<typeof State> = async (state) => {
  const aiMessage = state.messages.at(-1);
  const toolCall = aiMessage.tool_calls?.at(-1);

  const functionName = toolCall?.function?.name;
  if (functionName !== "get_items") {
    throw new Error(`Tool ${functionName} not supported`);
  }

  const functionArguments = toolCall?.function?.arguments;
  const args = JSON.parse(functionArguments);

  const functionResponse = await getItems.invoke(args);
  const toolMessage = {
    tool_call_id: toolCall.id,
    role: "tool",
    name: functionName,
    content: functionResponse,
  };
  return { messages: [toolMessage] };
};

const graph = new StateGraph(State)
  // this is the tool-calling graph node
  .addNode("callTool", callTool)
  .addEdge(START, "callTool")
  .compile();

```

Let's invoke the graph with an `@[AIMessage]` that includes a tool call:

```

const inputs = {
  messages: [
    {
      content: null,
      role: "assistant",
      tool_calls: [
        {
          id: "1",
          function: {
            arguments: '{"place":"bedroom"}',
            name: "get_items",
          },
          type: "function",
        }
      ],
    }
  ]
};

for await (const chunk of await graph.stream(
  inputs,
  { streamMode: "custom" }
)) {
  console.log(chunk.content + "|");
}

```

Disable streaming for specific chat models

If your application mixes models that support streaming with those that do not, you may need to explicitly disable streaming for models that do not support it.

python

Set `streaming=False` when initializing the model.

init_chat_model

```

from langchain.chat_models import init_chat_model

model = init_chat_model(
    "claude-sonnet-4-6",
    # Set streaming=False to disable streaming for the chat model
    streaming=False # [!code highlight]
)

```

Chat model interface

```
from langchain_openai import ChatOpenAI

# Set streaming=False to disable streaming for the chat model
model = ChatOpenAI(model="gpt-5.5", streaming=False)
```

js

Set `streaming: false` when initializing the model.

```
import { ChatOpenAI } from "@langchain/openai";

const model = new ChatOpenAI({
  model: "gpt-5.5",
  // Set streaming: false to disable streaming for the chat model
  streaming: false,
});
```

python

참고

Not all chat model integrations support the `streaming` parameter. If your model doesn't support it, use `disable_streaming=True` instead. This parameter is available on all chat models via the base class.

js

참고

Not all chat model integrations support the `streaming` parameter. If your model doesn't support it, use `disableStreaming: true` instead. This parameter is available on all chat models via the base class.

python

Migrate to v2

The v2 streaming format (used throughout this page) provides a unified output format. Here's a summary of the key differences and how to migrate:

Scenario	v1 (default)	v2 (version="v2")
Single stream mode	Raw data (dict)	StreamPart dict with type , ns , data
Multiple stream modes	(mode, data) tuples	Same StreamPart dict, filter on chunk["type"]
Subgraph streaming	(namespace, data) tuples	Same StreamPart dict, check chunk["ns"]
Multiple modes + subgraphs	(namespace, mode, data) triples	Same StreamPart dict
invoke() return type	Plain dict (state)	GraphOutput with .value and .interrupts
Interrupt location (stream)	__interrupt__ key in state dict	interrupts field on values stream parts
Interrupt location (invoke)	__interrupt__ key in result dict	.interrupts attribute on GraphOutput
Pydantic/dataclass output	Returns plain dict	Coerces to model/dataclass instance

v2 invoke format

When you pass `version="v2"` to `invoke()` or `ainvoke()`, it returns a `@[GraphOutput]` object with `.value` and `.interrupts` attributes:

```
from langgraph.types import GraphOutput

result = graph.invoke(inputs, version="v2")

assert isinstance(result, GraphOutput)
result.value      # your output – dict, Pydantic model, or dataclass
result.interrupts # tuple[Interrupt, ...], empty if none occurred
```

With any stream mode other than the default `"values"`, `invoke(..., stream_mode="updates", version="v2")` returns `list[StreamPart]` instead of `list[tuple]`.

주의

Dict-style access on `GraphOutput` (`result["key"]`, `"key" in result`, `result["__interrupt__"]`) still works for backwards compatibility but is **deprecated** and will be removed in a future version. Migrate to `result.value` and `result.interrupts`.

This separates state from interrupt metadata. With v1, interrupts are embedded in the returned dict under `__interrupt__`:

```
```python v2 (new) config = {"configurable": {"thread_id": "thread-1"}} result =
graph.invoke(inputs, config=config, version="v2")
```

```
if result.interrupts: print(result.interrupts[0].value) graph.invoke(Command(resume=True),
config=config, version="v2")
```

```
```python v1 (current default)
config = {"configurable": {"thread_id": "thread-1"}}
result = graph.invoke(inputs, config=config)

if "__interrupt__" in result:
    print(result["__interrupt__"][0].value)
    graph.invoke(Command(resume=True), config=config)
```

Pydantic and dataclass state coercion

When your graph state is a Pydantic model or dataclass, v2 `values` mode automatically coerces output to the correct type:

```
from pydantic import BaseModel
from typing import Annotated
import operator

class MyState(BaseModel):
    value: str
    items: Annotated[list[str], operator.add]

# With version="v2", chunk["data"] is a MyState instance
for chunk in graph.stream(
    {"value": "x", "items": []}, stream_mode="values", version="v2"
):
    print(type(chunk["data"])) # <class 'MyState'>
```

Async with Python < 3.11

In Python versions < 3.11, [asyncio tasks](#) do not support the `context` parameter. This limits LangGraph ability to automatically propagate context, and affects LangGraph's streaming mechanisms in two key ways:

1. You **must** explicitly pass [RunnableConfig](#) into async LLM calls (e.g., `ainvoke()`), as callbacks are not automatically propagated.
2. You **cannot** use `@get_stream_writer` in async nodes or tools—you must pass a `writer` argument directly.

Extended example: async LLM call with manual config

```

from typing import TypedDict
from langgraph.graph import START, StateGraph
from langchain.chat_models import init_chat_model

model = init_chat_model(model="gpt-5.4-mini")

class State(TypedDict):
    topic: str
    joke: str

# Accept config as an argument in the async node function
async def call_model(state, config):
    topic = state["topic"]
    print("Generating joke...")
    # Pass config to model.ainvoke() to ensure proper context propagation
    joke_response = await model.ainvoke( # [!code highlight]
        [{"role": "user", "content": f"Write a joke about {topic}"}],
        config,
    )
    return {"joke": joke_response.content}

graph = (
    StateGraph(State)
    .add_node(call_model)
    .add_edge(START, "call_model")
    .compile()
)

# Set stream_mode="messages" to stream LLM tokens
async for chunk in graph.astream(
    {"topic": "ice cream"},
    stream_mode="messages", # [!code highlight]
    version="v2", # [!code highlight]
):
    if chunk["type"] == "messages":
        message_chunk, metadata = chunk["data"]
        if message_chunk.content:
            print(message_chunk.content, end="|", flush=True)

```

Extended example: async custom streaming with stream writer

```

from typing import TypedDict
from langgraph.types import StreamWriter

class State(TypedDict):
    topic: str
    joke: str

# Add writer as an argument in the function signature of the async node or tool
# LangGraph will automatically pass the stream writer to the function

```

```

async def generate_joke(state: State, writer: StreamWriter): # [!code highlight]
    writer({"custom_key": "Streaming custom data while generating a joke"})
    return {"joke": f"This is a joke about {state['topic']}"}

graph = (
    StateGraph(State)
        .add_node(generate_joke)
        .add_edge(START, "generate_joke")
        .compile()
)

# Set stream_mode="custom" to receive the custom data in the stream # [!code highlight]
async for chunk in graph.astream(
    {"topic": "ice cream"},
    stream_mode="custom",
    version="v2",
):
    if chunk["type"] == "custom":
        print(chunk["data"])

```

LangSmith Studio

langgraph/studio.mdx

When building agents with LangChain locally, it's helpful to visualize what's happening inside your agent, interact with it in real-time, and debug issues as they occur. **LangSmith Studio** is a free visual interface for developing and testing your LangChain agents from your local machine.

Studio connects to your locally running agent to show you each step your agent takes: the prompts sent to the model, tool calls and their results, and the final output. You can test different inputs, inspect intermediate states, and iterate on your agent's behavior without additional code or deployment.

This page describes how to set up Studio with your local LangChain agent.

Prerequisites

Before you begin, ensure you have the following:

- **A LangSmith account:** Sign up (for free) or log in at smith.langchain.com.
- **A LangSmith API key:** Follow the [Create an API key](#) guide.
- If you don't want data [traced](#) to LangSmith, set `LANGSMITH_TRACING=false` in your application's `.env` file. With tracing disabled, no data leaves your local server.

Set up local Agent server

1. Install the LangGraph CLI

The [LangGraph CLI](#) provides a local development server (also called [Agent Server](#)) that connects your agent to Studio.

python

```
# Python >= 3.11 is required.  
pip install --upgrade "langgraph-cli[inmem]"
```

js

```
npx @langchain/langgraph-cli
```


2. Prepare your agent

If you already have a LangChain agent, you can use it directly. This example uses a simple email agent:

python

```
```python title="agent.py" from langchain.agents import create_agent

def send_email(to: str, subject: str, body: str): """Send an email""" email = { "to": to, "subject":
subject, "body": body } # ... email sending logic

return f"Email sent to {to}"

agent = create_agent("gpt-5.5", tools=[send_email], system_prompt="You are an email
assistant. Always use the send_email tool.",)
```

</div>

<div class="callout js" markdown="1">

<p class="callout-label">js</p>

```
```typescript title="agent.ts"
```

```
import { createAgent } from "@langchain/langgraph";
```

```
function sendEmail(to: string, subject: string, body: string): string {
```

```
  // Send an email
```

```
  const email = {
```

```
    to: to,
```

```
    subject: subject,
```

```
    body: body
```

```
  };
```

```
  // ... email sending logic
```

```
  return `Email sent to ${to}`;
```

```
}
```

```
const agent = createAgent({
```

```
  model: "gpt-5.5",
```

```
  tools: [sendEmail],
```

```
  systemPrompt: "You are an email assistant. Always use the sendEmail tool.",
```

```
});
```

```
export { agent };
```

3. Environment variables

Studio requires a LangSmith API key to connect your local agent. Create a `.env` file in the root of your project and add your API key from [LangSmith](#).

주의

Ensure your `.env` file is not committed to version control, such as Git.

```
```bash .env LANGSMITH_API_KEY=lsv2...
```

```
4. Create a LangGraph config file
```

The LangGraph CLI uses a configuration file to locate your agent and manage dependencies. Create a `langgraph.json` file in your app's directory:

```
<div class="callout python" markdown="1">
```

```
<p class="callout-label">python</p>
```

```
```json title="langgraph.json"
{
  "dependencies": ["."],
  "graphs": {
    "agent": "./src/agent.py:agent"
  },
  "env": ".env"
}
```

js

```
```json title="langgraph.json" { "dependencies": ["."], "graphs": { "agent":
"./src/agent.ts:agent" }, "env": ".env" }
```

```
</div>
```

```
<div class="callout python" markdown="1">
```

```
<p class="callout-label">python</p>
```

The `@[`create_agent`]` function automatically returns a compiled LangGraph graph, which is what the `graphs` key expects in the configuration file.

```
</div>
```

```
<div class="callout js" markdown="1">
```

```
<p class="callout-label">js</p>
```

The `@[`createAgent`]` function automatically returns a compiled LangGraph graph, which is what the ``graphs`` key expects in the configuration file.

```
</div>
```

```
<div class="callout info" markdown="1">
```

```
<p class="callout-label">정보</p>
```

For detailed explanations of each key in the JSON object of the configuration file, refer to the [LangGraph configuration file reference](/langsmith/cli#configuration-file).

```
</div>
```

At this point, the project structure will look like this:

```
<div class="callout python" markdown="1">
```

```
<p class="callout-label">python</p>
```

```
```bash
my-app/
├── src
│   └── agent.py
├── .env
└── langgraph.json
```

js

```
my-app/
├── src
│   └── agent.ts
├── .env
├── package.json
└── langgraph.json
```

5. Install dependencies

python

Install your project dependencies from the root directory:

```
```shell pip pip install langchain langchain-openai
```

```
```shell uv
uv add langchain langchain-openai
```

js

```
yarn install
```

6. View your agent in Studio

Start the development server to connect your agent to Studio:

python

```
langgraph dev
```

js

```
npx @langchain/langgraph-cli dev
```

주의

Safari blocks `localhost` connections to Studio. To work around this, run the above command with `--tunnel` to access Studio via a secure tunnel. You'll need to manually add the tunnel URL to allowed origins by clicking **Connect to a local server** in the Studio UI. See the [troubleshooting guide](#) for steps.

Once the server is running, your agent is accessible both via API at `http://127.0.0.1:2024` and through the Studio UI at `https://smith.langchain.com/studio/?baseUrl=http://127.0.0.1:2024`:

 Agent view in the Studio UI

With Studio connected to your local agent, you can iterate quickly on your agent's behavior. Run a test input, inspect the full execution trace including prompts, tool arguments, return values, and token/latency metrics in [LangSmith](#). When something goes wrong, Studio captures exceptions with the surrounding state to help you understand what happened.

The development server supports hot-reloading—make changes to prompts or tool signatures in your code, and Studio reflects them immediately. Re-run conversation threads from any step to test your changes without starting over. This workflow scales from simple single-tool agents to complex multi-node graphs.

For more information on how to run Studio, refer to the following guides in the [LangSmith docs](#):

- [Run application](#)
- [Manage assistants](#)
- [Manage threads](#)
- [Iterate on prompts](#)
- [Debug LangSmith traces](#)
- [Add node to dataset](#)

Video guide

Test

langgraph/test.mdx

After you've prototyped your LangGraph agent, a natural next step is to add tests. This guide covers some useful patterns you can use when writing unit tests.

python

Note that this guide is LangGraph-specific and covers scenarios around graphs with custom structures - if you are just getting started, check out [Test](#) that uses LangChain's built-in `@[ create_agent ]` instead.

js

Note that this guide is LangGraph-specific and covers scenarios around graphs with custom structures - if you are just getting started, check out [Test](#) that uses LangChain's built-in `@[ createAgent ]` instead.

Prerequisites

python

First, make sure you have [pytest](#) installed:

```
$ pip install -U pytest
```

js

First, make sure you have [vitest](#) installed:

```
$ npm install -D vitest
```

Getting started

Because many LangGraph agents depend on state, a useful pattern is to create your graph before each test where you use it, then compile it within tests with a new checkpoint instance.

The below example shows how this works with a simple, linear graph that progresses through `node1` and `node2`. Each node updates the single state key `my_key`:

python

```
import pytest

from typing_extensions import TypedDict
from langgraph.graph import StateGraph, START, END
from langgraph.checkpoint.memory import MemorySaver

def create_graph() -> StateGraph:
    class MyState(TypedDict):
        my_key: str

    graph = StateGraph(MyState)
    graph.add_node("node1", lambda state: {"my_key": "hello from node1"})
    graph.add_node("node2", lambda state: {"my_key": "hello from node2"})
    graph.add_edge(START, "node1")
    graph.add_edge("node1", "node2")
    graph.add_edge("node2", END)
    return graph

def test_basic_agent_execution() -> None:
    checkpointer = MemorySaver()
    graph = create_graph()
    compiled_graph = graph.compile(checkpointer=checkpointer)
    result = compiled_graph.invoke(
        {"my_key": "initial_value"},
        config={"configurable": {"thread_id": "1"}}
    )
    assert result["my_key"] == "hello from node2"
```

js

```
import { test, expect } from 'vitest';
import {
    StateGraph,
    StateSchema,
    START,
    END,
    MemorySaver,
} from '@langchain/langgraph';
import * as z from "zod";

const State = new StateSchema({
    my_key: z.string(),
});

const createGraph = () => {
    return new StateGraph(State)
```

```

        .addNode('node1', (state) => ({ my_key: 'hello from node1' })))
        .addNode('node2', (state) => ({ my_key: 'hello from node2' })))
        .addEdge(START, 'node1')
        .addEdge('node1', 'node2')
        .addEdge('node2', END);
};

test('basic agent execution', async () => {
    const uncompiledGraph = createGraph();
    const checkpointer = new MemorySaver();
    const compiledGraph = uncompiledGraph.compile({ checkpointer });
    const result = await compiledGraph.invoke(
        { my_key: 'initial_value' },
        { configurable: { thread_id: '1' } }
    );
    expect(result.my_key).toBe('hello from node2');
});

```

Testing individual nodes and edges

Compiled LangGraph agents expose references to each individual node as `graph.nodes`. You can take advantage of this to test individual nodes within your agent. Note that this will bypass any checkpointers passed when compiling the graph:

python

```

import pytest

from typing_extensions import TypedDict
from langgraph.graph import StateGraph, START, END
from langgraph.checkpoint.memory import MemorySaver

def create_graph() -> StateGraph:
    class MyState(TypedDict):
        my_key: str

    graph = StateGraph(MyState)
    graph.add_node("node1", lambda state: {"my_key": "hello from node1"})
    graph.add_node("node2", lambda state: {"my_key": "hello from node2"})
    graph.add_edge(START, "node1")
    graph.add_edge("node1", "node2")
    graph.add_edge("node2", END)
    return graph

def test_individual_node_execution() -> None:
    # Will be ignored in this example
    checkpointer = MemorySaver()
    graph = create_graph()

```



```

compiled_graph = graph.compile(checkpointer=checkpointer)
# Only invoke node 1
result = compiled_graph.nodes["node1"].invoke(
    {"my_key": "initial_value"},
)
assert result["my_key"] == "hello from node1"

```

js

```

import { test, expect } from 'vitest';
import {
  StateGraph,
  START,
  END,
  MemorySaver,
  StateSchema,
} from '@langchain/langgraph';
import * as z from "zod";

const State = new StateSchema({
  my_key: z.string(),
});

const createGraph = () => {
  return new StateGraph(State)
    .addNode('node1', (state) => ({ my_key: 'hello from node1' }))
    .addNode('node2', (state) => ({ my_key: 'hello from node2' }))
    .addEdge(START, 'node1')
    .addEdge('node1', 'node2')
    .addEdge('node2', END);
};

test('individual node execution', async () => {
  const uncompiledGraph = createGraph();
  // Will be ignored in this example
  const checkpointer = new MemorySaver();
  const compiledGraph = uncompiledGraph.compile({ checkpointer });
  // Only invoke node 1
  const result = await compiledGraph.nodes['node1'].invoke(
    { my_key: 'initial_value' },
  );
  expect(result.my_key).toBe('hello from node1');
});

```

Partial execution

For agents made up of larger graphs, you may wish to test partial execution paths within your agent rather than the entire flow end-to-end. In some cases, it may make semantic sense to [restructure these sections as subgraphs](#), which you can invoke in isolation as normal.

However, if you do not wish to make changes to your agent graph's overall structure, you can use LangGraph's persistence mechanisms to simulate a state where your agent is paused right before the beginning of the desired section, and will pause again at the end of the desired section. The steps are as follows:

python

1. Compile your agent with a checkpoint (the in-memory checkpointer `@[ InMemorySaver ]` will suffice for testing).
2. Call your agent's `update_state` method with an `as_node` parameter set to the name of the node *before* the one you want to start your test.
3. Invoke your agent with the same `thread_id` you used to update the state and an `interrupt_after` parameter set to the name of the node you want to stop at.

js

1. Compile your agent with a checkpoint (the in-memory checkpointer `MemorySaver` will suffice for testing).
2. Call your agent's `update_state` method with an `asNode` parameter set to the name of the node *before* the one you want to start your test.
3. Invoke your agent with the same `thread_id` you used to update the state and an `interruptBefore` parameter set to the name of the node you want to stop at.

Here's an example that executes only the second and third nodes in a linear graph:

python

```
import pytest

from typing_extensions import TypedDict
from langgraph.graph import StateGraph, START, END
from langgraph.checkpoint.memory import MemorySaver

def create_graph() -> StateGraph:
    class MyState(TypedDict):
        my_key: str

    graph = StateGraph(MyState)
    graph.add_node("node1", lambda state: {"my_key": "hello from node1"})
```

```

graph.add_node("node2", lambda state: {"my_key": "hello from node2"})
graph.add_node("node3", lambda state: {"my_key": "hello from node3"})
graph.add_node("node4", lambda state: {"my_key": "hello from node4"})
graph.add_edge(START, "node1")
graph.add_edge("node1", "node2")
graph.add_edge("node2", "node3")
graph.add_edge("node3", "node4")
graph.add_edge("node4", END)
return graph

def test_partial_execution_from_node2_to_node3() -> None:
    checkpointer = MemorySaver()
    graph = create_graph()
    compiled_graph = graph.compile(checkpointer=checkpointer)
    compiled_graph.update_state(
        config={
            "configurable": {
                "thread_id": "1"
            }
        },
        # The state passed into node 2 - simulating the state at
        # the end of node 1
        values={"my_key": "initial_value"},
        # Update saved state as if it came from node 1
        # Execution will resume at node 2
        as_node="node1",
    )
    result = compiled_graph.invoke(
        # Resume execution by passing None
        None,
        config={"configurable": {"thread_id": "1"}},
        # Stop after node 3 so that node 4 doesn't run
        interrupt_after="node3",
    )
    assert result["my_key"] == "hello from node3"

```

js

```

import { test, expect } from 'vitest';
import {
    StateGraph,
    StateSchema,
    START,
    END,
    MemorySaver,
} from '@langchain/langgraph';
import * as z from "zod";

const State = new StateSchema({

```

```

    my_key: z.string(),
  });

const createGraph = () => {
  return new StateGraph(State)
    .addNode('node1', (state) => ({ my_key: 'hello from node1' }))
    .addNode('node2', (state) => ({ my_key: 'hello from node2' }))
    .addNode('node3', (state) => ({ my_key: 'hello from node3' }))
    .addNode('node4', (state) => ({ my_key: 'hello from node4' }))
    .addEdge(START, 'node1')
    .addEdge('node1', 'node2')
    .addEdge('node2', 'node3')
    .addEdge('node3', 'node4')
    .addEdge('node4', END);
};

test('partial execution from node2 to node3', async () => {
  const uncompiledGraph = createGraph();
  const checkpointer = new MemorySaver();
  const compiledGraph = uncompiledGraph.compile({ checkpointer });
  await compiledGraph.updateState(
    { configurable: { thread_id: '1' } },
    // The state passed into node 2 - simulating the state at
    // the end of node 1
    { my_key: 'initial_value' },
    // Update saved state as if it came from node 1
    // Execution will resume at node 2
    'node1',
  );
  const result = await compiledGraph.invoke(
    // Resume execution by passing None
    null,
    {
      configurable: { thread_id: '1' },
      // Stop after node 3 so that node 4 doesn't run
      interruptAfter: ['node3']
    },
  );
  expect(result.my_key).toBe('hello from node3');
});

```

Thinking in LangGraph

langgraph/thinking-in-langgraph.mdx

When you build an agent with LangGraph, you will first break it apart into discrete steps called **nodes**. Then, you will describe the different decisions and transitions from each of your nodes. Finally, you connect nodes together through a shared **state** that each node can read from and write to.

In this walkthrough, we'll guide you through the thought process of building a customer support email agent with LangGraph.

Start with the process you want to automate

Imagine that you need to build an AI agent that handles customer support emails. Your product team has given you these requirements:

The agent should:

- Read incoming customer emails
- Classify them by urgency and topic
- Search relevant documentation to answer questions
- Draft appropriate responses
- Escalate complex issues to human agents
- Schedule follow-ups when needed

Example scenarios to handle:

1. Simple product question: "How do I reset my password?"
2. Bug report: "The export feature crashes when I select PDF format"
3. Urgent billing issue: "I was charged twice for my subscription!"
4. Feature request: "Can you add dark mode to the mobile app?"
5. Complex technical issue: "Our API integration fails intermittently with 504 errors"

To implement an agent in LangGraph, you will usually follow the same five steps.

Step 1: Map out your workflow as discrete steps

Start by identifying the distinct steps in your process. Each step will become a **node** (a function that does one specific thing). Then, sketch how these steps connect to each other.

```
flowchart TD
    A[START] --> B[Read Email]
    B --> C[Classify Intent]
    C --> D[Doc Search]
```

```

C --> E[Bug Track]
C --> F[Human Review]

D --> G[Draft Reply]
E --> G
F --> G

G --> H[Human Review]
G --> I[Send Reply]

H --> J[END]
I --> J[END]

classDef process fill:#E5F4FF,stroke:#006DDD,stroke-width:2px,color:#030710
class A,B,C,D,E,F,G,H,I,J process

```

The arrows in this diagram show possible paths, but the actual decision of which path to take happens inside each node.

Now that we've identified the components in our workflow, let's understand what each node needs to do:

- **Read Email**: Extract and parse the email content
- **Classify Intent**: Use an LLM to categorize urgency and topic, then route to appropriate action
- **Doc Search**: Query your knowledge base for relevant information
- **Bug Track**: Create or update issue in tracking system
- **Draft Reply**: Generate an appropriate response
- **Human Review**: Escalate to human agent for approval or handling
- **Send Reply**: Dispatch the email response

팁

Notice that some nodes make decisions about where to go next (**Classify Intent** , **Draft Reply** , **Human Review**), while others always proceed to the same next step (**Read Email** always goes to **Classify Intent** , **Doc Search** always goes to **Draft Reply**).

Step 2: Identify what each step needs to do

For each node in your graph, determine what type of operation it represents and what context it needs to work properly.

[LLM steps](#)

Use when you need to understand, analyze, generate text, or make reasoning decisions

Data steps

Use when you need to retrieve information from external sources

Action steps

Use when you need to perform external actions

User input steps

Use when you need human intervention

LLM steps

When a step needs to understand, analyze, generate text, or make reasoning decisions:

Classify intent

- Static context (prompt): Classification categories, urgency definitions, response format
- Dynamic context (from state): Email content, sender information
- Desired outcome: Structured classification that determines routing

Draft reply

- Static context (prompt): Tone guidelines, company policies, response templates
- Dynamic context (from state): Classification results, search results, customer history
- Desired outcome: Professional email response ready for review

Data steps

When a step needs to retrieve information from external sources:

Document search

- Parameters: Query built from intent and topic
- Retry strategy: Yes, with exponential backoff for transient failures
- Caching: Could cache common queries to reduce API calls

Customer history lookup

- Parameters: Customer email or ID from state
- Retry strategy: Yes, but with fallback to basic info if unavailable
- Caching: Yes, with time-to-live to balance freshness and performance

Action steps

When a step needs to perform an external action:

Send reply

- When to execute node: After approval (human or automated)
- Retry strategy: Yes, with exponential backoff for network issues
- Should not cache: Each send is a unique action

Bug track

- When to execute node: Always when intent is "bug"
- Retry strategy: Yes, critical to not lose bug reports
- Returns: Ticket ID to include in response

User input steps

When a step needs human intervention:

Human review node

- Context for decision: Original email, draft response, urgency, classification
- Expected input format: Approval boolean plus optional edited response
- When triggered: High urgency, complex issues, or quality concerns

Step 3: Design your state

State is the shared [memory](#) accessible to all nodes in your agent. Think of it as the notebook your agent uses to keep track of everything it learns and decides as it works through the process.

What belongs in state?

Ask yourself these questions about each piece of data:

Include in state

Does it need to persist across steps? If yes, it goes in state.

Don't store

Can you derive it from other data? If yes, compute it when needed instead of storing it in state.

For our email agent, we need to track:

- The original email and sender info (can't reconstruct these later)
- Classification results (needed by multiple later/downstream nodes)
- Search results and customer data (expensive to re-fetch)
- The draft response (needs to persist through review)
- Execution metadata (for debugging and recovery)

Keep state raw, format prompts on-demand

팁

A key principle: your state should store raw data, not formatted text. Format prompts inside nodes when you need them.

This separation means:

- Different nodes can format the same data differently for their needs
- You can change prompt templates without modifying your state schema
- Debugging is clearer—you see exactly what data each node received
- Your agent can evolve without breaking existing state

Let's define our state:

python

```
from typing import TypedDict, Literal

# Define the structure for email classification
class EmailClassification(TypedDict):
    intent: Literal["question", "bug", "billing", "feature", "complex"]
    urgency: Literal["low", "medium", "high", "critical"]
    topic: str
    summary: str

class EmailAgentState(TypedDict):
    # Raw email data
    email_content: str
    sender_email: str
    email_id: str

    # Classification result
    classification: EmailClassification | None

    # Raw search/API results
    search_results: list[str] | None # List of raw document chunks
    customer_history: dict | None # Raw customer data from CRM

    # Generated content
    draft_response: str | None
    messages: list[str] | None
```

js

```
import { StateSchema } from "@langchain/langgraph";
import * as z from "zod";

// Define the structure for email classification
const EmailClassificationSchema = z.object({
```

```

    intent: z.enum(["question", "bug", "billing", "feature", "complex"]),
    urgency: z.enum(["low", "medium", "high", "critical"]),
    topic: z.string(),
    summary: z.string(),
  });

  const EmailAgentState = new StateSchema({
    // Raw email data
    emailContent: z.string(),
    senderEmail: z.string(),
    emailId: z.string(),

    // Classification result
    classification: EmailClassificationSchema.optional(),

    // Raw search/API results
    searchResults: z.array(z.string()).optional(), // List of raw document chunks
    customerHistory: z.record(z.string(), z.any()).optional(), // Raw customer data from CRM

    // Generated content
    responseText: z.string().optional(),
  });

  type EmailClassificationType = z.infer<typeof EmailClassificationSchema>;

```

Notice that the state contains only raw data—no prompt templates, no formatted strings, no instructions. The classification output is stored as a single dictionary, straight from the LLM.

Step 4: Build your nodes

python

Now we implement each step as a function. A node in LangGraph is just a Python function that takes the current state and returns updates to it.

js

Now we implement each step as a function. A node in LangGraph is just a JavaScript function that takes the current state and returns updates to it.

Handle errors appropriately

Different errors need different handling strategies:

Error Type	Who Fixes It	Strategy	When to Use
Transient errors (network issues, rate limits)	System (automatic)	Retry policy	Temporary failures that usually resolve on retry
LLM-recoverable errors (tool failures, parsing issues)	LLM	Store error in state and loop back	LLM can see the error and adjust its approach
User-fixable errors (missing information, unclear instructions)	Human	Pause with <code>interrupt()</code>	Need user input to proceed
Recoverable failure after retries	Developer (declarative)	<code>error_handler</code>	Run a compensation/recovery branch after retry exhaustion
Unexpected errors	Developer	Let them bubble up	Unknown issues that need debugging

Transient errors

Add a retry policy to automatically retry network issues and rate limits.

python

Combine with `timeout=` to cap each attempt. See [Fault tolerance](#) for the full lifecycle.

```
from langgraph.types import RetryPolicy

workflow.add_node(
    "search_documentation",
    search_documentation,
    retry_policy=RetryPolicy(max_attempts=3, initial_interval=1.0)
)
```

js

```
import type { RetryPolicy } from "@langchain/langgraph";

workflow.addNode(
  "searchDocumentation",
  searchDocumentation,
  {
    retryPolicy: { maxAttempts: 3, initialInterval: 1.0 },
  },
);
```

LLM-recoverable

Store the error in state and loop back so the LLM can see what went wrong and try again:

python

```
from langgraph.types import Command

def execute_tool(state: State) -> Command[Literal["agent", "execute_tool"]]:
    try:
        result = run_tool(state['tool_call'])
        return Command(update={"tool_result": result}, goto="agent")
    except ToolError as e:
        # Let the LLM see what went wrong and try again
        return Command(
            update={"tool_result": f"Tool error: {str(e)}"},
            goto="agent"
        )
```

js

```
import { Command, GraphNode } from "@langchain/langgraph";

const executeTool: GraphNode<typeof State> = async (state, config) => {
    try {
        const result = await runTool(state.toolCall);
        return new Command({
            update: { toolResult: result },
            goto: "agent",
        });
    } catch (error) {
        // Let the LLM see what went wrong and try again
        return new Command({
            update: { toolResult: `Tool error: ${error}` },
            goto: "agent"
        });
    }
}
```

User-fixable

Pause and collect information from the user when needed (like account IDs, order numbers, or clarifications):

python

```
from langgraph.types import Command

def lookup_customer_history(
    state: State
) -> Command[Literal["lookup_customer_history", "draft_response"]]:
```

```

    if not state.get('customer_id'):
        user_input = interrupt({
            "message": "Customer ID needed",
            "request": "Please provide the customer's account ID to look up their subscrip
on history"
        })
        return Command(
            update={"customer_id": user_input['customer_id']},
            goto="lookup_customer_history"
        )
    # Now proceed with the lookup
    customer_data = fetch_customer_history(state['customer_id'])
    return Command(update={"customer_history": customer_data}, goto="draft_response")

```

js

```

import { Command, GraphNode, interrupt } from "@langchain/langgraph";

const lookupCustomerHistory: GraphNode<typeof State> = async (state, config) => {
    if (!state.customerId) {
        const userInput = interrupt({
            message: "Customer ID needed",
            request: "Please provide the customer's account ID to look up their subscription histo
ry",
        });
        return new Command({
            update: { customerId: userInput.customerId },
            goto: "lookupCustomerHistory",
        });
    }
    // Now proceed with the lookup
    const customerData = await fetchCustomerHistory(state.customerId);
    return new Command({
        update: { customerHistory: customerData },
        goto: "draftResponse",
    });
}

```

Unexpected

Let them bubble up for debugging. Don't catch what you can't handle:

python

```

def send_reply(state: EmailAgentState):
    try:
        email_service.send(state["draft_response"])

```

```
except Exception:
    raise # Surface unexpected errors
```

js

```
import { Command, GraphNode } from "@langchain/langgraph";

const sendReply: GraphNode<typeof EmailAgentState> = async (state, config) => {
  try {
    await emailService.send(state.responseText);
  } catch (error) {
    throw error; // Surface unexpected errors
  }
}
```

Saga / compensation

After retries are exhausted, run a recovery function that updates state and routes to a compensation branch.

python

See [Fault tolerance](#) for the full pattern.

참고

`error_handler` requires `langgraph>=1.2`.

```
from langgraph.errors import NodeError
from langgraph.types import Command, RetryPolicy

def payment_error_handler(state: State, error: NodeError) -> Command:
    return Command(
        update={"status": f"compensated: {error.error}"},
        goto="finalize",
    )

workflow.add_node(
    "charge_payment",
    charge_payment,
    retry_policy=RetryPolicy(max_attempts=3, retry_on=ConnectionError),
    error_handler=payment_error_handler,
)
```

To apply the same `retry_policy`, `timeout`, or `error_handler` to every node in a graph without repeating them on each `add_node`, use `StateGraph.set_node_defaults(...)`. Per-node values still

take precedence. See [Fault tolerance](#).

Implementing our email agent nodes

We'll implement each node as a simple function. Remember: nodes take state, do work, and return updates.

Read and classify nodes

python

```
from typing import Literal
from langgraph.graph import StateGraph, START, END
from langgraph.types import interrupt, Command, RetryPolicy
from langchain_openai import ChatOpenAI
from langchain.messages import HumanMessage

llm = ChatOpenAI(model="gpt-5-nano")

def read_email(state: EmailAgentState) -> dict:
    """Extract and parse email content"""
    # In production, this would connect to your email service
    return {
        "messages": [HumanMessage(content=f"Processing email: {state['email_content']}")]
    }

def classify_intent(state: EmailAgentState) -> Command[Literal["search_documentation", "human_review", "draft_response", "bug_tracking"]]:
    """Use LLM to classify email intent and urgency, then route accordingly"""

    # Create structured LLM that returns EmailClassification dict
    structured_llm = llm.with_structured_output(EmailClassification)

    # Format the prompt on-demand, not stored in state
    classification_prompt = f"""
    Analyze this customer email and classify it:

    Email: {state['email_content']}
    From: {state['sender_email']}

    Provide classification including intent, urgency, topic, and summary.
    """

    # Get structured response directly as dict
    classification = structured_llm.invoke(classification_prompt)

    # Determine next node based on classification
    if classification['intent'] == 'billing' or classification['urgency'] == 'critical':
        goto = "human_review"
```

```

elif classification['intent'] in ['question', 'feature']:
    goto = "search_documentation"
elif classification['intent'] == 'bug':
    goto = "bug_tracking"
else:
    goto = "draft_response"

# Store classification as a single dict in state
return Command(
    update={"classification": classification},
    goto=goto
)

```

js

```

import { StateGraph, START, END, GraphNode, Command } from "@langchain/langgraph";
import { HumanMessage } from "@langchain/core/messages";
import { ChatAnthropic } from "@langchain/anthropic";

const llm = new ChatAnthropic({ model: "claude-sonnet-4-6" });

const readEmail: GraphNode<typeof EmailAgentState> = async (state, config) => {
  // Extract and parse email content
  // In production, this would connect to your email service
  console.log(`Processing email: ${state.emailContent}`);
  return {};
}

const classifyIntent: GraphNode<typeof EmailAgentState> = async (state, config) => {
  // Use LLM to classify email intent and urgency, then route accordingly

  // Create structured LLM that returns EmailClassification object
  const structuredLlm = llm.withStructuredOutput(EmailClassificationSchema);

  // Format the prompt on-demand, not stored in state
  const classificationPrompt = `
Analyze this customer email and classify it:

Email: ${state.emailContent}
From: ${state.senderEmail}

Provide classification including intent, urgency, topic, and summary.
`;

  // Get structured response directly as object
  const classification = await structuredLlm.invoke(classificationPrompt);

  // Determine next node based on classification
  let nextNode: "searchDocumentation" | "humanReview" | "draftResponse" | "bugTracking";

```



```

if (classification.intent === "billing" || classification.urgency === "critical") {
  nextNode = "humanReview";
} else if (classification.intent === "question" || classification.intent === "feature") {
  nextNode = "searchDocumentation";
} else if (classification.intent === "bug") {
  nextNode = "bugTracking";
} else {
  nextNode = "draftResponse";
}

// Store classification as a single object in state
return new Command({
  update: { classification },
  goto: nextNode,
});
}

```

Search and tracking nodes

python

```

def search_documentation(state: EmailAgentState) -> Command[Literal["draft_response"]]:
    """Search knowledge base for relevant information"""

    # Build search query from classification
    classification = state.get('classification', {})
    query = f"{classification.get('intent', '')} {classification.get('topic', '')}"

    try:
        # Implement your search logic here
        # Store raw search results, not formatted text
        search_results = [
            "Reset password via Settings > Security > Change Password",
            "Password must be at least 12 characters",
            "Include uppercase, lowercase, numbers, and symbols"
        ]
    except SearchAPIError as e:
        # For recoverable search errors, store error and continue
        search_results = [f"Search temporarily unavailable: {str(e)}"]

    return Command(
        update={"search_results": search_results}, # Store raw results or error
        goto="draft_response"
    )

def bug_tracking(state: EmailAgentState) -> Command[Literal["draft_response"]]:
    """Create or update bug tracking ticket"""

```

```
# Create ticket in your bug tracking system
ticket_id = "BUG-12345" # Would be created via API

return Command(
    update={
        "search_results": [f"Bug ticket {ticket_id} created"],
        "current_step": "bug_tracked"
    },
    goto="draft_response"
)
```

js

```
import { Command, GraphNode } from "@langchain/langgraph";

const searchDocumentation: GraphNode<typeof EmailAgentState> = async (state, config) => {
    // Search knowledge base for relevant information

    // Build search query from classification
    const classification = state.classification!;
    const query = `${classification.intent} ${classification.topic}`;

    let searchResults: string[];

    try {
        // Implement your search logic here
        // Store raw search results, not formatted text
        searchResults = [
            "Reset password via Settings > Security > Change Password",
            "Password must be at least 12 characters",
            "Include uppercase, lowercase, numbers, and symbols",
        ];
    } catch (error) {
        // For recoverable search errors, store error and continue
        searchResults = [Search temporarily unavailable: ${error}];
    }

    return new Command({
        update: { searchResults }, // Store raw results or error
        goto: "draftResponse",
    });
}

const bugTracking: GraphNode<typeof EmailAgentState> = async (state, config) => {
    // Create or update bug tracking ticket

    // Create ticket in your bug tracking system
    const ticketId = "BUG-12345"; // Would be created via API
```

```

return new Command({
  update: { searchResults: [`Bug ticket ${ticketId} created`] },
  goto: "draftResponse",
});
}

```

Response nodes

python

```

def draft_response(state: EmailAgentState) -> Command[Literal["human_review", "send_reply"]]:
    """Generate response using context and route based on quality"""

    classification = state.get('classification', {})

    # Format context from raw state data on-demand
    context_sections = []

    if state.get('search_results'):
        # Format search results for the prompt
        formatted_docs = "\n".join([f"- {doc}" for doc in state['search_results']])
        context_sections.append(f"Relevant documentation:\n{formatted_docs}")

    if state.get('customer_history'):
        # Format customer data for the prompt
        context_sections.append(f"Customer tier: {state['customer_history'].get('tier', 'standard')}")

    # Build the prompt with formatted context
    draft_prompt = f"""
    Draft a response to this customer email:
    {state['email_content']}

    Email intent: {classification.get('intent', 'unknown')}
    Urgency level: {classification.get('urgency', 'medium')}

    {chr(10).join(context_sections)}

    Guidelines:
    - Be professional and helpful
    - Address their specific concern
    - Use the provided documentation when relevant
    """

    response = llm.invoke(draft_prompt)

    # Determine if human review needed based on urgency and intent
    needs_review = (

```

```

        classification.get('urgency') in ['high', 'critical'] or
        classification.get('intent') == 'complex'
    )

    # Route to appropriate next node
    goto = "human_review" if needs_review else "send_reply"

    return Command(
        update={"draft_response": response.content}, # Store only the raw response
        goto=goto
    )

def human_review(state: EmailAgentState) -> Command[Literal["send_reply", END]]:
    """Pause for human review using interrupt and route based on decision"""

    classification = state.get('classification', {})

    # interrupt() must come first - any code before it will re-run on resume
    human_decision = interrupt({
        "email_id": state.get('email_id', ''),
        "original_email": state.get('email_content', ''),
        "draft_response": state.get('draft_response', ''),
        "urgency": classification.get('urgency'),
        "intent": classification.get('intent'),
        "action": "Please review and approve/edit this response"
    })

    # Now process the human's decision
    if human_decision.get("approved"):
        return Command(
            update={"draft_response": human_decision.get("edited_response", state.get('draft_response', ''))},
            goto="send_reply"
        )
    else:
        # Rejection means human will handle directly
        return Command(update={}, goto=END)

def send_reply(state: EmailAgentState) -> dict:
    """Send the email response"""
    # Integrate with email service
    print(f"Sending reply: {state['draft_response'][:100]}...")
    return {}

```

js

```

import { Command, interrupt } from "@langchain/langgraph";

const draftResponse: GraphNode<typeof EmailAgentState> = async (state, config) => {

```

```

// Generate response using context and route based on quality

const classification = state.classification!;

// Format context from raw state data on-demand
const contextSections: string[] = [];

if (state.searchResults) {
  // Format search results for the prompt
  const formattedDocs = state.searchResults.map(doc => `- ${doc}`).join("\n");
  contextSections.push(`Relevant documentation:\n${formattedDocs}`);
}

if (state.customerHistory) {
  // Format customer data for the prompt
  contextSections.push(`Customer tier: ${state.customerHistory.tier ?? "standard"}`);
}

// Build the prompt with formatted context
const draftPrompt = `
Draft a response to this customer email:
${state.emailContent}

Email intent: ${classification.intent}
Urgency level: ${classification.urgency}

${contextSections.join("\n\n")}

Guidelines:
- Be professional and helpful
- Address their specific concern
- Use the provided documentation when relevant
`;

const response = await llm.invoke([new HumanMessage(draftPrompt)]);

// Determine if human review needed based on urgency and intent
const needsReview = (
  classification.urgency === "high" ||
  classification.urgency === "critical" ||
  classification.intent === "complex"
);

// Route to appropriate next node
const nextNode = needsReview ? "humanReview" : "sendReply";

return new Command({
  update: { responseText: response.content.toString() }, // Store only the raw response
  goto: nextNode,
});
}

```

```

const humanReview: GraphNode<typeof EmailAgentState> = async (state, config) => {
  // Pause for human review using interrupt and route based on decision
  const classification = state.classification!;

  // interrupt() must come first - any code before it will re-run on resume
  const humanDecision = interrupt({
    emailId: state.emailId,
    originalEmail: state.emailContent,
    draftResponse: state.responseText,
    urgency: classification.urgency,
    intent: classification.intent,
    action: "Please review and approve/edit this response",
  });

  // Now process the human's decision
  if (humanDecision.approved) {
    return new Command({
      update: { responseText: humanDecision.editedResponse || state.responseText },
      goto: "sendReply",
    });
  } else {
    // Rejection means human will handle directly
    return new Command({ update: {}, goto: END });
  }
}

const sendReply: GraphNode<typeof EmailAgentState> = async (state, config) => {
  // Send the email response
  // Integrate with email service
  console.log(`Sending reply: ${state.responseText!.substring(0, 100)}...`);
  return {};
}

```

Step 5: Wire it together

Now we connect our nodes into a working graph. Since our nodes handle their own routing decisions, we only need a few essential edges.

To enable [human-in-the-loop](#) with `interrupt()`, we need to compile with a [checkpoint](#) to save state between runs:

Graph compilation code

python

```

from langgraph.checkpoint.memory import MemorySaver
from langgraph.types import RetryPolicy

```

```

# Create the graph
workflow = StateGraph>EmailAgentState)

# Add nodes with appropriate error handling
workflow.add_node("read_email", read_email)
workflow.add_node("classify_intent", classify_intent)

# Add retry policy for nodes that might have transient failures
workflow.add_node(
    "search_documentation",
    search_documentation,
    retry_policy=RetryPolicy(max_attempts=3)
)
workflow.add_node("bug_tracking", bug_tracking)
workflow.add_node("draft_response", draft_response)
workflow.add_node("human_review", human_review)
workflow.add_node("send_reply", send_reply)

# Add only the essential edges
workflow.add_edge(START, "read_email")
workflow.add_edge("read_email", "classify_intent")
workflow.add_edge("send_reply", END)

# Compile with checkpointer for persistence, in case run graph with Local_Server --> Please
compile without checkpointer
memory = MemorySaver()
app = workflow.compile(checkpointer=memory)

```

js

```

import { MemorySaver, RetryPolicy } from "@langchain/langgraph";

// Create the graph
const workflow = new StateGraph>EmailAgentState)
  // Add nodes with appropriate error handling
  .addNode("readEmail", readEmail)
  .addNode("classifyIntent", classifyIntent)
  // Add retry policy for nodes that might have transient failures
  .addNode(
    "searchDocumentation",
    searchDocumentation,
    { retryPolicy: { maxAttempts: 3 } },
  )
  .addNode("bugTracking", bugTracking)
  .addNode("draftResponse", draftResponse)
  .addNode("humanReview", humanReview)
  .addNode("sendReply", sendReply)
  // Add only the essential edges

```

```

.addEdge(START, "readEmail")
.addEdge("readEmail", "classifyIntent")
.addEdge("sendReply", END);

// Compile with checkpointer for persistence
const memory = new MemorySaver();
const app = workflow.compile({ checkpointer: memory });

```

python

The graph structure is minimal because routing happens inside nodes through `@[Command]` objects. Each node declares where it can go using type hints like `Command[Literal["node1", "node2"]]`, making the flow explicit and traceable.

js

The graph structure is minimal because routing happens inside nodes through `Command` objects. Each node declares where it can go, making the flow explicit and traceable.

Try out your agent

Let's run our agent with an urgent billing issue that needs human review:

Testing the agent

python

js

```

// Test with an urgent billing issue
const initialState: EmailAgentStateType = {
  emailContent: "I was charged twice for my subscription! This is urgent!",
  senderEmail: "customer@example.com",
  emailId: "email_123"
};

// Run with a thread_id for persistence
const config = { configurable: { thread_id: "customer_123" } };
const result = await app.invoke(initialState, config);
// The graph will pause at human_review
console.log(`Draft ready for review: ${result.responseText?.substring(0, 100)}...`);

```

```

import { Command } from "@langchain/langgraph";

// When ready, provide human input to resume

```



```
const humanResponse = new Command({
  resume: {
    approved: true,
    editedResponse: "We sincerely apologize for the double charge. I've initiated an immediate refund...",
  }
});

// Resume execution
const finalResult = await app.invoke(humanResponse, config);
console.log("Email sent successfully!");
```

The graph pauses when it hits `interrupt()`, saves everything to the checkpoint, and waits. It can resume days later, picking up exactly where it left off. The `thread_id` ensures all state for this conversation is preserved together.

Summary and next steps

Key Insights

Building this email agent has shown us the LangGraph way of thinking:

[Break into discrete steps](#)

Each node does one thing well. This decomposition enables streaming progress updates, durable execution that can pause and resume, and clear debugging since you can inspect state between steps.

[State is shared memory](#)

Store raw data, not formatted text. This lets different nodes use the same information in different ways.

[Nodes are functions](#)

They take state, do work, and return updates. When they need to make routing decisions, they specify both the state updates and the next destination.

[Errors are part of the flow](#)

Transient failures get retries, LLM-recoverable errors loop back with context, user-fixable problems pause for input, and unexpected errors bubble up for debugging.

[Human input is first-class](#)

The `interrupt()` function pauses execution indefinitely, saves all state, and resumes exactly where it left off when you provide input. When combined with other operations in a node, it must come first.

[Graph structure emerges naturally](#)

You define the essential connections, and your nodes handle their own routing logic. This keeps control flow explicit and traceable - you can always understand what your agent will do next by looking at the current node.

Advanced considerations

Node granularity trade-offs

정보

This section explores the trade-offs in node granularity design. Most applications can skip this and use the patterns shown above.

You might wonder: why not combine `Read Email` and `Classify Intent` into one node?

Or why separate `Doc Search` from `Draft Reply`?

The answer involves trade-offs between resilience and observability.

The resilience consideration: LangGraph's [persistence layer](#) creates checkpoints at node boundaries. When a workflow resumes after an interruption or failure, it starts from the beginning of the node where execution stopped. Smaller nodes mean more frequent checkpoints, which means less work to repeat if something goes wrong. If you combine multiple operations into one large node, a failure near the end means re-executing everything from the start of that node.

Why we chose this breakdown for the email agent:

- **Isolation of external services:** `Doc Search` and `Bug Track` are separate nodes because they call external APIs. If the search service is slow or fails, we want to isolate that from the LLM calls. We can add retry policies to these specific nodes without affecting others.
- **Intermediate visibility:** Having `Classify Intent` as its own node lets us inspect what the LLM decided before taking action. This is valuable for debugging and monitoring—you can see exactly when and why the agent routes to human review.
- **Different failure modes:** LLM calls, database lookups, and email sending have different retry strategies. Separate nodes let you configure these independently.
- **Reusability and testing:** Smaller nodes are easier to test in isolation and reuse in other workflows.

A different valid approach: You could combine `Read Email` and `Classify Intent` into a single node. You'd lose the ability to inspect the raw email before classification and would repeat both operations on any failure in that node. For most applications, the observability and debugging benefits of separate nodes are worth the trade-off.

Application-level concerns: The caching discussion in Step 2 (whether to cache search results) is an application-level decision, not a LangGraph framework feature. You implement caching within your node functions based on your specific requirements—LangGraph doesn't prescribe this.

Performance considerations: More nodes doesn't mean slower execution. LangGraph writes checkpoints in the background by default ([async durability mode](#)), so your graph continues running without waiting for checkpoints to complete. This means you get frequent checkpoints with minimal performance impact. You can adjust this behavior if needed—use `"exit"` mode to checkpoint only at completion, or `"sync"` mode to block execution until each checkpoint is written.

Where to go from here

This was an introduction to thinking about building agents with LangGraph. You can extend this foundation with:

[Human-in-the-loop patterns](#)

Learn how to add tool approval before execution, batch approval, and other patterns

[Subgraphs](#)

Create subgraphs for complex multi-step operations

[Streaming](#)

Add streaming to show real-time progress to users

[Observability](#)

Add observability with LangSmith for debugging and monitoring

[Tool Integration](#)

Integrate more tools for web search, database queries, and API calls

[Retry Logic](#)

Implement retry logic with exponential backoff for failed operations

Agent Chat UI

langgraph/ui.mdx

Connect to your agent

Agent Chat UI can connect to both [local](#) and [deployed agents](#).

After starting Agent Chat UI, you'll need to configure it to connect to your agent:

1. **Graph ID:** Enter your graph name (find this under `graphs` in your `langgraph.json` file)
2. **Deployment URL:** Your Agent server's endpoint (e.g., `http://localhost:2024` for local development, or your deployed agent's URL)
3. **LangSmith API key (optional):** Add your LangSmith API key (not required if you're using a local Agent server)

Once configured, Agent Chat UI will automatically fetch and display any interrupted threads from your agent.

팁

Agent Chat UI has out-of-the-box support for rendering tool calls and tool result messages. To customize what messages are shown, see [Hiding Messages in the Chat](#).

Use the functional API

langgraph/use-functional-api.mdx

The **Functional API** allows you to add LangGraph's key features ([persistence](#), [memory](#), [human-in-the-loop](#), and [streaming](#)) to your applications with minimal changes to your existing code.

팁

For conceptual information on the functional API, see [Functional API](#).

Creating a simple workflow

When defining an `entrypoint`, input is restricted to the first argument of the function. To pass multiple inputs, you can use a dictionary.

python

```
@entrypoint(checkpointer=checkpointer)
def my_workflow(inputs: dict) -> int:
    value = inputs["value"]
    another_value = inputs["another_value"]
    ...

my_workflow.invoke({"value": 1, "another_value": 2})
```

js

```
const checkpointer = new MemorySaver();

const myWorkflow = entrypoint(
  { checkpointer, name: "myWorkflow" },
  async (inputs: { value: number; anotherValue: number }) => {
    const value = inputs.value;
    const anotherValue = inputs.anotherValue;
    // ...
  }
);

await myWorkflow.invoke({ value: 1, anotherValue: 2 });
```

Extended example: simple workflow

python

```
from langchain_core.utils.uuid import uuid7
from langgraph.func import entrypoint, task
from langgraph.checkpoint.memory import InMemorySaver

# Task that checks if a number is even
@task
def is_even(number: int) -> bool:
    return number % 2 == 0

# Task that formats a message
@task
def format_message(is_even: bool) -> str:
    return "The number is even." if is_even else "The number is odd."

# Create a checkpointer for persistence
checkerpoint = InMemorySaver()

@entrypoint(checkpointer=checkerpoint)
def workflow(inputs: dict) -> str:
    """Simple workflow to classify a number."""
    even = is_even(inputs["number"]).result()
    return format_message(even).result()

# Run the workflow with a unique thread ID
config = {"configurable": {"thread_id": str(uuid7())}}
result = workflow.invoke({"number": 7}, config=config)
print(result)
```

js

```
import { v7 as uuid7 } from "uuid";
import { entrypoint, task, MemorySaver } from "@langchain/langgraph";

// Task that checks if a number is even
const isEven = task("isEven", async (number: number) => {
    return number % 2 === 0;
});

// Task that formats a message
const formatMessage = task("formatMessage", async (isEven: boolean) => {
    return isEven ? "The number is even." : "The number is odd.";
});

// Create a checkpointer for persistence
const checkerpoint = new MemorySaver();
```

```

const workflow = entrypoint(
  { checkpointer, name: "workflow" },
  async (inputs: { number: number }) => {
    // Simple workflow to classify a number
    const even = await isEven(inputs.number);
    return await formatMessage(even);
  }
);

// Run the workflow with a unique thread ID
const config = { configurable: { thread_id: uuid7() } };
const result = await workflow.invoke({ number: 7 }, config);
console.log(result);

```

Extended example: Compose an essay with an LLM

This example demonstrates how to use the `@task` and `@entrypoint` decorators syntactically. Given that a checkpointer is provided, the workflow results will be persisted in the checkpointer.

python

```

import uuid
from langchain.chat_models import init_chat_model
from langgraph.func import entrypoint, task
from langgraph.checkpoint.memory import InMemorySaver

model = init_chat_model('gpt-3.5-turbo')

# Task: generate essay using an LLM
@task
def compose_essay(topic: str) -> str:
    """Generate an essay about the given topic."""
    return model.invoke([
        {"role": "system", "content": "You are a helpful assistant that writes essays."},
        {"role": "user", "content": f"Write an essay about {topic}."}
    ]).content

# Create a checkpointer for persistence
checkerpoint = InMemorySaver()

@entrypoint(checkpointer=checkerpoint)
def workflow(topic: str) -> str:
    """Simple workflow that generates an essay with an LLM."""
    return compose_essay(topic).result()

# Execute the workflow
config = {"configurable": {"thread_id": str(uuid7())}}

```

```
result = workflow.invoke("the history of flight", config=config)
print(result)
```

js

```
import { v7 as uuid7 } from "uuid";
import { ChatOpenAI } from "@langchain/openai";
import { entrypoint, task, MemorySaver } from "@langchain/langgraph";

const model = new ChatOpenAI({ model: "gpt-3.5-turbo" });

// Task: generate essay using an LLM
const composeEssay = task("composeEssay", async (topic: string) => {
  // Generate an essay about the given topic
  const response = await model.invoke([
    { role: "system", content: "You are a helpful assistant that writes essays." },
    { role: "user", content: `Write an essay about ${topic}.` },
  ]);
  return response.content as string;
});

// Create a checkpoint for persistence
const checkpoint = new MemorySaver();

const workflow = entrypoint(
  { checkpoint, name: "workflow" },
  async (topic: string) => {
    // Simple workflow that generates an essay with an LLM
    return await composeEssay(topic);
  }
);

// Execute the workflow
const config = { configurable: { thread_id: uuid7() } };
const result = await workflow.invoke("the history of flight", config);
console.log(result);
```

Parallel execution

Tasks can be executed in parallel by invoking them concurrently and waiting for the results. This is useful for improving performance in IO bound tasks (e.g., calling APIs for LLMs).

python

```
@task
def add_one(number: int) -> int:
```



```

    return number + 1

@entrypoint(checkpointer=checkpointer)
def graph(numbers: list[int]) -> list[str]:
    futures = [add_one(i) for i in numbers]
    return [f.result() for f in futures]

```

js

```

const addOne = task("addOne", async (number: number) => {
  return number + 1;
});

const graph = entrypoint(
  { checkpointer, name: "graph" },
  async (numbers: number[]) => {
    return await Promise.all(numbers.map(addOne));
  }
);

```

Extended example: parallel LLM calls

This example demonstrates how to run multiple LLM calls in parallel using `@task`. Each call generates a paragraph on a different topic, and results are joined into a single text output.

python

```

import uuid
from langchain.chat_models import init_chat_model
from langgraph.func import entrypoint, task
from langgraph.checkpoint.memory import InMemorySaver

# Initialize the LLM model
model = init_chat_model("gpt-3.5-turbo")

# Task that generates a paragraph about a given topic
@task
def generate_paragraph(topic: str) -> str:
    response = model.invoke([
        {"role": "system", "content": "You are a helpful assistant that writes educational paragraphs."},
        {"role": "user", "content": f"Write a paragraph about {topic}."}
    ])
    return response.content

# Create a checkpointer for persistence
checkpointer = InMemorySaver()

```

```

@entrypoint(checkpointer=checkpointer)
def workflow(topics: list[str]) -> str:
    """Generates multiple paragraphs in parallel and combines them."""
    futures = [generate_paragraph(topic) for topic in topics]
    paragraphs = [f.result() for f in futures]
    return "\n\n".join(paragraphs)

# Run the workflow
config = {"configurable": {"thread_id": str(uuid7())}}
result = workflow.invoke(["quantum computing", "climate change", "history of aviation"], config=config)
print(result)

```

js

```

import { v7 as uuid7 } from "uuid";
import { ChatOpenAI } from "@langchain/openai";
import { entrypoint, task, MemorySaver } from "@langchain/langgraph";

// Initialize the LLM model
const model = new ChatOpenAI({ model: "gpt-3.5-turbo" });

// Task that generates a paragraph about a given topic
const generateParagraph = task("generateParagraph", async (topic: string) => {
    const response = await model.invoke([
        { role: "system", content: "You are a helpful assistant that writes educational paragraphs." },
        { role: "user", content: `Write a paragraph about ${topic}.` }
    ]);
    return response.content as string;
});

// Create a checkpointer for persistence
const checkpointer = new MemorySaver();

const workflow = entrypoint(
    { checkpointer, name: "workflow" },
    async (topics: string[]) => {
        // Generates multiple paragraphs in parallel and combines them
        const paragraphs = await Promise.all(topics.map(generateParagraph));
        return paragraphs.join("\n\n");
    }
);

// Run the workflow
const config = { configurable: { thread_id: uuid7() } };
const result = await workflow.invoke(["quantum computing", "climate change", "history of avi

```

```
ation"], config);
console.log(result);
```

This example uses LangGraph's concurrency model to improve execution time, especially when tasks involve I/O like LLM completions.

Calling graphs

The **Functional API** and the [Graph API](#) can be used together in the same application as they share the same underlying runtime.

python

```
from langgraph.func import entrypoint
from langgraph.graph import StateGraph

builder = StateGraph()
...
some_graph = builder.compile()

@entrypoint()
def some_workflow(some_input: dict) -> int:
    # Call a graph defined using the graph API
    result_1 = some_graph.invoke(...)
    # Call another graph defined using the graph API
    result_2 = another_graph.invoke(...)
    return {
        "result_1": result_1,
        "result_2": result_2
    }
```

js

```
import { entrypoint } from "@langchain/langgraph";
import { StateGraph } from "@langchain/langgraph";

const builder = new StateGraph(/* ... */);
// ...
const someGraph = builder.compile();

const someWorkflow = entrypoint(
  { name: "someWorkflow" },
  async (someInput: Record<string, any>) => {
    // Call a graph defined using the graph API
    const result1 = await someGraph.invoke(/* ... */);
    // Call another graph defined using the graph API
```

```

    const result2 = await anotherGraph.invoke(/* ... */);
    return {
      result1,
      result2,
    };
  }
);

```

Extended example: calling a simple graph from the functional API

python

```

import uuid
from typing import TypedDict
from langgraph.func import entrypoint
from langgraph.checkpoint.memory import InMemorySaver
from langgraph.graph import StateGraph

# Define the shared state type
class State(TypedDict):
    foo: int

# Define a simple transformation node
def double(state: State) -> State:
    return {"foo": state["foo"] * 2}

# Build the graph using the Graph API
builder = StateGraph(State)
builder.add_node("double", double)
builder.set_entry_point("double")
graph = builder.compile()

# Define the functional API workflow
checkpointer = InMemorySaver()

@entrypoint(checkpointer=checkpointer)
def workflow(x: int) -> dict:
    result = graph.invoke({"foo": x})
    return {"bar": result["foo"]}

# Execute the workflow
config = {"configurable": {"thread_id": str(uuid7())}}
print(workflow.invoke(5, config=config)) # Output: {'bar': 10}

```

js

```

import { v7 as uuid7 } from "uuid";
import { entrypoint, MemorySaver, StateGraph, StateSchema } from "@langchain/langgraph";
import * as z from "zod";

// Define the shared state type
const State = new StateSchema({
  foo: z.number(),
});

// Build the graph using the Graph API
const builder = new StateGraph(State)
  .addNode("double", (state) => {
    return { foo: state.foo * 2 };
  })
  .addEdge("__start__", "double");
const graph = builder.compile();

// Define the functional API workflow
const checkpointer = new MemorySaver();

const workflow = entrypoint(
  { checkpointer, name: "workflow" },
  async (x: number) => {
    const result = await graph.invoke({ foo: x });
    return { bar: result.foo };
  }
);

// Execute the workflow
const config = { configurable: { thread_id: uuid7() } };
console.log(await workflow.invoke(5, config)); // Output: { bar: 10 }

```

Call other entrypoints

You can call other **entrypoints** from within an **entrypoint** or a **task**.

python

```

@entrypoint() # Will automatically use the checkpointer from the parent entrypoint
def some_other_workflow(inputs: dict) -> int:
    return inputs["value"]

@entrypoint(checkpointer=checkpointer)
def my_workflow(inputs: dict) -> int:
    value = some_other_workflow.invoke({"value": 1})
    return value

```

js

```
// Will automatically use the checkpointer from the parent entrypoint
const someOtherWorkflow = entrypoint(
  { name: "someOtherWorkflow" },
  async (inputs: { value: number }) => {
    return inputs.value;
  }
);

const myWorkflow = entrypoint(
  { checkpointer, name: "myWorkflow" },
  async (inputs: { value: number }) => {
    const value = await someOtherWorkflow.invoke({ value: 1 });
    return value;
  }
);
```

Extended example: calling another entrypoint

python

```
import uuid
from langgraph.func import entrypoint
from langgraph.checkpoint.memory import InMemorySaver

# Initialize a checkpointer
checkerpoint = InMemorySaver()

# A reusable sub-workflow that multiplies a number
@entrypoint()
def multiply(inputs: dict) -> int:
    return inputs["a"] * inputs["b"]

# Main workflow that invokes the sub-workflow
@entrypoint(checkpointer=checkerpoint)
def main(inputs: dict) -> dict:
    result = multiply.invoke({"a": inputs["x"], "b": inputs["y"]})
    return {"product": result}

# Execute the main workflow
config = {"configurable": {"thread_id": str(uuid7())}}
print(main.invoke({"x": 6, "y": 7}, config=config)) # Output: {'product': 42}
```

js

```

import { v7 as uuid7 } from "uuid";
import { entrypoint, MemorySaver } from "@langchain/langgraph";

// Initialize a checkpointer
const checkpointer = new MemorySaver();

// A reusable sub-workflow that multiplies a number
const multiply = entrypoint(
  { name: "multiply" },
  async (inputs: { a: number; b: number }) => {
    return inputs.a * inputs.b;
  }
);

// Main workflow that invokes the sub-workflow
const main = entrypoint(
  { checkpointer, name: "main" },
  async (inputs: { x: number; y: number }) => {
    const result = await multiply.invoke({ a: inputs.x, b: inputs.y });
    return { product: result };
  }
);

// Execute the main workflow
const config = { configurable: { thread_id: uuid7() } };
console.log(await main.invoke({ x: 6, y: 7 }, config)); // Output: { product: 42 }

```

Streaming

The **Functional API** uses the same streaming mechanism as the **Graph API**. Please read the [streaming guide](#) section for more details.

Example of using the streaming API to stream value chunks from a workflow run.

python

1. Import `@[get_stream_writer]` from `langgraph.config`.
2. Obtain a stream writer instance within the entrypoint.
3. Emit custom data before computation begins.
4. Emit another custom message after computing the result.
5. Use `stream_events()` to process streamed output.
6. Iterate over `(mode, chunk)` pairs from `interleave("values")`.

주의

Async with Python < 3.11 If using Python < 3.11 and writing async code, using `@[get_stream_writer]` will not work. Instead please use the `StreamWriter` class directly. See

[Async with Python < 3.11](#) for more details.

```
from langgraph.types import StreamWriter

@entrypoint(checkpointer=checkpointer)
async def main(inputs: dict, writer: StreamWriter) -> int: # [!code highlight]
    ...
```

js

1. Emit custom data before computation begins.
2. Emit another custom message after computing the result.
3. Use `streamEvents()` to process streamed output.
4. Iterate `stream.values` to receive value snapshots.

Retry policy

python

```
from langgraph.checkpoint.memory import InMemorySaver
from langgraph.func import entrypoint, task
from langgraph.types import RetryPolicy

# This variable is just used for demonstration purposes to simulate a network failure.
# It's not something you will have in your actual code.
attempts = 0

# Let's configure the RetryPolicy to retry on ValueError.
# The default RetryPolicy is optimized for retrying specific network errors.
retry_policy = RetryPolicy(retry_on=ValueError)

@task(retry_policy=retry_policy)
def get_info():
    global attempts
    attempts += 1

    if attempts < 2:
        raise ValueError('Failure')
    return "OK"

checkpointer = InMemorySaver()

@entrypoint(checkpointer=checkpointer)
def main(inputs, writer):
    return get_info().result()
```



```
config = {
  "configurable": {
    "thread_id": "1"
  }
}

main.invoke({'any_input': 'foobar'}, config=config)
```

```
'OK'
```

js

```
import {
  MemorySaver,
  entrypoint,
  task,
  RetryPolicy,
} from "@langchain/langgraph";

// This variable is just used for demonstration purposes to simulate a network failure.
// It's not something you will have in your actual code.
let attempts = 0;

// Let's configure the RetryPolicy to retry on ValueError.
// The default RetryPolicy is optimized for retrying specific network errors.
const retryPolicy: RetryPolicy = { retryOn: (error) => error instanceof Error };

const getInfo = task(
  {
    name: "getInfo",
    retry: retryPolicy,
  },
  () => {
    attempts += 1;

    if (attempts < 2) {
      throw new Error("Failure");
    }
    return "OK";
  }
);

const checkpointer = new MemorySaver();

const main = entrypoint(
  { checkpointer, name: "main" },
  async (inputs: Record<string, any>) => {
    return await getInfo();
  }
);
```

```

    }
);

const config = {
  configurable: {
    thread_id: "1",
  },
};

await main.invoke({ any_input: "foobar" }, config);

```

'OK'

python

Set task and entrypoint timeouts

Use the `timeout` parameter with `@task` or `@entrypoint` to limit how long a single async attempt can run. Provide the timeout in seconds or as a `datetime.timedelta`.

```

import asyncio

from langgraph.errors import NodeTimeoutError
from langgraph.func import entrypoint, task
from langgraph.types import RetryPolicy

@task(
    timeout=1.0,
    retry_policy=RetryPolicy(retry_on=NodeTimeoutError),
)
async def call_api(url: str) -> str:
    await asyncio.sleep(2)
    return f"result from {url}"

@entrypoint(timeout=5.0)
async def workflow(inputs: dict) -> str:
    return await call_api(inputs["url"])

try:
    await workflow.ainvoke({"url": "https://example.com"})
except NodeTimeoutError:
    print("Task timed out")

```

Timeouts are supported only for async tasks and entrypoints. If you set `timeout` on a sync function, LangGraph raises an error when the task or entrypoint is declared.

When a task or entrypoint exceeds its timeout, LangGraph raises `NodeTimeoutError`, which subclasses Python's built-in `TimeoutError`. If a retry policy retries `TimeoutError` or `NodeTimeoutError`, the timed-out attempt is retried. The timeout applies to each attempt independently, so the timer resets for every retry.

Caching tasks

python

```
import time
from langgraph.cache.memory import InMemoryCache
from langgraph.func import entrypoint, task
from langgraph.types import CachePolicy

@task(cache_policy=CachePolicy(ttl=120))    # [!code highlight]
def slow_add(x: int) -> int:
    time.sleep(1)
    return x * 2

@entrypoint(cache=InMemoryCache())
def main(inputs: dict) -> dict[str, int]:
    result1 = slow_add(inputs["x"]).result()
    result2 = slow_add(inputs["x"]).result()
    return {"result1": result1, "result2": result2}

stream = main.stream_events({"x": 5}, version="v3")
for snapshot in stream.values:
    print(snapshot)
```

1. `ttl` is specified in seconds. The cache will be invalidated after this time.

js

```
import {
  InMemoryCache,
  entrypoint,
  task,
  CachePolicy,
} from "@langchain/langgraph";

const slowAdd = task(
  {
    name: "slowAdd",
    cache: { ttl: 120 },    // [!code highlight]
  },
  async (x: number) => {
```

```

    await new Promise((resolve) => setTimeout(resolve, 1000));
    return x * 2;
  }
);

const main = entrypoint(
  { cache: new InMemoryCache(), name: "main" },
  async (inputs: { x: number }) => {
    const result1 = await slowAdd(inputs.x);
    const result2 = await slowAdd(inputs.x);
    return { result1, result2 };
  }
);

const stream = await main.streamEvents({ x: 5 }, { version: "v3" });
for await (const snapshot of stream.values) {
  console.log(snapshot);
}

```

1. `ttl` is specified in seconds. The cache will be invalidated after this time.

Resuming after an error

python

```

import time
from langgraph.checkpoint.memory import InMemorySaver
from langgraph.func import entrypoint, task
from langgraph.types import StreamWriter

# This variable is just used for demonstration purposes to simulate a network failure.
# It's not something you will have in your actual code.
attempts = 0

@task()
def get_info():
    """
    Simulates a task that fails once before succeeding.
    Raises an exception on the first attempt, then returns "OK" on subsequent tries.
    """
    global attempts
    attempts += 1

    if attempts < 2:
        raise ValueError("Failure") # Simulate a failure on the first attempt
    return "OK"

# Initialize an in-memory checkpointer for persistence

```

```

checkpointer = InMemorySaver()

@task
def slow_task():
    """
    Simulates a slow-running task by introducing a 1-second delay.
    """
    time.sleep(1)
    return "Ran slow task."

@entrypoint(checkpointer=checkpointer)
def main(inputs, writer: StreamWriter):
    """
    Main workflow function that runs the slow_task and get_info tasks sequentially.

    Parameters:
    - inputs: Dictionary containing workflow input values.
    - writer: StreamWriter for streaming custom data.

    The workflow first executes `slow_task` and then attempts to execute `get_info`,
    which will fail on the first invocation.
    """
    slow_task_result = slow_task().result() # Blocking call to slow_task
    get_info().result() # Exception will be raised here on the first attempt
    return slow_task_result

# Workflow execution configuration with a unique thread identifier
config = {
    "configurable": {
        "thread_id": "1" # Unique identifier to track workflow execution
    }
}

# This invocation will take ~1 second due to the slow_task execution
try:
    # First invocation will raise an exception due to the `get_info` task failing
    main.invoke({'any_input': 'foobar'}, config=config)
except ValueError:
    pass # Handle the failure gracefully

```

When we resume execution, we won't need to re-run the `slow_task` as its result is already saved in the checkpoint.

```
main.invoke(None, config=config)
```

```
'Ran slow task.'
```

```

import { entrypoint, task, MemorySaver } from "@langchain/langgraph";

// This variable is just used for demonstration purposes to simulate a network failure.
// It's not something you will have in your actual code.
let attempts = 0;

const getInfo = task("getInfo", async () => {
  /**
   * Simulates a task that fails once before succeeding.
   * Throws an exception on the first attempt, then returns "OK" on subsequent tries.
   */
  attempts += 1;

  if (attempts < 2) {
    throw new Error("Failure"); // Simulate a failure on the first attempt
  }
  return "OK";
});

// Initialize an in-memory checkpointer for persistence
const checkpointer = new MemorySaver();

const slowTask = task("slowTask", async () => {
  /**
   * Simulates a slow-running task by introducing a 1-second delay.
   */
  await new Promise((resolve) => setTimeout(resolve, 1000));
  return "Ran slow task.";
});

const main = entrypoint(
  { checkpointer, name: "main" },
  async (inputs: Record<string, any>) => {
    /**
     * Main workflow function that runs the slowTask and getInfo tasks sequentially.
     *
     * Parameters:
     * - inputs: Record<string, any> containing workflow input values.
     *
     * The workflow first executes `slowTask` and then attempts to execute `getInfo`,
     * which will fail on the first invocation.
     */
    const slowTaskResult = await slowTask(); // Blocking call to slowTask
    await getInfo(); // Exception will be raised here on the first attempt
    return slowTaskResult;
  }
);

// Workflow execution configuration with a unique thread identifier
const config = {

```

```

configurable: {
  thread_id: "1", // Unique identifier to track workflow execution
},
};

// This invocation will take ~1 second due to the slowTask execution
try {
  // First invocation will raise an exception due to the `getInfo` task failing
  await main.invoke({ any_input: "foobar" }, config);
} catch (err) {
  // Handle the failure gracefully
}

```

When we resume execution, we won't need to re-run the `slowTask` as its result is already saved in the checkpoint.

```
await main.invoke(null, config);
```

```
'Ran slow task.'
```

Human-in-the-loop

The functional API supports [human-in-the-loop](#) workflows using the `@[interrupt]` function and the `Command` primitive.

Basic human-in-the-loop workflow

We will create three [tasks](#):

1. Append `"bar"`.
2. Pause for human input. When resuming, append human input.
3. Append `"qux"`.

python

```

from langgraph.func import entrypoint, task
from langgraph.types import Command, interrupt

@task
def step_1(input_query):
    """Append bar."""
    return f"{input_query} bar"

@task
def human_feedback(input_query):
    """Append user input."""

```

```
feedback = interrupt(f"Please provide feedback: {input_query}")
return f"{input_query} {feedback}"
```

```
@task
def step_3(input_query):
    """Append qux."""
    return f"{input_query} qux"
```

js

```
import { entrypoint, task, interrupt, Command } from "@langchain/langgraph";

const step1 = task("step1", async (inputQuery: string) => {
  // Append bar
  return `${inputQuery} bar`;
});

const humanFeedback = task("humanFeedback", async (inputQuery: string) => {
  // Append user input
  const feedback = interrupt(`Please provide feedback: ${inputQuery}`);
  return `${inputQuery} ${feedback}`;
});

const step3 = task("step3", async (inputQuery: string) => {
  // Append qux
  return `${inputQuery} qux`;
});
```

We can now compose these tasks in an [entrypoint](#):

python

```
from langgraph.checkpoint.memory import InMemorySaver

checkpointer = InMemorySaver()

@entrypoint(checkpointer=checkpointer)
def graph(input_query):
    result_1 = step_1(input_query).result()
    result_2 = human_feedback(result_1).result()
    result_3 = step_3(result_2).result()

    return result_3
```

js


```
import { MemorySaver } from "@langchain/langgraph";

const checkpointer = new MemorySaver();

const graph = entrypoint(
  { checkpointer, name: "graph" },
  async (inputQuery: string) => {
    const result1 = await step1(inputQuery);
    const result2 = await humanFeedback(result1);
    const result3 = await step3(result2);

    return result3;
  }
);
```

[interrupt\(\)](#) is called inside a task, enabling a human to review and edit the output of the previous task. The results of prior tasks-- in this case `step_1` -- are persisted, so that they are not run again following the `@[interrupt]`.

Let's send in a query string:

python

```
config = {"configurable": {"thread_id": "1"}}

stream = graph.stream_events("foo", config, version="v3")
for message in stream.messages:
    for token in message.text:
        print(token, end="", flush=True)
```

js

```
const config = { configurable: { thread_id: "1" } };

const stream = await graph.streamEvents("foo", config, { version: "v3" });
for await (const message of stream.messages) {
  for await (const token of message.text) {
    process.stdout.write(token);
  }
}
```

Note that we've paused with an `@[interrupt]` after `step_1`. The interrupt provides instructions to resume the run. To resume, we issue a [Command](#) containing the data expected by the `human_feedback` task.

python

```
# Continue execution
stream = graph.stream_events(Command(resume="baz"), config, version="v3")
for message in stream.messages:
    for token in message.text:
        print(token, end="", flush=True)
```

js

```
// Continue execution
const stream = await graph.streamEvents(
  new Command({ resume: "baz" }),
  config,
  { version: "v3" }
);
for await (const message of stream.messages) {
  for await (const token of message.text) {
    process.stdout.write(token);
  }
}
```

After resuming, the run proceeds through the remaining step and terminates as expected.

Review tool calls

To review tool calls before execution, we add a `review_tool_call` function that calls `interrupt`. When this function is called, execution will be paused until we issue a command to resume it.

Given a tool call, our function will `@[interrupt]` for human review. At that point we can either:

- Accept the tool call
- Revise the tool call and continue
- Generate a custom tool message (e.g., instructing the model to re-format its tool call)

python

```
from typing import Union

def review_tool_call(tool_call: ToolCall) -> Union[ToolCall, ToolMessage]:
    """Review a tool call, returning a validated version."""
    human_review = interrupt(
        {
            "question": "Is this correct?",
            "tool_call": tool_call,
        }
    )
```

```

)
review_action = human_review["action"]
review_data = human_review.get("data")
if review_action == "continue":
    return tool_call
elif review_action == "update":
    updated_tool_call = {**tool_call, **{"args": review_data}}
    return updated_tool_call
elif review_action == "feedback":
    return ToolMessage(
        content=review_data, name=tool_call["name"], tool_call_id=tool_call["id"]
    )

```

js

```

import { ToolCall } from "@langchain/core/messages/tool";
import { ToolMessage } from "@langchain/core/messages";

function reviewToolCall(toolCall: ToolCall): ToolCall | ToolMessage {
    // Review a tool call, returning a validated version
    const humanReview = interrupt({
        question: "Is this correct?",
        tool_call: toolCall,
    });

    const reviewAction = humanReview.action;
    const reviewData = humanReview.data;

    if (reviewAction === "continue") {
        return toolCall;
    } else if (reviewAction === "update") {
        const updatedToolCall = { ...toolCall, args: reviewData };
        return updatedToolCall;
    } else if (reviewAction === "feedback") {
        return new ToolMessage({
            content: reviewData,
            name: toolCall.name,
            tool_call_id: toolCall.id,
        });
    }

    throw new Error(`Unknown review action: ${reviewAction}`);
}

```

We can now update our [entrypoint](#) to review the generated tool calls. If a tool call is accepted or revised, we execute in the same way as before. Otherwise, we just append the @[ToolMessage]

supplied by the human. The results of prior tasks—in this case the initial model call—are persisted, so that they are not run again following the `@interrupt`.

python

```
from langgraph.checkpoint.memory import InMemorySaver
from langgraph.graph.message import add_messages
from langgraph.types import Command, interrupt

checkpointer = InMemorySaver()

@entrypoint(checkpointer=checkpointer)
def agent(messages, previous):
    if previous is not None:
        messages = add_messages(previous, messages)

    model_response = call_model(messages).result()
    while True:
        if not model_response.tool_calls:
            break

        # Review tool calls
        tool_results = []
        tool_calls = []
        for i, tool_call in enumerate(model_response.tool_calls):
            review = review_tool_call(tool_call)
            if isinstance(review, ToolMessage):
                tool_results.append(review)
            else: # is a validated tool call
                tool_calls.append(review)
            if review != tool_call:
                model_response.tool_calls[i] = review # update message

        # Execute remaining tool calls
        tool_result_futures = [call_tool(tool_call) for tool_call in tool_calls]
        remaining_tool_results = [fut.result() for fut in tool_result_futures]

        # Append to message list
        messages = add_messages(
            messages,
            [model_response, *tool_results, *remaining_tool_results],
        )

        # Call model again
        model_response = call_model(messages).result()

        # Generate final response
```

```
messages = add_messages(messages, model_response)
return entrypoint.final(value=model_response, save=messages)
```

js

```
import {
  MemorySaver,
  entrypoint,
  interrupt,
  Command,
  addMessages,
} from "@langchain/langgraph";
import { ToolMessage, AIMessage, BaseMessage } from "@langchain/core/messages";

const checkpointer = new MemorySaver();

const agent = entrypoint(
  { checkpointer, name: "agent" },
  async (
    messages: BaseMessage[],
    previous?: BaseMessage[]
  ): Promise<BaseMessage> => {
    if (previous !== undefined) {
      messages = addMessages(previous, messages);
    }

    let modelResponse = await callModel(messages);
    while (true) {
      if (!modelResponse.tool_calls?.length) {
        break;
      }

      // Review tool calls
      const toolResults: ToolMessage[] = [];
      const toolCalls: ToolCall[] = [];

      for (let i = 0; i < modelResponse.tool_calls.length; i++) {
        const review = reviewToolCall(modelResponse.tool_calls[i]);
        if (review instanceof ToolMessage) {
          toolResults.push(review);
        } else {
          // is a validated tool call
          toolCalls.push(review);
          if (review !== modelResponse.tool_calls[i]) {
            modelResponse.tool_calls[i] = review; // update message
          }
        }
      }
    }
  }
}
```

```

// Execute remaining tool calls
const remainingToolResults = await Promise.all(
  toolCalls.map((toolCall) => callTool(toolCall))
);

// Append to message list
messages = addMessages(messages, [
  modelResponse,
  ...toolResults,
  ...remainingToolResults,
]);

// Call model again
modelResponse = await callModel(messages);
}

// Generate final response
messages = addMessages(messages, modelResponse);
return entrypoint.final({ value: modelResponse, save: messages });
}
);

```

Short-term memory

Short-term memory allows storing information across different **invocations** of the same **thread id**. See [short-term memory](#) for more details.

Manage checkpoints

You can view and delete the information stored by the checkpointer.

View thread state

python

```

config = {
  "configurable": {
    "thread_id": "1", # [!code highlight]
    # optionally provide an ID for a specific checkpoint,
    # otherwise the latest checkpoint is shown
    # "checkpoint_id": "1f029ca3-1f5b-6704-8004-820c16b69a5a" # [!code highlight]
  }
}
graph.get_state(config) # [!code highlight]

```

```

StateSnapshot(
  values={ 'messages': [HumanMessage(content="hi! I'm bob"), AIMessage(content='Hi Bob! How
are you doing today?'), HumanMessage(content="what's my name?"), AIMessage(content='Your name
is Bob.')] }, next=(),
  config={ 'configurable': { 'thread_id': '1', 'checkpoint_ns': '', 'checkpoint_id': '1f029c
a3-1f5b-6704-8004-820c16b69a5a' } },
  metadata={
    'source': 'loop',
    'writes': { 'call_model': { 'messages': AIMessage(content='Your name is Bob.')} },
    'step': 4,
    'parents': {},
    'thread_id': '1'
  },
  created_at='2025-05-05T16:01:24.680462+00:00',
  parent_config={ 'configurable': { 'thread_id': '1', 'checkpoint_ns': '', 'checkpoint_id':
'1f029ca3-1790-6b0a-8003-baf965b6a38f' } },
  tasks=(),
  interrupts=()
)

```

js

```

const config = {
  configurable: {
    thread_id: "1", // [!code highlight]
    // optionally provide an ID for a specific checkpoint,
    // otherwise the latest checkpoint is shown
    // checkpoint_id: "1f029ca3-1f5b-6704-8004-820c16b69a5a" [!code highlight]
  },
};
await graph.getState(config); // [!code highlight]

```

```

StateSnapshot {
  values: {
    messages: [
      HumanMessage { content: "hi! I'm bob" },
      AIMessage { content: "Hi Bob! How are you doing today?" },
      HumanMessage { content: "what's my name?" },
      AIMessage { content: "Your name is Bob." }
    ]
  },
  next: [],
  config: { configurable: { thread_id: '1', checkpoint_ns: '', checkpoint_id: '1f029ca3-1f5b
-6704-8004-820c16b69a5a' } },
  metadata: {
    source: 'loop',
    writes: { call_model: { messages: AIMessage { content: "Your name is Bob." } } },
    step: 4,

```

```

    parents: {},
    thread_id: '1'
  },
  createdAt: '2025-05-05T16:01:24.680462+00:00',
  parentConfig: { configurable: { thread_id: '1', checkpoint_ns: '', checkpoint_id: '1f029ca3-1790-6b0a-8003-baf965b6a38f' } },
  tasks: [],
  interrupts: []
}

```

View the history of the thread

python

```

config = {
    "configurable": {
        "thread_id": "1" # [!code highlight]
    }
}
list(graph.get_state_history(config)) # [!code highlight]

```

```

[
  StateSnapshot(
    values={'messages': [HumanMessage(content="hi! I'm bob"), AIMessage(content='Hi Bob! How are you doing today? Is there anything I can help you with?'), HumanMessage(content="what's my name?"), AIMessage(content='Your name is Bob.')]},
    next=(),
    config={'configurable': {'thread_id': '1', 'checkpoint_ns': '', 'checkpoint_id': '1f029ca3-1f5b-6704-8004-820c16b69a5a'}},
    metadata={'source': 'loop', 'writes': {'call_model': {'messages': AIMessage(content='Your name is Bob.')}}, 'step': 4, 'parents': {}, 'thread_id': '1'},
    created_at='2025-05-05T16:01:24.680462+00:00',
    parent_config={'configurable': {'thread_id': '1', 'checkpoint_ns': '', 'checkpoint_id': '1f029ca3-1790-6b0a-8003-baf965b6a38f'}},
    tasks=(),
    interrupts=()
  ),
  StateSnapshot(
    values={'messages': [HumanMessage(content="hi! I'm bob"), AIMessage(content='Hi Bob! How are you doing today? Is there anything I can help you with?'), HumanMessage(content="what's my name?")]},
    next=('call_model',),
    config={'configurable': {'thread_id': '1', 'checkpoint_ns': '', 'checkpoint_id': '1f029ca3-1790-6b0a-8003-baf965b6a38f'}},
    metadata={'source': 'loop', 'writes': None, 'step': 3, 'parents': {}, 'thread_id': '1'},
    created_at='2025-05-05T16:01:23.863421+00:00',
    parent_config=...
  )
]

```



```

        tasks=(PregelTask(id='8ab4155e-6b15-b885-9ce5-bed69a2c305c', name='call_model', path=
('__pregel_pull', 'call_model'), error=None, interrupts=(), state=None, result={'messages':
AIMessage(content='Your name is Bob.')})),
        interrupts=()
    ),
    StateSnapshot(
        values={'messages': [HumanMessage(content="hi! I'm bob"), AIMessage(content='Hi Bob!
How are you doing today? Is there anything I can help you with?')]},
        next=('__start__',),
        config=...,
        metadata={'source': 'input', 'writes': {'__start__': {'messages': [{'role': 'user',
'content': "what's my name?"}]}}, 'step': 2, 'parents': {}, 'thread_id': '1'},
        created_at='2025-05-05T16:01:23.863173+00:00',
        parent_config=...
        tasks=(PregelTask(id='24ba39d6-6db1-4c9b-f4c5-682aeaf38dcd', name='__start__', path=
('__pregel_pull', '__start__'), error=None, interrupts=(), state=None, result={'messages':
[{'role': 'user', 'content': "what's my name?"}]})),
        interrupts=()
    ),
    StateSnapshot(
        values={'messages': [HumanMessage(content="hi! I'm bob"), AIMessage(content='Hi Bob!
How are you doing today? Is there anything I can help you with?')]},
        next=(),
        config=...,
        metadata={'source': 'loop', 'writes': {'call_model': {'messages': AIMessage(content
='Hi Bob! How are you doing today? Is there anything I can help you with?')}}, 'step': 1, 'p
arents': {}, 'thread_id': '1'},
        created_at='2025-05-05T16:01:23.862295+00:00',
        parent_config=...
        tasks=(),
        interrupts=()
    ),
    StateSnapshot(
        values={'messages': [HumanMessage(content="hi! I'm bob")]},
        next=('call_model',),
        config=...,
        metadata={'source': 'loop', 'writes': None, 'step': 0, 'parents': {}, 'thread_id':
'1'},
        created_at='2025-05-05T16:01:22.278960+00:00',
        parent_config=...
        tasks=(PregelTask(id='8cbd75e0-3720-b056-04f7-71ac805140a0', name='call_model', path=
('__pregel_pull', 'call_model'), error=None, interrupts=(), state=None, result={'messages':
AIMessage(content='Hi Bob! How are you doing today? Is there anything I can help you wit
h?')})),
        interrupts=()
    ),
    StateSnapshot(
        values={'messages': []},
        next=('__start__',),
        config={'configurable': {'thread_id': '1', 'checkpoint_ns': '', 'checkpoint_id': '1f
029ca3-0870-6ce2-bfff-1f3f14c3e565'}},

```

```

        metadata={'source': 'input', 'writes': {'__start__': {'messages': [{'role': 'user',
'content': "hi! I'm bob"}]}}, 'step': -1, 'parents': {}, 'thread_id': '1'},
        created_at='2025-05-05T16:01:22.277497+00:00',
        parent_config=None,
        tasks=(PregelTask(id='d458367b-8265-812c-18e2-33001d199ce6', name='__start__', path=
('__pregel_pull', '__start__'), error=None, interrupts=(), state=None, result={'messages':
[{'role': 'user', 'content': "hi! I'm bob"}]})),),
        interrupts=()
    )
]

```

js

```

const config = {
  configurable: {
    thread_id: "1", // [!code highlight]
  },
};
const history = []; // [!code highlight]
for await (const state of graph.getStateHistory(config)) {
  history.push(state);
}

```

```

[
  StateSnapshot {
    values: {
      messages: [
        HumanMessage { content: "hi! I'm bob" },
        AIMessage { content: "Hi Bob! How are you doing today? Is there anything I can help
you with?" },
        HumanMessage { content: "what's my name?" },
        AIMessage { content: "Your name is Bob." }
      ]
    },
    next: [],
    config: { configurable: { thread_id: '1', checkpoint_ns: '', checkpoint_id: '1f029ca3-1f
5b-6704-8004-820c16b69a5a' } },
    metadata: { source: 'loop', writes: { call_model: { messages: AIMessage { content: "Your
name is Bob." } } }, step: 4, parents: {}, thread_id: '1' },
    createdAt: '2025-05-05T16:01:24.680462+00:00',
    parentConfig: { configurable: { thread_id: '1', checkpoint_ns: '', checkpoint_id: '1f029
ca3-1790-6b0a-8003-baf965b6a38f' } },
    tasks: [],
    interrupts: []
  },
  // ... more state snapshots
]

```

Decouple return value from saved value

Use `entrypoint.final` to decouple what is returned to the caller from what is persisted in the checkpoint. This is useful when:

- You want to return a computed result (e.g., a summary or status), but save a different internal value for use on the next invocation.
- You need to control what gets passed to the previous parameter on the next run.

python

```
from langgraph.func import entrypoint
from langgraph.checkpoint.memory import InMemorySaver

checkpointer = InMemorySaver()

@entrypoint(checkpointer=checkpointer)
def accumulate(n: int, *, previous: int | None) -> entrypoint.final[int, int]:
    previous = previous or 0
    total = previous + n
    # Return the *previous* value to the caller but save the *new* total to the checkpoint.
    return entrypoint.final(value=previous, save=total)

config = {"configurable": {"thread_id": "my-thread"}}

print(accumulate.invoke(1, config=config)) # 0
print(accumulate.invoke(2, config=config)) # 1
print(accumulate.invoke(3, config=config)) # 3
```

js

```
import { entrypoint, MemorySaver } from "@langchain/langgraph";

const checkpointer = new MemorySaver();

const accumulate = entrypoint(
  { checkpointer, name: "accumulate" },
  async (n: number, previous?: number) => {
    const prev = previous || 0;
    const total = prev + n;
    // Return the *previous* value to the caller but save the *new* total to the checkpoint.
    return entrypoint.final({ value: prev, save: total });
  }
);

const config = { configurable: { thread_id: "my-thread" } };

console.log(await accumulate.invoke(1, config)); // 0
```

```
console.log(await accumulate.invoke(2, config)); // 1
console.log(await accumulate.invoke(3, config)); // 3
```

Chatbot example

An example of a simple chatbot using the functional API and the @[InMemorySaver] checkpointer.

The bot is able to remember the previous conversation and continue from where it left off.

python

```
from langchain.messages import BaseMessage
from langgraph.graph import add_messages
from langgraph.func import entrypoint, task
from langgraph.checkpoint.memory import InMemorySaver
from langchain_anthropic import ChatAnthropic

model = ChatAnthropic(model="claude-sonnet-4-6")

@task
def call_model(messages: list[BaseMessage]):
    response = model.invoke(messages)
    return response

checkpointer = InMemorySaver()

@entrypoint(checkpointer=checkpointer)
def workflow(inputs: list[BaseMessage], *, previous: list[BaseMessage]):
    if previous:
        inputs = add_messages(previous, inputs)

    response = call_model(inputs).result()
    return entrypoint.final(value=response, save=add_messages(inputs, response))

config = {"configurable": {"thread_id": "1"}}
input_message = {"role": "user", "content": "hi! I'm bob"}
stream = workflow.stream_events([input_message], config, version="v3")
for snapshot in stream.values:
    print(snapshot)

input_message = {"role": "user", "content": "what's my name?"}
stream = workflow.stream_events([input_message], config, version="v3")
for snapshot in stream.values:
    print(snapshot)
```

js

```

import { BaseMessage } from "@langchain/core/messages";
import {
  addMessages,
  entrypoint,
  task,
  MemorySaver,
} from "@langchain/langgraph";
import { ChatAnthropic } from "@langchain/anthropic";

const model = new ChatAnthropic({ model: "claude-sonnet-4-6" });

const callModel = task(
  "callModel",
  async (messages: BaseMessage[]): Promise<BaseMessage> => {
    const response = await model.invoke(messages);
    return response;
  }
);

const checkpointer = new MemorySaver();

const workflow = entrypoint(
  { checkpointer, name: "workflow" },
  async (
    inputs: BaseMessage[],
    previous?: BaseMessage[]
  ): Promise<BaseMessage> => {
    let messages = inputs;
    if (previous) {
      messages = addMessages(previous, inputs);
    }

    const response = await callModel(messages);
    return entrypoint.final({
      value: response,
      save: addMessages(messages, response),
    });
  }
);

const config = { configurable: { thread_id: "1" } };
const inputMessage = { role: "user", content: "hi! I'm bob" };

const stream1 = await workflow.streamEvents([inputMessage], { ...config, version: "v3" });
for await (const snapshot of stream1.values) {
  console.log(snapshot);
}

const inputMessage2 = { role: "user", content: "what's my name?" };
const stream2 = await workflow.streamEvents([inputMessage2], { ...config, version: "v3" });

```

```
for await (const snapshot of stream2.values) {  
  console.log(snapshot);  
}
```

Long-term memory

[long-term memory](#) allows storing information across different **thread ids**. This could be useful for learning information about a given user in one conversation and using it in another.

Workflows

- [Workflows and agent](#) guide for more examples of how to build workflows using the Functional API.

Integrate with other libraries

- [Add LangGraph's features to other frameworks using the functional API](#): Add LangGraph features like persistence, memory and streaming to other agent frameworks that do not provide them out of the box.

Use the graph API

langgraph/use-graph-api.mdx

This guide demonstrates the basics of LangGraph's Graph API. It walks through [state](#), as well as composing common graph structures such as [sequences](#), [branches](#), and [loops](#). It also covers LangGraph's control features, including the [Send API](#) for map-reduce workflows and the [Command API](#) for combining state updates with "hops" across nodes.

Setup

python

Install langgraph:

```
```bash pip install -U langgraph
```

```
```bash uv
uv add langgraph
```

js

Install langgraph:

```
npm install @langchain/langgraph
```

팁

Set up LangSmith for better debugging

Sign up for [LangSmith](#) to quickly spot issues and improve the performance of your LangGraph projects. LangSmith lets you use trace data to debug, test, and monitor your LLM apps built with LangGraph—read more about how to get started in the [docs](#).

Define and update state

Here we show how to define and update [state](#) in LangGraph. We will demonstrate:

1. How to use state to define a graph's [schema](#)
2. How to use [reducers](#) to control how state updates are processed.

Define state

python

[State](#) in LangGraph can be a `TypedDict`, `Pydantic` model, or `dataclass`. Below we will use `TypedDict`. See [Use Pydantic models for graph state](#) for detail on using `Pydantic`.

js

[State](#) in LangGraph is defined using the `StateSchema` class. This provides a unified API that accepts [standard schemas](#) (like [Zod](#)) for individual fields along with special value types like `ReducedValue`, `MessagesValue`, and `UntrackedValue`.

By default, graphs will have the same input and output schema, and the state determines that schema. See [Define input and output schemas](#) for how to define distinct input and output schemas.

python

Let's consider a simple example using [messages](#). This represents a versatile formulation of state for many LLM applications. See our [concepts page](#) for more detail.

js

Let's consider a simple example using [messages](#). This represents a versatile formulation of state for many LLM applications. See our [concepts page](#) for more detail.

python

```
from langchain.messages import AnyMessage
from typing_extensions import TypedDict

class State(TypedDict):
    messages: list[AnyMessage]
    extra_field: int
```

This state tracks a list of [message](#) objects, as well as an extra integer field.

js

```
import { StateSchema, MessagesValue } from "@langchain/langgraph";
import * as z from "zod";

const State = new StateSchema({
```



```
messages: MessagesValue,  
extraField: z.number(),  
});
```

This state tracks a list of [message](#) objects, as well as an extra integer field.

Update state

python

Let's build an example graph with a single node. Our [node](#) is just a Python function that reads our graph's state and makes updates to it. The first argument to this function will always be the state:

```
from langchain.messages import AIMessage  
  
def node(state: State):  
    messages = state["messages"]  
    new_message = AIMessage("Hello!")  
    return {"messages": messages + [new_message], "extra_field": 10}
```

This node simply appends a message to our message list, and populates an extra field.

js

Let's build an example graph with a single node. Our [node](#) is just a TypeScript function that reads our graph's state and makes updates to it. The first argument to this function will always be the state:

```
import { AIMessage } from "@langchain/core/messages";  
import { GraphNode } from "@langchain/langgraph";  
  
const node: GraphNode<typeof State> = (state) => {  
    const messages = state.messages;  
    const newMessage = new AIMessage("Hello!");  
    return { messages: [newMessage], extraField: 10 };  
};
```

This node simply appends a message to our message list (the reducer handles concatenation), and populates an extra field.

주의

Nodes should return updates to the state directly, instead of mutating the state.

python

Let's next define a simple graph containing this node. We use [StateGraph](#) to define a graph that operates on this state. We then use [add_node](#) to populate our graph.

```
from langgraph.graph import StateGraph

builder = StateGraph(State)
builder.add_node(node)
builder.set_entry_point("node")
graph = builder.compile()
```

js

Let's next define a simple graph containing this node. We use [StateGraph](#) to define a graph that operates on this state. We then use [addNode](#) to populate our graph.

```
import { StateGraph } from "@langchain/langgraph";

const graph = new StateGraph(State)
  .addNode("node", node)
  .addEdge("__start__", "node")
  .compile();
```

LangGraph provides built-in utilities for visualizing your graph. Let's inspect our graph. See [Visualize your graph](#) for detail on visualization.

python

```
from IPython.display import Image, display

display(Image(graph.get_graph().draw_mermaid_png()))
```



Simple graph with single node

js

```
import * as fs from "node:fs/promises";

const drawableGraph = await graph.getGraphAsync();
const image = await drawableGraph.drawMermaidPng();
const imageBuffer = new Uint8Array(await image.arrayBuffer());
```

```
await fs.writeFile("graph.png", imageBuffer);
```

In this case, our graph just executes a single node. Let's proceed with a simple invocation:

python

```
from langchain.messages import HumanMessage
```

```
result = graph.invoke({"messages": [HumanMessage("Hi")]})
result
```

```
{'messages': [HumanMessage(content='Hi'), AIMessage(content='Hello!')], 'extra_field': 10}
```

js

```
import { HumanMessage } from "@langchain/core/messages";
```

```
const result = await graph.invoke({ messages: [new HumanMessage("Hi")], extraField: 0 });
console.log(result);
```

```
{ messages: [HumanMessage { content: 'Hi' }, AIMessage { content: 'Hello!' }], extraField: 1
0 }
```

Note that:

- We kicked off invocation by updating a single key of the state.
- We receive the entire state in the invocation result.

python

For convenience, we frequently inspect the content of [message objects](#) via pretty-print:

```
for message in result["messages"]:
    message.pretty_print()
```

```
===== Human Message =====
```

```
Hi
```

```
===== Ai Message =====
```

```
Hello!
```

js

For convenience, we frequently inspect the content of [message objects](#) via logging:

```
for (const message of result.messages) {  
  console.log(`${message.getType()}: ${message.content}`);  
}
```

```
human: Hi  
ai: Hello!
```

Process state updates with reducers

Each key in the state can have its own independent [reducer](#) function, which controls how updates from nodes are applied. If no reducer function is explicitly specified then it is assumed that all updates to the key should override it.

python

For `TypedDict` state schemas, we can define reducers by annotating the corresponding field of the state with a reducer function.

In the earlier example, our node updated the `"messages"` key in the state by appending a message to it. Below, we add a reducer to this key, such that updates are automatically appended:

```
from typing_extensions import Annotated  
  
def add(left, right):  
    """Can also import `add` from the `operator` built-in."""  
    return left + right  
  
class State(TypedDict):  
    messages: Annotated[list[AnyMessage], add] # [!code highlight]  
    extra_field: int
```

Now our node can be simplified:

```
def node(state: State):  
    new_message = AIMessage("Hello!")  
    return {"messages": [new_message], "extra_field": 10} # [!code highlight]
```

js

In our earlier example, we used `MessagesValue` which already has a built-in reducer. For custom fields, you can use `ReducedValue` to define how updates are applied.

In the earlier example, our node updated the `"messages"` key in the state by appending a message to it. The `MessagesValue` reducer handles this automatically:

```
import { StateSchema, MessagesValue, ReducedValue } from "@langchain/langgraph";
import * as z from "zod";

// MessagesValue already has a built-in reducer
const State = new StateSchema({
  messages: MessagesValue, // [!code highlight]
  extraField: z.number(),
});
```

Our node can simply return the new message (the reducer handles concatenation):

```
import { GraphNode } from "@langchain/langgraph";

const node: GraphNode<typeof State> = (state) => {
  const newMessage = new AIMessage("Hello!");
  return { messages: [newMessage], extraField: 10 }; // [!code highlight]
};
```

python

```
from langgraph.graph import START

graph = StateGraph(State).add_node(node).add_edge(START, "node").compile()

result = graph.invoke({"messages": [HumanMessage("Hi")]})

for message in result["messages"]:
    message.pretty_print()
```

```
===== Human Message =====

Hi
===== Ai Message =====

Hello!
```

js

```
import { START } from "@langchain/langgraph";
```

```

const graph = new StateGraph(State)
  .addNode("node", node)
  .addEdge(START, "node")
  .compile();

const result = await graph.invoke({ messages: [new HumanMessage("Hi")] });

for (const message of result.messages) {
  console.log(`${message.getType()}: ${message.content}`);
}

```

```

human: Hi
ai: Hello!

```

python

MessagesState

In practice, there are additional considerations for updating lists of messages:

- We may wish to update an existing message in the state.
- We may want to accept short-hands for [message formats](#), such as [OpenAI format](#).

LangGraph includes a built-in reducer `@[ add_messages ]` that handles these considerations:

```

from langgraph.graph.message import add_messages

class State(TypedDict):
    messages: Annotated[list[AnyMessage], add_messages] # [!code highlight]
    extra_field: int

def node(state: State):
    new_message = AIMessage("Hello!")
    return {"messages": [new_message], "extra_field": 10}

graph = StateGraph(State).add_node(node).set_entry_point("node").compile()

```

```

input_message = {"role": "user", "content": "Hi"} # [!code highlight]

result = graph.invoke({"messages": [input_message]})

for message in result["messages"]:
    message.pretty_print()

```

```

===== Human Message =====

Hi

```

```
===== Ai Message =====
```

```
Hello!
```

This is a versatile representation of state for applications involving [chat models](#). LangGraph includes a prebuilt `MessagesState` for convenience, so that we can have:

```
from langgraph.graph import MessagesState

class State(MessagesState):
    extra_field: int
```

js

MessagesValue

In practice, there are additional considerations for updating lists of messages:

- We may wish to update an existing message in the state.
- We may want to accept short-hands for [message formats](#), such as [OpenAI format](#).

LangGraph includes the built-in `MessagesValue` that handles these considerations:

```
import { StateSchema, StateGraph, MessagesValue, GraphNode, START } from "@langchain/langgraph";
import * as z from "zod";

const State = new StateSchema({ // [!code highlight]
  messages: MessagesValue,
  extraField: z.number(),
});

const node: GraphNode<typeof State> = (state) => {
  const newMessage = new AIMessage("Hello!");
  return { messages: [newMessage], extraField: 10 };
};

const graph = new StateGraph(State)
  .addNode("node", node)
  .addEdge(START, "node")
  .compile();
```

```
const inputMessage = { role: "user", content: "Hi" }; // [!code highlight]

const result = await graph.invoke({ messages: [inputMessage] });

for (const message of result.messages) {
```

```
console.log(`${message.getType()}: ${message.content}`);  
}
```

```
human: Hi  
ai: Hello!
```

This is a versatile representation of state for applications involving [chat models](#). LangGraph includes the prebuilt `MessagesValue` for convenience, so that we can have:

```
import { StateSchema, MessagesValue } from "@langchain/langgraph";  
import * as z from "zod";  
  
const State = new StateSchema({  
  messages: MessagesValue,  
  extraField: z.number(),  
});
```

python

Bypass reducers with `Overwrite`

In some cases, you may want to bypass a reducer and directly overwrite a state value. LangGraph provides the [Overwrite](#) type for this purpose. When a node returns a value wrapped with `overwrite`, the reducer is bypassed and the channel is set directly to that value.

This is useful when you want to reset or replace accumulated state rather than merge it with existing values.

```
from langgraph.graph import StateGraph, START, END  
from langgraph.types import Overwrite  
from typing_extensions import Annotated, TypedDict  
import operator  
  
class State(TypedDict):  
    messages: Annotated[list, operator.add]  
  
def add_message(state: State):  
    return {"messages": ["first message"]}  
  
def replace_messages(state: State):  
    # Bypass the reducer and replace the entire messages list  
    return {"messages": Overwrite(["replacement message"])}  
  
builder = StateGraph(State)  
builder.add_node("add_message", add_message)  
builder.add_node("replace_messages", replace_messages)  
builder.add_edge(START, "add_message")
```



```
builder.add_edge("add_message", "replace_messages")
builder.add_edge("replace_messages", END)

graph = builder.compile()

result = graph.invoke({"messages": ["initial"]})
print(result["messages"])
```

```
['replacement message']
```

You can also use JSON format with the special key `"__overwrite__"`:

```
def replace_messages(state: State):
    return {"messages": {"__overwrite__": ["replacement message"]}}
```

주의

When nodes execute in parallel, only one node can use `overwrite` on the same state key in a given super-step. If multiple nodes attempt to overwrite the same key in the same super-step, an `InvalidUpdateError` will be raised.

js

Bypass reducers with `overwrite`

In some cases, you may want to bypass a reducer and directly overwrite a state value. LangGraph provides the `overwrite` type for this purpose. When a node returns a value wrapped with `overwrite`, the reducer is bypassed and the channel is set directly to that value.

This is useful when you want to reset or replace accumulated state rather than merge it with existing values.

```
import {
    END,
    Overwrite,
    ReducedValue,
    START,
    StateGraph,
    StateSchema,
} from "@langchain/langgraph";
import { z } from "zod/v4";

const State = new StateSchema({
    messages: new ReducedValue(
        z.array(z.string()).default(() => []),
    ),
    {
```

```

    reducer: (current: string[], update: string[]) => current.concat(update),
  },
),
});

const addMessage = () => {
  return { messages: ["first message"] };
};

const replaceMessages = () => {
  // Bypass the reducer and replace the entire messages list
  return { messages: new Overwrite(["replacement message"]) };
};

const graph = new StateGraph(State)
  .addNode("add_message", addMessage)
  .addNode("replace_messages", replaceMessages)
  .addEdge(START, "add_message")
  .addEdge("add_message", "replace_messages")
  .addEdge("replace_messages", END)
  .compile();

const result = await graph.invoke({ messages: ["initial"] });
console.log(result.messages);

```

```
["replacement message"]
```

You can also use JSON format with the special key `"__overwrite__"`:

```

const replaceMessages = () => {
  return { messages: { __overwrite__: ["replacement message"] } };
};

```

주의

When nodes execute in parallel, only one node can use `overwrite` on the same state key in a given super-step. If multiple nodes attempt to overwrite the same key in the same super-step, an `InvalidUpdateError` is raised.

Define input and output schemas

By default, `StateGraph` operates with a single schema, and all nodes are expected to communicate using that schema. However, it's also possible to define distinct input and output schemas for a graph.

When distinct schemas are specified, an internal schema will still be used for communication between nodes. The input schema ensures that the provided input matches the expected structure, while the output schema filters the internal data to return only the relevant information according to the defined output schema.

Below, we'll see how to define distinct input and output schema.

python

```
from langgraph.graph import StateGraph, START, END
from typing_extensions import TypedDict

# Define the schema for the input
class InputState(TypedDict):
    question: str

# Define the schema for the output
class OutputState(TypedDict):
    answer: str

# Define the overall schema, combining both input and output
class OverallState(InputState, OutputState):
    pass

# Define the node that processes the input and generates an answer
def answer_node(state: InputState):
    # Example answer and an extra key
    return {"answer": "bye", "question": state["question"]}

# Build the graph with input and output schemas specified
builder = StateGraph(OverallState, input_schema=InputState, output_schema=OutputState)
builder.add_node(answer_node) # Add the answer node
builder.add_edge(START, "answer_node") # Define the starting edge
builder.add_edge("answer_node", END) # Define the ending edge
graph = builder.compile() # Compile the graph

# Invoke the graph with an input and print the result
print(graph.invoke({"question": "hi"}))
```

```
{'answer': 'bye'}
```

js

```
import { StateGraph, StateSchema, GraphNode, START, END } from "@langchain/langgraph";
import * as z from "zod";

// Define the schema for the input
const InputState = new StateSchema({
```

```

    question: z.string(),
  });

  // Define the schema for the output
  const OutputState = new StateSchema({
    answer: z.string(),
  });

  // Define the overall schema, combining both input and output
  const OverallState = new StateSchema({
    question: z.string(),
    answer: z.string(),
  });

  // Define the node that processes the input
  const answerNode: GraphNode<typeof OverallState> = (state) => {
    // Example answer and an extra key
    return { answer: "bye", question: state.question };
  };

  // Build the graph with input and output schemas specified
  const graph = new StateGraph({
    input: InputState,
    output: OutputState,
    state: OverallState,
  })
    .addNode("answerNode", answerNode)
    .addEdge(START, "answerNode")
    .addEdge("answerNode", END)
    .compile();

  // Invoke the graph with an input and print the result
  console.log(await graph.invoke({ question: "hi" }));

```

```
{ answer: 'bye' }
```

Notice that the output of invoke only includes the output schema.

Pass private state between nodes

In some cases, you may want nodes to exchange information that is crucial for intermediate logic but doesn't need to be part of the main schema of the graph. This private data is not relevant to the overall input/output of the graph and should only be shared between certain nodes.

Below, we'll create an example sequential graph consisting of three nodes (node_1, node_2 and node_3), where private data is passed between the first two steps (node_1 and node_2), while the third step (node_3) only has access to the public overall state.

python

```
from langgraph.graph import StateGraph, START, END
from typing_extensions import TypedDict

# The overall state of the graph (this is the public state shared across nodes)
class OverallState(TypedDict):
    a: str

# Output from node_1 contains private data that is not part of the overall state
class Node1Output(TypedDict):
    private_data: str

# The private data is only shared between node_1 and node_2
def node_1(state: OverallState) -> Node1Output:
    output = {"private_data": "set by node_1"}
    print(f"Entered node `node_1`: \n\tInput: {state}. \n\tReturned: {output}")
    return output

# Node 2 input only requests the private data available after node_1
class Node2Input(TypedDict):
    private_data: str

def node_2(state: Node2Input) -> OverallState:
    output = {"a": "set by node_2"}
    print(f"Entered node `node_2`: \n\tInput: {state}. \n\tReturned: {output}")
    return output

# Node 3 only has access to the overall state (no access to private data from node_1)
def node_3(state: OverallState) -> OverallState:
    output = {"a": "set by node_3"}
    print(f"Entered node `node_3`: \n\tInput: {state}. \n\tReturned: {output}")
    return output

# Connect nodes in a sequence
# node_2 accepts private data from node_1, whereas
# node_3 does not see the private data.
builder = StateGraph(OverallState).add_sequence([node_1, node_2, node_3])
builder.add_edge(START, "node_1")
graph = builder.compile()

# Invoke the graph with the initial state
response = graph.invoke(
    {
        "a": "set at start",
    }
)

print()
print(f"Output of graph invocation: {response}")
```

```

Entered node `node_1`:
  Input: {'a': 'set at start'}.
  Returned: {'private_data': 'set by node_1'}
Entered node `node_2`:
  Input: {'private_data': 'set by node_1'}.
  Returned: {'a': 'set by node_2'}
Entered node `node_3`:
  Input: {'a': 'set by node_2'}.
  Returned: {'a': 'set by node_3'}

Output of graph invocation: {'a': 'set by node_3'}

```

js

```

import { StateGraph, StateSchema, GraphNode, START, END } from "@langchain/langgraph";
import * as z from "zod";

// The overall state of the graph (this is the public state shared across nodes)
const OverallState = new StateSchema({
  a: z.string(),
});

// Output from node1 contains private data that is not part of the overall state
const Node1Output = new StateSchema({
  privateData: z.string(),
});

// Node 2 input only requests the private data available after node1
const Node2Input = new StateSchema({
  privateData: z.string(),
});

// The private data is only shared between node1 and node2
const node1: GraphNode<typeof OverallState> = (state) => {
  const output = { privateData: "set by node1" };
  console.log(`Entered node 'node1':\n\tInput: ${JSON.stringify(state)}.\n\tReturned: ${JSON.stringify(output)}`);
  return output;
};

const node2: GraphNode<typeof Node2Input> = (state) => {
  const output = { a: "set by node2" };
  console.log(`Entered node 'node2':\n\tInput: ${JSON.stringify(state)}.\n\tReturned: ${JSON.stringify(output)}`);
  return output;
};

// Node 3 only has access to the overall state (no access to private data from node1)
const node3: GraphNode<typeof OverallState> = (state) => {

```

```

const output = { a: "set by node3" };
console.log(`Entered node 'node3':\n\tInput: ${JSON.stringify(state)}.\n\tReturned: ${JSON.stringify(output)}`);
return output;
};

// Connect nodes in a sequence
// node2 accepts private data from node1, whereas
// node3 does not see the private data.
const graph = new StateGraph(OverallState)
  .addNode("node1", node1)
  .addNode("node2", node2, { input: Node2Input })
  .addNode("node3", node3)
  .addEdge(START, "node1")
  .addEdge("node1", "node2")
  .addEdge("node2", "node3")
  .addEdge("node3", END)
  .compile();

// Invoke the graph with the initial state
const response = await graph.invoke({ a: "set at start" });

console.log(`\nOutput of graph invocation: ${JSON.stringify(response)}`);

```

```

Entered node 'node1':
  Input: {"a":"set at start"}.
  Returned: {"privateData":"set by node1"}
Entered node 'node2':
  Input: {"privateData":"set by node1"}.
  Returned: {"a":"set by node2"}
Entered node 'node3':
  Input: {"a":"set by node2"}.
  Returned: {"a":"set by node3"}

Output of graph invocation: {"a":"set by node3"}

```

python

Use pydantic models for graph state

A `@[StateGraph]` accepts a `@[state_schema]` argument on initialization that specifies the "shape" of the state that the nodes in the graph can access and update.

In our examples, we typically use a python-native `TypedDict` or [dataclass](#) for `state_schema`, but `@[state_schema]` can be any [type](#).

Here, we'll see how a [Pydantic BaseModel](#) can be used for `@[state_schema]` to add run-time validation on **inputs**.

참고

Known Limitations

- * Currently, the output of the graph will **NOT** be an instance of a pydantic model.
- * Run-time validation only occurs on inputs to the first node in the graph, not on subsequent nodes or outputs.
- * The validation error trace from pydantic does not show which node the error arises in.
- * Pydantic's recursive validation can be slow. For performance-sensitive applications, you may want to consider using a `dataclass` instead.

```
from langgraph.graph import StateGraph, START, END
from typing_extensions import TypedDict
from pydantic import BaseModel

# The overall state of the graph (this is the public state shared across nodes)
class OverallState(BaseModel):
    a: str

def node(state: OverallState):
    return {"a": "goodbye"}

# Build the state graph
builder = StateGraph(OverallState)
builder.add_node(node) # node_1 is the first node
builder.add_edge(START, "node") # Start the graph with node_1
builder.add_edge("node", END) # End the graph after node_1
graph = builder.compile()

# Test the graph with a valid input
graph.invoke({"a": "hello"})
```

Invoke the graph with an **invalid** input

```
try:
    graph.invoke({"a": 123}) # Should be a string
except Exception as e:
    print("An exception was raised because `a` is an integer rather than a string.")
    print(e)
```

```
An exception was raised because `a` is an integer rather than a string.
1 validation error for OverallState
a
  Input should be a valid string [type=string_type, input_value=123, input_type=int]
  For further information visit https://errors.pydantic.dev/2.9/v/string_type
```

See below for additional features of Pydantic model state:

Serialization Behavior

When using Pydantic models as state schemas, it's important to understand how serialization works, especially when:

- Passing Pydantic objects as inputs
- Receiving outputs from the graph
- Working with nested Pydantic models

Let's see these behaviors in action.

```
from langgraph.graph import StateGraph, START, END
from pydantic import BaseModel

class NestedModel(BaseModel):
    value: str

class ComplexState(BaseModel):
    text: str
    count: int
    nested: NestedModel

def process_node(state: ComplexState):
    # Node receives a validated Pydantic object
    print(f"Input state type: {type(state)}")
    print(f"Nested type: {type(state.nested)}")
    # Return a dictionary update
    return {"text": state.text + " processed", "count": state.count + 1}

# Build the graph
builder = StateGraph(ComplexState)
builder.add_node("process", process_node)
builder.add_edge(START, "process")
builder.add_edge("process", END)
graph = builder.compile()

# Create a Pydantic instance for input
input_state = ComplexState(text="hello", count=0, nested=NestedModel(value="test"))
print(f"Input object type: {type(input_state)}")

# Invoke graph with a Pydantic instance
result = graph.invoke(input_state)
print(f"Output type: {type(result)}")
print(f"Output content: {result}")

# Convert back to Pydantic model if needed
output_model = ComplexState(**result)
print(f"Converted back to Pydantic: {type(output_model)}")
```

Runtime Type Coercion

Pydantic performs runtime type coercion for certain data types. This can be helpful but also lead to unexpected behavior if you're not aware of it.

```
from langgraph.graph import StateGraph, START, END
from pydantic import BaseModel

class CoercionExample(BaseModel):
    # Pydantic will coerce string numbers to integers
    number: int
    # Pydantic will parse string booleans to bool
    flag: bool

def inspect_node(state: CoercionExample):
    print(f"number: {state.number} (type: {type(state.number)})")
    print(f"flag: {state.flag} (type: {type(state.flag)})")
    return {}

builder = StateGraph(CoercionExample)
builder.add_node("inspect", inspect_node)
builder.add_edge(START, "inspect")
builder.add_edge("inspect", END)
graph = builder.compile()

# Demonstrate coercion with string inputs that will be converted
result = graph.invoke({"number": "42", "flag": "true"})

# This would fail with a validation error
try:
    graph.invoke({"number": "not-a-number", "flag": "true"})
except Exception as e:
    print(f"\nExpected validation error: {e}")
```

Working with Message Models

When working with LangChain message types in your state schema, there are important considerations for serialization. You should use `AnyMessage` (rather than `BaseMessage`) for proper serialization/deserialization when using message objects over the wire.

```
from langgraph.graph import StateGraph, START, END
from pydantic import BaseModel
from langchain.messages import HumanMessage, AIMessage, AnyMessage
from typing import List

class ChatState(BaseModel):
    messages: List[AnyMessage]
    context: str

def add_message(state: ChatState):
    return {"messages": state.messages + [AIMessage(content="Hello there!")]}
```

```

builder = StateGraph(ChatState)
builder.add_node("add_message", add_message)
builder.add_edge(START, "add_message")
builder.add_edge("add_message", END)
graph = builder.compile()

# Create input with a message
initial_state = ChatState(
    messages=[HumanMessage(content="Hi")], context="Customer support chat"
)

result = graph.invoke(initial_state)
print(f"Output: {result}")

# Convert back to Pydantic model to see message types
output_model = ChatState(**result)
for i, msg in enumerate(output_model.messages):
    print(f"Message {i}: {type(msg).__name__} - {msg.content}")

```

js

Alternative state definitions

While `StateSchema` is the recommended approach for defining state, LangGraph supports several other methods. This section covers all available options.

Channels API

The channels API provides low-level control over state management. LangGraph provides several built-in channel types:

Channel Type	Behavior	Use Case
<code>LastValue</code>	Stores the most recent value	Simple fields that get overwritten
<code>BinaryOperatorAggregate</code>	Combines values using a reducer function	Accumulating values (counters, lists)
<code>Topic</code>	Collects all values into a sequence	Event streams, audit logs
<code>EphemeralValue</code>	Value that resets between supersteps	Temporary computation state

Using the object shorthand:

When you pass an object with `reducer` and `default`, it creates a `BinaryOperatorAggregate` channel. Passing `null` creates a `LastValue` channel:

```
import { BaseMessage } from "@langchain/core/messages";
import { StateGraph } from "@langchain/langgraph";

interface WorkflowState {
  messages: BaseMessage[];
  question: string;
  answer: string;
}

const workflow = new StateGraph<WorkflowState>({
  channels: {
    // BinaryOperatorAggregate: combines values with a reducer
    messages: {
      reducer: (current, update) => current.concat(update),
      default: () => [],
    },
    // LastValue: stores the most recent value (null = no reducer)
    question: null,
    answer: null,
  },
});
```

Using channel classes directly:

For more control, you can instantiate channel classes directly:

```
import { BaseMessage } from "@langchain/core/messages";
import { StateGraph, LastValue, BinaryOperatorAggregate, Topic } from "@langchain/langgraph";

interface WorkflowState {
  messages: BaseMessage[];
  question: string;
  events: string[];
}

const workflow = new StateGraph<WorkflowState>({
  channels: {
    messages: new BinaryOperatorAggregate<BaseMessage[]>({
      (current, update) => current.concat(update),
      () => []
    }),
    question: new LastValue<string>(),
    // Topic collects all values pushed during execution
    events: new Topic<string>(),
  },
});
```

Annotation.Root

`Annotation.Root` provides a declarative way to define state with reducers. It's similar to `StateSchema` but uses a different syntax:

```
import { BaseMessage } from "@langchain/core/messages";
import { Annotation, StateGraph, messagesStateReducer } from "@langchain/langgraph";

const State = Annotation.Root({
  messages: Annotation<BaseMessage[]>({
    reducer: messagesStateReducer,
    default: () => [],
  }),
  question: Annotation<string>(),
  count: Annotation<number>({
    reducer: (current, update) => current + update,
    default: () => 0,
  }),
});

const graph = new StateGraph(State);
```

Zod objects with Zod v3

When using Zod v3, you can define state with plain `z.object()` schemas. `LangGraph` extends Zod v3 with a `.langgraph` plugin that provides `.reducer()` and `.metadata()` methods:

```
import { z } from "zod/v3";
import { BaseMessage } from "@langchain/core/messages";
import { StateGraph, messagesStateReducer } from "@langchain/langgraph";

const State = z.object({
  // Use .langgraph.reducer() to attach a reducer function
  messages: z
    .array(z.custom<BaseMessage>())
    .default([])
    .langgraph.reducer(messagesStateReducer),
  // Simple fields work directly (last-write-wins)
  question: z.string().optional(),
  answer: z.string().optional(),
  // Custom reducer for accumulating values
  count: z
    .number()
    .default(0)
    .langgraph.reducer((current, update) => current + update),
});

const graph = new StateGraph(State);
```

Zod objects with Zod v4

Zod v4 uses a registry-based approach. Use the LangGraph registry to attach metadata to schema fields:

```
import * as z from "zod";
import { BaseMessage } from "@langchain/core/messages";
import { StateGraph, MessagesZodMeta, messagesStateReducer } from "@langchain/langgraph";
import { registry } from "@langchain/langgraph/zod";

const State = z.object({
  // Use .register() with the LangGraph registry and MessagesZodMeta
  messages: z
    .array(z.custom<BaseMessage>())
    .default([])
    .register(registry, MessagesZodMeta),
  // Simple fields work directly (last-write-wins)
  question: z.string().optional(),
  answer: z.string().optional(),
  // Custom reducer via registry metadata
  count: z
    .number()
    .default(0)
    .register(registry, { reducer: (current: number, update: number) => current + update }),
});

const graph = new StateGraph(State);
```

Comparison table

Approach	Reducers	Type Safety	Zod Version	Recommended
StateSchema	✔ Built-in	✔ Full	v3 or v4	✔ Yes
Channels API	✔ Manual	⚠ Partial	N/A	For advanced cases
Annotation.Root	✔ Built-in	✔ Full	N/A	Legacy
Zod v3 + .langgraph	✔ Via plugin	✔ Full	v3 only	Legacy
Zod v4 + registry	✔ Via registry	✔ Full	v4 only	Legacy

Add runtime configuration

Sometimes you want to be able to configure your graph when calling it. For example, you might want to be able to specify what LLM or system prompt to use at runtime, *without polluting the graph state with these parameters*.

To add runtime configuration:

1. Specify a schema for your configuration
2. Add the configuration to the function signature for nodes or conditional edges
3. Pass the configuration into the graph.

See below for a simple example:

python

```
from langgraph.graph import END, StateGraph, START
from langgraph.runtime import Runtime
from typing_extensions import TypedDict

# 1. Specify config schema
class ContextSchema(TypedDict):
    my_runtime_value: str

# 2. Define a graph that accesses the config in a node
class State(TypedDict):
    my_state_value: str

def node(state: State, runtime: Runtime[ContextSchema]): # [!code highlight]
    if runtime.context["my_runtime_value"] == "a": # [!code highlight]
        return {"my_state_value": 1}
    elif runtime.context["my_runtime_value"] == "b": # [!code highlight]
        return {"my_state_value": 2}
    else:
        raise ValueError("Unknown values.")

builder = StateGraph(State, context_schema=ContextSchema) # [!code highlight]
builder.add_node(node)
builder.add_edge(START, "node")
builder.add_edge("node", END)

graph = builder.compile()

# 3. Pass in configuration at runtime:
print(graph.invoke({}, context={"my_runtime_value": "a"})) # [!code highlight]
print(graph.invoke({}, context={"my_runtime_value": "b"})) # [!code highlight]
```

```
{'my_state_value': 1}
{'my_state_value': 2}
```

js

```
import { StateGraph, StateSchema, GraphNode, END, START } from "@langchain/langgraph";
import * as z from "zod";

// 1. Specify config schema
```

```

const ContextSchema = z.object({
  myRuntimeValue: z.string(),
});

// 2. Define a graph that accesses the config in a node
const State = new StateSchema({
  myStateValue: z.number(),
});

const node: GraphNode<typeof State> = (state, runtime) => {
  if (runtime?.context?.myRuntimeValue === "a") { // [!code highlight]
    return { myStateValue: 1 };
  } else if (runtime?.context?.myRuntimeValue === "b") { // [!code highlight]
    return { myStateValue: 2 };
  } else {
    throw new Error("Unknown values.");
  }
};

const graph = new StateGraph(State, ContextSchema)
  .addNode("node", node)
  .addEdge(START, "node")
  .addEdge("node", END)
  .compile();

// 3. Pass in configuration at runtime:
console.log(await graph.invoke({}, { context: { myRuntimeValue: "a" } })); // [!code highlight]
console.log(await graph.invoke({}, { context: { myRuntimeValue: "b" } })); // [!code highlight]

```

```

{ myStateValue: 1 }
{ myStateValue: 2 }

```

Extended example: specifying LLM at runtime

python

Below we demonstrate a practical example in which we configure what LLM to use at runtime. We will use both OpenAI and Anthropic models.

```

from dataclasses import dataclass

from langchain.chat_models import init_chat_model
from langgraph.graph import MessagesState, END, StateGraph, START
from langgraph.runtime import Runtime
from typing_extensions import TypedDict

```



```

@dataclass
class ContextSchema:
    model_provider: str = "anthropic"

MODELS = {
    "anthropic": init_chat_model("claude-haiku-4-5-20251001"),
    "openai": init_chat_model("gpt-5.4-mini"),
}

def call_model(state: MessagesState, runtime: Runtime[ContextSchema]):
    model = MODELS[runtime.context.model_provider]
    response = model.invoke(state["messages"])
    return {"messages": [response]}

builder = StateGraph(MessagesState, context_schema=ContextSchema)
builder.add_node("model", call_model)
builder.add_edge(START, "model")
builder.add_edge("model", END)

graph = builder.compile()

# Usage
input_message = {"role": "user", "content": "hi"}
# With no configuration, uses default (Anthropic)
response_1 = graph.invoke({"messages": [input_message]}, context=ContextSchema())["messages"][-1]
# Or, can set OpenAI
response_2 = graph.invoke({"messages": [input_message]}, context={"model_provider": "openai"})["messages"][-1]

print(response_1.response_metadata["model_name"])
print(response_2.response_metadata["model_name"])

```

```

claude-haiku-4-5-20251001
gpt-5.4-mini

```

js

Below we demonstrate a practical example in which we configure what LLM to use at runtime. We will use both OpenAI and Anthropic models.

```

import { ChatOpenAI } from "@langchain/openai";
import { ChatAnthropic } from "@langchain/anthropic";
import { StateGraph, StateSchema, MessagesValue, GraphNode, START, END } from "@langchain/langgraph";
import * as z from "zod";

const ConfigSchema = z.object({

```

```

    modelProvider: z.string().default("anthropic"),
  });

  const State = new StateSchema({
    messages: MessagesValue,
  });

  const MODELS = {
    anthropic: new ChatAnthropic({ model: "claude-haiku-4-5-20251001" }),
    openai: new ChatOpenAI({ model: "gpt-5.4-mini" }),
  };

  const callModel: GraphNode<typeof State> = async (state, config) => {
    const modelProvider = config?.configurable?.modelProvider || "anthropic";
    const model = MODELS[modelProvider as keyof typeof MODELS];
    const response = await model.invoke(state.messages);
    return { messages: [response] };
  };

  const graph = new StateGraph(State, ConfigSchema)
    .addNode("model", callModel)
    .addEdge(START, "model")
    .addEdge("model", END)
    .compile();

  // Usage
  const inputMessage = { role: "user", content: "hi" };
  // With no configuration, uses default (Anthropic)
  const response1 = await graph.invoke({ messages: [inputMessage] });
  // Or, can set OpenAI
  const response2 = await graph.invoke(
    { messages: [inputMessage] },
    { configurable: { modelProvider: "openai" } },
  );

  console.log(response1.messages.at(-1)?.response_metadata?.model);
  console.log(response2.messages.at(-1)?.response_metadata?.model);

```

```

claude-haiku-4-5-20251001
gpt-5.4-mini

```

Extended example: specifying model and system message at runtime

python

Below we demonstrate a practical example in which we configure two parameters: the LLM and system message to use at runtime.

```

from dataclasses import dataclass
from langchain.chat_models import init_chat_model
from langchain.messages import SystemMessage
from langgraph.graph import END, MessagesState, StateGraph, START
from langgraph.runtime import Runtime
from typing_extensions import TypedDict

@dataclass
class ContextSchema:
    model_provider: str = "anthropic"
    system_message: str | None = None

MODELS = {
    "anthropic": init_chat_model("claude-haiku-4-5-20251001"),
    "openai": init_chat_model("gpt-5.4-mini"),
}

def call_model(state: MessagesState, runtime: Runtime[ContextSchema]):
    model = MODELS[runtime.context.model_provider]
    messages = state["messages"]
    if (system_message := runtime.context.system_message):
        messages = [SystemMessage(system_message)] + messages
    response = model.invoke(messages)
    return {"messages": [response]}

builder = StateGraph(MessagesState, context_schema=ContextSchema)
builder.add_node("model", call_model)
builder.add_edge(START, "model")
builder.add_edge("model", END)

graph = builder.compile()

# Usage
input_message = {"role": "user", "content": "hi"}
response = graph.invoke({"messages": [input_message]}, context={"model_provider": "openai",
"system_message": "Respond in Italian."})
for message in response["messages"]:
    message.pretty_print()

```

```

===== Human Message =====

hi
===== Ai Message =====

Ciao! Come posso aiutarti oggi?

```

js

Below we demonstrate a practical example in which we configure two parameters: the LLM and system message to use at runtime.

```
import { ChatOpenAI } from "@langchain/openai";
import { ChatAnthropic } from "@langchain/anthropic";
import { SystemMessage } from "@langchain/core/messages";
import { StateGraph, StateSchema, MessagesValue, GraphNode, START, END } from "@langchain/langgraph";
import * as z from "zod";

const ConfigSchema = z.object({
  modelProvider: z.string().default("anthropic"),
  systemMessage: z.string().optional(),
});

const State = new StateSchema({
  messages: MessagesValue,
});

const MODELS = {
  anthropic: new ChatAnthropic({ model: "claude-haiku-4-5-20251001" }),
  openai: new ChatOpenAI({ model: "gpt-5.4-mini" }),
};

const callModel: GraphNode<typeof State> = async (state, config) => {
  const modelProvider = config?.configurable?.modelProvider || "anthropic";
  const systemMessage = config?.configurable?.systemMessage;

  const model = MODELS[modelProvider as keyof typeof MODELS];
  let messages = state.messages;

  if (systemMessage) {
    messages = [new SystemMessage(systemMessage), ...messages];
  }

  const response = await model.invoke(messages);
  return { messages: [response] };
};

const graph = new StateGraph(State, ConfigSchema)
  .addNode("model", callModel)
  .addEdge(START, "model")
  .addEdge("model", END)
  .compile();

// Usage
const inputMessage = { role: "user", content: "hi" };
const response = await graph.invoke(
  { messages: [inputMessage] },
  {
```

```

    configurable: {
      modelProvider: "openai",
      systemMessage: "Respond in Italian."
    }
  }
);

for (const message of response.messages) {
  console.log(`${message.getType()}: ${message.content}`);
}

```

```

human: hi
ai: Ciao! Come posso aiutarti oggi?

```

Add retry policies

There are many use cases where you may wish for your node to have a custom retry policy, for example if you are calling an API, querying a database, or calling an LLM, etc. LangGraph lets you add retry policies to nodes.

python

To configure a retry policy, pass the `retry_policy` parameter to the `@[add_node]`. The `retry_policy` parameter takes in a `RetryPolicy` named tuple object. Below we instantiate a `RetryPolicy` object with the default parameters and associate it with a node:

```

from langgraph.types import RetryPolicy

builder.add_node(
    "node_name",
    node_function,
    retry_policy=RetryPolicy(),
)

```

By default, the `retry_on` parameter uses the `default_retry_on` function, which retries on any exception except for the following:

- `ValueError`
- `TypeError`
- `ArithmeticError`
- `ImportError`
- `LookupError`
- `NameError`
- `SyntaxError`
- `RuntimeError`

- `ReferenceError`
- `StopIteration`
- `StopAsyncIteration`
- `OSError`

In addition, for exceptions from popular http request libraries such as `requests` and `httpx` it only retries on 5xx status codes.

js

To configure a retry policy, pass the `retryPolicy` parameter to the `addNode`. The `retryPolicy` parameter takes in a `RetryPolicy` object. Below we instantiate a `RetryPolicy` object with the default parameters and associate it with a node:

```
import { RetryPolicy } from "@langchain/langgraph";

const graph = new StateGraph(State)
  .addNode("nodeName", nodeFunction, { retryPolicy: {} })
  .compile();
```

By default, the retry policy retries on any exception except for the following:

- `TypeError`
- `SyntaxError`
- `ReferenceError`

Extended example: customizing retry policies

python

Consider an example in which we are reading from a SQL database. Below we pass two different retry policies to nodes:

```
import sqlite3
from typing_extensions import TypedDict
from langchain.chat_models import init_chat_model
from langgraph.graph import END, MessagesState, StateGraph, START
from langgraph.types import RetryPolicy
from langchain.messages import AIMessage

con = sqlite3.connect(":memory:")
model = init_chat_model("claude-haiku-4-5-20251001")

def query_database(state: MessagesState):
    cursor = con.cursor()
    cursor.execute("SELECT * FROM Artist LIMIT 10;")
```

```

        query_result = str(cursor.fetchall())
        return {"messages": [AIMessage(content=query_result)]}

def call_model(state: MessagesState):
    response = model.invoke(state["messages"])
    return {"messages": [response]}

# Define a new graph
builder = StateGraph(MessagesState)
builder.add_node(
    "query_database",
    query_database,
    retry_policy=RetryPolicy(retry_on=sqlite3.OperationalError),
)
builder.add_node("model", call_model, retry_policy=RetryPolicy(max_attempts=5))
builder.add_edge(START, "model")
builder.add_edge("model", "query_database")
builder.add_edge("query_database", END)
graph = builder.compile()

```

js

Consider an example in which we are reading from a SQL database. Below we pass two different retry policies to nodes:

```

import Database from "better-sqlite3";
import { ChatAnthropic } from "@langchain/anthropic";
import { StateGraph, StateSchema, MessagesValue, GraphNode, START, END } from "@langchain/langgraph";
import { AIMessage } from "@langchain/core/messages";

const State = new StateSchema({
  messages: MessagesValue,
});

// Create an in-memory database
const db: typeof Database.prototype = new Database(":memory:");

const model = new ChatAnthropic({ model: "claude-sonnet-4-6" });

const callModel: GraphNode<typeof State> = async (state) => {
  const response = await model.invoke(state.messages);
  return { messages: [response] };
};

const queryDatabase: GraphNode<typeof State> = async (state) => {
  const queryResult: string = JSON.stringify(
    db.prepare("SELECT * FROM Artist LIMIT 10;").all(),
  );
};

```

```

    return { messages: [new AIMessage({ content: "queryResult" })] };
};

const workflow = new StateGraph(State)
// Define the two nodes we will cycle between
.addNode("call_model", callModel, { retryPolicy: { maxAttempts: 5 } })
.addNode("query_database", queryDatabase, {
    retryPolicy: {
        retryOn: (e: any): boolean => {
            if (e instanceof Database.SqliteError) {
                // Retry on "SQLITE_BUSY" error
                return e.code === "SQLITE_BUSY";
            }
            return false; // Don't retry on other errors
        },
    },
})
.addEdge(START, "call_model")
.addEdge("call_model", "query_database")
.addEdge("query_database", END);

const graph = workflow.compile();

```

python

Set node timeouts

Use the `timeout` parameter with `@[ add_node ]` to limit how long a single async node invocation can run. Provide the timeout in seconds or as a `datetime.timedelta`.

```

import asyncio
from typing_extensions import TypedDict

from langgraph.errors import NodeTimeoutError
from langgraph.graph import END, START, StateGraph

class State(TypedDict):
    value: str

async def call_model(state: State) -> State:
    await asyncio.sleep(2)
    return {"value": "done"}

builder = StateGraph(State)
builder.add_node("model", call_model, timeout=1.0)
builder.add_edge(START, "model")
builder.add_edge("model", END)

```



```
graph = builder.compile()

try:
    await graph.ainvoke({"value": "start"})
except NodeTimeoutError:
    print("Node timed out")
```

Node timeouts are supported only for async nodes. If you set `timeout` on a sync node, LangGraph raises an error when the graph is compiled because sync Python execution cannot be safely canceled in-process.

When a node exceeds its timeout, LangGraph raises `NodeTimeoutError`, which subclasses Python's built-in `TimeoutError`. If the node has a `retry_policy` that retries `TimeoutError` or `NodeTimeoutError`, the timed-out attempt is retried. The timeout applies to each attempt independently, so the timer resets for every retry.

Timed-out attempts do not commit their buffered writes. This prevents state updates or child-task scheduling from leaking out after the timeout boundary.

Configure node timeouts

The `timeout=` parameter on `@[ add_node ]` caps how long a single async node attempt may run. Pass a number (seconds), a `timedelta`, or a `@[ TimeoutPolicy ]` for finer control over run and idle timeouts. When the limit is exceeded, LangGraph raises `@[ NodeTimeoutError ]` and lets the retry policy decide whether to retry.

참고

Per-node timeouts require `langgraph>=1.2`.

```
from langgraph.types import TimeoutPolicy

builder.add_node(
    "call_model",
    call_model,
    timeout=TimeoutPolicy(run_timeout=120, idle_timeout=30),
)
```

See [Fault tolerance](#) for the full timeout lifecycle, idle-timeout refresh sources, and `runtime.heartbeat()`.

Handle node errors

The `error_handler=` parameter on `@[ add_node ]` registers a function that runs after a node fails and all retries are exhausted. The handler receives the current state and a typed `@[ NodeError ]`

with failure context, and can route to a recovery branch via `@[ Command ]`:

참고

Node-level error handlers require `langgraph>=1.2`.

```
from langgraph.errors import NodeError
from langgraph.types import Command, RetryPolicy

def payment_error_handler(state: State, error: NodeError) -> Command:
    return Command(
        update={"status": f"compensated: {error.error}"},
        goto="finalize",
    )

builder.add_node(
    "charge_payment",
    charge_payment,
    retry_policy=RetryPolicy(max_attempts=3, retry_on=ConnectionError),
    error_handler=payment_error_handler,
)
```

See [Fault tolerance](#) for compensation patterns and `Command` routing.

Set graph-wide node defaults

참고

Requires `langgraph>=1.2`.

Use `@[ set_node_defaults ]` to set `retry_policy`, `timeout`, `cache_policy`, or `error_handler` once for every node in a graph, instead of repeating them on each `@[ add_node ]` call. Per-node values always win, and defaults are applied at `@[ StateGraph.compile ]` time:

```
from langgraph.types import RetryPolicy, TimeoutPolicy

graph = (
    StateGraph(State)
    .set_node_defaults(
        retry_policy=RetryPolicy(max_attempts=3),
        timeout=TimeoutPolicy(run_timeout=30),
        error_handler=fallback_handler,
    )
    .add_node("a", node_a)
    .add_node("b", node_b, retry_policy=RetryPolicy(max_attempts=5)) # overrides default
    .add_edge(START, "a")
)
```

```
.compile()  
)
```

`retry_policy` and `timeout` defaults apply to every node, including error-handler nodes. `cache_policy` and `error_handler` defaults apply only to regular nodes—handlers never catch themselves, and caching a handler result is unsafe. Defaults are not inherited by subgraphs.

See [Fault tolerance](#) for the full precedence rules and applicability table.

python

Access execution info inside a node

You can access execution identity and retry information via `runtime.execution_info`. This surfaces thread, run, and checkpoint identifiers as well as retry state, without needing to read from `config` directly.

Attribute	Type	Description
<code>thread_id</code>	<code>str</code> \ <code>None</code>	Thread ID for the current execution. <code>None</code> without a checkpointer.
<code>run_id</code>	<code>str</code> \ <code>None</code>	Run ID for the current execution. <code>None</code> when not provided in <code>config</code> .
<code>checkpoint_id</code>	<code>str</code>	Checkpoint ID for the current execution.
<code>checkpoint_ns</code>	<code>str</code>	Checkpoint namespace for the current execution.
<code>task_id</code>	<code>str</code>	Task ID for the current execution.
<code>node_attempt</code>	<code>int</code>	Current execution attempt number (1-indexed). <code>1</code> on the first try, <code>2</code> on the first retry, etc.
<code>node_first_attempt_time</code>	<code>float</code> \ <code>None</code>	Unix timestamp (seconds) of when the first attempt started. Stays the same across retries.

Access thread and run IDs

Use `execution_info` to access the thread ID, run ID, and other identity fields inside a node:

```
from langgraph.graph import StateGraph, START, END  
from langgraph.runtime import Runtime  
from typing_extensions import TypedDict  
  
class State(TypedDict):  
    result: str  
  
def my_node(state: State, runtime: Runtime):  
    info = runtime.execution_info
```

```

print(f"Thread: {info.thread_id}, Run: {info.run_id}") # [!code highlight]
return {"result": "done"}

builder = StateGraph(State)
builder.add_node("my_node", my_node)
builder.add_edge(START, "my_node")
builder.add_edge("my_node", END)
graph = builder.compile()

```

Adjust behavior based on retry state

When a node has a retry policy, use `execution_info` to inspect the current attempt number and switch to a fallback after the first attempt fails:

```

from langgraph.graph import StateGraph, START, END
from langgraph.runtime import Runtime
from langgraph.types import RetryPolicy
from typing_extensions import TypedDict

class State(TypedDict):
    result: str

def my_node(state: State, runtime: Runtime):
    info = runtime.execution_info
    if info.node_attempt > 1: # [!code highlight]
        # use a fallback on retries
        return {"result": call_fallback_api()}
    return {"result": call_primary_api()}

builder = StateGraph(State)
builder.add_node("my_node", my_node, retry_policy=RetryPolicy(max_attempts=3))
builder.add_edge(START, "my_node")
builder.add_edge("my_node", END)
graph = builder.compile()

```

`execution_info` is available on the `Runtime` object even without a retry policy — `node_attempt` defaults to `1` and `node_first_attempt_time` is set to the time the node starts executing.

Access server info inside a node

When your graph runs on LangGraph Server, you can access server-specific metadata via `runtime.server_info`. This surfaces the assistant ID, graph ID, and authenticated user without needing to read from config metadata or configurable keys directly.

Attribute	Type	Description
<code>assistant_id</code>	<code>str</code>	The assistant ID for the current deployment.
<code>graph_id</code>	<code>str</code>	The graph ID for the current deployment.

Attribute	Type	Description
user	BaseUser \ None	The authenticated user, if custom auth is configured.

```

from langgraph.graph import StateGraph, START, END
from langgraph.runtime import Runtime
from typing_extensions import TypedDict

class State(TypedDict):
    result: str

def my_node(state: State, runtime: Runtime):
    server = runtime.server_info
    if server is not None:
        print(f"Assistant: {server.assistant_id}, Graph: {server.graph_id}") # [!code highlight]
        if server.user is not None:
            print(f"User: {server.user.identity}")
    return {"result": "done"}

builder = StateGraph(State)
builder.add_node("my_node", my_node)
builder.add_edge(START, "my_node")
builder.add_edge("my_node", END)
graph = builder.compile()

```

`server_info` is `None` when the graph is not running on LangGraph Server (e.g., during local development or testing).

참고

Requires `deepagents>=0.5.0` (or `langgraph>=1.1.5`) for `runtime.execution_info` and `runtime.server_info`.

Access drain state inside a node

When a [graceful shutdown](#) has been requested, `runtime.drain_requested` is `True`. Read this inside a node to skip expensive work before the next superstep boundary:

```

from langgraph.runtime import Runtime

def my_node(state: State, runtime: Runtime) -> State:
    if runtime.drain_requested: # [!code highlight]
        return {"status": "skipped", "reason": runtime.drain_reason}
    return {"status": do_work()}

```

Property	Type	Description
drain_request ed	bool	True if <code>RunControl.request_drain()</code> has been called for this run.
drain_reason	str \ \nNone	The reason string passed to <code>request_drain()</code> , or <code>None</code> if drain was not requested.

참고

Requires `langgraph>=1.2` . See [Graceful shutdown](#) for the full `RunControl` API.

js

Access execution info inside a node

You can access execution identity and retry information via `runtime.executionInfo` . This surfaces thread, run, and checkpoint identifiers as well as retry state, without needing to read from `config` directly.

Attribute	Type	Description
threadId	string \ \nundefined	Thread ID for the current execution.
runId	string \ \nundefined	Run ID for the current execution.
checkpointId	string	Checkpoint ID for the current execution.
checkpointNs	string	Checkpoint namespace for the current execution.
taskId	string	Task ID for the current execution.
nodeAttempt	number	Current execution attempt number (1-indexed).
nodeFirstAttemptTime	number \ \nundefined	Unix timestamp (seconds) of when the first attempt started. Stays the same across retries.

Access thread and run IDs

Use `executionInfo` to access the thread ID, run ID, and other identity fields inside a node:

```
import { StateGraph, StateSchema, START, END } from "@langchain/langgraph";
import * as z from "zod";

const State = new StateSchema({
  result: z.string(),
});
```

```
const myNode: GraphNode<typeof State> = async (state, runtime) => {
  const info = runtime.executionInfo;
  console.log(`Thread: ${info.threadId}, Run: ${info.runId}`); // [!code highlight]
  return { result: "done" };
};

const graph = new StateGraph(State)
  .addNode("my_node", myNode)
  .addEdge(START, "my_node")
  .addEdge("my_node", END)
  .compile();
```

Adjust behavior based on retry state

When a node has a retry policy, use `executionInfo` to inspect the current attempt number and switch to a fallback after the first attempt fails:

```
import { StateGraph, StateSchema, START, END } from "@langchain/langgraph";
import * as z from "zod";

const State = new StateSchema({
  result: z.string(),
});

const myNode: GraphNode<typeof State> = async (state, runtime) => {
  const info = runtime.executionInfo;
  if (info.nodeAttempt > 1) { // [!code highlight]
    // use a fallback on retries
    return { result: await callFallbackApi() };
  }
  return { result: await callPrimaryApi() };
};

const graph = new StateGraph(State)
  .addNode("my_node", myNode, { retryPolicy: { maxAttempts: 3 } })
  .addEdge(START, "my_node")
  .addEdge("my_node", END)
  .compile();
```

`executionInfo` is available on the `Runtime` object even without a retry policy — `nodeAttempt` defaults to 1 and `nodeFirstAttemptTime` is set to the time the node starts executing.

Access server info inside a node

When your graph runs on LangGraph Server, you can access server-specific metadata via `runtime.serverInfo`.

Attribute	Type	Description
assistantId	string	The assistant ID for the current deployment.
graphId	string	The graph ID for the current deployment.
user	BaseUser \ null	The authenticated user, if custom auth is configured.

```
const myNode: GraphNode<typeof State> = async (state, runtime) => {
  const server = runtime.serverInfo;
  if (server != null) {
    console.log(`Assistant: ${server.assistantId}, Graph: ${server.graphId}`); // [!code highlight]
    if (server.user != null) {
      console.log(`User: ${server.user.identity}`);
    }
  }
  return { result: "done" };
};
```

serverInfo is null when the graph is not running on LangGraph Server.

참고

Requires `deepagents>=1.9.0` (or `@langchain/langgraph>=1.2.8`) for `runtime.executionInfo` and `runtime.serverInfo`.

python

Add node caching

Node caching is useful in cases where you want to avoid repeating operations, like when doing something expensive (either in terms of time or cost). LangGraph lets you add individualized caching policies to nodes in a graph.

To configure a cache policy, pass the `cache_policy` parameter to the `@[add_node]` function. In the following example, a `@[CachePolicy]` object is instantiated with a time to live of 120 seconds and the default `key_func` generator. Then it is associated with a node:

```
from langgraph.types import CachePolicy

builder.add_node(
    "node_name",
    node_function,
    cache_policy=CachePolicy(ttl=120),
)
```


Then, to enable node-level caching for a graph, set the `cache` argument when compiling the graph. The example below uses `InMemoryCache` to set up a graph with in-memory cache, but `SQLiteCache` is also available.

```
from langgraph.cache.memory import InMemoryCache

graph = builder.compile(cache=InMemoryCache())
```

Create a sequence of steps

정보

Prerequisites This guide assumes familiarity with the above section on [state](#).

Here we demonstrate how to construct a simple sequence of steps. We will show:

1. How to build a sequential graph
2. Built-in short-hand for constructing similar graphs.

python

To add a sequence of nodes, we use the `@[ add_node ]` and `@[ add_edge ]` methods of our [graph](#):

```
from langgraph.graph import START, StateGraph

builder = StateGraph(State)

# Add nodes
builder.add_node(step_1)
builder.add_node(step_2)
builder.add_node(step_3)

# Add edges
builder.add_edge(START, "step_1")
builder.add_edge("step_1", "step_2")
builder.add_edge("step_2", "step_3")
```

We can also use the built-in shorthand `.add_sequence`:

```
builder = StateGraph(State).add_sequence([step_1, step_2, step_3])
builder.add_edge(START, "step_1")
```

js

To add a sequence of nodes, we use the `.addNode` and `.addEdge` methods of our [graph](#):

```
import { START, StateGraph } from "@langchain/langgraph";

const builder = new StateGraph(State)
  .addNode("step1", step1)
  .addNode("step2", step2)
  .addNode("step3", step3)
  .addEdge(START, "step1")
  .addEdge("step1", "step2")
  .addEdge("step2", "step3");
```

Why split application steps into a sequence with LangGraph?

LangGraph makes it easy to add an underlying persistence layer to your application. This allows state to be checkpointed in between the execution of nodes, so your LangGraph nodes govern:

- How state updates are [checkpointed](#)
- How interruptions are resumed in [human-in-the-loop](#) workflows
- How we can "rewind" and branch-off executions using LangGraph's [time travel](#) features

They also determine how execution steps are [streamed](#), and how your application is visualized and debugged using [Studio](#).

Let's demonstrate an end-to-end example. We will create a sequence of three steps:

1. Populate a value in a key of the state
2. Update the same value
3. Populate a different value

Let's first define our [state](#). This governs the [schema of the graph](#), and can also specify how to apply updates. See [Process state updates with reducers](#) for more detail.

In our case, we will just keep track of two values:

python

```
from typing_extensions import TypedDict

class State(TypedDict):
    value_1: str
    value_2: int
```

js

```
import { StateSchema, GraphNode } from "@langchain/langgraph";
import * as z from "zod";
```

```
const State = new StateSchema({
  value1: z.string(),
  value2: z.number(),
});
```

python

Our [nodes](#) are just Python functions that read our graph's state and make updates to it. The first argument to this function will always be the state:

```
def step_1(state: State):
    return {"value_1": "a"}

def step_2(state: State):
    current_value_1 = state["value_1"]
    return {"value_1": f"{current_value_1} b"}

def step_3(state: State):
    return {"value_2": 10}
```

js

Our [nodes](#) are just TypeScript functions that read our graph's state and make updates to it. The first argument to this function will always be the state:

```
const step1: GraphNode<typeof State> = (state) => {
  return { value1: "a" };
};

const step2: GraphNode<typeof State> = (state) => {
  const currentValue1 = state.value1;
  return { value1: `${currentValue1} b` };
};

const step3: GraphNode<typeof State> = (state) => {
  return { value2: 10 };
};
```

참고

Note that when issuing updates to the state, each node can just specify the value of the key it wishes to update.

By default, this will **overwrite** the value of the corresponding key. You can also use [reducers](#) to control how updates are processed—for example, you can append successive updates to a key

instead. See [Process state updates with reducers](#) for more detail.

Finally, we define the graph. We use [StateGraph](#) to define a graph that operates on this state.

python

We will then use [add_node](#) and [add_edge](#) to populate our graph and define its control flow.

```
from langgraph.graph import START, StateGraph

builder = StateGraph(State)

# Add nodes
builder.add_node(step_1)
builder.add_node(step_2)
builder.add_node(step_3)

# Add edges
builder.add_edge(START, "step_1")
builder.add_edge("step_1", "step_2")
builder.add_edge("step_2", "step_3")
```

js

We will then use [addNode](#) and [addEdge](#) to populate our graph and define its control flow.

```
import { START, StateGraph } from "@langchain/langgraph";

const graph = new StateGraph(State)
  .addNode("step1", step1)
  .addNode("step2", step2)
  .addNode("step3", step3)
  .addEdge(START, "step1")
  .addEdge("step1", "step2")
  .addEdge("step2", "step3")
  .compile();
```

python

팁

Specifying custom names You can specify custom names for nodes using `@[ add_node ]`:

```
builder.add_node("my_node", step_1)
```

js

팁

Specifying custom names You can specify custom names for nodes using `.addNode`:

```
const graph = new StateGraph(State)
  .addNode("myNode", step1)
  .compile();
```

Note that:

python

- `@[ add_edge ]` takes the names of nodes, which for functions defaults to `node.__name__`.
- We must specify the entry point of the graph. For this we add an edge with the [START node](#).
- The graph halts when there are no more nodes to execute.

We next [compile](#) our graph. This provides a few basic checks on the structure of the graph (e.g., identifying orphaned nodes). If we were adding persistence to our application via a [checkpointer](#), it would also be passed in here.

```
graph = builder.compile()
```

js

- `.addEdge` takes the names of nodes, which for functions defaults to `node.name`.
- We must specify the entry point of the graph. For this we add an edge with the [START node](#).
- The graph halts when there are no more nodes to execute.

We next [compile](#) our graph. This provides a few basic checks on the structure of the graph (e.g., identifying orphaned nodes). If we were adding persistence to our application via a [checkpointer](#), it would also be passed in here.

LangGraph provides built-in utilities for visualizing your graph. Let's inspect our sequence. See [Visualize your graph](#) for detail on visualization.

python

```
from IPython.display import Image, display
```

```
display(Image(graph.get_graph().draw_mermaid_png()))
```

 Sequence of steps graph

js

```
import * as fs from "node:fs/promises";

const drawableGraph = await graph.getGraphAsync();
const image = await drawableGraph.drawMermaidPng();
const imageBuffer = new Uint8Array(await image.arrayBuffer());

await fs.writeFile("graph.png", imageBuffer);
```

Let's proceed with a simple invocation:

python

```
graph.invoke({"value_1": "c"})
```

```
{'value_1': 'a b', 'value_2': 10}
```

js

```
const result = await graph.invoke({ value1: "c" });
console.log(result);
```

```
{ value1: 'a b', value2: 10 }
```

Note that:

- We kicked off invocation by providing a value for a single state key. We must always provide a value for at least one key.
- The value we passed in was overwritten by the first node.
- The second node updated the value.
- The third node populated a different value.

python

팁

Built-in shorthand `langgraph>=0.2.46` includes a built-in short-hand `add_sequence` for adding node sequences. You can compile the same graph as follows:

```

builder = StateGraph(State).add_sequence([step_1, step_2, step_3]) # [!code highlight]
builder.add_edge(START, "step_1")

graph = builder.compile()

graph.invoke({"value_1": "c"})

```

Create branches

Parallel execution of nodes is essential to speed up overall graph operation. LangGraph offers native support for parallel execution of nodes, which can significantly enhance the performance of graph-based workflows. This parallelization is achieved through fan-out and fan-in mechanisms, utilizing both standard edges and `@conditional_edges`[`add_conditional_edges`]. Below are some examples showing how to add create branching dataflows that work for you.

Run graph nodes in parallel

In this example, we fan out from `Node A` to `B` and `C` and then fan in to `D`. With our state, [we specify the reducer add operation](#). This will combine or accumulate values for the specific key in the State, rather than simply overwriting the existing value. For lists, this means concatenating the new list with the existing list. See the above section on [state reducers](#) for more detail on updating state with reducers.

python

```

import operator
from typing import Annotated, Any
from typing_extensions import TypedDict
from langgraph.graph import StateGraph, START, END

class State(TypedDict):
    # The operator.add reducer fn makes this append-only
    aggregate: Annotated[list, operator.add]

def a(state: State):
    print(f'Adding "A" to {state["aggregate"]}')
    return {"aggregate": ["A"]}

def b(state: State):
    print(f'Adding "B" to {state["aggregate"]}')
    return {"aggregate": ["B"]}

def c(state: State):
    print(f'Adding "C" to {state["aggregate"]}')
    return {"aggregate": ["C"]}

```

```

def d(state: State):
    print(f'Adding "D" to {state["aggregate"]}')
    return {"aggregate": ["D"]}

builder = StateGraph(State)
builder.add_node(a)
builder.add_node(b)
builder.add_node(c)
builder.add_node(d)
builder.add_edge(START, "a")
builder.add_edge("a", "b")
builder.add_edge("a", "c")
builder.add_edge("b", "d")
builder.add_edge("c", "d")
builder.add_edge("d", END)
graph = builder.compile()

```

js

```

import { StateGraph, StateSchema, ReducedValue, GraphNode, START, END } from "@langchain/langgraph";
import * as z from "zod";

const State = new StateSchema({
  // The reducer makes this append-only
  aggregate: new ReducedValue(
    z.array(z.string()).default(() => []),
    { reducer: (x, y) => x.concat(y) }
  ),
});

const nodeA: GraphNode<typeof State> = (state) => {
  console.log(`Adding "A" to ${state.aggregate}`);
  return { aggregate: ["A"] };
};

const nodeB: GraphNode<typeof State> = (state) => {
  console.log(`Adding "B" to ${state.aggregate}`);
  return { aggregate: ["B"] };
};

const nodeC: GraphNode<typeof State> = (state) => {
  console.log(`Adding "C" to ${state.aggregate}`);
  return { aggregate: ["C"] };
};

const nodeD: GraphNode<typeof State> = (state) => {
  console.log(`Adding "D" to ${state.aggregate}`);

```



```

    return { aggregate: ["D"] };
};

const graph = new StateGraph(State)
    .addNode("a", nodeA)
    .addNode("b", nodeB)
    .addNode("c", nodeC)
    .addNode("d", nodeD)
    .addEdge(START, "a")
    .addEdge("a", "b")
    .addEdge("a", "c")
    .addEdge("b", "d")
    .addEdge("c", "d")
    .addEdge("d", END)
    .compile();

```

python

```

from IPython.display import Image, display

display(Image(graph.get_graph().draw_mermaid_png()))

```



Parallel execution graph

js

```

import * as fs from "node:fs/promises";

const drawableGraph = await graph.getGraphAsync();
const image = await drawableGraph.drawMermaidPng();
const imageBuffer = new Uint8Array(await image.arrayBuffer());

await fs.writeFile("graph.png", imageBuffer);

```

With the reducer, you can see that the values added in each node are accumulated.

python

```

graph.invoke({"aggregate": []}, {"configurable": {"thread_id": "foo"}})

```

```

Adding "A" to []
Adding "B" to ['A']
Adding "C" to ['A']
Adding "D" to ['A', 'B', 'C']

```

js

```
const result = await graph.invoke({
  aggregate: [],
});
console.log(result);
```

```
Adding "A" to []
Adding "B" to ['A']
Adding "C" to ['A']
Adding "D" to ['A', 'B', 'C']
{ aggregate: ['A', 'B', 'C', 'D'] }
```

참고

In the above example, nodes "b" and "c" are executed concurrently in the same [superstep](#). Because they are in the same step, node "d" executes after both "b" and "c" are finished.

Importantly, updates from a parallel superstep may not be ordered consistently. If you need a consistent, predetermined ordering of updates from a parallel superstep, you should write the outputs to a separate field in the state together with a value with which to order them.

Exception handling?

LangGraph executes nodes within [supersteps](#), meaning that while parallel branches are executed in parallel, the entire superstep is **transactional**. If any of these branches raises an exception, **none** of the updates are applied to the state (the entire superstep errors).

Importantly, when using a [checkpointer](#), results from successful nodes within a superstep are saved, and don't repeat when resumed.

If you have error-prone (perhaps want to handle flakey API calls), LangGraph provides two ways to address this:

1. You can write regular python code within your node to catch and handle exceptions.
2. You can set a [RetryPolicy](#) to direct the graph to retry nodes that raise certain types of exceptions. Only failing branches are retried, so you needn't worry about performing redundant work.

Together, these let you perform parallel execution and fully control exception handling.

python

팁

Set max concurrency You can control the maximum number of concurrent tasks by setting `max_concurrency` in the [configuration](#) when invoking the graph. `max_concurrency` is a standalone config key, so set it at the top level of the config rather than inside `configurable`.

```
graph.invoke({"value_1": "c"}, {"max_concurrency": 10})
```

js

팁

Set max concurrency You can control the maximum number of concurrent tasks by setting `maxConcurrency` in the [configuration](#) when invoking the graph. `maxConcurrency` is a standalone config key, so set it at the top level of the config rather than inside `configurable`.

```
const result = await graph.invoke({ value1: "c" }, { maxConcurrency: 10 });
```

python

Defer node execution

Deferring node execution is useful when you want to delay the execution of a node until all other pending tasks are completed. This is particularly relevant when branches have different lengths, which is common in workflows like map-reduce flows.

The above example showed how to fan-out and fan-in when each path was only one step. But what if one branch had more than one step? Let's add a node `"b_2"` in the `"b"` branch:

```
import operator
from typing import Annotated, Any
from typing_extensions import TypedDict
from langgraph.graph import StateGraph, START, END

class State(TypedDict):
    # The operator.add reducer fn makes this append-only
    aggregate: Annotated[list, operator.add]

def a(state: State):
    print(f'Adding "A" to {state["aggregate"]}')
    return {"aggregate": ["A"]}

def b(state: State):
    print(f'Adding "B" to {state["aggregate"]}')
```

```

    return {"aggregate": ["B"]}

def b_2(state: State):
    print(f'Adding "B_2" to {state["aggregate"]}')
    return {"aggregate": ["B_2"]}

def c(state: State):
    print(f'Adding "C" to {state["aggregate"]}')
    return {"aggregate": ["C"]}

def d(state: State):
    print(f'Adding "D" to {state["aggregate"]}')
    return {"aggregate": ["D"]}

builder = StateGraph(State)
builder.add_node(a)
builder.add_node(b)
builder.add_node(b_2)
builder.add_node(c)
builder.add_node(d, defer=True) # [!code highlight]
builder.add_edge(START, "a")
builder.add_edge("a", "b")
builder.add_edge("a", "c")
builder.add_edge("b", "b_2")
builder.add_edge("b_2", "d")
builder.add_edge("c", "d")
builder.add_edge("d", END)
graph = builder.compile()

```

```

from IPython.display import Image, display

display(Image(graph.get_graph().draw_mermaid_png()))

```

Deferred execution graph

```
graph.invoke({"aggregate": []})
```

```

Adding "A" to []
Adding "B" to ['A']
Adding "C" to ['A']
Adding "B_2" to ['A', 'B', 'C']
Adding "D" to ['A', 'B', 'C', 'B_2']

```

In the above example, nodes "b" and "c" are executed concurrently in the same superstep. We set `defer=True` on node d so it will not execute until all pending tasks are finished. In this case, this means that "d" waits to execute until the entire "b" branch is finished.

When every branch always runs, you can wait with a list-form edge instead of `defer=True`. `add_edge` also accepts a list of start nodes. This is not shorthand for separate `add_edge` calls; the two forms behave differently:

```
builder.add_edge(["b_2", "c"], "d") # d runs once, after both b_2 and c complete
```

- **A list of start nodes** runs `d` once, after all of the listed nodes have completed. If one of them never runs (for example, a conditional edge does not select its branch), `d` never runs and no error is raised. State updates from branches that did complete remain in the graph state, but `d` does not consume them.
- **Separate edges** run `d` once per superstep in which any incoming branch completes. With branches of equal length, that is a single run; with branches of different lengths, `d` runs more than once.
- **Separate edges with `defer=True`** (the pattern in the example above) run `d` once, after every selected branch has completed, whether the fan-out selected all of the branches or only some of them.

`defer=True` postpones the node until no tasks are pending anywhere in the graph, not only in the branches that feed it. A `Send` addressed to the node still invokes it separately.

Conditional branching

python

If your fan-out should vary at runtime based on the state, you can use `@[ add_conditional_edges ]` to select one or more paths using the graph state. See example below, where node `a` generates a state update that determines the following node.

```
import operator
from typing import Annotated, Literal, Sequence
from typing_extensions import TypedDict
from langgraph.graph import StateGraph, START, END

class State(TypedDict):
    aggregate: Annotated[list, operator.add]
    # Add a key to the state. We will set this key to determine
    # how we branch.
    which: str

def a(state: State):
    print(f'Adding "A" to {state["aggregate"]}')
    return {"aggregate": ["A"], "which": "c"} # [!code highlight]

def b(state: State):
    print(f'Adding "B" to {state["aggregate"]}')
```

```

    return {"aggregate": ["B"]}

def c(state: State):
    print(f'Adding "C" to {state["aggregate"]}')
    return {"aggregate": ["C"]}

builder = StateGraph(State)
builder.add_node(a)
builder.add_node(b)
builder.add_node(c)
builder.add_edge(START, "a")
builder.add_edge("b", END)
builder.add_edge("c", END)

def conditional_edge(state: State) -> Literal["b", "c"]:
    # Fill in arbitrary logic here that uses the state
    # to determine the next node
    return state["which"]

builder.add_conditional_edges("a", conditional_edge) # [!code highlight]

graph = builder.compile()

```

```

from IPython.display import Image, display

display(Image(graph.get_graph().draw_mermaid_png()))

```

Conditional branching graph

```

result = graph.invoke({"aggregate": []})
print(result)

```

```

Adding "A" to []
Adding "C" to ['A']
{'aggregate': ['A', 'C'], 'which': 'c'}

```

js

If your fan-out should vary at runtime based on the state, you can use [addConditionalEdges](#) to select one or more paths using the graph state. See example below, where node `a` generates a state update that determines the following node.

```

import { StateGraph, StateSchema, ReducedValue, GraphNode, ConditionalEdgeRouter, START, END } from "@langchain/langgraph";
import * as z from "zod";

const State = new StateSchema({

```

```

    aggregate: new ReducedValue(
      z.array(z.string()).default(() => []),
      { reducer: (x, y) => x.concat(y) }
    ),
    // Add a key to the state. We will set this key to determine
    // how we branch.
    which: z.string(), // [!code highlight]
  });

const nodeA: GraphNode<typeof State> = (state) => {
  console.log(`Adding "A" to ${state.aggregate}`);
  return { aggregate: ["A"], which: "c" };
};

const nodeB: GraphNode<typeof State> = (state) => {
  console.log(`Adding "B" to ${state.aggregate}`);
  return { aggregate: ["B"] };
};

const nodeC: GraphNode<typeof State> = (state) => {
  console.log(`Adding "C" to ${state.aggregate}`);
  return { aggregate: ["C"] }; // [!code highlight]
};

const conditionalEdge: ConditionalEdgeRouter<{ InputSchema: typeof State; Nodes: "b" | "c" }
> = (state) => {
  // Fill in arbitrary logic here that uses the state
  // to determine the next node
  return state.which as "b" | "c";
};

const graph = new StateGraph(State)
  .addNode("a", nodeA)
  .addNode("b", nodeB)
  .addNode("c", nodeC)
  .addEdge(START, "a")
  .addEdge("b", END)
  .addEdge("c", END)
  .addConditionalEdges("a", conditionalEdge)
  .compile();

```

```

import * as fs from "node:fs/promises";

const drawableGraph = await graph.getGraphAsync();
const image = await drawableGraph.drawMermaidPng();
const imageBuffer = new Uint8Array(await image.arrayBuffer());

await fs.writeFile("graph.png", imageBuffer);

```

```
const result = await graph.invoke({ aggregate: [] });
console.log(result);
```

```
Adding "A" to []
Adding "C" to ['A']
{ aggregate: ['A', 'C'], which: 'c' }
```

팁

Your conditional edges can route to multiple destination nodes. For example:

python

```
def route_bc_or_cd(state: State) -> Sequence[str]:
    if state["which"] == "cd":
        return ["c", "d"]
    return ["b", "c"]
```

js

```
const routeBcOrCd: ConditionalEdgeRouter<{ InputSchema: typeof State; Nodes: "b" | "c"
| "d" }> = (state) => {
    if (state.which === "cd") {
        return ["c", "d"];
    }
    return ["b", "c"];
};
```

Map-Reduce and the send API

LangGraph supports map-reduce and other advanced branching patterns using the Send API. Here is an example of how to use it:

python

```
from langgraph.graph import StateGraph, START, END
from langgraph.types import Send
from typing_extensions import TypedDict, Annotated
import operator

class OverallState(TypedDict):
    topic: str
    subjects: list[str]
```



```

jokes: Annotated[list[str], operator.add]
best_selected_joke: str

def generate_topics(state: OverallState):
    return {"subjects": ["lions", "elephants", "penguins"]}

def generate_joke(state: OverallState):
    joke_map = {
        "lions": "Why don't lions like fast food? Because they can't catch it!",
        "elephants": "Why don't elephants use computers? They're afraid of the mouse!",
        "penguins": "Why don't penguins like talking to strangers at parties? Because they find it hard to break the ice."
    }
    return {"jokes": [joke_map[state["subject"]]]}

def continue_to_jokes(state: OverallState):
    return [Send("generate_joke", {"subject": s}) for s in state["subjects"]]

def best_joke(state: OverallState):
    return {"best_selected_joke": "penguins"}

builder = StateGraph(OverallState)
builder.add_node("generate_topics", generate_topics)
builder.add_node("generate_joke", generate_joke)
builder.add_node("best_joke", best_joke)
builder.add_edge(START, "generate_topics")
builder.add_conditional_edges("generate_topics", continue_to_jokes, ["generate_joke"])
builder.add_edge("generate_joke", "best_joke")
builder.add_edge("best_joke", END)
graph = builder.compile()

```

```

from IPython.display import Image, display

display(Image(graph.get_graph().draw_mermaid_png()))

```

Map-reduce graph with fanout

```

# Call the graph: here we call it to generate a list of jokes
stream = graph.stream_events({"topic": "animals"}, version="v3")
for message in stream.messages:
    for token in message.text:
        print(token, end="", flush=True)

```

```

{'generate_topics': {'subjects': ['lions', 'elephants', 'penguins']}}
{'generate_joke': {'jokes': ["Why don't lions like fast food? Because they can't catch it!"]}}
{'generate_joke': {'jokes': ["Why don't elephants use computers? They're afraid of the mouse!"]}}
{'generate_joke': {'jokes': ["Why don't penguins like talking to strangers at parties? Because they find it hard to break the ice."]}}

```

```
se they find it hard to break the ice.']]}}
{'best_joke': {'best_selected_joke': 'penguins'}}
```

js

```
import { StateGraph, StateSchema, ReducedValue, GraphNode, START, END, Send } from "@langchain/langgraph";
import * as z from "zod";

const OverallState = new StateSchema({
  topic: z.string(),
  subjects: z.array(z.string()),
  jokes: new ReducedValue(
    z.array(z.string()).default(() => []),
    { reducer: (x, y) => x.concat(y) }
  ),
  bestSelectedJoke: z.string(),
});

const generateTopics: GraphNode<typeof OverallState> = (state) => {
  return { subjects: ["lions", "elephants", "penguins"] };
};

const generateJoke: GraphNode<typeof OverallState> = (state) => {
  const jokeMap: Record<string, string> = {
    lions: "Why don't lions like fast food? Because they can't catch it!",
    elephants: "Why don't elephants use computers? They're afraid of the mouse!",
    penguins: "Why don't penguins like talking to strangers at parties? Because they find it hard to break the ice."
  };
  return { jokes: [jokeMap[state.subject]] };
};

const continueToJokes: ConditionalEdgeRouter<{ InputSchema: typeof OverallState; Nodes: "generateJoke" }> = (state) => {
  return state.subjects.map((subject) => new Send("generateJoke", { subject }));
};

const bestJoke: GraphNode<typeof OverallState> = (state) => {
  return { bestSelectedJoke: "penguins" };
};

const graph = new StateGraph(OverallState)
  .addNode("generateTopics", generateTopics)
  .addNode("generateJoke", generateJoke)
  .addNode("bestJoke", bestJoke)
  .addEdge(START, "generateTopics")
  .addConditionalEdges("generateTopics", continueToJokes)
  .addEdge("generateJoke", "bestJoke")
```

```
.addEdge("bestJoke", END)
.compile();
```

```
import * as fs from "node:fs/promises";

const drawableGraph = await graph.getGraphAsync();
const image = await drawableGraph.drawMermaidPng();
const imageBuffer = new Uint8Array(await image.arrayBuffer());

await fs.writeFile("graph.png", imageBuffer);
```

```
// Call the graph: here we call it to generate a list of jokes
const stream = await graph.streamEvents({ topic: "animals" }, { version: "v3" });
for await (const message of stream.messages) {
  for await (const token of message.text) {
    process.stdout.write(token);
  }
}
```

```
{ generateTopics: { subjects: [ 'lions', 'elephants', 'penguins' ] } }
{ generateJoke: { jokes: [ "Why don't lions like fast food? Because they can't catch it!" ] } }
{ generateJoke: { jokes: [ "Why don't elephants use computers? They're afraid of the mouse!" ] } }
{ generateJoke: { jokes: [ "Why don't penguins like talking to strangers at parties? Because they find it hard to break the ice." ] } }
{ bestJoke: { bestSelectedJoke: 'penguins' } }
```

Create and control loops

When creating a graph with a loop, we require a mechanism for terminating execution. This is most commonly done by adding a [conditional edge](#) that routes to the [END](#) node once we reach some termination condition.

You can also set the graph recursion limit when invoking or streaming the graph. The recursion limit sets the number of [super-steps](#) that the graph is allowed to execute before it raises an error. Read more about the [recursion limit concept](#).

Let's consider a simple graph with a loop to better understand how these mechanisms work.

팁

To return the last value of your state instead of receiving a recursion limit error, see the [next section](#).

When creating a loop, you can include a conditional edge that specifies a termination condition:

python

```
builder = StateGraph(State)
builder.add_node(a)
builder.add_node(b)

def route(state: State) -> Literal["b", END]:
    if termination_condition(state):
        return END
    else:
        return "b"

builder.add_edge(START, "a")
builder.add_conditional_edges("a", route)
builder.add_edge("b", "a")
graph = builder.compile()
```

js

```
const route: ConditionalEdgeRouter<{ InputSchema: typeof State; Nodes: "b" }> = (state) => {
  if (terminationCondition(state)) {
    return END;
  } else {
    return "b";
  }
};

const graph = new StateGraph(State)
  .addNode("a", nodeA)
  .addNode("b", nodeB)
  .addEdge(START, "a")
  .addConditionalEdges("a", route)
  .addEdge("b", "a")
  .compile();
```

python

To control the recursion limit, specify `"recursion_limit"` in the config. This will raise a `GraphRecursionError`, which you can catch and handle:

```
from langgraph.errors import GraphRecursionError

try:
    graph.invoke(inputs, {"recursion_limit": 3})
```

```
except GraphRecursionError:
    print("Recursion Error")
```

js

To control the recursion limit, specify `"recursionLimit"` in the config. This will raise a `GraphRecursionError`, which you can catch and handle:

```
import { GraphRecursionError } from "@langchain/langgraph";

try {
  await graph.invoke(inputs, { recursionLimit: 3 });
} catch (error) {
  if (error instanceof GraphRecursionError) {
    console.log("Recursion Error");
  }
}
```

Let's define a graph with a simple loop. Note that we use a conditional edge to implement a termination condition.

python

```
import operator
from typing import Annotated, Literal
from typing_extensions import TypedDict
from langgraph.graph import StateGraph, START, END

class State(TypedDict):
    # The operator.add reducer fn makes this append-only
    aggregate: Annotated[list, operator.add]

def a(state: State):
    print(f'Node A sees {state["aggregate"]}')
    return {"aggregate": ["A"]}

def b(state: State):
    print(f'Node B sees {state["aggregate"]}')
    return {"aggregate": ["B"]}

# Define nodes
builder = StateGraph(State)
builder.add_node(a)
builder.add_node(b)

# Define edges
def route(state: State) -> Literal["b", END]:
```

```

    if len(state["aggregate"]) < 7:
        return "b"
    else:
        return END

builder.add_edge(START, "a")
builder.add_conditional_edges("a", route)
builder.add_edge("b", "a")
graph = builder.compile()

from IPython.display import Image, display

display(Image(graph.get_graph().draw_mermaid_png()))

```

 Simple loop graph

js

```

import { StateGraph, StateSchema, ReducedValue, GraphNode, ConditionalEdgeRouter, START, END } from "@langchain/langgraph";
import * as z from "zod";

const State = new StateSchema({
  // The reducer makes this append-only
  aggregate: new ReducedValue(
    z.array(z.string()).default(() => []),
    { reducer: (x, y) => x.concat(y) }
  ),
});

const nodeA: GraphNode<typeof State> = (state) => {
  console.log(`Node A sees ${state.aggregate}`);
  return { aggregate: ["A"] };
};

const nodeB: GraphNode<typeof State> = (state) => {
  console.log(`Node B sees ${state.aggregate}`);
  return { aggregate: ["B"] };
};

// Define edges
const route: ConditionalEdgeRouter<{ InputSchema: typeof State; Nodes: "b" }> = (state) => {
  if (state.aggregate.length < 7) {
    return "b";
  } else {
    return END;
  }
};

```

```
const graph = new StateGraph(State)
  .addNode("a", nodeA)
  .addNode("b", nodeB)
  .addEdge(START, "a")
  .addConditionalEdges("a", route)
  .addEdge("b", "a")
  .compile();
```

```
import * as fs from "node:fs/promises";

const drawableGraph = await graph.getGraphAsync();
const image = await drawableGraph.drawMermaidPng();
const imageBuffer = new Uint8Array(await image.arrayBuffer());

await fs.writeFile("graph.png", imageBuffer);
```

This architecture is similar to a [ReAct agent](#) in which node "a" is a tool-calling model, and node "b" represents the tools.

In our `route` conditional edge, we specify that we should end after the "aggregate" list in the state passes a threshold length.

Invoking the graph, we see that we alternate between nodes "a" and "b" before terminating once we reach the termination condition.

python

```
graph.invoke({"aggregate": []})
```

```
Node A sees []
Node B sees ['A']
Node A sees ['A', 'B']
Node B sees ['A', 'B', 'A']
Node A sees ['A', 'B', 'A', 'B']
Node B sees ['A', 'B', 'A', 'B', 'A']
Node A sees ['A', 'B', 'A', 'B', 'A', 'B']
```

js

```
const result = await graph.invoke({ aggregate: [] });
console.log(result);
```

```
Node A sees []
Node B sees ['A']
Node A sees ['A', 'B']
```

```
Node B sees ['A', 'B', 'A']
Node A sees ['A', 'B', 'A', 'B']
Node B sees ['A', 'B', 'A', 'B', 'A']
Node A sees ['A', 'B', 'A', 'B', 'A', 'B']
{ aggregate: ['A', 'B', 'A', 'B', 'A', 'B', 'A'] }
```

Impose a recursion limit

In some applications, we may not have a guarantee that we will reach a given termination condition. In these cases, we can set the graph's [recursion limit](#). This will raise a `GraphRecursionError` after a given number of [supersteps](#). We can then catch and handle this exception:

python

```
from langgraph.errors import GraphRecursionError

try:
    graph.invoke({"aggregate": []}, {"recursion_limit": 4})
except GraphRecursionError:
    print("Recursion Error")
```

```
Node A sees []
Node B sees ['A']
Node C sees ['A', 'B']
Node D sees ['A', 'B']
Node A sees ['A', 'B', 'C', 'D']
Recursion Error
```

js

```
import { GraphRecursionError } from "@langchain/langgraph";

try {
    await graph.invoke({ aggregate: [] }, { recursionLimit: 4 });
} catch (error) {
    if (error instanceof GraphRecursionError) {
        console.log("Recursion Error");
    }
}
```

```
Node A sees []
Node B sees ['A']
Node A sees ['A', 'B']
Node B sees ['A', 'B', 'A']
```



```
Node A sees ['A', 'B', 'A', 'B']
Recursion Error
```

python

Extended example: return state on hitting recursion limit

Instead of raising `GraphRecursionError`, we can introduce a new key to the state that keeps track of the number of steps remaining until reaching the recursion limit. We can then use this key to determine if we should end the run.

`LangGraph` implements a special `RemainingSteps` annotation. Under the hood, it creates a `ManagedValue` channel -- a state channel that will exist for the duration of our graph run and no longer.

```
import operator
from typing import Annotated, Literal
from typing_extensions import TypedDict
from langgraph.graph import StateGraph, START, END
from langgraph.managed.is_last_step import RemainingSteps

class State(TypedDict):
    aggregate: Annotated[list, operator.add]
    remaining_steps: RemainingSteps

def a(state: State):
    print(f'Node A sees {state["aggregate"]}')
    return {"aggregate": ["A"]}

def b(state: State):
    print(f'Node B sees {state["aggregate"]}')
    return {"aggregate": ["B"]}

# Define nodes
builder = StateGraph(State)
builder.add_node(a)
builder.add_node(b)

# Define edges
def route(state: State) -> Literal["b", END]:
    if state["remaining_steps"] <= 2:
        return END
    else:
        return "b"

builder.add_edge(START, "a")
builder.add_conditional_edges("a", route)
builder.add_edge("b", "a")
```

```
graph = builder.compile()

# Test it out
result = graph.invoke({"aggregate": []}, {"recursion_limit": 4})
print(result)
```

```
Node A sees []
Node B sees ['A']
Node A sees ['A', 'B']
{'aggregate': ['A', 'B', 'A']}
```

python

Extended example: loops with branches

To better understand how the recursion limit works, let's consider a more complex example. Below we implement a loop, but one step fans out into two nodes:

```
import operator
from typing import Annotated, Literal
from typing_extensions import TypedDict
from langgraph.graph import StateGraph, START, END

class State(TypedDict):
    aggregate: Annotated[list, operator.add]

def a(state: State):
    print(f'Node A sees {state["aggregate"]}')
    return {"aggregate": ["A"]}

def b(state: State):
    print(f'Node B sees {state["aggregate"]}')
    return {"aggregate": ["B"]}

def c(state: State):
    print(f'Node C sees {state["aggregate"]}')
    return {"aggregate": ["C"]}

def d(state: State):
    print(f'Node D sees {state["aggregate"]}')
    return {"aggregate": ["D"]}

# Define nodes
builder = StateGraph(State)
builder.add_node(a)
builder.add_node(b)
builder.add_node(c)
builder.add_node(d)
```

```
# Define edges
def route(state: State) -> Literal["b", END]:
    if len(state["aggregate"]) < 7:
        return "b"
    else:
        return END

builder.add_edge(START, "a")
builder.add_conditional_edges("a", route)
builder.add_edge("b", "c")
builder.add_edge("b", "d")
builder.add_edge(["c", "d"], "a")
graph = builder.compile()
```

```
from IPython.display import Image, display

display(Image(graph.get_graph().draw_mermaid_png()))
```

Complex loop graph with branches

This graph looks complex, but can be conceptualized as loop of [supersteps](#):

1. Node A
2. Node B
3. Nodes C and D
4. Node A
5. ...

We have a loop of four supersteps, where nodes C and D are executed concurrently. The list-form edge waits for both "c" and "d" before returning to "a". For how list-form edges differ from separate edges into the same node, see [Defer node execution](#).

Invoking the graph as before, we see that we complete two full "laps" before hitting the termination condition:

```
result = graph.invoke({"aggregate": []})
```

```
Node A sees []
Node B sees ['A']
Node D sees ['A', 'B']
Node C sees ['A', 'B']
Node A sees ['A', 'B', 'C', 'D']
Node B sees ['A', 'B', 'C', 'D', 'A']
Node D sees ['A', 'B', 'C', 'D', 'A', 'B']
Node C sees ['A', 'B', 'C', 'D', 'A', 'B']
Node A sees ['A', 'B', 'C', 'D', 'A', 'B', 'C', 'D']
```

However, if we set the recursion limit to four, we only complete one lap because each lap is four supersteps:

```
from langgraph.errors import GraphRecursionError

try:
    result = graph.invoke({"aggregate": []}, {"recursion_limit": 4})
except GraphRecursionError:
    print("Recursion Error")
```

```
Node A sees []
Node B sees ['A']
Node C sees ['A', 'B']
Node D sees ['A', 'B']
Node A sees ['A', 'B', 'C', 'D']
Recursion Error
```

python

Async

Using the async programming paradigm can produce significant performance improvements when running [IO-bound](#) code concurrently (e.g., making concurrent API requests to a chat model provider).

To convert a `sync` implementation of the graph to an `async` implementation, you will need to:

1. Update `nodes` use `async def` instead of `def`.
2. Update the code inside to use `await` appropriately.
3. Invoke the graph with `.ainvoke` or `.astream` as desired.

Because many LangChain objects implement the [Runnable Protocol](#) which has `async` variants of all the `sync` methods it's typically fairly quick to upgrade a `sync` graph to an `async` graph.

See example below. To demonstrate async invocations of underlying LLMs, we will include a chat model:

```
from langchain.chat_models import init_chat_model
from langgraph.graph import MessagesState, StateGraph

async def node(state: MessagesState): # [!code highlight]
    new_message = await llm.ainvoke(state["messages"]) # [!code highlight]
    return {"messages": [new_message]}

builder = StateGraph(MessagesState).add_node(node).set_entry_point("node")
graph = builder.compile()
```

```
input_message = {"role": "user", "content": "Hello"}
result = await graph.ainvoke({"messages": [input_message]}) # [!code highlight]
```

팁

Async streaming See the [streaming guide](#) for examples of streaming with async.

Combine control flow and state updates with `Command`

It can be useful to combine control flow (edges) and state updates (nodes). For example, you might want to BOTH perform state updates AND decide which node to go to next in the SAME node. LangGraph provides a way to do so by returning a `@[Command]` object from node functions:

python

```
def my_node(state: State) -> Command[Literal["my_other_node"]]:
    return Command(
        # state update
        update={"foo": "bar"},
        # control flow
        goto="my_other_node"
    )
```

js

```
import { Command } from "@langchain/langgraph";

const myNode = (state: State): Command => {
  return new Command({
    // state update
    update: { foo: "bar" },
    // control flow
    goto: "myOtherNode"
  });
};
```

We show an end-to-end example below. Let's create a simple graph with 3 nodes: A, B and C. We will first execute node A, and then decide whether to go to Node B or Node C next based on the output of node A.

python

```

import random
from typing_extensions import TypedDict, Literal
from langgraph.graph import StateGraph, START
from langgraph.types import Command

# Define graph state
class State(TypedDict):
    foo: str

# Define the nodes

def node_a(state: State) -> Command[Literal["node_b", "node_c"]]:
    print("Called A")
    value = random.choice(["b", "c"])
    # this is a replacement for a conditional edge function
    if value == "b":
        goto = "node_b"
    else:
        goto = "node_c"

# note how Command allows you to BOTH update the graph state AND route to the next node
return Command(
    # this is the state update
    update={"foo": value},
    # this is a replacement for an edge
    goto=goto,
)

def node_b(state: State):
    print("Called B")
    return {"foo": state["foo"] + "b"}

def node_c(state: State):
    print("Called C")
    return {"foo": state["foo"] + "c"}

```

We can now create the `@[StateGraph]` with the above nodes. Notice that the graph doesn't have [conditional edges](#) for routing! This is because control flow is defined with `@[Command]` inside `node_a`.

```

builder = StateGraph(State)
builder.add_edge(START, "node_a")
builder.add_node(node_a)
builder.add_node(node_b)
builder.add_node(node_c)
# NOTE: there are no edges between nodes A, B and C!

graph = builder.compile()

```

주의

You might have noticed that we used `@[Command]` as a return type annotation, e.g. `Command[Literal["node_b", "node_c"]]`. This is necessary for the graph rendering and tells LangGraph that `node_a` can navigate to `node_b` and `node_c`.

```
from IPython.display import display, Image

display(Image(graph.get_graph().draw_mermaid_png()))
```

Command-based graph navigation

If we run the graph multiple times, we'd see it take different paths (A -> B or A -> C) based on the random choice in node A.

```
graph.invoke({"foo": ""})
```

```
Called A
Called C
```

js

```
import { StateGraph, StateSchema, GraphNode, Command, START } from "@langchain/langgraph";
import * as z from "zod";

// Define graph state
const State = new StateSchema({
  foo: z.string(),
});

// Define the nodes

const nodeA: GraphNode<{ InputSchema: typeof State; Nodes: "nodeB" | "nodeC" }> = (state) => {
  console.log("Called A");
  const value = Math.random() > 0.5 ? "b" : "c";
  // this is a replacement for a conditional edge function
  const goto = value === "b" ? "nodeB" : "nodeC";

  // note how Command allows you to BOTH update the graph state AND route to the next node
  return new Command({
    // this is the state update
    update: { foo: value },
    // this is a replacement for an edge
    goto,
  });
};
```

```
};

const nodeB: GraphNode<typeof State> = (state) => {
  console.log("Called B");
  return { foo: state.foo + "b" };
};

const nodeC: GraphNode<typeof State> = (state) => {
  console.log("Called C");
  return { foo: state.foo + "c" };
};
```

We can now create the `StateGraph` with the above nodes. Notice that the graph doesn't have [conditional edges](#) for routing! This is because control flow is defined with `Command` inside `nodeA`.

```
const graph = new StateGraph(State)
  .addNode("nodeA", nodeA, {
    ends: ["nodeB", "nodeC"],
  })
  .addNode("nodeB", nodeB)
  .addNode("nodeC", nodeC)
  .addEdge(START, "nodeA")
  .compile();
```

주의

You might have noticed that we used `ends` to specify which nodes `nodeA` can navigate to. This is necessary for the graph rendering and tells `LangGraph` that `nodeA` can navigate to `nodeB` and `nodeC`.

```
import * as fs from "node:fs/promises";

const drawableGraph = await graph.getGraphAsync();
const image = await drawableGraph.drawMermaidPng();
const imageBuffer = new Uint8Array(await image.arrayBuffer());

await fs.writeFile("graph.png", imageBuffer);
```

If we run the graph multiple times, we'd see it take different paths (A -> B or A -> C) based on the random choice in node A.

```
const result = await graph.invoke({ foo: "" });
console.log(result);
```

```
Called A
Called C
```



```
{ foo: 'cc' }
```

Navigate to a node in a parent graph

If you are using [subgraphs](#), you might want to navigate from a node within a subgraph to a different subgraph (i.e. a different node in the parent graph). To do so, you can specify `graph=Command.PARENT` in `Command`:

python

```
def my_node(state: State) -> Command[Literal["other_subgraph"]]:
    return Command(
        update={"foo": "bar"},
        goto="other_subgraph", # where `other_subgraph` is a node in the parent graph
        graph=Command.PARENT
    )
```

js

```
const myNode = (state: State): Command => {
  return new Command({
    update: { foo: "bar" },
    goto: "otherSubgraph", // where `otherSubgraph` is a node in the parent graph
    graph: Command.PARENT
  });
};
```

Let's demonstrate this using the above example. We'll do so by changing `nodeA` in the above example into a single-node graph that we'll add as a subgraph to our parent graph.

주의

State updates with `Command.PARENT` When you send updates from a subgraph node to a parent graph node for a key that's shared by both parent and subgraph [state schemas](#), you **must** define a [reducer](#) for the key you're updating in the parent graph state. See the example below.

python

```
import operator
from typing_extensions import Annotated

class State(TypedDict):
    # NOTE: we define a reducer here
    foo: Annotated[str, operator.add] # [!code highlight]
```

```

def node_a(state: State):
    print("Called A")
    value = random.choice(["a", "b"])
    # this is a replacement for a conditional edge function
    if value == "a":
        goto = "node_b"
    else:
        goto = "node_c"

# note how Command allows you to BOTH update the graph state AND route to the next node
return Command(
    update={"foo": value},
    goto=goto,
    # this tells LangGraph to navigate to node_b or node_c in the parent graph
    # NOTE: this will navigate to the closest parent graph relative to the subgraph
    graph=Command.PARENT, # [!code highlight]
)

subgraph = StateGraph(State).add_node(node_a).add_edge(START, "node_a").compile()

def node_b(state: State):
    print("Called B")
    # NOTE: since we've defined a reducer, we don't need to manually append
    # new characters to existing 'foo' value. instead, reducer will append these
    # automatically (via operator.add)
    return {"foo": "b"} # [!code highlight]

def node_c(state: State):
    print("Called C")
    return {"foo": "c"} # [!code highlight]

builder = StateGraph(State)
builder.add_edge(START, "subgraph")
builder.add_node("subgraph", subgraph)
builder.add_node(node_b)
builder.add_node(node_c)

graph = builder.compile()

```

```
graph.invoke({"foo": ""})
```

```
Called A
Called C
```

js

```

import { StateGraph, StateSchema, ReducedValue, GraphNode, Command, START } from "@langchain/langgraph";
import * as z from "zod";

const State = new StateSchema({
  // NOTE: we define a reducer here
  foo: new ReducedValue( // [!code highlight]
    z.string().default(""),
    { reducer: (x, y) => x + y }
  ),
});

const nodeA: GraphNode<{ InputSchema: typeof State; Nodes: "nodeB" | "nodeC" }> = (state) => {
  console.log("Called A");
  const value = Math.random() > 0.5 ? "nodeB" : "nodeC";

  // note how Command allows you to BOTH update the graph state AND route to the next node
  return new Command({
    update: { foo: "a" }, // [!code highlight]
    goto: value,
    // this tells LangGraph to navigate to nodeB or nodeC in the parent graph
    // NOTE: this will navigate to the closest parent graph relative to the subgraph
    graph: Command.PARENT,
  });
};

const subgraph = new StateGraph(State)
  .addNode("nodeA", nodeA, { ends: ["nodeB", "nodeC"] })
  .addEdge(START, "nodeA")
  .compile();

const nodeB: GraphNode<typeof State> = (state) => {
  console.log("Called B"); // [!code highlight]
  // NOTE: since we've defined a reducer, we don't need to manually append
  // new characters to existing 'foo' value. instead, reducer will append these
  // automatically
  return { foo: "b" };
}; // [!code highlight]

const nodeC: GraphNode<typeof State> = (state) => {
  console.log("Called C");
  return { foo: "c" };
};

const graph = new StateGraph(State)
  .addNode("subgraph", subgraph, { ends: ["nodeB", "nodeC"] })
  .addNode("nodeB", nodeB)
  .addNode("nodeC", nodeC)

```

```
.addEdge(START, "subgraph")
.compile();
```

```
const result = await graph.invoke({ foo: "" });
console.log(result);
```

```
Called A
Called C
{ foo: 'ac' }
```

Use inside tools

A common use case is updating graph state from inside a tool. For example, in a customer support application you might want to look up customer information based on their account number or ID in the beginning of the conversation. To update the graph state from the tool, you can return

`Command(update={"my_custom_key": "foo", "messages": [...]})` from the tool:

python

```
from langchain.tools import ToolRuntime

@tool
def lookup_user_info(runtime: ToolRuntime):
    """Use this to look up user information to better assist them with their questions."""
    user_info = get_user_info(runtime.server_info.user.identity) # [!code highlight]
    return Command(
        update={
            # update the state keys
            "user_info": user_info,
            # update the message history
            "messages": [ToolMessage("Successfully looked up user information", tool_call_id
=runtime.tool_call_id)]
        }
    )
```

js

```
import { tool } from "@langchain/core/tools";
import { Command } from "@langchain/langgraph";
import * as z from "zod";

const lookupUserInfo = tool(
  async (input, runtime) => {
    const userId = runtime.serverInfo?.user?.identity; // [!code highlight]
    const userInfo = getUserInfo(userId);
```

```

return new Command({
  update: {
    // update the state keys
    userInfo: userInfo,
    // update the message history
    messages: [{
      role: "tool",
      content: "Successfully looked up user information",
      tool_call_id: runtime.toolCall.id
    }]
  }
});
},
{
  name: "lookupUserInfo",
  description: "Use this to look up user information to better assist them with their questions.",
  schema: z.object({}),
}
);

```

주의

You MUST include `messages` (or any state key used for the message history) in `Command.update` when returning `@[Command]` from a tool and the list of messages in `messages` MUST contain a `ToolMessage`. This is necessary for the resulting message history to be valid (LLM providers require AI messages with tool calls to be followed by the tool result messages).

If you are using tools that update state via `@[Command]`, we recommend using prebuilt `@[ToolNode]` which automatically handles tools returning `@[Command]` objects and propagates them to the graph state. If you're writing a custom node that calls tools, you would need to manually propagate `@[Command]` objects returned by the tools as the update from the node.

Visualize your graph

Here we demonstrate how to visualize the graphs you create.

You can visualize any arbitrary [Graph](#), including `@[StateGraph]`.

python

Let's have some fun by drawing fractals :).

```

import random
from typing import Annotated, Literal
from typing_extensions import TypedDict

```

```

from langgraph.graph import StateGraph, START, END
from langgraph.graph.message import add_messages

class State(TypedDict):
    messages: Annotated[list, add_messages]

class MyNode:
    def __init__(self, name: str):
        self.name = name
    def __call__(self, state: State):
        return {"messages": [("assistant", f"Called node {self.name}")]}}

def route(state) -> Literal["entry_node", END]:
    if len(state["messages"]) > 10:
        return END
    return "entry_node"

def add_fractal_nodes(builder, current_node, level, max_level):
    if level > max_level:
        return
    # Number of nodes to create at this level
    num_nodes = random.randint(1, 3) # Adjust randomness as needed
    for i in range(num_nodes):
        nm = ["A", "B", "C"][i]
        node_name = f"node_{current_node}_{nm}"
        builder.add_node(node_name, MyNode(node_name))
        builder.add_edge(current_node, node_name)
        # Recursively add more nodes
        r = random.random()
        if r > 0.2 and level + 1 < max_level:
            add_fractal_nodes(builder, node_name, level + 1, max_level)
        elif r > 0.05:
            builder.add_conditional_edges(node_name, route, node_name)
        else:
            # End
            builder.add_edge(node_name, END)

def build_fractal_graph(max_level: int):
    builder = StateGraph(State)
    entry_point = "entry_node"
    builder.add_node(entry_point, MyNode(entry_point))
    builder.add_edge(START, entry_point)
    add_fractal_nodes(builder, entry_point, 1, max_level)
    # Optional: set a finish point if required
    builder.add_edge(entry_point, END) # or any specific node
    return builder.compile()

app = build_fractal_graph(3)

```

js

Let's create a simple example graph to demonstrate visualization.

```
import { StateGraph, StateSchema, MessagesValue, ReducedValue, GraphNode, ConditionalEdgeRouter, START, END } from "@langchain/langgraph";
import * as z from "zod";

const State = new StateSchema({
  messages: MessagesValue,
  value: new ReducedValue(
    z.number().default(0),
    { reducer: (x, y) => x + y }
  ),
});

const node1: GraphNode<typeof State> = (state) => {
  return { value: state.value + 1 };
};

const node2: GraphNode<typeof State> = (state) => {
  return { value: state.value * 2 };
};

const router: ConditionalEdgeRouter<{ InputSchema: typeof State; Nodes: "node2" }> = (state) => {
  if (state.value < 10) {
    return "node2";
  }
  return END;
};

const app = new StateGraph(State)
  .addNode("node1", node1)
  .addNode("node2", node2)
  .addEdge(START, "node1")
  .addConditionalEdges("node1", router)
  .addEdge("node2", "node1")
  .compile();
```

Mermaid

We can also convert a graph class into Mermaid syntax.

python

```
print(app.get_graph().draw_mermaid())
```

```

%%{init: {'flowchart': {'curve': 'linear'}}}%
graph TD;
    tart__([<p>__start__</p>]):::first
    ry_node(entry_node)
    e_entry_node_A(node_entry_node_A)
    e_entry_node_B(node_entry_node_B)
    e_node_entry_node_B_A(node_node_entry_node_B_A)
    e_node_entry_node_B_B(node_node_entry_node_B_B)
    e_node_entry_node_B_C(node_node_entry_node_B_C)
    nd__([<p>__end__</p>]):::last
    tart__ --> entry_node;
    ry_node --> __end__;
    ry_node --> node_entry_node_A;
    ry_node --> node_entry_node_B;
    e_entry_node_B --> node_node_entry_node_B_A;
    e_entry_node_B --> node_node_entry_node_B_B;
    e_entry_node_B --> node_node_entry_node_B_C;
    e_entry_node_A --> entry_node;
    e_entry_node_A --> __end__;
    e_node_entry_node_B_A --> entry_node;
    e_node_entry_node_B_A --> __end__;
    e_node_entry_node_B_B --> entry_node;
    e_node_entry_node_B_B --> __end__;
    e_node_entry_node_B_C --> entry_node;
    e_node_entry_node_B_C --> __end__;
    ssDef default fill:#f2f0ff,line-height:1.2
    ssDef first fill-opacity:0
    ssDef last fill:#bfb6fc

```

js

```

const drawableGraph = await app.getGraphAsync();
console.log(drawableGraph.drawMermaid());

```

```

%%{init: {'flowchart': {'curve': 'linear'}}}%
graph TD;
    tart__([<p>__start__</p>]):::first
    e1(node1)
    e2(node2)
    nd__([<p>__end__</p>]):::last
    tart__ --> node1;
    e1 --> node2;
    e1 --> __end__;
    e2 --> node1;
    ssDef default fill:#f2f0ff,line-height:1.2
    ssDef first fill-opacity:0
    ssDef last fill:#bfb6fc

```


PNG

python

If preferred, we could render the Graph into a `.png`. Here we could use three options:

- Using Mermaid.ink API (does not require additional packages)
- Using Mermaid + Pyppeteer (requires `pip install pyppeteer`)
- Using graphviz (which requires `pip install graphviz`)

Using Mermaid.Ink

By default, `draw_mermaid_png()` uses Mermaid.Ink's API to generate the diagram.

```
from IPython.display import Image, display
from langchain_core.runnables.graph import CurveStyle, MermaidDrawMethod, NodeStyles

display(Image(app.get_graph().draw_mermaid_png()))
```



Fractal graph visualization

Using Mermaid + Pyppeteer

```
import nest_asyncio

nest_asyncio.apply() # Required for Jupyter Notebook to run async functions

display(
    Image(
        app.get_graph().draw_mermaid_png(
            curve_style=CurveStyle.LINEAR,
            node_colors=NodeStyles(first="#ffdfba", last="#baffc9", default="#fad7de"),
            wrap_label_n_words=9,
            output_file_path=None,
            draw_method=MermaidDrawMethod.PYPETEER,
            background_color="white",
            padding=10,
        )
    )
)
```

Using Graphviz

```
try:
    display(Image(app.get_graph().draw_png()))
except ImportError:
    print(
        "You likely need to install dependencies for pygraphviz, see more here https://github.com
```

```
b.com/pygraphviz/pygraphviz/blob/main/INSTALL.txt"  
)
```

js

If preferred, we could render the Graph into a `.png`. This uses the Mermaid.ink API to generate the diagram.

```
import * as fs from "node:fs/promises";  
  
const drawableGraph = await app.getGraphAsync();  
const image = await drawableGraph.drawMermaidPng();  
const imageBuffer = new Uint8Array(await image.arrayBuffer());  
  
await fs.writeFile("graph.png", imageBuffer);
```

Subgraphs

langgraph/use-subgraphs.mdx

This guide explains the mechanics of using subgraphs. A subgraph is a [graph](#) that is used as a [node](#) in another graph.

Subgraphs are useful for:

- Building [multi-agent systems](#) - Reusing a set of nodes in multiple graphs
- Distributing development: when you want different teams to work on different parts of the graph independently, you can define each part as a subgraph, and as long as the subgraph interface (the input and output schemas) is respected, the parent graph can be built without knowing any details of the subgraph

Setup

python

```
```bash pip install -U langgraph
```

```
```bash uv  
uv add langgraph
```

js

```
npm install @langchain/langgraph
```

팁

Set up LangSmith for LangGraph development Sign up for [LangSmith](#) to quickly spot issues and improve the performance of your LangGraph projects. LangSmith lets you use trace data to debug, test, and monitor your LLM apps built with LangGraph—read more about [how to get started with LangSmith](#).

Define subgraph communication

When adding subgraphs, you need to define how the parent graph and the subgraph communicate:

Pattern	When to use	State schemas
Call a subgraph inside a node	Parent and subgraph have different state schemas (no shared keys), or you need to transform state between them	You write a wrapper function that maps parent state to subgraph input and subgraph output back to parent state
Add a subgraph as a node	Parent and subgraph share state keys —the subgraph reads from and writes to the same channels as the parent	You pass the compiled subgraph directly to <code>add_node</code> —no wrapper function needed

Call a subgraph inside a node

When the parent graph and subgraph have **different state schemas** (no shared keys), invoke the subgraph inside a node function. This is common when you want to keep a private message history for each agent in a [multi-agent](#) system.

The node function transforms the parent state to the subgraph state before invoking the subgraph, and transforms the results back to the parent state before returning.

python

```

from typing_extensions import TypedDict
from langgraph.graph.state import StateGraph, START

class SubgraphState(TypedDict):
    bar: str

# Subgraph

def subgraph_node_1(state: SubgraphState):
    return {"bar": "hi! " + state["bar"]}

subgraph_builder = StateGraph(SubgraphState)
subgraph_builder.add_node(subgraph_node_1)
subgraph_builder.add_edge(START, "subgraph_node_1")
subgraph = subgraph_builder.compile()

# Parent graph

class State(TypedDict):
    foo: str

def call_subgraph(state: State):
    # Transform the state to the subgraph state
    subgraph_output = subgraph.invoke({"bar": state["foo"]}) # [!code highlight]
    # Transform response back to the parent state
    return {"foo": subgraph_output["bar"]}

builder = StateGraph(State)

```

```
builder.add_node("node_1", call_subgraph)
builder.add_edge(START, "node_1")
graph = builder.compile()
```

js

```
import { StateGraph, StateSchema, START } from "@langchain/langgraph";
import * as z from "zod";

const SubgraphState = new StateSchema({
  bar: z.string(),
});

// Subgraph
const subgraphBuilder = new StateGraph(SubgraphState)
  .addNode("subgraphNode1", (state) => {
    return { bar: "hi! " + state.bar };
  })
  .addEdge(START, "subgraphNode1");

const subgraph = subgraphBuilder.compile();

// Parent graph
const State = new StateSchema({
  foo: z.string(),
});

// Transform the state to the subgraph state and back
const builder = new StateGraph(State)
  .addNode("node1", async (state) => {
    const subgraphOutput = await subgraph.invoke({ bar: state.foo });
    return { foo: subgraphOutput.bar };
  })
  .addEdge(START, "node1");

const graph = builder.compile();
```

Full example: different state schemas

python

```
from typing_extensions import TypedDict
from langgraph.graph.state import StateGraph, START
from langgraph.stream import UpdatesTransformer

# Define subgraph
class SubgraphState(TypedDict):
```

```

# note that none of these keys are shared with the parent graph state
bar: str
baz: str

def subgraph_node_1(state: SubgraphState):
    return {"baz": "baz"}

def subgraph_node_2(state: SubgraphState):
    return {"bar": state["bar"] + state["baz"]}

subgraph_builder = StateGraph(SubgraphState)
subgraph_builder.add_node(subgraph_node_1)
subgraph_builder.add_node(subgraph_node_2)
subgraph_builder.add_edge(START, "subgraph_node_1")
subgraph_builder.add_edge("subgraph_node_1", "subgraph_node_2")
subgraph = subgraph_builder.compile()

# Define parent graph
class ParentState(TypedDict):
    foo: str

def node_1(state: ParentState):
    return {"foo": "hi! " + state["foo"]}

def node_2(state: ParentState):
    # Transform the state to the subgraph state
    response = subgraph.invoke({"bar": state["foo"]})
    # Transform response back to the parent state
    return {"foo": response["bar"]}

builder = StateGraph(ParentState)
builder.add_node("node_1", node_1)
builder.add_node("node_2", node_2)
builder.add_edge(START, "node_1")
builder.add_edge("node_1", "node_2")
graph = builder.compile()

stream = graph.stream_events({"foo": "foo"}, version="v3", transformers=[UpdatesTransformer])
for event in stream:
    if event["method"] == "updates":
        print(event["params"]["namespace"], event["params"]["data"])

```

```

[] {'node_1': {'foo': 'hi! foo'}}
['node_2:577b710b-64ae-31fb-9455-6a4d4cc2b0b9'] {'subgraph_node_1': {'baz': 'baz'}}
['node_2:577b710b-64ae-31fb-9455-6a4d4cc2b0b9'] {'subgraph_node_2': {'bar': 'hi! foobaz'}}
[] {'node_2': {'foo': 'hi! foobaz'}}

```

```

import { StateGraph, StateSchema, START } from "@langchain/langgraph";
import * as z from "zod";

// Define subgraph
const SubgraphState = new StateSchema({
  // note that none of these keys are shared with the parent graph state
  bar: z.string(),
  baz: z.string(),
});

const subgraphBuilder = new StateGraph(SubgraphState)
  .addNode("subgraphNode1", (state) => {
    return { baz: "baz" };
  })
  .addNode("subgraphNode2", (state) => {
    return { bar: state.bar + state.baz };
  })
  .addEdge(START, "subgraphNode1")
  .addEdge("subgraphNode1", "subgraphNode2");

const subgraph = subgraphBuilder.compile();

// Define parent graph
const ParentState = new StateSchema({
  foo: z.string(),
});

const builder = new StateGraph(ParentState)
  .addNode("node1", (state) => {
    return { foo: "hi! " + state.foo };
  })
  .addNode("node2", async (state) => {
    const response = await subgraph.invoke({ bar: state.foo }); // [!code highlight]
    return { foo: response.bar }; // [!code highlight]
  })
  .addEdge(START, "node1")
  .addEdge("node1", "node2");

const graph = builder.compile();

const stream = await graph.streamEvents(
  { foo: "foo" },
  { subgraphs: true, version: "v3" }
);
for await (const message of stream.messages) {
  for await (const token of message.text) {
    process.stdout.write(token);
  }
}

```

1. Transform the state to the subgraph state
2. Transform response back to the parent state

```
[[], { node1: { foo: 'hi! foo' } }] [['node2:9c36dd0f-151a-cbad-fa2f851f9ab7'], {
subgraphNode1: { baz: 'baz' } }] [['node2:9c36dd0f-151a-cbad-fa2f851f9ab7'], {
subgraphNode2: { bar: 'hi! foobaz' } }] [[], { node2: { foo: 'hi! foobaz' } }]
```

Full example: different state schemas (two levels of subgraphs)

This is an example with two levels of subgraphs: parent -> child -> grandchild.

python

```
# Grandchild graph
from typing_extensions import TypedDict
from langgraph.graph.state import StateGraph, START, END
from langgraph.stream import UpdatesTransformer

class GrandChildState(TypedDict):
    my_grandchild_key: str

def grandchild_1(state: GrandChildState) -> GrandChildState:
    # NOTE: child or parent keys will not be accessible here
    return {"my_grandchild_key": state["my_grandchild_key"] + ", how are you"}

grandchild = StateGraph(GrandChildState)
grandchild.add_node("grandchild_1", grandchild_1)

grandchild.add_edge(START, "grandchild_1")
grandchild.add_edge("grandchild_1", END)

grandchild_graph = grandchild.compile()

# Child graph
class ChildState(TypedDict):
    my_child_key: str

def call_grandchild_graph(state: ChildState) -> ChildState:
    # NOTE: parent or grandchild keys won't be accessible here
    grandchild_graph_input = {"my_grandchild_key": state["my_child_key"]}
    grandchild_graph_output = grandchild_graph.invoke(grandchild_graph_input)
    return {"my_child_key": grandchild_graph_output["my_grandchild_key"] + " today?"}

child = StateGraph(ChildState)
# We're passing a function here instead of just compiled graph (`grandchild_graph`)
child.add_node("child_1", call_grandchild_graph)
child.add_edge(START, "child_1")
child.add_edge("child_1", END)
child_graph = child.compile()
```



```

# Parent graph
class ParentState(TypedDict):
    my_key: str

def parent_1(state: ParentState) -> ParentState:
    # NOTE: child or grandchild keys won't be accessible here
    return {"my_key": "hi " + state["my_key"]}

def parent_2(state: ParentState) -> ParentState:
    return {"my_key": state["my_key"] + " bye!"}

def call_child_graph(state: ParentState) -> ParentState:
    child_graph_input = {"my_child_key": state["my_key"]}
    child_graph_output = child_graph.invoke(child_graph_input)
    return {"my_key": child_graph_output["my_child_key"]}

parent = StateGraph(ParentState)
parent.add_node("parent_1", parent_1)
# We're passing a function here instead of just a compiled graph (`child_graph`)
parent.add_node("child", call_child_graph)
parent.add_node("parent_2", parent_2)

parent.add_edge(START, "parent_1")
parent.add_edge("parent_1", "child")
parent.add_edge("child", "parent_2")
parent.add_edge("parent_2", END)

parent_graph = parent.compile()

stream = parent_graph.stream_events({"my_key": "Bob"}, version="v3", transformers=[UpdatesTransformer])
for event in stream:
    if event["method"] == "updates":
        print(event["params"]["namespace"], event["params"]["data"])

```

```

[] {'parent_1': {'my_key': 'hi Bob'}}
['child:2e26e9ce-602f-862c-aa66-1ea5a4655e3b', 'child_1:781bb3b1-3971-84ce-810b-acf819a03f9c'] {'grandchild_1': {'my_grandchild_key': 'hi Bob, how are you'}}
['child:2e26e9ce-602f-862c-aa66-1ea5a4655e3b'] {'child_1': {'my_child_key': 'hi Bob, how are you today?'}}
[] {'child': {'my_key': 'hi Bob, how are you today?'}}
[] {'parent_2': {'my_key': 'hi Bob, how are you today? bye!'}}

```

js

```

import { StateGraph, StateSchema, START, END } from "@langchain/langgraph";
import * as z from "zod";

```

```

// Grandchild graph
const GrandChildState = new StateSchema({
  myGrandchildKey: z.string(),
});

const grandchild = new StateGraph(GrandChildState)
  .addNode("grandchild1", (state) => {
    // NOTE: child or parent keys will not be accessible here
    return { myGrandchildKey: state.myGrandchildKey + ", how are you" };
  })
  .addEdge(START, "grandchild1")
  .addEdge("grandchild1", END);

const grandchildGraph = grandchild.compile();

// Child graph
const ChildState = new StateSchema({
  myChildKey: z.string(),
});

const child = new StateGraph(ChildState)
  .addNode("child1", async (state) => {
    // NOTE: parent or grandchild keys won't be accessible here
    const grandchildGraphInput = { myGrandchildKey: state.myChildKey }; // [!code highlight]

    const grandchildGraphOutput = await grandchildGraph.invoke(grandchildGraphInput);
    return { myChildKey: grandchildGraphOutput.myGrandchildKey + " today?" }; // [!code highlight]
  }) // [!code highlight]
  .addEdge(START, "child1")
  .addEdge("child1", END);

const childGraph = child.compile();

// Parent graph
const ParentState = new StateSchema({
  myKey: z.string(),
});

const parent = new StateGraph(ParentState)
  .addNode("parent1", (state) => {
    // NOTE: child or grandchild keys won't be accessible here
    return { myKey: "hi " + state.myKey };
  })
  .addNode("child", async (state) => {
    const childGraphInput = { myChildKey: state.myKey }; // [!code highlight]
    const childGraphOutput = await childGraph.invoke(childGraphInput);
    return { myKey: childGraphOutput.myChildKey }; // [!code highlight]
  }) // [!code highlight]
  .addNode("parent2", (state) => {
    return { myKey: state.myKey + " bye!" };
  });

```

```

})
.addEdge(START, "parent1")
.addEdge("parent1", "child")
.addEdge("child", "parent2")
.addEdge("parent2", END);

const parentGraph = parent.compile();

const stream = await parentGraph.streamEvents(
  { myKey: "Bob" },
  { subgraphs: true, version: "v3" }
);
for await (const message of stream.messages) {
  for await (const token of message.text) {
    process.stdout.write(token);
  }
}

```

1. We're transforming the state from the child state channels (`myChildKey`) to the grandchild state channels (`myGrandchildKey`)
2. We're transforming the state from the grandchild state channels (`myGrandchildKey`) back to the child state channels (`myChildKey`)
3. We're passing a function here instead of just compiled graph (`grandchildGraph`)
4. We're transforming the state from the parent state channels (`myKey`) to the child state channels (`myChildKey`)
5. We're transforming the state from the child state channels (`myChildKey`) back to the parent state channels (`myKey`)
6. We're passing a function here instead of just a compiled graph (`childGraph`)

```

[[[], { parent1: { myKey: 'hi Bob' } }]] [['child:2e26e9ce-602f-862c-aa66-1ea5a4655e3b',
'child1:781bb3b1-3971-84ce-810b-acf819a03f9c'], { grandchild1: { myGrandchildKey: 'hi Bob, how
are you' } }]] [['child:2e26e9ce-602f-862c-aa66-1ea5a4655e3b'], { child1: { myChildKey: 'hi Bob,
how are you today?' } }]] [[[], { child: { myKey: 'hi Bob, how are you today?' } }]] [[[], {
parent2: { myKey: 'hi Bob, how are you today? bye!' } }]]

```

Add a subgraph as a node

When the parent graph and subgraph **share state keys**, you can pass a compiled subgraph directly to `add_node`. No wrapper function is needed—the subgraph reads from and writes to the parent's state channels automatically. For example, in [multi-agent](#) systems, the agents often communicate over a shared [messages](#) key.



SQL agent graph

If your subgraph shares state keys with the parent graph, you can follow these steps to add it to your graph:

python

1. Define the subgraph workflow (`subgraph_builder` in the example below) and compile it
2. Pass compiled subgraph to the `@[ add_node ]` method when defining the parent graph workflow

```
from typing_extensions import TypedDict
from langgraph.graph.state import StateGraph, START

class State(TypedDict):
    foo: str

# Subgraph

def subgraph_node_1(state: State):
    return {"foo": "hi! " + state["foo"]}

subgraph_builder = StateGraph(State)
subgraph_builder.add_node(subgraph_node_1)
subgraph_builder.add_edge(START, "subgraph_node_1")
subgraph = subgraph_builder.compile()

# Parent graph

builder = StateGraph(State)
builder.add_node("node_1", subgraph) # [!code highlight]
builder.add_edge(START, "node_1")
graph = builder.compile()
```

js

1. Define the subgraph workflow (`subgraphBuilder` in the example below) and compile it
2. Pass compiled subgraph to the `.addNode` method when defining the parent graph workflow

```
import { StateGraph, StateSchema, START } from "@langchain/langgraph";
import * as z from "zod";

const State = new StateSchema({
  foo: z.string(),
});

// Subgraph
const subgraphBuilder = new StateGraph(State)
  .addNode("subgraphNode1", (state) => {
    return { foo: "hi! " + state.foo };
  })
  .addEdge(START, "subgraphNode1");
```

```

const subgraph = subgraphBuilder.compile();

// Parent graph
const builder = new StateGraph(State)
  .addNode("node1", subgraph)
  .addEdge(START, "node1");

const graph = builder.compile();

```

Full example: shared state schemas

python

```

from typing_extensions import TypedDict
from langgraph.graph.state import StateGraph, START
from langgraph.stream import UpdatesTransformer

# Define subgraph
class SubgraphState(TypedDict):
    foo: str # shared with parent graph state
    bar: str # private to SubgraphState

def subgraph_node_1(state: SubgraphState):
    return {"bar": "bar"}

def subgraph_node_2(state: SubgraphState):
    # note that this node is using a state key ('bar') that is only available in the subgraph
    # and is sending update on the shared state key ('foo')
    return {"foo": state["foo"] + state["bar"]}

subgraph_builder = StateGraph(SubgraphState)
subgraph_builder.add_node(subgraph_node_1)
subgraph_builder.add_node(subgraph_node_2)
subgraph_builder.add_edge(START, "subgraph_node_1")
subgraph_builder.add_edge("subgraph_node_1", "subgraph_node_2")
subgraph = subgraph_builder.compile()

# Define parent graph
class ParentState(TypedDict):
    foo: str

def node_1(state: ParentState):
    return {"foo": "hi! " + state["foo"]}

builder = StateGraph(ParentState)
builder.add_node("node_1", node_1)
builder.add_node("node_2", subgraph)

```

```

builder.add_edge(START, "node_1")
builder.add_edge("node_1", "node_2")
graph = builder.compile()

stream = graph.stream_events({"foo": "foo"}, version="v3", transformers=[UpdatesTransformer])
for event in stream:
    if event["method"] == "updates" and not event["params"]["namespace"]:
        print(event["params"]["data"])

```

```

{'node_1': {'foo': 'hi! foo'}}
{'node_2': {'foo': 'hi! foobar'}}

```

js

```

import { StateGraph, StateSchema, START } from "@langchain/langgraph";
import * as z from "zod";

// Define subgraph
const SubgraphState = new StateSchema({
  foo: z.string(),    // [!code highlight]
  bar: z.string(),    // [!code highlight]
});

const subgraphBuilder = new StateGraph(SubgraphState)
  .addNode("subgraphNode1", (state) => {
    return { bar: "bar" };
  })
  .addNode("subgraphNode2", (state) => {
    // note that this node is using a state key ('bar') that is only available in the subgraph
    // and is sending update on the shared state key ('foo')
    return { foo: state.foo + state.bar };
  })
  .addEdge(START, "subgraphNode1")
  .addEdge("subgraphNode1", "subgraphNode2");

const subgraph = subgraphBuilder.compile();

// Define parent graph
const ParentState = new StateSchema({
  foo: z.string(),
});

const builder = new StateGraph(ParentState)
  .addNode("node1", (state) => {
    return { foo: "hi! " + state.foo };
  })
  .addNode("node2", subgraph)

```

```

.addEdge(START, "node1")
.addEdge("node1", "node2");

const graph = builder.compile();

const stream = await graph.streamEvents({ foo: "foo" }, { version: "v3" });
for await (const message of stream.messages) {
  for await (const token of message.text) {
    process.stdout.write(token);
  }
}

```

1. This key is shared with the parent graph state
2. This key is private to the `SubgraphState` and is not visible to the parent graph

```
{ node1: { foo: 'hi! foo' } } { node2: { foo: 'hi! foobar' } }
```

Subgraph persistence

When you use a subgraph, you need to decide what happens to its internal data between calls. Consider a customer support bot that delegates to specialist subagents: should the "billing expert" subagent remember the customer's earlier questions, or start fresh each time it's called?

The `checkpointer` parameter on `.compile()` controls subgraph persistence:

Mode	checkpoint ter=	Behavior
Per- invocation	None (default)	Each call starts fresh and inherits the parent's checkpointer to support interrupts and durable execution within a single call.
Per- thread	True	State accumulates across calls on the same thread. Each call picks up where the last one left off.
Stateless	False	No checkpointing at all—runs like a plain function call. No interrupts or durable execution.

Per-invocation is the right choice for most applications, including [multi-agent](#) systems where subagents handle independent requests. Use per-thread when a subagent needs multi-turn conversation memory (for example, a research assistant that builds context over several exchanges).

참고

The parent graph must be compiled with a checkpointer for subgraph persistence features (interrupts, state inspection, per-thread memory) to work. See [persistence](#).

정보

The examples below use LangChain's `@create_agent`, which is a common way to build agents. `create_agent` produces a [LangGraph graph](#) under the hood, so all subgraph persistence concepts apply directly. If you're building with raw `LangGraph` `StateGraph`, the same patterns and configuration options apply—see the [Graph API](#) for details.

Stateful

Stateful subgraphs inherit the parent graph's checkpoint, which enables [interrupts](#), [persistence](#), and state inspection. The two stateful modes differ in how long state is retained.

Per-invocation (default)

팁

This is the recommended mode for most applications, including [multi-agent](#) systems where subagents are invoked as tools. It supports [interrupts](#), [persistence](#), and parallel calls while keeping each invocation isolated.

Use per-invocation persistence when each call to the subgraph is independent and the subagent doesn't need to remember anything from previous calls. This is the most common pattern, especially for [multi-agent](#) systems where subagents handle one-off requests like "look up this customer's order" or "summarize this document."

Omit `checkpointer` or set it to `None`. Each call starts fresh, but within a single call the subgraph inherits the parent's checkpoint and can use `interrupt()` to pause and resume.

The following examples use two subagents (fruit expert, veggie expert) wrapped as tools for an outer agent:

python

```
from langchain.agents import create_agent
from langchain.tools import tool
from langgraph.checkpoint.memory import MemorySaver
from langgraph.types import Command, interrupt

@tool
def fruit_info(fruit_name: str) -> str:
    """Look up fruit info."""
    return f"Info about {fruit_name}"

@tool
def veggie_info(veggie_name: str) -> str:
```



```

    """Look up veggie info."""
    return f"Info about {veggie_name}"

# Subagents - no checkpointer setting (inherits parent)
fruit_agent = create_agent(
    model="gpt-5.4-mini",
    tools=[fruit_info],
    prompt="You are a fruit expert. Use the fruit_info tool. Respond in one sentence.",
)

veggie_agent = create_agent(
    model="gpt-5.4-mini",
    tools=[veggie_info],
    prompt="You are a veggie expert. Use the veggie_info tool. Respond in one sentence.",
)

# Wrap subagents as tools for the outer agent
@tool
def ask_fruit_expert(question: str) -> str:
    """Ask the fruit expert. Use for ALL fruit questions."""
    response = fruit_agent.invoke(
        {"messages": [{"role": "user", "content": question}]}
    )
    return response["messages"][-1].content

@tool
def ask_veggie_expert(question: str) -> str:
    """Ask the veggie expert. Use for ALL veggie questions."""
    response = veggie_agent.invoke(
        {"messages": [{"role": "user", "content": question}]}
    )
    return response["messages"][-1].content

# Outer agent with checkpointer
agent = create_agent(
    model="gpt-5.4-mini",
    tools=[ask_fruit_expert, ask_veggie_expert],
    prompt=(
        "You have two experts: ask_fruit_expert and ask_veggie_expert. "
        "ALWAYS delegate questions to the appropriate expert."
    ),
    checkpointer=MemorySaver(),
)

```

Interrupts

Each invocation can use `interrupt()` to pause and resume. Add `interrupt()` to a tool function to require user approval before proceeding:

```
@tool
def fruit_info(fruit_name: str) -> str:
    """Look up fruit info."""
    interrupt("continue?") # [!code highlight]
    return f"Info about {fruit_name}"
```

Multi-turn

Each invocation starts with a fresh subagent state. The subagent does not remember previous calls:

```
config = {"configurable": {"thread_id": "1"}}

# First call
response = agent.invoke(
    {"messages": [{"role": "user", "content": "Tell me about apples"}]},
    config=config,
)
# Subagent message count: 4

# Second call - subagent starts fresh, no memory of apples
response = agent.invoke(
    {"messages": [{"role": "user", "content": "Now tell me about bananas"}]},
    config=config,
)
# Subagent message count: 4 (still fresh!)
```

Multiple subgraph calls

Multiple calls to the same subgraph work without conflicts, since each invocation gets its own checkpoint namespace:

```
config = {"configurable": {"thread_id": "1"}}

# LLM calls ask_fruit_expert for both apples and bananas
response = agent.invoke(
    {"messages": [{"role": "user", "content": "Tell me about apples and bananas"}]},
    config=config,
)
# Subagent message count: 4 (apples - fresh)
# Subagent message count: 4 (bananas - fresh)
```

js

```
import { createAgent, tool } from "langchain";
import { MemorySaver, Command, interrupt } from "@langchain/langgraph";
import * as z from "zod";
```

```

const fruitInfo = tool(
  (input) => `Info about ${input.fruitName}`,
  {
    name: "fruit_info",
    description: "Look up fruit info.",
    schema: z.object({ fruitName: z.string() }),
  }
);

const veggieInfo = tool(
  (input) => `Info about ${input.veggieName}`,
  {
    name: "veggie_info",
    description: "Look up veggie info.",
    schema: z.object({ veggieName: z.string() }),
  }
);

// Subagents - no checkpoint setting (inherits parent)
const fruitAgent = createAgent({
  model: "gpt-5.4-mini",
  tools: [fruitInfo],
  prompt: "You are a fruit expert. Use the fruit_info tool. Respond in one sentence.",
});

const veggieAgent = createAgent({
  model: "gpt-5.4-mini",
  tools: [veggieInfo],
  prompt: "You are a veggie expert. Use the veggie_info tool. Respond in one sentence.",
});

// Wrap subagents as tools for the outer agent
const askFruitExpert = tool(
  async (input) => {
    const response = await fruitAgent.invoke({
      messages: [{ role: "user", content: input.question }],
    });
    return response.messages[response.messages.length - 1].content;
  },
  {
    name: "ask_fruit_expert",
    description: "Ask the fruit expert. Use for ALL fruit questions.",
    schema: z.object({ question: z.string() }),
  }
);

const askVeggieExpert = tool(
  async (input) => {
    const response = await veggieAgent.invoke({
      messages: [{ role: "user", content: input.question }],
    });
  },
  {
    name: "ask_veggie_expert",
    description: "Ask the veggie expert. Use for ALL veggie questions.",
    schema: z.object({ question: z.string() }),
  }
);

```

```

    });
    return response.messages[response.messages.length - 1].content;
  },
  {
    name: "ask_veggie_expert",
    description: "Ask the veggie expert. Use for ALL veggie questions.",
    schema: z.object({ question: z.string() }),
  }
});

// Outer agent with checkpointer
const agent = createAgent({
  model: "gpt-5.4-mini",
  tools: [askFruitExpert, askVeggieExpert],
  prompt:
    "You have two experts: ask_fruit_expert and ask_veggie_expert. " +
    "ALWAYS delegate questions to the appropriate expert.",
  checkpointer: new MemorySaver(),
});

```

Interrupts

Each invocation can use `interrupt()` to pause and resume. Add `interrupt()` to a tool function to require user approval before proceeding:

```

const fruitInfo = tool(
  (input) => {
    interrupt("continue?"); // [!code highlight]
    return `Info about ${input.fruitName}`;
  },
  {
    name: "fruit_info",
    description: "Look up fruit info.",
    schema: z.object({ fruitName: z.string() }),
  }
);

```

```

const config = { configurable: { thread_id: "1" } };

// Invoke - the subagent's tool calls interrupt()
let response = await agent.invoke(
  { messages: [{ role: "user", content: "Tell me about apples" }] },
  config,
);
// response contains __interrupt__

// Resume - approve the interrupt
response = await agent.invoke(new Command({ resume: true }), config); // [!code highlight]
// Subagent message count: 4

```

Multi-turn

Each invocation starts with a fresh subagent state. The subagent does not remember previous calls:

```
const config = { configurable: { thread_id: "1" } };

// First call
let response = await agent.invoke(
  { messages: [{ role: "user", content: "Tell me about apples" }] },
  config,
);
// Subagent message count: 4

// Second call - subagent starts fresh, no memory of apples
response = await agent.invoke(
  { messages: [{ role: "user", content: "Now tell me about bananas" }] },
  config,
);
// Subagent message count: 4 (still fresh!)
```

Multiple subgraph calls

Multiple calls to the same subgraph work without conflicts, since each invocation gets its own checkpoint namespace:

```
const config = { configurable: { thread_id: "1" } };

// LLM calls ask_fruit_expert for both apples and bananas
const response = await agent.invoke(
  { messages: [{ role: "user", content: "Tell me about apples and bananas" }] },
  config,
);
// Subagent message count: 4 (apples - fresh)
// Subagent message count: 4 (bananas - fresh)
```

Per-thread

Use per-thread persistence when a subagent needs to remember previous interactions. For example, a research assistant that builds up context over several exchanges, or a coding assistant that tracks what files it has already edited. The subagent's conversation history and data accumulate across calls on the same thread. Each call picks up where the last one left off.

Compile with `checkpointinter=True` to enable this behavior.

주의

Per-thread subgraphs do not support parallel tool calls. When an LLM has access to a per-thread subagent as a tool, it may try to call that tool multiple times in parallel (for example, asking the fruit expert about apples and bananas simultaneously). This causes checkpoint conflicts because both calls write to the same namespace.

The examples below use LangChain's `ToolCallLimitMiddleware` to prevent this. If you're building with pure LangGraph `StateGraph`, you need to prevent parallel tool calls yourself—for example, by configuring your model to disable parallel tool calling or by adding logic to ensure the same subgraph is not invoked multiple times in parallel.

The following examples use a fruit expert subagent compiled with `checkpointer=True`:

python

```
from langchain.agents import create_agent
from langchain.agents.middleware import ToolCallLimitMiddleware
from langchain.tools import tool
from langgraph.checkpoint.memory import MemorySaver
from langgraph.types import Command, interrupt

@tool
def fruit_info(fruit_name: str) -> str:
    """Look up fruit info."""
    return f"Info about {fruit_name}"

# Subagent with checkpointer=True for persistent state
fruit_agent = create_agent(
    model="gpt-5.4-mini",
    tools=[fruit_info],
    prompt="You are a fruit expert. Use the fruit_info tool. Respond in one sentence.",
    checkpointer=True, # [!code highlight]
)

# Wrap subagent as a tool for the outer agent
@tool
def ask_fruit_expert(question: str) -> str:
    """Ask the fruit expert. Use for ALL fruit questions."""
    response = fruit_agent.invoke(
        {"messages": [{"role": "user", "content": question}]}
    )
    return response["messages"][-1].content

# Outer agent with checkpointer
# Use ToolCallLimitMiddleware to prevent parallel calls to per-thread subagents,
# which would cause checkpoint conflicts.
agent = create_agent(
    model="gpt-5.4-mini",
    tools=[ask_fruit_expert],
```

```

prompt="You have a fruit expert. ALWAYS delegate fruit questions to ask_fruit_expert.",
middleware=[ # [!code highlight]
    ToolCallLimitMiddleware(tool_name="ask_fruit_expert", run_limit=1), # [!code highlight]
], # [!code highlight]
checkpointer=MemorySaver(),
)

```

Interrupts

Per-thread subagents support `interrupt()` just like per-invocation. Add `interrupt()` to a tool function to require user approval:

```

@tool
def fruit_info(fruit_name: str) -> str:
    """Look up fruit info."""
    interrupt("continue?") # [!code highlight]
    return f"Info about {fruit_name}"

```

Multi-turn

State accumulates across invocations—the subagent remembers past conversations:

```

config = {"configurable": {"thread_id": "1"}}

# First call
response = agent.invoke(
    {"messages": [{"role": "user", "content": "Tell me about apples"}]},
    config=config,
)
# Subagent message count: 4

# Second call - subagent REMEMBERS apples conversation
response = agent.invoke(
    {"messages": [{"role": "user", "content": "Now tell me about bananas"}]},
    config=config,
)
# Subagent message count: 8 (accumulated!)

```

Multiple subgraph calls

When you have multiple **different** per-thread subgraphs (for example, a fruit expert and a veggie expert), each one needs its own storage space so their checkpoints don't overwrite each other. This is called **namespace isolation**.

If you [call subgraphs inside a node](#), LangGraph assigns namespaces based on call order (first call, second call, etc.). This means reordering your calls can mix up which subgraph loads which

state. To avoid this, wrap each subagent in its own `StateGraph` with a unique node name—this gives each subgraph a stable, unique namespace:

```
from langgraph.graph import MessagesState, StateGraph

def create_sub_agent(model, *, name, **kwargs):
    """Wrap an agent with a unique node name for namespace isolation."""
    agent = create_agent(model=model, name=name, **kwargs)
    return (
        StateGraph(MessagesState)
        .add_node(name, agent) # unique name → stable namespace # [!code highlight]
        .add_edge("__start__", name)
        .compile()
    )

fruit_agent = create_sub_agent(
    "gpt-5.4-mini", name="fruit_agent",
    tools=[fruit_info], prompt="...", checkpointeer=True,
)
veggie_agent = create_sub_agent(
    "gpt-5.4-mini", name="veggie_agent",
    tools=[veggie_info], prompt="...", checkpointeer=True,
)

config = {"configurable": {"thread_id": "1"}}

# First call - LLM calls both fruit and veggie experts
response = agent.invoke(
    {"messages": [{"role": "user", "content": "Tell me about cherries and broccoli"}]},
    config=config,
)
# Fruit subagent message count: 4
# Veggie subagent message count: 4

# Second call - both agents accumulate independently
response = agent.invoke(
    {"messages": [{"role": "user", "content": "Now tell me about oranges and carrots"}]},
    config=config,
)
# Fruit subagent message count: 8 (remembers cherries!)
# Veggie subagent message count: 8 (remembers broccoli!)
```

Subgraphs [added as nodes](#) already get name-based namespaces automatically, so they don't need this wrapper.

js


```

import { createAgent, tool, toolCallLimitMiddleware } from "langchain";
import { MemorySaver, Command, interrupt } from "@langchain/langgraph";
import * as z from "zod";

const fruitInfo = tool(
  (input) => `Info about ${input.fruitName}`,
  {
    name: "fruit_info",
    description: "Look up fruit info.",
    schema: z.object({ fruitName: z.string() }),
  }
);

// Subagent with checkpointer=true for persistent state
const fruitAgent = createAgent({
  model: "gpt-5.4-mini",
  tools: [fruitInfo],
  prompt: "You are a fruit expert. Use the fruit_info tool. Respond in one sentence.",
  checkpointer: true, // [!code highlight]
});

// Wrap subagent as a tool for the outer agent
const askFruitExpert = tool(
  async (input) => {
    const response = await fruitAgent.invoke({
      messages: [{ role: "user", content: input.question }],
    });
    return response.messages[response.messages.length - 1].content;
  },
  {
    name: "ask_fruit_expert",
    description: "Ask the fruit expert. Use for ALL fruit questions.",
    schema: z.object({ question: z.string() }),
  }
);

// Outer agent with checkpointer
// Use toolCallLimitMiddleware to prevent parallel calls to per-thread subagents,
// which would cause checkpoint conflicts.
const agent = createAgent({
  model: "gpt-5.4-mini",
  tools: [askFruitExpert],
  prompt: "You have a fruit expert. ALWAYS delegate fruit questions to ask_fruit_expert.",
  middleware: [ // [!code highlight]
    toolCallLimitMiddleware({ toolName: "ask_fruit_expert", runLimit: 1 }), // [!code highlight]
  ], // [!code highlight]
  checkpointer: new MemorySaver(),
});

```

Interrupts

Per-thread subagents support `interrupt()` just like per-invocation. Add `interrupt()` to a tool function to require user approval:

```
const fruitInfo = tool(
  (input) => {
    interrupt("continue?"); // [!code highlight]
    return `Info about ${input.fruitName}`;
  },
  {
    name: "fruit_info",
    description: "Look up fruit info.",
    schema: z.object({ fruitName: z.string() }),
  }
);
```

```
const config = { configurable: { thread_id: "1" } };

// Invoke - the subagent's tool calls interrupt()
let response = await agent.invoke(
  { messages: [{ role: "user", content: "Tell me about apples" }] },
  config,
);
// response contains __interrupt__

// Resume - approve the interrupt
response = await agent.invoke(new Command({ resume: true }), config); // [!code highlight]
// Subagent message count: 4
```

Multi-turn

State accumulates across invocations—the subagent remembers past conversations:

```
const config = { configurable: { thread_id: "1" } };

// First call
let response = await agent.invoke(
  { messages: [{ role: "user", content: "Tell me about apples" }] },
  config,
);
// Subagent message count: 4

// Second call - subagent REMEMBERS apples conversation
response = await agent.invoke(
  { messages: [{ role: "user", content: "Now tell me about bananas" }] },
  config,
);
// Subagent message count: 8 (accumulated!)
```

Multiple subgraph calls

When you have multiple **different** per-thread subgraphs (for example, a fruit expert and a veggie expert), each one needs its own storage space so their checkpoints don't overwrite each other. This is called **namespace isolation**.

If you [call subgraphs inside a node](#), LangGraph assigns namespaces based on call order (first call, second call, etc.). This means reordering your calls can mix up which subgraph loads which state. To avoid this, wrap each subagent in its own `StateGraph` with a unique node name—this gives each subgraph a stable, unique namespace:

```
import { StateGraph, StateSchema, MessagesValue, START } from "@langchain/langgraph";

function createSubAgent(model: string, { name, ...kwargs }: { name: string; [key: string]: any }) {
  const agent = createAgent({ model, name, ...kwargs });
  return new StateGraph(new StateSchema({ messages: MessagesValue }))
    .addNode(name, agent) // unique name → stable namespace // [!code highlight]
    .addEdge(START, name)
    .compile();
}

const fruitAgent = createSubAgent("gpt-5.4-mini", {
  name: "fruit_agent", tools: [fruitInfo], prompt: "...", checkpointer: true,
});
const veggieAgent = createSubAgent("gpt-5.4-mini", {
  name: "veggie_agent", tools: [veggieInfo], prompt: "...", checkpointer: true,
});
const config = { configurable: { thread_id: "1" } };

// First call - LLM calls both fruit and veggie experts
let response = await agent.invoke(
  { messages: [{ role: "user", content: "Tell me about cherries and broccoli" }] },
  config,
);
// Fruit subagent message count: 4
// Veggie subagent message count: 4

// Second call - both agents accumulate independently
response = await agent.invoke(
  { messages: [{ role: "user", content: "Now tell me about oranges and carrots" }] },
  config,
);
// Fruit subagent message count: 8 (remembers cherries!)
// Veggie subagent message count: 8 (remembers broccoli!)
```

Subgraphs [added as nodes](#) already get name-based namespaces automatically, so they don't need this wrapper.

Stateless

Use this when you want to run a subagent like a plain function call with no checkpointing overhead. The subgraph cannot pause/resume and does not benefit from [durable execution](#). Compile with `checkpointer=False`.

주의

Without checkpointing, the subgraph has no durable execution. If the process crashes mid-run, the subgraph cannot recover and must be re-run from the beginning.

python

```
subgraph_builder = StateGraph(...)
subgraph = subgraph_builder.compile(checkpointer=False) # [!code highlight]
```

js

```
const subgraphBuilder = new StateGraph(...);
const subgraph = subgraphBuilder.compile({ checkpointer: false }); // [!code highlight]
```

Checkpointer reference

Control subgraph persistence with the `checkpointer` parameter on `.compile()`:

python

```
subgraph = builder.compile(checkpointer=False) # or True / None
```

js

```
const subgraph = builder.compile({ checkpointer: false }); // or true, or null
```

Feature	Per-invocation (default)	Per-thread	Stateless
<code>checkpointer=</code>	None	True	False
Interrupts (HITL)	✓	✓	✗
Multi-turn memory	✗	✓	✗
Multiple calls (different subgraphs)	✓		



|  | | Multiple calls (same subgraph) |  |  |  | | State inspection |



|  |  |

- **Interrupts (HITL):** The subgraph can use [interrupt\(\)](#) to pause execution and wait for user input, then resume where it left off.
- **Multi-turn memory:** The subgraph retains its state across multiple invocations within the same [thread](#). Each call picks up where the last one left off rather than starting fresh.
- **Multiple calls (different subgraphs):** Multiple different subgraph instances can be invoked within a single node without checkpoint namespace conflicts.
- **Multiple calls (same subgraph):** The same subgraph instance can be invoked multiple times within a single node. With stateful persistence, these calls write to the same checkpoint namespace and conflict—use per-invocation persistence instead.
- **State inspection:** The subgraph's state is available via `get_state(config, subgraphs=True)` for debugging and monitoring.

View subgraph state

When you enable [persistence](#), you can inspect the subgraph state using the `subgraphs` option. With [stateless](#) checkpointing (`checkpointer=False`), no subgraph checkpoints are saved, so subgraph state is not available.

참고

Viewing subgraph state requires that LangGraph can **statically discover** the subgraph—i.e., it is [added as a node](#) or [called inside a node](#). It does not work when a subgraph is called inside a [tool](#) function or other indirection (e.g., the [subagents](#) pattern). Interrupts still propagate to the top-level graph regardless of nesting.

python

Per-invocation

Returns subgraph state for the **current invocation only**. Each invocation starts fresh.

```
from langgraph.graph import START, StateGraph
from langgraph.checkpoint.memory import MemorySaver
from langgraph.types import interrupt, Command
from typing_extensions import TypedDict

class State(TypedDict):
    foo: str

# Subgraph
```

```

def subgraph_node_1(state: State):
    value = interrupt("Provide value:")
    return {"foo": state["foo"] + value}

subgraph_builder = StateGraph(State)
subgraph_builder.add_node(subgraph_node_1)
subgraph_builder.add_edge(START, "subgraph_node_1")
subgraph = subgraph_builder.compile() # inherits parent checkpointer

# Parent graph
builder = StateGraph(State)
builder.add_node("node_1", subgraph)
builder.add_edge(START, "node_1")

checkpointer = MemorySaver()
graph = builder.compile(checkpointer=checkpointer)

config = {"configurable": {"thread_id": "1"}}

graph.invoke({"foo": ""}, config)

# View subgraph state for the current invocation
subgraph_state = graph.get_state(config, subgraphs=True).tasks[0].state # [!code highlight]

# Resume the subgraph
graph.invoke(Command(resume="bar"), config)

```

Per-thread

Returns **accumulated** subgraph state across all invocations on this thread.

```

from langgraph.graph import START, StateGraph, MessagesState
from langgraph.checkpoint.memory import MemorySaver

# Subgraph with its own persistent state
subgraph_builder = StateGraph(MessagesState)
# ... add nodes and edges
subgraph = subgraph_builder.compile(checkpointer=True) # [!code highlight]

# Parent graph
builder = StateGraph(MessagesState)
builder.add_node("agent", subgraph)
builder.add_edge(START, "agent")

checkpointer = MemorySaver()
graph = builder.compile(checkpointer=checkpointer)

config = {"configurable": {"thread_id": "1"}}

graph.invoke({"messages": [{"role": "user", "content": "hi"}]}, config)

```

```
graph.invoke({"messages": [{"role": "user", "content": "what did I say?"}]}, config)

# View accumulated subgraph state (includes messages from both invocations)
subgraph_state = graph.get_state(config, subgraphs=True).tasks[0].state # [!code highlight]
```

js

Per-invocation

Returns subgraph state for the **current invocation only**. Each invocation starts fresh.

```
import { StateGraph, StateSchema, START, MemorySaver, interrupt, Command } from "@langchain/
langgraph";
import * as z from "zod";

const State = new StateSchema({
  foo: z.string(),
});

// Subgraph
const subgraphBuilder = new StateGraph(State)
  .addNode("subgraphNode1", (state) => {
    const value = interrupt("Provide value:");
    return { foo: state.foo + value };
  })
  .addEdge(START, "subgraphNode1");

const subgraph = subgraphBuilder.compile(); // inherits parent checkpointer

// Parent graph
const builder = new StateGraph(State)
  .addNode("node1", subgraph)
  .addEdge(START, "node1");

const checkpointer = new MemorySaver();
const graph = builder.compile({ checkpointer });

const config = { configurable: { thread_id: "1" } };

await graph.invoke({ foo: "" }, config);

// View subgraph state for the current invocation
const subgraphState = (await graph.getState(config, { subgraphs: true })).tasks[0].state;
// [!code highlight]

// Resume the subgraph
await graph.invoke(new Command({ resume: "bar" }), config);
```

Per-thread

Returns **accumulated** subgraph state across all invocations on this thread.

```
import { StateGraph, StateSchema, MessagesValue, START, MemorySaver } from "@langchain/langgraph";

// Subgraph with its own persistent state
const SubgraphState = new StateSchema({
  messages: MessagesValue,
});

const subgraphBuilder = new StateGraph(SubgraphState);
// ... add nodes and edges
const subgraph = subgraphBuilder.compile({ checkpointer: true }); // [!code highlight]

// Parent graph
const builder = new StateGraph(SubgraphState)
  .addNode("agent", subgraph)
  .addEdge(START, "agent");

const checkpointer = new MemorySaver();
const graph = builder.compile({ checkpointer });

const config = { configurable: { thread_id: "1" } };

await graph.invoke({ messages: [{ role: "user", content: "hi" }] }, config);
await graph.invoke({ messages: [{ role: "user", content: "what did I say?" }] }, config);

// View accumulated subgraph state (includes messages from both invocations)
const subgraphState = (await graph.getState(config, { subgraphs: true })).tasks[0].state;
// [!code highlight]
```

Stream subgraph outputs

To observe nested graph executions, we recommend [event streaming](#): the `stream.subgraphs` projection discovers each nested run and exposes its `path`, `messages`, and `values` without parsing namespace strings.

python

```
stream = graph.stream_events({"foo": "foo"}, version="v3") # [!code highlight]

for subgraph in stream.subgraphs:
    print(subgraph.graph_name, subgraph.path)

    for snapshot in subgraph.values:
        print(subgraph.path, snapshot)
```


If you need the raw protocol events, iterate the stream directly and filter on `event["method"]` and `event["params"]["namespace"]`:

```
from langgraph.stream import UpdatesTransformer

stream = graph.stream_events({"foo": "foo"}, version="v3", transformers=[UpdatesTransformer])
for event in stream:
    if event["method"] == "updates":
        print(event["params"]["namespace"], event["params"]["data"])
```

js

```
const stream = await graph.streamEvents(
  { foo: "foo" },
  {
    subgraphs: true, // [!code highlight]
    version: "v3",
  }
);
for await (const snapshot of stream.values) {
  console.log(snapshot);
}
```

1. Set `subgraphs: true` to stream outputs from subgraphs.

Stream from subgraphs

python

```
from typing_extensions import TypedDict
from langgraph.graph.state import StateGraph, START
from langgraph.stream import UpdatesTransformer

# Define subgraph
class SubgraphState(TypedDict):
    foo: str
    bar: str

def subgraph_node_1(state: SubgraphState):
    return {"bar": "bar"}

def subgraph_node_2(state: SubgraphState):
    # note that this node is using a state key ('bar') that is only available in the subgraph
    # and is sending update on the shared state key ('foo')
    return {"foo": state["foo"] + state["bar"]}
```

```

subgraph_builder = StateGraph(SubgraphState)
subgraph_builder.add_node(subgraph_node_1)
subgraph_builder.add_node(subgraph_node_2)
subgraph_builder.add_edge(START, "subgraph_node_1")
subgraph_builder.add_edge("subgraph_node_1", "subgraph_node_2")
subgraph = subgraph_builder.compile()

# Define parent graph
class ParentState(TypedDict):
    foo: str

def node_1(state: ParentState):
    return {"foo": "hi! " + state["foo"]}

builder = StateGraph(ParentState)
builder.add_node("node_1", node_1)
builder.add_node("node_2", subgraph)
builder.add_edge(START, "node_1")
builder.add_edge("node_1", "node_2")
graph = builder.compile()

stream = graph.stream_events({"foo": "foo"}, version="v3", transformers=[UpdatesTransformer]) # [!code highlight]
for event in stream:
    if event["method"] == "updates":
        print(event["params"]["namespace"], event["params"]["data"])

```

```

[] {'node_1': {'foo': 'hi! foo'}}
['node_2:e58e5673-a661-ebb0-70d4-e298a7fc28b7'] {'subgraph_node_1': {'bar': 'bar'}}
['node_2:e58e5673-a661-ebb0-70d4-e298a7fc28b7'] {'subgraph_node_2': {'foo': 'hi! foobar'}}
[] {'node_2': {'foo': 'hi! foobar'}}

```

js

```

import { StateGraph, StateSchema, START } from "@langchain/langgraph";
import * as z from "zod";

// Define subgraph
const SubgraphState = new StateSchema({
  foo: z.string(),
  bar: z.string(),
});

const subgraphBuilder = new StateGraph(SubgraphState)
  .addNode("subgraphNode1", (state) => {
    return { bar: "bar" };
  })
  .addNode("subgraphNode2", (state) => {

```

```

    // note that this node is using a state key ('bar') that is only available in the subgraph
    // and is sending update on the shared state key ('foo')
    return { foo: state.foo + state.bar };
  })
  .addEdge(START, "subgraphNode1")
  .addEdge("subgraphNode1", "subgraphNode2");

const subgraph = subgraphBuilder.compile();

// Define parent graph
const ParentState = new StateSchema({
  foo: z.string(),
});

const builder = new StateGraph(ParentState)
  .addNode("node1", (state) => {
    return { foo: "hi! " + state.foo };
  })
  .addNode("node2", subgraph)
  .addEdge(START, "node1")
  .addEdge("node1", "node2");

const graph = builder.compile();

const stream = await graph.streamEvents(
  { foo: "foo" },
  {
    subgraphs: true,    // [!code highlight]
    version: "v3",
  }
);

for await (const snapshot of stream.values) {
  console.log(snapshot);
}

```

1. Set `subgraphs: true` to stream outputs from subgraphs.

```

[[[], { node1: { foo: 'hi! foo' } }]] [['node2:e58e5673-a661-ebb0-70d4-e298a7fc28b7'], {
subgraphNode1: { bar: 'bar' } }]] [['node2:e58e5673-a661-ebb0-70d4-e298a7fc28b7'], {
subgraphNode2: { foo: 'hi! foobar' } }]] [[[], { node2: { foo: 'hi! foobar' } }]]

```

Use time-travel

langgraph/use-time-travel.mdx

Overview

LangGraph supports time travel through [checkpoints](#):

- **Replay**: Retry from a prior checkpoint.
- **Fork**: Branch from a prior checkpoint with modified state to explore an alternative path.

Both work by resuming from a prior checkpoint. Nodes before the checkpoint are not re-executed (results are already saved). Nodes after the checkpoint re-execute, including any LLM calls, API requests, and [interrupts](#) (which may produce different results).

Replay

Invoke the graph with a prior checkpoint's config to replay from that point.

주의

Replay re-executes nodes—it doesn't just read from cache. LLM calls, API requests, and [interrupts](#) fire again and may return different results. Replaying from the final checkpoint (no next nodes) is a no-op.



python

Use `@[get_state_history]` to find the checkpoint you want to replay from, then call `@[invoke]` `[CompiledStateGraph.invoke]` with that checkpoint's config:

```
from langgraph.graph import StateGraph, START
from langgraph.checkpoint.memory import InMemorySaver
from typing_extensions import TypedDict, NotRequired
from langchain_core.utils.uuid import uuid7

class State(TypedDict):
    topic: NotRequired[str]
    joke: NotRequired[str]

def generate_topic(state: State):
    return {"topic": "socks in the dryer"}

def write_joke(state: State):
```

```

        return {"joke": f"Why do {state['topic']} disappear? They elope!"}

checkpointer = InMemorySaver()
graph = (
    StateGraph(State)
    .add_node("generate_topic", generate_topic)
    .add_node("write_joke", write_joke)
    .add_edge(START, "generate_topic")
    .add_edge("generate_topic", "write_joke")
    .compile(checkpointer=checkpointer)
)

# Step 1: Run the graph
config = {"configurable": {"thread_id": str(uuid7())}}
result = graph.invoke({}, config)

# Step 2: Find a checkpoint to replay from
history = list(graph.get_state_history(config))
# History is in reverse chronological order
for state in history:
    print(f"next={state.next}, checkpoint_id={state.config['configurable']['checkpoint_id']}")

# Step 3: Replay from a specific checkpoint
# Find the checkpoint before write_joke
before_joke = next(s for s in history if s.next == ("write_joke",))
replay_result = graph.invoke(None, before_joke.config)
# write_joke re-executes (runs again), generate_topic does not

```

js

Use `@[getStateHistory]` to find the checkpoint you want to replay from, then call `@[invoke]` `[CompiledStateGraph.invoke]` with that checkpoint's config:

```

import { v7 as uuid7 } from "uuid";
import { StateGraph, MemorySaver, START } from "@langchain/langgraph";

const StateAnnotation = Annotation.Root({
  topic: Annotation<string>(),
  joke: Annotation<string>(),
});

function generateTopic(state: typeof StateAnnotation.State) {
  return { topic: "socks in the dryer" };
}

function writeJoke(state: typeof StateAnnotation.State) {
  return { joke: `Why do ${state.topic} disappear? They elope!` };
}

```

```

const checkpointer = new MemorySaver();
const graph = new StateGraph(StateAnnotation)
    .addNode("generateTopic", generateTopic)
    .addNode("writeJoke", writeJoke)
    .addEdge(START, "generateTopic")
    .addEdge("generateTopic", "writeJoke")
    .compile({ checkpointer });

// Step 1: Run the graph
const config = { configurable: { thread_id: uuid7() } };
const result = await graph.invoke({}, config);

// Step 2: Find a checkpoint to replay from
const states = [];
for await (const state of graph.getStateHistory(config)) {
    states.push(state);
}

// Step 3: Replay from a specific checkpoint
const beforeJoke = states.find((s) => s.next.includes("writeJoke"));
const replayResult = await graph.invoke(null, beforeJoke.config);
// writeJoke re-executes (runs again), generateTopic does not

```

Fork

Fork creates a new branch from a past checkpoint with modified state. Call `@[update_state]` on a prior checkpoint to create the fork, then `@[invoke]` `[CompiledStateGraph.invoke]` with `None` to continue execution.



주의

`update_state` does **not** roll back a thread. It creates a new checkpoint that branches from the specified point. The original execution history remains intact.

python

```

# Find checkpoint before write_joke
history = list(graph.get_state_history(config))
before_joke = next(s for s in history if s.next == ("write_joke",))

# Fork: update state to change the topic
fork_config = graph.update_state(
    before_joke.config,
    values={"topic": "chickens"},

```

```
)

# Resume from the fork – write_joke re-executes with the new topic
fork_result = graph.invoke(None, fork_config)
print(fork_result["joke"]) # A joke about chickens, not socks
```

js

```
// Find checkpoint before writeJoke
const states = [];
for await (const state of graph.getStateHistory(config)) {
  states.push(state);
}
const beforeJoke = states.find((s) => s.next.includes("writeJoke"));

// Fork: update state to change the topic
const forkConfig = await graph.updateState(
  beforeJoke.config,
  { topic: "chickens" },
);

// Resume from the fork – writeJoke re-executes with the new topic
const forkResult = await graph.invoke(null, forkConfig);
console.log(forkResult.joke); // A joke about chickens, not socks
```

From a specific node

When you call `@[update_state]`, values are applied using the specified node's writers (including [reducers](#)). The checkpoint records that node as having produced the update, and execution resumes from that node's successors.

By default, LangGraph infers `as_node` from the checkpoint's version history. When forking from a specific checkpoint, this inference is almost always correct.

Specify `as_node` explicitly when:

- **Parallel branches:** Multiple nodes updated state in the same step, and LangGraph can't determine which was last (`InvalidUpdateError`).
- **No execution history:** Setting up state on a fresh thread (common in [testing](#)).
- **Skipping nodes:** Set `as_node` to a later node to make the graph think that node already ran.

python

```
# graph: generate_topic -> write_joke

# Treat this update as if generate_topic produced it.
```

```
# Execution resumes at write_joke (the successor of generate_topic).
fork_config = graph.update_state(
    before_joke.config,
    values={"topic": "chickens"},
    as_node="generate_topic",
)
```

js

```
// graph: generateTopic -> writeJoke

// Treat this update as if generateTopic produced it.
// Execution resumes at writeJoke (the successor of generateTopic).
const forkConfig = await graph.updateState(
    beforeJoke.config,
    { topic: "chickens" },
    { asNode: "generateTopic" },
);
```

Interrupts

If your graph uses `@[interrupt]` for [human-in-the-loop](#) workflows, interrupts are always re-triggered during time travel. The node containing the interrupt re-executes, and `interrupt()` pauses for a new `Command(resume=...)`.

python

```
from langgraph.types import interrupt, Command

class State(TypedDict):
    value: list[str]

def ask_human(state: State):
    answer = interrupt("What is your name?")
    return {"value": [f"Hello, {answer}!"]}

def final_step(state: State):
    return {"value": ["Done"]}

graph = (
    StateGraph(State)
    .add_node("ask_human", ask_human)
    .add_node("final_step", final_step)
    .add_edge(START, "ask_human")
    .add_edge("ask_human", "final_step")
    .compile(checkpointer=InMemorySaver())
)
```



```

)

config = {"configurable": {"thread_id": "1"}}

# First run: hits interrupt
graph.invoke({"value": []}, config)
# Resume with answer
graph.invoke(Command(resume="Alice"), config)

# Replay from before ask_human
history = list(graph.get_state_history(config))
before_ask = [s for s in history if s.next == ("ask_human",)][-1]

replay_result = graph.invoke(None, before_ask.config)
# Pauses at interrupt – waiting for new Command(resume=...)

# Fork from before ask_human
fork_config = graph.update_state(before_ask.config, {"value": ["forked"]})
fork_result = graph.invoke(None, fork_config)
# Pauses at interrupt – waiting for new Command(resume=...)

# Resume the forked interrupt with a different answer
graph.invoke(Command(resume="Bob"), fork_config)
# Result: {"value": ["forked", "Hello, Bob!", "Done"]}

```

js

```

import { interrupt, Command } from "@langchain/langgraph";

function askHuman(state: { value: string[] }) {
  const answer = interrupt("What is your name?");
  return { value: [`Hello, ${answer}!`] };
}

function finalStep(state: { value: string[] }) {
  return { value: ["Done"] };
}

// ... build graph with checkpointing ...

// First run: hits interrupt
await graph.invoke({ value: [] }, config);
// Resume with answer
await graph.invoke(new Command({ resume: "Alice" }), config);

// Replay from before askHuman
const states = [];
for await (const state of graph.getStateHistory(config)) {
  states.push(state);
}

```

```

}
const beforeAsk = states.filter((s) => s.next.includes("askHuman")).pop();

const replayResult = await graph.invoke(null, beforeAsk.config);
// Pauses at interrupt – waiting for new Command({ resume: ... })

// Fork from before askHuman
const forkConfig = await graph.updateState(beforeAsk.config, { value: ["forked"] });
const forkResult = await graph.invoke(null, forkConfig);
// Pauses at interrupt – waiting for new Command({ resume: ... })

// Resume the forked interrupt with a different answer
await graph.invoke(new Command({ resume: "Bob" }), forkConfig);
// Result: { value: ["forked", "Hello, Bob!", "Done"] }

```

Multiple interrupts

If your graph collects input at several points (for example, a multi-step form), you can fork from between the interrupts to change a later answer without re-asking earlier questions.

python

```

def ask_name(state):
    name = interrupt("What is your name?")
    return {"value": [f"name:{name}"]}

def ask_age(state):
    age = interrupt("How old are you?")
    return {"value": [f"age:{age}"]}

# Graph: ask_name -> ask_age -> final
# After completing both interrupts:

# Fork from BETWEEN the two interrupts (after ask_name, before ask_age)
history = list(graph.get_state_history(config))
between = [s for s in history if s.next == ("ask_age",)][-1]

fork_config = graph.update_state(between.config, {"value": ["modified"]})
result = graph.invoke(None, fork_config)
# ask_name result preserved ("name:Alice")
# ask_age pauses at interrupt – waiting for new answer

```

js

```

// Fork from BETWEEN the two interrupts (after askName, before askAge)
const states = [];
for await (const state of graph.getStateHistory(config)) {

```

```

states.push(state);
}
const between = states.filter((s) => s.next.includes("askAge")).pop();

const forkConfig = await graph.updateState(between.config, { value: ["modified"] });
const result = await graph.invoke(null, forkConfig);
// askName result preserved ("name:Alice")
// askAge pauses at interrupt – waiting for new answer

```

Subgraphs

Time travel with [subgraphs](#) depends on whether the subgraph has its own checkpointer. This determines the granularity of checkpoints you can time travel from.

Inherited checkpointer (default)

By default, a subgraph inherits the parent's checkpointer. The parent treats the entire subgraph as a **single super-step** — there is only one parent-level checkpoint for the whole subgraph execution. Time traveling from before the subgraph re-executes it from scratch.

You cannot time travel to a point *between* nodes in a default subgraph — you can only time travel from the parent level.

python

```

# Subgraph without its own checkpointer (default)
subgraph = (
    StateGraph(State)
    .add_node("step_a", step_a)      # Has interrupt()
    .add_node("step_b", step_b)      # Has interrupt()
    .add_edge(START, "step_a")
    .add_edge("step_a", "step_b")
    .compile() # No checkpointer – inherits from parent
)

graph = (
    StateGraph(State)
    .add_node("subgraph_node", subgraph)
    .add_edge(START, "subgraph_node")
    .compile(checkpointer=InMemorySaver())
)

config = {"configurable": {"thread_id": "1"}}

# Complete both interrupts
graph.invoke({"value": []}, config)      # Hits step_a interrupt
graph.invoke(Command(resume="Alice"), config) # Hits step_b interrupt
graph.invoke(Command(resume="30"), config) # Completes

```

```

# Time travel from before the subgraph
history = list(graph.get_state_history(config))
before_sub = [s for s in history if s.next == ("subgraph_node",)][-1]

fork_config = graph.update_state(before_sub.config, {"value": ["forked"]})
result = graph.invoke(None, fork_config)
# The entire subgraph re-executes from scratch
# You cannot time travel to a point between step_a and step_b

```

js

```

// Subgraph without its own checkpointer (default)
const subgraph = new StateGraph(StateAnnotation)
  .addNode("stepA", stepA)          // Has interrupt()
  .addNode("stepB", stepB)          // Has interrupt()
  .addEdge(START, "stepA")
  .addEdge("stepA", "stepB")
  .compile(); // No checkpointer – inherits from parent

const graph = new StateGraph(StateAnnotation)
  .addNode("subgraphNode", subgraph)
  .addEdge(START, "subgraphNode")
  .compile({ checkpointer });

// Complete both interrupts
await graph.invoke({ value: [] }, config);
await graph.invoke(new Command({ resume: "Alice" }), config);
await graph.invoke(new Command({ resume: "30" }), config);

// Time travel from before the subgraph
const states = [];
for await (const state of graph.getStateHistory(config)) {
  states.push(state);
}
const beforeSub = states.filter((s) => s.next.includes("subgraphNode")).pop();

const forkConfig = await graph.updateState(beforeSub.config, { value: ["forked"] });
const result = await graph.invoke(null, forkConfig);
// The entire subgraph re-executes from scratch
// You cannot time travel to a point between stepA and stepB

```

Subgraph checkpointer

Set `checker=true` on the subgraph to give it its own checkpoint history. This creates checkpoints at each step **within** the subgraph, allowing you to time travel from a specific point inside it — for example, between two interrupts.

Use `@[get_state]` with `subgraphs=True` to access the subgraph's own checkpoint config, then fork from it:

python

```
# Subgraph with its own checkpointer
subgraph = (
    StateGraph(State)
    .add_node("step_a", step_a)      # Has interrupt()
    .add_node("step_b", step_b)      # Has interrupt()
    .add_edge(START, "step_a")
    .add_edge("step_a", "step_b")
    .compile(checkpointer=True) # Own checkpoint history
)

graph = (
    StateGraph(State)
    .add_node("subgraph_node", subgraph)
    .add_edge(START, "subgraph_node")
    .compile(checkpointer=InMemorySaver())
)

config = {"configurable": {"thread_id": "1"}}

# Run until step_a interrupt
graph.invoke({"value": []}, config)

# Resume step_a -> hits step_b interrupt
graph.invoke(Command(resume="Alice"), config)

# Get the subgraph's own checkpoint (between step_a and step_b)
parent_state = graph.get_state(config, subgraphs=True)
sub_config = parent_state.tasks[0].state.config

# Fork from the subgraph checkpoint
fork_config = graph.update_state(sub_config, {"value": ["forked"]})
result = graph.invoke(None, fork_config)
# step_b re-executes, step_a's result is preserved
```

js

```
// Subgraph with its own checkpointer
const subgraph = new StateGraph(StateAnnotation)
    .addNode("stepA", stepA)      // Has interrupt()
    .addNode("stepB", stepB)      // Has interrupt()
    .addEdge(START, "stepA")
    .addEdge("stepA", "stepB")
    .compile({ checkpointer: true }); // Own checkpoint history
```

```
const graph = new StateGraph(StateAnnotation)
  .addNode("subgraphNode", subgraph)
  .addEdge(START, "subgraphNode")
  .compile({ checkpointer });

// Run until stepA interrupt, then resume -> hits stepB interrupt
await graph.invoke({ value: [] }, config);
await graph.invoke(new Command({ resume: "Alice" }), config);

// Get the subgraph's own checkpoint (between stepA and stepB)
const parentState = await graph.getState(config, { subgraphs: true });
const subConfig = parentState.tasks[0].state.config;

// Fork from the subgraph checkpoint
const forkConfig = await graph.updateState(subConfig, { value: ["forked"] });
const result = await graph.invoke(null, forkConfig);
// stepB re-executes, stepA's result is preserved
```

See [subgraph persistence](#) for more on configuring subgraph checkpointers.

Workflows and agents

langgraph/workflows-agents.mdx

This guide reviews common workflow and agent patterns.

- Workflows have predetermined code paths and are designed to operate in a certain order.
- Agents are dynamic and define their own processes and tool usage.

Agent Workflow

LangGraph offers several benefits when building agents and workflows, including [persistence](#), [streaming](#), and support for debugging as well as [deployment](#).

팁

Trace and compare these workflow patterns with [LangSmith](#). Follow the [tracing quickstart](#) to see how data flows through each step. We recommend you also set up [LangSmith Engine](#) which monitors your traces, detects issues, and proposes fixes.

Setup

To build a workflow or agent, you can use [any chat model](#) that supports structured outputs and tool calling. The following example uses Anthropic:

python

1. Install dependencies:

```
pip install langchain_core langchain-anthropic langgraph
```

2. Initialize the LLM:

```
import os
import getpass

from langchain_anthropic import ChatAnthropic

def _set_env(var: str):
    if not os.environ.get(var):
        os.environ[var] = getpass.getpass(f"{var}: ")

_set_env("ANTHROPIC_API_KEY")
```

```
llm = ChatAnthropic(model="claude-sonnet-4-6")
```

js

1. Install dependencies

```
```bash npm npm install @langchain/langgraph @langchain/core
```

```
```bash pnpm  
pnpm add @langchain/langgraph @langchain/core
```

```
```bash yarn yarn add @langchain/langgraph @langchain/core
```

```
```bash bun  
bun add @langchain/langgraph @langchain/core
```

2. Initialize the LLM:

```
import { ChatAnthropic } from "@langchain/anthropic";  
  
const llm = new ChatAnthropic({  
  model: "claude-sonnet-4-6",  
  apiKey: "<your_anthropic_key>"  
});
```

LLMs and augmentations

Workflows and agentic systems are based on LLMs and the various augmentations you add to them. [Tool calling](#), [structured outputs](#), and [short term memory](#) are a few options for tailoring LLMs to your needs.

 LLM augmentations

python

```
# Schema for structured output  
from pydantic import BaseModel, Field  
  
class SearchQuery(BaseModel):  
    search_query: str | None = Field(  
        default=None, description="Query that is optimized web search."  
    )
```



```

    justification: str | None = Field(
        default=None, description="Why this query is relevant to the user's request."
    )

# Augment the LLM with schema for structured output
structured_llm = llm.with_structured_output(SearchQuery)

# Invoke the augmented LLM
output = structured_llm.invoke("How does Calcium CT score relate to high cholesterol?")
print(output) # The model returns an instance of SearchQuery.

# Define a tool
def multiply(a: int, b: int) -> int:
    return a * b

# Augment the LLM with tools
llm_with_tools = llm.bind_tools([multiply])

# Invoke the LLM with input that triggers the tool call
msg = llm_with_tools.invoke("What is 2 times 3?")
print(msg.tool_calls) # The model returns a request to call the tool.

```

js

```

import * as z from "zod";
import { tool } from "langchain";

// Schema for structured output
const SearchQuery = z.object({
  search_query: z.string().describe("Query that is optimized web search."),
  justification: z
    .string()
    .describe("Why this query is relevant to the user's request."),
});

// Augment the LLM with schema for structured output
const structuredLlm = llm.withStructuredOutput(SearchQuery);

// Invoke the augmented LLM
const output = await structuredLlm.invoke(
  "How does Calcium CT score relate to high cholesterol?"
);

// Define a tool
const multiply = tool(
  ({ a, b }) => {
    return a * b;
  },

```

```

{
  name: "multiply",
  description: "Multiply two numbers",
  schema: z.object({
    a: z.number(),
    b: z.number(),
  }),
}
);

// Augment the LLM with tools
const llmWithTools = llm.bindTools([multiply]);

// Invoke the LLM with input that triggers the tool call
const msg = await llmWithTools.invoke("What is 2 times 3?");

// Get the tool call
console.log(msg.tool_calls);

```

Prompt chaining

Prompt chaining is when each LLM call processes the output of the previous call. It's often used for performing well-defined tasks that can be broken down into smaller, verifiable steps. Some examples include:

- Translating documents into different languages
- Verifying generated content for consistency



python

```

` `` python Graph API from typing_extensions import TypedDict from langgraph.graph import
StateGraph, START, END from IPython.display import Image, display

```

Graph state

```

class State(TypedDict): topic: str joke: str improved_joke: str final_joke: str

```

Nodes

```

def generate_joke(state: State): """First LLM call to generate initial joke"""
msg = llm.invoke(f"Write a short joke about {state['topic']}") return {"joke": msg.content}

```

```
def check_punchline(state: State): """Gate function to check if the joke has a punchline"""
```

Simple check - does the joke contain "?" or "!"

```
if "?" in state["joke"] or "!" in state["joke"]: return "Pass" return "Fail"
```

```
def improve_joke(state: State): """Second LLM call to improve the joke"""
```

```
msg = llm.invoke(f"Make this joke funnier by adding wordplay: {state['joke']}") return {"improved_joke": msg.content}
```

```
def polish_joke(state: State): """Third LLM call for final polish""" msg = llm.invoke(f"Add a surprising twist to this joke: {state['improved_joke']}") return {"final_joke": msg.content}
```

Build workflow

```
workflow = StateGraph(State)
```

Add nodes

```
workflow.add_node("generate_joke", generate_joke) workflow.add_node("improve_joke", improve_joke) workflow.add_node("polish_joke", polish_joke)
```

Add edges to connect nodes

```
workflow.add_edge(START, "generate_joke") workflow.add_conditional_edges("generate_joke", check_punchline, {"Fail": "improve_joke", "Pass": END}) workflow.add_edge("improve_joke", "polish_joke") workflow.add_edge("polish_joke", END)
```

Compile

```
chain = workflow.compile()
```

Show workflow

```
display(Image(chain.get_graph().draw_mermaid_png()))
```

Invoke

```
state = chain.invoke({"topic": "cats"}) print("Initial joke:") print(state["joke"]) print("\n--- --- ---\n") if "improved_joke" in state: print("Improved joke:") print(state["improved_joke"]) print("\n--- --- ---\n")
```

```
print("Final joke:")
print(state["final_joke"])
```

```
else: print("Final joke:") print(state["joke"])
```

```
```python Functional API
from langgraph.func import entrypoint, task

Tasks
@task
def generate_joke(topic: str):
 """First LLM call to generate initial joke"""
 msg = llm.invoke(f"Write a short joke about {topic}")
 return msg.content

def check_punchline(joke: str):
 """Gate function to check if the joke has a punchline"""
 # Simple check - does the joke contain "?" or "!"
 if "?" in joke or "!" in joke:
 return "Fail"

 return "Pass"

@task
def improve_joke(joke: str):
 """Second LLM call to improve the joke"""
 msg = llm.invoke(f"Make this joke funnier by adding wordplay: {joke}")
 return msg.content

@task
def polish_joke(joke: str):
 """Third LLM call for final polish"""
 msg = llm.invoke(f"Add a surprising twist to this joke: {joke}")
 return msg.content

@entrypoint()
def prompt_chaining_workflow(topic: str):
 original_joke = generate_joke(topic).result()
 if check_punchline(original_joke) == "Pass":
 return original_joke

 improved_joke = improve_joke(original_joke).result()
```

```

return polish_joke(improved_joke).result()

Invoke
stream = prompt_chaining_workflow.stream_events("cats", version="v3")
for snapshot in stream.values:
 print(snapshot)
 print("\n")

```

js

```

````typescript
import { StateGraph, StateSchema, GraphNode, ConditionalEdgeRouter } from "@langchain/langgraph";
import { z } from "zod/v4";

// Graph state
const State = new StateSchema({ topic: z.string(), joke: z.string(), improvedJoke: z.string(), finalJoke: z.string() });

// Define node functions

// First LLM call to generate initial joke
const generateJoke: GraphNode = async (state) => {
  const msg = await llm.invoke(`Write a short joke about ${state.topic}`);
  return { joke: msg.content };
};

// Gate function to check if the joke has a punchline
const checkPunchline: ConditionalEdgeRouter<{ InputSchema: typeof State; Nodes: "improveJoke" }> = (state) => {
  // Simple check - does the joke contain "?" or "!"
  if (state.joke?.includes("?") || state.joke?.includes("!")) {
    return "Pass";
  }
  return "Fail";
};

// Second LLM call to improve the joke
const improveJoke: GraphNode = async (state) => {
  const msg = await llm.invoke(`Make this joke funnier by adding wordplay: ${state.joke}`);
  return { improvedJoke: msg.content };
};

// Third LLM call for final polish
const polishJoke: GraphNode = async (state) => {
  const msg = await llm.invoke(`Add a surprising twist to this joke: ${state.improvedJoke}`);
  return { finalJoke: msg.content };
};

// Build workflow
const chain = new StateGraph(State)
  .addNode("generateJoke", generateJoke)
  .addNode("improveJoke", improveJoke)
  .addNode("polishJoke", polishJoke)
  .addEdge("start", "generateJoke")
  .addConditionalEdges("generateJoke", checkPunchline, {
    Pass: "improveJoke",
    Fail: "end"
  })
  .addEdge("improveJoke", "polishJoke")
  .addEdge("polishJoke", "end")
  .compile();

// Invoke
const state = await chain.invoke({ topic: "cats" });
console.log("Initial joke:");
console.log(state.joke);
console.log("\n--- --- ---\n");
if (state.improvedJoke !== undefined) {
  console.log("Improved joke:");
  console.log(state.improvedJoke);
  console.log("\n--- --- ---\n");
}
console.log("Final joke:");
console.log(state.finalJoke);
} else {
  console.log("Joke failed quality gate - no punchline detected!");
}

```

```

````typescript Functional API
import { task, entrypoint } from "@langchain/langgraph";

// Tasks

// First LLM call to generate initial joke
const generateJoke = task("generateJoke", async (topic: string) => {
 const msg = await llm.invoke(`Write a short joke about ${topic}`);
 return msg.content;
});

// Gate function to check if the joke has a punchline
function checkPunchline(joke: string) {
 // Simple check - does the joke contain "?" or "!"
 if (joke.includes("?") || joke.includes("!")) {
 return "Pass";
 }
 return "Fail";
}

// Second LLM call to improve the joke
const improveJoke = task("improveJoke", async (joke: string) => {
 const msg = await llm.invoke(
 `Make this joke funnier by adding wordplay: ${joke}`
);
 return msg.content;
});

// Third LLM call for final polish
const polishJoke = task("polishJoke", async (joke: string) => {
 const msg = await llm.invoke(
 `Add a surprising twist to this joke: ${joke}`
);
 return msg.content;
});

const workflow = entrypoint(
 "jokeMaker",
 async (topic: string) => {
 const originalJoke = await generateJoke(topic);
 if (checkPunchline(originalJoke) === "Pass") {
 return originalJoke;
 }
 const improvedJoke = await improveJoke(originalJoke);
 const polishedJoke = await polishJoke(improvedJoke);
 return polishedJoke;
 }
);

const stream = await workflow.streamEvents("cats", { version: "v3" });

```

```
for await (const snapshot of stream.values) {
 console.log(snapshot);
}
```

## Parallelization

With parallelization, LLMs work simultaneously on a task. This is either done by running multiple independent subtasks at the same time, or running the same task multiple times to check for different outputs. Parallelization is commonly used to:

- Split up subtasks and run them in parallel, which increases speed
- Run tasks multiple times to check for different outputs, which increases confidence

Some examples include:

- Running one subtask that processes a document for keywords, and a second subtask to check for formatting errors
- Running a task multiple times that scores a document for accuracy based on different criteria, like the number of citations, the number of sources used, and the quality of the sources

parallelization.png

### python

```python Graph API

Graph state

```
class State(TypedDict): topic: str joke: str story: str poem: str combined_output: str
```

Nodes

```
def call_llm_1(state: State): """First LLM call to generate initial joke"""
```

```
msg = llm.invoke(f"Write a joke about {state['topic']}") return {"joke": msg.content}
```

```
def call_llm_2(state: State): """Second LLM call to generate story"""
```

```
msg = llm.invoke(f"Write a story about {state['topic']}") return {"story": msg.content}
```

```
def call_llm_3(state: State): """Third LLM call to generate poem"""
```

```
msg = llm.invoke(f"Write a poem about {state['topic']}") return {"poem": msg.content}
```

```
def aggregator(state: State): """Combine the joke, story and poem into a single output"""
```

```
combined = f"Here's a story, joke, and poem about {state['topic']}!\n\n" combined +=  
f"STORY:\n{state['story']}\n\n" combined += f"JOKE:\n{state['joke']}\n\n" combined +=  
f"POEM:\n{state['poem']}" return {"combined_output": combined}
```

Build workflow

```
parallel_builder = StateGraph(State)
```

Add nodes

```
parallel_builder.add_node("call_llm_1", call_llm_1) parallel_builder.add_node("call_llm_2",  
call_llm_2) parallel_builder.add_node("call_llm_3", call_llm_3)  
parallel_builder.add_node("aggregator", aggregator)
```

Add edges to connect nodes

```
parallel_builder.add_edge(START, "call_llm_1") parallel_builder.add_edge(START, "call_llm_2")  
parallel_builder.add_edge(START, "call_llm_3") parallel_builder.add_edge("call_llm_1",  
"aggregator") parallel_builder.add_edge("call_llm_2", "aggregator")  
parallel_builder.add_edge("call_llm_3", "aggregator") parallel_builder.add_edge("aggregator",  
END) parallel_workflow = parallel_builder.compile()
```

Show workflow

```
display(Image(parallel_workflow.get_graph().draw_mermaid_png()))
```

Invoke

```
state = parallel_workflow.invoke({"topic": "cats"}) print(state["combined_output"])
```

```
```python Functional API  
@task
def call_llm_1(topic: str):
 """First LLM call to generate initial joke"""
 msg = llm.invoke(f"Write a joke about {topic}")
 return msg.content

@task
def call_llm_2(topic: str):
 """Second LLM call to generate story"""
 msg = llm.invoke(f"Write a story about {topic}")
```



```

 return msg.content

@task
def call_llm_3(topic):
 """Third LLM call to generate poem"""
 msg = llm.invoke(f"Write a poem about {topic}")
 return msg.content

@task
def aggregator(topic, joke, story, poem):
 """Combine the joke and story into a single output"""

 combined = f"Here's a story, joke, and poem about {topic}!\n\n"
 combined += f"STORY:\n{story}\n\n"
 combined += f"JOKE:\n{joke}\n\n"
 combined += f"POEM:\n{poem}"
 return combined

Build workflow
@entrypoint()
def parallel_workflow(topic: str):
 joke_fut = call_llm_1(topic)
 story_fut = call_llm_2(topic)
 poem_fut = call_llm_3(topic)
 return aggregator(
 topic, joke_fut.result(), story_fut.result(), poem_fut.result()
).result()

Invoke
stream = parallel_workflow.stream_events("cats", version="v3")
for snapshot in stream.values:
 print(snapshot)
 print("\n")

```

**js**

```

````typescript
Graph API import { StateGraph, StateSchema, GraphNode } from
"@langchain/langgraph"; import * as z from "zod";

// Graph state
const State = new StateSchema({
  topic: z.string(),
  joke: z.string(),
  story: z.string(),
  poem: z.string(),
  combinedOutput: z.string(),
});

// Nodes
// First LLM call to generate initial joke
const callLlm1: GraphNode = async (state) => {
  const msg = await llm.invoke(`Write a joke about ${state.topic}`);
  return { joke: msg.content };
};

// Second LLM call to generate story
const callLlm2: GraphNode = async (state) => {
  const msg = await llm.invoke(`Write a story about ${state.topic}`);
  return { story: msg.content };
};

```

```
// Third LLM call to generate poem const callLlm3: GraphNode = async (state) => { const msg =
await llm.invoke(Write a poem about ${state.topic}); return { poem: msg.content }; };

// Combine the joke, story and poem into a single output const aggregator: GraphNode = async
(state) => { const combined = Here's a story, joke, and poem about ${state.topic}!\n\n +
STORY:\n${state.story}\n\n + JOKE:\n${state.joke}\n\n + POEM:\n${state.poem}; return {
combinedOutput: combined }; };

// Build workflow const parallelWorkflow = new StateGraph(State) .addNode("callLlm1",
callLlm1) .addNode("callLlm2", callLlm2) .addNode("callLlm3", callLlm3)
.addNode("aggregator", aggregator) .addEdge("start", "callLlm1") .addEdge("start", "callLlm2")
.addEdge("start", "callLlm3") .addEdge("callLlm1", "aggregator") .addEdge("callLlm2",
"aggregator") .addEdge("callLlm3", "aggregator") .addEdge("aggregator", "end") .compile();

// Invoke const result = await parallelWorkflow.invoke({ topic: "cats" });
console.log(result.combinedOutput);
```

```
```typescript Functional API
import { task, entrypoint } from "@langchain/langgraph";

// Tasks

// First LLM call to generate initial joke
const callLlm1 = task("generateJoke", async (topic: string) => {
 const msg = await llm.invoke(`Write a joke about ${topic}`);
 return msg.content;
});

// Second LLM call to generate story
const callLlm2 = task("generateStory", async (topic: string) => {
 const msg = await llm.invoke(`Write a story about ${topic}`);
 return msg.content;
});

// Third LLM call to generate poem
const callLlm3 = task("generatePoem", async (topic: string) => {
 const msg = await llm.invoke(`Write a poem about ${topic}`);
 return msg.content;
});

// Combine outputs
const aggregator = task("aggregator", async (params: {
 topic: string;
 joke: string;
 story: string;
 poem: string;
}) => {
 const { topic, joke, story, poem } = params;
 return `Here's a story, joke, and poem about ${topic}!\n\n` +
```

```

 `STORY:\n${story}\n\n` +
 `JOKE:\n${joke}\n\n` +
 `POEM:\n${poem}`;
 });

// Build workflow
const workflow = entrypoint(
 "parallelWorkflow",
 async (topic: string) => {
 const [joke, story, poem] = await Promise.all([
 callLlm1(topic),
 callLlm2(topic),
 callLlm3(topic),
]);

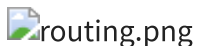
 return aggregator({ topic, joke, story, poem });
 }
);

// Invoke
const stream = await workflow.streamEvents("cats", { version: "v3" });
for await (const snapshot of stream.values) {
 console.log(snapshot);
}

```

## Routing

Routing workflows process inputs and then directs them to context-specific tasks. This allows you to define specialized flows for complex tasks. For example, a workflow built to answer product related questions might process the type of question first, and then route the request to specific processes for pricing, refunds, returns, etc.



### python

```

` `` python Graph API from typing_extensions import Literal from langchain.messages import
HumanMessage, SystemMessage

```

## Schema for structured output to use as routing logic

```

class Route(BaseModel): step: Literal["poem", "story", "joke"] = Field(None, description="The
next step in the routing process")

```

# Augment the LLM with schema for structured output

---

```
router = llm.with_structured_output(Route)
```

## State

---

```
class State(TypedDict): input: str decision: str output: str
```

## Nodes

---

```
def llm_call_1(state: State): """Write a story"""

result = llm.invoke(state["input"]) return {"output": result.content}

def llm_call_2(state: State): """Write a joke"""

result = llm.invoke(state["input"]) return {"output": result.content}

def llm_call_3(state: State): """Write a poem"""

result = llm.invoke(state["input"]) return {"output": result.content}

def llm_call_router(state: State): """Route the input to the appropriate node"""
```

## Run the augmented LLM with structured output to serve as routing logic

---

```
decision = router.invoke([SystemMessage(content="Route the input to story, joke, or poem
based on the user's request."), HumanMessage(content=state["input"]),])

return {"decision": decision.step}
```

## Conditional edge function to route to the appropriate node

---

```
def route_decision(state: State): # Return the node name you want to visit next if
state["decision"] == "story": return "llm_call_1" elif state["decision"] == "joke": return
"llm_call_2" elif state["decision"] == "poem": return "llm_call_3"
```

## Build workflow

---

```
router_builder = StateGraph(State)
```

## Add nodes

---

```
router_builder.add_node("llm_call_1", llm_call_1) router_builder.add_node("llm_call_2",
llm_call_2) router_builder.add_node("llm_call_3", llm_call_3)
router_builder.add_node("llm_call_router", llm_call_router)
```

## Add edges to connect nodes

---

```
router_builder.add_edge(START, "llm_call_router") router_builder.add_conditional_edges(
"llm_call_router", route_decision, { # Name returned by route_decision : Name of next node to
visit "llm_call_1": "llm_call_1", "llm_call_2": "llm_call_2", "llm_call_3": "llm_call_3", },)
router_builder.add_edge("llm_call_1", END) router_builder.add_edge("llm_call_2", END)
router_builder.add_edge("llm_call_3", END)
```

## Compile workflow

---

```
router_workflow = router_builder.compile()
```

## Show the workflow

---

```
display(Image(router_workflow.get_graph().draw_mermaid_png()))
```

## Invoke

---

```
state = router_workflow.invoke({"input": "Write me a joke about cats"}) print(state["output"])
```

```
```python Functional API  
from typing_extensions import Literal  
from pydantic import BaseModel  
from langchain.messages import HumanMessage, SystemMessage  
  
# Schema for structured output to use as routing logic  
class Route(BaseModel):  
    step: Literal["poem", "story", "joke"] = Field(  
        None, description="The next step in the routing process"  
    )
```

```

# Augment the LLM with schema for structured output
router = llm.with_structured_output(Route)

@task
def llm_call_1(input_: str):
    """Write a story"""
    result = llm.invoke(input_)
    return result.content

@task
def llm_call_2(input_: str):
    """Write a joke"""
    result = llm.invoke(input_)
    return result.content

@task
def llm_call_3(input_: str):
    """Write a poem"""
    result = llm.invoke(input_)
    return result.content

def llm_call_router(input_: str):
    """Route the input to the appropriate node"""
    # Run the augmented LLM with structured output to serve as routing logic
    decision = router.invoke(
        [
            SystemMessage(
                content="Route the input to story, joke, or poem based on the user's request."
            ),
            HumanMessage(content=input_),
        ]
    )
    return decision.step

# Create workflow
@entrypoint()
def router_workflow(input_: str):
    next_step = llm_call_router(input_)
    if next_step == "story":
        llm_call = llm_call_1
    elif next_step == "joke":
        llm_call = llm_call_2
    elif next_step == "poem":
        llm_call = llm_call_3

    return llm_call(input_).result()

# Invoke
stream = router_workflow.stream_events("Write me a joke about cats", version="v3")

```

```

for snapshot in stream.values:
    print(snapshot)
    print("\n")

```

js

```

````typescript
Graph API import { StateGraph, StateSchema, GraphNode,
ConditionalEdgeRouter } from "@langchain/langgraph"; import * as z from "zod";

// Schema for structured output to use as routing logic
const routeSchema = z.object({
 step: z.enum(["poem", "story", "joke"]).describe("The next step in the routing process"),
});

// Augment the LLM with schema for structured output
const router = llm.withStructuredOutput(routeSchema);

// Graph state
const State = new StateSchema({
 input: z.string(),
 decision: z.string(),
 output: z.string(),
});

// Nodes
// Write a story
const llmCall1: GraphNode = async (state) => {
 const result = await llm.invoke([
 { role: "system", content: "You are an expert storyteller." },
 { role: "user", content: state.input },
]);
 return { output: result.content };
};

// Write a joke
const llmCall2: GraphNode = async (state) => {
 const result = await llm.invoke([
 { role: "system", content: "You are an expert comedian." },
 { role: "user", content: state.input },
]);
 return { output: result.content };
};

// Write a poem
const llmCall3: GraphNode = async (state) => {
 const result = await llm.invoke([
 { role: "system", content: "You are an expert poet." },
 { role: "user", content: state.input },
]);
 return { output: result.content };
};

const llmCallRouter: GraphNode = async (state) => {
 // Route the input to the appropriate node
 const decision = await router.invoke([
 { role: "system", content: "Route the input to story, joke, or poem based on the user's request." },
 { role: "user", content: state.input },
]);

 return { decision: decision.step };
};

// Conditional edge function to route to the appropriate node
const routeDecision: ConditionalEdgeRouter<
 InputSchema: typeof State;
 Nodes: "llmCall1" | "llmCall2" | "llmCall3"
> = (state) => {
 // Return the node name you want to visit next if (state.decision === "story")
 if (state.decision === "story") { return "llmCall1"; }
 else if (state.decision === "joke") { return "llmCall2"; }
 else { return "llmCall3"; }
};

// Build workflow
const routerWorkflow = new StateGraph(State)
 .addNode("llmCall1", llmCall1)
 .addNode("llmCall2", llmCall2)
 .addNode("llmCall3", llmCall3)
 .addNode("llmCallRouter", llmCallRouter)
 .addEdge("start", "llmCallRouter")
 .addConditionalEdges("llmCallRouter", routeDecision, ["llmCall1", "llmCall2", "llmCall3"])
 .addEdge("llmCall1", "end")
 .addEdge("llmCall2", "end")
 .addEdge("llmCall3", "end")
 .compile();

```

```
// Invoke const state = await routerWorkflow.invoke({ input: "Write me a joke about cats" });
console.log(state.output);
```

```
```typescript Functional API
import * as z from "zod";
import { task, entrypoint } from "@langchain/langgraph";

// Schema for structured output to use as routing logic
const routeSchema = z.object({
  step: z.enum(["poem", "story", "joke"]).describe(
    "The next step in the routing process"
  ),
});

// Augment the LLM with schema for structured output
const router = llm.withStructuredOutput(routeSchema);

// Tasks
// Write a story
const llmCall1 = task("generateStory", async (input: string) => {
  const result = await llm.invoke([
    {
      role: "system",
      content: "You are an expert storyteller.",
    },
    {
      role: "user",
      content: input
    }
  ]);
  return result.content;
});

// Write a joke
const llmCall2 = task("generateJoke", async (input: string) => {
  const result = await llm.invoke([
    {
      role: "system",
      content: "You are an expert comedian.",
    },
    {
      role: "user",
      content: input
    }
  ]);
  return result.content;
});

// Write a poem
const llmCall3 = task("generatePoem", async (input: string) => {
  const result = await llm.invoke([
    {
      role: "system",
      content: "You are an expert poet.",
    },
    {
      role: "user",
      content: input
    }
  ]);
  return result.content;
});
```



```

    });
    return result.content;
  });

  // Route the input to the appropriate node
  const llmCallRouter = task("router", async (input: string) => {
    const decision = await router.invoke([
      {
        role: "system",
        content: "Route the input to story, joke, or poem based on the user's request."
      },
      {
        role: "user",
        content: input
      },
    ]);
    return decision.step;
  });

  // Build workflow
  const workflow = entrypoint(
    "routerWorkflow",
    async (input: string) => {
      const nextStep = await llmCallRouter(input);

      let llmCall;
      if (nextStep === "story") {
        llmCall = llmCall1;
      } else if (nextStep === "joke") {
        llmCall = llmCall2;
      } else if (nextStep === "poem") {
        llmCall = llmCall3;
      }

      const finalResult = await llmCall(input);
      return finalResult;
    }
  );

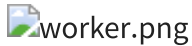
  // Invoke
  const stream = await workflow.streamEvents("Write me a joke about cats", { version: "v3" });
  for await (const snapshot of stream.values) {
    console.log(snapshot);
  }
}

```

Orchestrator-worker

In an orchestrator-worker configuration, the orchestrator:

- Breaks down tasks into subtasks
- Delegates subtasks to workers
- Synthesizes worker outputs into a final result



Orchestrator-worker workflows provide more flexibility and are often used when subtasks cannot be predefined the way they can with [parallelization](#). This is common with workflows that write code or need to update content across multiple files. For example, a workflow that needs to update installation instructions for multiple Python libraries across an unknown number of documents might use this pattern.

python

```
```python Graph API from typing import Annotated, List import operator
```

## Schema for structured output to use in planning

---

```
class Section(BaseModel): name: str = Field(description="Name for this section of the report.",
) description: str = Field(description="Brief overview of the main topics and concepts to be
covered in this section.",)
```

```
class Sections(BaseModel): sections: List[Section] = Field(description="Sections of the report.",
)
```

## Augment the LLM with schema for structured output

---

```
planner = llm.with_structured_output(Sections)
```

```
```python Functional API
from typing import List

# Schema for structured output to use in planning
class Section(BaseModel):
    name: str = Field(
        description="Name for this section of the report.",
    )
    description: str = Field(
        description="Brief overview of the main topics and concepts to be covered in this se
ction.",
    )
```

```

class Sections(BaseModel):
    sections: List[Section] = Field(
        description="Sections of the report.",
    )

# Augment the LLM with schema for structured output
planner = llm.with_structured_output(Sections)

@task
def orchestrator(topic: str):
    """Orchestrator that generates a plan for the report"""
    # Generate queries
    report_sections = planner.invoke(
        [
            SystemMessage(content="Generate a plan for the report."),
            HumanMessage(content=f"Here is the report topic: {topic}"),
        ]
    )

    return report_sections.sections

@task
def llm_call(section: Section):
    """Worker writes a section of the report"""

    # Generate section
    result = llm.invoke(
        [
            SystemMessage(content="Write a report section."),
            HumanMessage(
                content=f"Here is the section name: {section.name} and description: {section.description}"
            ),
        ]
    )

    # Write the updated section to completed sections
    return result.content

@task
def synthesizer(completed_sections: list[str]):
    """Synthesize full report from sections"""
    final_report = "\n\n---\n\n".join(completed_sections)
    return final_report

@entrypoint()
def orchestrator_worker(topic: str):
    sections = orchestrator(topic).result()
    section_futures = [llm_call(section) for section in sections]
    final_report = synthesizer(

```

```

        [section_fut.result() for section_fut in section_futures]
    ).result()
    return final_report

# Invoke
report = orchestrator_worker.invoke("Create a report on LLM scaling laws")
from IPython.display import Markdown
Markdown(report)

```

js

```typescript Graph API

```

type SectionSchema = { name: string; description: string; } type SectionsSchema = { sections:
SectionSchema[]; }

```

```

// Augment the LLM with schema for structured output const planner =
llm.withStructuredOutput(sectionsSchema);

```

```

```typescript Functional API
import * as z from "zod";
import { task, entrypoint } from "@langchain/langgraph";

// Schema for structured output to use in planning
const sectionSchema = z.object({
  name: z.string().describe("Name for this section of the report."),
  description: z.string().describe(
    "Brief overview of the main topics and concepts to be covered in this section."
  ),
});

const sectionsSchema = z.object({
  sections: z.array(sectionSchema).describe("Sections of the report."),
});

// Augment the LLM with schema for structured output
const planner = llm.withStructuredOutput(sectionsSchema);

// Tasks
const orchestrator = task("orchestrator", async (topic: string) => {
  // Generate queries
  const reportSections = await planner.invoke([
    { role: "system", content: "Generate a plan for the report." },
    { role: "user", content: `Here is the report topic: ${topic}` },
  ]);

  return reportSections.sections;
});

```

```

const llmCall = task("sectionWriter", async (section: z.infer<typeof sectionSchema>) => {
  // Generate section
  const result = await llm.invoke([
    {
      role: "system",
      content: "Write a report section.",
    },
    {
      role: "user",
      content: `Here is the section name: ${section.name} and description: ${section.description}`,
    },
  ]);

  return result.content;
});

const synthesizer = task("synthesizer", async (completedSections: string[]) => {
  // Synthesize full report from sections
  return completedSections.join("\n\n---\n\n");
});

// Build workflow
const workflow = entrypoint(
  "orchestratorWorker",
  async (topic: string) => {
    const sections = await orchestrator(topic);
    const completedSections = await Promise.all(
      sections.map((section) => llmCall(section))
    );
    return synthesizer(completedSections);
  }
);

// Invoke
const stream = await workflow.streamEvents("Create a report on LLM scaling laws", { version: "v3" });
for await (const snapshot of stream.values) {
  console.log(snapshot);
}

```

Creating workers in LangGraph

Orchestrator-worker workflows are common and LangGraph has built-in support for them. The `Send` API lets you dynamically create worker nodes and send them specific inputs. Each worker has its own state, and all worker outputs are written to a shared state key that is accessible to the orchestrator graph. This gives the orchestrator access to all worker output and allows it to

synthesize them into a final output. The example below iterates over a list of sections and uses the `Send` API to send a section to each worker.

python

```
from langgraph.types import Send

# Graph state
class State(TypedDict):
    topic: str # Report topic
    sections: list[Section] # List of report sections
    completed_sections: Annotated[
        list, operator.add
    ] # All workers write to this key in parallel
    final_report: str # Final report

# Worker state
class WorkerState(TypedDict):
    section: Section
    completed_sections: Annotated[list, operator.add]

# Nodes
def orchestrator(state: State):
    """Orchestrator that generates a plan for the report"""

    # Generate queries
    report_sections = planner.invoke(
        [
            SystemMessage(content="Generate a plan for the report."),
            HumanMessage(content=f"Here is the report topic: {state['topic']}"),
        ]
    )

    return {"sections": report_sections.sections}

def llm_call(state: WorkerState):
    """Worker writes a section of the report"""

    # Generate section
    section = llm.invoke(
        [
            SystemMessage(
                content="Write a report section following the provided name and description. Include no preamble for each section. Use markdown formatting."
            ),
            HumanMessage(
                content=f"Here is the section name: {state['section'].name} and description: {state['section'].description}"
            ),
        ]
    )
```

```

)

# Write the updated section to completed sections
return {"completed_sections": [section.content]}

def synthesizer(state: State):
    """Synthesize full report from sections"""

    # List of completed sections
    completed_sections = state["completed_sections"]

    # Format completed section to str to use as context for final sections
    completed_report_sections = "\n\n---\n\n".join(completed_sections)

    return {"final_report": completed_report_sections}

# Conditional edge function to create llm_call workers that each write a section of the report
def assign_workers(state: State):
    """Assign a worker to each section in the plan"""

    # Kick off section writing in parallel via Send() API
    return [Send("llm_call", {"section": s}) for s in state["sections"]]

# Build workflow
orchestrator_worker_builder = StateGraph(State)

# Add the nodes
orchestrator_worker_builder.add_node("orchestrator", orchestrator)
orchestrator_worker_builder.add_node("llm_call", llm_call)
orchestrator_worker_builder.add_node("synthesizer", synthesizer)

# Add edges to connect nodes
orchestrator_worker_builder.add_edge(START, "orchestrator")
orchestrator_worker_builder.add_conditional_edges(
    "orchestrator", assign_workers, ["llm_call"]
)
orchestrator_worker_builder.add_edge("llm_call", "synthesizer")
orchestrator_worker_builder.add_edge("synthesizer", END)

# Compile the workflow
orchestrator_worker = orchestrator_worker_builder.compile()

# Show the workflow
display(Image(orchestrator_worker.get_graph().draw_mermaid_png()))

# Invoke
state = orchestrator_worker.invoke({"topic": "Create a report on LLM scaling laws"})

```

```
from IPython.display import Markdown
Markdown(state["final_report"])
```

js

```
import { StateGraph, StateSchema, ReducedValue, GraphNode, Send } from "@langchain/langgraph";
import * as z from "zod";

// Graph state
const State = new StateSchema({
  topic: z.string(),
  sections: z.array(z.custom<SectionsSchema>()),
  completedSections: new ReducedValue(
    z.array(z.string()).default(() => []),
    { reducer: (a, b) => a.concat(b) }
  ),
  finalReport: z.string(),
});

// Worker state
const WorkerState = new StateSchema({
  section: z.custom<SectionsSchema>(),
  completedSections: new ReducedValue(
    z.array(z.string()).default(() => []),
    { reducer: (a, b) => a.concat(b) }
  ),
});

// Nodes
const orchestrator: GraphNode<typeof State> = async (state) => {
  // Generate queries
  const reportSections = await planner.invoke([
    { role: "system", content: "Generate a plan for the report." },
    { role: "user", content: `Here is the report topic: ${state.topic}` },
  ]);

  return { sections: reportSections.sections };
};

const llmCall: GraphNode<typeof WorkerState> = async (state) => {
  // Generate section
  const section = await llm.invoke([
    {
      role: "system",
      content: "Write a report section following the provided name and description. Include no preamble for each section. Use markdown formatting.",
    },
    {

```



```

        role: "user",
        content: `Here is the section name: ${state.section.name} and description: ${state.section.description}`,
    },
]);

// Write the updated section to completed sections
return { completedSections: [section.content] };
};

const synthesizer: GraphNode<typeof State> = async (state) => {
    // List of completed sections
    const completedSections = state.completedSections;

    // Format completed section to str to use as context for final sections
    const completedReportSections = completedSections.join("\n\n---\n\n");

    return { finalReport: completedReportSections };
};

// Conditional edge function to create llm_call workers that each write a section of the report
const assignWorkers: ConditionalEdgeRouter<{ InputSchema: typeof State; Nodes: "llmCall" }>
= (state) => {
    // Kick off section writing in parallel via Send() API
    return state.sections.map((section) =>
        new Send("llmCall", { section })
    );
};

// Build workflow
const orchestratorWorker = new StateGraph(State)
    .addNode("orchestrator", orchestrator)
    .addNode("llmCall", llmCall)
    .addNode("synthesizer", synthesizer)
    .addEdge("__start__", "orchestrator")
    .addConditionalEdges(
        "orchestrator",
        assignWorkers,
        ["llmCall"]
    )
    .addEdge("llmCall", "synthesizer")
    .addEdge("synthesizer", "__end__")
    .compile();

// Invoke
const state = await orchestratorWorker.invoke({
    topic: "Create a report on LLM scaling laws"
});
console.log(state.finalReport);

```

Evaluator-optimizer

In evaluator-optimizer workflows, one LLM call creates a response and the other evaluates that response. If the evaluator or a [human-in-the-loop](#) determines the response needs refinement, feedback is provided and the response is recreated. This loop continues until an acceptable response is generated.

Evaluator-optimizer workflows are commonly used when there's particular success criteria for a task, but iteration is required to meet that criteria. For example, there's not always a perfect match when translating text between two languages. It might take a few iterations to generate a translation with the same meaning across the two languages.

evaluator_optimizer.png

python

```
```python Graph API
```

### Graph state

---

```
class State(TypedDict): joke: str topic: str feedback: str funny_or_not: str
```

### Schema for structured output to use in evaluation

---

```
class Feedback(BaseModel): grade: Literal["funny", "not funny"] = Field(description="Decide if the joke is funny or not.",) feedback: str = Field(description="If the joke is not funny, provide feedback on how to improve it.",)
```

### Augment the LLM with schema for structured output

---

```
evaluator = llm.with_structured_output(Feedback)
```

### Nodes

---

```
def llm_call_generator(state: State): """LLM generates a joke"""
```

```
if state.get("feedback"): msg = llm.invoke(f"Write a joke about {state['topic']} but take into account the feedback: {state['feedback']}") else: msg = llm.invoke(f"Write a joke about {state['topic']}") return {"joke": msg.content}
```

```
def llm_call_evaluator(state: State): """LLM evaluates the joke"""
```

```
grade = evaluator.invoke(f"Grade the joke {state['joke']}") return {"funny_or_not": grade.grade, "feedback": grade.feedback}
```

## Conditional edge function to route back to joke generator or end based upon feedback from the evaluator

---

```
def route_joke(state: State): """Route back to joke generator or end based upon feedback from the evaluator"""
```

```
if state["funny_or_not"] == "funny": return "Accepted" elif state["funny_or_not"] == "not funny": return "Rejected + Feedback"
```

## Build workflow

---

```
optimizer_builder = StateGraph(State)
```

## Add the nodes

---

```
optimizer_builder.add_node("llm_call_generator", llm_call_generator)
optimizer_builder.add_node("llm_call_evaluator", llm_call_evaluator)
```

## Add edges to connect nodes

---

```
optimizer_builder.add_edge(START, "llm_call_generator")
optimizer_builder.add_edge("llm_call_generator", "llm_call_evaluator")
optimizer_builder.add_conditional_edges("llm_call_evaluator", route_joke, { # Name returned by route_joke : Name of next node to visit "Accepted": END, "Rejected + Feedback": "llm_call_generator", },)
```

## Compile the workflow

---

```
optimizer_workflow = optimizer_builder.compile()
```

# Show the workflow

---

```
display(Image(optimizer_workflow.get_graph().draw_mermaid_png()))
```

## Invoke

---

```
state = optimizer_workflow.invoke({"topic": "Cats"}) print(state["joke"])
```

```
```python Functional API
# Schema for structured output to use in evaluation
class Feedback(BaseModel):
    grade: Literal["funny", "not funny"] = Field(
        description="Decide if the joke is funny or not.",
    )
    feedback: str = Field(
        description="If the joke is not funny, provide feedback on how to improve it.",
    )

# Augment the LLM with schema for structured output
evaluator = llm.with_structured_output(Feedback)

# Nodes
@task
def llm_call_generator(topic: str, feedback: Feedback):
    """LLM generates a joke"""
    if feedback:
        msg = llm.invoke(
            f"Write a joke about {topic} but take into account the feedback: {feedback}"
        )
    else:
        msg = llm.invoke(f"Write a joke about {topic}")
    return msg.content

@task
def llm_call_evaluator(joke: str):
    """LLM evaluates the joke"""
    feedback = evaluator.invoke(f"Grade the joke {joke}")
    return feedback

@entrypoint()
def optimizer_workflow(topic: str):
    feedback = None
    while True:
        joke = llm_call_generator(topic, feedback).result()
        feedback = llm_call_evaluator(joke).result()
        if feedback.grade == "funny":
            break
```

```

return joke

# Invoke
stream = optimizer_workflow.stream_events("Cats", version="v3")
for snapshot in stream.values:
    print(snapshot)
    print("\n")

```

js

```

````typescript
Graph API import { StateGraph, StateSchema, GraphNode,
ConditionalEdgeRouter } from "@langchain/langgraph"; import * as z from "zod";

// Graph state
const State = new StateSchema({ joke: z.string(), topic: z.string(), feedback:
z.string(), funnyOrNot: z.string(), });

// Schema for structured output to use in evaluation
const feedbackSchema = z.object({ grade:
z.enum(["funny", "not funny"]).describe("Decide if the joke is funny or not."), feedback:
z.string().describe("If the joke is not funny, provide feedback on how to improve it."), });

// Augment the LLM with schema for structured output
const evaluator =
llm.withStructuredOutput(feedbackSchema);

// Nodes
const llmCallGenerator: GraphNode = async (state) => { // LLM generates a joke
let msg; if (state.feedback) { msg = await llm.invoke(`Write a joke about ${state.topic} but take
into account the feedback: ${state.feedback} ` ); } else { msg = await llm.invoke( `Write a joke
about ${state.topic} `); } return { joke: msg.content }; };

const llmCallEvaluator: GraphNode = async (state) => { // LLM evaluates the joke
const grade = await evaluator.invoke(`Grade the joke ${state.joke} `); return { funnyOrNot: grade.grade,
feedback: grade.feedback }; };

// Conditional edge function to route back to joke generator or end based upon feedback from
the evaluator
const routeJoke: ConditionalEdgeRouter<{ InputSchema: typeof State; Nodes:
"llmCallGenerator" }> = (state) => { // Route back to joke generator or end based upon feedback
from the evaluator
if (state.funnyOrNot === "funny") { return "Accepted"; } else { return
"Rejected + Feedback"; } };

// Build workflow
const optimizerWorkflow = new StateGraph(State)
.addNode("llmCallGenerator", llmCallGenerator)
.addNode("llmCallEvaluator", llmCallEvaluator)
.addEdge("start", "llmCallGenerator")
.addEdge("llmCallGenerator", "llmCallEvaluator")
.addConditionalEdges("llmCallEvaluator", routeJoke, { // Name returned by
routeJoke : Name of next node to visit
"Accepted": "end",
"Rejected + Feedback": "llmCallGenerator",
})
.compile();

// Invoke
const state = await optimizerWorkflow.invoke({ topic: "Cats" }); console.log(state.joke);

```

```

```typescript Functional API
import * as z from "zod";
import { task, entrypoint } from "@langchain/langgraph";

// Schema for structured output to use in evaluation
const feedbackSchema = z.object({
  grade: z.enum(["funny", "not funny"]).describe(
    "Decide if the joke is funny or not."
  ),
  feedback: z.string().describe(
    "If the joke is not funny, provide feedback on how to improve it."
  ),
});

// Augment the LLM with schema for structured output
const evaluator = llm.withStructuredOutput(feedbackSchema);

// Tasks
const llmCallGenerator = task("jokeGenerator", async (params: {
  topic: string;
  feedback?: z.infer<typeof feedbackSchema>;
}) => {
  // LLM generates a joke
  const msg = params.feedback
    ? await llm.invoke(
        `Write a joke about ${params.topic} but take into account the feedback: ${params.feedback.feedback}`
      )
    : await llm.invoke(`Write a joke about ${params.topic}`);
  return msg.content;
});

const llmCallEvaluator = task("jokeEvaluator", async (joke: string) => {
  // LLM evaluates the joke
  return evaluator.invoke(`Grade the joke ${joke}`);
});

// Build workflow
const workflow = entrypoint(
  "optimizerWorkflow",
  async (topic: string) => {
    let feedback: z.infer<typeof feedbackSchema> | undefined;
    let joke: string;

    while (true) {
      joke = await llmCallGenerator({ topic, feedback });
      feedback = await llmCallEvaluator(joke);

      if (feedback.grade === "funny") {
        break;
      }
    }
  }
);

```

```

    }
  }

  return joke;
}
);

// Invoke
const stream = await workflow.streamEvents("Cats", { version: "v3" });
for await (const snapshot of stream.values) {
  console.log(snapshot);
  console.log("\n");
}

```

Agents

Agents are typically implemented as an LLM performing actions using [tools](#). They operate in continuous feedback loops, and are used in situations where problems and solutions are unpredictable. Agents have more autonomy than workflows, and can make decisions about the tools they use and how to solve problems. You can still define the available toolset and guidelines for how agents behave.



참고

To get started with agents, see the [quickstart](#) or read more about [how they work](#) in LangChain.

python

```
```python Using tools from langchain.tools import tool
```

## Define tools

```
@tool def multiply(a: int, b: int) -> int: """Multiply a and b .
```

```
Args: a: First int b: Second int """ return a * b
```

```
@tool def add(a: int, b: int) -> int: """Adds a and b .
```

```
Args: a: First int b: Second int """ return a + b
```

```
@tool def divide(a: int, b: int) -> float: """Divide a and b .
```

```
Args: a: First int b: Second int """ return a / b
```

# Augment the LLM with tools

```
tools = [add, multiply, divide] tools_by_name = {tool.name: tool for tool in tools}
llm_with_tools = llm.bind_tools(tools)
```

```
```python Graph API
from langgraph.graph import MessagesState
from langchain.messages import SystemMessage, HumanMessage, ToolMessage

# Nodes
def llm_call(state: MessagesState):
    """LLM decides whether to call a tool or not"""

    return {
        "messages": [
            llm_with_tools.invoke(
                [
                    SystemMessage(
                        content="You are a helpful assistant tasked with performing arithmetic o
n a set of inputs."
                    )
                ]
                + state["messages"]
            )
        ]
    }

def tool_node(state: MessagesState):
    """Performs the tool call"""

    result = []
    for tool_call in state["messages"][-1].tool_calls:
        tool = tools_by_name[tool_call["name"]]
        observation = tool.invoke(tool_call["args"])
        result.append(ToolMessage(content=observation, tool_call_id=tool_call["id"]))
    return {"messages": result}

# Conditional edge function to route to the tool node or end based upon whether the LLM made
a tool call
def should_continue(state: MessagesState) -> Literal["tool_node", END]:
    """Decide if we should continue the loop or stop based upon whether the LLM made a tool
call"""

    messages = state["messages"]
    last_message = messages[-1]

    # If the LLM makes a tool call, then perform an action
    if last_message.tool_calls:
```



```

        return "tool_node"

# Otherwise, we stop (reply to the user)
return END

# Build workflow
agent_builder = StateGraph(MessagesState)

# Add nodes
agent_builder.add_node("llm_call", llm_call)
agent_builder.add_node("tool_node", tool_node)

# Add edges to connect nodes
agent_builder.add_edge(START, "llm_call")
agent_builder.add_conditional_edges(
    "llm_call",
    should_continue,
    ["tool_node", END]
)
agent_builder.add_edge("tool_node", "llm_call")

# Compile the agent
agent = agent_builder.compile()

# Show the agent
display(Image(agent.get_graph(xray=True).draw_mermaid_png()))

# Invoke
messages = [HumanMessage(content="Add 3 and 4.")]
messages = agent.invoke({"messages": messages})
for m in messages["messages"]:
    m.pretty_print()

```

```

```python
Functional API from langgraph.graph import add_messages from
langchain.messages import (SystemMessage, HumanMessage, ToolCall,) from
langchain_core.messages import BaseMessage

```

```

@task def call_llm(messages: list[BaseMessage]): """LLM decides whether to call a tool or
not""" return llm_with_tools.invoke([SystemMessage(content="You are a helpful assistant
tasked with performing arithmetic on a set of inputs.")] + messages)

```

```

@task def call_tool(tool_call: ToolCall): """Performs the tool call""" tool =
tools_by_name[tool_call["name"]] return tool.invoke(tool_call)

```

```

@entrypoint() def agent(messages: list[BaseMessage]): llm_response =
call_llm(messages).result()

```

```

while True: if not llm_response.tool_calls: break

```

# Execute tools

---

```
tool_result_futures = [call_tool(tool_call) for tool_call in llm_response.tool_calls] tool_results
= [fut.result() for fut in tool_result_futures] messages = add_messages(messages,
[llm_response, *tool_results]) llm_response = call_llm(messages).result()
```

```
messages = add_messages(messages, llm_response) return messages
```

## Invoke

---

```
messages = [HumanMessage(content="Add 3 and 4.")] stream = agent.stream_events(messages,
version="v3") for snapshot in stream.values: print(snapshot) print("\n")
```

```
</div>

<div class="callout js" markdown="1">

<p class="callout-label">js</p>

```typescript Using tools
import { tool } from "@langchain/core/tools";
import * as z from "zod";

// Define tools
const multiply = tool(
  ({ a, b }) => {
    return a * b;
  },
  {
    name: "multiply",
    description: "Multiply two numbers together",
    schema: z.object({
      a: z.number().describe("first number"),
      b: z.number().describe("second number"),
    }),
  }
);

const add = tool(
  ({ a, b }) => {
    return a + b;
  },
  {
    name: "add",
    description: "Add two numbers together",
    schema: z.object({
```

```

        a: z.number().describe("first number"),
        b: z.number().describe("second number"),
    }},
    }
);

const divide = tool(
  ({ a, b }) => {
    return a / b;
  },
  {
    name: "divide",
    description: "Divide two numbers",
    schema: z.object({
      a: z.number().describe("first number"),
      b: z.number().describe("second number"),
    }),
  }
);

// Augment the LLM with tools
const tools = [add, multiply, divide];
const toolsByName = Object.fromEntries(tools.map((tool) => [tool.name, tool]));
const llmWithTools = llm.bindTools(tools);

```

```

```typescript
import { StateGraph, StateSchema, MessagesValue, GraphNode, ConditionalEdgeRouter } from "@langchain/langgraph";
import { ToolNode } from "@langchain/langgraph/prebuilt";
import { SystemMessage, ToolMessage } from "@langchain/core/messages";

// Graph state
const State = new StateSchema({ messages: MessagesValue, });

// Nodes
const llmCall: GraphNode = async (state) => { // LLM decides whether to call a tool or not
 const result = await llmWithTools.invoke([{ role: "system", content: "You are a helpful assistant tasked with performing arithmetic on a set of inputs." }, ...state.messages]);

 return { messages: [result] };
};

const toolNode = new ToolNode(tools);

// Conditional edge function to route to the tool node or end
const shouldContinue: ConditionalEdgeRouter<{ InputSchema: typeof State; Nodes: "toolNode" }> = (state) => {
 const messages = state.messages;
 const lastMessage = messages.at(-1);

 // If the LLM makes a tool call, then perform an action if (lastMessage?.tool_calls?.length) {
 return "toolNode";
} // Otherwise, we stop (reply to the user) return "end";

// Build workflow
const agentBuilder = new StateGraph(State)
 .addNode("llmCall", llmCall)
 .addNode("toolNode", toolNode)
 // Add edges to connect nodes
 .addEdge("start", "llmCall")

```

```
.addConditionalEdges("llmCall", shouldContinue, ["toolNode", "end"]) .addEdge("toolNode", "llmCall") .compile();
```

```
// Invoke const messages = [{ role: "user", content: "Add 3 and 4." }]; const result = await agentBuilder.invoke({ messages }); console.log(result.messages);
```

```
```typescript Functional API
import { task, entrypoint, addMessages } from "@langchain/langgraph";
import { BaseMessageLike, ToolCall } from "@langchain/core/messages";

const callLlm = task("llmCall", async (messages: BaseMessageLike[]) => {
  // LLM decides whether to call a tool or not
  return llmWithTools.invoke([
    {
      role: "system",
      content: "You are a helpful assistant tasked with performing arithmetic on a set of inputs."
    },
    ...messages
  ]);
});

const callTool = task("toolCall", async (toolCall: ToolCall) => {
  // Performs the tool call
  const tool = toolsByName[toolCall.name];
  return tool.invoke(toolCall.args);
});

const agent = entrypoint(
  "agent",
  async (messages) => {
    let llmResponse = await callLlm(messages);

    while (true) {
      if (!llmResponse.tool_calls?.length) {
        break;
      }
    }

    // Execute tools
    const toolResults = await Promise.all(
      llmResponse.tool_calls.map((toolCall) => callTool(toolCall))
    );

    messages = addMessages(messages, [llmResponse, ...toolResults]);
    llmResponse = await callLlm(messages);
  }

  messages = addMessages(messages, [llmResponse]);
  return messages;
}
```

```

);

// Invoke
const messages = [{
  role: "user",
  content: "Add 3 and 4."
}];

const stream = await agent.streamEvents([messages], { version: "v3" });
for await (const snapshot of stream.values) {
  console.log(snapshot);
}

```

ToolNode

`@ToolNode` is a prebuilt node that executes tools in LangGraph workflows. It handles parallel tool execution, error handling, and state injection automatically.

Use `@ToolNode` when you need fine-grained control over how your graph executes tools. This is the building block that powers tool execution in many LangGraph agent patterns.

python

```

from langchain.tools import tool
from langgraph.prebuilt import ToolNode
from langgraph.graph import MessagesState, StateGraph

@tool
def search(query: str) -> str:
    """Search for information."""
    return f"Results for: {query}"

@tool
def calculator(expression: str) -> str:
    """Evaluate a math expression."""
    return str(eval(expression))

builder = StateGraph(MessagesState)
builder.add_node("tools", ToolNode([search, calculator]))
# ... add other nodes and edges
graph = builder.compile()

```

js

```

import { ToolNode } from "@langchain/langgraph/prebuilt";
import { tool } from "@langchain/core/tools";
import * as z from "zod";

```

```

const search = tool(
  ({ query }) => `Results for: ${query}`,
  {
    name: "search",
    description: "Search for information.",
    schema: z.object({ query: z.string() }),
  }
);

const calculator = tool(
  ({ expression }) => String(eval(expression)),
  {
    name: "calculator",
    description: "Evaluate a math expression.",
    schema: z.object({ expression: z.string() }),
  }
);

const toolNode = new ToolNode([search, calculator]);

```

Access graph state and context from tools

Tools executed by `ToolNode` receive the arguments generated by the model as their first argument. To read graph-side data that was not generated by the model, use one of these options:

- In Python, read state and run-scoped context from the injected `@[ToolRuntime]` argument.
- In JavaScript, read state and run-scoped context from the tool's second argument, typed as `@[ToolRuntime]`.

참고

Tools can only access the state values passed to the `ToolNode`. When `ToolNode` is added directly as a `StateGraph` node, that input is the current graph state. If you invoke a `ToolNode` manually from another node, pass the full state when tools need custom state fields. For example, `tool_node.invoke(state)` OR `toolNode.invoke(state, config)` exposes the full state, while passing only `{"messages": state["messages"]}` OR `{ messages: state.messages }` only exposes `messages`.

python

js

langgraph / errors

GRAPH_RECURSION_LIMIT

langgraph/errors/GRAPH_RECURSION_LIMIT.mdx

Your LangGraph @[StateGraph] reached the maximum number of steps before hitting a stop condition. This is often due to an infinite loop caused by code like the example below:

python

```
from typing import TypedDict

from langgraph.graph import StateGraph

class State(TypedDict):
    some_key: str

builder = StateGraph(State)
builder.add_node("a", ...)
builder.add_node("b", ...)
builder.add_edge("a", "b")
builder.add_edge("b", "a")
...

graph = builder.compile()
```

js

```
import { StateGraph, StateSchema } from "@langchain/langgraph";
import { z } from "zod/v4";

const State = new StateSchema({
  someKey: z.string(),
});

const builder = new StateGraph(State)
  .addNode("a", ...)
  .addNode("b", ...)
  .addEdge("a", "b")
  .addEdge("b", "a")
  ...

const graph = builder.compile();
```

However, complex graphs may hit the default limit naturally.

Troubleshooting

- If you are not expecting your graph to go through many iterations, you likely have a cycle. Check your logic for infinite loops.

python

- If you have a complex graph, you can pass in a higher `recursion_limit` value into your `config` object when invoking your graph like this:

```
graph.invoke({...}, {"recursion_limit": 1000})
```

js

- If you have a complex graph, you can pass in a higher `recursionLimit` value into your `config` object when invoking your graph like this:

```
await graph.invoke({...}, { recursionLimit: 1000 });
```

INVALID_CHAT_HISTORY

langgraph/errors/INVALID_CHAT_HISTORY.mdx

python

This error is raised in the prebuilt `@[ create_agent ]` when the `call_model` graph node receives a malformed list of messages. Specifically, it is malformed when there are `AIMessages` with `tool_calls` (LLM requesting to call a tool) that do not have a corresponding `@[ ToolMessage ]` (result of a tool invocation to return to the LLM).

js

This error is raised in the prebuilt `@[ createAgent ]` when the `callModel` graph node receives a malformed list of messages. Specifically, it is malformed when there are `AIMessage`s with `tool_calls` (LLM requesting to call a tool) that do not have a corresponding `@[ ToolMessage ]` (result of a tool invocation to return to the LLM).

There could be a few reasons you're seeing this error:

python

1. You manually passed a malformed list of messages when invoking the graph, e.g.

```
graph.invoke({'messages': [AIMessage(..., tool_calls=[...])])
```
2. The graph was interrupted before receiving updates from the `tools` node (i.e. a list of `@[ ToolMessage ]`) and you invoked it with an input that is not `None` or a `ToolMessage`, e.g.

```
graph.invoke({'messages': [HumanMessage(...)], config}).
```

 This interrupt could have been triggered in one of the following ways:
 - * You manually set `interrupt_before = ['tools']` in `create_agent`
 - One of the tools raised an error that wasn't handled by the `@[ ToolNode ] ( "tools" )`

js

1. You manually passed a malformed list of messages when invoking the graph, e.g.

```
graph.invoke({messages: [new AIMessage({..., tool_calls: [...])])})
```
2. The graph was interrupted before receiving updates from the `tools` node (i.e. a list of `@[ ToolMessage ]`) and you invoked it with an input that is not `null` or a `ToolMessage`, e.g.

```
graph.invoke({messages: [new HumanMessage(...)], config}).
```

 This interrupt could have been triggered in one of the following ways:
 - You manually set `interruptBefore: ['tools']` in `createAgent`

- One of the tools raised an error that wasn't handled by the `@[ ToolNode ] ( "tools" )`

Troubleshooting

To resolve this, you can do one of the following:

1. Don't invoke the graph with a malformed list of messages
2. In case of an interrupt (manual or due to an error) you can:

python

- provide `@[ ToolMessage ]` objects that match existing tool calls and call `graph.invoke({'messages': [ToolMessage(...)]})`. **NOTE:** this will append the messages to the history and run the graph from the START node.
- manually update the state and resume the graph from the interrupt:
 1. get the list of most recent messages from the graph state with `graph.get_state(config)`
 2. modify the list of messages to either remove unanswered tool calls from `AIMessages`

or add `@[ ToolMessage ]` objects with `tool_call_ids` that match unanswered tool calls 3. call `graph.update_state(config, {'messages': ...})` with the modified list of messages 4. resume the graph, e.g. call `graph.invoke(None, config)`

js

- provide `ToolMessage` objects that match existing tool calls and call `graph.invoke({messages: [new ToolMessage(...)]})`. **NOTE:** this will append the messages to the history and run the graph from the START node.
- manually update the state and resume the graph from the interrupt:
 1. get the list of most recent messages from the graph state with `graph.getState(config)`
 2. modify the list of messages to either remove unanswered tool calls from `AIMessages`

or add `ToolMessage` objects with `toolCallIds` that match unanswered tool calls 3. call `graph.updateState(config, {messages: ...})` with the modified list of messages 4. resume the graph, e.g. call `graph.invoke(null, config)`

INVALID_CONCURRENT_GRAPH_UPDATE

langgraph/errors/INVALID_CONCURRENT_GRAPH_UPDATE.mdx

A LangGraph @[StateGraph] received concurrent updates to its state from multiple nodes to a state property that doesn't support it.

One way this can occur is if you are using a [fanout](#) or other parallel execution in your graph and you have defined a graph like this:

python

```
from typing import TypedDict

from langgraph.graph import START, StateGraph

class State(TypedDict):
    some_key: str # [!code highlight]

def node(state: State):
    return {"some_key": "some_string_value"}

def other_node(state: State):
    return {"some_key": "some_string_value"}

builder = StateGraph(State)
builder.add_node(node)
builder.add_node(other_node)
builder.add_edge(START, "node")
builder.add_edge(START, "other_node")
graph = builder.compile()
```

js

```
import { StateGraph, StateSchema, START } from "@langchain/langgraph";
import { z } from "zod/v4"; // [!code highlight]

const State = new StateSchema({
  someKey: z.string(),
});

const builder = new StateGraph(State)
  .addNode("node", (state) => {
    return { someKey: "some_string_value" };
  })
  .addNode("otherNode", (state) => {
    return { someKey: "some_string_value" };
  });
```

```

    })
    .addEdge(START, "node")
    .addEdge(START, "otherNode");

const graph = builder.compile();

```

python

If a node in the above graph returns `{ "some_key": "some_string_value" }`, this will overwrite the state value for `"some_key"` with `"some_string_value"`. However, if multiple nodes in e.g. a fanout within a single step return values for `"some_key"`, the graph will throw this error because there is uncertainty around how to update the internal state.

js

If a node in the above graph returns `{ someKey: "some_string_value" }`, this will overwrite the state value for `someKey` with `"some_string_value"`. However, if multiple nodes in e.g. a fanout within a single step return values for `someKey`, the graph will throw this error because there is uncertainty around how to update the internal state.

To get around this, you can define a reducer that combines multiple values:

python

```

import operator
from typing import Annotated

class State(TypedDict):
    # The operator.add reducer fn makes this append-only # [!code highlight]
    some_key: Annotated[list, operator.add] # [!code highlight]

```

js

```

import { StateSchema, ReducedValue } from "@langchain/langgraph";
import { z } from "zod/v4";

const State = new StateSchema({ // [!code highlight]
  someKey: new ReducedValue( // [!code highlight]
    z.array(z.string()).default(() => []), // [!code highlight]
    {
      inputSchema: z.array(z.string()),
      reducer: (existing, update) => existing.concat(update), // [!code highlight]
    }
  )
});

```

```
),  
});
```

This will allow you to define logic that handles the same key returned from multiple nodes executed in parallel.

Troubleshooting

The following may help resolve this error:

- If your graph executes nodes in parallel, make sure you have defined relevant state keys with a reducer.

INVALID_GRAPH_NODE_RETURN_VALUE

langgraph/errors/INVALID_GRAPH_NODE_RETURN_VALUE.mdx

python

A `LangGraph @ [StateGraph]` received a non-dict return type from a node. Here's an example:

```
from typing import TypedDict

from langgraph.graph import StateGraph

class State(TypedDict):
    some_key: str

def bad_node(state: State):
    # Should return a dict with a value for "some_key", not a list
    return ["whoops"]

builder = StateGraph(State)
builder.add_node(bad_node)
...

graph = builder.compile()
```

Invoking the above graph will result in an error like this:

```
graph.invoke({ "some_key": "someval" });
```

```
InvalidUpdateError: Expected dict, got ['whoops']
For troubleshooting, visit: https://docs.langchain.com/oss/python/langgraph/errors/INVALID\_GRAPH\_NODE\_RETURN\_VALUE
```

Nodes in your graph must return a dict containing one or more keys defined in your state.

js

A `LangGraph @ [StateGraph]` received a non-object return type from a node. Here's an example:

```
import { z } from "zod/v4";
import { StateGraph, StateSchema, GraphNode } from "@langchain/langgraph";

const State = new StateSchema({
  someKey: z.string(),
});
```

```
const badNode: GraphNode<typeof State> = (state) => {  
  // Should return an object with a value for "someKey", not an array  
  return ["whoops"];  
};  
  
const builder = new StateGraph(State).addNode("badNode", badNode);  
// ...  
  
const graph = builder.compile();
```

Invoking the above graph will result in an error like this:

```
await graph.invoke({ someKey: "someval" });
```

```
InvalidUpdateError: Expected object, got ['whoops']  
For troubleshooting, visit: https://langchain-ai.github.io/langgraphjs/troubleshooting/errors/INVALID\_GRAPH\_NODE\_RETURN\_VALUE
```

Nodes in your graph must return an object containing one or more keys defined in your state.

Troubleshooting

The following may help resolve this error:

python

- If you have complex logic in your node, make sure all code paths return an appropriate dict for your defined state.

js

- If you have complex logic in your node, make sure all code paths return an appropriate object for your defined state.

MISSING_CHECKPOINTER

langgraph/errors/MISSING_CHECKPOINTER.mdx

You are attempting to use built-in LangGraph persistence without providing a checkpoint.

python

This happens when a `checkpoint` is missing in the `compile()` method of `@[StateGraph]` `[StateGraph]` or `@[entrypoint]`.

js

This happens when a `checkpoint` is missing in the `compile()` method of `@[StateGraph]` `[StateGraph]` or `@[entrypoint]`.

Troubleshooting

The following may help resolve this error:

python

- Initialize and pass a checkpoint to the `compile()` method of `@[StateGraph]` `[StateGraph]` or `@[entrypoint]`.

js

- Initialize and pass a checkpoint to the `compile()` method of `@[StateGraph]` `[StateGraph]` or `@[entrypoint]`.

python

```
from langgraph.checkpoint.memory import InMemorySaver
checkpointer = InMemorySaver()

# Graph API
from langgraph.graph import StateGraph
graph = StateGraph(...).compile(checkpointer=checkpointer)

# Functional API
from langgraph.func import entrypoint
@entrypoint(checkpointer=checkpointer)
```

```
def workflow(messages: list[str]) -> str:
    ...
```

js

```
import { InMemorySaver, StateGraph } from "@langchain/langgraph";
const checkpointers = new InMemorySaver();

// Graph API
import { StateGraph } from "@langchain/langgraph";
const graph = new StateGraph(...).compile({ checkpointers });

// Functional API
import { entrypoint } from "@langchain/langgraph";
const workflow = entrypoint(
  { checkpointers, name: "workflow" },
  async (messages: string[]) => {
    // ...
  }
);
```

- Use the LangGraph API so you don't need to implement or configure checkpointers manually. The API handles all persistence infrastructure for you.

Related

- Read more about [persistence](#).

MULTIPLE_SUBGRAPHS

langgraph/errors/MULTIPLE_SUBGRAPHS.mdx

This error occurs when you [call a subgraph inside a node](#) multiple times, and the subgraph is compiled with `checkpointer=True` (continuations mode).

Troubleshooting

Choose one of the following based on your requirements:

python

1. **Don't need interrupts?** Use `checkpointer=False` to opt out of checkpointing entirely:

```
python subgraph = subgraph_builder.compile(checkpointer=False)
```
2. **Need interrupts but not cross-invocation persistence?** Use the default inherited mode by omitting `checkpointer`:

```
python subgraph = subgraph_builder.compile()
```

 Each invocation gets a unique namespace, so parallel execution works. The subgraph starts fresh each time but can use `interrupt()`.
3. **Need cross-invocation persistence?** Use `checkpointer=True`. LangGraph assigns each invocation a position-based namespace suffix (`calling_node`, `calling_node|1`, etc.) to prevent conflicts. For stable, name-based namespaces, wrap each subgraph with a unique node name — see [parallel subgraphs](#).

js

1. **Don't need interrupts?** Use `checkpointer: false` to opt out of checkpointing entirely:

```
typescript const subgraph = subgraphBuilder.compile({ checkpointer: false });
```
2. **Need interrupts but not cross-invocation persistence?** Use the default inherited mode by omitting `checkpointer`:

```
typescript const subgraph = subgraphBuilder.compile();
```

 Each invocation gets a unique namespace, so parallel execution works. The subgraph starts fresh each time but can use `interrupt()`.
3. **Need cross-invocation persistence?** Use `checkpointer: true`. LangGraph assigns each invocation a position-based namespace suffix (`calling_node`, `calling_node|1`, etc.) to prevent conflicts. For stable, name-based namespaces, wrap each subgraph with a unique node name — see [parallel subgraphs](#).

Related

- [Subgraph persistence](#) — full comparison of checkpointer modes
- [Persistence](#) — how checkpointers work in LangGraph